

FULL STACK WEB DEVELOPMENT

Complete Course Book

HTML • CSS • JavaScript • React • Next.js

T. SACHINTHA IMESH
[FYZIE]

A Complete Learning Guide to Modern Web Development

ප්‍රකාශන හිමිකම්

© 2026 T. SACHINTHA IMESH [FYZIE]

මෙම පොතේ සියලුම ප්‍රකාශන හිමිකම් කර්තෘ සතුය.

කර්තෘගේ ලිඛිත අවසරයකින් තොරව මෙම පොතේ කිසිදු කොටසක් පිටපත් කිරීම, නැවත ප්‍රකාශනය කිරීම, බෙදාහැරීම හෝ වාණිජමය වශයෙන් භාවිත කිරීම තහනම් වේ.

පෙරවදන

FULL STACK WEB DEVELOPMENT යනු නූතන වෙබ් සංවර්ධනය ඉගෙනගැනීමට කැමති පාඨකයෙකු සඳහා මූලික සංකල්පවලින් ආරම්භ වී ක්‍රමයෙන් වඩාත් උසස් තාක්ෂණයන් වෙත ගමන් කරන සම්පූර්ණ පාඨමාලා මාර්ගෝපදේශයකි. මෙහි HTML සහ CSS මගින් වෙබ් පිටු ගොඩනැගීමේ පදනම තැබීමෙන් පසු JavaScript, React, Next.js සහ TypeScript වැනි නූතන තාක්ෂණයන් වෙත පියවරෙන් පියවර ගමන් කිරීමට අවකාශ සලසා ඇත.

මෙම පොත නිර්මාණය කිරීමේ අරමුණ වූයේ ඉගෙනගන්නා කෙනෙකුට තනි තනි තොරතුරු අතර අතරමං නොවී, සම්බන්ධිත සහ පිළිවෙළකට සැකසූ ඉගෙනුම් ගමනක් ලබාදීමයි. සංකල්පයක් කියවා ඉදිරියට යාම පමණක් ප්‍රමාණවත් නොවේ; එය කේතයක් ලෙස ලියා, ප්‍රතිඵලය නිරීක්ෂණය කර, වෙනස්කම් කර බලමින් අත්දැකීමෙන් අවබෝධ කරගැනීම වඩාත් වැදගත්ය.

එබැවින් මෙම පොත භාවිතා කරන විට ඉක්මනින් අවසානයට යාමට උත්සාහ නොකරන්න. එක් පරිච්ඡේදයක අදහස හොඳින් තේරුම්ගෙන, උදාහරණ ඔබම ලියා ක්‍රියාත්මක කර, වැරදි ඇති වූ විට ඒවා අසාර්ථකත්වයක් ලෙස නොව ඉගෙනුම් අවස්ථාවක් ලෙස භාවිත කරන්න. නිතර පුහුණුව, කුතුහලය සහ නැවත නැවත උත්සාහ කිරීම වෙබ් සංවර්ධනය ඉගෙනගැනීමේ සැබෑ ශක්තිය වේ.

මෙම පොත ඔබට සරලව තාක්ෂණ ලැයිස්තුවක් මතක තබාගැනීමට නොව, ඒවා එකිනෙකට සම්බන්ධ කරමින් සැබෑ ව්‍යාපෘති නිර්මාණය කළ හැකි ආකාරයට සිතීමට උපකාරී වනු ඇතැයි බලාපොරොත්තු වෙමි. ඔබගේ ඉගෙනුම් ගමනේදී මෙම පොත මාර්ගෝපදේශයක් ලෙස භාවිත කරමින්, ඔබේම අත්දැකීම් සහ නිර්මාණ සමඟ එය ඉදිරියට ගෙන යන්න.

කර්තෘ ගැන

T. SACHINTHA IMESH

[FYZIE]

T. SACHINTHA IMESH යනු වෙබ් තාක්ෂණයන්, programming සහ digital creativity පිළිබඳ උනන්දුවක් ඇති ඉගෙනගන්නෙකු සහ වෙබ් සංවර්ධකයෙකු ලෙස තම දැනුම පුළුල් කරගනිමින් සිටින කෙනෙකි. ඉගෙනගන්නා සංකල්ප ක්‍රමානුකූලව සකස් කිරීම, කේත සමඟ අත්හදා බැලීම සහ තාක්ෂණික අදහස් වඩාත් පැහැදිලි ඉගෙනුම් මාර්ගයක් බවට පත් කිරීම පිළිබඳ ඔහුට විශේෂ උනන්දුවක් ඇත.

මෙම පොත එම ඉගෙනුම් ගමනේ ප්‍රතිඵලයක් ලෙස සකස් කර ඇති අතර, වෙබ් සංවර්ධනය ඉගෙනගැනීමට කැමති අනෙකුත් පාඨකයන්ටද පිළිවෙළකට ගමන් කළ හැකි මාර්ගෝපදේශයක් ලබාදීම එහි අරමුණයි.

මෙම පොත භාවිතා කරන ආකාරය

1. පරිච්ඡේද අනුපිළිවෙළින් අනුගමනය කරන්න — මූලික සංකල්ප මගහැර උසස් කොටස් වෙත නොයන්න.
2. කේත ඔබම ටයිප් කරන්න — කියවීම පමණක් නොව, සෑම උදාහරණයක්ම ඔබේම පරිගණකයේ ලියා බලන්න.
3. පුහුණු වී අත්හදා බලන්න — අගයන්, styles සහ logic වෙනස් කර ප්‍රතිඵල නිරීක්ෂණය කරන්න.
4. වැරදිවලින් ඉගෙනගන්න — error message එකක් ලැබුණු විට එය තේරුම්ගෙන හේතුව සොයා නිවැරදි කිරීමට උත්සාහ කරන්න.
5. කුඩා ව්‍යාපෘති ගොඩනගන්න — ඉගෙනගත් සංකල්ප එකට යොදා සරල නිර්මාණ සෑදීමෙන් දැනුම තහවුරු කරගන්න.
6. පෙර සංකල්ප වෙත නැවත යන්න — අලුත් මාතෘකා ඉගෙනගන්නා විට අවශ්‍ය මූලික කරුණු නැවත සමාලෝචනය කරන්න.
7. අවශ්‍ය විට official documentation භාවිත කරන්න — නිවැරදි syntax, නවතම විශේෂාංග සහ වැඩිදුර පැහැදිලි කිරීම් සඳහා මුල් ලේඛන පරිශීලනය කරන්න.

පාඨමාලා මාර්ග සිතියම

- 01 — වෙබ් සහ HTML මූලික කරුණු
- 02 — CSS සහ ප්‍රතිචාරාත්මක වෙබ් නිර්මාණය
- 03 — JavaScript මූලික කරුණු
- 04 — උසස් JavaScript
- 05 — Git, GitHub සහ ප්‍රකාශනය
- 06 — React.js සහ නූතන ඉදිරිපස සංවර්ධනය
- 07 — Next.js සහ නූතන වෙබ් යෙදුම්
- 08 — TypeScript සහ වඩාත් ආරක්ෂිත JavaScript සංවර්ධනය
- 09 — ප්‍රායෝගික ව්‍යාපෘති සහ සැබෑ ලෝක සංවර්ධනය

පටුන [Table of Contents]

පරිච්ඡේදය 1: පරිගණක ක්‍රියාකාරිත්වය සහ HTML මූලධර්මය

- පරිච්ඡේදය 1-4: පරිගණකය, අන්තර්ජාලය සහ බ්‍රවුසරය 10
- පරිච්ඡේදය 5-10: HTML පදනම සහ උසස් භාවිතය 19
- පරිච්ඡේදය 11-12: උසස් වෙබ් ඉංජිනේරු විද්‍යාව සහ HTML රහස් 38

පරිච්ඡේදය 2: CSS සහ වෙබ් අඩවි අලංකරණය

- පරිච්ඡේදය 13-17: තේරීම්කාරක, වර්ණ, අකුරු සහ පෙට්ටි ආකෘතිය 46
- පරිච්ඡේදය 18-23: Flexbox, Grid සහ Position 61
- පරිච්ඡේදය 24-26: ප්‍රතිචාරාත්මක නිර්මාණය සහ පෝට්‍රෙට්‍රියෝ ව්‍යාපෘතිය 79
- පරිච්ඡේදය 27: CSS ගෘහ නිර්මාණ ශිල්පය (විශේෂ) 88

පරිච්ඡේදය 3: ජාවාස්ක්‍රිප්ට් මූලධර්ම

- පරිච්ඡේදය 28-30: හැඳින්වීම, පරිසරය සහ විචල්‍යයන් 96
- පරිච්ඡේදය 31-36: දත්ත වර්ග, ක්‍රියාකරුවන්, ලූප් සහ ශ්‍රිත 107
- පරිච්ඡේදය 37-40: වස්තූන්, ඩොම් පාලනය සහ ප්‍රායෝගික ව්‍යාපෘතිය 130
- පරිච්ඡේදය 41-43: ජාවාස්ක්‍රිප්ට් ඉංජිනේරු විද්‍යාව 146

පරිච්ඡේදය 4: උසස් ජාවාස්ක්‍රිප්ට්

- පරිච්ඡේදය 44-45: නූතන ජාවාස්ක්‍රිප්ට් (ES6+) සහ මොඩියුල 159
- පරිච්ඡේදය 46-49: Async/Await, Fetch API සහ දෝෂ හැසිරවීම 164
- පරිච්ඡේදය 50-52: ලෝකල් ස්ටෝරේජ්, පන්ති සහ SkyCast Pro 176

පරිච්ඡේදය 5: වෘත්තීය මෙවලම් සහ වෙබ් අඩවි ප්‍රසිද්ධ කිරීම

- පරිච්ඡේදය 53-55: ටර්මිනලය, ගිට් සහ ගිට්හබ් 190
- පරිච්ඡේදය 56-58: ප්‍රසිද්ධ කිරීම, SEO සහ විගණනය 198
- පරිච්ඡේදය 59-60: ගිට් අභ්‍යන්තරය (විශේෂ) 206

පරිච්ඡේදය 6: රියැක්ට් (React.js) - නූතන ඉදිරිපෙළ සංවර්ධනය

- පරිච්ඡේදය 61-63: හැඳින්වීම, පරිසරය සහ JSX 213
- පරිච්ඡේදය 64-70: සංරචක, ප්‍රොප්ස්, ස්ටේට් සහ ඉලෙක්ට්ස් 226
- පරිච්ඡේදය 71-76: පෝරම, රවුටින් සහ TaskMaster ව්‍යාපෘතිය 255
- පරිච්ඡේදය 77-82: ටෙල්වින්ඩ් CSS සහ උසස් සංරචක (විශේෂ) 282

පරිච්ඡේදය 7: ටයිප්ස්ක්‍රිප්ට් ප්‍රවීණතාවය

- පරිච්ඡේදය 83-86: හැඳින්වීම, දත්ත වර්ග සහ ශ්‍රිත 306
- පරිච්ඡේදය 87-90: ඉන්ටර්ෆේස් සහ ජෙනරික්ස් (විශේෂ) 318

පරිච්ඡේදය 8: නෙක්ස්ට් ජේ.එස් (Next.js) - අති නවීන වෙබ් රාමුව

- පරිච්ඡේදය 91-93: හැඳින්වීම, පද්ධතිය සහ ගොනු රටාවන් 331
- පරිච්ඡේදය 94-97: සර්වර්/ක්ලයන්ට් සංරචක සහ දත්ත ප්‍රවාහය 339
- පරිච්ඡේදය 98-101: සර්වර් ඇක්ෂන්ස් සහ InsightBlog ව්‍යාපෘතිය 349

පරිච්ඡේදය 9: වෘත්තීය ඉදිරිපෙළ සංවර්ධනය සහ වෘත්තීය ගමන

- පරිච්ඡේදය 102-104: පෝට්ෆෝලියෝව, නිදහස් ගමන සහ පාරිභෝගිකයන් 363
- පරිච්ඡේදය 105-107: සම්මුඛ පරීක්ෂණ, මාර්ග සිතියම සහ CV 370

පාඨමාලා අවසාන ප්‍රතිඵල

මෙම පාඨමාලාව අවසන් වන විට පාඨකයාට නූතන වෙබ් සංවර්ධනයේ මූලික සිට උසස් සංකල්ප දක්වා සම්බන්ධිත අවබෝධයක් ගොඩනගා ගැනීමට සහ ඒවා ප්‍රායෝගිකව භාවිත කිරීමට හැකි වන ලෙස මෙම ඉගෙනුම් මාර්ගය සැලසුම් කර ඇත.

- HTML භාවිතයෙන් නිසි ව්‍යුහයක් සහිත වෙබ් පිටු නිර්මාණය කිරීම.
- CSS භාවිතයෙන් layout, styling සහ responsive design සකස් කිරීම.
- JavaScript මගින් interactive සහ dynamic වෙබ් අන්දැකීම් ගොඩනැගීම.
- React භාවිතයෙන් component-based user interfaces නිර්මාණය කිරීම.
- Next.js භාවිතයෙන් නූතන web applications සකස් කිරීම.
- Git සහ GitHub භාවිතයෙන් code version control සහ project collaboration පිළිබඳ මූලික අවබෝධයක් ලබාගැනීම.
- Deployment ක්‍රියාවලිය හරහා වෙබ් ව්‍යාපෘති සජීවී පරිසරයක ප්‍රකාශනය කිරීම.
- TypeScript භාවිතයෙන් වඩාත් ආරක්ෂිත සහ කළමනාකරණය කළ හැකි JavaScript සංවර්ධනය පිළිබඳ අවබෝධයක් ලබාගැනීම.
- ඉගෙනගත් සංකල්ප ඒකාබද්ධ කර practical projects නිර්මාණය කිරීම සහ සැබෑ ලෝකයේ ගැටලු සඳහා සුදුසු විසඳුම් ගැන සිතීම.

නිර්මාණය පිළිබඳ සටහන

මෙම පොත ආරම්භ වූයේ සරල අදහසකිනි — මම ඉගෙනගන්නා දේ තනි තනි සටහන් ලෙස තබා නොගෙන, මටත් තවත් කෙනෙකුටත් නැවත කියවා ඉගෙනගත හැකි සම්පූර්ණ පොතක් බවට පත් කළ යුතුය යන අදහසෙනි.

එම අදහස පොතක් බවට පත් වීම එක් දිනකින් සිදු වූ දෙයක් නොවේ. මට අවශ්‍ය තොරතුරු සෙවීමට, විවිධ මාතෘකා අධ්‍යයනය කිරීමට, ඒවා තේරුම් ගැනීමට සහ එකිනෙකට සම්බන්ධ වන ආකාරය සොයා බැලීමට කාලය ගත විය. කේත ලියා අත්හදා බැලූ අවස්ථා තිබුණි; වැරදි සිදු වූ අවස්ථාද තිබුණි. සමහර සංකල්ප පළමු වරට කියවූ විට පැහැදිලි නොවූ අතර, නැවත කියවා, වෙනත් ආකාරවලින් සිතා, නැවත උත්සාහ කරමින් ඒවා තේරුම් ගැනීමට සිදු විය.

එම ක්‍රියාවලියේදී මගේ අරමුණ වූයේ තොරතුරු එකතු කිරීම පමණක් නොව, ඒවා ඉගෙනගන්නෙකුට අනුගමනය කළ හැකි පිළිවෙළකට සකස් කිරීමයි. කුමන සංකල්පය පළමුව පැමිණිය යුතුද, ඊළඟට කුමක් ඉගෙනගත යුතුද, මූලික දැනුමෙන් උසස් තාක්ෂණයන් වෙත ගමන් කරන්නේ කෙසේද යන්න ගැන සිතා මෙම පොතේ ව්‍යුහය සකස් කළෙමි. පර්යේෂණය, තොරතුරු සංවිධානය, අන්තර්ගතයේ පිළිවෙළ සහ අවසාන ඉදිරිපත් කිරීමේ ප්‍රධාන කාර්යය මෙම සිදු කළෙමි.

මෙම පොත සකස් කිරීමට වෙනත් කෙනෙකු විසින් මට සම්පූර්ණ පොතක් සූදානම් කර දී නොමැත. අවශ්‍ය තැන්වල තොරතුරු පරීක්ෂා කිරීම, අදහස් සාකච්ඡා කිරීම හෝ සමහර කරුණු සංවිධානය කිරීම සඳහා Google AI Studio කුඩා සහායක මෙවලමක් ලෙස භාවිත කළෙමි. එහෙත් මෙම පොතේ අවසාන අන්තර්ගතය, ව්‍යුහය, පරිච්ඡේද අනුපිළිවෙළ සහ ඉදිරිපත් කිරීම තීරණය කර සකස් කළේ මමමය.

ඒ නිසා මෙම පොත පිටු ගණනක් එකට බැඳුණු නිර්මාණයක් පමණක් නොවේ. එය සෙවීමේ, ඉගෙනීමේ, නොතේරුණු දේ නැවත තේරුම් ගැනීමට උත්සාහ කිරීමේ, වැරදි නිවැරදි කිරීමේ සහ අත් නොහැර ඉදිරියට ගමන් කිරීමේ ක්‍රියාවලියක සටහනකි. මෙහි සෑම කොටසක්ම නිසි තැනට ගෙන ඒම සඳහා වැය කළ කාලය, උත්සාහය, ඉවසීම සහ අඛණ්ඩතාව මෙම නිර්මාණයේ කොටසකි.

මෙය මගේ පොතක් පමණක් නොවේ — මම තනිවම ඉගෙනගත් ගමනක මතකයක්ද වේ.

පරිගණක ක්‍රියාකාරිත්වය සහ HTML මූලධර්ම

පළමුවන පරිච්ඡේදය: පරිගණකයක් යනු කුමක්ද සහ එය ක්‍රියා කරන්නේ කෙසේද?

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

- පරිගණකයක් යනු කුමක්දැයි සරලව පැහැදිලි කිරීම.
- දෘඩාංග (Hardware) සහ මෘදුකාංග (Software) අතර වෙනස හඳුනා ගැනීම.
- පරිගණකයක් දත්ත සකසන ආකාරය (Input-Process-Output) තේරුම් ගැනීම.
- ද්විමය පද්ධතිය (Binary System) යනු කුමක්දැයි අවබෝධ කර ගැනීම.
- වෙබ් සංවර්ධකයෙකුට (Web Developer) පරිගණක ක්‍රියාකාරිත්වය වැදගත් වන්නේ ඇයි දැයි තේරුම් ගැනීම.

1.1 හැඳින්වීම (Introduction)

ඔබ වෙබ් අඩවි නිර්මාණය කිරීමට ඉගෙන ගැනීමට පෙර, ඔබ භාවිතා කරන උපකරණය එනම් "පරිගණකය" ඇත්තටම ක්‍රියා කරන්නේ කෙසේදැයි දැනගත යුතුය. බොහෝ දෙනෙක් සිතන්නේ පරිගණකය යනු මැෂින් පෙට්ටියක් කියාය. නමුත් එය ඉතාමත් ක්‍රමානුකූලව වැඩ කරන යන්ත්‍රයකි.

ඔබ අනාගතයේදී වෙබ් අඩවියක් නිර්මාණය කරන විට, ඔබ කරන්නේ පරිගණකයට යම් උපදෙස් මාලාවක් ලබා දීමයි. පරිගණකය ක්‍රියා කරන ආකාරය ඔබ නොදන්නේ නම්, නිවැරදි උපදෙස් ලබා දීම අපහසු වේ.

1.2 පරිගණකය යනු කුමක්ද? (Definition of a Computer)

සරලවම කිවහොත්, පරිගණකය (Computer) යනු අප ලබා දෙන දත්ත (Data) ලබාගෙන, ඒවා අපට අවශ්‍ය පරිදි සකස් කර (Process), අපට අවශ්‍ය තොරතුරු (Information) ලබා දෙන විද්‍යුත් යන්ත්‍රයකි.

සැබෑ ලෝකයේ උදාහරණයක්: සහල් මෝලක් ගැන සිතන්න.

- ඔබ මෝලට වී (Raw Material) ඇතුළු කරයි.
- මෝල තුළදී එම වී කෙටීම සිදු වේ.
- අවසානයේදී ඔබට සහල් (Finished Product) ලැබේ.

පරිගණකයක් මෙලෙසමයි. ඔබ යතුරුපුවරුවෙන් (Keyboard) යමක් ටයිප් කරන විට එය "වී" වැනිය. පරිගණකය තුළ එය සැකසීම "කෙටීම" වැනිය. තිරය මත අකුරු දිස්වීම "සහල්" වැනිය.

1.3 දෘඩාංග සහ මෘදුකාංග (Hardware and Software)

පරිගණකයක් ප්‍රධාන කොටස් දෙකකින් සමන්විත වේ.

- දෘඩාංග (Hardware): පරිගණකයක අපට ඇල්ලිය හැකි, ස්පර්ශ කළ හැකි සියලුම භෞතික කොටස් මෙයට අයත් වේ.
 - Monitor (මොනිටරය): රූප පෙනෙන තිරය.
 - Keyboard (යතුරුපුවරුව): අකුරු ටයිප් කරන උපකරණය.
 - CPU (සී.පී.යූ): පරිගණකයේ මොළය.
 - RAM (රැම්): පරිගණකයේ තාවකාලික මතකය.

2. මෘදුකාංග (Software): පරිගණකයේ දෘඩාංග වලට වැඩ කිරීමට ලබා දෙන උපදෙස් මාලාවන් මෘදුකාංග ලෙස හැඳින්වේ. අපට මේවා ඇල්ලිය නොහැක.

- Web Browser (වෙබ් බ්‍රවුසරය): උදාහරණයක් ලෙස Google Chrome. අන්තර්ජාලයට පිවිසීමට මෙය භාවිතා කරයි.
- Operating System (මෙහෙයුම් පද්ධතිය): උදාහරණයක් ලෙස Windows 10 හෝ Windows 11. පරිගණකය පණ ගැන්වූ විට මුලින්ම වැඩ කරන්නේ මෙයයි.

1.4 පරිගණකය දත්ත සකසන ආකාරය (How Computers Process Data)

පරිගණකයක් ප්‍රධාන පියවර හතරකින් ක්‍රියා කරයි. මෙයට IPOS වක්‍රය යැයි කියනු ලැබේ.

- 1. Input (ආදානය):** පරිගණකයට දත්ත ඇතුළු කිරීම. (උදා: **Keyboard** එකෙන් නමක් ටයිප් කිරීම)
- 2. Process (සැකසීම):** ඇතුළු කළ දත්ත **CPU** එක මගින් ගණනය කිරීම හෝ සකස් කිරීම.
- 3. Output (ප්‍රතිදානය):** සැකසූ තොරතුරු අපට පෙන්වීම. (උදා: **Screen** එකේ නම පෙන්වීම)
- 4. Storage (ගබඩා කිරීම):** පසුව ප්‍රයෝජනයට ගැනීම සඳහා දත්ත තැන්පත් කර තැබීම. (උදා: **Hard Drive** එකක **Save** කිරීම)

1.5 ද්විමය පද්ධතිය (Binary System)

- පරිගණකයේ භාෂාව

මෙය ඉතාමත් වැදගත් කොටසකි. මිනිසුන් වන අපට සිංහල හෝ ඉංග්‍රීසි භාෂාව වැටහුණත්, පරිගණකයට මේ කිසිවක් තේරෙන්නේ නැත.

පරිගණකය යනු විදුලියෙන් ක්‍රියා කරන යන්ත්‍රයකි. එබැවින් එයට තේරෙන්නේ අවස්ථා දෙකක් පමණි:

1. විදුලිය ඇත (ON)
2. විදුලිය නැත (OFF)

මෙම අවස්ථා දෙක හැඟවීමට අපි **0** සහ **1** යන ඉලක්කම් භාවිතා කරමු. මෙයට **Binary** (ද්විමය) පද්ධතිය යැයි කියනු ලැබේ.

- **0** යනු විදුලිය නැති බවයි (Off).
- **1** යනු විදුලිය ඇති බවයි (On).

ඔබ පරිගණකයේ "A" අකුර ටයිප් කළත්, පරිගණකය එය දකින්නේ **01000001** වැනි **0** සහ **1** සමූහයක් ලෙසයි. වෙනි සංවර්ධකයෙකු ලෙස ඔබ ලියන සෑම කේතයක්ම (Code) අවසානයේදී පරිගණකය තේරුම් ගන්නේ මෙලෙස **0** සහ **1** ලෙසටයි.

1.6 මතකය සහ ගබඩා කිරීම (Memory and Storage)

වෙනි අඩවි වේගවත් වීමට නම් පරිගණකයේ මතකය ගැන අවබෝධයක් තිබිය යුතුය.

- 1. RAM (Random Access Memory):** මෙය පරිගණකයේ "කෙටි කාලීන මතකයයි". ඔබ වෙබ් බ්‍රවුසරයක් විවෘත කළ විට, එය තාවකාලිකව රඳවා ගන්නේ **RAM** එක තුළයි. පරිගණකය ක්‍රියා විරහිත කළහොත් මෙහි ඇති දත්ත මැකී යයි.
- 2. Hard Drive / SSD (Storage):** මෙය "දිගු කාලීන මතකයයි". ඔබගේ වීඩියෝ, පින්තූර සහ

ගොනු (Files) මෙහි සඳාකාලිකව ගබඩා වේ.

උපමාවක්: ඔබ පාඩම් කරන මේසය ගැන සිතන්න.

- ඔබ දැනට කියවන පොත මේසය උඩ ඇත. එය **RAM** එක වැනිය. ඔබට ඉක්මනින් එය කියවිය හැක.
- අනෙක් සියලුම පොත් අල්මාරියේ ඇත. එය **Hard Drive** එක වැනිය. අල්මාරියෙන් පොතක් ගැනීමට වැඩි වෙලාවක් ගත වේ, නමුත් එහි ගොඩක් පොත් තැබිය හැක.

1.7 වෙබ් සංවර්ධකයෙකුට මෙය වැදගත් වන්නේ ඇයි?

ඔබ **Frontend Developer** කෙනෙකු ලෙස නිර්මාණය කරන වෙබ් අඩවියක් තවත් කෙනෙකුගේ පරිගණකයක වැඩ කරන ආකාරය තේරුම් ගැනීමට මෙම මූලික කරුණු වැදගත් වේ.

- වෙබ් අඩවියක ඇති පින්තූර විශාල වැඩි නම්, එය **RAM** එකට බරක් වී පරිගණකය හිර විය හැක.
- ඔබ ලියන කේතය (Code) **CPU** එක මගින් කාර්යක්ෂමව සැකසීමට හැකි විය යුතුය.

1.8 සාරාංශය (Key Points)

- පරිගණකය දත්ත ලබාගෙන (**Input**), සකසා (**Process**), ප්‍රතිඵල ලබා දෙයි (**Output**).
- දෘඩාංග යනු භෞතික කොටස් වන අතර මෘදුකාංග යනු උපදෙස් මාලාවන්ය.
- පරිගණකය තේරුම් ගන්නා එකම භාෂාව **Binary** (**0** සහ **1**) වේ.
- **RAM** යනු වේගවත් තාවකාලික මතකය වන අතර **Storage** යනු ස්ථිර මතකයයි.

1.9 අභ්‍යාස (Exercises)

1. ඔබ අවට ඇති දෘඩාංග (**Hardware**) 5ක් නම් කරන්න.
2. ඔබ භාවිතා කරන මෘදුකාංග (**Software**) 3ක් නම් කරන්න.
3. පරිගණකයක **Input** සහ **Output** අතර වෙනස සරලව පැහැදිලි කරන්න.
4. **RAM** එක සහ **Hard Drive** එක අතර ඇති ප්‍රධාන වෙනස කුමක්ද?

1.10 ප්‍රායෝගික ක්‍රියාකාරකම (Practical Task)

ඔබගේ පරිගණකයේ (හෝ ලැප්ටොප් එකේ) පහත විස්තර පරීක්ෂා කර බලන්න:

- එහි ඇති **Operating System** එක කුමක්ද? (Windows, Mac, or Linux?)
- එහි **RAM** එක කොපමණද? (උදා: 4GB, 8GB, 16GB)
- පරිගණකයේ **Processor** එකේ නම කුමක්ද? (උදා: Intel Core i5, AMD Ryzen 5)

දෙවන පරිච්ඡේදය: අන්තර්ජාලය සහ වෙබ් අඩවි ක්‍රියා කරන ආකාරය (How the Internet and Web Work)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

1. අන්තර්ජාලය (Internet) සහ ලෝක ව්‍යාප්ත වෙබය (World Wide Web) අතර වෙනස හඳුනාගැනීම.
2. සේවාලාභියා (Client) සහ සේවාදායකයා (Server) යනු කවුරුන්දැයි වටහා ගැනීම.
3. IP ලිපිනයක් (IP Address) යනු කුමක්දැයි තේරුම් ගැනීම.
4. DNS (Domain Name System) ක්‍රියා කරන ආකාරය අවබෝධ කර ගැනීම.
5. ඔබ වෙබ් ලිපිනයක් ටයිප් කළ විට පසුබිමෙහි සිදුවන ක්‍රියාවලිය පියවරෙන් පියවර තේරුම් ගැනීම.

2.1 හැඳින්වීම (Introduction)

පෙර පරිච්ඡේදයේදී අපි පරිගණකයක් ක්‍රියා කරන ආකාරය ඉගෙන ගත්තෙමු. දැන් අපි ඉගෙන ගැනීමට යන්නේ මෙම පරිගණක දහස් ගණනක් එකිනෙකට සම්බන්ධ වී "අන්තර්ජාලය" නිර්මාණය වන ආකාරයයි. වෙබ් සංවර්ධකයෙකු (Web Developer) ලෙස ඔබ නිර්මාණය කරන වෙබ් අඩවිය ලෝකයේ ඕනෑම තැනක සිටින අයෙකුට දැක ගැනීමට ලැබෙන්නේ මෙම තාක්ෂණය නිසාය.

2.2 අන්තර්ජාලය යනු කුමක්ද? (What is the Internet?)

අන්තර්ජාලය (Internet) යනු ලොව පුරා විසිරී ඇති පරිගණක කෝටි ගණනක් එකිනෙක සම්බන්ධ කර ඇති දැවැන්ත ජාලයකි (Network).

සැබෑ ලෝකයේ උදාහරණයක්: අපේ රටේ මාර්ග පද්ධතිය ගැන සිතන්න. සෑම නිවසක්ම තවත් නිවසකට පාරවල් මගින් සම්බන්ධ වී ඇත. මේ නිසා ඔබට එක් තැනක සිට තවත් තැනකට ගමන් කළ හැක. අන්තර්ජාලය යනු පරිගණක සඳහා ඇති එවැනි "පාරවල්" පද්ධතියකි.

වැදගත්: අන්තර්ජාලය සහ වෙබය (Web) යනු දෙකකි.

- අන්තර්ජාලය (Internet): යනු භෞතිකව පවතින කේබල්, වයර්, සහ පරිගණක ජාලයයි. (පාරවල් පද්ධතිය වැනිය)
- වෙබය (Web): යනු එම ජාලය හරහා අප බලන වෙබ් අඩවි සහ තොරතුරුයි. (පාරේ ගමන් කරන වාහන වැනිය)

2.3 සේවාලාභියා සහ සේවාදායකයා (Client and Server)

අන්තර්ජාලය හරහා තොරතුරු හුවමාරු වීමේදී ප්‍රධාන වර්ත දෙකක් ඇත.

1. සේවාලාභියා (Client): තොරතුරු ඉල්ලා සිටින පුද්ගලයා හෝ උපකරණයයි. ඔබ ඔබේ ජංගම දුරකථනයෙන් හෝ ලැප්ටොප් එකෙන් වෙබ් අඩවියකට පිවිසෙන විට, ඔබේ උපකරණය Client එකක් ලෙස හැඳින්වේ. ඔබ භාවිතා කරන Google Chrome හෝ Firefox වැනි Browser එක මෙහිදී ප්‍රධාන කාර්යයක් ඉටු කරයි.

2. සේවාදායකයා (Server): තොරතුරු ගබඩා කර තබාගෙන සිටින සහ ඒවා ලබා දෙන ප්‍රබල පරිගණකයකි. වෙබ් අඩවියක ඇති සියලුම පින්තූර, විඩියෝ සහ අකුරු ගබඩා කර ඇත්තේ මෙම Server එක තුළයි. මෙය දවසේ පැය 24 පුරාම ක්‍රියාත්මක වී පවතී.

උපමාවක් (ආපනශාලා උදාහරණය):

- ඔබ (පාරිභෝගිකයා) = Client
- වේටර්වරයා = Request/Response (අන්තර්ජාලය)
- කුස්සිය (ආහාර පිළියෙළ කරන තැන) = Server

ඔබ ආහාරයක් ඉල්ලූ විට (Request), වේටර්වරයා එය කුස්සියෙන් ගෙනැවිත් ඔබට ලබා දෙයි (Response).

2.4 IP ලිපිනයක් යනු කුමක්ද? (What is an IP Address?)

ලෝකයේ සෑම නිවසකටම අනන්‍ය වූ ලිපිනයක් ඇත. ලිපිනයක් නැතිනම් ලිපුමක් නිවැරදිව බෙදා දිය නොහැක. එමෙන්ම අන්තර්ජාලයට සම්බන්ධ සෑම උපකරණයකටම අනන්‍ය වූ අංකයක් ඇත. මෙයට **IP Address (Internet Protocol Address)** යැයි කියනු ලැබේ. එය මෙවැනි ආකාරයේ අංක සමූහයකි: **192.168.1.1** හෝ **172.217.161.142**.

2.5 DNS යනු කුමක්ද? (What is DNS?)

අපට **172.217.161.142** වැනි අංක මතක තබා ගැනීම අපහසුය. නමුත් **google.com** හෝ **facebook.com** වැනි නම් මතක තබා ගැනීම පහසුය.

DNS (Domain Name System) යනු අන්තර්ජාලයේ ඇති "දුරකථන නාමාවලිය" (**Phonebook**) වැනිය. ඔබ බ්‍රවුසරයේ **google.com** කියා ටයිප් කළ විට, **DNS** මගින් එම නමට අදාළ **IP ලිපිනය (172.217.161.142)** සොයා දෙයි.

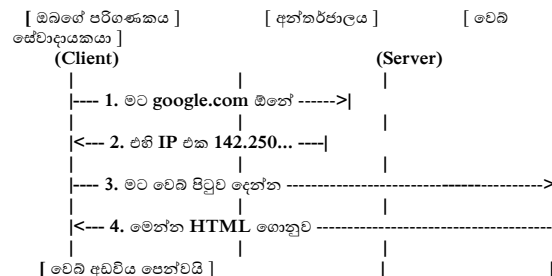
2.6 වෙබ් අඩවියක් ක්‍රියා කරන පියවර (How a Website Loads - Step by Step)

ඔබ බ්‍රවුසරයේ වෙබ් ලිපිනයක් (**URL**) ටයිප් කර **Enter** කළ විට සිදුවන දේ පියවරෙන් පියවර බලමු:

- 1. Request (ඉල්ලීම):** ඔබ **www.google.com** යන්න ටයිප් කරයි.
- 2. DNS Lookup:** බ්‍රවුසරය **DNS** සේවාදායකයාගෙන් අසයි, "Google ගේ **IP ලිපිනය** මොකක්ද?" කියා.
- 3. Getting IP:** **DNS** විසින් **IP ලිපිනය (උදා: 172.217.161.142)** බ්‍රවුසරයට ලබා දෙයි.
- 4. Connecting to Server:** ඇත් බ්‍රවුසරය එම **IP ලිපිනය** ඇති සේවාදායකයා (**Server**) වෙත සම්බන්ධ වේ.
- 5. Sending Request:** බ්‍රවුසරය සේවාදායකයාට පවසයි, "මට මේ වෙබ් අඩවියේ ඇති **HTML** ගොනුව ලබා දෙන්න" කියා.
- 6. Response (ප්‍රතිචාරය):** සේවාදායකයා එම ගොනුව බ්‍රවුසරය වෙත එවයි.

7. Rendering: බ්‍රවුසරය එම කේත (**Code**) කියවා අපට පෙනෙන ලස්සන වෙබ් අඩවියක් ලෙස තිරය මත පෙන්වයි.

2.7 ASCII රූපසටහන: Client-Server ක්‍රියාවලිය



2.8 වෘත්තීය පැහැදිලි කිරීම (Professional Explanation)

Frontend Developer කෙනෙකු ලෙස ඔබ ලියන **HTML, CSS** සහ **JavaScript** ගොනු මුලින්ම සේවාදායකයක් (**Server**) මත තැන්පත් (**Host**) කළ යුතුය. පරිශීලකයෙකු වෙබ් අඩවියට පිවිසෙන විට, බ්‍රවුසරය මගින් **HTTP (HyperText Transfer Protocol)** නමැති නීති මාලාව භාවිතා කරමින් සේවාදායකයෙන් එම ගොනු ඉල්ලා සිටියි. සේවාදායකය එම ඉල්ලීම සකසා නැවත බ්‍රවුසරය වෙත දත්ත එවයි.

2.9 පොදු වැරදි (Common Mistakes)

- අන්තර්ජාලය සහ වෙබය පටලවා ගැනීම: මතක තබා ගන්න, අන්තර්ජාලය යනු යටිතල පහසුකමයි (**Infrastructure**), වෙබය යනු එහි ඇති සේවාවකි.
- **IP ලිපිනය** සහ **Domain Name** එක පටලවා ගැනීම: **google.com** යනු **Domain Name** එකයි. **142.250...** යනු **IP ලිපිනය**යි.

2.10 සාරාංශය (Key Points)

- **Internet:** ලොව පුරා ඇති පරිගණක ජාලය.
- **Web (WWW):** අන්තර්ජාලය හරහා බලන තොරතුරු පද්ධතිය.
- **Client:** අපේ පරිගණකය හෝ බ්‍රවුසරය.
- **Server:** වෙබ් අඩවි ගබඩා කර ඇති ප්‍රබල පරිගණකය.
- **IP Address:** සෑම පරිගණකයකටම ඇති අංකය.

- **DNS:** නමකට අදාළ IP ලිපිනය සොයා දෙන පද්ධතිය.

2.11 අභ්‍යාස (Exercises)

1. **Client** සහ **Server** අතර ඇති ප්‍රධාන වෙනස ඔබේ වචනයෙන් පැහැදිලි කරන්න.
2. **DNS** පද්ධතියක් නොතිබුණේ නම් අන්තර්ජාලය භාවිතා කිරීමේදී අපට ඇතිවන ගැටලුව කුමක්ද?
3. අන්තර්ජාලය හරහා වෙබ් පිටුවක් අපේ පරිගණකයට ලැබෙන තෙක් සිදුවන ක්‍රියාවලිය පියවර 5කින් ලියන්න.
4. ඔබ දන්නා වෙබ් බ්‍රවුසර (Web Browsers) 3ක් නම් කරන්න.

2.12 ප්‍රායෝගික ක්‍රියාකාරකම (Practical Task)

ඔබේ පරිගණකයේ **Command Prompt** (Windows සඳහා) හෝ **Terminal** (Mac සඳහා) විවෘත කර පහත විධානය (**Command**) ටයිප් කර **Enter** කරන්න: **nslookup google.com**

ඔබට **Google** සමාගමට අයත් **IP** ලිපිනය දැකගත හැකි වනු ඇත. එය සටහන් කරගෙන බ්‍රවුසරයේ **URL** එක ලෙස එම අංකය ටයිප් කර බලන්න. **Google** වෙබ් අඩවිය විවෘත වේදැයි පරීක්ෂා කරන්න.

මිලිභ පරිච්ඡේදයෙන් අපි වෙබ් බ්‍රවුසරයක් ඇත්තටම **HTML** කේත කියවා වෙබ් අඩවියක් නිර්මාණය කරන්නේ කෙසේද (**Browser Fundamentals**) යන්න ඉගෙන ගනිමු.

තෙවන පරිච්ඡේදය: බ්‍රවුසරයක ක්‍රියාකාරිත්වය (Browser Fundamentals)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

1. වෙබ් බ්‍රවුසරයක් යනු කුමක්දැයි වෘත්තීය මට්ටමින් තේරුම් ගැනීම.
2. ප්‍රධාන වෙබ් බ්‍රවුසර සහ ඒවායේ වෙනස්කම් හඳුනා ගැනීම.

3. බ්‍රවුසර එන්ජිම (**Browser Engine**) යනු කුමක්දැයි තේරුම් ගැනීම.

4. බ්‍රවුසරයක් විසින් කේත (**Code**) කියවා වෙබ් පිටුවක් නිර්මාණය කරන පියවර (**Rendering Process**) අවබෝධ කර ගැනීම.

5. සංවර්ධක මෙවලම් (**Developer Tools**) ගැන මූලික හැඳින්වීමක් ලබා ගැනීම.

3.1 හැඳින්වීම (Introduction)

පෙර පරිච්ඡේදයේදී අප ඉගෙන ගත්තේ අන්තර්ජාලය හරහා සේවාදායකයක (**Server**) සිට අපේ පරිගණකයට දත්ත පැමිණෙන ආකාරයයි. නමුත් එම දත්ත අපට කෙලින්ම කියවිය නොහැක. ඒවා පැමිණෙන්නේ කේත (**Code**) ලෙසය. මෙම කේත කියවා, ඒවා අපට තේරුම් ගත හැකි රූප, අකුරු සහ වීඩියෝ බවට පත් කරන්නේ "වෙබ් බ්‍රවුසරය" මගිනි.

වෙබ් සංවර්ධකයෙකුට බ්‍රවුසරය යනු ඔහුගේ ප්‍රධානතම මෙවලමයි. ඔබ සාදන වෙබ් අඩවිය පරිශීලකයාට පෙනෙන්නේ බ්‍රවුසරය හරහා බැවින්, බ්‍රවුසරය ඇතුළත සිදුවන දේ දැනගැනීම අත්‍යවශ්‍ය වේ.

3.2 වෙබ් බ්‍රවුසරයක් යනු කුමක්ද? (What is a Web Browser?)

වෙබ් බ්‍රවුසරය (**Web Browser**) යනු වෙබ් අඩවි නැරඹීමට සහ අන්තර්ජාලය සමඟ ගනුදෙනු කිරීමට භාවිතා කරන මෘදුකාංගයකි (**Software Application**).

සරල පැහැදිලි කිරීම: බ්‍රවුසරය යනු "පරිවර්තකයෙකු" (**Translator**) වැනිය. සේවාදායකයෙන් ලැබෙන **HTML**, **CSS** සහ **JavaScript** වැනි තාක්ෂණික භාෂාවන් කියවා, ඒවා සාමාන්‍ය මිනිසෙකුට තේරෙන දෘශ්‍යමාන පිටුවක් බවට එය පත් කරයි.

ප්‍රධාන වෙබ් බ්‍රවුසර කිහිපයක්:

- **Google Chrome:** ලොව වැඩිපුරම භාවිතා කරන බ්‍රවුසරය.
- **Mozilla Firefox:** නිදහස් සහ විවෘත මෘදුකාංගයකි.
- **Microsoft Edge:** Windows සමඟ එන බ්‍රවුසරය.
- **Apple Safari:** Apple උපාංග (iPhone, Mac) සඳහා පමණක් ඇති බ්‍රවුසරය.

3.3 බ්‍රවුසර එන්ජිම (Browser Engine)

සෑම බ්‍රවුසරයක්ම ඇතුළත ඉතාමත් සංකීර්ණ වැඩසටහනක් ඇත. මෙය **Rendering Engine** (රෙන්ඩරින් එන්ජිම) ලෙස හැඳින්වේ. බ්‍රවුසරයේ "හඳවන" මෙයයි.

- **Render** (රෙන්ඩර්): යනු කේත කියවා රූපයක් හෝ පිටුවක් ඇඳ පෙන්වීමයි.

විවිධ බ්‍රවුසර විවිධ එන්ජින් භාවිතා කරයි:

- **Blink:** Google Chrome සහ Microsoft Edge භාවිතා කරයි.
- **WebKit:** Apple Safari භාවිතා කරයි.
- **Gecko:** Mozilla Firefox භාවිතා කරයි.

වෙබ් සංවර්ධකයෙකු ලෙස ඔබ දැනගත යුතු වැදගත් කරුණක් වන්නේ, සමහර විට එක් බ්‍රවුසරයක ලස්සනට පෙනෙන වෙබ් අඩවියක් තවත් බ්‍රවුසරයක මඳක් වෙනස් වී පෙනෙන්නට පුළුවන් බවයි. එයට හේතුව මෙම එන්ජින් කේත කියවන ආකාරය සුළු වශයෙන් වෙනස් වීමයි.

3.4 බ්‍රවුසරයක් වෙබ් පිටුවක් නිර්මාණය කරන ආකාරය (The Rendering Process)

මෙය ඉතාමත් වැදගත් කොටසකි. ඔබ ටයිප් කරන කේතයක් තිරය මත පෙනෙන්නට පියවර 5ක් පසු කළ යුතුය.

පියවර 1: DOM නිර්මාණය කිරීම (Creating the DOM)

බ්‍රවුසරය HTML ගොනුව කියවීමට පටන් ගනී. එය HTML හි ඇති සෑම ටැග් (Tag) එකක්ම ගසක අතු බෙදී යන ආකාරයට සකස් කරයි. මෙයට **DOM** (Document Object Model) යැයි කියනු ලැබේ.

- **DOM** යනු කුමක්දැයි ඉදිරි පරිච්ඡේදවලදී විස්තරාත්මකව ඉගෙන ගනිමු.

පියවර 2: CSSOM නිර්මාණය කිරීම (Creating the CSSOM)

බ්‍රවුසරය HTML සමඟ ඇති **CSS (Style)** ගොනු කියවයි. අකුරුවල පාට, විශාලත්වය වැනි දේ මෙහිදී සකස් වේ. මෙයට **CSSOM** (CSS Object Model) යැයි කියනු ලැබේ.

පියවර 3: Render Tree එක සෑදීම (Creating the Render Tree)

දැන් බ්‍රවුසරය **DOM** එක සහ **CSSOM** එක එකට එකතු කරයි. දැන් බ්‍රවුසරය දන්නවා කුමන අකුරට කුමන පාටද ලබා දිය යුත්තේ කියා.

පියවර 4: Layout (පිරිසැලසුම)

මෙහිදී බ්‍රවුසරය තීරණය කරනවා එක් එක් කොටස තිරයේ කුමන ස්ථානයක තිබිය යුතුද කියා. (උදා: පින්තූරය වම් පැත්තේ, අකුරු දකුණු පැත්තේ).

පියවර 5: Painting (ආලේප කිරීම)

අවසාන පියවර ලෙස බ්‍රවුසරය තිරයේ ඇති පික්සල් (**Pixels**) මත පාට තවරා අපට පෙනෙන වෙබ් පිටුව නිර්මාණය කරයි.

3.5 ASCII රූපසටහන: බ්‍රවුසර ක්‍රියාවලිය

```
[ HTML ] -----> [ DOM Tree ] --\
                                     \
                                     > [ Render Tree ] -----> [ Layout ] -----> [
Painting ] -----> [ වෙබ් පිටුව ]
                                     /
                                     /
[ CSS ] -----> [ CSSOM Tree ] -/
```

3.6 සංවර්ධක මෙවලම් (Developer Tools / Inspect Element)

සෑම බ්‍රවුසරයකම පාහේ වෙබ් සංවර්ධකයන් සඳහාම වෙන්වූ මෙවලම් කට්ටලයක් ඇත. මෙය **Inspect Element** ලෙසද හඳුන්වයි.

මෙය විවෘත කරන්නේ කෙසේද?

1. ඕනෑම වෙබ් අඩවියකට යන්න (උදා: google.com).
2. මවුසයේ දකුණු පැත්ත ක්ලික් කරන්න (**Right Click**).
3. **Inspect** යන්න තෝරන්න. (එසේත් නැතිනම් කීබෝඩ් එකේ **F12** බොත්තම ඔබන්න).

මෙමගින් ඔබට:

- වෙබ් අඩවිය සාදා ඇති **HTML** කේත දැකගත හැක.
- **CSS** මගින් වර්ණ වෙනස් කර බැලිය හැක.
- වෙබ් අඩවියේ ඇති වැරදි (**Errors**) සොයාගත හැක.

3.7 වෘත්තීය පැහැදිලි කිරීම (Professional Explanation)

බ්‍රවුසරයක් යනු හුදෙක් ගොනු පෙන්වන මෘදුකාංගයක් පමණක් නොවේ. එය ඉතාමත් දියුණු **Runtime Environment** එකකි. නවීන බ්‍රවුසර වල **V8 Engine** වැනි බලවත් **JavaScript** එන්ජින් අඩංගු වන අතර, එමගින් ඉතා සංකීර්ණ ගණනය කිරීම් සහ ඇනිමේෂන් තත්පරයකින් දහයෙන් පංගුවක කාලයකදී සිදු කිරීමට හැකියාව ලැබේ.

3.8 පොදු වැරදි (Common Mistakes)

- සියලුම බ්‍රවුසර සමාන යැයි සිතීම: සමහර පැරණි බ්‍රවුසර (උදා: **Internet Explorer**) නවීන කේත වර්ග තේරුම් නොගනී. එම නිසා අපේ වෙබ් අඩවිය සියලුම බ්‍රවුසර වල පරීක්ෂා කිරීම වැදගත් වේ.
- බ්‍රවුසර **Cache** එක ගැන නොදැනීම: සමහර විට ඔබ කේතය වෙනස් කළත් බ්‍රවුසරය පරණ පිටුවම පෙන්වයි. එයට හේතුව වේගය වැඩි කර ගැනීමට බ්‍රවුසරය විසින් පරණ දත්ත මතක තබා ගැනීමයි (**Caching**).

3.9 සාරාංශය (Key Points)

- බ්‍රවුසරය යනු කේත කියවා දැරාගන්නා පිටු සාදන මෘදුකාංගයකි.
- ප්‍රධාන බ්‍රවුසර එන්ජින් වන්නේ **Blink**, **WebKit** සහ **Gecko** ය.
- පිටුවක් පෙන්වීමට පෙර **DOM**, **CSSOM** සහ **Render Tree** එක සෑදිය යුතුය.
- **Inspect Element** මෙවලම වෙබ් සංවර්ධකයෙකුට අත්‍යවශ්‍ය වේ.

3.10 අභ්‍යාස (Exercises)

1. ඔබ දන්නා වෙබ් බ්‍රවුසර 4ක් නම් කරන්න.
2. **Rendering Engine** එකක් යනු කුමක්දැයි සරලව පැහැදිලි කරන්න.
3. බ්‍රවුසරයක් විසින් **HTML** කේත කියවා තිරය මත පෙන්වන තෙක් සිදුවන පියවර 5 අනුපිළිවෙලින් ලියන්න.
4. **DOM** සහ **CSSOM** යනු මොනවාද?

3.11 ප්‍රායෝගික ක්‍රියාකාරකම (Practical Task)

1. ඔබේ පරිගණකයේ **Google Chrome** හෝ **Firefox** විවෘත කරන්න.
2. **youtube.com** හෝ වෙනත් ප්‍රියතම වෙබ් අඩවියකට යන්න.
3. **F12** යතුර ඔබා **Developer Tools** විවෘත කරන්න.
4. එහි ඇති **Elements** ටැබ් එක මත ක්ලික් කර එම වෙබ් අඩවියේ ඇති **HTML** කේත දෙස බලන්න.
5. ඕනෑම අකුරක් මත මවුසය තබා **Right Click** කර **'Edit as HTML'** දී එම අකුරු වෙනස් කර බලන්න. (මෙය සිදු වන්නේ ඔබේ පරිගණකයේ පමණක් බව මතක තබා ගන්න, වෙබ් අඩවියට හානියක් නොවේ).

මිළඟ පරිච්ඡේදයෙන් අපි වෙබ් අඩවි ලිවීමට භාවිතා කරන ප්‍රධාන මෘදුකාංගය වන **VS Code** ස්ථාපනය කරන ආකාරය සහ භාවිතා කරන ආකාරය ඉගෙන ගනිමු.

හතරවන පරිච්ඡේදය: VS Code - අපගේ වැඩබිම් සකස් කරගනිමු (Visual Studio Code)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

1. කේත සංස්කාරකයක් (**Code Editor**) යනු කුමක්දැයි තේරුම් ගැනීම.
2. ලෝකයේ ජනප්‍රියම සංස්කාරකය වන **VS Code** තෝරා ගැනීමට හේතු.
3. **VS Code** පරිගණකයට ස්ථාපනය (**Install**) කරන ආකාරය.
4. **VS Code** හි මූලික කොටස් සහ අතුරු මුහුණත (**Interface**) හඳුනා ගැනීම.
5. වැඩ කටයුතු පහසු කරවන **Extensions** (අමතර උපාංග) ස්ථාපනය කිරීම.

4.1 හැඳින්වීම (Introduction)

ලේඛකයෙකුට පොතක් ලිවීමට පැනක් සහ කඩදාසියක් අවශ්‍ය වන්නා සේම, වඩු කාර්මිකයෙකුට උපකරණ කට්ටලයක් අවශ්‍ය වන්නා සේම, වෙබ් සංවර්ධකයෙකුට කේත (**Code**) ලිවීමට විශේෂිත මෘදුකාංගයක් අවශ්‍ය වේ. අපට සාමාන්‍ය **Notepad** එකක වුවද කේත ලිවිය හැකිය,

නමුත් වෘත්තීය මට්ටමේ වැඩ කිරීමට නම් අපට බලවත් උපකරණයක් අවශ්‍ය වේ.

අද වන විට ලෝකයේ සිටින දක්ෂතම මෘදුකාංග ඉංජිනේරුවන් පවා භාවිතා කරන ප්‍රධානතම මෙවලම වන්නේ **Visual Studio Code** හෙවත් **VS Code** ය.

4.2 කේත සංස්කාරකයක් යනු කුමක්ද? (What is a Code Editor?)

කේත සංස්කාරකයක් (Code Editor) යනු පරිගණක කේත ලිවීම සඳහාම විශේෂයෙන් නිර්මාණය කරන ලද මෘදුකාංගයකි.

ඇයි අපි Notepad භාවිතා නොකරන්නේ? සාමාන්‍ය Notepad එකක අප ලියන දේ හුදෙක් කළු පැහැති අකුරු පමණි. නමුත් කේත සංස්කාරකයක් මගින්:

- කේතයේ ඇති වැරදි පෙන්වා දෙයි.
- විවිධ කේත වර්ග (Tags, Properties) හඳුනා ගැනීමට විවිධ වර්ණ ලබා දෙයි (Syntax Highlighting).
- අප ටයිප් කිරීමට යන වචන කලින්ම අනුමාන කර පෙන්වයි (Auto-completion).

4.3 VS Code යනු කුමක්ද? (What is VS Code?)

Visual Studio Code (VS Code) යනු Microsoft සමාගම විසින් නිර්මාණය කරන ලද, නොමිලේ ලබා දෙන, ඉතාමත් වේගවත් සහ බලවත් කේත සංස්කාරකයකි.

මෙය භාවිතා කිරීමට ප්‍රධාන හේතු:

1. නොමිලේ ලබා දීම (Free): කිසිදු මුදලක් ගෙවිය යුතු නැත.
2. වේගය (Fast): ඕනෑම පරිගණකයක ඉතා හොඳින් ක්‍රියා කරයි.
3. බහුකාර්යතාව (Cross-platform): Windows, Mac, Linux යන ඕනෑම මෙහෙයුම් පද්ධතියක භාවිතා කළ හැක.
4. අමතර පහසුකම් (Extensions): අපට අවශ්‍ය ඕනෑම නව පහසුකමක් මෙයට එක් කරගත හැක.

4.4 VS Code ස්ථාපනය කිරීම (Installation Step-by-Step)

දැන් අපි පියවරෙන් පියවර **VS Code** ඔබේ පරිගණකයට දාගන්නා ආකාරය බලමු.

1. වෙබ් අඩවියට යන්න: ඔබගේ බ්‍රවුසරයේ (Chrome) code.visualstudio.com යන ලිපිනයට යන්න.
2. Download කරන්න: එහි ඇති "Download for Windows" (හෝ ඔබේ OS එකට අදාළ බොත්තම) ක්ලික් කරන්න.
3. Setup එක විවෘත කරන්න: බාගත වූ (Downloaded) ගොනුව මත දෙවරක් ක්ලික් (Double click) කර එය විවෘත කරන්න.
4. ගිවිසුම පිළිගන්න: "I accept the agreement" යන්න තෝරා Next කරන්න.
5. වැදගත් පියවරක්: ස්ථාපනය වන විට ලැබෙන විකල්ප අතරින් "Add to PATH" සහ "Create a desktop icon" යන කොටු සලකුණු (Tick) කරන්න.
6. අවසන් කිරීම: Install බොත්තම ඔබන්න. පසුව Finish කරන්න.

දැන් ඔබේ පරිගණක තිරයේ (Desktop) ඇති VS Code අයිකනය මත ක්ලික් කර එය විවෘත කරගන්න.

4.5 VS Code අතුරු මුහුණත හඳුනා ගැනීම (Interface Overview)

VS Code විවෘත වූ පසු ඔබට ප්‍රධාන කොටස් කිහිපයක් දැකගත හැක.

1. Activity Bar (ක්‍රියාකාරකම් තීරුව): වම් පැත්තේ ඇති කුඩා අයිකන පෙළයි. මෙහි ගොනු සෙවීමට සහ Extensions සොයා ගැනීමට බොත්තම් ඇත.
2. Side Bar (Explorer): ඔබ වැඩ කරන වෙබ් අඩවියේ ඇති සියලුම ගොනු (Files) සහ ෆෝල්ඩර් (Folders) මෙහි ලැයිස්තුවක් ලෙස පෙන්වයි.
3. Editor Area (සංස්කරණ ප්‍රදේශය): ඔබ කේත ටයිප් කරන ප්‍රධාන මැද කොටසයි.
4. Status Bar (තත්ව තීරුව): පහළින්ම ඇති තීරුවයි. ඔබ දැනට ඉන්නේ කිවෙහි පේළියේද, කුමන භාෂාවෙන් (HTML/CSS) ලියනවාද යන්න මෙහි පෙන්වයි.

4.6 Extensions (අමතර උපාංග) ස්ථාපනය කිරීම

VS Code හි ඇති හොඳම දේ වන්නේ එහි වැඩ පහසු කරන **Extensions** තිබීමයි. ආරම්භකයෙකු ලෙස ඔබට පහත **Extensions** දෙක ඉතාමත් වැදගත් වේ:

1. Live Server:

- එය කුමක්ද? ඔබ කේතයේ යමක් වෙනස් කර **Save** කළ සැණින්, බ්‍රවුසරය **Refresh** නොකරම එම වෙනස බලා ගැනීමට මෙය උදව් වේ.
- ස්ථාපනය කරන හැටි: **Activity Bar** එකේ ඇති **Extensions** අයිකනය (කොටු 4ක රූපය) ක්ලික් කර "**Live Server**" ලෙස සර්ච් කරන්න. ඉන්පසු '**Install**' කරන්න.

2. Prettier - Code Formatter:

- එය කුමක්ද? ඔබ කේතය අවුල් සහගත ලෙස ටයිප් කළත්, මෙය විසින් කේතය පිළිවෙළකට සකස් කර දෙයි.
- ස්ථාපනය කරන හැටි: පෙර පරිදීම සර්ච් කර **Install** කරන්න.

4.7 වැදගත් කෙටිමං (Useful Shortcuts)

වැඩ වේගවත් කර ගැනීමට මෙම කීබෝඩ් කෙටිමං පුරුදු වන්න:

- **Ctrl + S** : ලියූ කේතය **Save** (සුරැකීම) කිරීමට. (නිතරම මෙය කිරීමට පුරුදු වන්න).
- **Ctrl + N** : අලුත් ගොනුවක් (**New File**) විවෘත කිරීමට.
- **Ctrl + B** : පැත්තේ ඇති **Side Bar** එක පෙන්වීමට හෝ සැඟවීමට.
- **Ctrl + /** : කේතයේ යම් ජේලියක් **Comment** කිරීමට (පරිගණකයට නොපෙනෙන සේ සැඟවීමට).

4.8 පොදු වැරදි සහ විසඳුම් (Common Mistakes & Debugging)

- කේතය **Save** නොකිරීම: බොහෝ සිසුන් කේතය ලියා බ්‍රවුසරයේ බලන විට වෙනසක් සිදු වී නැති බව පවසයි. එයට හේතුව **Ctrl + S** මගින් **Save** නොකිරීමයි. (**VS Code Settings** වල "**Auto Save**" යන්න "**onFocusChange**" ලෙස සැකසීමෙන් මෙය විසඳාගත හැක).

- වැරදි **Extension** එකක් දැමීම: එකම නම ඇති **Extensions** කිහිපයක් තිබිය හැක. සැමවිටම වැඩිපුරම **Download** කර ඇති සහ විශ්වාසවන්ත එක තෝරාගන්න.

4.9 සාරාංශය (Key Points)

- **VS Code** යනු වෘත්තීය වෙබ් සංවර්ධකයන් භාවිතා කරන **Code Editor** එකකි.
- එය නොමිලේ ලබා ගත හැකි අතර ඕනෑම පරිගණකයක වැඩ කරයි.
- **Live Server** වැනි **Extensions** මගින් අපගේ වැඩ වේගවත් කර ගත හැක.
- කේතය පිළිවෙළට තබා ගැනීමට **Prettier** භාවිතා කරයි.

4.10 අභ්‍යාස (Exercises)

1. කේත සංස්කාරකයක් (**Code Editor**) භාවිතා කිරීමේ වාසි 3ක් ලියන්න.
2. ඔබේ පරිගණකයේ **VS Code** ස්ථාපනය කර එහි **Live Server Extension** එක ඇතුළත් කරන්න.
3. **VS Code** හි '**Editor Area**' සහ '**Side Bar**' මගින් කෙරෙන කාර්යයන් මොනවාද?
4. කේතයක් **Save** කිරීමට භාවිතා කරන කෙටිමහ කුමක්ද?

4.11 ප්‍රායෝගික ක්‍රියාකාරකම (Practical Task)

1. ඔබේ පරිගණකයේ **Desktop** එකේ **MyFirstProject** නමින් අලුත් **Folder** එකක් සාදන්න.
2. **VS Code** විවෘත කර **File -> Open Folder** ගොස් එම **Folder** එක තෝරන්න.
3. **Side Bar** එකේ ඇති '**New File**' අයිකනය ක්ලික් කර **index.html** නමින් ගොනුවක් සාදන්න.
4. එහි **Hello World** කියා ටයිප් කර **Ctrl + S** ඔබා **Save** කරන්න.
5. දැන් **VS Code** හි යට දකුණු කෙළවරේ ඇති "**Go Live**" බොත්තම ඔබා බ්‍රවුසරයේ එම වචනය දිස්වන්නේ දැයි බලන්න.

සුභ පැතුම්! ඔබ දැන් වෙබ් අඩවි නිර්මාණය කිරීමට අවශ්‍ය වටපිටාව සකසා අවසානය. මිලහ පරිච්ඡේදයේ සිට අපි සැබෑ **HTML** කේත ලිවීමට පටන් ගනිමු.

පස්වන පරිච්ඡේදය: HTML ආරම්භය - වෙබ් අඩවියක සැකිල්ල (HTML Beginner)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

1. HTML යනු කුමක්ද සහ එහි ඉතිහාසය කෙටියෙන් තේරුම් ගැනීම.
2. HTML ටැග් (Tags) සහ මූලිකාංග (Elements) යනු මොනවාදැයි ඉගෙන ගැනීම.
3. HTML ලේඛනයක මූලික ව්‍යුහය (Basic Structure) තේරුම් ගැනීම.
4. මාතෘකා (Headings) සහ ඡේද (Paragraphs) භාවිතා කරන ආකාරය.
5. ලැයිස්තු (Lists) සකස් කරන ආකාරය.
6. සැබෑ ලෝකයේ උදාහරණයක් ඇසුරින් වෙබ් පිටුවක් නිර්මාණය කිරීම.

5.1 HTML යනු කුමක්ද? (What is HTML?)

HTML (HyperText Markup Language) යනු වෙබ් අඩවි නිර්මාණය කිරීමට ලෝකයේ භාවිතා කරන ප්‍රධානතම භාෂාවයි. මෙය "ක්‍රමලේඛන භාෂාවක්" (Programming Language) ලෙස හඳුන්වනවාට වඩා "සලකුණු භාෂාවක්" (Markup Language) ලෙස හැඳින්වීම නිවැරදි වේ.

සරල පැහැදිලි කිරීම: අප නිවසක් තනන විට මූලිකම ගඩොල් සහ සිමෙන්ති යොදා "සැකිල්ල" (Structure) සාදා ගනිමු. ඉන්පසුව එයට පාට (CSS) ගා, විදුලිය සහ ජලය (JavaScript) ලබා දෙමු. වෙබ් අඩවියක "ගඩොල් සහ සිමෙන්ති" හෙවත් එහි සැකිල්ල වන්නේ HTML ය.

- **HyperText:** යනු වෙබ් පිටු එකිනෙක සම්බන්ධ කරන ලින්ක් (Links) වලට කියන නමයි.
- **Markup Language:** යනු සාමාන්‍ය අකුරු වලට විශේෂ කේත (Tags) එක් කර, ඒවා වෙබ් පිටුවක පෙන්විය යුතු ආකාරය බ්‍රවුසරයට පවසන භාෂාවයි.

5.2 HTML ඉතිහාසය (History of HTML)

HTML ප්‍රථම වරට නිර්මාණය කළේ ටිම් බර්නර්ස් ලී (Tim Berners-Lee) මහතා විසින් 1991

වසරේදීය. ඔහුට අවශ්‍ය වූයේ විද්‍යාඥයන් අතර තොරතුරු පහසුවෙන් හුවමාරු කර ගැනීමට ක්‍රමයක් සකස් කිරීමටයි. අද වන විට අපි භාවිතා කරන්නේ එහි නවතම සහ දියුණුම සංස්කරණය වන HTML5 ය.

5.3 HTML ටැග් සහ මූලිකාංග (Tags and Elements)

HTML ක්‍රියා කරන්නේ "ටැග්" (Tags) හරහාය. ටැග් එකක් යනු සාමාන්‍ය වචනයක් දෙපැත්තෙන් කෝණික වරහන් < > යෙදීමයි.

ටැග් එකක ව්‍යුහය: බොහෝ HTML ටැග් වලට ආරම්භයක් සහ අවසානයක් ඇත.

<h1>මෙය මාතෘකාවකි</h1>

1. **Opening Tag** (ආරම්භක ටැගය): <h1> - මෙයින් බ්‍රවුසරයට පවසන්නේ මාතෘකාව මෙතැනින් පටන් ගන්නා බවයි.
2. **Content** (අන්තර්ගතය): මෙය මාතෘකාවකි - පරිශීලකයාට පෙනෙන කොටස මෙයයි.
3. **Closing Tag** (අවසාන ටැගය): </h1> - මෙහි ඇති ඇල ඉර (Forward Slash /) මගින් පවසන්නේ මාතෘකාව මෙතැනින් අවසන් වන බවයි.
4. **Element** (මූලිකාංගය): ආරම්භක ටැගයේ සිට අවසාන ටැගය දක්වා මුළු කොටසම "Element" එකක් ලෙස හැඳින්වේ.

5.4 HTML ලේඛනයක මූලික ව්‍යුහය (Basic Structure)

ඕනෑම HTML ගොනුවක් ආරම්භ කළ යුත්තේ නිශ්චිත සැකිල්ලකට අනුවය. VS Code හි අලුත් index.html ගොනුවක් සාදා එහි ! ලකුණ ටයිප් කර Enter කළ විට ඔබට පහත කේතය ලැබෙනු ඇත.

කේතය:

```
<!DOCTYPE html>
<html>
<head>
  <title>මගේ පළමු වෙබ් පිටුව</title>
</head>
<body>
  <h1>ආයුබෝවන්!</h1>
  <p>මම වෙබ් සංවර්ධනය ඉගෙන ගනිමි.</p>
</body>
</html>
```


සෑම ජේළියකම තේරුම බලමු:

1. **<!DOCTYPE html>**: මෙය බ්‍රවුසරයට පවසන්නේ අප භාවිතා කරන්නේ නවතම **HTML5** සංස්කරණය බවයි.
2. **<html>**: මෙය වෙබ් පිටුවේ ප්‍රධානම උග්‍රයයි (**Root Element**). අනෙක් සියලුම උග්‍ර නිශ්චය යුත්තේ මෙයට ඇතුළතය.
3. **<head>**: මෙම කොටස වෙබ් අඩවියේ පිටතට නොපෙනෙන "මොළය" වැනිය. මෙහි වෙබ් අඩවිය පිළිබඳ තොරතුරු (**Metadata**) අඩංගු වේ.
4. **<title>**: බ්‍රවුසරයේ ඉහළ ඇති ටැබ් (**Tab**) එකේ පෙන්වන නම මෙහි ලියනු ලැබේ.
5. **<body>**: වෙබ් අඩවියේ අපට පෙනෙන සියලුම දේ (අකුරු, පින්තූර, විඩියෝ) අඩංගු වන්නේ මෙහිය. එය "ශරීරය" වැනිය.

5.5 මාතෘකා සහ ඡේද (Headings and Paragraphs)

වෙබ් පිටුවක අකුරු පෙන්වීමට විවිධ මට්ටමේ උග්‍ර ඇත.

Headings (මාතෘකා): HTML හි මාතෘකා වර්ග 6ක් ඇත.

- **<h1>** : විශාලම සහ වැදගත්ම මාතෘකාව. (වෙබ් පිටුවකට සාමාන්‍යයෙන් එකක් පමණක් යොදා ගනී).
- **<h2>** : උප මාතෘකාව.
- **<h3>** සිට **<h6>** දක්වා : ක්‍රමයෙන් කුඩා වන මාතෘකා.

Paragraphs (ඡේද): දිගු වාක්‍ය ලිවීම සඳහා **<p>** උග්‍රය භාවිතා කරයි.

උදාහරණ:

```
<h1>Frontend Development</h1>
<h2>HTML ඉගෙන ගනිමු</h2>
<p>HTML යනු වෙබ් අඩවියක අත්තිවාරමයි.</p>
```

5.6 ලැයිස්තු (Lists)

වෙබ් අඩවියක කරුණු පෙළ ගැස්වීමට ක්‍රම දෙකක් ඇත.

1. **Ordered List (පිළිවෙළක් ඇති ලැයිස්තු):** අංක (1, 2, 3) යෙදූ ලැයිස්තු.
 - මීට **** උග්‍රය භාවිතා වේ.

2. **Unordered List (පිළිවෙළක් නැති ලැයිස්තු):** බුලට් පොයින්ට්ස් (**Bullets**) යෙදූ ලැයිස්තු.
 - මීට **** උග්‍රය භාවිතා වේ.

සටහන: ලැයිස්තුවේ ඇති සෑම කරුණක්ම **** (**List Item**) උග්‍රය ඇතුළත ලිවිය යුතුය.

උදාහරණ:

```
<ul>
  <li>ඇපල්</li>
  <li>කෙසෙල්</li>
  <li>දොඩම්</li>
</ul>
```

5.7 පොදු වැරදි (Common Mistakes)

1. අවසාන උග්‍රය (**Closing Tag**) අමතක වීම:


```
<h1>Hello ලෙස ලිවීම වැරදිය. එය
<h1>Hello</h1> ලෙස නිවැරදිව අවසන් කළ යුතුය.
```
2. අකුරු වැරදි (**Typos**): **<p>** වෙනුවට **<P>** හෝ **<pp>** ලෙස ලිවීමෙන් කේතය ක්‍රියා නොකරනු ඇත.
3. වැරදි ලෙස **Nesting** කිරීම: උග්‍ර එකක් ඇතුළත තවත් උග්‍රයක් ලියන විට ඒවා පිළිවෙළට වැසිය යුතුය.
 - නිවැරදි:

```
<p><b>නිවැරදිය</b></p>
```
 - වැරදි:

```
<p><b>වැරදිය</p></b>
```

5.8 වෘත්තීය පැහැදිලි කිරීම (Professional Explanation)

සැබෑ වෘත්තීය වෙබ් සංවර්ධකයෙකු **HTML** ලියන විට එහි අර්ථය (**Semantic**) ගැන සැලකිලිමත් වේ. උදාහරණයක් ලෙස, අකුරු ලොකු කිරීමට පමණක් **<h1>** භාවිතා නොකර, එය වෙබ් පිටුවේ ප්‍රධානම මාතෘකාව සඳහා පමණක් වෙන් කරයි. මෙය සෙවුම් යන්ත්‍ර (**Google Search**) වලට ඔබේ වෙබ් අඩවිය හඳුනා ගැනීමට පහසු කරයි (**SEO - Search Engine Optimization**).

5.9 සාරාංශය (Key Points)

- **HTML** වෙබ් අඩවියක සැකිල්ල සාදයි.
- උග්‍ර (**Tags**) මගින් අන්තර්ගතය විස්තර කරයි.
- **index.html** යනු වෙබ් අඩවියක ප්‍රධාන ගොනුවයි.
- **<h1>** සිට **<h6>** දක්වා මාතෘකා ඇත.
- **** සහ **** මගින් ලැයිස්තු සාදයි.

5.10 අභ්‍යාස (Exercises)

1. HTML5 ලේඛනයක තිබිය යුතු මූලික වැග් 4ක් ලියන්න.
2. `<ul>` සහ `<ol>` අතර වෙනස පැහැදිලි කරන්න.
3. පහත දැක්වෙන තොරතුරු පෙන්වීමට අවශ්‍ය HTML කේතය ලියන්න:
 - ප්‍රධාන මාතෘකාව: "මගේ ප්‍රියතම ආහාර"
 - ඡේදයක්: "මම පහත ආහාර වලට ඉතා කැමතියි."
 - පිළිවෙළක් නැති ලැයිස්තුවක්: "බත්", "පාන්", "රොටි".

5.11 කුඩා ව්‍යාපෘතිය (Mini Project)

ව්‍යාපෘතිය: ඔබේ ජීව දත්ත පත්‍රිකාව (Simple Resume)

VS Code භාවිතා කර `my-resume.html` නමින් අලුත් ගොනුවක් සාදා, එහි පහත තොරතුරු HTML භාවිතා කර ඇතුළත් කරන්න:

1. ඔබේ නම ප්‍රධාන මාතෘකාවක් ලෙස.
2. "මම ගැන" උප මාතෘකාවක් යටතේ ඔබ ගැන කෙටි විස්තරයක්.
3. "මගේ අධ්‍යාපනය" උප මාතෘකාවක් යටතේ ඔබේ පාසලේ විස්තර.
4. "මගේ දක්ෂතා" උප මාතෘකාවක් යටතේ ලැයිස්තුවක් (උදා: චිත්‍ර ඇඳීම, පිහිනීම).

මූලික පරිච්ඡේදයෙන් අපි HTML Advanced කොටස යටතේ වෙබ් පිටුවකට පින්තූර (Images), ලින්ක් (Links) සහ වගු (Tables) එකතු කරන ආකාරය ඉගෙන ගනිමු.

භයවන පරිච්ඡේදය: HTML උසස් මට්ටම - පින්තූර, සබැඳි සහ වගු (HTML Advanced)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

1. ගුණාංග (Attributes) යනු කුමක්දැයි තේරුම් ගැනීම.
2. වෙබ් පිටු එකිනෙක සම්බන්ධ කරන ආකාරය (Links - `<a>`).
3. වෙබ් පිටුවකට පින්තූර එකතු කරන ආකාරය (Images - `<img>`).

4. දත්ත ක්‍රමවත්ව පෙන්වීමට වගු (Tables) භාවිතා කරන ආකාරය.

5. ස්වයං-වසාගන්නා වැග් (Self-closing tags) හඳුනා ගැනීම.

6.1 ගුණාංග යනු මොනවාද? (What are Attributes?)

පෙර පරිච්ඡේදයේදී අපි මූලික වැග් ගැන ඉගෙන ගත්තෙමු. නමුත් සමහර වැග් වලට වැඩ කිරීමට අමතර තොරතුරු අවශ්‍ය වේ. මෙම අමතර තොරතුරු Attributes (ගුණාංග) ලෙස හැඳින්වේ.

සැබෑ ලෝකයේ උදාහරණයක්: "ළමයා" යනු වැග් එකක් යැයි සිතන්න. නමුත් එම ළමයාගේ "උස", "බර", සහ "ඇස්වල වර්ණය" එම ළමයා පිළිබඳ අමතර විස්තර හෙවත් Attributes වේ.

HTML හි ස්වරූපය: Attributes සැමවිටම ලියනු ලබන්නේ ආරම්භක වැගය (Opening Tag) තුළය. `<tag attribute="අගය">` අන්තර්ගතය `</tag>`

6.2 සබැඳි හෙවත් ලින්ක්ස් (Hyperlinks - `<a>` Tag)

වෙබ් අඩවියක් යනු වෙබ් පිටු විශාල සංඛ්‍යාවක එකතුවකි. මෙම පිටු එකිනෙක සම්බන්ධ කිරීමට සබැඳි (Links) භාවිතා කරයි. මේ සඳහා භාවිතා කරන්නේ Anchor Tag හෙවත් `<a>` වැගයයි.

භාවිතා කරන ආකාරය: `<a>` වැගයට අනිවාර්යයෙන්ම `href` නමැති Attribute එක තිබිය යුතුය.

- `href` (Hypertext Reference): මෙයින් පවසන්නේ ලින්ක් එක ක්ලික් කළ විට යා යුතු ස්ථානයයි.

උදාහරණය:

```
<a href="https://www.google.com">ගූගල් වෙත යන්න</a>
```

Target Attribute: ඔබට ලින්ක් එක ක්ලික් කළ විට එය අලුත් වැඩ (New Tab) එකක විවෘත වීමට අවශ්‍ය නම්, `target="_blank"` යන්න භාවිතා කළ හැක.

```
<a href="https://www.google.com" target="_blank">අලුත් වැඩ එකක ගූගල් විවෘත කරන්න</a>
```

6.3 පින්තූර එකතු කිරීම (Images - Tag)

වෙබ් පිටුවක් ලස්සන කිරීමට පින්තූර අත්‍යවශ්‍ය වේ. මේ සඳහා ටැගය භාවිතා කරයි.

විශේෂත්වය: ටැගයට අවසාන ටැගයක් (Closing tag) නැත. මෙය Self-closing tag එකක් ලෙස හැඳින්වේ.

අත්‍යවශ්‍ය ගුණාංග (Attributes):

1. **src (Source):** පින්තූරය තිබෙන ස්ථානය (File path හෝ URL එක).
2. **alt (Alternative Text):** කිසියම් හේතුවක් නිසා පින්තූරය නොපෙනී ගියහොත් පෙන්විය යුතු අකුරු පෙළ. (මෙය ඇස් පෙනීම දුර්වල අයට වෙබ් අඩවිය කියවා දෙන මෘදුකාංග සඳහා ඉතා වැදගත් වේ).

උදාහරණය:

```

```

6.4 වගු නිර්මාණය (Tables - <table>)

දත්ත ලැයිස්තුවක් පේළි (Rows) සහ තීරු (Columns) ලෙස පෙන්වීමට වගු භාවිතා කරයි. වගුවක් සෑදීමට ටැග් කිහිපයක එකතුවක් අවශ්‍ය වේ.

1. **<table>:** වගුවේ ආරම්භය සහ අවසානය.
2. **<tr> (Table Row):** වගුවේ පේළියක් නිරූපණය කරයි.
3. **<th> (Table Header):** වගුවේ ප්‍රධාන මාතෘකා අඩංගු කොටුව (අකුරු තදින් පෙනේ).
4. **<td> (Table Data):** වගුවේ සාමාන්‍ය දත්ත අඩංගු කොටුව.

වගුවක ව්‍යුහය (ASCII Diagram):

```
+-----+-----+
| මාතෘකාව 1 (th) | මාතෘකාව 2 (th) | <- Row 1 (tr)
+-----+-----+
| දත්ත 1 (td)    | දත්ත 2 (td)    | <- Row 2 (tr)
+-----+-----+
```

සම්පූර්ණ කේතය:

```
<table border="1">
  <tr>
    <th>විෂයය</th>
    <th>ලකුණු</th>
  </tr>
  <tr>
    <td>ගණිතය</td>
    <td>85</td>
  </tr>
  <tr>
    <td>විද්‍යාව</td>
    <td>90</td>
  </tr>
</table>
```

(සටහන: border="1" යනු වගුවේ ඉරි පෙන්වීමට යොදන පැරණි ක්‍රමයකි. පසුව අපි මෙය CSS හරහා ලස්සනට කරන ආකාරය ඉගෙන ගනිමු.)

6.5 වෘත්තීය උපදෙස් (Professional Tips)

- **Accessibility (ප්‍රවේශ්‍යතාව):** සැමවිටම ටැගය සඳහා alt අගයක් ලබා දෙන්න. එය වෘත්තීය වෙබ් සංවර්ධකයෙකුගේ ලක්ෂණයකි.
- **Path (මග):** පින්තූරයක් ඇතුළත් කිරීමේදී එම පින්තූරය සහ HTML ගොනුව එකම Folder එකක තිබේ නම් src="photo.jpg" ලෙසද, වෙනත් Folder එකක නම් src="images/photo.jpg" ලෙසද නිවැරදිව ලිවිය යුතුය.

6.6 පොදු වැරදි (Common Mistakes)

1. **src අගය වැරදීම:** පින්තූරයේ නම හෝ වර්ගය (.jpg, .png) සුළු වශයෙන් හෝ වැරදුනහොත් පින්තූරය දිස් නොවේ.
2. **Self-closing ටැග් වැසීමට උත්සාහ කිරීම:** ලෙස ලිවීම වැරදිය. ප්‍රමාණවත්ය.
3. **වගු පටලවා ගැනීම:** <td> සහ <tr> පටලවා ගැනීම නිසා වගුවේ හැඩය වෙනස් විය හැක. සැමවිටම පේළියක් (<tr>) ඇතුළත කොටු (<td>) ලියන්න.

6.7 සාරාංශය (Key Points)

- **Attributes** මගින් ටැග් එකකට අමතර විස්තර ලබා දෙයි.

- `<a>` ටැගය `href` සමඟ සබැඳි සාදයි.
- `<img>` ටැගය `src` සහ `alt` සමඟ පින්තූර පෙන්වයි.
- වගුවක් සෑදීමට `<table>`, `<tr>`, `<th>`, සහ `<td>` භාවිතා වේ.

6.8 අභ්‍යාස (Exercises)

1. පින්තූරයක් සඳහා `alt Attribute` එක වැදගත් වන්නේ ඇයි?
2. අලුත් ටැබ් එකක වෙබ් අඩවියක් විවෘත කිරීමට භාවිතා කරන `Attribute` එක කුමක්ද?
3. පහත දත්ත ඇතුළත් වගුවක් සඳහා `HTML` කේතය ලියන්න:
 - තීරු දෙකකි: "නම" සහ "වයස".
 - පේළි දෙකක දත්ත: (කමල්, 16) සහ (නිමල්, 17).
4. `src` සහ `href` අතර වෙනස කුමක්ද?

6.9 කුඩා ව්‍යාපෘතිය (Mini Project)

ව්‍යාපෘතිය: මගේ සංචාරක මගපෙන්වන්නා (Simple Travel Guide)

පෙර පාඩම් වල ඉගෙන ගත් දේ සහ මෙම පාඩමේ කරුණු එකතු කර `travel.html` නමින් පිටුවක් සාදන්න. එහි තිබිය යුතු දේ:

1. ඔබ යාමට කැමතිම නගරයක නම ප්‍රධාන මාතෘකාව ලෙස.
2. එම නගරයේ ලස්සන පින්තූරයක්.
3. එම නගරය ගැන කුඩා විස්තරයක් (Paragraph).
4. එහි ඇති නැරඹිය යුතු ස්ථාන 3ක් (Unordered List).
5. එම නගරය ගැන වැඩි විස්තර දැනගැනීමට විකිපීඩියා (Wikipedia) වෙබ් අඩවියට යන සබැඳියක් (Link).
6. නගරයට යන බස් ගාස්තු හෝ දුම්රිය වේලාවන් පෙන්වන කුඩා වගුවක්.

මිලිභ පරිච්ඡේදයෙන් අපි වෙබ් අඩවියක ඉතාමත් වැදගත් කොටසක් වන "ෆෝම්" (Forms) හෙවත් පරිශීලකයාගෙන් දත්ත ලබා ගන්නා ආකාරය ඉගෙන ගනිමු.

හත්වන පරිච්ඡේදය: ෆෝම් - පරිශීලක දත්ත ලබා ගැනීම (Forms)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

1. `HTML` ෆෝම් (Forms) යනු මොනවාද සහ ඒවායේ වැදගත්කම.
2. `<form>` ටැගය සහ එහි මූලික ගුණාංග.
3. විවිධ වර්ගයේ `Input` (දත්ත ඇතුළත් කිරීමේ කොටු) හඳුනා ගැනීම.
4. ලේබල් (Labels) භාවිතා කිරීමේ වැදගත්කම.
5. `Dropdown` මෙනු සහ `Textarea` භාවිතා කරන ආකාරය.
6. සරල ලියාපදිංචි වීමේ පෝරමයක් (Registration Form) නිර්මාණය කිරීම.

7.1 හැඳින්වීම (Introduction)

පෙර පරිච්ඡේදවලදී අප ඉගෙන ගත්තේ තොරතුරු පෙන්වන ආකාරයයි. නමුත් සැබෑ වෙබ් අඩවියකදී අපට පරිශීලකයාගෙන් (User) තොරතුරු ලබා ගැනීමටද සිදු වේ.

උදාහරණයක් ලෙස:

- Facebook වෙත ඇතුළු වීමට (Login) ඔබගේ විද්‍යුත් තැපෑල සහ මුරපදය ටයිප් කිරීම.
- Google හි යමක් සෙවීමට වචනයක් ටයිප් කිරීම.
- Online ඇණවුමක් කිරීමට ඔබගේ ලිපිනය ඇතුළත් කිරීම.

මෙම සියලු අවස්ථාවලදී භාවිතා වන්නේ **Forms** (ෆෝම්) හෙවත් දත්ත ලබා ගන්නා පෝරමයන්ය.

7.2 ෆෝම් එකක මූලික ව්‍යුහය (Basic Form Structure)

`HTML` හි පෝරමයක් සෑදීමට ප්‍රධාන වශයෙන් `<form>` ටැගය භාවිතා කරයි. මෙය අනෙකුත් සියලුම `Input` කොටස් රඳවා තබා ගන්නා භාජනයක් (Container) වැනිය.

මූලික කේතය:

```
<form action="/save-data" method="POST">
  <!-- දත්ත ලබා ගන්නා කොටස් මෙහි ලියනු ලැබේ -->
</form>
```

ප්‍රධාන ගුණාංග (Attributes):

1. **action** (ක්‍රියාව): ෆෝම් එකේ දත්ත යැවිය යුත්තේ කුමන ස්ථානයටද (සර්වර් එකේ ලිපිනය) යන්න මෙහි සඳහන් කරයි.
2. **method** (ක්‍රමය): දත්ත යවන ක්‍රමයයි. සාමාන්‍යයෙන් රහස්‍ය දත්ත සඳහා **POST** ක්‍රමයත්, සෙවුම් වැනි දේ සඳහා **GET** ක්‍රමයත් භාවිතා කරයි.

7.3 ඉන්පුට් ටැගය (The <input> Tag)

පරිශීලකයාගෙන් දත්ත ලබා ගන්නා ප්‍රධානතම මෙවලම වන්නේ <input> ටැගයයි. මෙයද **Self-closing** (තනිව වැසෙන) ටැගයකි. එහි ඇති **type** ගුණාංගය වෙනස් කිරීමෙන් අපට විවිධ ආකාරයේ කොටු ලබා ගත හැක.

1. Text Input (සරල අකුරු සඳහා)

නම, ලිපිනය වැනි දේ ලබා ගැනීමට මෙය භාවිතා කරයි.

```
<input type="text" placeholder="ඔබේ නම මෙහි ටයිප් කරන්න">
```

- **placeholder**: කොටුව ඇතුළත ලා පැහැයෙන් පෙනෙන උපදෙස් වැකියයි.

2. Password Input (මුරපද සඳහා)

මෙහිදී ටයිප් කරන අකුරු තීන් හෝ තරු ලකුණු ලෙස සැඟවී පෙනේ.

```
<input type="password"
placeholder="මුරපදය ඇතුළත් කරන්න">
```

3. Email Input (විද්‍යුත් තැපෑල සඳහා)

මෙහිදී පරිශීලකයා @ ලකුණ සහිත නිවැරදි Email එකක් ටයිප් කර ඇත්දැයි බුදුසරය මගින්ම පරීක්ෂා කරයි.

```
<input type="email">
```

4. Checkbox (තේරීම් සඳහා)

ලැයිස්තුවකින් එකකට වඩා වැඩි ගණනක් තේරීමට මෙය භාවිතා කරයි.

```
<input type="checkbox"> මම කොන්දේසි වලට එකඟ වෙමි
```

5. Radio Buttons (එකක් පමණක් තේරීමට)

ලැයිස්තුවකින් එකක් පමණක් තේරීමට මෙය භාවිතා කරයි (උදා: ස්ත්‍රී/පුරුෂ භාවය). මෙහිදී **name** ගුණාංගය සමාන විය යුතුය.

```
<input type="radio" name="gender"> පිරිමි
<input type="radio" name="gender"> ගැහැණු
```

7.4 ලේබල් එකක වැදගත්කම (The <label> Tag)

භූදෙක් ඉන්පුට් කොටුවක් පමණක් තිබීමෙන් එය කුමක් සඳහාදැයි පරිශීලකයාට පැහැදිලි නැත. එම නිසා සෑම ඉන්පුට් එකකටම ලේබලයක් තිබිය යුතුය.

නිවැරදි ක්‍රමය:

```
<label for="username">පරිශීලක නාමය:</label>
<input type="text" id="username" name="username">
```

- **id**: ඉන්පුට් එකට ලබා දෙන අනන්‍ය නමකි.
- **for**: ලේබලය කුමන ඉන්පුට් එකට අදාළදැයි බුදුසරයට පවසයි. (මෙහිදී **for** සහ **id** සමාන විය යුතුය).
- ප්‍රයෝජනය: ලේබලය මත ක්ලික් කළ විටත් අදාළ ඉන්පුට් කොටුව ක්‍රියාත්මක (**Focus**) වේ.

7.5 අනෙකුත් පෝරම කොටස් (Other Form Elements)

1. Textarea (දිගු විස්තර සඳහා)

පේළි කිහිපයක විස්තරයක් (උදා: පණිවිඩයක්) ලිවීමට මෙය භාවිතා කරයි.

```
<label>ඔබේ පණිවිඩය:</label>
<textarea rows="4" cols="50"></textarea>
```

2. Select (Dropdown ලිස්ට්)

බොහෝ තේරීම් අතරින් එකක් තේරීමට පහතට දිග හැරෙන මෙනුවක් සාදාගත හැක.

```
<label>නගරය තෝරන්න:</label>
<select>
  <option value="colombo">කොළඹ</option>
  <option value="kandy">මහනුවර</option>
  <option value="galle">ගාල්ල</option>
</select>
```

3. Submit Button (දත්ත යැවීමේ බොත්තම)

ෆෝම් එකේ පුරවන ලද දත්ත සර්වර් එකට යැවීමට මෙම බොත්තම අත්‍යවශ්‍ය වේ.

```
<input type="submit" value="ලියාපදිංචි වන්න">
```

7.6 සම්පූර්ණ උදාහරණය (Complete Registration Form)

පහත දැක්වෙන්නේ සැබෑ ලෝකයේ භාවිතා කළ හැකි සරල ලියාපදිංචි වීමේ පෝරමයකි.

```
<!DOCTYPE html>
<html>
<head>
  <title>ලියාපදිංචි වීමේ පෝරමය</title>
</head>
<body>
  <h2>අප හා සම්බන්ධ වන්න</h2>
  <form>
    <p>
      <label for="fname">මුල් නම:</label><br>
      <input type="text" id="fname" required>
    </p>

    <p>
      <label for="my-email">විද්‍යුත් තැපෑල:</label><br>
      <input type="email" id="my-email" required>
    </p>

    <p>
      <label>ස්ත්‍රී/පුරුෂ භාවය:</label><br>
      <input type="radio" name="gen" id="m"><label for="m">පිරිමි</label>
      <input type="radio" name="gen" id="f"><label for="f">ගැහැණු</label>
    </p>

    <p>
      <label for="msg">ඔබේ අදහස:</label><br>
      <textarea id="msg"></textarea>
    </p>

    <input type="submit" value="දත්ත යවන්න">
  </form>
</body>
</html>
```

7.7 වෘත්තීය උපදෙස් (Professional Tips)

- **Validation** (වලංගුතාවය): යම් කොටුවක් අනිවාර්යයෙන්ම පිරවිය යුතු නම් **required** යන වචනය ඉන්පුටු වැගය තුළ ඇතුළත් කරන්න.
- **Accessibility** (ප්‍රවේශ්‍යතාව): කිසිවිටකත් ලේබල් (<label>) භාවිතය මඟ හරින්න එපා. එය ඇස් පෙනීම අඩු අයට වෙබ් අඩවිය භාවිතා කිරීමට බෙහෙවින් උදව් වේ.
- **Naming**: සෑම ඉන්පුටු එකකටම **name** එකක් ලබා දෙන්න. එවිට සර්වර් එකට දත්ත ලැබෙන විට එය කුමක්දැයි හඳුනා ගැනීමට පහසු වේ.

7.8 පොදු වැරදි (Common Mistakes)

1. **name** ගුණාංගය අමතක වීම: සර්වර් එකට දත්ත යවන විට **name** එක නොමැති නම් එම දත්ත හඳුනාගත නොහැක.
2. **Radio Buttons** වල **name** වෙනස් වීම: **name** එක සමාන නොකළහොත් රේඩියෝ බොත්තම් සියල්ලම එකවර තේරීමට (**Select**) හැකි වේ. එය වැරදිය.
3. **<form>** වැගය අමතක වීම: සියලුම ඉන්පුටු කොටස් **<form>** වැගය ඇතුළත තිබිය යුතුය. නැතිනම් **Submit** බොත්තම ක්‍රියා නොකරනු ඇත.

7.9 අභ්‍යාස (Exercises)

1. **type="text"** සහ **type="password"** අතර ඇති වෙනස කුමක්ද?
2. **Checkbox** සහ **Radio Button** භාවිතා කළ යුතු අවස්ථා වෙන් වෙන්ව පැහැදිලි කරන්න.
3. පරිශීලකයා විසින් ඉන්පුටු කොටුවක් පිරවීම අනිවාර්ය කරන්නේ කෙසේද?
4. **<label>** වැගයේ ඇති **for** ගුණාංගය සහ ඉන්පුටු එකේ **id** එක අතර ඇති සම්බන්ධය කුමක්ද?

7.10 කුඩා ව්‍යාපෘතිය (Mini Project)

ව්‍යාපෘතිය: පාසල් ඇතුළත් වීමේ පෝරමය (School Admission Form)

පහත විස්තර ලබා ගත හැකි පරිදි **HTML** ෆෝම් එකක් නිර්මාණය කරන්න:

1. ශිෂ්‍යයාගේ සම්පූර්ණ නම (**Text**).
2. උපන් දිනය (ඉණිය: **type="date"** භාවිතා කර බලන්න).

3. ඉගෙන ගැනීමට කැමති විෂයයන් (Checkboxes: ගණිතය, විද්‍යාව, තොරතුරු තාක්ෂණය).
4. පදිංචි දිස්ත්‍රික්කය (Dropdown Select).
5. මීට පෙර ඉගෙන ගත් පාසල ගැන කෙටි විස්තරයක් (Textarea).
6. "ඇතුළත් වන්න" නමින් Submit බොත්තමක්.

මිළන පරිච්ඡේදයෙන් අපි වෙබ් අඩවියක් අර්ථවත් ලෙස බෙදා වෙන් කරන "සෙමැන්ටික් HTML" (Semantic HTML) ගැන ඉගෙන ගනිමු.

අටවන පරිච්ඡේදය: සෙමැන්ටික් HTML - අර්ථවත් වෙබ් පිටු (Semantic HTML)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

1. සෙමැන්ටික් (Semantic) යන්නෙහි තේරුම වටහා ගැනීම.
2. අර්ථවත් ටැග් (Semantic Tags) සහ අර්ථයක් නැති ටැග් (Non-semantic Tags) අතර වෙනස හඳුනා ගැනීම.
3. වෙබ් පිටුවක ව්‍යුහය නිවැරදිව බෙදා වෙන් කිරීමට භාවිතා කරන ටැග් ඉගෙන ගැනීම.
4. සෙවුම් යන්ත්‍ර ප්‍රශස්තකරණය (SEO) සහ ප්‍රවේශ්‍යතාව (Accessibility) සඳහා මෙහි ඇති වැදගත්කම තේරුම් ගැනීම.
5. වෙබ් පිටුවක සම්මත සැකිල්ලක් (Standard Layout) නිර්මාණය කිරීම.

8.1 සෙමැන්ටික් (Semantic) යනු කුමක්ද?

සෙමැන්ටික් (Semantic) යනු "අර්ථය පිළිබඳ" යන්නයි. HTML වලදී සෙමැන්ටික් ටැග් යනු එම ටැග් තුළ ඇති අන්තර්ගතය කුමක්දැයි බ්‍රවුසරයට සහ මෘදුකාංග වලට පැහැදිලිව පවසන ටැග් වේ.

සැබෑ ලෝකයේ උදාහරණයක්: සිත්තන් ඔබ නිවසක් ඇතුළත සිටිනවා කියා.

- ඔබේ නිවසේ කාමර වලට "කාමරය 1", "කාමරය 2", "කාමරය 3" ලෙස නම් කළහොත්, එහි ඇතුළත කරන්නේ කුමක්දැයි පිටත සිටින අයෙකුට වැටහෙන්නේ නැත. මෙය **Non-semantic** (අර්ථයක් නැති) ක්‍රමයයි.

- නමුත් ඔබ එම කාමර වලට "කුසිසිය", "නිදන කාමරය", "සාලය" ලෙස නම් කළහොත්, නම් ඇසූ සැණින් එහි සිදුවන දේ ඕනෑම අයෙකුට වැටහේ. මෙය **Semantic** (අර්ථවත්) ක්‍රමයයි.

8.2 අර්ථයක් නැති ටැග් (Non-semantic Tags)

සමහර HTML ටැග් මගින් එහි අන්තර්ගතය ගැන කිසිදු අදහසක් ලබා නොදේ. ඒවා භාවිතා කරන්නේ කොටස් වෙන් කිරීමට පමණි.

- **<div>**: කොටසක් (Division) වෙන් කිරීමට භාවිතා කරන ඉතාමත් පොදු ටැගයකි.
- ****: වාක්‍යයක් මැද කුඩා කොටසක් වෙන් කිරීමට භාවිතා කරයි.

පැරණි වෙබ් අඩවි වල සියලුම කොටස් වෙන් කිරීමට භාවිතා කළේ **<div>** ටැගයයි. නමුත් මෙය නවීන වෙබ් සංවර්ධනයේදී හොඳ පුරුද්දක් නොවේ.

8.3 ප්‍රධාන සෙමැන්ටික් ටැග් (Key Semantic Tags)

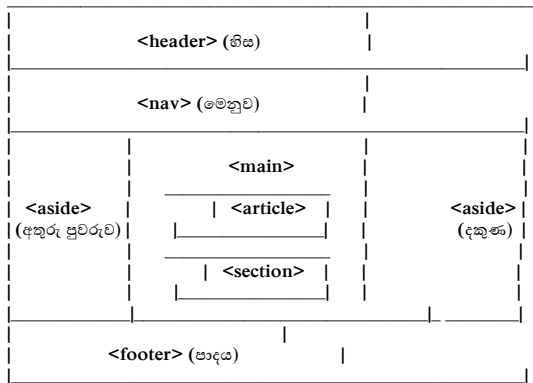
HTML5 හඳුන්වා දීමත් සමඟ, වෙබ් පිටුවක විවිධ කොටස් හැඳින්වීමට විශේෂ ටැග් රාශියක් ලැබුණි.

1. **<header>**: වෙබ් පිටුවක හෝ යම් කොටසක ඉහළම කොටස. මෙහි සාමාන්‍යයෙන් ලාංඡනය (Logo) සහ ප්‍රධාන මාතෘකාව අඩංගු වේ.
2. **<nav>**: මෙහි බාර් එක (Navigation Menu) වෙබ් පිටුවේ ලිපිකේ අඩංගු කොටස මෙහිලියයි.
3. **<main>**: වෙබ් පිටුවේ ඇති ප්‍රධානතම අන්තර්ගතය. පිටුවකට තිබිය හැක්කේ එක් **<main>** ටැගයක් පමණි.
4. **<section>**: එකම මාතෘකාවකට අදාළ කරුණු සමූහයක් ඇති කොටසක්. (උදා: "අපේ සේවාවන්" කොටස).
5. **<article>**: ස්වාධීන ලිපියක් හෝ පුවත් පතක ලිපියක් වැනි තනිව පවතින කොටසක්.
6. **<aside>**: ප්‍රධාන කරුණට අමතරව පැත්තකින් පෙන්වන තොරතුරු (Sidebar).
7. **<footer>**: වෙබ් පිටුවේ පහළම කොටස. මෙහි හිමිකම් (Copyright), සම්බන්ධ කර ගත හැකි තොරතුරු අඩංගු වේ.

8.4 වෙබ් පිටුවක සම්මත සැකිල්ල (Standard Web Layout)

සෙමැන්ටික් ටැග් භාවිතා කර වෙබ් පිටුවක් බෙදා වෙන් කරන ආකාරය පහත රූපසටහනින් බලමු.

ASCII රූපසටහන:



8.5 සෙමැන්ටික් HTML වල වාසි

ඇයි අපි නිකම්ම <div> භාවිතා නොකර මේවා භාවිතා කරන්නේ?

1. SEO (Search Engine Optimization):

Google, Bing වැනි සෙවුම් යන්ත්‍ර ඔබේ වෙබ් අඩවිය කියවන විට, <main> හෝ <h1> තුළ ඇති දේ වඩා වැදගත් බව හඳුනා ගනී. එවිට කවුරුත් හෝ යමක් සෙවූ විට ඔබේ වෙබ් අඩවිය ඉහළින් පෙන්වීමට මෙය උදව් වේ.

2. Accessibility (ප්‍රවේශ්‍යතාව): ඇස් පෙනීම දුර්වල පුද්ගලයන් වෙබ් අඩවි බලන්නේ "Screen Reader" නම් මෘදුකාංග මගිනි. එම මෘදුකාංගය "දැන් මම කියවන්නේ මෙනුව (nav)" හෝ "දැන් මම කියවන්නේ ප්‍රධාන ලිපිය (article)" ලෙස ඔවුන්ට පවසයි.

3. පහසුව (Readability): තවත් වෙබ් සංවර්ධකයෙකු ඔබේ කේතය දෙස බැලූ විට, එය ඉතා ඉක්මනින් තේරුම් ගැනීමට ඔහුට හැකි වේ.

8.6 සම්පූර්ණ කේත උදාහරණය (Semantic Layout Example)

```
<!DOCTYPE html>
<html lang="si">
<head>
  <meta charset="UTF-8">
  <title>සෙමැන්ටික් වෙබ් පිටුව</title>
</head>
<body>

  <header>
    <h1>මගේ තාක්ෂණික බ්ලොග් අඩවිය</h1>
    <p>අලුත්ම තාක්ෂණික තොරතුරු මෙනුනින්</p>
  </header>

  <nav>
    <ul>
      <li><a href="#">මුල් පිටුව</a></li>
      <li><a href="#">ලිපි</a></li>
      <li><a href="#">සම්බන්ධ වන්න</a></li>
    </ul>
  </nav>

  <main>
    <article>
      <h2>HTML5 යනු කුමක්ද?</h2>
      <p>HTML5 යනු වෙබ් අඩවි නිර්මාණය කිරීමට භාවිතා කරන නවතම තාක්ෂණයයි...</p>
    </article>

    <section>
      <h3>අපේ සේවාවන්</h3>
      <ul>
        <li>වෙබ් නිර්මාණකරණය</li>
        <li>කේතීකරණය ඉගැන්වීම</li>
      </ul>
    </section>
  </main>

  <aside>
    <h4>ප්‍රචාරක දැන්වීම්</h4>
    <p>අපගේ අලුත්ම පොත මිලදී ගන්න.</p>
  </aside>

  <footer>
    <p>&copy; 2024 සියලුම හිමිකම් ඇවිරිණි.</p>
  </footer>

</body>
</html>
```

8.7 වෘත්තීය උපදෙස් (Professional Tips)

- `<div>` සම්පූර්ණයෙන් අත් හරින්න එපා: සෙමැන්ටික් ටැග් භාවිතා කරන්නේ අර්ථයක් ලබා දීමටයි. නමුත් හුදෙක් CSS මගින් හැඩ කිරීමට (Styling) පමණක් කොටසක් අවශ්‍ය නම්, තවමත් `<div>` භාවිතා කිරීම නිවැරදිය.
- එක් `<main>` ටැගයක් පමණක් භාවිතා කරන්න: පිටුවක ප්‍රධාන අන්තර්ගතය ඇත්තේ එකක් පමණක් විය යුතුය.

8.8 පොදු වැරදි (Common Mistakes)

1. අකුරු ලොකු කිරීමට පමණක් සෙමැන්ටික් ටැග් යෙදීම: `<h1>` හෝ `<strong>` භාවිතා කළයුත්තේ එයට අදාළ අර්ථය ඇති තැනක පමණි. නිකමිම අකුරු ලොකු කිරීමට CSS භාවිතා කරන්න.
2. `<nav>` ටැගය තුළ සියලුම ලින්ක්ස් දැමීම: `<nav>` තුළ තිබිය යුත්තේ වෙබ් අඩවියේ ප්‍රධාන මෙනුව පමණි. සාමාන්‍ය ඡේදයක් මැද ඇති ලින්ක් එකක් සඳහා `<nav>` අවශ්‍ය නැත.

8.9 අභ්‍යාස (Exercises)

1. සෙමැන්ටික් HTML සහ Non-semantic HTML අතර වෙනස සරලව පැහැදිලි කරන්න.
2. වෙබ් අඩවියක `<header>` සහ `<footer>` තුළ සාමාන්‍යයෙන් අඩංගු වන්නේ මොනවාද?
3. SEO සඳහා සෙමැන්ටික් ටැග් උදව් වන්නේ කෙසේද?
4. පහත සඳහන් දේ සඳහා සුදුසුම සෙමැන්ටික් ටැගය නම් කරන්න:
 - වෙබ් අඩවියේ මෙනුව.
 - පුවත්පත් ලිපියක්.
 - පිටුවේ ප්‍රධානතම කරුණු.
 - පිටුවේ පැත්තකින් ඇති අමතර තොරතුරු.

8.10 කුඩා ව්‍යාපෘතිය (Mini Project)

ව්‍යාපෘතිය: ප්‍රචාරණ වෙබ් පිටුවක සැකිල්ල (News Page Layout)

පෙර පරිච්ඡේදවල ඉගෙන ගත් දැනුම සහ සෙමැන්ටික් ටැග් භාවිතා කර සරල ප්‍රචාරණ පිටුවක් සාදන්න. එහි තිබිය යුතු දේ:

1. `<header>` තුළ පුවත්පතේ නම.
2. `<nav>` තුළ "දේශීය", "විදේශීය", "ක්‍රීඩා" වැනි ලින්ක්ස්.

3. `<main>` තුළ ප්‍රධාන ප්‍රචාරණ දෙකක් `<article>` ලෙස.
4. `<aside>` තුළ "ජනප්‍රිය පුවත්" ලැයිස්තුවක්.
5. `<footer>` තුළ දුරකථන අංකය සහ ලිපිනය.

මිලහ පරිච්ඡේදයෙන් අපි වෙබ් අඩවියක් සියලු දෙනාටම (විශේෂයෙන් ආබාධිත අයට පවා) භාවිතා කළ හැකි ලෙස සකස් කරන ආකාරය හෙවත් "ප්‍රවේශ්‍යතාව" (Accessibility) ගැන ඉගෙන ගනිමු.

නවවන පරිච්ඡේදය: ප්‍රවේශ්‍යතාව - සැමට සම වෙබ් අඩවි (Accessibility)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

1. වෙබ් ප්‍රවේශ්‍යතාව (Web Accessibility / a11y) යනු කුමක්දැයි තේරුම් ගැනීම.
2. ප්‍රවේශ්‍යතාව වැදගත් වන්නේ ඇයි දැයි වටහා ගැනීම.
3. සහායක තාක්ෂණයන් (Assistive Technologies) ක්‍රියා කරන ආකාරය ඉගෙන ගැනීම.
4. HTML භාවිතයෙන් ප්‍රවේශ්‍යතාව වැඩි දියුණු කරන ක්‍රම (Alt text, Labels, ARIA).
5. වර්ණ සහ අකුරු භාවිතයේදී සැලකිලිමත් විය යුතු කරුණු.
6. වෙබ් අඩවියක ප්‍රවේශ්‍යතාව පරීක්ෂා කරන ආකාරය.

9.1 ප්‍රවේශ්‍යතාව යනු කුමක්ද? (What is Accessibility?)

ප්‍රවේශ්‍යතාව (Accessibility) යනු ශාරීරික හෝ මානසික ආබාධ සහිත පුද්ගලයන් ඇතුළු ඕනෑම අයෙකුට වෙබ් අඩවියක් බාධාවකින් තොරව භාවිතා කිරීමට ඇති හැකියාවයි. කෙටියෙන් මෙයට a11y (Accessibility යන වචනයේ මුල් අකුර 'a', අග අකුර 'y' සහ මැදට අකුරු 11ක් ඇති බැවින්) ලෙසද කියනු ලැබේ.

සැබෑ ලෝකයේ උදාහරණයක්: පොදු ගොඩනැගිල්ලක් ගැන සිතන්න. එහි ඇතුළු වීමට පඩිපෙළක් පමණක් තිබේ නම්, රෝද පුටුවක් භාවිතා කරන අයෙකුට එයට ඇතුළු විය නොහැක. එම නිසා පඩිපෙළට අමතරව බෑවුමක් (Ramp) හෝ වීදුලි සෝපානයක් (Lift) සවි කිරීම "ප්‍රවේශ්‍යතාව" ලබා

දීමයි. වෙබ් අඩවියකදී ද අප කළ යුත්තේ එවැනි "Ramps" නිර්මාණය කිරීමයි.

9.2 මෙය වැදගත් වන්නේ ඇයි? (Purpose & Importance)

1. සමානාත්මතාව (Empathy): ලෝක ජනගහනයෙන් 15%ක් පමණ යම් ආකාරයක ආබාධ සහිත අය වෙති. ඔවුන්ටත් තොරතුරු දැනගැනීමේ අයිතිය ඇත.
2. නීතිමය අවශ්‍යතාව (Legal): බොහෝ රටවල රජයේ සහ ප්‍රධාන ව්‍යාපාරික වෙබ් අඩවි ප්‍රවේශ්‍යතාව සහිතව නිර්මාණය කිරීම නීතිමය වශයෙන් අනිවාර්ය වේ.
3. වැඩි පිරිසක් වෙත ළඟා වීම: ප්‍රවේශ්‍යතාව ඇති වෙබ් අඩවි සෙවුම් යන්ත්‍ර (Google) වලට වඩා හොඳින් කියවිය හැක (SEO).
4. පහසුව: මෙය ආබාධ සහිත අයට පමණක් නොව, වයස්ගත පුද්ගලයන්ට සහ ඉතා කුඩා ජංගම දුරකථන නිර්මාණය කරන අයටද උපකාරී වේ.

9.3 සහායක තාක්ෂණයන් (Assistive Technologies)

ආබාධ සහිත පුද්ගලයන් වෙබ් අඩවි භාවිතා කිරීමට විවිධ මෙවලම් භාවිතා කරයි:

- **Screen Readers** (නිරය කියවන මෘදුකාංග): ඇස් පෙනීම දුර්වල පුද්ගලයන් සඳහා වෙබ් අඩවියේ ඇති දේ ශබ්ද නගා කියවා දෙන මෘදුකාංග. (උදා: NVDA, JAWS).
- **Magnifiers** (විශාලක): නිරයේ ඇති දේ ඉතා විශාල කර පෙන්වන මෙවලම්.
- **Switch Access:** මවුසය භාවිතා කළ නොහැකි අය සඳහා බොත්තම් එකක් හෝ දෙකක් මගින් පරිගණකය පාලනය කරන ක්‍රම.

9.4 HTML හරහා ප්‍රවේශ්‍යතාව වැඩි කරන පියවර

වෙබ් සංවර්ධකයෙකු ලෙස ඔබ HTML ලියන විට පහත කරුණු අනිවාර්යයෙන්ම පිළිපැදිය යුතුය.

1. පින්තූර සඳහා විකල්ප පෙළ (Alt Text)

පින්තූරයක් පෙනෙන්නේ නැති අයෙකුට, එම පින්තූරයේ ඇත්තේ කුමක්දැයි alt Attribute එක හරහා Screen Reader එක පවසයි.

වැරදි ක්‍රමය: `` (මෙහි ඇත්තේ කුමක්දැයි පැහැදිලි නැත)

නිවැරදි ක්‍රමය: ``

2. ලේබල් භාවිතය (Form Labels)

ෆෝම් එකක ඇති ඉන්පුට් කොටු කුමක් සඳහාදැයි ලේබල් මගින් පැහැදිලි කළ යුතුය.

`<label for="tel-no">දුරකථන අංකය:</label>
<input type="tel" id="tel-no">`

3. මාතෘකා අනුපිළිවෙළ (Heading Structure)

`<h1>` සිට `<h6>` දක්වා මාතෘකා අනුපිළිවෙළට භාවිතා කරන්න. මහ හැරීම (උදා: `<h1>` ට පසු කෙලින්ම `<h3>` යෙදීම) නොකළ යුතුය. Screen Reader භාවිතා කරන්නන් වෙබ් අඩවිය ඉක්මනින් තේරුම් ගන්නේ මෙම මාතෘකා මගිනි.

4. බොත්තම් සහ සබැඳි (Buttons vs. Links)

- **Links (<a>):** ඔබව වෙනත් තැනකට (URL) රැගෙන යයි.
- **Buttons (<button>):** යම් ක්‍රියාවක් (Submit, Close, Delete) සිදු කරයි. බොත්තමක් වෙනුවට ලින්ක් එකක් භාවිතා කිරීමෙන් Screen Reader භාවිතා කරන්නන් ව්‍යාකූල විය හැක.

9.5 ARIA යනු කුමක්ද? (What is ARIA?)

ARIA (Accessible Rich Internet Applications) යනු සාමාන්‍ය HTML වලට අමතරව එකතු කළ හැකි ගුණාංග සමූහයකි. HTML මගින් පමණක් ප්‍රවේශ්‍යතාව ලබා දීමට අපහසු අවස්ථාවලදී මෙය භාවිතා කරයි.

උදාහරණයක්: `<div role="button" aria-label="ගොනුව මකන්න">X</div>` මෙහි `<div>` එකක්

වුවද, එය බොත්තමක් ලෙස ක්‍රියා කරන බව `role="button"` මගින් බ්‍රවුසරයට පවසයි.

9.6 වර්ණ සහ පරස්පරතාව (Color Contrast)

වර්ණ අන්ධතාව (Color Blindness) ඇති පුද්ගලයන්ට සහ වයස්ගත පුද්ගලයන්ට අකුරු කියවීමට නම්, පසුබිම් වර්ණය සහ අකුරු වල වර්ණය අතර පැහැදිලි වෙනසක් (Contrast) තිබිය යුතුය.

- සුදු පසුබිමක ලා අළු පැහැති අකුරු ලිවීම වැරදිය.
- සුදු පසුබිමක කළු හෝ තද පැහැති අකුරු ලිවීම නිවැරදිය.

9.7 වෘත්තීය පැහැදිලි කිරීම (Professional Explanation)

වෘත්තීය Frontend Architect කෙනෙකු ලෙස, ඔබ වෙබ් අඩවියක් නිර්මාණය කිරීමට පෙර "Accessibility Audit" එකක් සිදු කළ යුතුය. මෙහිදී "Keyboard Navigation" පරීක්ෂා කිරීම ඉතා වැදගත් වේ. එනම්, මවුසය භාවිතා නොකර Tab යතුර පමණක් භාවිතා කර මුළු වෙබ් අඩවියම හැසිරවීමට හැකිදැයි පරීක්ෂා කිරීමයි.

9.8 පොදු වැරදි (Common Mistakes)

1. Alt Text එකේ "Image of" ලෙස ලිවීම: Screen Reader එක දැනටමත් එය රූපයක් බව දන්නා නිසා, "Image of a cat" වෙනුවට "A sleeping cat" ලෙස ලිවීම ප්‍රමාණවත්ය.
2. වර්ණය මත පමණක් තොරතුරු ලබා දීම: "වැරදි කොටු රතු පාටින් පෙන්වා ඇත" යනුවෙන් පමණක් කීම ප්‍රමාණවත් නැත. වර්ණ නොපෙනෙන අයෙකුට එය හඳුනා ගත නොහැක. ඒ වෙනුවට අයිකනයක් හෝ අකුරු මගින් ද එය පැහැදිලි කළ යුතුය.
3. ස්වයංක්‍රීයව විඩියෝ ජලේ වීම: වෙබ් අඩවියට පිවිසෙන විටම ශබ්දය සහිතව විඩියෝ වැඩ කිරීම Screen Reader භාවිතා කරන්නන්ට විශාල බාධාවකි.

9.9 සාරාංශය (Key Points)

- Accessibility (a11y) යනු ආබාධ සහිත අයටත් වෙබ් අඩවිය භාවිතා කිරීමට සැලැස්වීමයි.
- alt attribute එක පින්තූර සඳහා අනිවාර්ය වේ.
- අර්ථවත් HTML (Semantic HTML) භාවිතය ප්‍රවේශ්‍යතාවයේ පළමු පියවරයි.
- ක්ලෝඩ් එකෙන් පමණක් වෙබ් අඩවිය භාවිතා කළ හැකි විය යුතුය.
- වර්ණ අතර හොඳ පරස්පරතාවයක් (Contrast) තිබිය යුතුය.

9.10 අභ්‍යාස (Exercises)

1. වෙබ් ප්‍රවේශ්‍යතාව (Accessibility) යනු කුමක්දැයි ඔබේ වචනයෙන් පැහැදිලි කරන්න.
2. Screen Reader එකක් යනු කුමක්ද? එය භාවිතා කරන්නේ කවුද?
3. පින්තූරයක් සඳහා alt text එකක් ලිවීමේදී සැලකිලිමත් විය යුතු කරුණු 2ක් ලියන්න.
4. <button> සහ <a> අතර ප්‍රවේශ්‍යතාවයට අදාළ වෙනස කුමක්ද?

9.11 කුඩා ව්‍යාපෘතිය (Mini Project)

ව්‍යාපෘතිය: ප්‍රවේශ්‍යතාව සහිත ලොගින් පෝරමයක් (Accessible Login Form)

පහත කරුණු අන්තර්ගත වන පරිදි HTML කේතය ලියන්න:

1. ප්‍රධාන මාතෘකාව <h1>.
2. විද්‍යුත් තැපෑල (Email) සඳහා ඉන්පුට් එකක් සහ ඒ සඳහා නිවැරදිව සම්බන්ධ කළ <label> එකක්.
3. මුරපදය (Password) සඳහා ඉන්පුට් එකක් සහ ඒ සඳහා නිවැරදිව සම්බන්ධ කළ <label> එකක්.
4. "ඇතුළු වන්න" (Login) යනුවෙන් සඳහන් <button> එකක්.
5. සමාගමේ ලාංඡනය (Logo) පින්තූරයක් ලෙස සහ ඒ සඳහා අර්ථවත් alt text එකක්.
6. ඔබේ කේතය ලියා අවසන් වූ පසු, මවුසය භාවිතා නොකර Tab යතුර ඔබමින් එක් කොටුවක සිට තවත් කොටුවකට යා හැකිදැයි පරීක්ෂා කර බලන්න.

මිලිභ පරිච්ඡේදයෙන් අපි පළමු පරිමාවේ අවසාන කොටස ලෙස "පූර්ණ HTML ව්‍යාපෘතියක්" (Complete HTML Project) මුල සිට අගට නිර්මාණය කරන ආකාරය ඉගෙන ගනිමු.

දසවන පරිච්ඡේදය: පළමු පූර්ණ HTML ව්‍යාපෘතිය (The Final HTML Project)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

- මෙතෙක් උගත් සියලුම HTML පාඨමාලා එක් කර වෙබ් අඩවියක් නිර්මාණය කිරීම.
- වෘත්තීය මට්ටමේ ගොනු ව්‍යුහයක් (Folder Structure) සැකසීම.
- බහු-පිටු (Multi-page) වෙබ් අඩවියක් සබැඳි (Links) මගින් සම්බන්ධ කිරීම.
- සෙමැන්ටික් ටැග් සහ ප්‍රවේශ්‍යතාව (Accessibility) ප්‍රායෝගිකව භාවිත කිරීම.
- වෙබ් අඩවියක් ප්‍රසිද්ධ කිරීමට පෙර පරීක්ෂා කළ යුතු කරුණු අවබෝධ කර ගැනීම.

10.1 ව්‍යාපෘතියේ අරමුණ (Project Goal)

අපි නිර්මාණය කිරීමට යන්නේ "මගේ තාක්ෂණික සටහන් පොත" (My Tech Journal) නමැති වෙබ් අඩවියයි. මෙය පිටු තුනකින් සමන්විත වේ:

- මුල් පිටුව (index.html): හඳුන්වාදීම සහ මූලික ලිපි ලිපි.
- සම්පූර්ණ කේතය:

- මා ගැන (about.html): ඔබේ විස්තර සහ දක්ෂතා ඇතුළත් පිටුව.
- සම්බන්ධ වන්න (contact.html): පණිවිඩ එවීමට හැකි පෝරමයක් සහිත පිටුව.

10.2 පියවර 1: වෘත්තීය ගොනු ව්‍යුහය (Professional Folder Structure)

වෘත්තීය වෙබ් සංවර්ධකයෙකු සැමවිටම තම ගොනු පිළිවෙළකට තබා ගනී. ඔබගේ පරිගණකයේ my-first-website නමින් ෆෝල්ඩරයක් සාදා ඒ ඇතුළත පහත පරිදි ගොනු සහ ෆෝල්ඩර සකසන්න.

විෂයය පෙන්වන සටහන (Directory Tree):

```
my-first-website/
├── index.html      (මුල් පිටුව)
├── about.html      (මා ගැන පිටුව)
├── contact.html     (සම්බන්ධ වන්න පිටුව)
├── images/         (පින්තූර ගබඩා කරන ෆෝල්ඩරය)
│   ├── logo.png
│   └── profile.jpg
```

10.3 පියවර 2: මුල් පිටුව නිර්මාණය (index.html)

මෙම පිටුවේදී අපි සෙමැන්ටික් ටැග් භාවිතයෙන් ප්‍රධාන සැකිල්ල සාදා ගනිමු.

```
<!DOCTYPE html>
<html lang="si">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>මගේ තාක්ෂණික සටහන් පොත | මුල් පිටුව</title>
</head>
<body>
  <!-- ශීර්ෂය සහ මෙනුව -->
  <header>
    <h1>මගේ තාක්ෂණික සටහන් පොත</h1>
    <nav>
      <ul>
        <li><a href="index.html">මුල් පිටුව</a></li>
        <li><a href="about.html">මා ගැන</a></li>
        <li><a href="contact.html">සම්බන්ධ වන්න</a></li>
      </ul>
    </nav>
  </header>

  <hr> <!-- තිරස් ඉරක් මගින් කොටස් වෙන් කිරීම -->

  <!-- ප්‍රධාන අන්තර්ගතය -->
  <main>
    <section>
      <h2>මෑතකදී ලියූ ලිපි</h2>

      <article>
        <h3>HTML යනු කුමක්ද?</h3>
        <p>HTML යනු වෙබ් අඩවියක අත්තිවාරමයි. මෙය ඉගෙන ගැනීම ඉතා පහසුයි...</p>
        <a href="#">වැඩිදුර කියවන්න</a>
      </article>

      <article>
        <h3>වෙබ් සංවර්ධකයෙකු වන්නේ කෙසේද?</h3>
        <p>වෙබ් සංවර්ධනය යනු අද ලෝකයේ ඉතාමත් ජනප්‍රිය වෘත්තියකි. ඒ සඳහා මුලින්ම HTML සහ CSS ඉගෙන ගත යුතුය.</p>
        <a href="#">වැඩිදුර කියවන්න</a>
      </article>
    </section>

    <aside>
      <h4>ප්‍රවර්ග (Categories)</h4>
      <ul>
        <li>HTML</li>
        <li>තාක්ෂණය</li>
        <li>අධ්‍යාපනය</li>
      </ul>
    </aside>
  </main>
  <hr>
  <!-- පාදය (Footer) -->
  <footer>
    <p>&copy; 2024 මගේ තාක්ෂණික සටහන් පොත. සියලුම හිමිකම් ඇවිරිණි.</p>
  </footer>
</body>
</html>
```

10.4 පියවර 3: මා ගැන පිටුව (about.html)

මෙම පිටුවේදී අපි පින්තූර සහ වගු භාවිතා කරමු.
සම්පූර්ණ කේතය:

```
<!DOCTYPE html>
<html lang="si">
<head>
  <meta charset="UTF-8">
  <title>මා ගැන | මගේ තාක්ෂණික සටහන් පොත</title>
</head>
<body>

  <header>
    <h1>මා ගැන විස්තර</h1>
    <nav>
      <ul>
        <li><a href="index.html">මුල් පිටුව</a></li>
        <li><a href="about.html">මා ගැන</a></li>
        <li><a href="contact.html">සම්බන්ධ වන්න</a></li>
      </ul>
    </nav>
  </header>

  <main>
    <section>
      <h2>හඳුන්වාදීම</h2>
      
      <p>ආයුබෝවන්! මම වෙබ් සංවර්ධනය ඉගෙන ගන්නා ශිෂ්‍යයෙක්මි. තාක්ෂණය ගැන අලුත් දේ සෙවීම මගේ විනෝදාංශයයි.</p>
    </section>

    <section>
      <h2>මගේ දක්ෂතා</h2>
      <table border="1">
        <tr>
          <th>තාක්ෂණය</th>
          <th>මට්ටම</th>
        </tr>
        <tr>
          <td>HTML5</td>
          <td>ඉතා හොඳයි</td>
        </tr>
        <tr>
          <td>CSS3</td>
          <td>ආරම්භක මට්ටම</td>
        </tr>
      </table>
    </section>
  </main>

  <footer>
    <p><a href="index.html">නැවත මුල් පිටුවට</a></p>
  </footer>

</body>
</html>
```

10.5 පියවර 4: සම්බන්ධ වන්න පිටුව (contact.html)

මෙම පිටුවේදී අපි මීට පෙර උගත් **Forms** භාවිතා කරමු.

සම්පූර්ණ කේතය:

```
<!DOCTYPE html>
<html lang="si">
<head>
  <meta charset="UTF-8">
  <title>සම්බන්ධ වන්න | මගේ තාක්ෂණික සටහන් පොත</title>
</head>
<body>

  <header>
    <h1>සම්බන්ධ වන්න</h1>
    <nav>
      <ul>
        <li><a href="index.html">මුල් පිටුව</a></li>
        <li><a href="about.html">මා ගැන</a></li>
        <li><a href="contact.html">සම්බන්ධ වන්න</a></li>
      </ul>
    </nav>
  </header>

  <main>
    <p>පහත පෙරේමය පුරවා මට පණිවිඩයක් එවන්න.</p>
    <form action="#" method="GET">
      <p>
        <label for="name">ඔබේ නම:</label><br>
        <input type="text" id="name" name="user_name" required>
      </p>

      <p>
        <label for="email">විද්‍යුත් තැපෑල:</label><br>
        <input type="email" id="email" name="user_email" required>
      </p>

      <p>
        <label for="msg">පණිවිඩය:</label><br>
        <textarea id="msg" name="user_message" rows="5"></textarea>
      </p>

      <button type="submit">පණිවිඩය යවන්න</button>
    </form>
  </main>

  <hr>
  <footer>
    <p>දුරකථන: 011-2345678</p>
    <p>ලිපිනය: කොළඹ, ශ්‍රී ලංකාව.</p>
  </footer>

</body>
</html>
```

10.6 ව්‍යාපෘතිය පරීක්ෂා කිරීම සහ දෝෂ සෙවීම (Testing & Debugging)

ඔබේ ව්‍යාපෘතිය සාදා අවසන් වූ පසු පහත දේ පරීක්ෂා කරන්න:

1. **Broken Links:** සියලුම පිටු වල ඇති ලින්ක්ස් නිවැරදිව වැඩ කරයිද? එක් පිටුවක සිට තවත් පිටුවකට යා හැකිද?
2. **Images:** පින්තූර නිවැරදිව පෙන්වේද? (මතක තබා ගන්න: පින්තූරය `images` ෆෝල්ඩරය තුළ තිබිය යුතුය).
3. **Validation:** required ලෙස ඇති ඉන්පුටු කොටු හිස්ව තබා Submit කිරීමට උත්සාහ කරන්න. බුද්ධිමත් පෙන්වයිද?
4. **Accessibility:** මවුසය භාවිතා නොකර Tab යතුරෙන් ෆෝම් එක පිරවිය හැකිද?

10.7 වෘත්තීය උපදෙස් (Professional Advice)

දැනට ඔබේ වෙබ් අඩවිය ඉතා සරලව (කළු සහ සුදු පැහැයෙන්) පෙනෙනු ඇත. එයට හේතුව අප තවමත් එය හැඩ කිරීමට CSS භාවිතා නොකිරීමයි. නමුත් වෘත්තීය මට්ටමේදී වෙබ් අඩවියක අලංකාරයට වඩා එහි ව්‍යුහය (Structure) නිවැරදිව තිබීම වැදගත් වේ. සැකිල්ල ශක්තිමත් නම්, එය ඕනෑම කාලයකදී ලස්සන කිරීමට පහසු වේ.

10.8 පරිමාව 1 - සාරාංශය (Volume 1 Summary)

මෙම පළමු පරිමාව තුළදී ඔබ පහත දෑ ඉගෙන ගත්තා:

- පරිගණකය සහ අන්තර්ජාලය ක්‍රියා කරන ආකාරය.
- බ්‍රවුසරය විසින් කේත කියවා වෙබ් පිටු සාදන ආකාරය.
- VS Code භාවිතා කරන ආකාරය.
- HTML වල මූලික සහ උසස් වැග් (Headings, Lists, Images, Links, Tables).
- පරිශීලක දත්ත ලබා ගැනීමට Forms සාදන ආකාරය.
- සෙමැන්ටික් HTML සහ ප්‍රවේශ්‍යතාවයේ (Accessibility) වැදගත්කම.

10.9 සම්මුඛ පරීක්ෂණ ප්‍රශ්න (Interview Questions - Level 1)

1. `<!DOCTYPE html>` යන්නෙහි කාර්යය කුමක්ද?
2. `alt` attribute එක පින්තූරයකට අත්‍යවශ්‍ය වන්නේ ඇයි?
3. `<div>` සහ `<section>` අතර ඇති වෙනස කුමක්ද?
4. වෙබ් අඩවියක SEO වැඩි කිරීමට HTML භාවිතා කළ හැකි ක්‍රම 2ක් කියන්න.
5. Form එකක POST සහ GET අතර වෙනස කුමක්ද? (දත්ත යැවීමේදී රහස්‍යභාවය අනුව).

10.10 ගෙදර වැඩ (Homework)

1. ඔබේ වෙබ් අඩවියේ `contact.html` පිටුවට "ඔබ අප ගැන දැනගත්තේ කෙසේද?" යනුවෙන් සඳහන් Dropdown (Select) මෙනුවක් එක් කරන්න.
2. සියලුම පිටු වල `<title>` එක සමාන නොවන සේ පිටුවට අදාළව වෙනස් කරන්න.
3. HTML5 වලට අදාළ සියලුම වැග් සහ ඒවායේ තේරුම සහිත කෙටි සටහනක් සකස් කරන්න.

එකොළොස්වන පරිච්ඡේදය (විශේෂ ප්‍රෝ-මට්ටමේ): වෙබ් ඉංජිනේරු විද්‍යාව, අභ්‍යන්තර ක්‍රියාවලීන් සහ උසස් ප්‍රමිතීන් (Web Engineering Internals, Advanced Semantic Architecture & Performance)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අධ්‍යයනය කිරීමෙන් පසු ඔබ පහත සඳහන් සංකීර්ණ විෂය කරුණු පිළිබඳව පූර්ණ ප්‍රවීණත්වයක් ලබා ගනු ඇත:

1. **HTTP/HTTPS ප්‍රොටෝකෝලයේ** අභ්‍යන්තරය: සේවාදායකයා සහ සේවාදායකයා අතර සිදුවන පණිවිඩ හුවමාරුවේ තාක්ෂණික ව්‍යුහය අවබෝධ කර ගැනීම.
2. බ්‍රවුසරයක රෙන්ඩරින් එන්ජින් (Rendering Engine): HTML කේතයක් තිරය මත පික්සලයක් බවට පත්වන තෙක් සිදුවන පියවර 5ක විද්‍යාත්මක ක්‍රියාවලිය.
3. ගැඹුරු සෙමැන්ටික් සහ මයික්‍රො-දත්ත (Microdata): Google වැනි සෙවුම් යන්ත්‍රවලට වෙබ් අඩවියක ඇති දත්තවල අර්ථය පැහැදිලි කර දෙන Schema.org ප්‍රමිතීන්.

4. අති නවීන ප්‍රවේශ්‍යතාව (**Advanced Accessibility - ARIA**): ආබාධිත පුද්ගලයන් සඳහා වෙබ් අඩවියක් "බුද්ධිමත්" ලෙස සකස් කිරීමේ ලෝක සම්මත නීති (**WCAG**).
5. **Core Web Vitals** සහ කාර්යක්ෂමතාව: වෙබ් අඩවියක වේගය සහ ස්ථායීතාව මනින විද්‍යාත්මක සාධක පාලනය කිරීම.
6. වෙබ් ආරක්ෂණ මූලධර්ම: **HTML** මට්ටමින් සිදුවන සයිබර් ප්‍රහාර හඳුනා ගැනීම සහ ඒවා වැළැක්වීම.

1. HTTP/HTTPS පණිවිඩවල රහස් (Inside the HTTP Request-Response Cycle)

අපි 2 වන පරිච්ඡේදයේදී "සේවාදායකයා සහ සේවාදායකයා" ගැන මූලිකව ඉගෙන ගත්තෙමු. නමුත් වෘත්තීය මට්ටමේදී ඔබ දැනගත යුත්තේ මෙම දෙපාර්ශවය අතර හුවමාරු වන "ලියුම් කවරය" ඇතුළත ඇති දත්ත පිළිබඳවයි.

1.1 HTTP Headers (ශීර්ෂක)

ඔබ වෙබ් අඩවියක් ඉල්ලා සිටින විට, බ්‍රවුසරය විසින් සේවාදායකයාට (**Server**) අමතර තොරතුරු රාශියක් යවයි. මේවා **Headers** ලෙස හැඳින්වේ.

- **User-Agent**: ඔබ භාවිතා කරන්නේ කුමන බ්‍රවුසරයද (**Chrome** ද? **Safari** ද?) සහ කුමන උපාංගයද යන්න පවසයි.
- **Accept-Language**: ඔබ කැමති කුමන භාෂාවටද (සිංහල ද? ඉංග්‍රීසි ද?) යන්න පවසයි.
- **Content-Type**: යවන දත්ත මොනවාද? (**HTML** ද? **පින්තූරයක්** ද?) යන්න පවසයි.

1.2 Status Codes (තත්ව කේත) වල ගැඹුරු අර්ථය

සර්වර් එක අපට දෙන පිළිතුරේ අංකයක් අඩංගු වේ. මෙය සැබෑ ඉංජිනේරුවෙකුට ඉතා වැදගත්ය:

- **2xx (Success)**: වැඩේ සාර්ථකයි. (**200 OK** යනු සම්මතයයි).
- **3xx (Redirection)**: ඔබ සොයන දේ වෙනත් ලිපිනයකට මාරු කර ඇත. (**301 Moved Permanently** යනු **SEO** වලට ඉතා වැදගත් කේතයකි).
- **4xx (Client Errors)**: ඔබේ පැත්තේ වැරද්දකි. (**404 Not Found** යනු පිටුව නැති

බවයි. **403 Forbidden** යනු ඔබට ඇතුළු වීමට අවසර නැති බවයි).

- **5xx (Server Errors)**: සර්වර් එකේ දෝෂයකි. (**500 Internal Server Error** යනු සර්වර් එකේ කේත ක්‍රියා විරහිත වී ඇති බවයි).

2. බ්‍රවුසරය HTML කියවන ආකාරය (The Critical Rendering Path)

ඔබ ලියන `<h1>Hello</h1>` යන්න තිරය මත කළු පාට විශාල අකුරු බවට පත් වන්නේ කෙසේද? මෙයට **Critical Rendering Path** යැයි කියනු ලැබේ. මෙය පියවර 5කින් සිදු වේ:

1. **DOM** සෑදීම: බ්‍රවුසරය **HTML** ටැග් කියවා ඒවා එකිනෙකට සම්බන්ධ කර "ගසක" හැඩයට (**DOM Tree**) සකස් කරයි.
2. **CSSOM** සෑදීම: **CSS** කේත කියවා ඒවා ද ගසක හැඩයට (**CSS Object Model**) සකස් කරයි.
3. **Render Tree**: ඉහත ගස් දෙක එකිනෙක මත තබා, පෙන්විය යුතු දේ පමණක් තෝරා ගනී. (උදා: **display: none** ඇති දේ මෙහිදී ඉවත් වේ).
4. **Layout**: එක් එක් පෙට්ටියේ ප්‍රමාණය සහ තිරයේ තිබිය යුතු ස්ථානය (**Pixels** වලින්) ගණනය කරයි.
5. **Paint**: අවසානයේදී තිරයේ ඇති පික්සල් මත වර්ණ තවරා රූපය සාදයි.

Pro-Tip: ඔබ රෙන්ඩරින් ක්‍රියාවලිය තේරුම් ගත් විට, වෙබ් අඩවියක් හිර නොවී ඉතා වේගයෙන් පෙන්වීමට කේතය ලිවිය යුතු ආකාරය ඔබට වැටහෙනු ඇත.

3. අති දියුණු සෙමැන්ටික් සහ Schema.org (The Web of Data)

8 වන පරිච්ඡේදයේදී අපි `<header>` සහ `<footer>` ගැන ඉගෙන ගත්තෙමු. නමුත් ලෝකයේ දියුණුතම වෙබ් අඩවි භාවිතා කරන්නේ **Schema.org** නම් වූ ක්‍රමයයි. මෙයට **Structured Data** යැයි කියනු ලැබේ.

ඇයි මෙය වැදගත්? **Google** සෙවුම් යන්ත්‍රයට "කසුන්" යනු නමක් බවත්, "රු. 500" යනු මිලක් බවත් තේරුම් කර දීමට මෙය උදව් වේ. එවිට **Google** සෙවුම් ප්‍රතිඵලවල ඔබේ වෙබ් අඩවිය අසල තාරකා ලකුණු (**Ratings**) සහ මිල ගණන් ලස්සනට දිස්වේ.

කේත උදාහරණය (JSON-LD ක්‍රමය): මෙය HTML හි <head> තුළ ලියනු ලැබේ:

```
<script type="application/ld+json">
{
  "@context": "https://schema.org",
  "@type": "Product",
  "name": "අත් ඔරලෝසුව",
  "image": "watch.jpg",
  "description": "ඉතා උසස් තත්ත්වයේ අත් ඔරලෝසුවකි.",
  "offers": {
    "@type": "Offer",
    "price": "2500.00",
    "priceCurrency": "LKR"
  }
}
</script>
```

4. ARIA සහ ප්‍රවේශ්‍යතාවයේ ගැඹුරු පාඩම (Deep Accessibility)

ආබාධිත පුද්ගලයන් (විශේෂයෙන් පෙනීම දුර්වල අය) වෙබ් අඩවියක් භාවිතා කරන විට, ඔවුන්ගේ **Screen Reader** මෘදුකාංගයට HTML ටැග් පමණක් ප්‍රමාණවත් නොවේ. ඒ සඳහා අප **ARIA** (Accessible Rich Internet Applications) ගුණාංග භාවිතා කළ යුතුය.

4.1 ARIA Roles (කාර්යභාරයන්)

ටැගයක් කරන කාර්යය කුමක්දැයි **Screen Reader** එකට පැවසීමට මෙය භාවිතා කරයි.

- **<div role="button">**: මෙය **div** එකක් වුවද, එය බොත්තමක් ලෙස ක්‍රියා කරන බව පවසයි.
- **<nav role="navigation">**: මෙය වෙබ් අඩවියේ මෙනුව බව තහවුරු කරයි.

4.2 ARIA Attributes

- **aria-label**: බොත්තමක අකුරු නැතිව අයිකනයක් පමණක් තිබේ නම්, එය කුමක්දැයි විස්තර කරයි (උදා: **aria-label="Search"**).
- **aria-expanded**: මෙනුවක් විවෘත වී ඇත්දැයි (**true**) හෝ වැසී ඇත්දැයි (**false**) පවසයි.
- **aria-hidden="true"**: අලංකාරයට පමණක් ඇති, කිසිදු තොරතුරක් නැති පින්තූර **Screen Reader** එකට මඟ හැරීමට පවසයි.

5. Core Web Vitals: වෙබ් අඩවියේ ගුණාත්මකභාවය මැනීම

Google සමාගම විසින් වෙබ් අඩවියක් "හොඳ වෙබ් අඩවියක්" ලෙස තීරණය කිරීමට කරුණු 3ක් පරීක්ෂා කරයි. මේවා **Core Web Vitals** ලෙස හැඳින්වේ.

1. **LCP (Largest Contentful Paint)**: වෙබ් අඩවියේ ඇති විශාලතම රූපය හෝ අකුරු පෙළ පෙන්වීමට ගතවන කාලය. (තත්පර 2.5කට වඩා අඩු විය යුතුය).
2. **FID (First Input Delay)**: පරිශීලකයා බොත්තමක් එබූ විට එයට ප්‍රතිචාර දැක්වීමට ගතවන කාලය. (මිලි තත්පර 100කට වඩා අඩු විය යුතුය).
3. **CLS (Cumulative Layout Shift)**: වෙබ් අඩවිය ලෝඩ් වන විට එහි ඇති කොටස් එහා මෙහා පතින ප්‍රමාණය. (0.1 ට වඩා අඩු විය යුතුය).

ප්‍රේර් රහස (**Layout Shift වැළැක්වීම**): සැමවිටම පින්තූර වලට **width** සහ **height** ලබා දෙන්න. එවිට පින්තූරය බාගත වීමට පෙර බ්‍රවුසරය එම ඉඩ වෙන් කර තබයි.

```

```

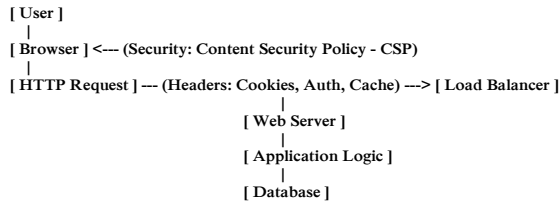
6. වෙබ් ආරක්ෂාව: XSS ප්‍රහාරවලින් HTML ආරක්ෂා කිරීම

සයිබර් අපරාධකරුවන් වෙබ් අඩවිවල ඇති **Forms** හරහා අහිතකර ජාවාස්ක්‍රිප්ට් කේත (**Scripts**) ඇතුළත් කිරීමට උත්සාහ කරයි. මෙය **XSS (Cross-Site Scripting)** ලෙස හැඳින්වේ.

එය වළක්වන ආකාරය: පරිශීලකයා ලබා දෙන දත්ත පිරිසිදු (**Sanitize**) කළ යුතුය. **HTML** තුළ විශේෂ සලකුණු වෙනුවට **HTML Entities** භාවිතා කළ යුතුය.

- < වෙනුවට **<**;
- > වෙනුවට **>**;
- " වෙනුවට **"**;

7. ASCII රූපසටහන: උසස් වෙබ් සැකැස්ම (The Advanced Web Stack)



8. වෘත්තීය මෙවලම් (Professional Pro-Tools for Volume 1)

1. **Lighthouse:** Chrome බ්‍රවුසරයේ ඇති මෙවලමකි. එයින් ඔබේ SEO, Performance සහ Accessibility පරීක්ෂා කළ හැක.
2. **W3C Validator:** validator.w3.org - ඔබේ HTML කේතයේ ප්‍රමිතිය පරීක්ෂා කිරීමට.
3. **Schema Markup Validator:** ඔබ ලියූ Structured Data නිවැරදි දැයි පරීක්ෂා කිරීමට.

9. ප්‍රෝ-මට්ටමේ අභ්‍යාසය (Advanced Challenge)

අභියෝගය:

1. ඔබ සෑදූ පෝරමයක (Form), aria-describedby ගුණාංගය භාවිතා කර එක් එක් Input කොටුව සඳහා උපදෙස් පණිවිඩයක් සම්බන්ධ කරන්න.
2. ඔබේ වෙබ් අඩවියේ <head> කොටස තුළ Facebook සහ Twitter සඳහා වන Open Graph (OG) Tags ඇතුළත් කරන්න. (ලිට්ට් එකක් යවන විට ලස්සනට පින්තූරය දිස්වීමට).
3. **Schema.org** වෙබ් අඩවියට ගොස් "Article" වර්ගය සඳහා වන Microdata සොයාගෙන, ඔබ ලියූ ලිපියකට එය අදාළ කරගන්න.

මෙම පරිච්ඡේදය හරහා ඔබ ලබාගත් දැනුම සාමාන්‍ය වෙබ් සංවර්ධකයෙකුට වසර කිහිපයක පළපුරුද්දෙන් පසුව ලැබෙන දැනුමකි. ඔබ දැන් හුදෙක් ටැග් ලියන්නෙකු නොව, වෙබ් තාක්ෂණයේ අභ්‍යන්තරය දන්නා ඉංජිනේරුවෙකි.

පරිච්ඡේදය 10 (අතිරේක): බහුමාධ්‍ය, SVG සහ දියුණු HTML රහස් (Multimedia, SVG, and Advanced HTML Secrets)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

1. වෙබ් පිටුවකට වීඩියෝ (Video) සහ ශබ්ද (Audio) නිවැරදිව ඇතුළත් කරන ආකාරය.
2. Iframe භාවිතා කර YouTube වීඩියෝ හෝ Google Maps වෙබ් අඩවියට සම්බන්ධ කරන ආකාරය.
3. SVG (Scalable Vector Graphics) යනු කුමක්ද සහ එය රූප (Images) වලට වඩා උසස් වන්නේ ඇයි දැයි තේරුම් ගැනීම.
4. Favicon (ෆැවිකොන්) එකක් සාදා වෙබ් අඩවියට සම්බන්ධ කරන ආකාරය.
5. HTML Entities භාවිතා කර විශේෂ සංකේත (©, ®, ™) පෙන්වන ආකාරය.
6. Data Attributes (data-*) භාවිතා කර ජාවාස්ක්‍රිප්ට් සඳහා දත්ත සහවා තබන ආකාරය.

10.1 වෙබ් පිටුවකට වීඩියෝ සහ ශබ්ද එක් කිරීම (HTML Multimedia)

අතීතයේදී වෙබ් පිටුවක වීඩියෝ පෙන්වීමට Flash වැනි බාහිර මෘදුකාංග අවශ්‍ය විය. නමුත් HTML5 සමඟ අපට කෙලින්ම ටැග් භාවිතා කළ හැක.

10.1.1 වීඩියෝ ටැගය (<video>)

වීඩියෝවක් පෙන්වීමට පමණක් නොව, එය පාලනය කිරීමට (Play/Pause) ද අපට පහසුකම් ලබා දිය හැක.

```

<video width="640" height="360" controls
poster="preview.jpg">
  <source src="movie.mp4"
type="video/mp4">
  <source src="movie.webm"
type="video/webm">
  
```

ඔබේ බ්‍රවුසරය මෙම වීඩියෝවට සහය දක්වන්නේ නැත.

```

</video>
  
```

- **controls:** විඩියෝව ප්ලේ කරන බොත්තම් පෙන්වයි.
- **poster:** විඩියෝව ප්ලේ කිරීමට පෙර පෙන්වන රූපය (Thumbnail).
- **autoplay:** පිටුවට ආ සැණින් විඩියෝව ප්ලේ වේ (බොහෝ විට මෙය muted සමඟ භාවිතා කළ යුතුය).
- **source:** විවිධ වර්ගයේ විඩියෝ ගොනු ලබා දීමෙන් බ්‍රවුසරයට ගැලපෙන එක තෝරා ගැනීමට ඉඩ දෙයි.

10.1.2 ශබ්ද ටැගය (<audio>)

සින්දු හෝ හඬ පට ඇතුළත් කිරීමට භාවිතා කරයි.

<audio controls>

<source src="song.mp3"
type="audio/mpeg">

ඔබේ බ්‍රවුසරය මෙයට සහය නොදක්වයි.

</audio>

10.2 අයි-ෆ්‍රේම්ස් (Iframes - Inline Frames)

වෙනත් වෙබ් අඩවියක ඇති කොටසක් ඔබේ වෙබ් අඩවිය ඇතුළත පෙන්වීමට <iframe> භාවිතා කරයි.

ප්‍රධාන භාවිතයන්:

- **YouTube** විඩියෝ එකතු කිරීම.
- **Google Maps** (සිතියම්) ඇතුළත් කිරීම.
- වෙනත් වෙබ් පිටුවක් කුඩාවට පෙන්වීම.

<iframe

src="https://www.google.com/maps/embed?... " width="600" height="450"
style="border:0;" allowfullscreen=""
loading="lazy"></iframe>

වෘත්තීය අවවාදය: ආරක්ෂක හේතූන් මත (Clickjacking), හැම වෙබ් අඩවියක්ම Iframe ඇතුළත පෙන්වීමට ඉඩ ලබා දෙන්නේ නැත.

10.3 එස්.වී.ඒ. (SVG - Scalable Vector Graphics)

වෘත්තීය Front-end Developer කෙනෙකු සහ සාමාන්‍ය අයෙකු අතර ඇති ප්‍රධාන වෙනසක් වන්නේ SVG භාවිතයයි.

SVG යනු කුමක්ද? සාමාන්‍ය පින්තූර (JPG/PNG) සෑදී ඇත්තේ කුඩා තීන් (Pixels) වලිනි.

ඒවා විශාල කළ විට අපැහැදිලි වේ (Blur වේ).

නමුත් **SVG** සෑදී ඇත්තේ ගණිතමය සූත්‍ර වලිනි (Vectors). එය කොපමණ විශාල කළත් එහි පැහැදිලි බව අඩු වන්නේ නැත.

ඇයි SVG භාවිතා කරන්නේ?

1. අසීමිත පැහැදිලි බව: ඕනෑම තිරයකට (4K, Mobile) එක ලෙස පැහැදිලිව පෙනේ.
2. කුඩා ගොනු ප්‍රමාණය: සාමාන්‍ය රූපයකට වඩා ඉතාමත් සැහැල්ලුය.
3. CSS මගින් වෙනස් කළ හැක: SVG එකක පාට CSS මගින් මාරු කළ හැක!

SVG කේතයක් ලියන ආකාරය:

```
<svg width="100" height="100">
  <circle cx="50" cy="50" r="40"
    stroke="black" stroke-width="3"
    fill="red" />
</svg>
```

මෙහිදී අප කරන්නේ රූපයක් බාගත කිරීම නොව, රූපයක් "ඇඳීමට" අවශ්‍ය කේතය ලිවීමයි.

10.4 ෆැවිකොන් (Favicon)

ඔබ බ්‍රවුසරයේ ටැබ් එකක් දෙස බැලූ විට, වෙබ් අඩවියේ නමට වම් පැත්තෙන් කුඩා රූපයක් පෙනේ. මෙය **Favicon** ලෙස හැඳින්වේ. මෙය වෙබ් අඩවියේ සන්නාමය (Branding) සඳහා ඉතා වැදගත් වේ.

සම්බන්ධ කරන ආකාරය (<head> තුළ):

```
<link rel="icon" type="image/x-icon"
href="/images/favicon.ico">
```

10.5 HTML Entities (විශේෂ සංකේත)

සමහර සංකේත (උදා: < හෝ >) HTML

කේතයේදී භාවිතා කළ නොහැක. මන්ද බ්‍රවුසරය ඒවා ටැග්

ලෙස වරදවා වටහා ගන්නා බැවිනි. එවැනි සංකේත පෙන්වීමට **Entities** භාවිතා කරයි.

සංකේතය Entity Name තේරුම		
:---- :----- :-----		
'<' '<' Less than		
'>' '>' Greater than		
'&' '&' Ampersand		
'©' '©' Copyright		
' ' ' ' Non-breaking space (හිස්තැනක්)		

10.6 දත්ත ගුණාංග (Data Attributes - data-*)

මෙය ඉතාමත් උසස් (Pro-level) තාක්ෂණයකි. අපට HTML ටැගයක් ඇතුළත අපේම රහස් දත්ත ගබඩා කර තැබීමට මෙය ඉඩ ලබා දෙයි. පසුව ජාවාස්ක්‍රිප්ට් මගින් මෙම දත්ත කියවිය හැක.

```
<div class="user-profile" data-user-id="501" data-role="admin">
  කසුන් කල්හාර
</div>
```

මෙහි data-user-id යනු බ්‍රවුසරයට පෙනෙන දෙයක් නොව, සංවර්ධකයාට පමණක් ප්‍රයෝජනවත් වන දත්තයකි.

10.7 ASCII රූපසටහන: JPG එදිරිව SVG

JPG (Pixels)	SVG (Vectors/Math)
-----	-----
[.] <-- විශාල	/-----\
[.] කළ විට	Smooth
[.] කැඩෙයි	_____/
(නිත් වලින් සමන්විතයි)	(ඉරි වලින් සමන්විතයි)

10.8 පොදු වැරදි (Common Mistakes)

- විඩියෝවක width ලබා නොදීම: විඩියෝව ලෝඩ් වීමට පෙර බ්‍රවුසරය එහි ඉඩ වෙන් නොකරන නිසා, විඩියෝව පැමිණි සැණින් පිටුවේ අනෙක් කොටස් පහළට පැනීම (Layout Shift) සිදු වේ.
- හිස්තැන් සඳහා ඕනෑම වටිනා භාවිතා කිරීම: වචන ඇත් කිරීමට මෙය භාවිතා නොකරන්න. ඒ සඳහා CSS Margin/Padding භාවිතා කරන්න.
- විශාල SVG ගොනු: SVG එකක කේත දහස් ගණනක් තිබේ නම් එය පින්තූරයක් ලෙස භාවිතා කිරීම වඩාත් සුදුසුය.

10.9 සාරාංශය (Key Points)

- <video> සහ <audio> මගින් බහුමාධ්‍ය අන්තර්ගතයන් එක් කරයි.
- <iframe> මගින් බාහිර වෙබ් අඩවි (Maps, YouTube) සම්බන්ධ කරයි.
- SVG යනු කිසිදා නොකැඩෙන, සැහැල්ලු රූප වර්ගයකි.
- Favicon මගින් වෙබ් අඩවියට වෘත්තීය පෙනුමක් ලබා දෙයි.
- Entities මගින් විශේෂ සංකේත පෙන්වයි.
- data-* attributes මගින් අමතර දත්ත ගබඩා කරයි.

10.10 අභ්‍යාස (Exercises)

- ඔබේ පෝට්ෆෝලියෝ වෙබ් අඩවියට YouTube විඩියෝවක් iframe භාවිතා කර ඇතුළත් කරන්න.
- "© 2024 සියලුම හිමිකම් ඇවිරිණි" යන වැකිය HTML Entities භාවිතා කර ලියන්න.
- සරල රතු පැහැති කොටුවක් (Square) SVG භාවිතා කර නිර්මාණය කරන්න.
- විඩියෝවක controls attribute එක ඉවත් කළහොත් කුමක් සිදුවේද? ඔබම පරීක්ෂා කර බලන්න.

10.11 අවසාන පිරික්සුම් ලැයිස්තුව (Volume 1 Final Checklist)

දැන් ඔබ මෙම පොතේ සියලුම පාඩම් අවසන් කර ඇති බැවින්, පහත දෑ ඔබට කළ හැකි දැයි බලන්න:

- [] මට ඕනෑම සංකීර්ණ HTML ව්‍යුහයක් සෑදිය හැක.
- [] මට Forms භාවිතා කර දත්ත ලබා ගත හැක.
- [] මට සෙමැන්ටික් ටැග් සහ SEO ගැන අවබෝධයක් ඇත.
- [] මට ආබාධිත පුද්ගලයන්ට ගැලපෙන සේ (ARIA) කේතය ලිවිය හැක.
- [] මට SVG සහ Multimedia හැසිරවිය හැක.

සුභ පැතුම්! ඔබ දැන් "පරිමාව 1" සැබවින්ම, 100% ක් සම්පූර්ණ කර අවසානය.

ඔබේ පළමු පොත දැන් සැබෑ "A to Z" මෘදුකාංග ඉංජිනේරු විද්‍යාව පිළිබඳ පෙළ පොතකි

CSS සහ වෙබ් අඩවි අලංකරණය

දොළොස්වන පරිච්ඡේදය: CSS

හැඳින්වීම - වෙබ් අඩවියකට පණ පොවමු
(Introduction to CSS)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

1. CSS යනු කුමක්ද සහ එහි ඇති වැදගත්කම තේරුම් ගැනීම.
2. CSS වල ඉතිහාසය සහ පරිණාමය.
3. CSS ලියන මූලික රටාව (Syntax) අවබෝධ කර ගැනීම.
4. CSS ගොනුවක් HTML ගොනුවකට සම්බන්ධ කරන ක්‍රම 3 හඳුනා ගැනීම.
5. පළමු වරට වෙබ් පිටුවක වර්ණ සහ හැඩතල වෙනස් කිරීම.

12.1 CSS යනු කුමක්ද? (What is CSS?)

CSS යනු Cascading Style Sheets යන්නෙහි කෙටි නමයි. HTML මගින් වෙබ් අඩවියක සැකිල්ල (Structure) සාදන අතර, CSS මගින් එම සැකිල්ල අලංකාර කිරීම සිදු කරයි.

සරල පැහැදිලි කිරීම: අපි කලින් පරිච්ඡේදවලදී කී පරිදි HTML යනු නිවසක ගඩොල් සැකිල්ලයි. CSS යනු එම නිවසේ බිත්තිවල ගාන "සායම්", දොර පේන්තෙල් වල "හැඩය", සහ ගෘහ භාණ්ඩ තබන "ස්ථානය" තීරණය කරන සැලසුම් ශිල්පියා වැනිය.

- **Cascading:** මෙහි තේරුම "පියවරෙන් පියවර ගලා යන" යන්නයි. එනම්, එකම දේකට නීති දෙකක් තිබුණහොත්, වඩාත්ම නිවැරදි හෝ අවසානයට ඇති නීතිය ක්‍රියාත්මක වීමයි.

- **Style:** අකුරු වල පාට, ප්‍රමාණය, පින්තූර වල හැඩය වැනි දෘශ්‍යමාන දේ.

- **Sheets:** මෙම නීති රීති ලියා තබන කඩදාසි හෝ ගොනු.

12.2 CSS වල ඉතිහාසය (History of CSS)

වෙබ් අඩවි වල මුල් කාලයේදී CSS තිබුණේ නැත. සියලුම හැඩසවි ගැන්වීම් HTML තුළම කිරීමට සිදු විය. එය ඉතාමත් අපහසු ක්‍රියාවක් විය. 1996 දී

Håkon Wium Lie විසින් CSS ප්‍රථම වරට හඳුන්වා දෙන ලදී. අද වන විට අපි භාවිතා කරන්නේ ඉතාමත් බලවත් CSS3 සංස්කරණයයි.

12.3 CSS ඇයි අපට අවශ්‍ය වන්නේ? (Purpose)

1. සැලසුම සහ අන්තර්ගතය වෙන් කිරීම: HTML තුළ අකුරු පමණක්ද, CSS තුළ අලංකරණය පමණක්ද තබා ගැනීමෙන් වෙබ් අඩවිය කළමනාකරණය පහසු වේ.
2. කාලය ඉතිරි වීම: එකම CSS ගොනුවක් පිටු සිය ගණනකට භාවිතා කළ හැක.
3. විවිධ උපාංග වලට ගැලපීම: පරිගණකයකට පෙනෙන ආකාරය සහ දුරකථනයකට පෙනෙන ආකාරය වෙනස් කිරීමට CSS උදව් වේ.

12.4 CSS ලියන රටාව (Syntax)

CSS ලියන ආකාරය HTML වලට වඩා සම්පූර්ණයෙන්ම වෙනස්ය. එහි ප්‍රධාන කොටස් 3ක් ඇත.

රූපසටහන:

```
Selector {
  Property: Value;
}
```

1. **Selector** (තේරීම්කාරකය): අප හැඩ කිරීමට බලාපොරොත්තු වන HTML ටැගය. (උදා: **h1**, **p**)
2. **Property** (ගුණාංගය): අප වෙනස් කිරීමට යන දේ. (උදා: **color**, **font-size**)
3. **Value** (අගය): අප ලබා දෙන අගය. (උදා: **red**, **20px**)

උදාහරණය:

```
h1 {
  color: blue;
  font-size: 30px;
}
```

මෙම කේතයෙන් පවසන්නේ: "සියලුම **h1** ටැග් වල අකුරු නිල් පාට කරන්න, එහි ප්‍රමාණය පික්සල් 30ක් කරන්න."

12.5 CSS එකතු කරන ක්‍රම 3 (Three Ways to Add CSS)

අප ලියන CSS කේත HTML ගොනුවට සම්බන්ධ කිරීමට ක්‍රම 3ක් ඇත.

1. Inline CSS (ජේළිය තුළ CSS): HTML ටැගය ඇතුළතම style attribute එක භාවිතා කර ලියන ක්‍රමයයි.

```
<h1 style="color: red;">ආයුබෝවන්</h1>
```

මෙය ඉතා කුඩා වෙනස්කම් සඳහා පමණක් භාවිතා කරයි. විශාල වැඩ වලදී මෙය භාවිතා කිරීම වෘත්තීය මට්ටමේ ක්‍රමයක් නොවේ.

2. Internal CSS (අභ්‍යන්තර CSS): HTML ගොනුවේ <head> කොටස ඇතුළත <style> ටැගය භාවිතා කර ලිවීමයි.

```
<head>
  <style>
    p {
      color: green;
    }
  </style>
</head>
```

මෙය එක් පිටුවකට පමණක් බලපාන CSS සඳහා සුදුසුය.

3. External CSS (බාහිර CSS) - [වඩාත්ම නිර්දේශිත ක්‍රමය]: CSS වෙනමම ගොනුවක (උදා: style.css) ලියා, එය HTML පිටුවට සම්බන්ධ කිරීමයි.

පියවර:

1. style.css නමින් අලුත් ගොනුවක් සාදන්න.
2. HTML ගොනුවේ <head> කොටස තුළ පහත ජේළිය ලියන්න:

```
<link rel="stylesheet" href="style.css">
```

12.6 පළමු CSS අත්දැකීම (Practical Example)

HTML ගොනුව (index.html):

```
<!DOCTYPE html>
```

```
<html>
<head>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>මගේ අලංකාර වෙබ් අඩවිය</h1>
  <p>මම අද CSS ඉගෙන ගැනීමට පටන් ගත්තෙමි.</p>
</body>
</html>
```

CSS ගොනුව (style.css):

/* මෙයට Comment එකක් යැයි කියනු ලැබේ.
මෙය පරිගණකය කියවන්නේ නැත */

```
h1 {
  color: darkblue;    /* අකුරු තද නිල් පාට කරයි */
  text-align: center; /* මාතෘකාව මැදට ගෙන එයි */
  text-decoration: underline; /* ඉරක් අදියි */
}

p {
  color: dimgray;     /* අකුරු අළු පාට කරයි */
  font-family: Arial; /* අකුරු වල හැඩය (Font) වෙනස් කරයි */
  line-height: 1.6;   /* ජේළි අතර පරතරය වැඩි කරයි */
}
```

12.7 වෘත්තීය උපදෙස් (Professional Tips)

- External CSS භාවිතා කරන්න: සැමවිටම බාහිර CSS ගොනු භාවිතා කරන්න. එවිට ඔබේ කේතය පිරිසිදුව පවතින අතර එකම සැකසුම පිටු කිහිපයකට පහසුවෙන් ලබා දිය හැක.
- Comments භාවිතා කරන්න: CSS කේතය තුළ /* comment */ ලෙස සටහන් යොදන්න. එවිට වෙනත් කෙනෙකුට හෝ ඔබට පසුව කේතය තේරුම් ගැනීමට පහසු වේ.

12.8 පොදු වැරදි (Common Mistakes)

1. අවසාන සෙමිකෝලනය (;) අමතක වීම: සෑම CSS නීතියක් අවසානයේදීම ; ලකුණ තැබිය යුතුය. නැතිනම් ඊළඟ ජේළිය ක්‍රියා නොකරයි.

```
/* වැරදි */
h1 { color: red }
p { color: blue }
```

```
/* නිවැරදි */
h1 { color: red; }
p { color: blue; }
```

- වරහන් වැසීමට අමතක වීම: { වරහනක් ඇරියහොත් එය අනිවාර්යයෙන්ම } මගින් වැසිය යුතුය.
- වැරදි File Path: HTML එකට ලිස්සා ඇති CSS ගොනුවේ නම හෝ නිවැරදි ස්ථානය වැරදි නොවන අලංකාරය දී ඇත.

12.9 සාරාංශය (Key Points)

- CSS මගින් වෙබ් පිටුව වල පෙනුම පාලනය කරයි.
- CSS වල ප්‍රධාන කොටස් වන්නේ **Selector, Property සහ Value** ය.
- හොඳම ක්‍රමය වන්නේ **External CSS** (බාහිර ගොනුව) භාවිතා කිරීමයි.
- සෑම **Property** එකක් අවසානයේම සෙමිකෝලනයක් (;) තිබිය යුතුය.

12.10 අභ්‍යාස (Exercises)

- CSS යන්නෙහි සම්පූර්ණ නම කුමක්ද?
- External CSS භාවිතා කිරීමේ වාසි 2ක් ලියන්න.
- පහත CSS කේතයේ ඇති වැරදි සොයා නිවැරදි කරන්න:


```
h2 (
  color: red
  font-size 20px;
)
```
- සියලුම ඡේද (<p>) වල අකුරු මැදට (center) කිරීමට සහ අකුරු තද කිරීමට (bold) අවශ්‍ය CSS ලියන්න.

12.11 ප්‍රායෝගික ක්‍රියාකාරකම (Practical Task)

- පසුගිය පරිච්ඡේදයේ ඔබ සෑදූ "Tech Journal" ව්‍යාපෘතිය විවෘත කරන්න.
- එයට **style.css** නමින් ගොනුවක් එකතු කර එය HTML පිටුව වලට සම්බන්ධ කරන්න.
- වෙබ් අඩවියේ පසුබිම් වර්ණය (Background color) ලා නිල් පාටක් කර, මාතෘකා වල පාට තද රතු පාට කරන්න.

මූලික පරිච්ඡේදයෙන් අපි CSS වල ඉතාමත් වැදගත් කොටසක් වන "සෙලෙක්ටර්ස්" (Selectors) හෙවත් නිවැරදිව වැග් තෝරා ගන්නා ආකාරය ගැන ඉගෙන ගනිමු.

දහනුවන පරිච්ඡේදය: සී.එස්.එස්. තේරීම්කාරක - නිවැරදි ඉලක්කය තෝරා ගැනීම (CSS Selectors)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

- CSS Selector** (තේරීම්කාරකයක්) යනු කුමක්දැයි වෘත්තීය මට්ටමින් තේරුම් ගැනීම.
- මූලික තේරීම්කාරක වර්ග (Universal, Element, Class, ID) හඳුනා ගැනීම.
- Class** සහ **ID** අතර ඇති තාක්ෂණික සහ ප්‍රායෝගික වෙනස්කම් ඉතා පැහැදිලිව වටහා ගැනීම.
- සමූහ තේරීම්කාරක (Grouping Selectors) සහ පැවත එන්නන් තේරීම (Descendant Selectors).
- CSS ප්‍රමුඛතාවය (Specificity)** හෙවත් විවිධ නීති අතරින් බ්‍රවුසරය එකක් තෝරා ගන්නා ආකාරය.
- තේරීම්කාරක නිවැරදිව භාවිතා කිරීමේ හොඳම පුරුදු (Best Practices).

13.1 හැඳින්වීම (Introduction)

HTML මගින් වෙබ් අඩවියක සැකිල්ල සාදන අතර CSS මගින් එය හැඩගන්වයි. නමුත් මෙම දෙක එකිනෙක සම්බන්ධ කරන්නේ කෙසේද? HTML පිටුවක වැග් සිය ගණනක් තිබිය හැක. ජාවාස්ක්‍රිප්ට් වලට හෝ CSS වලට එම වැග් අතරින් අපට අවශ්‍ය නිශ්චිත කොටස පමණක් වෙන් කර හඳුනා ගැනීමට උපකාරී වන්නේ **Selector** (තේරීම්කාරකය) යි.

සරල පැහැදිලි කිරීම: තේරීම්කාරකයක් යනු බ්‍රවුසරයට ලබා දෙන "ලිපිනයක්" වැනිය. එම ලිපිනය දෙස බලා බ්‍රවුසරය තීරණය කරනවා අප ලියන වර්ණය හෝ හැඩය ලබා දිය යුත්තේ කුමන HTML ටැගයටද කියා.

13.2 මූලික තේරීම්කාරක වර්ග (Basic Selectors)

- Universal Selector** (විශ්ව තේරීම්කාරකය) - *

මෙය වෙබ් පිටුවේ ඇති සියලුම **HTML** ටැග් එකවර තෝරා ගැනීමට භාවිතා කරයි. තාරකාව (*) ලකුණ මෙයට යොදා ගනී.

භාවිතය: සාමාන්‍යයෙන් වෙබ් පිටුවක ඇති ස්වයංක්‍රීය ඉඩකඩ (**Default margins/padding**) ඉවත් කිරීමට මෙය භාවිතා කරයි.

```
* {
  margin: 0;
  padding: 0;
}
```

2. Element Selector (මූලිකාංග තේරීම්කාරකය)

HTML ටැගයේ නම කෙලින්ම භාවිතා කරයි. පිටුවේ ඇති එම වර්ගයේ සියලුම ටැග් වලට මෙය බලපායි.

```
h1 {
  color: darkred;
}
```

3. Class Selector (පන්ති තේරීම්කාරකය) - .

HTML ටැගයකට **class** යනුවෙන් නමක් ලබා දී, **CSS** වලදී එම නම ඉදිරියෙන් තිත්කේ (.) තබා භාවිතා කරයි.

ලක්ෂණය	Class (පන්තිය)	ID (අන්‍යන්‍යතාවය)
:-----	:-----	:-----
සංකේතය	තිත් (.)	හැෂ් (\#)
පිටුවක භාවිතය	එකම නම ඕනෑම වාර ගණනක් භාවිතා කළ හැක.	එක් නමක් පිටුවකට එක් වරක් පමණි.
ප්‍රමුඛතාවය	අඩුයි.	වැඩියි (බලවත් වැඩියි).
ටැගයකට ප්‍රමාණය	එක් ටැගයකට Class කිහිපයක් තිබිය හැක.	එක් ටැගයකට තිබිය හැක්කේ එක් ID එකක්

සැබෑ ලෝකයේ උපමාව (පාසලක උදාහරණය):

- **Class:** "11 ශ්‍රේණියේ සිසුන්" යනු පන්තියකි. එම පන්තියට සිසුන් 40ක් පමණ අයත් විය හැක. ඔවුන් සියල්ලන්ටම පොදු ලක්ෂණ ඇත. (**CSS** වලදී **grade-11**)

- **ID:** "ජාතික හැඳුනුම්පත් අංකය" (**NIC**) එක් අයෙකුට ඇත්තේ එකකි. එය තවත් අයෙකුට තිබිය නොහැක. (**CSS** වලදී **#nic-123456789v**)

වැදගත් තාක්ෂණික කරුණක්: ඔබ එකම ටැගයකට **Class** එකකින් "නිල්" පාටත්, **ID** එකකින් "රතු" පාටත් ලබා දුන්නොත්, බ්‍රවුසරය සැමවිටම තෝරා ගන්නේ **ID** එකේ ඇති "රතු" පාටයි. මන්ද **ID** එකක ඇති ප්‍රමුඛතාවය **Class** එකකට වඩා වැඩි බැවිනි.

```
<p class="highlight">මෙය විශේෂ  
ෂේදයකි.</p>
```

```
.highlight {
  background-color: yellow;
}
```

4. ID Selector (අන්‍යන්‍ය තේරීම්කාරකය) -

යම් ටැගයකට **id** නමක් ලබා දී, **CSS** වලදී එම නම ඉදිරියෙන් හැෂ් (#) ලකුණ තබා භාවිතා කරයි.

```
<nav id="main-menu">...</nav>
```

```
#main-menu {
  background-color: #333;
}
```

13.3 Class සහ ID අතර ඇති ප්‍රධාන වෙනස (Class vs ID)

මෙය වෙබ් සංවර්ධනයේදී ඔබ දැනගත යුතු වැදගත්ම වෙනසයි. බොහෝ ශිෂ්‍යයන් මේ දෙක පටලවා ගනී.

13.4 උසස් තේරීම්කාරක (Advanced Selectors)

1. Grouping Selector (සමූහ තේරීම්කාරකය)

ටැග් කිහිපයකට එකම **CSS** නීතිය ලබා දීමට අවශ්‍ය වූ විට, ඒවා කොමාවකින් (,) වෙන් කර ලිවිය හැක. මෙය කේතය කෙටි කිරීමට උදව් වේ.

```
h1, h2, p {
  text-align: center;
}
```

2. Descendant Selector (පැවත එන්නන් තේරීම)

යම් ටැගයක් ඇතුළත ඇති තවත් ටැගයක් තේරීමට හිස්තැනක් (**Space**) භාවිතා කරයි.

```
/* div එකක් ඇතුළත ඇති p ටැග් පමණක් තෝරයි
*/
div p {
  color: gray;
}
```

13.5 ASCII රූපසටහන: ප්‍රමුඛතාවය (Specificity)

CSS නීති අතර ගැටුමක් ඇති වූ විට (Conflict), බ්‍රවුසරය තීරණය ගන්නේ මෙම පිළිවෙළටය:

අඩු බලය (Low Power) -----> වැඩි බලය (High Power)

[*] < [Element (p)] < [Class (.name)] < [ID (#id)]

13.6 වෘත්තීය පැහැදිලි කිරීම (Professional Explanation)

වෘත්තීය Frontend Developer කෙනෙකු ලෙස ඔබ හැඩතල ගැන්වීම් (Styling) සඳහා 95% ක්ම භාවිතා කළ යුත්තේ Class ය.

- ඇයි? මන්ද Class භාවිතා කිරීමෙන් ඔබේ කේතය නැවත නැවත භාවිතා කළ හැකි (Reusable) සහ නම්‍යශීලී (Flexible) වන බැවිනි.
- ID භාවිතා කළ යුත්තේ සාමාන්‍යයෙන් JavaScript මගින් යම් කොටසක් තෝරා ගැනීමට හෝ වෙබ් පිටුවේ ඇතුළත ලිපි (Internal Anchors) සෑදීමට පමණි.

13.7 පොදු වැරදි (Common Mistakes)

1. තිත් (.) හෝ හැඟුණ (#) අමතක වීම: Class එකක් ලියන විට තිත් අමතක වීම නිසා CSS ක්‍රියා නොකරයි.
2. ID එකක් පිටුවකට දෙවරක් යෙදීම: මෙය HTML වැරද්දකි. බ්‍රවුසරය සමහර විට හැඩතල පෙන්වුවත්, JavaScript වලදී මෙය විශාල ගැටලු ඇති කරයි.
3. නම අංකයකින් ආරම්භ කිරීම: Class හෝ ID නමක් කිසිවිටකත් අංකයකින් ආරම්භ නොකළ යුතුය. (උදා: .1st-box වැරදියි, .box-1 නිවැරදියි).
4. Case Sensitivity: .MainBox සහ .mainbox යනු CSS වලදී දෙකකි. අකුරු වල හැඩය ගැන සැලකිලිමත් වන්න.

13.8 සාරාංශය (Key Points)

- * : සියලුම ටැග් තෝරා ගනී.
- .class : එකම ගතිලක්ෂණ ඇති ටැග් සමූහයක් හැඩගැන්වීමට (Reusable).
- #id : එක් විශේෂිත ටැගයක් පමණක් හැඩගැන්වීමට (Unique).
- ID සතු ප්‍රමුඛතාවය (Specificity) Class එකට වඩා වැඩිය.
- කොමාව (,) මගින් ටැග් කිහිපයක් එකවර තෝරාගත හැක.

13.9 අභ්‍යාස (Exercises)

1. class="main-text" ඇති සියලුම ඡේදවල අකුරු ප්‍රමාණය 18px කිරීමට අවශ්‍ය CSS ලියන්න.
2. id="footer-section" ඇති කොටසක පසුබිම් වර්ණය කළු කිරීමට අවශ්‍ය CSS ලියන්න.
3. පහත කේතයේ p ටැගයට ලැබෙන අවසාන වර්ණය කුමක්ද? ඇයි?

```
p { color: blue; }
.text-red { color: red; }
```

 (HTML: <p class="text-red">මොකක්ද පාට?</p>)

4. එකම HTML ටැගයකට Class දෙකක් ලබා දෙන ආකාරය උදාහරණයකින් පෙන්වන්න.

13.10 ප්‍රායෝගික ක්‍රියාකාරකම (Practical Task)

1. අලුත් HTML ගොනුවක් සාදා එහි ඡේද (Paragraphs) 3ක් ලියන්න.
2. පළමු ඡේදයට first නමින් ID එකක් ලබා දෙන්න.
3. දෙවන සහ තෙවන ඡේදවලට common නමින් Class එකක් ලබා දෙන්න.
4. CSS මගින්:
 - ID එක භාවිතා කර පළමු ඡේදය කොළ පාට කරන්න.
 - Class එක භාවිතා කර අනෙක් ඡේද දෙක තැඹිලි පාට කරන්න.
 - සියලුම ඡේද මැදට (center) පෙළගස්වන්න.

දහහතරවන පරිච්ඡේදය: වර්ණ සහ පසුබිම් - වෙබ් අඩවිය අලංකාර කිරීම (Colors and Backgrounds)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

1. CSS මගින් වර්ණ ලබා දීමේ විවිධ ක්‍රමවේද (Color Systems) හඳුනා ගැනීම.
2. අකුරු සඳහා වර්ණ (Foreground Color) සහ පසුබිම් වර්ණ (Background Color) සැකසීම.
3. වර්ණවල විනිවිදභාවය (Opacity / Transparency) පාලනය කරන ආකාරය.
4. පසුබිමට පින්තූරයක් එක් කිරීම සහ එහි හැසිරීම පාලනය කරන ගුණාංග (Properties) ඉගෙන ගැනීම.
5. වර්ණ අනුක්‍රමණ (Gradients) භාවිතා කර ආකර්ෂණීය පසුබිම් නිර්මාණය කිරීම.
6. වර්ණ තෝරා ගැනීමේදී සැලකිලිමත් විය යුතු ප්‍රවේශ්‍යතා නීති (Color Accessibility).

14.1 වර්ණ හැඳින්වීම (Introduction to Colors)

වෙබ් අඩවියක පළමු ආකර්ෂණය රඳා පවතින්නේ ඔබ භාවිතා කරන වර්ණ සංකලනය මතයි. CSS මගින් අපට මූලිකාංගයක (Element) ප්‍රධාන කොටස් දෙකකට වර්ණ ලබා දිය හැක:

1. **Foreground Color** (පෙරබිම් වර්ණය): මෙයින් අදහස් කරන්නේ අකුරු වල වර්ණයයි. මේ සඳහා **color** යන ගුණාංගය භාවිතා කරයි.
2. **Background Color** (පසුබිම් වර්ණය): මූලිකාංගය පිටුපස ඇති පසුබිමේ වර්ණයයි. මේ සඳහා **background-color** භාවිතා කරයි.

14.2 වර්ණ ලබා දිය හැකි ක්‍රම 4 (Ways to Define Colors)

පරිගණක තිරයක වර්ණ නිර්මාණය වන්නේ රතු (Red), කොළ (Green) සහ නිල් (Blue) යන මූලික වර්ණ තුනේ එකතුවෙනි. CSS හිදී මෙම වර්ණ හැඳින්වීමට ක්‍රම 4ක් ඇත.

1. Color Names (වර්ණවල නම්)

ඉතාමත් සරලම ක්‍රමයයි. අපට වර්ණයේ ඉංග්‍රීසි නම කෙලින්ම භාවිතා කළ හැක. CSS තුළ මෙවැනි නම් 140ක් පමණ හඳුනාගත හැක.

- උදාහරණ: **red, blue, green, orange, hotpink, skyblue.**

```
h1 {
  color: tomato;
}
```

2. HEX Codes (හෙක්සාඩෙසිමල් කේත)

වෘත්තීය මට්ටමේදී වැඩිපුරම භාවිතා වන ක්‍රමය මෙයයි. මෙය ආරම්භ වන්නේ හැෂ් (#) ලකුණකිනි. ඉන්පසුව අකුරු හෝ අංක 6ක් යොදනු ලැබේ.

- ව්‍යුහය: **#RRGGBB** (පළමු දෙක රතු, මැද දෙක කොළ, අවසන් දෙක නිල්).

- උදාහරණ:

- **#FF0000** - රතු (උපරිම රතු)
- **#00FF00** - කොළ
- **#0000FF** - නිල්
- **#FFFFFF** - සුදු
- **#000000** - කළු

```
p {
  color: #3498db; /* නිල් පැහැති විශේෂිත වර්ණයක් */
}
```

3. RGB System (ආර්.ජී.බී. පද්ධතිය)

මෙහිදී රතු, කොළ සහ නිල් වර්ණ මිශ්‍ර කරන ප්‍රමාණය 0 සිට 255 දක්වා වූ අංකයකින් ලබා දෙයි.

- ව්‍යුහය: **rgb(red, green, blue)**
- උදාහරණ: **rgb(255, 0, 0)** යනු රතු වර්ණයයි.

```
div {
  background-color: rgb(255, 165, 0); /* තැඹිලි පාට */
}
```

4. RGBA System (විනිවිදභාවය සහිත ක්‍රමය)

මෙය **RGB** වලට සමාන වුවත්, අවසානයට '**A**' (**Alpha**) අගයක් එක් කරයි. **Alpha** අගය මගින් වර්ණයේ විනිවිදභාවය පාලනය කරයි.

- අගය: 0.0 (සම්පූර්ණයෙන්ම නොපෙනෙන) සිට 1.0 (සම්පූර්ණයෙන්ම පැහැදිලි) දක්වා.


```
.overlay {
  background-color: rgba(0, 0, 0, 0.5); /*
  විනිවිද පෙනෙන කළු පැහැති පසුබිමක් */
}
```

14.3 පසුබිම් පින්තූර පාලනය (Background Images)

වෙබ් අඩවියක පසුබිමට පින්තූරයක් එක් කිරීමේදී ටැග් එකක් නොව CSS ගුණාංග භාවිතා කිරීම වඩාත් වෘත්තීය ක්‍රමයයි.

1. background-image

පසුබිමට පින්තූරය ඇතුළත් කිරීමට භාවිතා කරයි.

```
body {
  background-image:
  url('images/background.jpg');
}
```

2. background-repeat

පින්තූරය කුඩා නම් එය මුළු තිරය පුරාම ටයිල් (Tiles) මෙන් නැවත නැවත දිස්වේ. එය පාලනය කිරීමට මෙය භාවිතා කරයි.

- repeat (සම්මත අගය)
- no-repeat (එක් වරක් පමණක් පෙන්වයි)
- repeat-x (හරහට පමණක්)
- repeat-y (පහළට පමණක්)

3. background-size

පින්තූරයේ විශාලත්වය තීරණය කරයි.

- cover: පින්තූරය කැපී ගියත් මුළු පසුබිමම වැසෙන සේ විශාල කරයි. (වඩාත්ම ජනප්‍රියයි)
- contain: පින්තූරය සම්පූර්ණයෙන්ම පෙනෙන සේ සකස් කරයි.

4. background-position

පින්තූරය තිබිය යුතු ස්ථානය.

- center, top, bottom, left, right.

5. background-attachment

තිරය පහළට ස්ක්‍රෝල් (Scroll) කරන විට පින්තූරයත් සමග ගමන් කළ යුතුද නැද්ද යන්න.

- scroll: අකුරු සමග පින්තූරයත් ඉහළට යයි.
- fixed: පින්තූරය එක තැනක තිබියදී අකුරු පමණක් උඩින් ගමන් කරයි (Parallax Effect).

14.4 වර්ණ අනුක්‍රමණ (Gradients)

Gradients (ග්‍රේඩියන්ට්ස්) යනු වර්ණ එකකට වඩා වැඩි ගණනක් මෘදු ලෙස එකිනෙක මුසු වෙමින් පවතින පසුබිම් වේ. මෙය පින්තූරයක් ලෙස නොව CSS කේතයක් ලෙස භාවිතා කරයි.

1. Linear Gradient (සරල රේඛීය)

වර්ණ එක් දිශාවකට ගලා යයි.

```
.box {
  background-image: linear-gradient(to
  right, red, yellow);
}
```

14.5 ASCII රූපසටහන: RGB වර්ණ මිශ්‍රණය

```

      (රතු) 255,0,0
        \  /
         \ /
          \ /
(කහ)  \ \ (දම්)
      255,255,0 X 255,0,255
         / \
        /  \
       /    \
      (කොළ)  (නිල්)
      0,255,0  0,0,255
  
```

14.6 වෘත්තීය පැහැදිලි කිරීම (Professional Explanation)

වෘත්තීය Frontend Developer කෙනෙකු පසුබිම් පින්තූර භාවිතා කිරීමේදී "Fallback Color" එකක් භාවිතා කිරීමට අමතක නොකරයි. කිසියම් හේතුවක් නිසා පින්තූරය ලෝඩ් (Load) නොවුවහොත්, ඒ වෙනුවට පෙන්වීමට වර්ණයක් ලබා දීම මින් අදහස් වේ.

```
/* පින්තූරය එනතෙක් හෝ පින්තූරය වැරදුණහොත්
නිල් පාට පෙන්වයි */
```

```
.hero {
  background-color: blue;
  background-image: url('hero.jpg');
}
```

14.7 ප්‍රවේශ්‍යතාව සහ වර්ණ (Color Accessibility)

වෙබ් අඩවියක් සාදන විට වර්ණ අන්ධතාව (Color Blindness) ඇති පුද්ගලයන් ගැන සිතීම ඉතා වැදගත් වේ.

- **Contrast** (පරස්පරතාව): පසුබිම් වර්ණය සහ අකුරු වර්ණය අතර පැහැදිලි වෙනසක් තිබිය යුතුය. උදාහරණයක් ලෙස, තද අළු පසුබිමක ලා අළු අකුරු ලිවීම වැරදිය. (සුදු පසුබිමේ කළු අකුරු ලිවීම ඉතා හොඳ පරස්පරතාවයකි).

14.8 පොදු වැරදි (Common Mistakes)

1. **HEX Code වැරදීම:** # ලකුණ අමතක වීම හෝ අකුරු 6ක් වෙනුවට 5ක් ලිවීම.
2. පින්තූරයේ **Path** වැරදීම: **url()** තුළ පින්තූරය ඇති නිවැරදි ෆෝල්ඩරය සඳහන් නොකිරීම.
3. **Color vs Background-color:** අකුරු වල පාට වෙනස් කිරීමට **background-color** භාවිතා කිරීමට උත්සාහ කිරීම. (අකුරු සඳහා නිතරම **color** පමණක් භාවිතා කරන්න).

14.9 සාරාංශය (Key Points)

- **color:** අකුරු වල වර්ණය.
- **background-color:** පසුබිමේ වර්ණය.
- **HEX, RGB, RGBA** මගින් නිශ්චිත වර්ණ ලබා ගත හැක.
- **background-size: cover:** පසුබිම් පින්තූරයක් පෙන්වීමට ඇති හොඳම ක්‍රමයයි.
- **Contrast** එක ගැන සැලකිලිමත් වීම වෘත්තීය ගුණාංගයකි.

14.10 අභ්‍යාස (Exercises)

1. පහත සඳහන් වර්ණවලට අදාළ **HEX Codes** සොයා ලියන්න: සුදු, කළු, රතු, නිල්.
2. පිටුවේ පසුබිමට පින්තූරයක් එක් කර, එය ස්කෝල් නොවන ලෙස (**fixed**) තබා ගැනීමට අවශ්‍ය **CSS** ලියන්න.

3. **rgba** භාවිතා කර 70% ක් පෙනෙන (0.7 **opacity**) කොළ පැහැති පසුබිමක් සාදන්න.

4. **Linear Gradient** එකක් භාවිතා කර නිල් සිට කොළ දක්වා වර්ණ මාරු වන පසුබිමක් සහිත **div** එකක් සාදන්න.

14.11 ප්‍රායෝගික ව්‍යාපෘතිය (Mini Project)

ව්‍යාපෘතිය: "**Hero Section**" එකක් නිර්මාණය කිරීම

1. **HTML** හි **section** ටැගයක් සාදා එයට "**hero**" නමින් **Class** එකක් දෙන්න.
2. එහි ඇතුළත **h1** එකක් සහ **p** එකක් ලියන්න.
3. **CSS** මගින්:
 - **Hero section** එකට ලස්සන පසුබිම් පින්තූරයක් එක් කරන්න.
 - එම පින්තූරය **cover** සහ **center** කරන්න.
 - පින්තූරය මත ඇති අකුරු කියවීමට අපහසු නම්, **rgba** භාවිතා කර පින්තූරය උඩින් අඳුරු ස්ථරයක් (**Overlay**) දමන්න.
 - අකුරු සියල්ල සුදු පැහැයට හරවන්න.

පහළොස්වන පරිච්ඡේදය: අකුරු හැඩගැන්වීම - කියවීමේ පහසුව සහ කලාව (Typography - Readability and Art)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

1. ටයිපොග්‍රැෆි (**Typography**) යනු කුමක්ද සහ වෙබ් අඩවියක එහි ඇති වැදගත්කම අවබෝධ කර ගැනීම.
2. අකුරු පවුල් (**Font Families**) සහ **Serif**, **Sans-serif**, **Monospace** අතර වෙනස හඳුනා ගැනීම.
3. අකුරු වල ප්‍රමාණය (**Font Size**) සඳහා භාවිතා කරන විවිධ ඒකක (**px**, **rem**, **em**) තේරුම් ගැනීම.
4. අකුරු වල බර (**Weight**), රටාව (**Style**) සහ සැරසිලි (**Decoration**) පාලනය කිරීම.
5. ජේලි අතර පරතරය (**Line Height**) සහ අකුරු අතර පරතරය (**Letter Spacing**) හැසිරවීම.
6. අකුරු පෙළගැස්වීම (**Text Alignment**) සහ පරිවර්තනය (**Text Transform**).
7. **Google Fonts** වැනි බාහිර අකුරු වර්ග වෙබ් අඩවියට සම්බන්ධ කරන ආකාරය.

8. ප්‍රවේශ්‍යතාව (**Accessibility**) සඳහා අකුරු භාවිතයේදී සැලකිලිමත් විය යුතු කරුණු.

15.1 ටයිපොග්‍රැෆි (Typography) යනු කුමක්ද?

ටයිපොග්‍රැෆි යනු අකුරු පෙළක් කියවීමට පහසු වන ලෙස සහ දෘශ්‍යමය වශයෙන් ආකර්ෂණීය වන ලෙස සකස් කිරීමේ කලාවයි. වෙබ් අඩවියක ඇති තොරතුරු වලින් 90% කට වඩා අඩංගු වන්නේ අකුරු (**Text**) ලෙසය. එම නිසා, ඔබ තෝරා ගන්නා අකුරු වර්ගය සහ ඒවා පෙන්වන ආකාරය වෙබ් අඩවියේ සාර්ථකත්වයට සෘජුවම බලපායි.

සැබෑ ලෝකයේ උපමාව: සිතන්න ඔබ ඉතා වැදගත් ලිපියක් කියවනවා කියා.

- එහි අකුරු ඉතා කුඩා නම් හෝ අකුරු එකිනෙක ඇලී තිබේ නම් ඔබට එය කියවීමට අපහසු වේ.
- නමුත් අකුරු පැහැදිලිව, පේළි අතර නිසි පරතරයක් සහිතව තිබේ නම් ඔබට එය වෙහෙසකින් තොරව කියවිය හැක. වෙබ් අඩවියකදී ද අප උත්සාහ කරන්නේ පරිශීලකයාගේ ඇස් වෙහෙසට පත් නොකර තොරතුරු ලබා දීමටයි.

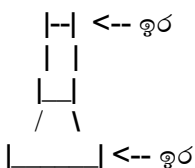
15.2 අකුරු පවුල් (Font Families)

CSS වලදී අකුරු ප්‍රධාන කාණ්ඩ (**Generic Families**) කිහිපයකට බෙදා ඇත.

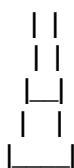
1. **Serif** (සෙරිෆ්): අකුරුවල කෙළවර කුඩා ඉරි සහිත සම්ප්‍රදායික අකුරු වර්ගයයි. (උදා: **Times New Roman**). මෙය මුද්‍රිත පොත් සඳහා වැඩිපුර භාවිතා වේ.
2. **Sans-serif** (සැන්-සෙරිෆ්): අකුරුවල කෙළවර අමතර ඉරි නැති ඉතා පිරිසිදු නූතන අකුරු වර්ගයයි. (උදා: **Arial, Roboto**). වෙබ් අඩවි වල කියවීමේ පහසුව සඳහා වැඩිපුරම භාවිතා කරන්නේ මේවාය.
3. **Monospace** (මොනොස්පේස්): සෑම අකුරකටම සමාන පළලක් ඇති අකුරු වර්ගයයි. (උදා: **Courier**). මෙය කේත (**Code**) පෙන්වීමට භාවිතා කරයි.

ASCII රූපසටහන: **Serif** එදිරිව **Sans-serif**

Serif (ඉරි සහිත)



Sans-serif (ඉරි රහිත)



භාවිතා කරන ආකාරය (**font-family**): සැමවිටම අකුරු වර්ග කිහිපයක් ලැයිස්තුවක් ලෙස ලබා දීම වෘත්තීය ක්‍රමයයි. පරිගණකයේ පළමු අකුරු වර්ගය නැතිනම් දෙවන වර්ගය භාවිතා කිරීමට බ්‍රවුසරයට ඉඩ ලබා දෙයි.

```
body {
  font-family: "Roboto", Arial, sans-serif;
}
```

15.3 අකුරු වල ප්‍රමාණය (Font Size)

අකුරු වල විශාලත්වය පාලනය කිරීමට **font-size** ගුණාංගය භාවිතා කරයි. මෙහිදී ඒකක (**Units**) වර්ග කිහිපයක් ඇත.

1. **px (Pixels)**: ස්ථාවර ඒකකයකි. (උදා: 16px). මෙය වෙනස් නොවන ප්‍රමාණයකි.
2. **rem (Root em)**: වඩාත්ම නිර්දේශිත ඒකකයයි. මෙය වෙබ් අඩවියේ මූලික අකුරු ප්‍රමාණයට (සාමාන්‍යයෙන් 16px) සාපේක්ෂව ගණනය වේ.
 - 1rem = 16px (සාමාන්‍යයෙන්)
 - 2rem = 32px
 - වාසිය: පරිශීලකයා බ්‍රවුසරයේ **Settings** වලින් අකුරු විශාල කළහොත්, **rem** භාවිතා කර ඇති වෙබ් අඩවි වල අකුරු ද ඒ අනුව ලස්සනට විශාල වේ.

15.4 අකුරු හැඩගැන්වීමේ ගුණාංග (Font Styling)

1. **font-weight** (අකුරේ බර): අකුරු තද කිරීමට (**Bold**) භාවිතා කරයි.
 - අගයන්: **normal, bold** හෝ 100 සිට 900 දක්වා අංක.
2. **font-style** (රටාව): අකුරු ඇල කිරීමට (**Italic**) භාවිතා කරයි.
 - අගය: **italic**.
3. **text-decoration** (සැරසිලි): අකුරු යටින් ඉරක් ඇඳීමට හෝ ඉරි ඉවත් කිරීමට.
 - අගයන්: **underline, none** (ලිත්ක් වල ඇති ඉර ඉවත් කිරීමට මෙය නිතර භාවිතාවේ).
4. **text-transform** (පරිවර්තනය): අකුරු කැපිටල් හෝ සිම්පල් කිරීමට.
 - **uppercase** (සියල්ල කැපිටල්), **lowercase**, **capitalize** (මුල් අකුර පමණක් කැපිටල්).

15.5 පරතරය සහ පෙළගැස්ම (Spacing and Alignment)

1. **line-height** (පේළි අතර උස): මෙය ඉතා වැදගත්ය. පේළි දෙකක් අතර පරතරය මෙයින් පාලනය වේ. කියවීමේ පහසුව සඳහා සාමාන්‍යයෙන් 1.5 හෝ 1.6 අගයක් භාවිතා කරයි.
2. **letter-spacing**: අකුරු දෙකක් අතර පරතරය.
3. **text-align**: අකුරු පෙළගස්වන දිශාව.
 - left, center, right, justify (පේළියේ දෙපැත්තම සමාන කිරීමට).

15.6 Google Fonts භාවිතා කිරීම (Using External Fonts)

ඔබට ඔබේ වෙබ් අඩවියට ඉතා ලස්සන අකුරු වර්ග අවශ්‍ය නම් fonts.google.com වෙත ගොස් ඒවා ලබා ගත හැක.

පියවර:

1. **Google Fonts** වෙබ් අඩවියට ගොස් අකුරු වර්ගයක් තෝරන්න (උදා: **Poppins**).
2. ඔබට අවශ්‍ය **Styles (Regular 400, Bold 700)** තෝරා **"Get embed code"** ක්ලික් කරන්න.
3. එහි ලැබෙන **<link>** එක ඔබේ **HTML** හි **<head>** තුළට ඇතුළත් කරන්න.
4. **CSS** වලදී එම අකුරු පවුලේ නම භාවිතා කරන්න.

```
body {
  font-family: 'Poppins', sans-serif;
}
```

15.7 සම්පූර්ණ කේත උදාහරණය

HTML:

```
<div class="article">
  <h1>වෙබ් නිර්මාණකරණයේ රහස්</h1>
  <p class="intro">වයිපොග්‍රැෆි කියන්නේ නිකමම අකුරු වර්ගයක් තෝරන එක නෙවෙයි.</p>
  <p>හොඳ වෙබ් අඩවියක් කියවන්න පහසු වෙන්න ඕනේ. ඒකට පේළි අතර පරතරය සහ නිවැරදි අකුරු ප්‍රමාණය වැදගත්.</p>
</div>
```

CSS:

```
.article {
  max-width: 600px;
  margin: auto;
}

h1 {
  font-family: 'Arial', sans-serif;
  font-size: 2.5rem; /* 40px පමණ */
  font-weight: 700;
  text-transform: capitalize;
  color: #2c3e50;
  line-height: 1.2;
}

p {
  font-size: 1.1rem; /* 17.6px පමණ */
  line-height: 1.6; /* කියවීමේ පහසුව සඳහා හොඳ පරතරයක් */
  color: #34495e;
  text-align: justify;
}

.intro {
  font-style: italic;
  font-weight: bold;
  color: #e67e22;
}
```

15.8 වෘත්තීය උපදෙස් (Professional Tips)

- අකුරු වර්ග සීමා කරන්න: එක් වෙබ් අඩවියකට අකුරු වර්ග (Fonts) 2කට හෝ 3කට වඩා භාවිතා නොකරන්න. ඕනෑවට වඩා අකුරු වර්ග තිබීම වෙබ් අඩවියේ වේගය අඩු කරයි.
- පේළියක දිග (Line Length): පේළියක ඇති අකුරු ගණන 45-75 අතර තිබීම කියවීමට වඩාත් සුදුසු බව පර්යේෂණ වලින් හෙළි වී ඇත. පේළිය ඉතා දිගු වූ විට ඇසට ඊළඟ පේළිය සොයා ගැනීමට අපහසු වේ.
- **Contrast**: කළු පසුබිමක තද අළු පාටින් අකුරු ලිවීමෙන් වළකින්න.

15.9 පොදු වැරදි (Common Mistakes)

1. **Line-height** කුඩා වීම: පේළි එකිනෙක ඇති තිබීම නිසා කියවීමට අපහසු වීම.
2. **Px** ඒකකය පමණක් භාවිතා කිරීම: ජංගම දුරකථන සහ ප්‍රවේශ්‍යතාව සඳහා **rem** භාවිතා නොකිරීම.

3. **Justify** වැරදි ලෙස භාවිතය: වෙබ් අඩවි වල **text-align: justify** භාවිතා කරන විට අකුරු අතර අමුතු හිස්තැන් ඇති විය හැක. මෙය පරීක්ෂා කර බැලිය යුතුය.

15.10 සාරාංශය (Key Points)

- **Typography** යනු වෙබ් අඩවියක තොරතුරු කියවීමේ පහසු වයි.
- **Sans-serif** අකුරු වෙබ් අඩවි සඳහා වඩාත් සිදුසුය.
- **rem** ඒකකය භාවිතා කිරීම ප්‍රවේශ්‍යතාව (Accessibility) සඳහා වැදගත් වේ.
- **line-height** අගය 1.5 ක් පමණ තැබීම කියවීමේ පහසුව වැඩි කරයි.
- **Google Fonts** හරහා විවිධ අකුරු වර්ග ලබා ගත හැක.

15.11 අභ්‍යාස (Exercises)

1. **Serif** සහ **Sans-serif** අකුරු අතර ඇති වෙනස පින්තූරයකින් හෝ රූප සටහනකින් පැහැදිලි කරන්න.
2. ලින්ක් එකක () යටින් ඇති ඉර ඉවත් කිරීමට සහ මවුසය උඩට ගෙන ගිය විට පමණක් ඉරක් ඒමට අවශ්‍ය **CSS** ලියන්න.
3. **Google Fonts** වෙත ගොස් "Montserrat" නමැති අකුරු වර්ගය ඔබේ වෙබ් අඩවියට සම්බන්ධ කර පෙන්වන්න.
4. පහත දත්ත ඇතුළත් කර **CSS** නීතියක් ලියන්න:
 - අකුරු ප්‍රමාණය: 20px (rem වලින්).
 - පේළි උස: 1.8.
 - අකුරු සියල්ල **Uppercase** කරන්න.

15.12 ප්‍රායෝගික ව්‍යාපෘතිය (Mini Project)

ව්‍යාපෘතිය: බ්ලොග් ලිපියක පිටුවක් හැඩගැන්වීම (Blog Post Styling)

1. **HTML** හි බ්ලොග් ලිපියක ව්‍යුහයක් සාදන්න (මාතෘකාව, උප මාතෘකාව, දිගු ඡේද 2ක්).
2. **CSS** භාවිතා කර:
 - මාතෘකාව සඳහා ඉතා තද (**Bold**) සහ විශාල **Sans-serif** අකුරු ලබා දෙන්න.
 - ඡේද සඳහා කියවීමට පහසු **line-height** එකක් සහ **rem** ඒකකයෙන් අකුරු ප්‍රමාණය ලබා දෙන්න.
 - ඡේදයේ පළමු අකුර පමණක් විශාල කර පෙන්වීමට උත්සාහ කරන්න (ඉභිය: **::first-letter selector** එක භාවිතා කර බලන්න).

- පසුබිම් වර්ණය සහ අකුරු වර්ණය අතර හොඳ **Contrast** එකක් පවත්වා ගන්න.

දහසයවන පරිච්ඡේදය: සී.එස්.එස්. පෙට්ටි ආකෘතිය - වෙබ් අඩවියක පදනම (CSS Box Model)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

1. "සෑම **HTML** මූලිකාංගයක්ම පෙට්ටියකි" යන සංකල්පය වෘත්තීය මට්ටමින් තේරුම් ගැනීම.
2. පෙට්ටි ආකෘතියේ (**Box Model**) ප්‍රධාන කොටස් හතර (**Content, Padding, Border, Margin**) හඳුනා ගැනීම.
3. මූලිකාංගයක සැබෑ පළල සහ උස (**Total Width/Height**) ගණනය කරන ආකාරය ඉගෙන ගැනීම.
4. **box-sizing: border-box** ගුණාංගය මගින් පිරිසැලසුම් (**Layouts**) වල ඇතිවන ගැටලු විසඳන ආකාරය.
5. **margin: auto** භාවිතා කර මූලිකාංග මැදට (**Center**) කරන ආකාරය.
6. කෙටි ක්‍රම (**Shorthand properties**) භාවිතා කර කේතය කෙටි කරන ආකාරය.

16.1 හැඳින්වීම: සෑම දෙයක්ම පෙට්ටියකි (Everything is a Box)

CSS ඉගෙනීමේදී ඔබට හමුවන වැදගත්ම සහ මූලිකම සංකල්පය මෙයයි. බ්‍රවුසරයක් ඕනෑම **HTML** ටැගයක් (එය **<h1>** වේවා, **<p>** වේවා, එසේත් නැතිනම් **** වේවා) දකින්නේ සෘජුකෝණාස්‍රාකාර පෙට්ටියක් ලෙසය.

වෙබ් අඩවියක පිරිසැලසුමක් (**Layout**) සෑදීම යනු මෙම විවිධ ප්‍රමාණයේ පෙට්ටි එක මත එක හෝ එක ළඟ පිළිවෙළකට තැබීමයි. මෙම පෙට්ටි එකිනෙක ගැටෙන්නේ කෙසේද, ඒවා අතර පරතරය කොපමණද යන්න තීරණය කරන්නේ **Box Model** (පෙට්ටි ආකෘතිය) මගිනි.

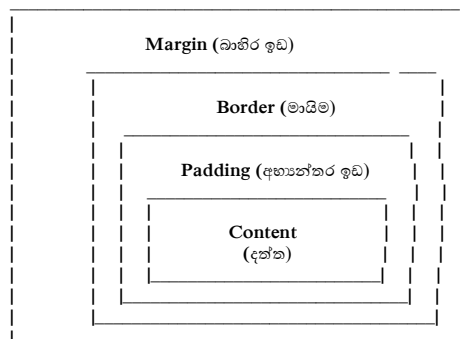
16.2 පෙට්ටි ආකෘතියේ කොටස් 4 (The 4 Layers)

සෑම CSS පෙට්ටියක්ම ස්ථර හතරකින් සමන්විත වේ. අපි මෙය සැබෑ ලෝකයේ උදාහරණයකින් තේරුම් ගනිමු.

සැබෑ ලෝකයේ උපමාව (වටිනා බඳුනක් පාර්සල් කිරීම):

- 1. Content** (අන්තර්ගතය): ඔබ විදේශගත වන මිතුරෙකුට යවන වටිනා "බඳුන" වැනිය. (අතුරු හෝ පින්තූර).
- 2. Padding** (අභ්‍යන්තර පරතරය): බඳුන ආරක්ෂා කිරීමට පෙට්ටිය ඇතුළත දමන "බුබුළු දවටන" (Bubble wrap) වැනිය. මෙය බඳුන සහ පෙට්ටියේ බිත්තිය අතර ඉඩයි.
- 3. Border** (මායිම): බඳුන අඩංගු "කාඩ්බෝඩ් පෙට්ටිය" වැනිය.
- 4. Margin** (බාහිර පරතරය): මෙම පෙට්ටිය තවත් පෙට්ටියක් සමඟ ගැටීම වැළැක්වීමට පෙට්ටියෙන් පිටත තබන නිස් ඉඩයි.

ASCII රූපසටහන: CSS Box Model



16.3 එක් එක් ගුණාංගය සවිස්තරාත්මකව (Detailed Properties)

- ## 1. Content (අන්තර්ගතය)

මෙහි ප්‍රමාණය පාලනය කරන්නේ **width** සහ **height** මගිනි.

- width: 300px;
- height: 200px;

- ## 2. Padding (අභ්‍යන්තර පරතරය)

අන්තර්ගතය සහ මායිම (Border) අතර ඉඩයි. මෙය පෙට්ටිය ඇතුළත ඇති ඉඩක් බැවින්, **Padding** වැඩි කළ විට පසුබිම් වර්ණය (Background color) එම ඉඩෙහි දී පෙනේ.

- padding-top, padding-right, padding-bottom, padding-left.**

3. Border (මායිම)

පැඩිං එක වටා ඇති ඉරයි. මෙයට ප්‍රධාන කොටස් 3ක් ඇත:

- border-width: 5px; (ඝනකම)
- border-style: solid; (රටාව - solid, dashed, dotted)
- border-color: black; (වර්ණය)
- කෙටි කමය: border: 5px solid black;

4. Margin (බාහිර පරතරය)

පෙට්ටියෙන් පිටත ඇති ඉඩයි. මෙය වෙනත් මූලිකාංග තල්ලු කිරීමට භාවිතා කරයි. **Margin** වලදී පසුබිම් වර්ණය පෙනෙන්නේ නැත (එය විනිවිද පෙනෙන ඉඩකඩ).

- margin-top, margin-right, margin-bottom, margin-left.**

16.4 සම්පූර්ණ ප්‍රමාණය ගණනය කිරීම (Calculating Total Size)

ආරම්භකයින් වැඩිපුරම වැරදි සිදු කරන්නේ මෙතැනදීය. ඔබ මූලික-ගයක **width** එක 300px ලෙස ලබා දුන් පමණින් එහි සම්පූර්ණ පළල 300px නොවේ.

උදාහරණයක්:

```
.box {
  width: 300px;
  padding: 20px;
  border: 10px solid black;
  margin: 30px;
}
```

පළල ගණනය කිරීම:

- Content Width: 300px
- Padding Left + Right: 20px + 20px = 40px
- Border Left + Right: 10px + 10px = 20px

- **Total Width** (නිරයේ ගන්නා ඉඩ): $300 + 40 + 20 = 360\text{px}$ (**Margin** යනු අසල ඇති පෙට්ටි තල්ලු කිරීමට ගන්නා ඉඩකඩ).

16.5 වෘත්තීය විසඳුම: **box-sizing: border-box**

ඉහත පරිදි ගණනය කිරීම ඉතා කරදරකාරී වැඩකි. අපට අවශ්‍ය වන්නේ පළල **300px** කියා ලබා දුන් විට, පැඩිං සහ බෝඩර් කොපමණ දැමීමත් මුළු පළල **300px** ලෙසම පවත්වා ගැනීමටයි. සඳහා මෙම ගුණාංගය භාවිතා කරයි.

```
* {
  box-sizing: border-box;
}
```

මෙය සිදු වූ විට: ඔබ ලබා දෙන **width (300px)** ඇතුළතටම පැඩිං සහ බෝඩර් ද එකතු වේ. අන්තර්ගතය (**Content**) ඒ අනුව කුඩා වේ. නවීන වෙබ් සංවර්ධනයේදී සියලුම සංවර්ධකයන් මෙම ක්‍රමය භාවිතා කරයි.

16.6 කෙටි ක්‍රම (Shorthand Properties)

පේළි හතරක් ලියනවා වෙනුවට එක පේළියකින් **Padding** සහ **Margin** ලිවිය හැක. මෙහිදී අගයන් ලියන පිළිවෙළ ඔරලෝසුවක කටු කැරකෙන දිශාවට සමානය (ඉහළ -> දකුණ -> පහළ -> වම).

1. **padding: 10px 20px 30px 40px;** (Top 10, Right 20, Bottom 30, Left 40)
2. **padding: 10px 20px;** (Top/Bottom 10, Right/Left 20)
3. **padding: 10px;** (හතර පැත්තටම 10)

16.7 ප්‍රායෝගික කේත උදාහරණය

HTML:

```
<div class="card">
  <h2>Box Model ඉගෙන ගනිමු</h2>
  <p>සෑම දෙයක්ම පෙට්ටියක් ලෙස සිතන්න.</p>
</div>
```

CSS:

```
* {
  box-sizing: border-box; /* වෘත්තීය ක්‍රමය */
  margin: 0;
  padding: 0;
}

.card {
  width: 300px;
  background-color: #f1f1f1;

  /* පෙට්ටිය ඇතුළත ඉඩ */
  padding: 20px;

  /* මායිම */
  border: 5px solid #3498db;

  /* පෙට්ටිය පිටත ඉඩ - ඉහළින් සහ පහළින් 50px,
  දෙපැත්තෙන් auto */
  margin: 50px auto;

  /* auto මගින් පෙට්ටිය තිරයේ මැදට (Center) පැමිණේ */

  border-radius: 10px; /* කොන් වටකුරු කිරීමට */
}
```

16.8 වෘත්තීය උපදෙස් (Professional Tips)

- **Margin Collapse:** පේළි දෙකක ඇති පෙට්ටි දෙකක **margin-bottom** සහ **margin-top** එකිනෙක ගැටුණු විට, ඒවා එකතු වන්නේ නැත. ඒ වෙනුවට විශාලතම මාර්ජින් එක පමණක් ක්‍රියාත්මක වේ. මෙය "**Margin Collapse**" ලෙස හැඳින්වේ.
- **Centering:** මූලිකාංගයක් තිරයේ තිරස් අතට මැදට ගැනීමට නම්, අනිවාර්යයෙන්ම එයට **width** එකක් ලබා දී **margin: auto** භාවිතා කරන්න.
- **Debug Tool:** බ්‍රවුසරයේ **Inspect Element (F12)** විවෘත කළ විට, පහළ කෙළවරේ **Box Model** එක වර්ණ ගැන්වූ රූපසටහනක් ලෙස දැකගත හැක. ඔබේ පරතරයන් පරීක්ෂා කිරීමට එය භාවිතා කරන්න.

16.9 පොදු වැරදි (Common Mistakes)

1. **Padding** සහ **Margin** පටලවා ගැනීම: බොහෝවිට ඇතුළත අකුරු වලට ඉඩ දීමට **margin** භාවිතා කිරීමට උත්සාහ කිරීම (එහිදී **padding** භාවිතා කළ යුතුය).
2. **Width 100%** සහ **Padding:** පෙට්ටියකට **width: 100%** ලබා දී පැඩිං එකතු කළහොත්, එය තිරයෙන් පිටතට ගොස් පෙනෙනු ඇත (**Horizontal scroll**). මෙය වැළැක්වීමට **box-sizing: border-box** අනිවාර්යයෙන්ම යොදා ගන්න.

16.10 සාරාංශය (Key Points)

- සෑම HTML ටැගයක්ම පෙට්ටියකි.
- Content -> Padding -> Border -> Margin යනු පිළිවෙළයි.
- **Padding** පෙට්ටිය ඇතුළත; **Margin** පෙට්ටිය පිටතය.
- **box-sizing: border-box** මගින් ගණනය කිරීම් පහසු කරයි.
- **margin: auto** මගින් මැදට (Center) පෙළගස්වයි.

16.11 අභ්‍යාස (Exercises)

1. පහත කේතයේ මූලිකාංගය තිරයේ ගන්නා සම්පූර්ණ පළල (Total Width) කොපමණද? (**Box-sizing** භාවිතා කර නැත). **width: 250px; padding: 15px; border: 2px solid black; margin: 10px;**
2. පැඩ් (Padding) සහ මාර්ජින් (Margin) අතර ඇති ප්‍රධාන තාක්ෂණික වෙනස කුමක්ද?
3. පැඩ් සඳහා භාවිතා කළ හැකි කෙටි ක්‍රමය (Shorthand) උදාහරණයකින් පෙන්වන්න.
4. සතර පැත්තටම 20px පරතරයක් සහ රතු පැහැති තිත් සහිත (dotted) මායිමක් ඇති div එකක් සඳහා CSS ලියන්න.

16.12 ප්‍රායෝගික ව්‍යාපෘතිය (Mini Project)

ව්‍යාපෘතිය: ආකර්ෂණීය "තොරතුරු කොටුවක්" නිර්මාණය කිරීම (Info Box)

1. **HTML** වල **div** එකක් සාදා එයට "info-box" නමින් **Class** එකක් ලබා දෙන්න.
2. **CSS** භාවිතා කර:
 - එයට 400px පළලක් ලබා දෙන්න.
 - සෑම පැත්තකටම 25px ක **Padding** එකක් ලබා දෙන්න.
 - වම් පැත්තේ පමණක් තද නිල් පැහැති 10px ඝනකම බෝධරයක් ලබා දෙන්න (**border-left**).
 - ලා නිල් පැහැති පසුබිම් වර්ණයක් ලබා දෙන්න.
 - එය තිරයේ මැදට ගෙන එන්න (**margin: auto**).
 - **box-sizing: border-box** භාවිතා කර පිරිසිදුව කේතය ලියන්න.

දහනත්වන පරිච්ඡේදය: පිරිසැලසුම් මූලධර්ම - ඩිස්ප්ලේ ගුණාංගය (Layout Fundamentals - Display Property)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

1. **display** (ඩිස්ප්ලේ) ගුණාංගය යනු කුමක්ද සහ එය පිරිසැලසුමකට (Layout) බලපාන ආකාරය තේරුම් ගැනීම.
2. බ්ලොක් මට්ටමේ මූලිකාංග (Block-level Elements) සහ ඒවායේ හැසිරීම් හඳුනා ගැනීම.
3. ඉන්ලයින් මූලිකාංග (Inline Elements) සහ ඒවායේ සීමාවන් වටහා ගැනීම.
4. ඉන්ලයින්-බ්ලොක් (Inline-block) මූලිකාංග භාවිතා කරමින් දියුණු පිරිසැලසුම් සකස් කරන ආකාරය.
5. **display: none** භාවිතා කර මූලිකාංගයක් තාවකාලිකව වෙබ් පිටුවෙන් ඉවත් කරන ආකාරය.
6. මූලිකාංග එක ළඟින් තැබීමේ මූලික නීති අවබෝධ කර ගැනීම.

17.1 හැඳින්වීම: මූලිකාංගවල හැසිරීම (Element Behavior)

පෙර පරිච්ඡේදයේදී අප ඉගෙන ගත්තේ සෑම HTML ටැගයක්ම පෙට්ටියක් (Box) ලෙස හැසිරෙන බවයි. නමුත් මෙම පෙට්ටි සියල්ලම එක සමාන ලෙස වෙබ් පිටුවක හැසිරෙන්නේ නැත. සමහර පෙට්ටි මුළු පේළියම තනිවම ලබා ගන්නා අතර, සමහර පෙට්ටි තවත් පෙට්ටියක් අසලින් සිටීමට ඉඩ ලබා දෙයි.

මෙම හැසිරීම පාලනය කරන ප්‍රධානතම CSS ගුණාංගය වන්නේ **display** ගුණාංගයයි. වෙබ් අඩවියක පිරිසැලසුම (Layout) හෙවත් කොටස් පෙළගස්වන ආකාරය තීරණය කරන මූලික අඩිතාලම මෙයයි.

සැබෑ ලෝකයේ උපමාව (සිතමා ශාලාවක අසුන් පැනවීම):

- **Block:** VIP පේළියක් වැනිය. ඔබ එම පේළියේ එක් අසුනක් වෙන් කර ගත්තද, එම මුළු පේළියේම වෙනත් කිසිවෙකුට වාඩි වීමට ඉඩ නොදී තනිවම ලබා ගැනීමයි.
- **Inline:** සාමාන්‍ය අසුන් පෙළක් වැනිය. ඔබ වාඩි වන්නේ ඔබට අවශ්‍ය ඉඩ ප්‍රමාණයේ පමණි. තවත් අයෙකුට ඔබ අසලින්ම වාඩි විය හැක.

17.2 බ්ලොක් මට්ටමේ මූලිකාංග (Block-level Elements)

මෙම වර්ගයේ මූලිකාංග සැමවිටම අලුත් ජේළියකින් ආරම්භ වේ. එමෙන්ම ඒවාට පවතින සම්පූර්ණ පළල (Full width) වමේ සිට දකුණට ලබා ගනී.

ප්‍රධාන ලක්ෂණ:

1. සැමවිටම අලුත් ජේළියකින් පටන් ගනී.
2. ඒවාට ලබා දිය හැකි උපරිම පළල (100% width) ස්වයංක්‍රීයව ලබා ගනී.
3. මේවාට width (පළල) සහ height (උස) ලබා දිය හැක.
4. මේවායේ හතර පැත්තටම Padding සහ Margin නිවැරදිව ලබා දිය හැක.

පොදු බ්ලොක් මූලිකාංග: <div>, <h1> සිට <h6>, <p>, , , <header>, <footer>, <section>.

ASCII රූපසටහන: Block-level Behavior

[h1 වැගය - මුළු ජේළියම ලබා ගනී]
[p වැගය - අලුත් ජේළියකින් ආරම්භ වේ]

17.3 ඉන්ලයින් මූලිකාංග (Inline Elements)

මෙම වර්ගයේ මූලිකාංග අලුත් ජේළියකින් ආරම්භ වන්නේ නැත. ඒවා ලබා ගන්නේ ඒවායේ අන්තර්ගතයට (අකුරු වලට) අවශ්‍ය අවම ඉඩ ප්‍රමාණය පමණි. තවත් ඉන්ලයින් මූලිකාංගයක් තිබුණහොත් එයට ඒ අසලින්ම සිටීමට ඉඩ ලබා දෙයි.

ප්‍රධාන ලක්ෂණ:

1. අලුත් ජේළියකින් පටන් ගන්නේ නැත.
2. ලබා ගන්නේ අන්තර්ගතයට අවශ්‍ය ඉඩ පමණි.
3. වැදගත්: ඉන්ලයින් මූලිකාංග වලට width හෝ height ලබා දිය නොහැක (බ්‍රවුසරය ඒවා නොසලකා හරියි).
4. මේවාට වම් සහ දකුණු පසට (Left/Right) පමණක් Margin ලබා දිය හැක. ඉහළ සහ පහළ Margin නිවැරදිව ක්‍රියා නොකරයි.

පොදු ඉන්ලයින් මූලිකාංග: , <a>, , , .

ASCII රූපසටහන: Inline Behavior

[span 1] [span 2] [a link] (ඉතිරි ඉඩ)

17.4 ඉන්ලයින්-බ්ලොක් (Inline-block) මූලිකාංග

මෙය බ්ලොක් සහ ඉන්ලයින් යන දෙකේම හොඳම ලක්ෂණ එකතු කර සාදන ලද "මිශ්‍ර" (Hybrid) අවස්ථාවකි.

ඇයි අපට මෙය අවශ්‍ය වන්නේ? සමහර විට අපට අවශ්‍ය වෙනවා මූලිකාංග කිහිපයක් එක ජේළියේ තබන්න (Inline මෙන්), නමුත් ඒ සෑම එකකටම නිශ්චිත පළලක් සහ උසක් ලබා දෙන්න (Block මෙන්).

ප්‍රධාන ලක්ෂණ:

1. එක ජේළියේ සිටීමට ඉඩ ලබා දෙයි.
2. width සහ height ලබා දීමට හැකියාව ඇත.
3. හතර පැත්තටම Padding සහ Margin ලබා දිය හැක.

ප්‍රායෝගික භාවිතය: වෙබ් අඩවි වල Navbar එකේ ලිත්කස් හෝ ජේළියට පෙන්වන බොත්තම් (Buttons) සෑදීමට මෙය බහුලව භාවිතා කරයි.

17.5 සංසන්දනාත්මක වගුව (Comparison Table)

ලක්ෂණය	Block	Inline	Inline-block
අලුත් ජේළියකින් ඇරඹේද?	ඔව්	නැත	නැත
පළල (Width) දිය හැකිද?	ඔව්	නැත	ඔව්
උස (Height) දිය හැකිද?	ඔව්	නැත	ඔව්
ජේළියක කිහිපයක් සිටිය හැකිද?	නැත	ඔව්	ඔව්

17.6 මූලිකාංග සභවා තැබීම (display: none)

සමහර අවස්ථාවලදී අපට වෙබ් පිටුවේ ඇති යම් කොටසක් තාවකාලිකව නොපෙනී යාමට සැලැස්විය යුතුය.

- **display: none;** මූලිකාංගය සම්පූර්ණයෙන්ම වෙබ් පිටුවෙන් ඉවත් කරයි. එයට වෙන් වී තිබූ ඉඩ ප්‍රමාණය ද නැති වී යන අතර අසල ඇති අනෙක් පෙට්ටි එම ඉඩ ලබා ගනී. (එය හරියට වෙබ් පිටුවේ එම වැගය නැත්තාක් මෙනි).

සටහන: **visibility: hidden;** යනුවෙන් තවත් ක්‍රමයක් ඇත. එයින් මූලිකාංගය නොපෙනී ගියද, එයට වෙන්වූ ඉඩ ප්‍රමාණය තවමත් හිස්ව පවතී (හොල්මනක් මෙන්).

17.7 ප්‍රායෝගික කේත උදාහරණය

HTML:

```
<div class="box block-box">මම බ්ලොක් එකක්
(Full Width)</div>
<div class="box block-box">මම තවත් බ්ලොක්
එකක්</div>
<span class="inline-box">මම ඉන්ලයින්
1</span>
<span class="inline-box">මම ඉන්ලයින්
2</span>
<div class="nav-container">
  <div class="nav-item">මෙනුව 1</div>
  <div class="nav-item">මෙනුව 2</div>
  <div class="nav-item">මෙනුව 3</div>
</div>
```

CSS:

```
/* පොදු හැඩතල */
.box {
  background-color: lightblue;
  margin: 10px 0;
  border: 1px solid blue;
}

/* 1. Block: තනි පේළියක් ගනී */
.block-box {
  display: block;
  padding: 10px;
}

/* 2. Inline: අකුරු වල ඉඩ පමණක් ගනී */
.inline-box {
  display: inline;
  background-color: lightgreen;
  padding: 10px; /* උස වෙනස් නොවේ */
}

/* 3. Inline-block: පේළියේ සිටින අතර පළල/උස ලබා
දිය හැක */
.nav-item {
  display: inline-block;
  width: 100px;
  height: 50px;
  background-color: tomato;
  color: white;
  text-align: center;
  line-height: 50px; /* අකුරු සිරස් අතට මැදට
ගැනීමට */
  margin-right: 10px;
}
```

17.8 වෘත්තීය උපදෙස් (Professional Tips)

- ඉන්ලයින් වැග් ඇතුළත බ්ලොක් වැග් ලියන්න එපා: උදාහරණයක් ලෙස **** වැගයක් ඇතුළත **<div>** වැගයක් ලිවීම වැරදිය. සෑම විටම විශාල පෙට්ටිය (Block) ඇතුළත කුඩා පෙට්ටි (Inline) තිබිය යුතුය.

- **Vertical Alignment:** ඉන්ලයින්-බ්ලොක් මූලිකාංග භාවිතා කරන විට ඒවා උස අනුව නිවැරදිව පෙළගැසී නැතිනම්, **vertical-align: top;** වැනි ගුණාංග භාවිතා කරන්න.

- **Whitespace Bug:** ඉන්ලයින්-බ්ලොක් මූලිකාංග දෙකක් අතර HTML කේතයේ හිස්තැනක් (Space/Enter) තිබුණහොත්, තිරය මතද කුඩා හිස්තැනක් දිස්වේ. මෙය වැළැක්වීමට වැග් දෙක අතර හිස්තැන් නැතිව ලිවීමට හෝ මව් වැගයේ (Parent) අකුරු ප්‍රමාණය **0** කිරීමට වෘත්තීය සංවර්ධකයන් පුරුදුව සිටිති.

17.9 පොදු වැරදි (Common Mistakes)

1. **Span එකකට Width ලබා දීමට උත්සාහ කිරීම:** බොහෝ සිසුන් **** එකකට **width: 200px** ලබා දී එය වැඩ නොකරන බව පවසයි. එයට හේතුව එය ඉන්ලයින් මූලිකාංගයක් වීමයි. එයට පළල ලබා දීමට නම් මූලිකවම **display: inline-block** හෝ **display: block** කළ යුතුය.

2. **Margin Top සහ Inline:** ඉන්ලයින් මූලිකාංග වලට **margin-top** ලබා දීමෙන් ඒවා පහළට තල්ලු නොවේ.

17.10 සාරාංශය (Key Points)

- **display** මගින් මූලිකාංගයක දෘශ්‍ය හැසිරීම පාලනය කරයි.
- **Block** මූලිකාංග මුළු පේළියම ලබා ගනී.
- **Inline** මූලිකාංග අවශ්‍ය ඉඩ පමණක් ලබා ගෙන එක පේළියේ සිටී.
- **Inline-block** මගින් එක පේළියේ සිටින අතරම පළල සහ උස ලබා දිය හැක.
- **display: none** මගින් මූලිකාංගය සම්පූර්ණයෙන්ම සඟවයි.

17.11 අභ්‍යාස (Exercises)

1. **Block** සහ **Inline** මූලිකාංග අතර ඇති ප්‍රධාන වෙනස්කම් 3ක් ලියන්න.
2. පහත දැක්වෙන HTML වැග් බ්ලොක් (Block) ද ඉන්ලයින් (Inline) ද කියා වෙන් කරන්න: **<h1>**, **<a>**, **<div>**, ****, **<p>**, ****.

3. ලැයිස්තුවක ඇති කරුණු () හරහට (Horizontal) පෙන්වීමට සුදුසුම Display අගය කුමක්ද?

4. පින්තූරයකට () width සහ height ලබා දීමට හැකි වන්නේ ඇයි? (ඉභිය: පින්තූර යනු ඉන්ලයින් චුවත් ස්වභාවයෙන්ම වෙනස් ලෙස හැසිරෙන මූලිකාංගයකි).

17.12 ප්‍රායෝගික ව්‍යාපෘතිය (Mini Project)

ව්‍යාපෘතිය: තිරස් Navigation Bar එකක් නිර්මාණය කිරීම

1. HTML වල nav ටැගයක් ඇතුළත a ටැග් 4ක් සාදන්න (Home, About, Services, Contact).

2. CSS භාවිතා කර:

- එම ලින්ක්ස් වලට display: inline-block ලබා දෙන්න.

- සෑම ලින්ක් එකකටම 120px පළලක් සහ 40px උසක් ලබා දෙන්න.

- පසුබිම් වර්ණයක් සහ අකුරු මැදට (Center) පෙළගැස්වීම් සිදු කරන්න.

- මවුසය උඩට ගෙන ගිය විට (:hover) පසුබිම් වර්ණය වෙනස් කරන්න.

- ඒවා එක පේළියක ලස්සනට පෙනේදැයි පරීක්ෂා කරන්න.

දහඅටවන පරිච්ඡේදය: සී.එස්.එස්. පිහිටීම පාලනය කිරීම - මූලිකාංග කැමති තැනක තබමු (CSS Position Property)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

1. සාමාන්‍ය ලේඛන ප්‍රවාහය (Normal Document Flow) යනු කුමක්දැයි අවබෝධ කර ගැනීම.

2. position (පිහිටීම) ගුණාංගය භාවිතා කර මූලිකාංග තිබෙන ස්ථානය වෙනස් කරන ආකාරය.

3. ප්‍රධාන පිහිටීම් වර්ග 5 (Static, Relative, Absolute, Fixed, Sticky) ඉතා පැහැදිලිව වටහා ගැනීම.

4. දිශා පාලක (Top, Right, Bottom, Left) භාවිතා කරන ආකාරය.

5. මූලිකාංග එකක් මත එකක් තැබීමේදී z-index (ස්ථර පාලනය) භාවිතා කරන ආකාරය.

6. සැබෑ ලෝකයේ උදාහරණ ලෙස "තිරයේ ඇලි පවතින මෙනු" සහ "පාවෙන බොත්තම්" නිර්මාණය කිරීම.

18.1 හැදින්වීම: සාමාන්‍ය ප්‍රවාහයෙන් ඔබ්බට (Beyond Normal Flow)

පෙර පරිච්ඡේදවලදී අප ඉගෙන ගත්තේ HTML ටැග් එකකට පසුව තවත් ටැග් එකක් ලියූ විට, ඒවා ඔවුසරයේ එකකට යටින් එකක් හෝ එකක් අසල එකක් ලෙස පෙළගැසෙන බවයි. මෙයට Normal Document Flow (සාමාන්‍ය ලේඛන ප්‍රවාහය) යැයි කියනු ලැබේ.

නමුත් සමහර අවස්ථාවලදී අපට අවශ්‍ය වෙනවා:

- පින්තූරයක් මත අකුරු ටිකක් ලියන්න (එකක් මත එකක් තබන්න).

- වෙබ් පිටුව පහළට ස්ක්‍රෝල් (Scroll) කරන විටත්, මෙනුව තිරයේ ඉහළම ඇලි තිබෙන්නට සලස්වන්න.

- තිරයේ කෙළවරක "Help" බොත්තමක් ස්ථාවරව තබන්න.

සාමාන්‍ය ප්‍රවාහයට අනුව මේවා කළ නොහැක. ඒ සඳහා අප භාවිතා කරන්නේ CSS Position ගුණාංගයයි.

18.2 පිහිටීම පාලනය කරන දිශා (Top, Right, Bottom, Left)

යම් මූලිකාංගයක පිහිටීම වෙනස් කිරීමට අපට ප්‍රධාන දිශා 4ක් භාවිතා කළ හැක. මේවා භාවිතා කළ හැකි වන්නේ position: static නොවන අනෙක් සෑම අවස්ථාවකදීම පමණි.

- top: ඉහළ සිට ඇති පරතරය.

- right: දකුණු පස සිට ඇති පරතරය.

- bottom: පතුලේ සිට ඇති පරතරය.

- left: වම් පස සිට ඇති පරතරය.

18.3 Position අගයන් පහ (The Five Position Values)

CSS වල පිහිටීම් වර්ග 5ක් ඇත. අපි ඒවා එකින් එක සවිස්තරාත්මකව බලමු.

1. Static (සම්මත පිහිටීම)

මෙය සෑම HTML ටැග් එකකටම ස්වයංක්‍රීයව ලැබෙන අගයයි.

- හැසිරීම: ටැග් එක ලියා ඇති පිළිවෙලට තිරයේ දිස්වේ.
- විශේෂත්වය: **top, left** වැනි අගයන් මෙයට ලබා දුන්නද කිසිදු වෙනසක් සිදු නොවේ.

2. Relative (සාපේක්ෂ පිහිටීම)

ටැගය සාමාන්‍යයෙන් තිබිය යුතු ස්ථානයේ සිට සුළු වශයෙන් වෙනස් කිරීමට මෙය භාවිතා කරයි.

- හැසිරීම: ඔබ **top: 20px** ලෙස ලබා දුන්නොත්, එය සාමාන්‍යයෙන් තිබිය යුතු තැන සිට **20px** ක් පහළට ගමන් කරයි.
- වැදගත්: එය ඕට් පෙර සිටි ස්ථානය බ්‍රවුසරය විසින් තවමත් මතක තබා ගනී (එහි හිස් ඉඩක් පවතී). අනෙකුත් ටැග් එම හිස් ඉඩට පැමිණෙන්නේ නැත. එය "හොල්මනක්" වැනිය; ශරීරය වෙන තැනක තිබුණත්, එහි මුල් ස්ථානය තවමත් වෙන් වී පවතී.

3. Absolute (නිරපේක්ෂ පිහිටීම)

මෙය ඉතාමත් බලවත් අගයකි. මෙය භාවිතා කළ විට එම ටැගය සාමාන්‍ය ලේඛන ප්‍රවාහයෙන් සම්පූර්ණයෙන්ම ඉවත් වේ.

- හැසිරීම: මෙය තමන්ගේ "දෙමාපිය ටැගයට" (Parent) සාපේක්ෂව පිහිටීම තීරණය කරයි.
- නීතිය: මෙය නිවැරදිව වැඩ කිරීමට නම්, එහි දෙමාපිය ටැගයට **position: relative** ලබා දී තිබිය යුතුය.
- උපමාවක්: මෙය පියාසර කරන "ඩ්‍රෝන්" (Drone) එකක් වැනිය. එය පොළොවේ ඉඩක් ගන්නේ නැත, එයට ඕනෑම තැනක පාවෙමින් සිටිය හැක.

4. Fixed (ස්ථාවර පිහිටීම)

වෙබ් පිටුව කොපමණ පහළට ස්ක්‍රෝල් (Scroll) කළත්, මෙය තිරයේ එකම තැනක සිටි පවතී.

- හැසිරීම: මෙය බ්‍රවුසරයේ අපට පෙනෙන තිරයට (Viewport) සාපේක්ෂව ක්‍රියා කරයි.
- භාවිතය: "Back to Top" බොත්තම් හෝ මැසෙන්ජර් වැට් බොත්තම් සාදන්නේ මෙලෙසයි.

5. Sticky (ඇලෙන සුළු පිහිටීම)

මෙය **Relative** සහ **Fixed** යන දෙකේම මිශ්‍රණයකි.

- හැසිරීම: යම් සීමාවකට එන තෙක් එය සාමාන්‍ය පරිදි ස්ක්‍රෝල් වේ. නමුත් එම සීමාවට (උදා: **top: 0**) පැමිණි පසු එය එතැන ඇලී පවතී.
- භාවිතය: වගුවක මාතෘකා පේළිය (Table Header) ස්ක්‍රෝල් කරන විට පෙනෙන සේ තබා

ගැනීමට මෙය ඉතා සුදුසුයි.

18.4 ASCII රූපසටහන: Relative එදිරිව Absolute

Normal Flow:	Relative:	Absolute:
[Box 1]	[Box 1]	[Box 1]
[Box 2]	[Gap] <-- ඉඩ ඇත	[Box 3] <-- උඩට එයි
[Box 3]	[Box 2] <-- පහළට ගියා	[Box 2] <-- පාවෙයි

18.5 Z-Index: ස්ථර පාලනය (Layering)

අපි ටැග් දෙකක් එකක් මත එකක් තැබූ විට (**Position** භාවිතා කර), උඩින් පෙනිය යුත්තේ කුමන එකද යන්න තීරණය කරන්නේ **z-index** මගිනි.

- පරිගණක තිරය **X** (නිරස්) සහ **Y** (සිරස්) අක්ෂ වලින් සමන්විතයි. **Z** යනු ඔබ දෙසට දිවෙන අක්ෂයයි.
- **z-index: 1** ඇති ටැගයට වඩා **z-index: 10** ඇති ටැගය ඉදිරියෙන් (උඩින්) පෙනේ.
- වැදගත්: **z-index** ක්‍රියා කරන්නේ **position** අගයක් ලබා දී ඇති ටැග් සඳහා පමණි.

18.6 ප්‍රායෝගික කේත උදාහරණය: පින්තූරයක් මත "New" ලේබලය තැබීම

HTML:

```
<div class="product-card">
  <span class="badge">NEW</span>
  
  <h3>Professional Camera</h3>
</div>
```

```
<button class="chat-btn">Chat with
Us</button>
```


CSS:

/* 1. දෙමාපිය පෙට්ටිය සාපේක්ෂ (Relative) විය යුතුය */

```
.product-card {
  position: relative;
  width: 300px;
  border: 1px solid #ddd;
}
```

```
.product-card img {
  width: 100%;
}
```

/* 2. ලේඛලය පාවෙමින් (Absolute) තිබිය යුතුය */

```
.badge {
  position: absolute;
  top: 10px; /* ඉහළ සිට 10px */
  right: 10px; /* දකුණේ සිට 10px */
  background-color: red;
  color: white;
  padding: 5px 10px;
  font-weight: bold;
  z-index: 5; /* පින්තූරයට උඩින් පෙනීමට */
}
```

/* 3. වැට් බොත්තම තිරයේ ඇලි (Fixed) තිබිය යුතුය */

```
.chat-btn {
  position: fixed;
  bottom: 20px; /* පතුලේ සිට 20px */
  right: 20px; /* දකුණේ සිට 20px */
  padding: 15px;
  background-color: #0084ff;
  color: white;
  border-radius: 50px;
}
```

18.7 වෘත්තීය උපදෙස් (Professional Tips)

- The "Relative Parent" Rule: ඔබ මූලිකාංගයක් **position: absolute** කරන්නේ නම්, 99% කදී එහි දෙමාපිය වැගය **position: relative** කළ යුතුය. නැතිනම් එම මූලිකාංගය මුළු වෙබ් පිටුවටම (body) සාපේක්ෂව පාවී ගොස් ඔබේ Layout එක කැඩී යනු ඇත.
- Sticky Header: ඔබේ මෙනු බාර් එක (Navigation) තිරයේ මුදුනේ ඇලවීමට **position: sticky; top: 0;** භාවිතා කරන්න.

- **Fixed and Mobile:** ජංගම දුරකථන වලදී **fixed** මූලිකාංග ඕනෑවට වඩා භාවිතා කිරීමෙන් වළකින්න. මන්ද ඒවා කුඩා තිරයේ වැඩි ඉඩක් ලබා ගන්නා බැවිනි.

18.8 පොදු වැරදි (Common Mistakes)

1. **Z-index සහ Static position** එකක් ලබා නොදී **z-index** ලබා දීම. (එය කිසිවිටකත් ක්‍රියා නොකරයි).
2. දත්ත වැසී යාම: **Navigation Bar** එකක් **fixed** කළ විට, එය සාමාන්‍ය ප්‍රවාහයෙන් ඉවත් වන නිසා පිටුවේ මුල ඇති අකුරු ඒ යටට ගොස් වැසී යයි. මෙය වැළැක්වීමට **body** එකට පැඩිං (**padding-top**) එකක් ලබා දිය යුතුය.
3. අනවශ්‍ය ලෙස **Position** භාවිතා කිරීම: සෑම දෙයකටම **Position** භාවිතා නොකරන්න. මුලින්ම **Flexbox** හෝ **Grid** (ඉදිරි පාඩම්) මගින් පිරිසැලසුම සෑදීමට උත්සාහ කරන්න.

18.9 සාරාංශය (Key Points)

- **relative:** තමා සිටි තැනට සාපේක්ෂව වලනය වේ, නමුත් මුල් ඉඩ රඳවා ගනී.
- **absolute:** සාපේක්ෂ දෙමාපියයකට සාපේක්ෂව පාවෙයි, මුල් ඉඩ අත්හරියි.
- **fixed:** තිරයට (**Viewport**) සාපේක්ෂව ඇලී පවතී.
- **sticky:** ස්ක්‍රෝල් කිරීමේදී යම් සීමාවකදී ඇලී පවතී.
- **z-index:** ස්ථර වල පිළිවෙළ (ඉදිරියට/පසුපසට) තීරණය කරයි.

18.10 අභ්‍යාස (Exercises)

1. **Relative සහ Absolute** අතර ඇති ප්‍රධාන වෙනස ඔබේ වචනයෙන් පැහැදිලි කරන්න.
2. පිටුව ස්ක්‍රෝල් කරන විටත් පෙනෙන සේ "Contact Support" බොත්තමක් තිරයේ පහළ දකුණු කෙළවරේ සෑදීමට අවශ්‍ය **CSS** ලියන්න.
3. පහත කේතයේ වැරද්ද කුමක්ද?


```
.box {
        z-index: 10;
        top: 20px;
      }
```
4. පින්තූර දෙකක් එකක් මත එකක් අර්ධ වශයෙන් වැටී ඇති සේ (**Overlapping**) සැකසීමට අවශ්‍ය **CSS** ලියන්න.

18.11 ප්‍රායෝගික ව්‍යාපෘතිය (Mini Project)

ව්‍යාපෘතිය: "පාවෙන විස්තර කාඩ්පතක්" (Floating Info Card)

1. **HTML** වල **div** එකක් සාදා ඒ තුළ පින්තූරයක් සහ තවත් කුඩා **div** එකක් (**Caption**) සාදන්න.
2. **CSS** භාවිතා කර:
 - විශාල **div** එක 400px පළලට සකසා එය **relative** කරන්න.
 - කුඩා **Caption div** එක **absolute** කර එය පින්තූරයේ පතුලටම වන්නට සකසන්න.
 - **Caption** එකට විනිවිද පෙනෙන අඳුරු පසුබිමක් (**rgba**) සහ සුදු පැහැති අකුරු ලබා දෙන්න.
 - මුළු කාඩ්පතම තිරයේ මැදට ගෙන එන්න.

දහනවවන පරිච්ඡේදය:
 ෆ්ලෙක්ස්බොක්ස් - නම්‍යශීලී
 පිරිසැලසුම් (CSS Flexbox -
 Flexible Box Layout)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

1. **Flexbox** යනු කුමක්ද සහ එය නිර්මාණය කිරීමට හේතුව.
2. **Flex Container** සහ **Flex Items** අතර වෙනස.
3. ප්‍රධාන අක්ෂය (**Main Axis**) සහ හරස් අක්ෂය (**Cross Axis**) හඳුනා ගැනීම.
4. **Flexbox** හි දෙමාපිය වැගයට (**Parent**) අදාළ සියලුම ගුණාංග ඉගෙන ගැනීම.
5. මූලිකාංග හරියටම මැදට (**Center**) ගෙන එන ආකාරය.
6. සැබෑ ලෝකයේ **Navigation Bar** එකක් **Flexbox** මගින් සාදන ආකාරය.

19.1 Flexbox යනු කුමක්ද? (What is Flexbox?)

Flexbox යනු වෙබ් පිටුවක ඇති මූලිකාංග (**Elements**) එක දිශාවකට (පේළියක් ලෙස හෝ තිරුවක් ලෙස) ඉතාමත් පහසුවෙන් සහ නම්‍යශීලීව පෙළගැස්වීමට භාවිතා කරන **CSS** ක්‍රමයකි.

ඇයි අපිට **Flexbox** අවශ්‍ය වන්නේ? **Flexbox** හඳුන්වා දීමට පෙර, වෙබ් අඩවියක කොටස් දෙකක් එක ළඟින් තැබීමට (**Side by side**) **float** හෝ **display: inline-block** වැනි ක්‍රම භාවිතා කිරීමට සිදු විය. ඒවා ඉතාමත් අපහසු සහ දෝෂ (**Bugs**) සහිත ක්‍රම විය. උදාහරණයක් ලෙස, යමක් හරියටම මැදට (**Center**) ගැනීමට එකල ඉතා සංකීර්ණ කේත ලිවීමට සිදු විය. **Flexbox** මෙම සියලු ගැටලු විසඳා දැමීය.

19.2 මූලික සංකල්ප: Container සහ Items

Flexbox ක්‍රියාත්මක වීමට නම් මට්ටම් දෙකක සම්බන්ධතාවයක් තිබිය යුතුය.

1. **Flex Container** (දෙමාපිය වැගය): අපට පෙළගැස්වීමට අවශ්‍ය මූලිකාංග සියල්ල අඩංගු වන ප්‍රධාන පෙට්ටියයි. මෙයට **display: flex;** යන ගුණාංගය ලබා දිය යුතුය.
2. **Flex Items** (දරුවෝ): **Container** එක ඇතුළත සිටින සියලුම සෘජු මූලිකාංග (**Direct children**) මෙලෙස හැඳින්වේ.

උපමාවක්: කේක් අඩංගු පෙට්ටියක් ගැන සිතන්න.

- ලොකු පෙට්ටිය = **Flex Container**
- පෙට්ටිය ඇතුළේ ඇති කේක් කැලි = **Flex Items**

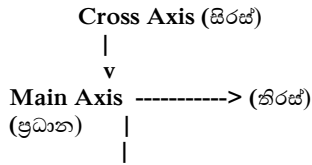
ඔබ ලොකු පෙට්ටියට "**Flex**" නියෝගය දුන් සැණින්, ඇතුළේ ඇති කේක් කැලි සියල්ල ඔබ පවසන නීති රීති වලට අනුව පෙළගැසීමට පටන් ගනී.

19.3 අක්ෂ දෙක (The Two Axes)

Flexbox තේරුම් ගැනීමට ඇති වැදගත්ම රහස මෙයයි. **Flexbox** සැමවිටම අක්ෂ දෙකක් ඔස්සේ වැඩ කරයි.

1. **Main Axis** (ප්‍රධාන අක්ෂය): මෙය **Flex Items** පෙළගැසී ඇති ප්‍රධාන දිශාවයි. සාමාන්‍යයෙන් මෙය වම් සිට දකුණට (**Row**) දිවෙයි.
2. **Cross Axis** (හරස් අක්ෂය): මෙය ප්‍රධාන අක්ෂයට ලම්බකව (සිරස් අතට) දිවෙන අක්ෂයයි.

ASCII සටහන:



19.4 Flex Container සඳහා වන ගුණාංග (Parent Properties)

අපි දැන් බලමු දෙමාපිය වැගයට ලබා දිය හැකි නියෝග මොනවාද කියා.

1. display: flex;

මෙය **Flexbox** ආරම්භ කරන ස්විචය වැනිය. මෙය නොමැතිව **Flexbox** වැඩ කරන්නේ නැත.

2. flex-direction (දිශාව)

දරුවන් පෙළගැසිය යුත්තේ කුමන පැත්තටදැයි තීරණය කරයි.

- **row** (සම්මත අගය): වමේ සිට දකුණට.
- **row-reverse**: දකුණේ සිට වමට.
- **column**: ඉහළ සිට පහළට.
- **column-reverse**: පහළ සිට ඉහළට.

3. justify-content (ප්‍රධාන අක්ෂයේ පෙළගැස්ම)

ප්‍රධාන අක්ෂය (වමේ සිට දකුණට) දිගේ දේවල් පවතින්නේ කෙසේදැයි මෙය පාලනය කරයි. මෙය ඉතා වැදගත්ය.

- **flex-start**: සියල්ල මුලට (වම් පැත්තට) ගොනු කරයි.
- **flex-end**: සියල්ල අගට (දකුණු පැත්තට) ගොනු කරයි.
- **center**: සියල්ල හරියටම මැදට ගෙන එයි.
- **space-between**: පළමු සහ අවසාන අයිතම දෙපැත්තට තල්ලු කර, මැද ඇති අයිතම අතර සමාන ඉඩක් තබයි.
- **space-around**: සෑම අයිතමයක් වටාම සමාන ඉඩක් තබයි.

4. align-items (හරස් අක්ෂයේ පෙළගැස්ම)

සිරස් අතට (උස අනුව) දේවල් පවතින්නේ කෙසේදැයි පාලනය කරයි.

- **stretch** (සම්මත අගය): සියලුම දරුවන් **Container** එකේ උසට සමාන වන සේ ඇදෙයි.
- **center**: සිරස් අතට හරියටම මැදට ගෙන එයි.
- **flex-start**: ඉහළට ගොනු කරයි.
- **flex-end**: පහළට ගොනු කරයි.

19.5 ප්‍රායෝගික කේත උදාහරණය: මූලිකාංගයක් මැදට ගැනීම

ඕනෑම දෙයක් (උදා: **div** එකක් ඇතුළේ ඇති තවත් **div** එකක්) තිරයේ හරියටම මැදට ගැනීමට ඇති කෙටිම සහ හොඳම ක්‍රමය මෙයයි.

HTML:

```
<div class="box-container">
  <div class="inner-box">මම මැද සිටිමි</div>
</div>
```

CSS:

```
.box-container {
  display: flex;           /* Flexbox සක්‍රීය කිරීම */
  justify-content: center; /* තිරස් අතට මැදට ගැනීම */
  align-items: center;     /* සිරස් අතට මැදට ගැනීම */
  height: 300px;          /* පෙනීමට අවශ්‍ය නිසා උසක් ලබා දීම */
  border: 2px solid black;
}

.inner-box {
  background-color: orange;
  padding: 20px;
}
```

19.6 Flex Wrap (පේළි මාරු කිරීම)

සාමාන්‍යයෙන් **Flex Items** කොපමණ තිබුණත් ඒවා එකම පේළියක සිටීමට උත්සාහ කරයි. එම නිසා දේවල් හීනි වී පොඩි විය හැක. මෙය වැළැක්වීමට **flex-wrap: wrap**; භාවිතා කරයි. එවිට ඉඩ මදි වූ විට අයිතම ස්වයංක්‍රීයව ඊළඟ පේළියට යයි.

19.7 වෘත්තීය උපදෙස් (Professional Tips)

- **Navigation Bars:** වෙබ් අඩවියක ඉහළ ඇති Menu එකක් සෑදීමට **justify-content: space-between;** ඉතාමත් ප්‍රයෝජනවත් වේ (එක් පැත්තක Logo එකක් අනෙක් පැත්තේ Links ටිකක් තැබීමට).
- **Mobile Friendly:** දුරකථනයකදී (Mobile) **flex-direction: column;** ලෙස වෙනස් කිරීමෙන් ඉතා පහසුවෙන් වෙබ් අඩවිය **Responsive** කළ හැක.

19.8 පොදු වැරදි (Common Mistakes)

1. **display: flex** ලබා දීමට අමතක වීම: මෙය නොමැතිව අනෙක් කිසිදු **Flex** ගුණාංගයක් ක්‍රියා නොකරයි.
2. **Parent** සහ **Child** පටලවා ගැනීම: **justify-content** වැනි නීති ලබා දිය යුත්තේ දෙමාපිය ටැගයට (**Parent**) මිස ඇතුළේ සිටින දරුවන්ට නොවේ.

19.9 අභ්‍යාස (Exercises)

1. **Flex Container** සහ **Flex Items** අතර ඇති වෙනස ඔබේ වචනයෙන් පැහැදිලි කරන්න.
2. **justify-content** සහ **align-items** අතර ඇති ප්‍රධාන වෙනස කුමක්ද?
3. පහත දැක්වෙන අවශ්‍යතාවය සඳහා **CSS** ලියන්න:
 - එක් දිගු පේළියක (**Row**) අයිතම 3ක් තිබිය යුතුය.
 - ඒවා අතර සමාන ඉඩක් (**Space**) තිබිය යුතුය.
 - ඒවා සිරස් අතට මැදට (**Center**) වී තිබිය යුතුය.

19.10 ප්‍රායෝගික ව්‍යාපෘතිය (Mini Project)

ව්‍යාපෘතිය: සරල **Navbar** එකක් නිර්මාණය කිරීම

1. **header** ටැගයක් සාදා ඒ තුළ **div.logo** සහ **nav** ටැගයක් ඇතුළත් කරන්න.
2. **nav** ටැගය තුළ ලින්ක්ස් (**Home, About, Contact**) 3ක් ලියන්න.
3. **Flexbox** භාවිතා කර:
 - **Logo** එක වම් පැත්තටත්, **Menu** එක දකුණු පැත්තටත් තල්ලු කරන්න.

- **Menu** එක ඇතුළේ ඇති ලින්ක්ස් 3 එක ළඟින් (නිරස් අතට) තබන්න.
- සියල්ල සිරස් අතට මැදට පෙළගස්වන්න.

මිළඟ පරිච්ඡේදයෙන් අපි **Flexbox** හි ඉතිරි කොටස වන "දරුවන්ට ලබා දිය හැකි ගුණාංග" (**Flex Grow, Shrink, Basis**) සහ සංකීර්ණ පිරිසැලසුම් සාදන ආකාරය ඉගෙන ගනිමු.

විසිවන පරිච්ඡේදය:

ෆ්ලෙක්ස්බොක්ස් අයිතම පාලනය (Flexbox Items - Deep Dive)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

1. **Flex Items** සඳහා පමණක් භාවිතා වන විශේෂිත **CSS** ගුණාංග 5 හඳුනා ගැනීම.
2. **flex-grow** මගින් ඉතිරි ඉඩ ලබා ගන්නා ආකාරය ගණිතමය වශයෙන් තේරුම් ගැනීම.
3. **flex-shrink** මගින් අයිතමයක් අනවශ්‍ය ලෙස හැකිලීම වළක්වන ආකාරය.
4. **flex-basis** සහ **width** අතර ඇති සියුම් වෙනස වටහා ගැනීම.
5. **flex** නමැති කෙටි යෙදුම (**Shorthand**) නිවැරදිව භාවිතා කරන ආකාරය.
6. **align-self** මගින් තනි අයිතමයක පිහිටීම වෙනස් කරන ආකාරය.
7. **order** ගුණාංගය මගින් **HTML** කේතය වෙනස් නොකර අයිතමවල අනුපිළිවෙළ මාරු කිරීම.

20.1 හැදින්වීම (Introduction)

දහනවවන පරිච්ඡේදයේදී අපි ඉගෙන ගත්තේ "දෙමාපිය පෙට්ටිය" (**Flex Container**) මගින් එහි සිටින සියලුම දරුවන් පොදුවේ පාලනය කරන්නේ කෙසේද කියාය. නමුත් සැබෑ වෙබ් අඩවි නිර්මාණයේදී අපට එක් එක් දරුවා (**Item**) වෙත වෙනම පාලනය කිරීමට සිදු වේ.

උදාහරණයක් ලෙස, වෙබ් අඩවියක **Navigation Bar** එකක් සලකන්න. එහි **Logo** එකට වැඩි ඉඩක් ලබා දී, අනෙක් ලින්ක්ස් (**Links**) වලට සාමාන්‍ය ඉඩක් ලබා දීමට අපට අවශ්‍ය විය හැක. මෙවැනි අවස්ථාවලදී අප භාවිතා කරන්නේ **Flex Item Properties** ය.

20.2 Flex Grow: ඉතිරි ඉඩ බෙදා ගැනීම

Flex Grow (ෆ්ලෙක්ස් ග්‍රෝ) යනු මූලිකාංගයකට ලබා දී ඇති ඉඩට වඩා වැඩිපුර ඉඩක් පවතින විට, එම ඉඩෙන් කොපමණ ප්‍රමාණයක් අදාළ අයිතමය ලබා ගත යුතුද යන්න තීරණය කරන අංකයකි.

- **Default Value** (සම්මත අගය): **0** (එයින් අදහස් වන්නේ ඉතිරි ඉඩ ලබාගෙන අයිතමය විශාල නොවන බවයි).

ගණිතමය පැහැදිලි කිරීම: සිත්තන **Container** එකක පළල **500px** කියා, එහි අයිතම **3**ක් ඇත. එම අයිතම **3**ම **100px** බැගින් පළලයි. එවිට තවත් **200px** ඉතිරි වේ.

1. සියල්ලටම **flex-grow: 0** නම්: අයිතම **3**ම **100px** ලෙස පවතී. ඉතිරි **200px** හිස්ව පවතී.
2. එක අයිතමයකට **flex-grow: 1** ලබා දුන්නොත්: එම අයිතමය ඉතිරි **200px**ම ලබා ගෙන **300px** දක්වා විශාලවේ.
3. අයිතම **3**ටම **flex-grow: 1** ලබා දුන්නොත්: ඉතිරි **200px** කොටස් **3**කට බෙදා සමානව ලබා ගනී.

උපමාවක්: සහෝදරයන් තිදෙනෙකුට අම්මා කේක් ගෙඩියක් ගෙන ආවා යැයි සිතන්න. අම්මා පවසනවා "ඔයාලට බඩ පිරෙනකම් කෑවට පස්සේ ඉතිරි වෙන කොටස ඔයාලා අතරේ බෙදාගන්න" කියා. මෙහිදී **flex-grow** යනු එම ඉතිරි කොටස බෙදා ගන්නා "අනුපාතයයි".

20.3 Flex Shrink: හැකිලීම පාලනය කිරීම

Flex Shrink (ෆ්ලෙක්ස් ශ්‍රින්ක්) යනු **Container** එකේ ඉඩ මදි වූ විට, අදාළ අයිතමය කොපමණ ප්‍රමාණයකින් හැකිලිය යුතුද යන්න තීරණය කරන අංකයකි.

- **Default Value** (සම්මත අගය): **1** (එයින් අදහස් වන්නේ ඉඩ මදි වූ විට සියලුම අයිතම සමානව හැකිලෙන බවයි).

ප්‍රායෝගික භාවිතය: ඔබට කිසියම් අයිතමයක් (උදා: පින්තූරයක් හෝ **Logo** එකක්) කිසිසේත්ම හැකිලීමට අවශ්‍ය නැතිනම්, එයට **flex-shrink: 0;** ලබා දිය යුතුය. එවිට අනෙක් අයිතම හැකිලී එම අයිතමයට අවශ්‍ය ඉඩ ලබා දෙයි.

20.4 Flex Basis: මූලික ප්‍රමාණය

Flex Basis (ෆ්ලෙක්ස් බේසිස්) යනු **flex-grow** හෝ **flex-shrink** ක්‍රියාත්මක වීමට පෙර අයිතමයකට තිබිය යුතු මූලික පළල (හෝ උස) වේ.

- ප්‍රශ්නය: මෙය **width** එකකට සමාන නොවේද?
- පිළිතුර: බොහෝ දුරට සමානය. නමුත් **flex-direction: column** වූ විට, **flex-basis** මගින් පාලනය කරන්නේ පළල නොව උස (**Height**) ය. එනම්, මෙය ප්‍රධාන අක්ෂයේ (**Main Axis**) මූලික ප්‍රමාණයයි.

20.5 Flex shorthand: කෙටි ක්‍රමය

වෘත්තීය වෙබ් සංවර්ධකයන් **grow**, **shrink** සහ **basis** වෙන වෙනම ලියන්නේ නැත. ඒ වෙනුවට ඔවුන් **flex** නමැති කෙටි යෙදුම භාවිතා කරයි.

විෂුභය: **flex: [grow] [shrink] [basis];**

උදාහරණය:

```
.item {
  flex: 1 0 200px;
}
```

මෙහි තේරුම: ඉතිරි ඉඩ තිබේ නම් ලබා ගන්න (1), ඉඩ මදි වුවද හැකිලෙන්න එපා (0), මූලික ප්‍රමාණය **200px** විය යුතුය.

20.6 Align Self: තනිව කැපී පෙනීම

අපි 19 වන පරිච්ඡේදයේදී **align-items** ගැන ඉගෙන ගත්තෙමු. එයින් සියලුම දරුවන් එකවර පාලනය කරයි. නමුත් **Align Self** මගින් එක් දරුවෙකුට පමණක් අනෙක් අයගෙන් වෙන් වී වෙනස් ලෙස පෙළගැසීමට ඉඩ ලබා දෙයි.

උදාහරණය: අනෙක් සියලුම අයිතම **Container** එකේ ඉහළ (**flex-start**) සිටින විට, එක් අයිතමයක් පමණක් පතුලට (**flex-end**) ගෙන යාමට මෙය භාවිතා කළ හැක.

20.7 Order: පෙත්වන පිළිවෙළ වෙනස් කිරීම

සාමාන්‍යයෙන් වෙබ් පිටුවක දේවල් පෙන්නෙන්නේ **HTML** කේතයේ ඒවා ලියා ඇති පිළිවෙළටය. නමුත් **Order** (ඕඩර්) ගුණාංගය මගින් අපට මෙම පිළිවෙළ ඕනෑම වෙලාවක වෙනස් කළ හැක.

- සම්මත අගය: 0.
- අංකය කුඩා වන තරමට අයිතමය මූලින් දිස්වේ. අංකය විශාල වන තරමට අයිතමය අගින් දිස්වේ.

ප්‍රයෝජනය: දුරකථනයකදී (Mobile) වෙබ් අඩවිය බලන විට, මූලින් පෙනෙන පින්තූරය පසුව පෙන්වීමටත්, පසුව ඇති ලිපිය මූලින් පෙන්වීමටත් මෙය ඉතා ප්‍රයෝජනවත් වේ.

20.8 සම්පූර්ණ කේත උදාහරණය (Complete Example)

```
<div class="flex-container">
  <div class="item logo">LOGO</div>
  <div class="item nav">Home, About,
  Contact</div>
  <div class="item btn">Login</div>
</div>
```

```
.flex-container {
  display: flex;
  background-color: #f1f1f1;
  padding: 10px;
  align-items: center;
}
```

```
.item {
  padding: 20px;
  background-color: skyblue;
  margin: 5px;
}
```

```
/* Logo එකට ඉතිරි සියලුම ඉඩ ලබා දෙන්න */
.logo {
  flex-grow: 1;
}
```

```
/* Login බොත්තම කිසිවිටකත් හැකිලෙන්න එපා */
.btn {
  flex-shrink: 0;
  align-self: flex-end; /* මෙම බොත්තම පමණක් පහළට කරන්න */
}
```

```
/* Navigation ලැයිස්තුව අන්තිමට පෙන්වන්න */
.nav {
  order: 2;
}
```

20.9 ASCII රූපසටහන: Flex Grow ක්‍රියා කරන ආකාරය

Container (500px)

[Item 1]	[Item 2]	[Item 3]	භිස් ඉඩ
100px	100px	100px	200px

Item 2 හට flex-grow: 1 ලබා දුන් පසු:

[Item 1]	[Item 2]	[Item 3]
100px	300px	100px

20.10 පොදු වැරදි (Common Mistakes)

1. **Flex Basis** සහ **Width** පටලවා ගැනීම: **Flexbox** භාවිතා කරන විට **width** භාවිතා කරනවාට වඩා **flex-basis** භාවිතා කිරීම වෘත්තීය මට්ටමේ ක්‍රමයයි.
2. **Order** භාවිතා කිරීමේදී පරෙස්සම් වන්න: පෙනෙන පිළිවෙළ වෙනස් වුවද, **Screen Readers** (නිරය කියවන මෘදුකාංග) තවමත් කියවන්නේ **HTML** හි ඇති පිළිවෙළටමය. මෙය ආබාධිත පුද්ගලයන්ට ගැටලුවක් විය හැක.

20.11 සාරාංශය (Key Points)

- **flex-grow**: භිස් ඉඩ ලබා ගැනීමට.
- **flex-shrink**: ඉඩ මදි වූ විට හැකිලීමට.
- **flex-basis**: ප්‍රධාන අක්ෂයේ මූලික ප්‍රමාණය.
- **align-self**: තනි අයිතමයක සිරස් පෙළගැස්ම.
- **order**: පෙන්වන අනුපිළිවෙළ වෙනස් කිරීම.

20.12 අභ්‍යාස (Exercises)

1. **flex: 0 1 auto**; යන්නෙහි තේරුම කොටස් වශයෙන් පැහැදිලි කරන්න.
2. පෙට්ටි 3ක් ඇති **Flex Container** එකක මැද පෙට්ටියට පමණක් ඉතිරි ඉඩ ලබා ගැනීමට අවශ්‍ය **CSS** ලියන්න.
3. **HTML** කේතයේ මූලින්ම ඇති **div** එකක්, තිරයේ අන්තිමට පෙන්වීමට කළ යුත්තේ කුමක්ද?

මිළඟ පරිච්ඡේදයෙන් අපි **CSS** වල ඇති බලවත්ම සහ විශාලතම පිරිසැලසුම් ක්‍රමය වන "සි.එස්.එස්. ග්‍රිඩ්" (**CSS Grid**) ගැන ඉතාමත් විස්තරාත්මකව ඉගෙන ගනිමු.

විසිඑක්වන පරිච්ඡේදය: සී.එස්.එස්. ග්‍රිඩ් - ද්විමාන පිරිසැලසුම් (CSS Grid Layout - 2D Layouts)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

1. CSS Grid යනු කුමක්ද සහ එය Flexbox වලින් වෙනස් වන්නේ කෙසේද?
2. Grid Container සහ Grid Item අතර වෙනස හඳුනා ගැනීම.
3. තීරු (Columns) සහ පේළි (Rows) නිර්මාණය කරන ආකාරය.
4. fr (Fractional Unit) නමැති නවීන ඒකකය භාවිතා කරන ආකාරය.
5. repeat() සහ minmax() වැනි විශේෂ ශ්‍රිත (Functions) භාවිතා කිරීම.
6. ග්‍රිඩ් එකක කොටස් අතර පරතරය (Gap) පාලනය කිරීම.
7. Grid Areas භාවිතා කර මුළු වෙබ් පිටුවක සැකිල්ලක් (Layout) සාදන ආකාරය.

21.1 හැඳින්වීම (Introduction)

පෙර පරිච්ඡේද දෙකේදී අපි Flexbox ගැන ඉගෙන ගත්තෙමු. Flexbox ඉතා විශිෂ්ට වුවත්, එය නිර්මාණය කර ඇත්තේ එක දිශාවකට (පේළියක් හෝ තීරුවක් ලෙස) දේවල් පෙළගැස්වීමටය. නමුත් වෙබ් අඩවියක මුළු සැකිල්ලම (Layout) නිර්මාණය කිරීමේදී අපට පේළි සහ තීරු යන දෙකම එකවර පාලනය කිරීමට සිදු වේ. මෙන්න මෙතැනදී අපට CSS Grid උදව් වේ.

CSS Grid යනු CSS වල ඇති බලවත්ම සහ සංකීර්ණම පිරිසැලසුම් පද්ධතියයි. මෙය ද්විමාන (2D) පද්ධතියකි. එනම් මෙහිදී අපට තිරස් අතට (Horizontal) සහ සිරස් අතට (Vertical) යන දිශා දෙකම එකවර පාලනය කළ හැක.

21.2 Flexbox සහ Grid අතර වෙනස (Comparison)

ලක්ෂණය	Flexbox	CSS Grid
මානය (Dimension)	ඒකමාන (1D) - පේළියක් හෝ තීරුවක් පමණි.	ද්විමාන (2D) - පේළි සහ තීරු දෙකම.
ප්‍රධාන අරමුණ	අයිතම එක පෙළට පෙළගැස්වීම.	මුළු පිටුවේ ව්‍යුහය (Structure) නිර්මාණය කිරීම.
භාවිතය	Navbar, Buttons වැනි කුඩා කොටස් සඳහා.	වෙබ් අඩවියක ප්‍රධාන Layout එක සඳහා.

21.3 ග්‍රිඩ් එකක මූලික පාරිභාෂික වචන (Terminology)

Grid එකක් තේරුම් ගැනීමට පෙර මෙම වචන දැන සිටීම අත්‍යවශ්‍ය වේ:

1. **Grid Container:** `display: grid;` ලබා දෙන ප්‍රධාන පෙට්ටිය.
2. **Grid Item:** **Container** එක ඇතුළත සිටින දරුවන්.
3. **Grid Line:** කොටු වෙන් කරන ඉරි.
4. **Grid Track:** ඡේද්‍රයක් හෝ තීරුවක්. (ඡේද්‍ර දෙකක් අතර ඇති ඉඩ).
5. **Grid Cell:** ග්‍රිඩ් එකක ඇති කුඩාම තනි කොටුව. (වගුවක ඇති කොටුවක් වැනිය).
6. **Grid Area:** කොටු කිහිපයක් එකතු වී සෑදෙන විශාල කොටසක්.

21.4 ඡේද්‍ර සහ තීරු සෑදීම (Defining Columns and Rows)

ග්‍රිඩ් එකක් සෑදීමට අප මූලිකව පවසන්න ඕනේ එහි තීරු කීයක් සහ ඡේද්‍ර කීයක් තිබිය යුතුද කියා.

HTML:

```
<div class="container">
  <div class="item">1</div>
  <div class="item">2</div>
  <div class="item">3</div>
  <div class="item">4</div>
</div>
```

CSS:

```
.container {
  display: grid;
  /* තීරු 3ක් සාදමු. පළල 100px බැගින් */
  grid-template-columns: 100px 100px 100px;
  /* ඡේද්‍ර 2ක් සාදමු. උස 50px බැගින් */
  grid-template-rows: 50px 50px;
}
```

21.5 නවීන ඒකකය: fr (Fractional Unit)

වෙබ් අඩවි නිර්මාණයේදී **px** භාවිතා කිරීම සැමවිටම සාර්ථක නැත. මන්ද විවිධ තිර වල ප්‍රමාණය වෙනස් නිසාය. ඒ සඳහා **CSS Grid** වලටම ආවේණික වූ **fr** නමැති ඒකකය හඳුන්වා දී ඇත.

fr යනු "**Fraction**" (භාගය) යන්නයි. මෙයින් කරන්නේ පවතින මුළු ඉඩ කොටස් වලට බෙදා දීමයි.

උදාහරණය:

```
grid-template-columns: 1fr 1fr 1fr;
```

මෙහිදී තීරු 3ක් සෑදෙන අතර, ඒ සෑම එකක්ම පවතින ඉඩෙන් $\frac{1}{3}$ ක් බැගින් (සමානව) ලබා ගනී.

21.6 කෙටි ක්‍රම: repeat() සහ minmax()

ඔබට සමාන තීරු 10ක් අවශ්‍ය නම් **1fr** දස වතාවක් ටයිප් කිරීම අපහසුය. ඒ සඳහා **repeat()** භාවිතා කළ හැක.

```
/* 1fr තීරු 3ක් සාදන්න */
grid-template-columns: repeat(3, 1fr);
```

minmax() යනු අයිතමයක අවම සහ උපරිම ප්‍රමාණය තීරණය කිරීමට භාවිතා කරන ශ්‍රිතයකි.

```
/* අවම 100px ද උපරිම පවතින ඉඩ ද (1fr) ලබා ගන්න */
grid-template-columns: repeat(3, minmax(100px, 1fr));
```

21.7 පරතරය තැබීම (Gap)

ග්‍රිඩ් එකේ කොටු එකිනෙක ඇළි පවතින විට ඒවා වෙන් කිරීමට **gap** ගුණාංගය භාවිතා කරයි.

```
.container {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 20px; /* ඡේද්‍ර සහ තීරු අතර 20px ඉඩක් තබයි */
}
```

21.8 Grid Areas මගින් Layout එකක් සෑදීම (Professional Method)

මෙය **CSS Grid** වල ඇති ඉතාමත් පහසු සහ රූපසටහනක් අඳින්නාක් වැනි ක්‍රමයකි.

උදාහරණය:

```
.container {
  display: grid;
  grid-template-areas:
    "header header header"
    "sidebar main main"
    "footer footer footer";
  grid-template-columns: 200px 1fr 1fr;
}
```

```
.header { grid-area: header; }
.sidebar { grid-area: sidebar; }
.main { grid-area: main; }
.footer { grid-area: footer; }
```

මෙය ක්‍රියා කරන්නේ මෙලෙසයි (ASCII Diagram):

HEADER		
SIDEBAR (200px)	MAIN (1fr)	
FOOTER		

21.9 වෘත්තීය උපදෙස් (Professional Tips)

- කවදාද **Grid** භාවිතා කරන්නේ? ඔබ වෙබ් පිටුවක ප්‍රධාන සැකිල්ල (**Outer layout**) සාදන විට සැමවිටම **Grid** භාවිතා කරන්න.
- කවදාද **Flexbox** භාවිතා කරන්නේ? එම සැකිල්ල ඇතුළත ඇති කුඩා දේවල් (උදා: **Navigation links, Buttons**) එක පෙළට පෙළගැස්වීමට **Flexbox** භාවිතා කරන්න.
- **Inspect Element: Chrome** බ්‍රවුසරයේ **Inspect** කර බැලීමේදී **Grid** එකක ඉරි (**Lines**) පෙන්වීමට විශේෂ පහසුකමක් ඇත. එය භාවිතා කර කේතය පරීක්ෂා කරන්න.

21.10 පොදු වැරදි (Common Mistakes)

1. දරුවන්ට **Display Grid** ලබා දීම: **display: grid;** ලබා දිය යුත්තේ දෙමාපිය ටැගයට පමණි. දරුවන් ස්වයංක්‍රීයව **Grid Items** බවට පත් වේ.
2. **fr** සහ **auto** පටලවා ගැනීම: **auto** ලබා දුන් විට එය දත්ත වල ප්‍රමාණයට අනුව සකස් වේ. **fr** ලබා

දුන් විට එය ඉතිරිව ඇති සියලුම ඉඩ ලබා ගැනීමට උත්සාහ කරයි.

21.11 සාරාංශය (Key Points)

- **CSS Grid** යනු ද්විමාන (ජ්‍යෙෂ්ඨ සහ තීරු) පිරිසැලසුම් පද්ධතියකි.
- **grid-template-columns** සහ **grid-template-rows** මගින් ව්‍යුහය සාදයි.
- **fr** ඒකකය මගින් ඉඩ ප්‍රමාණයන් භාග ලෙස බෙදා දෙයි.
- **grid-area** මගින් වෙබ් පිටුවේ කොටස් නම් කර ලේසිව සැකිල්ලක් සෑදිය හැක.

21.12 අභ්‍යාස (Exercises)

1. **Flexbox** සහ **CSS Grid** අතර ඇති ප්‍රධාන වෙනස්කම් 3ක් ලියන්න.
2. සමාන ප්‍රමාණයේ තීරු 4ක් සෑදීමට අවශ්‍ය **CSS** කේතය ක්‍රම දෙකකට ලියන්න.
3. පහත දැක්වෙන **Layout** එක සඳහා **CSS Grid Areas** භාවිතා කර කේතය ලියන්න:
 - ඉහළින් **Header** එක.
 - මැද තීරු දෙකකි (**Content** සහ **Sidebar**).
 - පහළින් **Footer** එක.

21.13 ප්‍රායෝගික ව්‍යාපෘතිය (Mini Project)

ව්‍යාපෘතිය: පුවත්පත් වෙබ් පිටුවක සැකිල්ල (**News Portal Layout**)

1. **HTML** වල **div.container** එකක් සාදා ඒ තුළ **header, nav, main, sidebar** සහ **footer** යන ටැග් ඇතුළත් කරන්න.
2. **CSS Grid** භාවිතා කර **header** එක මුදුනේ සම්පූර්ණ පළල ගන්නා ලෙසත්, **nav** එක ඊට පහළින් ද, **main** සහ **sidebar** එක ළඟින් ද (**Sidebar** එක **300px**), **footer** එක යටින් ද පවතින සේ සකසන්න.
3. කොටස් හඳුනා ගැනීමට විවිධ පසුබිම් වර්ණ (**Background colors**) ලබා දෙන්න.

මිලහ පරිවිෂේදයෙන් අපි මූලිකාංග තිබිය යුතු ස්ථානය තීරණය කරන ආකාර 5 හෙවත් "සි.එස්.එස්. පොසිෂන්" (**CSS Position**) ගැන ඉතාමත් විස්තරාත්මකව ඉගෙන ගනිමු.

විසිදෙන පරිච්ඡේදය: පිහිටීම පාලනය කිරීම (CSS Position Property)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

1. **CSS Position** යනු කුමක්ද සහ එය භාවිතා කරන අවස්ථා තේරුම් ගැනීම.
2. සාමාන්‍ය ලේඛන ප්‍රවාහය (**Normal Document Flow**) යනු කුමක්දැයි වටහා ගැනීම.
3. **Position** අගයන් 5 (**Static, Relative, Absolute, Fixed, Sticky**) ඉතා පැහැදිලිව හඳුනා ගැනීම.
4. **top, bottom, left, right** යන ගුණාංග භාවිතා කරන ආකාරය.
5. **z-index** මගින් ස්ථර (**Layers**) පාලනය කරමින් මූලිකාංග එකක් මත එකක් තබන ආකාරය.

22.1 හැඳින්වීම (Introduction)

අපි **HTML** වල වැග් එකකට පස්සේ තව වැග් එකක් ලියනකොට, බ්‍රවුසරය ඒ වැග් එක පෙන්වන්නේ කලින් වැග් එකට යටින් හෝ ඒ අසලින්මයි. මෙයට **Normal Document Flow** (සාමාන්‍ය ලේඛන ප්‍රවාහය) කියා කියනවා.

නමුත් සමහර අවස්ථාවලදී අපිට අවශ්‍ය වෙනවා යම් කොටසක් (උදාහරණයක් ලෙස **"Chat"** බොත්තමක්) තිරයේ එකම තැනක සිර කර තබන්න, නැතිනම් පින්තූරයක් උඩින් අකුරු ටිකක් ලියන්න. සාමාන්‍ය ක්‍රමයට මේ දේවල් කරන්න අපහසුයි. මෙන්න මේ වගේ වෙලාවට තමයි අපි **CSS Position** ගුණාංගය භාවිතා කරන්නේ.

22.2 Position අගයන් පහ (The Five Values)

CSS වල පිහිටීම පාලනය කිරීමට ප්‍රධාන අගයන් 5ක් තිබෙනවා. අපි ඒවා එකින් එක සවිස්තරාත්මකව බලමු.

1. Static (සම්මත පිහිටීම)

මෙය සෑම **HTML** වැග් එකකටම ස්වයංක්‍රීයව ලැබෙන අගයයි.

- හැසිරීම: වැග් එක ලියා ඇති පිළිවෙලට තිරයේ දිස්වේ.
- විශේෂත්වය: **top, bottom, left, right** වැනි අගයන් මෙයට ලබා දුන්නද කිසිදු වෙනසක් සිදු නොවේ.

2. Relative (සාපේක්ෂ පිහිටීම)

මෙය වැගය සාමාන්‍යයෙන් තිබිය යුතු ස්ථානයේ සිට සුළු වශයෙන් වෙනස් කිරීමට භාවිතා කරයි.

- හැසිරීම: **top: 20px;** ලෙස ලබා දුන්නොත්, එය සාමාන්‍යයෙන් තිබිය යුතු තැන සිට **20px** ක් පහළට ගමන් කරයි.
- විශේෂත්වය: එය මීට පෙර සිටි ස්ථානය බ්‍රවුසරය විසින් තවමත් මතක තබා ගනී (හිස් ඉඩක් ලෙස පවතී). අනෙකුත් වැග් එම හිස් ඉඩට පැමිණෙන්නේ නැත. එය "හොල්මනක්" වැනිය; ශරීරය වෙන තැනක තිබුණත්, එහි ඉඩ තවමත් වෙන් වී පවතී.

3. Absolute (නිරපේක්ෂ පිහිටීම)

මෙය ඉතාමත් බලවත් අගයකි. මෙය භාවිතා කළ විට එම වැගය සාමාන්‍ය ලේඛන ප්‍රවාහයෙන් සම්පූර්ණයෙන්ම ඉවත් වේ.

- හැසිරීම: මෙය තමන්ගේ දෙමාපිය වැගයට (**Parent**) සාපේක්ෂව පිහිටීම තීරණය කරයි.
- කොන්දේසිය: මෙය නිවැරදිව වැඩ කිරීමට නම්, එහි දෙමාපිය වැගයට **position: relative;** ලබා දී තිබිය යුතුය. දෙමාපියෙකුට **position** එකක් නැතිනම්, මෙය මුළු වෙබ් පිටුවටම (**<body>**) සාපේක්ෂව ක්‍රියා කරයි.
- උපමාවක්: මෙය පියාසර කරන "ඩ්‍රෝන්" (**Drone**) එකක් වැනිය. එය පොළොවේ ඉඩක් ගන්නේ නැත, එයට ඕනෑම තැනක පාවෙමින් සිටිය හැක.

4. Fixed (ස්ථාවර පිහිටීම)

වෙබ් පිටුව කොපමණ පහළට ස්ක්‍රෝල් (**Scroll**) කළත්, මෙය තිරයේ එකම තැනක සිරවී පවතී.

- හැසිරීම: මෙය බ්‍රවුසරයේ අපට පෙනෙන තිරයට (**Viewport**) සාපේක්ෂව ක්‍රියා කරයි.
- ප්‍රායෝගික භාවිතය: වෙබ් අඩවි වල පහළ කෙළවරේ ඇති **"Back to Top"** බොත්තම් හෝ මැසෙන්ජර් වැට් බොත්තම් සාදන්නේ මෙලෙසයි.

5. Sticky (ඇලෙන සුළු පිහිටීම)

මෙය **Relative** සහ **Fixed** යන දෙකේම මිශ්‍රණයකි.

- හැසිරීම: යම් සීමාවකට එන තෙක් එය සාමාන්‍ය පරිදි ස්ක්‍රෝල් වේ. නමුත් එම සීමාව (උදා: **top: 0;**) පැමිණි පසු එය එතැන ඇලී පවතී.

- ප්‍රායෝගික භාවිතය: වගුවක මාතෘකා පේළිය (Table Header) ස්ක්‍රෝල් කරන විට පෙනෙන සේ තබා ගැනීමට මෙය ඉතා සුදුසුයි.

22.3 දිශා පාලනය (Top, Bottom, Left, Right)

Position එකක් ලබා දුන් පසු, එය තිබිය යුතු නිවැරදි ස්ථානය පවසන්නේ මෙම ගුණාංග මගිනි.

- **top: 50px;** - ඉහළ සිට 50px ක පරතරයක් තබන්න.
- **bottom: 0;** - පතුලටම අලවන්න.
- **left: 10%;** - වම් පස සිට 10% ක ඉඩක් තබන්න.

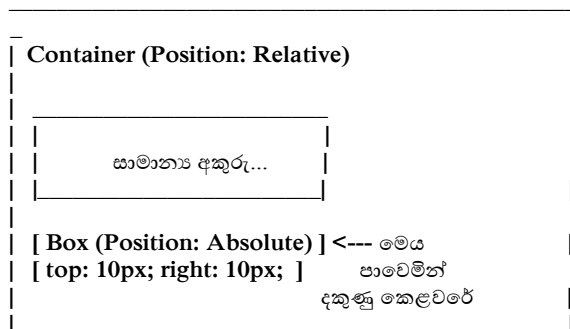
22.4 Z-Index: ස්ථර පාලනය (Layering)

අපි ටැග් දෙකක් එකක් මත එකක් තැබූ විට (**Position** භාවිතා කර), උඩින් පෙනිය යුත්තේ කුමන එකද යන්න තීරණය කරන්නේ **z-index** මගිනි.

- පරිගණක තිරය **X** (තිරස්) සහ **Y** (සිරස්) අක්ෂ වලින් සමන්විතයි. **Z** යනු ඔබ දෙසට දිවෙන අක්ෂයයි.
- **z-index: 1;** ඇති ටැගයට වඩා **z-index: 10;** ඇති ටැගය ඉදිරියෙන් (උඩින්) පෙනේ.

වැදගත්: **z-index** ක්‍රියා කරන්නේ **static** නොවන (එනම් **relative**, **absolute**, **fixed** හෝ **sticky**) පිහිටීමක් ඇති ටැග් සඳහා පමණි.

22.5 ASCII රූපසටහන: Position Absolute ක්‍රියා කරන ආකාරය



22.6 ප්‍රායෝගික කේත උදාහරණය: පින්තූරයක් මත අකුරු තැබීම

HTML:

```
<div class="card">
  
  <div class="overlay-text">ලස්සන
ලංකාව</div>
</div>
```

CSS:

```
.card {
  position: relative; /* දෙමාපිය ටැගය සාපේක්ෂයි */
  width: 300px;
}

.card img {
  width: 100%;
  display: block;
}

.overlay-text {
  position: absolute; /* අකුරු ටික පාවෙමින් පවතී */
  bottom: 20px; /* පතුලේ සිට 20px ඉහළින් */
  left: 20px; /* වම් පැත්තේ සිට 20px ඇතුළතින් */
  color: white;
  background-color: rgba(0, 0, 0, 0.5); /* කළු පැහැති විනිවිද පෙනෙන පසුබිමක් */
  padding: 10px;
}
```

22.7 පොදු වැරදි (Common Mistakes)

1. දෙමාපිය ටැගයට **Relative** ලබා දීමට අමතක වීම: **absolute** දරුවෙකු සැමවිටම සොයන්නේ **relative** දෙමාපියෙකු වෙයි. ඔහු නැතිනම් දරුවා මුළු පිටුවේම කොහේ හෝ තැනකට විසි වී යයි.
2. **Z-index** සහ **Static: position** එකක් ලබා නොදී **z-index** ලබා දීමෙන් කිසිදු ප්‍රයෝජනයක් නැත.
3. **Fixed Position** නිසා දත්ත වැසී යාම: **Navigation Bar** එකක් **fixed** කළ විට, පිටුවේ මුළු ඇති අකුරු ඒ යටට ගොස් වැසී යා හැක. මෙය

වැළැක්වීමට **body** එකට ඉහළින් පැඩ් (padding-top) එකක් ලබා දිය යුතුය.

22.8 වෘත්තීය උපදෙස් (Professional Tips)

- **Sticky Headers:** නවීන වෙබ් අඩවි වල Menu එක පෙනෙන්නට තබා ගැනීමට **position: sticky; top: 0;** භාවිතා කිරීම ඉතාමත් ජනප්‍රියයි.

- **Centering with Absolute:** යමක් හරියටම මැදට ගැනීමට **top: 50%; left: 50%; transform: translate(-50%, -50%);** යන කේත සමූහය නිතර භාවිතා වේ.

22.9 අභ්‍යාස (Exercises)

1. **Relative සහ Absolute** අතර ඇති ප්‍රධාන වෙනස කුමක්ද?
2. වෙබ් පිටුව ස්ක්‍රෝල් කරන විටත් පෙනෙන සේ "Chat with us" බොත්තමක් සැදීමට සුදුසුම **Position** අගය කුමක්ද?
3. **z-index** භාවිතා කිරීමට පෙර සපුරාලිය යුතු ප්‍රධාන කොන්දේසිය කුමක්ද?
4. පහත කේතයේ **box** එක තිබෙන ස්ථානය විස්තර කරන්න:

```
.box {
  position: fixed;
  top: 0;
  right: 0;
}
```

22.10 ප්‍රායෝගික ව්‍යාපෘතිය (Mini Project)

ව්‍යාපෘතිය: පාවෙන පණිවිඩ කොටුවක් (Modal/Popup Box)

1. මුළු තිරයම වැසෙන සේ කළු පැහැති, විනිවිද පෙනෙන (**0.5 opacity**) **div** එකක් සාදන්න. එය **position: fixed** විය යුතුය.
2. එම **div** එක ඇතුළත සුදු පැහැති කුඩා පෙට්ටියක් (**Box**) සාදා, එය තිරයේ හරියටම මැදට (**Center**) ගන්න.
3. සුදු පෙට්ටියේ ඉහළ දකුණු කෙළවරේ "**X**" යනුවෙන් කුඩා බොත්තමක් **position: absolute** භාවිතා කර සවි කරන්න.
4. අවසානයේදී **z-index** භාවිතා කර මෙම මුළු කොටසම අනෙකුත් සියලුම දේට වඩා උඩින් පෙනෙන සේ සකසන්න.

මූලික පරිච්ඡේදයෙන් අපි වෙබ් අඩවියක් විවිධ තිර (Mobile, Tablet, Desktop) වලට ගැලපෙන සේ සකස් කරන "ප්‍රතිචාරාත්මක වෙබ් නිර්මාණය" (Responsive Web Design) ගැන ඉතාමත් විස්තරාත්මකව ඉගෙන ගනිමු.

විසිතුන්වන පරිච්ඡේදය: ප්‍රතිචාරාත්මක වෙබ් නිර්මාණය (Responsive Web Design)

ඉගෙනුම් අරමුණු (Learning Objectives)
මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

1. ප්‍රතිචාරාත්මක වෙබ් නිර්මාණය (Responsive Web Design - RWD) යනු කුමක්දැයි අවබෝධ කර ගැනීම.
2. **Viewport Meta Tag** එකේ වැදගත්කම සහ එය භාවිතා කරන ආකාරය.
3. **Media Queries** (මාධ්‍ය විමසුම්) භාවිතා කර විවිධ තිර ප්‍රමාණයන් සඳහා **CSS** ලියන ආකාරය.
4. **Breakpoints** (බිඳීමේ ලක්ෂ්‍යයන්) යනු මොනවාදැයි හඳුනා ගැනීම.
5. **Mobile-First** (මුලින්ම ජංගම දුරකථනයට) සැලසුම් කිරීමේ සංකල්පය.
6. නම්‍යශීලී ඒකක (Flexible Units) සහ පින්තුර භාවිතා කරන ආකාරය.

23.1 හැඳින්වීම (Introduction)

වෙබ් අඩවි නිර්මාණය වූ මුල් කාලයේදී මිනිසුන් අන්තර්ජාලයට පිවිසුණේ පරිගණක (Desktop Computers) හරහා පමණි. එම නිසා එකල වෙබ් අඩවි නිර්මාණය කළේ එක් ස්ථාවර පළලකට (උදාහරණයක් ලෙස 960px) අනුවය.

නමුත් අද තත්ත්වය සම්පූර්ණයෙන්ම වෙනස්ය. මිනිසුන් වෙබ් අඩවි බලන්නේ:

- කුඩා ස්මාර්ට් දුරකථන (Smartphones)
- ටැබ්ලට් පරිගණක (Tablets)
- ලැප්ටොප් (Laptops)
- විශාල රූපවාහිනී තිර (Smart TVs)

ඔබේ වෙබ් අඩවිය පරිගණකයකට පමණක් ගැලපෙන සේ සාදා තිබුණහොත්, එය ජංගම දුරකථනයකින් බලන විට අකුරු ඉතා කුඩා වී පෙනේ, නැතහොත් වෙබ් අඩවියේ කොටස් තිරයෙන් පිටතට ගොස් පෙනේ. මෙය පරිශීලකයාට ඉතා අපහසු අත්දැකීමකි.

23.2 ප්‍රතිචාරාත්මක වෙබ් නිර්මාණය යනු කුමක්ද? (What is Responsive Web Design?)

ප්‍රතිචාරාත්මක වෙබ් නිර්මාණය (**Responsive Web Design**) යනු ඕනෑම උපාංගයක තිරයේ ප්‍රමාණයට (**Screen Size**) ස්වයංක්‍රීයව හැඩගැසෙන සේ වෙබ් අඩවි නිර්මාණය කිරීමේ කලාවයි.

සැබෑ ලෝකයේ උදාහරණයක් (වතුර සහ බඳුන): ප්‍රසිද්ධ සටන් කලා ශිල්පී බෲස් ලී පැවසූ පරිදි, "වතුර බෝතලයකට දැමූ විට එය බෝතලයේ හැඩය ගනී, කෝප්පයකට දැමූ විට එය කෝප්පයේ හැඩය ගනී." වෙබ් අඩවියක්ද "වතුර" මෙන් විය යුතුය. එය විවෘත කරන ඕනෑම බඳුනක (තිරයක) හැඩය එය ගත යුතුය.

23.3 දර්ශන පථය: Viewport Meta Tag

ඕනෑම ප්‍රතිචාරාත්මක වෙබ් අඩවියක් නිර්මාණය කිරීමේ පළමු පියවර වන්නේ **HTML** ගොනුවේ **<head>** කොටස තුළ පහත කේතය ඇතුළත් කිරීමයි.

```
<meta name="viewport"
content="width=device-width, initial-scale=1.0">
```

මෙය කරන්නේ කුමක්ද?

- **Viewport** (දර්ශන පථය): යනු බ්‍රවුසරයේ වෙබ් අඩවිය පෙනෙන ප්‍රදේශයයි.
 - **width=device-width**: වෙබ් අඩවියේ පළල, පරිශීලකයා භාවිතා කරන උපාංගයේ පළලට සමාන කරන ලෙස බ්‍රවුසරයට පවසයි.
 - **initial-scale=1.0**: පිටුව මුලින්ම ලෝඩ් (**Load**) වන විට එය සාමාන්‍ය ප්‍රමාණයෙන් (**Zoom** නොවී) පෙන්වන ලෙස පවසයි.
- වැදගත්: මෙම ජෙළිය නොමැති නම්, ඔබ කොපමණ **CSS** ලිව්වත් ජංගම දුරකථන වලදී වෙබ් අඩවිය නිවැරදිව පෙනෙන්නේ නැත.

23.4 Media Queries (මාධ්‍ය විමසුම්)

ප්‍රතිචාරාත්මක වෙබ් අඩවියක "මොළය" වන්නේ **Media Queries** ය. මෙහිදී අපි බ්‍රවුසරයෙන් ප්‍රශ්නයක් අසමු: "තිරයේ පළල 600px ට වඩා අඩුද?" පිළිතුර "ඔව්" නම්, අපි ඒ සඳහා වෙනම **CSS** නීති මාලාවක් ලබා දෙමු.

Syntax (ලිවීමේ රටාව):

```
@media screen and (max-width: 600px) {
  /* තිරය 600px හෝ ඊට වඩා කුඩා වූ විට ක්‍රියාත්මක වන CSS */
  body {
    background-color: lightblue;
  }
}
```

- **@media**: මෙය මාධ්‍ය විමසුමක් බව පවසයි.
- **screen**: මෙය තිරයක් සහිත උපාංගයක් බව පවසයි.
- **max-width: 600px**: "උපරිම පළල 600px" යන්නයි. එනම් 0 සිට 600px දක්වා තිර සඳහා මෙය බලපායි.

23.5 Breakpoints (බිඳීමේ ලක්ෂ්‍යයන්)

වෙබ් අඩවියක සැකැස්ම (**Layout**) වෙනස් විය යුතු නිශ්චිත තිර ප්‍රමාණයන් **Breakpoints** ලෙස හැඳින්වේ. සාමාන්‍යයෙන් ලෝක සම්මත තිර ප්‍රමාණයන් කිහිපයක් මෙසේය:

- **480px** සහ ඊට අඩු: ජංගම දුරකථන (**Phones**)
- **481px** සිට **726px** දක්වා: ටැබ්ලට් (**Tablets / iPads**)
- **769px** සිට **1024px** දක්වා: ලැප්ටොප් (**Laptops**)
- **1025px** සහ ඊට වැඩි: විශාල තිර (**Desktops**)

23.6 Mobile-First ප්‍රවේශය (Mobile-First Approach)

වෘත්තීය වෙබ් සංවර්ධකයන් අනුගමනය කරන හොඳම ක්‍රමය මෙයයි.

1. මුලින්ම සාමාන්‍ය **CSS** ලෙස ජංගම දුරකථනයට ගැලපෙන කේත ලියන්න.
 2. ඉන්පසු **Media Queries** භාවිතා කර තිරය විශාල වන විට (**min-width** භාවිතා කර) අමතර වෙනස්කම් සිදු කරන්න.
- උදාහරණය:

```
/* 1. ජංගම දුරකථන සඳහා (සම්මත) */
.container {
  display: flex;
  flex-direction: column; /* කොටු එක යට එක */
}
```

```
/* 2. තිරය 726px ට වඩා වැඩි වන විට (Tablet/Desktop) */
@media screen and (min-width: 726px) {
  .container {
    flex-direction: row; /* කොටු එක පෙළට (හරහට) */
  }
}
```

23.7 නම්‍යශීලී ඒකක (Relative Units)

ප්‍රතිචාරාත්මක වෙබ් අඩවියකදී **px (Pixels)** වැනි ස්ථාවර ඒකක වෙනුවට නම්‍යශීලී ඒකක භාවිතා කිරීම ඉතා වැදගත් වේ.

- **% (Percentage):** දෙමාපිය ටැගයේ පළලට සාපේක්ෂව ප්‍රතිශතයක් ලෙස.
- **vw (Viewport Width):** තිරයේ පළලෙන් 1% ක්. (100vw යනු මුළු පළලයි).
- **vh (Viewport Height):** තිරයේ උසින් 1% ක්.
- **rem / em:** අකුරු වල ප්‍රමාණයට සාපේක්ෂව.

උදාහරණය: **width: 500px;** වෙනුවට **width: 90%;** භාවිතා කිරීමෙන් ඕනෑම තිරයකට ගැලපෙන සේ පළල වෙනස් වේ.

23.8 ASCII රූපසටහන: Responsive Layout වෙනස් වන ආකාරය

Desktop (තිරය 1024px+)

LOGO	MENU
CONTENT (70%)	SIDEBAR (30%)

Mobile (තිරය 480px-)

LOGO
MENU
CONTENT
SIDEBAR

23.9 පින්තූර ප්‍රතිචාරාත්මක කිරීම (Responsive Images)

පින්තූරයක් තිරයෙන් පිටතට පැනීම වැළැක්වීමට මෙම CSS ඡේදය ඉතා වැදගත් වේ:

```
img {
  max-width: 100%;
  height: auto;
}
```

මෙහි තේරුම: "පින්තූරයේ උපරිම පළල එහි දෙමාපිය ටැගයට වඩා වැඩි නොවන්න, උස ස්වයංක්‍රීයව සකසා ගන්න."

23.10 පොදු වැරදි (Common Mistakes)

1. **Viewport Meta Tag** එක අමතක වීම: **Media Queries** නිවැරදිව ලිවීමේදී මෙය නැතිනම් ජංගම දුරකථනයකදී එය ක්‍රියා නොකරයි.
2. ස්ථාවර පළල (**Fixed Width**) භාවිතා කිරීම: **width: 1000px;** ලෙස ලිවීමෙන් වෙබ් අඩවිය කුඩා තිර වලදී "කැඩී" යයි. සැමවිටම **max-width** හෝ **%** භාවිතා කරන්න.
3. අකුරු කුඩා වීම: දුරකථනයකදී අකුරු කියවීමට හැකි අවම ප්‍රමාණය **16px** පමණ වේ. **Media Queries** හරහා අකුරු වල ප්‍රමාණය පාලනය කරන්න.

23.11 වෘත්තීය උපදෙස් (Professional Tips)

- **Chrome DevTools** භාවිතා කරන්න: බ්‍රවුසරයේ **Inspect Element (F12)** විවෘත කර, **"Device Toggle"** අයිකනය (දුරකථනයක රූපයක් සහිත) ක්ලික් කරන්න. එවිට ඔබට විවිධ දුරකථන වල ඔබේ වෙබ් අඩවිය පෙනෙන ආකාරය පරීක්ෂා කළ හැක.
- **Content is King:** සැකැස්ම වෙනස් කළද, තොරතුරු (**Content**) කියවීමට ඇති හැකියාවට හානි නොවිය යුතුය.

23.12 අභ්‍යාස (Exercises)

1. **Responsive Web Design** යන්නෙහි අවශ්‍යතාවය සරලව පැහැදිලි කරන්න.
2. **Viewport Meta Tag** එක මගින් සිදුකරන ප්‍රධාන කාර්යයන් දෙක කුමක්ද?
3. තිරයක පළල **1024px** සහ **726px** අතර පවතින විට පමණක් ක්‍රියාත්මක වන **Media Query** එකක් ලියන්න.
4. **width: 100%;** සහ **max-width: 100%;** අතර ඇති වෙනස කුමක්ද?

23.13 ප්‍රායෝගික ව්‍යාපෘතිය (Mini Project)

ව්‍යාපෘතිය: නම්‍යශීලී තොරතුරු කාඩ්පත් (**Responsive Cards**)

1. **HTML** වල **div.container** එකක් සාදා ඒ තුළ **div.card** 3ක් සාදන්න.
2. **CSS Flexbox** භාවිතා කර:
 - පරිගණක තිර වලදී කාඩ්පත් 3 එක ඡේද්‍රීයක (**Horizontal**) පෙන්වන්න.

- ටැබ්ලට් තිර වලදී කාඩ්පත් 2ක් එක් ජේළියකත්, අනෙක ඊළඟ ජේළියෙන් පෙන්වන්න.
- ජංගම දුරකථන තිර වලදී කාඩ්පත් 3ම එක යට එක (Vertical) පෙන්වන්න.

3. Media Queries භාවිතා කර එක් එක් අවස්ථාවේදී පසුබිම් වර්ණය වෙනස් කරන්න.

මිළඟ පරිච්ඡේදයෙන් අපි CSS වල ඇති සජීවීකරණ (Animations) සහ මෘදු ලෙස වර්ණ/හැඩ වෙනස් කරන ආකාරය (Transitions) ගැන ඉතාමත් විස්තරාත්මකව ඉගෙන ගනිමු.

විසිහනරවන පරිච්ඡේදය: සී.එස්.එස්. විචල්‍යයන් සහ හොඳම පුරුදු (CSS Variables & Best Practices)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

1. CSS විචල්‍යයන් (CSS Variables / Custom Properties) යනු කුමක්දැයි තේරුම් ගැනීම.
2. එකම අගය තැන් කිහිපයක භාවිතා වන විට විචල්‍යයන් භාවිතා කිරීමේ වාසි වටහා ගැනීම.
3. :root සලකුණ භාවිතයෙන් ගෝලීය විචල්‍යයන් (Global Variables) සාදන ආකාරය.
4. කේතය පිරිසිදුව තබා ගැනීමට උපකාරී වන BEM නාමකරණ ක්‍රමය (Naming Convention) ඉගෙන ගැනීම.
5. වෘත්තීය වෙබ් සංවර්ධකයෙකු අනුගමනය කළ යුතු හොඳම පුරුදු (Best Practices) හඳුනා ගැනීම.

24.1 හැඳින්වීම (Introduction)

ඔබ විශාල වෙබ් අඩවියක් නිර්මාණය කරන විට, එහි එකම වර්ණය (උදා: තද නිල් පාට) තැන් 50ක පමණ භාවිතා කිරීමට සිදු විය හැක. සිතන්න, පසුව ඔබේ සේවාදායකයා (Client) පවසනවා එම නිල් පාට වෙනුවට රතු පාට භාවිතා කරන ලෙස. දැන් ඔබට සිදු වන්නේ CSS ගොනුව පුරාම ගොස් එම ස්ථාන 50ම එකින් එක සොයා වෙනස් කිරීමටයි. මෙය ඉතා වෙහෙසකර මෙන්ම වැරදි සිදු වීමට ඉඩ ඇති වැඩකි.

මෙම ගැටලුවට විසඳුම ලෙස CSS Variables හඳුන්වා දී ඇත. මෙහිදී අප කරන්නේ යම් වර්ණයක් හෝ අගයක් එක් විශේෂ නමක ගබඩා කර තැබීමයි. පසුව අවශ්‍ය තැන්වලදී එම නම භාවිතා කරයි. වර්ණය වෙනස් කිරීමට අවශ්‍ය වූ විට, එම නම ගබඩා කර ඇති ස්ථානයේ පමණක් වෙනස් කිරීම ප්‍රමාණවත් වේ.

24.2 CSS විචල්‍යයක් යනු කුමක්ද? (What is a Variable?)

Variable (විචල්‍යය) යනු යම් අගයක් (වර්ණයක්, ප්‍රමාණයක්, අකුරු වර්ගයක්) තැන්පත් කර තැබිය හැකි "පෙට්ටියක්" වැනිය.

සැබෑ ලෝකයේ උදාහරණයක්: සිතන්න ඔබ නිවසක් සායම් කරනවා කියා. ඔබ විශාල බාල්දියක විශේෂිත නිල් වර්ණයක් මිශ්‍ර කර ගන්නවා. දැන් ඔබ නිවසේ සෑම කාමරයකටම සායම් ගන්නේ එම බාල්දියෙන්. ඔබට වර්ණය මදක් වෙනස් කිරීමට අවශ්‍ය නම්, ඔබ කරන්නේ බාල්දියේ ඇති සායම් වලට වෙනත් වර්ණයක් එක් කිරීමයි. එවිට ස්වයංක්‍රීයවම ඔබ එම බාල්දියෙන් සායම් ගෑ සෑම බිත්තියකම වර්ණය වෙනස් වේ.

මෙහිදී "සායම් බාල්දිය" යනු විචල්‍යයයි (Variable).

24.3 විචල්‍යයන් භාවිතා කරන ආකාරය (Syntax)

CSS විචල්‍යයක් සෑදීමේදී පියවර දෙකක් ඇත:

1. විචල්‍යය ප්‍රකාශ කිරීම (Declaring the variable).
2. විචල්‍යය භාවිතා කිරීම (Using the variable).

24.3.1 විචල්‍යය ප්‍රකාශ කිරීම (:root)

සාමාන්‍යයෙන් අප විචල්‍යයන් ලියන්නේ CSS ගොනුවේ ඉහළින්ම ඇති :root යන කොටස තුළයි. මෙයට ලියන දේ මුළු වෙබ් අඩවියටම බලපායි.

- විචල්‍ය නාමයක් සැමවිටම පටන් ගත යුත්තේ ඉරි කැලි දෙකකින් (--) ය.

```
:root {
  --primary-color: #3498db; /* ප්‍රධාන වර්ණය */
  --secondary-color: #2ecc71; /* දෙවන වර්ණය */
  --main-padding: 20px; /* පොදු පැඩිං ප්‍රමාණය */
}
```

24.3.2 විචල්‍යය භාවිතා කිරීම (var())

විචල්‍යයක ඇති අගය ලබා ගැනීමට var() යන ශ්‍රිතය (Function) භාවිතා කරයි.

```
h1 {
  color: var(--primary-color); /* මෙහිදී #3498db අගය ලැබෙනු ඇත */
}
```

```
.box {
  padding: var(--main-padding);
  background-color: var(--secondary-color);
}
```

24.4 කේතය සංවිධානය කිරීමේ හොඳම පුරුදු (Best Practices)

වෘත්තීය මට්ටමේ වෙබ් සංවර්ධකයෙකු වීමට නම්, ඔබ ලියන කේතය වෙනත් අයෙකුට කියවීමට සහ තේරුම් ගැනීමට පහසු විය යුතුය. ඒ සඳහා පහත ක්‍රම අනුගමනය කරන්න.

1. BEM නාමකරණ ක්‍රමය (Block Element Modifier)

Class වලට නම් තැබීමේදී ලෝකයේ පිළිගත් ක්‍රමය මෙයයි. මෙය කේතය අවුල් වීම වළක්වයි.

- **Block** (බ්ලොක්): ස්වාධීන කොටස (උදා: **card**)
- **Element** (එලිමන්ට්): බ්ලොක් එක ඇතුළත ඇති කුඩා කොටස. මෙය ලියන්නේ යට ඉරි දෙකක් (__) මගිනි. (උදා: **card__title**)
- **Modifier** (මොඩිෆයර්): එම කොටසේ වෙනස්කම්. මෙය ලියන්නේ ඉරි කැලි දෙකක් (--) මගිනි. (උදා: **card--dark**)

උදාහරණ:

```
<div class="card card--dark">
  <h2 class="card__title">මාතෘකාව</h2>
  <button class="card__button">මිලදී
ගන්න</button>
</div>
```

2. DRY සංකල්පය (Don't Repeat Yourself)

"ඔබම නැවත සිදු නොකරන්න." එකම CSS කේතය නැවත නැවත ලිවීමෙන් වළකින්න. පොදු හැඩතල (Styles) සඳහා එක් **Class** එකක් සාදා එය අවශ්‍ය තැන්වලදී භාවිතා කරන්න.

3. අදහස් දැක්වීම (Comments)

ඔබ කේතය ලිවීමේ ඇයි දැයි පැහැදිලි කිරීමට **Comments** භාවිතා කරන්න.

```
/* මෙය වෙබ් අඩවියේ ප්‍රධාන මෙනුව සඳහා වන
CSS වේ */
.navbar { ... }
```

4. දේපල පෙළගැස්වීම (Property Ordering)

ගුණාංග (Properties) සැමවිටම එකම පිළිවෙළකට ලිවීමට උත්සාහ කරන්න. (උදා: මූලික Position, පසුව Box Model (Width/Height/Padding), අවසානයේ Typography).

24.5 ASCII සටහන: විචල්‍යයන්ගේ ප්‍රයෝජනය

සාමාන්‍ය ක්‍රමය (විචල්‍ය නැත):

```
Header { color: blue; } <-- වෙනස් කරන්න ඕනේ
Button { color: blue; } <-- වෙනස් කරන්න ඕනේ
Footer { color: blue; } <-- වෙනස් කරන්න ඕනේ
(තැන් 3කම වෙනස් කිරීමට සිදු වේ)
```

විචල්‍ය භාවිතා කරන ක්‍රමය:

```
:root { --my-color: blue; } <-- මෙතැන පමණක්
වෙනස් කිරීම ප්‍රමාණවත්!
```

```
Header { color: var(--my-color); }
Button { color: var(--my-color); }
Footer { color: var(--my-color); }
```

24.6 වෘත්තීය උපදෙස් (Professional Tips)

- **Dark Mode** සෑදීම: CSS Variables භාවිතා කිරීමෙන් වෙබ් අඩවියකට "Dark Mode" එකක් එක් කිරීම ඉතා පහසුය. ඔබ කළ යුත්තේ **:root** තුළ ඇති වර්ණ විචල්‍යයන්ගේ අගයන් පමණක් වෙනස් කිරීමයි.

- **Fallback Values:** සමහර විට විචල්‍යය ක්‍රියා නොකළහොත් භාවිතා කිරීමට අතිරේක අගයක් (Fallback) ලබා දිය හැක: **var(--my-color, black);**

24.7 පොදු වැරදි (Common Mistakes)

1. ඉරි කැලි අමතක වීම: විචල්‍යයේ නම ඉරිරියට -- යෙදීමට අමතක කිරීම.

2. **Case Sensitivity: --Main-Color** සහ **--main-color** යනු විචල්‍යයන් දෙකකි. සැමවිටම **simple** අකුරු භාවිතා කිරීම වඩාත් සුදුසුයි.

3. **Scope** එක වැරදීම: ඔබ විචල්‍යය **.box** එකක් ඇතුළත ලිවුවොත්, එය එම පෙට්ටියෙන් පිටතදී භාවිතා කළ නොහැක. ඒ නිසා සැමවිටම පොදු විචල්‍යයන් **:root** තුළ ලියන්න.

24.8 සාරාංශය (Key Points)

- CSS Variables මගින් කේතය නඩත්තු කිරීම (Maintain) ඉතා පහසු කරයි.
- :root යනු විචල්‍යයන් ගබඩා කිරීමට ඇති ප්‍රධාන ස්ථානයයි.
- -- මගින් විචල්‍යය සාදන අතර var() මගින් එය භාවිතා කරයි.
- BEM ක්‍රමය මගින් කේතය පිරිසිදුව තබා ගනී.

24.9 අභ්‍යාස (Exercises)

1. CSS විචල්‍යයන් භාවිතා කිරීමේ ප්‍රධාන වාසි 2ක් ලියන්න.
2. BEM ක්‍රමය භාවිතා කරමින් "පුවත්පත් ලිපියක්" (Article) සඳහා අවශ්‍ය HTML Class නම් සාදන්න. (ලිපිය, ලිපියේ රූපය, ලිපියේ මාතෘකාව).
3. පහත දැක්වෙන CSS කේතය විචල්‍යයන් භාවිතා කර ප්‍රතිනිර්මාණය කරන්න:


```
.header { background-color: #ff5733; padding: 15px; }
.footer { background-color: #ff5733; padding: 15px; }
```

24.10 ප්‍රායෝගික ව්‍යාපෘතිය (Mini Project)

ව්‍යාපෘතිය: වර්ණ තේමා මාරු කරන වෙබ් පිටුව (Theming Project)

1. HTML වල සරල පිටුවක් සාදන්න (මාතෘකාවක්, ඡේදයක් සහ බොත්තමක් සහිතව).
2. CSS හි :root තුළ --bg-color, --text-color, සහ --btn-color ලෙස විචල්‍යයන් 3ක් සාදන්න.
3. එම විචල්‍යයන් භාවිතා කර පිටුව හැඩගන්වන්න.
4. දැන්, :root හි ඇති අගයන් වෙනස් කර මුළු පිටුවේම වර්ණ තේමාව (Theme) තනි පියවරකින් වෙනස් වන ආකාරය අත්හදා බලන්න.

මිලහ පරිච්ඡේදයෙන් අපි පරිමාව 2 හි අවසාන කොටස ලෙස, මෙතෙක් ඉගෙන ගත් සියලුම CSS දැනුම භාවිතා කර වෘත්තීය මට්ටමේ "පෝට්ෆෝලියෝ වෙබ් අඩවියක්" (Professional Portfolio) මුල සිට අගට නිර්මාණය කරන ආකාරය ඉගෙන ගනිමු.

විසිපස්වන පරිච්ඡේදය: වෘත්තීය පෝට්ෆෝලියෝ ව්‍යාපෘතිය (The Professional Portfolio Project)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

1. මෙතෙක් උගත් සියලුම CSS සංකල්ප (Box Model, Flexbox, Grid, Positioning, Responsive Design, Animations, Variables) එක් කර සම්පූර්ණ ව්‍යාපෘතියක් නිර්මාණය කිරීම.
2. වෙබ් අඩවියක් නිර්මාණය කිරීමට පෙර සැලසුම් කරන ආකාරය (Planning).
3. වෘත්තීය මට්ටමේ CSS ගොනුවක් සංවිධානය කරන ආකාරය.
4. සැබෑ ලෝකයේ භාවිත කළ හැකි අලංකාර සහ ප්‍රතිචාරාත්මක (Responsive) වෙබ් අඩවියක් ප්‍රසිද්ධ කිරීමට සූදානම් කිරීම.

25.1 හැඳින්වීම (Introduction)

පරිමාව 2 හි අවසානයට ඔබ පැමිණ ඇත. ඔබ දැන් CSS පිළිබඳව විශාල දැනුමක් ලබා සිටියි. නමුත් වෘත්තීය Frontend Developer කෙනෙකු වන්නේ එම දැනුම භාවිතා කර සැබෑ නිර්මාණයක් කළ විට පමණි.

පෝට්ෆෝලියෝ (Portfolio) වෙබ් අඩවියක් යනු ඔබ කවුද, ඔබ දන්නා දේ කුමක්ද සහ ඔබ කළ නිර්මාණ මොනවාදැයි ලෝකයට පෙන්වන කැඩපතයි. ඔබ රැකියාවකට හෝ Freelancing වැඩකට අයදුම් කරන විට, සේවාදායකයා මුලින්ම බලන්නේ ඔබේ මෙම වෙබ් අඩවියයි.

අපි මෙම ව්‍යාපෘතිය පියවරෙන් පියවර ඉතාමත් සෙමින් නිර්මාණය කරමු.

25.2 පියවර 1: ව්‍යාපෘතිය සැලසුම් කිරීම (The Plan)

අපගේ පෝට්ෆෝලියෝ වෙබ් අඩවියේ ප්‍රධාන කොටස් 5ක් තිබිය යුතුය:

1. Navigation Bar (මෙනුව): වෙබ් අඩවියේ ලාංඡනය සහ ලින්ක්ස්. (Flexbox භාවිතා කරමු).

2. **Hero Section** (හඳුන්වාදීමේ කොටස): ඔබේ නම සහ රූපය. (**Positioning** සහ **Animations** භාවිතා කරමු).

3. **Skills Section** (දක්ෂතා): ඔබ දන්නා තාක්ෂණයන්. (**CSS Grid** භාවිතා කරමු).

4. **Projects Section** (නිර්මාණ): ඔබ කළ වැඩ පෙන්වන කාඩ්පත්. (**Responsive Grid** භාවිතා කරමු).

5. **Footer** (පහළ කොටස): සම්බන්ධ කර ගත හැකි තොරතුරු.

25.3 පියවර 2: ගොනු ව්‍යුහය සහ විචල්‍යයන් (Setup & Variables)

මූලිකව ඔබේ **CSS** ගොනුවේ වර්ණ සහ පොදු අගයන් විචල්‍යයන් (**Variables**) ලෙස සකස් කර ගන්න. මෙය පසුව වර්ණ වෙනස් කිරීමට ඉතා පහසු වේ.

style.css:

```
/* 1. විචල්‍යයන් නිර්වචනය කිරීම */
:root {
  --primary-color: #2563eb; /* තද නිල් */
  --secondary-color: #64748b; /* අළු පැහැය */
  --bg-color: #ffffff; /* සුදු පසුබිම */
  --text-color: #1e293b; /* තද අකුරු */
  --card-bg: #f8fafc; /* ලා අළු කාඩ්පත් */
  --transition: all 0.3s ease; /* පොදු සංක්‍රාන්තිය */
}
```

```
/* 2. මූලික සැකසුම් (Reset) */
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box; /* Box Model එක නිවැරදිව කටයුතු ගැනීමට */
  font-family: 'Segoe UI', sans-serif;
}
```

```
body {
  background-color: var(--bg-color);
  color: var(--text-color);
  line-height: 1.6;
}
```

```
/* Scroll කිරීම මෘදු කිරීමට */
html {
  scroll-behavior: smooth;
}
```

25.4 පියවර 3: මෙහෙයුම් නිර්මාණය (Navigation - Flexbox)

HTML:

```
<header class="navbar">
  <div class="logo">MyPortfolio</div>
  <nav>
    <ul class="nav-links">
      <li><a href="#home">Home</a></li>
      <li><a href="#skills">Skills</a></li>
      <li><a href="#projects">Projects</a></li>
    </ul>
  </nav>
</header>
```

CSS:

```
.navbar {
  display: flex;
  justify-content: space-between; /* දෙපැත්තට නල්ලු කිරීමට */
  align-items: center;
  padding: 20px 10%;
  background: white;
  box-shadow: 0 2px 10px rgba(0,0,0,0.1);
  position: sticky; /* ස්ක්‍රෝල් කරන විට උඩ ඇඳී තිබීමට */
  top: 0;
  z-index: 1000;
}
```

```
.nav-links {
  display: flex;
  list-style: none;
}
```

```
.nav-links li {
  margin-left: 30px;
}
```

```
.nav-links a {
  text-decoration: none;
  color: var(--text-color);
  font-weight: 600;
  transition: var(--transition);
}
```

```
.nav-links a:hover {
  color: var(--primary-color);
}
```

25.5 පියවර 4: හඳුන්වාදීමේ කොටස (Hero Section - Animation)

HTML:

```
<section id="home" class="hero">
  <div class="hero-content">
```

```

<h1 class="fade-in">ආයුබෝවන්, මම
<span>කසුන්</span></h1>
<p>මම වෘත්තීය Frontend Developer
කෙනෙක්.</p>
<a href="#projects" class="btn">මගේ වැඩ
බලන්න</a>
</div>
</section>

```

CSS:

```

.hero {
  height: 80vh; /* තිරයේ උසින් 80% ක් */
  display: flex;
  justify-content: center;
  align-items: center;
  text-align: center;
  background: linear-gradient(135deg,
    #f0f9ff 0%, #e0f2fe 100%);
}

.hero h1 {
  font-size: 3.5rem;
  margin-bottom: 1rem;
}

.hero span {
  color: var(--primary-color);
}

/* Animation එකක් එකතු කිරීම */
@keyframes fadeIn {
  from { opacity: 0; transform:
    translateY(20px); }
  to { opacity: 1; transform: translateY(0); }
}

.fade-in {
  animation: fadeIn 1.5s ease-out;
}

.btn {
  display: inline-block;
  padding: 12px 30px;
  background: var(--primary-color);
  color: white;
  text-decoration: none;
  border-radius: 5px;
  margin-top: 20px;
  transition: var(--transition);
}

.btn:hover {

```

```

  transform: scale(1.05); /* මගේ විශාල කිරීම */
  box-shadow: 0 5px 15px rgba(37, 99, 235,
    0.4);
}

```

25.6 පියවර 5: දක්ෂතා පෙන්නුම් (Skills Grid)

HTML:

```

<section id="skills" class="skills">
  <h2>මගේ දක්ෂතා</h2>
  <div class="skills-grid">
    <div class="skill-card">HTML5</div>
    <div class="skill-card">CSS3</div>
    <div class="skill-
      card">Flexbox/Grid</div>
    <div class="skill-card">Responsive
      Design</div>
    </div>
  </section>

```

CSS:

```

.skills {
  padding: 80px 10%;
  text-align: center;
}

.skills-grid {
  display: grid;
  /* තිරයේ ප්‍රමාණය අනුව තීරු ගණන ස්වයංක්‍රීයව
    සකස් වේ */
  grid-template-columns: repeat(auto-fit,
    minmax(150px, 1fr));
  gap: 20px;
  margin-top: 40px;
}

.skill-card {
  background: var(--card-bg);
  padding: 30px;
  border-radius: 10px;
  font-weight: bold;
  border-bottom: 4px solid var(--primary-
    color);
  transition: var(--transition);
}

.skill-card:hover {
  transform: translateY(-10px);
}

```

25.7 පියවර 6: ප්‍රතිචාරාත්මක බව (Mobile Responsive)

දැන් අපි ජංගම දුරකථනයෙන් බලන විට මෙනුව සහ අකුරු වෙනස් වන ආකාරය සකසමු.

CSS:

```
/* තිරය 726px (Tablet/Mobile) ට වඩා කුඩා වූ විට */
@media (max-width: 726px) {
  .hero h1 {
    font-size: 2.5rem;
  }

  .navbar {
    padding: 20px 5%;
  }

  .nav-links li {
    margin-left: 15px;
  }

  /* අකුරු ප්‍රමාණයන් මඳක් කුඩා කිරීම */
  html {
    font-size: 14px;
  }
}
```

25.8 වෘත්තීය උපදෙස් (Professional Tips)

- පින්තූර (Images): ඔබ පින්තූර භාවිතා කරනවා නම් ඒවායේ ප්‍රමාණය (File size) අඩු කිරීමට වග බලා ගන්න. (TinyPNG වැනි වෙබ් අඩවියක් භාවිතා කරන්න).
- ප්‍රවේශ්‍යතාව (Accessibility): අකුරු වල පාට සහ පසුබිම් පාට අතර හොඳ වෙනසක් (Contrast) පවත්වා ගන්න. (නම්‍යශීලී වෙබ් අඩවියක් සැමටම කියවිය හැකි විය යුතුය).
- කේත පරීක්ෂාව: සැමවිටම විවිධ බ්‍රවුසර වල (Chrome, Firefox, Safari) වෙබ් අඩවිය පරීක්ෂා කරන්න.

25.9 පොදු වැරදි (Common Mistakes)

1. **Over-animating:** ඕනෑම වඩා animations භාවිතා කිරීම නිසා වෙබ් අඩවිය "සෙල්ලම් බඩුවක්"

මෙන් පෙනෙන්නට පුළුවන. සජීවීකරණ ඉතා සියුම්ව භාවිතා කරන්න.

2. **Hardcoding units:** px භාවිතා කිරීම අඩු කරන්න. ඒ වෙනුවට %, rem, fr වැනි නම්‍යශීලී ඒකක භාවිතා කරන්න.

3. **Horizontal Scroll:** ජංගම දුරකථනයකදී වෙබ් අඩවිය පැත්තට ගමන් කිරීම (Horizontal scroll) ඉතා නරක ලක්ෂණයකි. මෙය සිදුවන්නේ යම් මූලිකාංගයක පළල තිරයට වඩා වැඩි වූ විටය. max-width: 100% භාවිතා කර මෙය වළක්වන්න.

25.10 අභ්‍යාස (Exercises)

1. පෝට්ෆෝලියෝ වෙබ් අඩවියේ පසුබිම "Dark Mode" කිරීමට අවශ්‍ය CSS විචල්‍යයන් නැවත ලියන්න.
2. **Skills Section** එකේ ඇති කාඩ්පත් මත මවුසය ගෙන ගිය විට ඒවායේ වර්ණය වෙනස් වන සේ CSS Transition එකක් එක් කරන්න.
3. අප සාදන ලද පෝට්ෆෝලියෝ එකට "Projects" නමින් අලුත් කොටසක් (Section) Grid භාවිතා කර නිර්මාණය කරන්න.

විසිභයවන පරිච්ඡේදය (විශේෂ ප්‍රෝ-මට්ටමේ): සී.එස්.එස්. ගෘහ නිර්මාණ ශිල්පය, පද්ධති නිර්මාණය සහ උසස් තාක්ෂණික රහස් (Advanced CSS Architecture, Design Systems & Engineering)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබ සාමාන්‍ය වෙබ් අඩවි නිර්මාණය කරන්නෙකුගේ මට්ටම සම්පූර්ණයෙන් ඉක්මවා ගොස් පහත කරුණු පිළිබඳ පූර්ණ ප්‍රවීණත්වයක් ලබා ගනු ඇත:

1. **CSS Architecture:** විශාල මෘදුකාංග සඳහා කේතය සංවිධානය කරන වෘත්තීය ක්‍රමවේද (BEM & ITCSS).
2. **Advanced Variables & Tokens:** වර්ණ සහ පරතරයන් නිර්මාණ පද්ධතියක් (Design System) ලෙස කළමනාකරණය කිරීම.

3. **Dark Mode Engineering: prefers-color-scheme** භාවිතා කර ස්වයංක්‍රීයව තේමාවන් මාරු කරන ආකාරය.

4. **Modern CSS Math: clamp(), calc(), සහ aspect-ratio** භාවිතා කර බුද්ධිමත් මූලිකාංග සැකසීම.

5. **Browser Rendering Internal: බ්‍රවුසරය CSS කියවන ආකාරය (Reflow & Repaint)** සහ එය වේගවත් කරන රහස්.

6. **Pseudo-element Mastery: HTML** කේතයෙන් තොරව CSS හරහා සංකීර්ණ සැරසිලි සහ අයිකන නිර්මාණය කිරීම.

1. බ්‍රවුසරයේ අභ්‍යන්තර ක්‍රියාවලිය (How the Browser Sees Your CSS)

ඔබ ලියන CSS කේතය නිරය මත රූපයක් බවට පත්වීමට පෙර බ්‍රවුසරය තුළ මහා පරිමාණ ඉංජිනේරු ක්‍රියාවලියක් සිදු වේ. මෙය තේරුම් ගැනීම වෘත්තීය මට්ටමේදී ඉතා වැදගත්ය.

1.1 CSSOM සහ Render Tree

බ්‍රවුසරය HTML කියවනවා සේම, ඔබේ CSS ගොනුව කියවා මතකය තුළ CSSOM (CSS Object Model) නමැති ව්‍යුහයක් සාදා ගනී. පසුව එය HTML (DOM) සහ CSS (CSSOM) එකිනෙක මත තබා Render Tree එක සාදයි.

1.2 Reflow (පිරිසැලසුම් නැවත ගණනය කිරීම)

ඔබ මූලිකාංගයක ප්‍රමාණය (width) හෝ පිහිටීම (position) වෙනස් කළහොත්, බ්‍රවුසරයට සිදු වන්නේ මුළු වෙබ් පිටුවේම ඇති සියලුම පෙට්ටිවල පිහිටීම නැවත ගණනය කිරීමයි. මෙය පරිගණකයේ වේගය (CPU) විශාල ලෙස වැය කරන වැඩකි.

1.3 Repaint (නැවත වර්ණ ගැන්වීම)

වර්ණයක් (color) හෝ පසුබිමක් පමණක් වෙනස් වන විට බ්‍රවුසරය කරන්නේ අදාළ පින්සල් මත නැවත පාට ගැම පමණි. මෙය වේගවත්ය.

Pro-Tip: ඇනිමේෂන් (Animations) කරන විට සැමවිටම transform භාවිතා කරන්න. එය බ්‍රවුසරයේ Layout එකට බලපෑමක් නොකරන නිසා ඉතා වේගවත් වේ.

2. වෘත්තීය නාමකරණය: BEM හි ගැඹුරු භාවිතය

විශාල ව්‍යාපෘතිවලදී CSS නීති එකිනෙක පටලැවීම (Collision) වළක්වා ගැනීමට BEM (Block Element Modifier) අනිවාර්යයෙන්ම භාවිතා කළ යුතුය.

- **Block** (බ්ලොක්): ස්ථායී කොටස (උදා: card)
- **Element** (එලිමන්ට්): බ්ලොක් එක ඇතුළත ඇති කොටස. මෙය යට ඉරි දෙකක් (__) මගින් ලියයි (උදා: card__title).
- **Modifier** (මොඩිෆයර්): එම කොටසේම වෙනස් අවස්ථාවක්. මෙය මැද ඉරි දෙකක් (--) මගින් ලියයි (උදා: card--dark).

ප්‍රෝ-කේත උදාහරණය:

```
<article class="product">
  
  <h2 class="product__title">Travel Bag</h2>
  <button class="product__btn product__btn--featured">Buy Now</button>
</article>
```

3. නිර්මාණ පද්ධති සහ CSS විචල්‍යයන් (Design Tokens)

වෘත්තීය මට්ටමේදී අප color: blue ලෙස ලියන්නේ නැත. ඒ වෙනුවට අප Design Tokens භාවිතා කරමු. මෙහිදී මුලින්ම සියලුම අගයන් :root තුළ විචල්‍යයන් ලෙස තැන්පත් කරයි.

```
:root {
  /* 1. වර්ණ (Colors) */
  --clr-primary-400: #3498db;
  --clr-primary-700: #2980b9;
  --clr-neutral-100: #ffffff;
  --clr-neutral-900: #1a1a1a;

  /* 2. පරතරයන් (Spacing) */
  --space-sm: 0.5rem;
  --space-md: 1.5rem;
  --space-lg: 3rem;

  /* 3. සෙවනැලි (Shadows) */
  --shadow-main: 0 10px 15px -3px rgba(0, 0, 0, 0.1);
}
```

4. ප්‍රෝ රහස: Dark Mode නිර්මාණය කිරීම (The Modern Way)

දැන් අපි බලමු පරිශීලකයාගේ පරිගණකයේ ඇති **Settings** වලට අනුව ස්වයංක්‍රීයව වෙබ් අඩවිය **Dark Mode** බවට පත් කරන්නේ කෙසේද කියා. මෙයට අපට අමතර **JavaScript** අවශ්‍ය නොවේ.

පියවර 1: විචල්‍යයන් සාදා ගැනීම

```
:root {
  /* සම්මත තේමාව (Light Mode) */
  --bg-page: var(--clr-neutral-100);
  --text-main: var(--clr-neutral-900);
  --accent: var(--clr-primary-400);
}
```

පියවර 2: Media Query එක භාවිතා කිරීම

```
@media (prefers-color-scheme: dark) {
  :root {
    /* මෙහිදී විචල්‍යයන්ගේ අගයන් පමණක් මාරු කරන්න */
    --bg-page: var(--clr-neutral-900);
    --text-main: var(--clr-neutral-100);
    --accent: #f1c40f; /* අඳුරු පසුබිමට ගැලපෙන කහ පැහැයක් */
  }
}
```

```
/* පියවර 3: විචල්‍යයන් භාවිතා කර හැඩගැන්වීම */
body {
  background-color: var(--bg-page);
  color: var(--text-main);
  transition: background-color 0.5s ease; /* මෘදු ලෙස මාරු වීමට */
}
```

පැහැදිලි කිරීම: මෙහිදී ඔබ **HTML** එකේ කිසිදු වෙනසක් කරන්නේ නැත. ඔවුසරය විසින් පරිශීලකයා "Dark Mode" කැමති බව හඳුනා ගත් සැණින්, **:root** හි ඇති අගයන් වෙනස් කරයි.

5. බුද්ධිමත් පිරිසැලසුම් සහ CSS Math

ස්ථාවර **px** අගයන් භාවිතා කිරීම වෙනුවට ඔවුසරයට ගණනය කිරීම් කිරීමට ඉඩ දෙන්න.

5.1 clamp() - පිරිසැලසුම් වල මැෂික් එක

මෙම ශ්‍රිතය මගින් අගයන් 3ක් පාලනය කරයි: (අවම අගය, කැමති අගය, උපරිම අගය).

```
h1 {
  /* අකුරු 2rem ට වඩා කුඩා නොවේ, තිරයේ පළලින් 5% ක් ලෙස පවතී, 5rem ට වඩා විශාල නොවේ */
  font-size: clamp(2rem, 5vw + 1rem, 5rem);
}
```

මෙහි ඇති වාසිය නම්, ඔබට ජංගම දුරකථනයට සහ පරිගණකයට වෙන වෙනම **Media Queries** සිය ගණනක් ලිවීමට අවශ්‍ය නොවීමයි.

5.2 aspect-ratio

පින්තූර හෝ වීඩියෝ පෙට්ටි සැමවිටම නිවැරදි හැඩයෙන් තබා ගැනීමට මෙය භාවිතා කරයි.

```
.card-img {
  width: 100%;
  aspect-ratio: 16 / 9; /* සැමවිටම 16:9 අනුපාතයට පවතී */
  object-fit: cover; /* පින්තූරය ඇද නොවී පෙනීමට */
}
```

6. ව්‍යාජ මූලිකාංග හරහා නිර්මාණකරණය (Advanced Pseudo-elements)

වෘත්තීය සංවර්ධකයන් **::before** සහ **::after** භාවිතා කරන්නේ අමුතු හැඩතල සෑදීමටය.

උදාහරණය: බොත්තමක් උඩින් මවුසය ගෙන ගිය විට එන සැරසිල්ලක් (**Overlay**)

```
.btn {
  position: relative;
  overflow: hidden; /* ඇතුළත ඇති දේ පිටතට නොපෙනීමට */
}
```

```
.btn::after {
  content: "";
  position: absolute;
  top: 0;
  left: -100%; /* බොත්තමෙන් පිටත වම් පැත්තේ ඇත */
  width: 100%;
  height: 100%;
  background-color: rgba(255, 255, 255, 0.2);
  transition: left 0.3s ease;
}
```



```
.btn:hover::after {
  left: 0; /* මවුසය ගෙන ගිය විට ඇතුළට පැමිණේ */
}
```

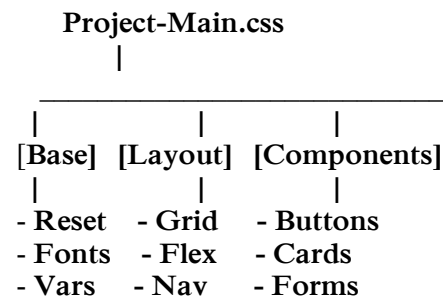
7. CSS Logical Properties (අනාගතය සඳහා සුදානම් වීම)

අද ලෝකයේ වෙබ් අඩවි විවිධ භාෂාවලට (උදා: අරාබි භාෂාව දකුණේ සිට වම්ට ලියයි) ගැලපිය යුතුය. වෘත්තීය මට්ටමේදී අප **margin-left** වෙනුවට **margin-inline-start** භාවිතා කරයි.

- **margin-inline-start**: ආරම්භක පැත්තෙන් ඉඩ තබන්න.
- **margin-inline-end**: අවසාන පැත්තෙන් ඉඩ තබන්න.

මෙසේ ලිවීමෙන් ඔබේ වෙබ් අඩවිය ලෝකයේ ඕනෑම භාෂාවකට ස්වයංක්‍රීයව හැඩ ගැසේ.

8. ASCII රූපසටහන: උසස් CSS Architecture



9. වෘත්තීය උපදෙස් සහ පිරිසිදු කේතනය (Clean Code)

- අනවශ්‍ය **!important** භාවිතා නොකරන්න: මෙය කේතය අවුල් කරන ප්‍රධාන සාධකයකි. ඒ වෙනුවට **Specificity** එක නිවැරදිව පාලනය කරන්න.
- **Dry Principle (Don't Repeat Yourself)**: එකම කේතය දෙපාරක් ලියනවා නම් එය විවලායක් හෝ **Utility Class** එකක් (උදා: **.flex-center**) බවට පත් කරන්න.
- **CSS Linters**: ඔබේ කේතයේ වැරදි සහ ප්‍රමිතිය පරීක්ෂා කරන මෘදුකාංග (උදා: **Stylelint**) භාවිතා කරන්න.

10. ප්‍රෝ-මට්ටමේ අභ්‍යාසය (Advanced Challenge)

අභියෝගය:

1. **CSS Variables** භාවිතා කරමින් **Light** සහ **Dark** තේමාවන් දෙකක් සාදන්න.
2. **clamp()** ශ්‍රිතය භාවිතා කරමින් ජංගම දුරකථනයේදී කුඩා වන, පරිගණකයේදී විශාල වන **"Card Grid"** එකක් නිර්මාණය කරන්න.
3. **::after** භාවිතා කරමින් බොත්තමකට **"Hover Effect"** එකක් ලබා දෙන්න.
4. ඔබේ වෙබ් අඩවිය ඇතුළත සියලුම **Margin** සහ **Padding** සඳහා **Logical Properties (inline-start, block-start)** භාවිතා කිරීමට උත්සාහ කරන්න.

දායා ප්‍රයෝග, උසස් තේරීම්කාරක සහ රූප පාලනය (Visual Effects, Advanced Selectors, and Image Mastery)

ඉගෙනුම් අරමුණු (Learning Objectives) මෙම අතිශය වැදගත් පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව පූර්ණ වෘත්තීය අවබෝධයක් ලැබෙනු ඇත:

1. **Shadows (සෙවනැලි)**: වෙබ් අඩවියක ඇති කොටස්වලට සහ අකුරුවලට සැබෑ පෙනුමක් (**3D Look**) ලබා දෙන ආකාරය සහ එහි ගණිතමය ව්‍යුහය.
2. **CSS Filters: රූප (Images) වලට** මෘදුකාංගයක් මගින් මෙන් විවිධ ප්‍රයෝග (Effects) ලබා දෙන ආකාරය.
3. **Backdrop-filter**: නවීන වෙබ් නිර්මාණකරණයේ ඉතා ජනප්‍රිය **"විදුරු පෙනුම" (Glassmorphism)** නිර්මාණය කිරීමේ රහස්.
4. **Object-fit & Object-position**: ඕනෑම ප්‍රමාණයක රූපයක්, පෙට්ටියක් ඇතුළත ඇද නොවී (**Stretching**) නිවැරදිව ප්‍රදර්ශනය කරන ආකාරය.
5. **Advanced Selectors: Attribute selectors** සහ **.not()** වැනි ඉතා බලවත් තේරීම්කාරක භාවිතා කර කේතය (**Code**) කෙටි කරන ආකාරය.
6. **Modern CSS Reset**: සියලුම වෙබ් බ්‍රවුසර (Browsers) තුළ වෙබ් අඩවිය එක හා සමානව පෙන්වීමට වෘත්තීය මට්ටමේ අත්තිවාරමක් සකසන ආකාරය.

27.1 හැඳින්වීම (Introduction)

පෙර පරිච්ඡේදවලදී ඔබ පිරිසැලසුම් (Layouts) සහ වර්ණ (Colors) භාවිතා කරන ආකාරය ඉගෙන ගත්තා. නමුත් වර්තමානයේ වෙබ් අඩවි හුදෙක් තොරතුරු පෙන්වන පිටු පමණක් නොවේ. ඒවා ඉතාමත් ආකර්ෂණීය (Attractive) විය යුතුය. වෙබ් අඩවියක් "Premium" හෝ "Professional" මට්ටමට ගෙන එන්නේ මෙම පරිච්ඡේදයේ අප සාකච්ඡා කරන කුඩා නමුත් ඉතාමත් වැදගත් තාක්ෂණික කරුණු මගිනි.

සිතන්න ඔබ Apple හෝ Microsoft සමාගමේ වෙබ් අඩවියකට පිවිසෙනවා කියා. එහි ඇති මෙනු (Menu) සහ පින්තූර (Images) ඉතාමත් සියුම් ලෙස සකස් කර ඇති ආකාරය ඔබට පෙනෙනු ඇත. මෙම පරිච්ඡේදයෙන් අපි ඉගෙන ගන්නේ එම "වෘත්තීය නිමාව" (Professional Finish) ලබා දෙන ආකාරයයි.

27.2 සෙවනැලි භාවිතය (CSS Shadows)

සෙවනැලි භාවිතා කරන්නේ වෙබ් පිටුවක ඇති සමතල පෙනුම (Flat design) ඉවත් කර එයට යම් ගැඹුරක් (Depth) ලබා දීමටයි. මෙය පරිශීලකයාට (User) හැඟෙන්නේ යම් කොටසක් තිරයෙන් මදක් ඉදිරියට නෙරා ඇති ආකාරයටයි.

27.2.1 Box Shadow (මූලිකාංග සඳහා සෙවනැලි)

ඕනෑම HTML පෙට්ටියක් (Box) වටා සෙවනැලි ලක් ඇති කිරීමට box-shadow ගුණාංගය (Property) භාවිතා කරයි.

ව්‍යුහය (Syntax): box-shadow: [h-offset] [v-offset] [blur] [spread] [color];

1. **h-offset** (තිරස් පරතරය): සෙවනැලි වම් පසට හෝ දකුණු පසට ගමන් කළ යුතු ප්‍රමාණය.

(ධන අගයක් දුන්නොත් දකුණටත්, සෘණ අගයක් දුන්නොත් වමටත් ගමන් කරයි).

2. **v-offset** (සිරස් පරතරය): සෙවනැලි ඉහළට හෝ පහළට ගමන් කළ යුතු ප්‍රමාණය. (ධන අගයක් දුන්නොත් පහළටත්, සෘණ අගයක් දුන්නොත් ඉහළටත් ගමන් කරයි).

3. **blur** (බොද වීම): සෙවනැලි කෙළවර කොපමණ ප්‍රමාණයකට බොද විය යුතුද යන්නයි. (මෙය වැඩි වන තරමට සෙවනැලි මෘදු ලෙස පෙනේ).

4. **spread** (පැතිරීම): සෙවනැලි කොපමණ විශාල විය යුතුද යන්නයි. (මෙය අනිවාර්ය නොවේ).

5. **color** (වර්ණය): සෙවනැලි වර්ණය. සාමාන්‍යයෙන් මේ සඳහා විනිවිද පෙනෙන කළු පැහැය

(**rgba(0,0,0,0.2)**) භාවිතා කරයි.

සම්පූර්ණ කේත උදාහරණය:

```
/* වෘත්තීය මට්ටමේ කාඩ්පතක සෙවනැලි ලක් */
.professional-card {
  width: 300px;
  height: 200px;
  background-color: white;
  /* තිරස් 0, සිරස් 10px, බොදවීම 20px, වර්ණය ලා කළු */
  box-shadow: 0px 10px 20px rgba(0, 0, 0, 0.15);
  border-radius: 12px; /* කොන් වටකුරු කිරීම */
}
```

27.2.2 Text Shadow (අකුරු සඳහා සෙවනැලි)

අකුරු පින්තූරයක් උඩින් ලියන විට හෝ ඒවා වඩාත් පැහැදිලිව පෙන්වීමට අවශ්‍ය වූ විට

text-shadow භාවිතා කරයි. මෙයට **spread** අගය නොමැති අතර ඉතිරි අගයන් **box-shadow** වලට සමාන වේ.

උදාහරණය:

```
h1 {
  color: white;
  /* තිරස් 2px, සිරස් 2px, බොදවීම 5px, කළු වර්ණය */
  text-shadow: 2px 2px 5px rgba(0, 0, 0, 0.7);
}
```

27.3 ෆිල්ටර්ස් සහ වීදුරු පෙනුම (Filters & Glassmorphism)

පින්තූර සහ පසුබිම් වෙනස් කිරීමට CSS විසින් අපට "Filters" (පෙරහන්) ලබා දෙයි.

27.3.1 Filter (රූප සඳහා ප්‍රයෝග)

පින්තූරයක් කළු-සුදු කිරීමට හෝ බොද කිරීමට මෙය ඉතා පහසු ක්‍රමයකි.

- **grayscale(100%)**: පින්තූරය සම්පූර්ණයෙන්ම කළු-සුදු කරයි.

- **blur(8px)**: පින්තූරය තදින් බොද කරයි.

- **brightness(150%)**: පින්තූරයේ ආලෝකය වැඩි කරයි.

```
.gallery-img {
  /* මවුසය ගෙන ගිය විට පින්තූරය කළු-සුදු කිරීමට */
  filter: grayscale(100%);
}
```

```

transition: 0.3s; /* මෘදු ලෙස සිදු වීමට */
}

.gallery-img:hover {
  filter: grayscale(0%); /* මවුසය උඩදී පාට පෙනේ */
}

```

27.3.2 Backdrop-filter (විදුරු පෙනුම හෙවත් Glassmorphism)

මෙය නවීන වෙබ් අඩවිවල දකින ඉතාමත් ආකර්ෂණීය ප්‍රයෝගයකි. මෙහිදී කරන්නේ මූලිකාංගයක (Element) පසුබිම නොව, එම මූලිකාංගයට පිටුපසින් ඇති දේ (Background) බොද කර පෙන්වීමයි.

ASCII සටහන: Glassmorphism ව්‍යුහය

```

[ පිටුපස ඇති රූපය (Background Image) ]
|
[ විදුරු පෙට්ටිය (Glass Box) - background: rgba(255,255,255,0.2) ]
[ Backdrop-filter: blur(10px) ]
|
[ පරිශීලකයාට පෙනෙන විදුරු වැනි පෙනුම ]

```

සැබෑ කේතය:

```

.glass-panel {
  background: rgba(255, 255, 255, 0.1); /* ඉතා ලා
පැහැති විනිවිද පෙනෙන පසුබිමක් */
  backdrop-filter: blur(15px); /* පිටුපස ඇති දේ
බොද කරයි */
  border: 1px solid rgba(255, 255, 255, 0.2); /* විදුරුවේ
කෙළවර පෙනීමට */
  border-radius: 20px;
  padding: 20px;
  color: white;
}

```

27.4 රූප පාලනය (Object-fit) - ඉතා වැදගත්!

වෘත්තීය මට්ටමේදී අපට විවිධ ප්‍රමාණයේ (Sizes) පින්තූර ලැබිය හැක. නමුත් අපේ වෙබ් අඩවියේ කාඩ්පත්වලට (Cards) ඒවා එකම ප්‍රමාණයකින් තිබිය යුතුය. පින්තූරයක පළල සහ උස වෙනස් කළ විට එය ඇද (Stretch) විමට හෝ මිරිකීමට (Squeeze) ඉඩ ඇත.

Object-fit මගින් පින්තූරයක් එහි වෙන් කළ ඉඩ ඇතුළත හැසිරිය යුතු ආකාරය තීරණය කරයි.

1. **fill** (සම්මත අගය): පින්තූරය පෙට්ටියට ගැලපීමට බලහත්කාරයෙන් ඇද දමයි. (පින්තූරය කැන වේ).

2. **cover**: පින්තූරය ඇද නොවී, එහි මැද කොටස පෙනෙන සේ පෙට්ටිය සම්පූර්ණයෙන්ම වසයි. (වඩාත් වෘත්තීය ක්‍රමයයි).

3. **contain**: පින්තූරය සම්පූර්ණයෙන්ම පෙන්වීමට පෙට්ටිය ඇතුළත කුඩා වේ. (පැතිවල හිස් ඉඩ ඉතිරි විය හැක).

උදාහරණය:

```

.user-profile-img {
  width: 200px;
  height: 200px;
  object-fit: cover; /* පින්තූරය ඇද නොවී ලස්සනට
පෙන්වේ */
  object-position: center; /* පින්තූරයේ මැද කොටස
පෙන්වීමට */
}

```

27.5 උසස් තේරීම්කාරක (Advanced Selectors)

අපට ඉතාමත් සංකීර්ණ ලෙස ටැග් තෝරා ගැනීමට මෙම ක්‍රමවේද උදව් වේ.

27.5.1 Attribute Selector (ගුණාංග මත පදනම්ව තේරීම)

යම් ටැගයක ඇති **Attribute** එකක් අනුව එය තෝරා ගැනීමට.

- **input[type="text"]**: Text input පමණක් තෝරා ගනී.
- **a[href\$=".pdf"]**: PDF ගොනුවකට ලින්ක් කර ඇති සබැඳි පමණක් තෝරා ගනී.

27.5.2 :not() Pseudo-class

යම් විශේෂ කොටසක් හැර අනෙක් සියල්ල තේරීමට.

```

/* .special class එක නැති සියලුම p ටැග් අළු පාට
කරන්න */
p:not(.special) {
  color: gray;
}

```

27.6 Modern CSS Reset (වෘත්තීය අත්තිවාරම)

සෑම වෙබ් බ්‍රවුසරයකටම (Chrome, Firefox, Safari) ඔවුන්ටම ආවේණික වූ කුඩා පරතරයන් (Default Margins) ඇත. මෙම පරතරයන් නිසා ඔබ සාදන වෙබ් අඩවිය එක් එක් බ්‍රවුසරයේදී

මදක් වෙනස් වී පෙනේ. වෘත්තීය මට්ටමේදී අපි මුලින්ම මේවා "සීරෝ" (Reset) කළ යුතුය.

ප්‍රෝ-මට්ටමේ රහස් **Reset** කේතය (Boilerplate):
මෙම කේතය ඔබ ලියන සෑම CSS ගොනුවකම ඉහළින්ම තිබිය යුතුය.

```
/* 1. සියලුම මූලිකාංගවල පරතරයන් ඉවත් කිරීම */
*, *::before, *::after {
  margin: 0;
  padding: 0;
  box-sizing: border-box; /* පැඩිං නිසා ප්‍රමාණය වෙනස් වීම වළක්වයි */
}
```

```
/* 2. රූප සහ වීඩියෝ පෙට්ටිවලින් පිටතට පැනීම වැළැක්වීම */
img, picture, svg, video {
  display: block;
  max-width: 100%;
}
```

```
/* 3. අකුරු වඩාත් මෘදු ලෙස පෙන්වීමට (Antialiasing) */
body {
  line-height: 1.5;
  -webkit-font-smoothing: antialiased;
}
```

27.7 පොදු වැරදි සහ විසඳුම් (Common Mistakes & Debugging)

- Backdrop-filter** වැඩ නොකිරීම: බොහෝ විට මෙය සිදු වන්නේ ඔබ එම මූලිකාංගයට තද වර්ණයක් (Solid color) ලබා දුන් විටයි. විදුරු ප්‍රයෝගය පෙනීමට නම් පසුබිම් වර්ණය විනිවිද පෙනෙන (Semi-transparent) එකක් විය යුතුය (උදා: `rgba(255,255,255,0.1)`).
- සෙවනැලි ඕනෑවට වඩා තද වීම: ආරම්භකයින් සෙවනැලිලට තද කළ පැහැය (black) ලබා දෙයි. එය වෙබ් අඩවිය අවලස්සන කරයි. සැමවිටම ඉතා ලා පැහැති `rgba` වර්ණ භාවිතා කරන්න.
- Object-fit** සහ **Width/Height**: පින්තූරයකට **width** සහ **height** යන දෙකම ලබා නොදුන්නොත් **object-fit** ක්‍රියා නොකරයි.

27.8 සාරාංශය (Key Points)

- **box-shadow** මගින් වෙබ් අඩවියට ගැඹුරක් (3D effect) ලබා දෙයි.
- **backdrop-filter: blur()** මගින් නවීන **Glassmorphism** පෙනුම නිර්මාණය කරයි.
- **object-fit: cover** භාවිතා කිරීමෙන් පින්තූර ඇඳ වීම වළක්වා ගත හැක.
- **CSS Reset** එකක් භාවිතා කිරීම ඔවුසර අතර ඇති වෙනස්කම් ඉවත් කිරීමට උදව් වේ.
- **Attribute selectors** සහ **:not()** මගින් කේතය වඩාත් කාර්යක්ෂම කරයි.

27.9 අභ්‍යාස (Exercises)

- අලංකාර සෙවනැල්ලක් සහිත "ප්‍රෝෆයිල් කාඩ්පතක්" (Profile Card) නිර්මාණය කරන්න.
- පින්තූරයක් ගෙන එයට මවුසය උඩට ගෙන යන විට (Hover) එය **Grayscale** වල සිට සාමාන්‍ය පාටට හැරෙන සේ **filter** භාවිතා කරන්න.
- විදුරු පෙනුම (Glassmorphism) සහිත මෙනුවක් (Navbar) සකස් කරන්න.
- input** කොටු සඳහා **Attribute selector** භාවිතා කරමින් **required** ලෙස ඇති කොටු වල මායිම (Border) රතු පාට කරන්න.

27.10 කුඩා ව්‍යාපෘතිය (Mini Project)

ව්‍යාපෘතිය: නවීන වෙබ් අඩවි මූලාංග පද්ධතියක් (UI Kit Component)

පහත කොටස් අඩංගු කුඩා වෙබ් පිටුවක් සාදන්න:

- ලස්සන පසුබිම් පින්තූරයක් මුළු පිටුවටම ලබා දෙන්න.
- එම පින්තූරය උඩින් "විදුරු" වැනි පෙනුමක් ඇති **div** එකක් සාදන්න.
- එම **div** එක ඇතුළත වටකුරු ප්‍රෝෆයිල් පින්තූරයක් තබන්න (**object-fit: cover** භාවිතා කරන්න).
- පින්තූරය යටින් ලස්සන **text-shadow** සහිතව ඔබේ නම ලියන්න.
- කාඩ්පත වටා ඉතා මෘදු සෙවනැල්ලක් (Shadow) ලබා දෙන්න.

සුභ පැතුම්! ඔබ දැන් "පරිමාව 2" සම්පූර්ණයෙන්ම අවසන් කර ඇත. දැන් ඔබ සතුව පිරිසැලසුම් සෑදීමට පමණක් නොව, ඒවා ලෝක මට්ටමේ (World-class) අලංකාර නිර්මාණ බවට පත් කිරීමට අවශ්‍ය සියලුම දැනුම තිබේ.

ජාවාස්ක්‍රිප්ට් මූලධර්ම (JavaScript Fundamentals)

විසිභත්වන පරිච්ඡේදය: ජාවාස්ක්‍රිප්ට් හැඳින්වීම සහ ඉතිහාසය (Introduction and History of JavaScript)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

1. ජාවාස්ක්‍රිප්ට් (JavaScript) යනු කුමක්දැයි වෘත්තීය මට්ටමින් තේරුම් ගැනීම.
2. වෙබ් අඩවියක ජාවාස්ක්‍රිප්ට් වල සැබෑ කාර්යභාරය (Role) හඳුනා ගැනීම.
3. ජාවාස්ක්‍රිප්ට් වල ඉතිහාසය සහ එය නිර්මාණය වූ ආකාරය පිළිබඳව දැනුම ලබා ගැනීම.
4. ජාවා සහ ජාවාස්ක්‍රිප්ට් (Java vs JavaScript) අතර ඇති වෙනස පැහැදිලිව වටහා ගැනීම.
5. බ්‍රවුසරයක් තුළ ජාවාස්ක්‍රිප්ට් ක්‍රියා කරවන "එන්ජින්" (Engine) යනු කුමක්දැයි ඉගෙන ගැනීම.
6. නූතන ලෝකයේ ජාවාස්ක්‍රිප්ට් භාවිතා වන විවිධ ක්ෂේත්‍ර පිළිබඳව අවබෝධයක් ලබා ගැනීම.

27.1 හැඳින්වීම (Introduction)

ඔබ මෙතෙක් ඉගෙන ගත් HTML සහ CSS භාවිතා කර ඔබට ලස්සන වෙබ් අඩවි නිර්මාණය කළ හැකිය.

නමුත් එම වෙබ් අඩවි "ස්ථාවර" (Static) ඒවා වේ. එනම්, එම වෙබ් පිටුවල අන්තර්ගතය වෙනස් වීමට නම් ඔබට නැවතත් කේතය (Code) වෙනස් කිරීමට සිදු වේ. පරිශීලකයා (User) කරන දේට ක්ෂණිකව ප්‍රතිචාර දැක්වීමට එම වෙබ් අඩවි වලට හැකියාවක් නැත.

වෙබ් අඩවියකට "බුද්ධිය" සහ "පණ" ලබා දෙන්නේ ජාවාස්ක්‍රිප්ට් මගිනි. ඔබ බොත්තමක් ක්ලික් කළ විට පණිවිඩයක් දිස්වීම, පින්තූර ස්ලයිඩරයක් මාරු වීම, හෝ වෙබ් පිටුව රිලෝඩ් (Reload) නොවී අලුත් දත්ත පෙන්වීම යන සියල්ල පිටුපස ඇත්තේ ජාවාස්ක්‍රිප්ටය.

27.2 ජාවාස්ක්‍රිප්ට් යනු කුමක්ද? (What is JavaScript?)

ජාවාස්ක්‍රිප්ට් (JavaScript) යනු වෙබ් අඩවි වලට අන්තර්ක්‍රියාකාරී බව (Interactivity) ලබා දීම සඳහා භාවිතා කරන ලොව ජනප්‍රියම ක්‍රමලේඛන භාෂාවයි (Programming Language).

සරල සිංහල පැහැදිලි කිරීම: HTML මගින් වෙබ් අඩවියේ ශරීරයන් (Structure), CSS මගින් එහි හැඩයන් (Design) නිර්මාණය කරන විට, ජාවාස්ක්‍රිප්ට් මගින් එම ශරීරයේ "හැසිරීම" (Behavior) තීරණය කරයි.

සැබෑ ලෝකයේ උපමාව (මෝටර් රථයක්):

- HTML: මෝටර් රථයේ ඇටසැකිල්ල, රෝද, දොරවල් සහ එන්ජින් තිබෙන තැන පෙන්වයි.
- CSS: රථයේ වර්ණය, සිටි වල ඇති ලස්සන සහ පෙනුම තීරණය කරයි.
- JavaScript: රථයේ එන්ජින් ක්‍රියා කරන ආකාරය, බ්‍රේක් අල්ලන ආකාරය සහ ලයිට් දැල්වීමේ ක්‍රියාවලිය පාලනය කරයි.

27.3 ජාවාස්ක්‍රිප්ට් ඉතිහාසය (The History of JavaScript)

ජාවාස්ක්‍රිප්ට් වල උපත ඉතා රසවත් කතාවකි. මෙය ලොව පවතින අනෙකුත් භාෂා වලට වඩා ඉතා වේගයෙන් නිර්මාණය වූවකි.

1. නිර්මාණය: 1995 වසරේදී බ්‍රෙන්ඩන් අයික් (Brendan Eich) නමැති දක්ෂ ඉංජිනේරුවා විසින් මෙය නිර්මාණය කරන ලදී. ඔහු එකල ප්‍රධානතම බ්‍රවුසරය වූ Netscape සමාගමේ සේවය කළේය.
2. දින 10ක ආශ්චර්යය: පුද්ගලයාට කරුණ නම්, බ්‍රෙන්ඩන් අයික් ජාවාස්ක්‍රිප්ට් වල පළමු සංස්කරණය නිර්මාණය කිරීමට ගත කළේ දින 10ක් පමණක් වීමයි!
3. නමේ වෙනස: මුලින්ම මෙයට Mocha ලෙස නම් තැබූ අතර, පසුව එය LiveScript ලෙස වෙනස් විය.
4. නම ජාවාස්ක්‍රිප්ට් වූ හැටි: එකල Java නමැති ක්‍රමලේඛන භාෂාව ලෝකයේ ඉතාමත් ජනප්‍රියව තිබුණි. Netscape සමාගමට අවශ්‍ය වූණා තමන්ගේ අලුත් භාෂාවටත් ඒ ජනප්‍රියත්වය ලබා ගැනීමටයි. ඒ නිසා ඔවුන් "Marketing" උපක්‍රමයක් ලෙස LiveScript යන නම JavaScript ලෙස වෙනස් කළා.

27.4 වැදගත්ම පැහැදිලි කිරීම: Java සහ JavaScript අතර වෙනස

බොහෝ දෙනෙක් සිතන්නේ Java සහ JavaScript යනු එකම භාෂාවක සංස්කරණ දෙකක් කියාය. නමුත් මෙය සම්පූර්ණයෙන්ම වැරදි මතයකි. සංසන්දනය: Java සහ JavaScript යනු සම්පූර්ණයෙන්ම වෙනස් භාෂා දෙකකි. ඒවායේ කිසිදු තාක්ෂණික සම්බන්ධයක් නැත.

- උදාහරණයක්: "කාර්" (Car) සහ "කාපට්" (Carpet) යන දෙකෙහිම "කාර්" යන අකුරු තිබුණද, ඒවා සම්පූර්ණයෙන්ම වෙනස් දේවල් දෙකක් වැනිය.

27.5 ඔවුසරය තුළ ජාවාස්ක්‍රිප්ට් ක්‍රියා කරන ආකාරය (The JavaScript Engine)

අප ලියන ජාවාස්ක්‍රිප්ට් කේතය (Code) පරිගණකයේ ප්‍රොසෙසරයට (Processor) කෙලින්ම තේරෙන්නේ නැත. එය තේරුම් කර දීමට ඔවුසරය තුළ විශේෂ මෘදුකාංගයක් ඇත. මෙයට JavaScript Engine (ජාවාස්ක්‍රිප්ට් එන්ජිම) යැයි කියනු ලැබේ.

ප්‍රධාන එන්ජිම කිහිපයක්:

- V8 Engine: Google Chrome සහ Microsoft Edge ඔවුසර වල ඇත. (මෙය ඉතා වේගවත්ය).
- SpiderMonkey: Firefox ඔවුසරයේ ඇත.
- JavaScriptCore: Apple Safari ඔවුසරයේ ඇත.

ඔබ ලියන කේතය මෙම එන්ජිම විසින් ඉතා වේගයෙන් කියවා, එය පරිගණකයට තේරෙන භාෂාවට (Machine Code) හරවා ක්‍රියාත්මක කරයි.

27.6 සැබෑ ලෝකයේ භාවිතය (Real-world Usage)

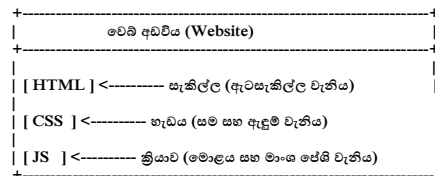
අද වන විට ජාවාස්ක්‍රිප්ට් භාවිතා නොවන තැනක් නැත:

1. Frontend Web Development: වෙබ් අඩවි වල පරිශීලකයාට පෙනෙන කොටස ක්‍රියාත්මක කිරීමට.
2. Backend Development: සේවාදායකයන් (Servers) නිර්මාණය කිරීමට (Node.js හරහා).

3. Mobile Apps: Android සහ iOS දුරකථන සඳහා මෘදුකාංග සෑදීමට.

4. Desktop Apps: පරිගණක මෘදුකාංග සෑදීමට (උදාහරණයක් ලෙස ඔබ භාවිතා කරන VS Code මෘදුකාංගය සාදා ඇත්තේ ද ජාවාස්ක්‍රිප්ට් වලිනි).

27.7 ASCII රූපසටහන: වෙබ් අඩවියක කාර්යභාරය



27.8 වෘත්තීය පැහැදිලි කිරීම (Professional Insight)

ජාවාස්ක්‍රිප්ට් යනු Interpreted (අර්ථකථනය කරන ලද) භාෂාවකි. මෙයින් අදහස් කරන්නේ කේතය ක්‍රියාත්මක කිරීමට පෙර වෙනත් භාෂාවකට පරිවර්තනය (Compile) කිරීමට අවශ්‍ය නොවන බවයි; ඔවුසරය විසින් එය එසැණින් කියවා ක්‍රියාත්මක කරයි. මෙය වෙබ් සංවර්ධකයෙකුට තම කේතයේ ප්‍රතිඵල වහාම දැක ගැනීමට උපකාරී වේ.

27.9 සාරාංශය (Key Points)

- ජාවාස්ක්‍රිප්ට් වෙබ් අඩවි වලට අන්තර්ක්‍රියාකාරී බව (Interactivity) ලබා දෙයි.
- එය 1995 දී බ්‍රොන්ඩන් අයිස් විසින් දින 10ක් ඇතුළත නිර්මාණය කරන ලදී.
- Java සහ JavaScript යනු සම්පූර්ණයෙන්ම වෙනස් භාෂා දෙකකි.
- සෑම ඔවුසරයකම ජාවාස්ක්‍රිප්ට් ක්‍රියාත්මක කිරීමට විශේෂ "එන්ජිමක්" ඇත.
- ජාවාස්ක්‍රිප්ට් යනු අද ලෝකයේ වැඩිපුරම භාවිතා වන ක්‍රමලේඛන භාෂාවයි.

27.10 අභ්‍යාස (Exercises)

1. වෙබ් අඩවියක් නිර්මාණය කිරීමේදී HTML, CSS සහ JavaScript වල කාර්යභාරයන් වෙන් වෙන්ව පැහැදිලි කරන්න.
2. ජාවාස්ක්‍රිප්ට් නිර්මාණය කළ පුද්ගලයා සහ සමාගම කවුද?
3. Java සහ JavaScript අතර ඇති ප්‍රධාන වෙනස කුමක්ද?

4. ඔබේ පරිගණකයේ ඔබ භාවිතා කරන බ්‍රවුසරය කුමක්ද? එහි ඇති ජාවාස්ක්‍රිප්ට් එන්ජින් කුමක් විය හැකිද?

විසිඳුවන පරිච්ඡේදය: වැඩබිම් සැකසීම සහ ප්‍රථම ජාවාස්ක්‍රිප්ට් කේතය (Environment Setup & Your First JavaScript Code)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

- ජාවාස්ක්‍රිප්ට් ලිවීමට අවශ්‍ය මෘදුකාංග පරිසරය සකසා ගැනීම.
- HTML ගොනුවකට ජාවාස්ක්‍රිප්ට් සම්බන්ධ කරන ක්‍රම 3 (Inline, Internal, External) හඳුනා ගැනීම.
- බාහිර ජාවාස්ක්‍රිප්ට් ගොනුවක් (External Script) භාවිතා කිරීමේ වෘත්තීය වාසි තේරුම් ගැනීම.
- බ්‍රවුසරයේ "Console" (කන්සෝල්) යනු කුමක්ද සහ එය භාවිතා කරන්නේ ඇයි දැයි වටහා ගැනීම.
- ඔබගේ ප්‍රථම ජාවාස්ක්‍රිප්ට් විධානය වන `console.log()` නිවැරදිව ලිවීම.
- කේතය තුළ සටහන් (Comments) යෙදීමේ වැදගත්කම සහ ක්‍රමවේදයන්.

28.1 ජාවාස්ක්‍රිප්ට් ලියන්නේ කොහේද? (Where to write JavaScript?)

අප HTML සහ CSS ලිවීමට භාවිතා කළ VS Code මෘදුකාංගයම ජාවාස්ක්‍රිප්ට් සඳහා ද භාවිතා කළ හැකිය. ජාවාස්ක්‍රිප්ට් කේතයක් ක්‍රියාත්මක වීමට නම් එය වෙබ් බ්‍රවුසරයක් (Browser) තුළට ඇතුළත් කළ යුතුය. මේ සඳහා ප්‍රධාන ක්‍රම 3ක් ඇත.

1. Inline JavaScript (වැගය තුළ ලිවීම)

මෙහිදී HTML වැගයක් ඇතුළතම `onclick` වැනි විශේෂ ගුණාංග භාවිතා කර කේතය ලියනු ලැබේ.

```
<button onclick="alert('ඔබ මාව එබුවා!')">මාව ඔබන්න</button>
```

- වෘත්තීය මතය: මෙය ඉතාමත් තරක පුරුද්දකි. කුඩා වැඩකට වුවද මෙය භාවිතා නොකිරීමට වගබලා ගන්න. මන්දයත් HTML සහ JavaScript

එකිනෙක පටලවා ගැනීමෙන් කේතය පිරිසිදුව තබා ගැනීමට අපහසු වේ.

2. Internal JavaScript (අභ්‍යන්තර ලිවීම)

මෙහිදී HTML ගොනුවේ `<head>` හෝ `<body>` කොටස තුළ `<script>` නමැති වැගය භාවිතා කර එහි ඇතුළත කේතය ලියනු ලැබේ.

```
<script>
```

```
    console.log("මම HTML පිටුව ඇතුළේ ඉන්න කේතයක්!");
```

```
</script>
```

- භාවිතය: මෙය ඉතා කුඩා පරික්ෂණයක් කිරීමට සුදුසු වුවද, විශාල ව්‍යාපෘති සඳහා භාවිතා නොකරයි.

3. External JavaScript (බාහිර ලිවීම - වෘත්තීය ක්‍රමය)

මෙහිදී ජාවාස්ක්‍රිප්ට් කේත සියල්ල වෙනම ගොනුවක (උදා: `script.js`) ලියා, එම ගොනුව HTML පිටුවට සම්බන්ධ කරනු ලැබේ.

- වාසි: කේතය පිරිසිදුව පවතී. HTML සහ JavaScript වෙන් වෙන්ව පවතී. එකම ජාවාස්ක්‍රිප්ට් ගොනුව පිටු කිහිපයකට භාවිතා කළ හැක.

28.2 බාහිර ගොනුවක් සම්බන්ධ කරන ආකාරය (Linking an External Script)

මෙය ඉතාමත් වැදගත් පියවරකි. අප සැමවිටම ජාවාස්ක්‍රිප්ට් ලිවිය යුත්තේ `.js` යනුවෙන් අවසන් වන වෙනම ගොනුවක ය.

පියවර:

1. ඔබගේ ව්‍යාපෘති ෆෝල්ඩරය (Project Folder) තුළ `index.html` සහ `script.js` ලෙස ගොනු දෙකක් සාදන්න.

2. `index.html` ගොනුවේ `</body>` (Body අවසාන වැගය) ට මඳක් ඉහළින් පහත පේළිය ඇතුළත් කරන්න:

```
<script src="script.js"></script>
```

ඇයි අපි මෙය පතුලේම (Bottom) ලියන්නේ? ඔවුසරය වෙබ් පිටුවක් කියවන්නේ ඉහළ සිට පහළටය. ජාවාස්ක්‍රිප්ට් ගොනුව ඉහළින්ම (<head> තුළ) තිබුණහොත්, එම විශාල ගොනුව බාගත වී අවසන් වන තෙක් වෙබ් අඩවියේ අකුරු සහ පින්තූර පරිශීලකයාට පෙනෙන්නේ නැත. නමුත් පතුලේ ලියූ විට, මුලින්ම වෙබ් අඩවිය දිස්වන අතර, පසුව පසුබිමෙන් ජාවාස්ක්‍රිප්ට් ක්‍රියාත්මක වේ. මෙය වෙබ් අඩවියේ වේගය (Performance) වැඩි කිරීමට උදව් වේ.

28.3 ඔවුසර කන්සෝලය (The Browser Console)

ජාවාස්ක්‍රිප්ට් ඉගෙන ගන්නා ශිෂ්‍යයෙකුට ඇති වැදගත්ම උපකරණය වන්නේ Console (කන්සෝලය) යි.

එය විවෘත කරන්නේ කෙසේද?

1. ඕනෑම වෙබ් අඩවියකට ගොස් මවුසයේ දකුණු පස ක්ලික් කරන්න (Right Click).
2. Inspect (පරීක්ෂා කරන්න) යන්න තෝරන්න.
3. ඉහළින් ඇති ටැබ් (Tabs) අතරින් Console යන්න තෝරන්න.

කන්සෝලයේ කාර්යය: කන්සෝලය යනු වෙබ් සංවර්ධකයෙකුට ඔවුසරය සමඟ "කතා කිරීමට" ඇති පණිවිඩ පුවරුවයි. අප ලියන කේතයේ ප්‍රතිඵල බැලීමට සහ කේතයේ වැරදි (Errors) තිබේ නම් ඒවා හඳුනා ගැනීමට මෙය භාවිතා කරයි.

28.4 ප්‍රථම කේතය: console.log()

දැන් අපි අපේ පළමු ජාවාස්ක්‍රිප්ට් විධානය (Command) ලියමු.

විධානය:

```
console.log("ආයුබෝවන් ජාවාස්ක්‍රිප්ට්!");
```

සෑම වචනයකම තේරුම බලමු:

- console: මෙය ඔවුසරය තුළ ඇති වස්තුවකි (Object). අපට පණිවිඩ පෙන්වීමට අවශ්‍ය උපකරණ මෙහි අඩංගු වේ.
- . (නිත): මෙයින් අදහස් කරන්නේ console නමැති උපකරණ පෙට්ටිය ඇතුළත ඇති යම් දෙයක් අප භාවිතා කරන බවයි.
- log: මෙය ශ්‍රිතයකි (Function / Method). මෙයින් කරන්නේ වරහන් ඇතුළත අප ලබා දෙන දත්තය කන්සෝලය මත මුද්‍රණය කිරීමයි.

- "..." (උද්ධෘත ලකුණු): අප ලියන්නේ සාමාන්‍ය අකුරු වැලක් (String) බව පරිගණකයට හැඟවීමට මේවා භාවිතා කරයි.

- ; (සෙමිකෝලනය): මෙය වාක්‍යයක අවසානයට තබන තීත වැනිය. විධානයක් මෙතැනින් අවසන් බව එයින් අදහස් වේ. (නූතන ජාවාස්ක්‍රිප්ට් වල මෙය අනිවාර්ය නැතත්, එය තැබීම ඉතා හොඳ පුරුද්දකි).

28.5 කේතය තුළ සටහන් (Comments in JavaScript)

අප ලියන කේතය ගැන අපට හෝ වෙනත් සංවර්ධකයෙකුට විස්තරයක් ලිවීමට Comments භාවිතා කරයි. මෙම සටහන් පරිගණකය විසින් කියවනු නොලැබේ; ඒවා ඇත්තේ මිනිසුන්ගේ ප්‍රයෝජනය සඳහා පමණි.

ක්‍රම දෙකක් ඇත:

1. තනි පේළියක සටහන් (Single-line comments): ඉරි කැලි දෙකක් (//) භාවිතා කරයි.

```
// මෙය පණිවිඩයක් පෙන්වන කේතයකි  
console.log("Hello");
```

2. පේළි කිහිපයක සටහන් (Multi-line comments): /* සහ */ භාවිතා කරයි.

```
/*  
මෙම කේතය මගින්  
වෙබ් අඩවියේ වර්ණ  
වෙනස් කිරීමට හැකියාව ඇත. */
```

28.6 සම්පූර්ණ ප්‍රායෝගික පියවර (Step-by-Step Practical)

අපි දැන් සියල්ල එකට සම්බන්ධ කරමු.

1. ෆෝල්ඩරයක් සාදන්න: ඔබේ පරිගණකයේ JS_Practice නමින් ෆෝල්ඩරයක් සාදන්න.
2. VS Code විවෘත කරන්න: එම ෆෝල්ඩරය VS Code හරහා විවෘත කරන්න.
3. HTML ගොනුව: index.html සාදා එහි මූලික HTML සැකිල්ල ඇතුළත් කරන්න. <body> ටැගය අවසානයට මඳක් ඉහළින්

```
<script src="script.js"></script> ලියන්න.
```


4. JS ගොනුව: script.js නමින් ගොනුවක් සාදා එහි පහත දේ ටයිප් කරන්න:

```
console.log("මගේ වැඩබිම් දැන් සුදානම්!");
```

```
console.log(10 + 20); // ගණිතමය ගණනය කිරීමක්
```

5. ක්‍රියාත්මක කරන්න: index.html ගොනුව ඔවුසරයෙන් විවෘත කරන්න (Right click ->Open with Live Server).

6. පරීක්ෂා කරන්න: ඔවුසරයේ Console එක විවෘත කර බලන්න. එහි ඔබ ලියූ පණිවිඩය සහ 30 යන අංකය පෙනෙනු ඇත.

28.7 වෘත්තීය පැහැදිලි කිරීම (Professional Explanation)

සැබෑ මෘදුකාංග ඉංජිනේරුවෙකු කිසිවිටකත් `console.log()` පණිවිඩ පාරිභෝගිකයාට පෙනෙන සේ අවසාන නිෂ්පාදනයේ (Production) ඉතිරි කරන්නේ නැත. ඒවා භාවිතා කරන්නේ කේතය සාදන අවස්ථාවේදී (Development) එහි ක්‍රියාකාරිත්වය පරීක්ෂා කිරීමට (Debugging) පමණි. එසේම, වෘත්තීය මට්ටමේදී බාහිර ජාවාස්ක්‍රිප්ට් ගොනු සම්බන්ධ කිරීමේදී `async` සහ `defer` වැනි නවීන ගුණාංග භාවිතා කළද, ආරම්භකයෙකු ලෙස `</body>` ට ඉහළින් සම්බන්ධ කිරීම ප්‍රමාණවත්ය.

28.8 පොදු වැරදි (Common Mistakes)

- අක්ෂර වින්‍යාස වැරදි (Typos): `Console.log` (C කැපිටල්) හෝ `concole.log` ලෙස ලිවීම. ජාවාස්ක්‍රිප්ට් වල අකුරු වල හැඩය (Case Sensitivity) ඉතා වැදගත් වේ. සැමවිටම `console` යන්න `simple` අකුරින් විය යුතුය.
- ගොනු නාම වැරදීම: HTML හි `src="script.js"` ලෙස ලියා, සැබෑ ගොනුවේ නම `main.js` ලෙස තිබීම.
- සටහන් සලකුණු පටලවා ගැනීම: CSS හි සටහන් (`/* */`) සහ HTML හි සටහන් (`<!-- -->`) ජාවාස්ක්‍රිප්ට් තුළ භාවිතා කිරීමට උත්සාහ කිරීම.

28.9 සාරාංශය (Key Points)

- ජාවාස්ක්‍රිප්ට් ලිවීමට හොඳම ක්‍රමය බාහිර (External) .js ගොනුවක් භාවිතා කිරීමයි.

- `<script>` ටැගය HTML පිටුවේ පතුලේ (`</body>` ට පෙර) තැබීම වේගය වැඩි කිරීමට උදව්වේ.
- `console.log()` යනු කන්සෝලය මත තොරතුරු පෙන්වීමට භාවිතා කරන ප්‍රධාන විධානයයි.
- කේතයේ අරමුණ පැහැදිලි කිරීමට Comments (`//` හෝ `/* */`) භාවිතා කළ යුතුය.

28.10 අභ්‍යාස (Exercises)

- ඔබේ පරිගණකයේ `script.js` ගොනුවක් සාදා එහි ඔබේ නම සහ වයස කන්සෝලය මත පෙන්වන කේතය ලියන්න.
- Internal JavaScript සහ External JavaScript අතර ඇති වෙනස ඔබේ වචනයෙන් පැහැදිලි කරන්න.
- `console.log(20 + 30)`; යන විධානයේ ප්‍රතිඵලය කුමක් විය හැකිද?
- ජාවාස්ක්‍රිප්ට් කේතයක් පතුලේ සම්බන්ධ කිරීමේ වාසිය කුමක්ද?

28.11 සම්මුඛ පරීක්ෂණ ප්‍රශ්න (Interview Questions)

- ප්‍රශ්නය: ඔබට ඔවුසරයේ Alert පණිවිඩයක් පෙන්විය හැක්කේ කෙසේද?
- පිළිතුර: `alert("ඔබේ පණිවිඩය");` යන විධානය භාවිතා කිරීමෙනි. (නමුත් වෘත්තීය කටයුතු වලදී මෙය භාවිතා කිරීම අවම කරයි).

මිළඟ පරිච්ඡේදයෙන් අපි ජාවාස්ක්‍රිප්ට් වල දත්ත තාවකාලිකව ගබඩා කර තබන "විචල්‍යයන්" (Variables) ගැන ඉතාමත් විස්තරාත්මකව ඉගෙන ගනිමු.

විසිනවවන පරිච්ඡේදය: විචල්‍යයන් - දත්ත ගබඩා කිරීම (Variables)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

- විචල්‍යයක් (Variable) යනු කුමක්දැයි සැබෑ ලෝකයේ උදාහරණ ඇසුරින් වටහා ගැනීම.
- විචල්‍යයන් ප්‍රකාශ කිරීම (Declaration) සහ අගයන් ලබා දීම (Assignment) යන ක්‍රියාවලීන් තේරුම් ගැනීම.

3. var, let සහ const යන මූලපද (Keywords)

අතර ඇති වෙනස හඳුනා ගැනීම.

4. විචල්‍යයන් නම් කිරීමේදී (Naming

Conventions) අනුගමනය කළ යුතු නීති සහ

සම්ප්‍රදායන් ඉගෙන ගැනීම.

5. නූතන ජාවාස්ක්‍රිප්ට් සංවර්ධනයේදී let සහ const

භාවිතා කළ යුතු අවස්ථා වටහා ගැනීම.

6. CamelCase නාමකරණ ක්‍රමය හඳුනා ගැනීම.

29.1 විචල්‍යයක් යනු කුමක්ද? (What is a Variable?)

පරිගණක වැඩසටහනක් (Program) යනු සරලවම කිවහොත් දත්ත (Data) සමඟ වැඩ කිරීමයි. මෙම දත්ත පරිගණකයේ මතකය (Memory/RAM) තුළ කොහේ හෝ තැනක තාවකාලිකව තබා ගත යුතුය. පසුව අපට එම දත්තය නැවත ලබා ගැනීමට නම්, එය තබා ඇති ස්ථානයට නමක් තිබිය යුතුය.

විචල්‍යයක් (Variable) යනු දත්තයක් ගබඩා කර තබන "පෙට්ටියක්" හෝ "බඳුනක්" වැනි දෙයකි. අප එම පෙට්ටියට නමක් ලබා දී, ඒ ඇතුළත දත්තයක් තැන්පත් කරමු.

සැබෑ ලෝකයේ උදාහරණයක් (කුස්සියේ බෝතල්): ඔබේ නිවසේ කුස්සියේ ඇති විවිධ බෝතල් ගැන සිතන්න.

- එක් බෝතලයකට ඔබ "සීනි" (Sugar) ලෙස ලේබලයක් අලවා ඒ තුළට සීනි දමයි.
- තවත් බෝතලයකට "ලුණු" (Salt) ලෙස ලේබලයක් අලවා ඒ තුළට ලුණු දමයි.
- පසුව ඔබට සීනි අවශ්‍ය වූ විට, ඔබ කරන්නේ "සීනි" යන ලේබලය ඇති බෝතලය සොයා එහි ඇති දේ ලබා ගැනීමයි.

මෙහිදී බෝතලයට අලවා ඇති නම Variable Name වන අතර, බෝතලය ඇතුළත ඇති දේ Value (අගය) වේ.

29.2 විචල්‍යයක් භාවිතා කරන පියවර (Process of Using Variables)

ජාවාස්ක්‍රිප්ට් වලදී විචල්‍යයක් භාවිතා කිරීමට ප්‍රධාන පියවර දෙකක් ඇත.

1. විචල්‍යය ප්‍රකාශ කිරීම (Variable Declaration)

මෙයින් කරන්නේ "මම අහවල් නමින් පෙට්ටියක් හඳුනවා" කියා පරිගණකයට පැවසීමයි.

```
let myName;
```

මෙහිදී let යනු විචල්‍යයක් සාදන බව පවසන මූලපදයයි. myName යනු අප ලබා දුන් නමයි.

2. අගයක් ලබා දීම (Value Assignment)

සෑදූ පෙට්ටිය ඇතුළට දත්තයක් ඇතුළු කිරීම මෙහිදී සිදු වේ. මෙයට සමාන ලකුණ (=) භාවිතා කරයි.

```
myName = "කසුන්";
```

දැන් myName නමැති පෙට්ටිය ඇතුළත "කසුන්" යන අකුරු වැල ගබඩා වී ඇත.

විශේෂත්වය: අපට මෙම පියවර දෙකම එකවර කළ හැකිය. මෙයට මූල පිරිම (Initialization) යැයි කියනු ලැබේ.

```
let myAge = 16;
```

29.3 var, let සහ const අතර වෙනස

විචල්‍යයක් ප්‍රකාශ කිරීමට ජාවාස්ක්‍රිප්ට් වල වචන තුනක් ඇත. මෙය ඉතාමත් වැදගත් කොටසකි.

1. var (පැරණි ක්‍රමය)

මෙය 1995 සිට 2015 දක්වා ජාවාස්ක්‍රිප්ට් වල තිබූ එකම ක්‍රමයයි. මෙහි සමහර තාක්ෂණික දෝෂ පවතින බැවින් නවීන වෙබ් සංවර්ධනයේදී (ES6+) මෙය භාවිතා නොකරන තරම්ය.

2. let (නූතන ක්‍රමය - වෙනස් කළ හැකි)

ඔබට පසුව අගය වෙනස් කිරීමට අවශ්‍ය විචල්‍යයන් සඳහා මෙය භාවිතා කරයි.

```
let score = 10;
```

```
score = 20; // මෙය කළ හැක. දැන් අගය 20 වේ.
```

3. const (නූතන ක්‍රමය - ස්ථාවර)

Constant යන්නෙහි කෙටි නමයි. වරක් අගයක් ලබා දුන් පසු එය කිසිම විටක වෙනස් කළ නොහැක.

```
const birthYear = 2008;
```

```
// birthYear = 2010; // මෙය කළ නොහැක.
```

Error (වැරද්දක්) පෙන්වයි.

වෘත්තීය උපදෙස: සැමවිටම විචල්‍යයක් සාදන විට එය **const** ලෙස සාදන්න. ඔබට එය පසුවෙනස් කිරීමට අවශ්‍ය යැයි හැඟේ නම් පමණක් **let** බවට හරවන්න.

29.4 විචල්‍යයන් නම් කිරීමේ නීති (Naming Rules)

ජාවාස්ක්‍රිප්ට් වල විචල්‍යයන්ට අපට කැමති නම් ලබා දිය නොහැක. ඒ සඳහා දැඩි නීති කිහිපයක් ඇත:

- අකුරු, ඉලක්කම් සහ සංකේත: නම් වල අඩංගු විය හැක්කේ අකුරු (Letters), ඉලක්කම් (Digits), යට ඉර (Underscore `_`) සහ ඩොලර් ලකුණ (\$) පමණි.
- අංකයකින් පටන් ගත නොහැක: නමක මුලට කිසිවිටකත් ඉලක්කමක් යෙදිය නොහැක. (උදා: `1name` වැරදි, `name1` නිවැරදි).
- හිස්තැන් තැබිය නොහැක: නම මැද Space තැබිය නොහැක. (උදා: `my name` වැරදි).
- Reserved Keywords:** ජාවාස්ක්‍රිප්ට් භාෂාවටම වෙන්වූ වචන භාවිතා කළ නොහැක. (උදා: `let`, `const`, `function`, `if` වැනි වචන විචල්‍ය නාම ලෙස යෙදිය නොහැක).

29.5 CamelCase නාමකරණ සම්ප්‍රදාය

වෘත්තීය ජාවාස්ක්‍රිප්ට් සංවර්ධකයන් නම් තැබීමේදී **camelCase** (ඔටුවෙකුගේ පිට වැනි) ක්‍රමය අනුගමනය කරයි.

- පළමු වචනය **simple** අකුරින් පටන් ගනී.
- දෙවන වචනයේ සිට සෑම වචනයකම මුල් අකුර **Capital** අකුරින් ලියයි.

උදාහරණ:

- `firstname` (වැරදි නැත, නමුත් වෘත්තීය නැත)
- `first_name` (Python වැනි භාෂා වල භාවිතා කරයි)
- `firstName` (නිවැරදි camelCase ක්‍රමය)
- `userLoginStatus` (camelCase ක්‍රමය)

29.6 ASCII රූපසටහන: මතකයේ සිදුවන දේ

පරිගණක මතකය (RAM)

ලිපිනය	නම	අගය	
0x001	age	16	<code><-- let age = 16;</code>
0x002	name	"Ali"	<code><-- const name = "Ali";</code>
0x003	[හිස්]	[හිස්]	

29.7 ප්‍රායෝගික උදාහරණය (Practical Example)

ඔබේ `script.js` ගොනුවේ පහත කේතය ලියා කන්සෝලය පරීක්ෂා කරන්න.

// 1. const භාවිතා කිරීම (වෙනස් නොවන දත්ත)

```
const country = "Sri Lanka";
```

// 2. let භාවිතා කිරීම (වෙනස් වන දත්ත)

```
let currentYear = 2024;
```

```
console.log("මගේ රට: " + country);
```

```
console.log("වත්මන් වසර: " + currentYear);
```

// වසර වෙනස් කරමු

```
currentYear = 2025;
```

```
console.log("මිලභ වසර: " + currentYear);
```

// පහත ඡේළියේ ඇති // ඉවත් කර බලන්න කුමක් සිදුවේද කියා

```
// country = "India";
```

29.8 පොදු වැරදි (Common Mistakes)

1. **Case Sensitivity:** ජාවාස්ක්‍රිප්ට් වල apple සහ Apple යනු සම්පූර්ණයෙන්ම වෙනස් විචල්‍යයන් දෙකකි. අකුරු වල හැඩය (Capital/Simple) වැරදුනහොත් කේතය ක්‍රියා නොකරයි.
2. අගයක් නැතිව const ප්‍රකාශ කිරීම: const x; ලෙස ලිවිය නොහැක. const එකක් සාදන විටම එයට අගයක් ලබා දිය යුතුය.
3. අකුරු වැරදි (Typos): විචල්‍යය සාදන විට myName ලෙස සාදා, පසුව භාවිතා කරන විට myname ලෙස ලිවීම.

29.9 වෘත්තීය උපදෙස් (Professional Tips)

- අර්ථවත් නම් ලබා දෙන්න: let a = 10; ලෙස ලියනවාට වඩා let userAge = 10; ලෙස ලිවීමෙන් කේතය කියවන ඕනෑම අයෙකුට එය වැටහේ.
- Global Variables අවම කරන්න: සියලුම විචල්‍යයන් එකම තැනක ගොඩගසන්නේ නැතිව, ඒවා අවශ්‍ය තැන පමණක් ප්‍රකාශ කිරීමට පුරුදු වන්න. (මෙය ඉදිරියේදී Scope පාඩමේදී විස්තරාත්මකව ඉගෙන ගනිමු).

29.10 සාරාංශය (Key Points)

- විචල්‍යයන් දත්ත ගබඩා කිරීමට භාවිතා කරයි.
- let පසුව වෙනස් කළ හැකි අගයන් සඳහා භාවිතා වේ.
- const වරක් ලබා දුන් පසු වෙනස් කළ නොහැකි අගයන් සඳහා භාවිතා වේ.
- නම් තැබීමේදී camelCase ක්‍රමය භාවිතා කිරීම වෘත්තීය පුරුද්දකි.
- විචල්‍ය නාමයක් අංකයකින් පටන් ගත නොහැක.

29.11 අභ්‍යාස (Exercises)

1. පහත සඳහන් විචල්‍ය නාම වලින් නිවැරදි සහ වැරදි ඒවා වෙන් කරන්න:

- my_variable
- 2ndPlace
- totalAmount
- let
- \$price

2. ඔබේ උපන් වසර ගබඩා කිරීමට සුදුසු let ද const ද? හේතුව පැහැදිලි කරන්න.

3. පහත තොරතුරු ගබඩා කිරීමට අවශ්‍ය විචල්‍යයන් 3ක් නිවැරදි camelCase ක්‍රමයට

සාදන්න:

- ඔබේ පාසලේ නම.
- ඔබේ පන්තිය.
- අද දවසේ උෂ්ණත්වය.

29.12 සම්මුඛ පරීක්ෂණ ප්‍රශ්න (Interview Questions)

- ප්‍රශ්නය: var සහ let අතර ඇති ප්‍රධාන වෙනස කුමක්ද?
- පිළිතුර: var යනු පැරණි ක්‍රමයයි, එයට "Function Scope" එකක් ඇත. let යනු නූතන ක්‍රමයයි, එයට "Block Scope" එකක් ඇත. එසේම var මගින් එකම නමින් විචල්‍යයන් දෙකක් සෑදිය හැකි වුවද let මගින් එය කළ නොහැක. (Scope ගැන ඉදිරි පරිච්ඡේදවලදී විස්තර කෙරේ).

මිලඟ පරිච්ඡේදයෙන් අපි ජාවාස්ක්‍රිප්ට් වල අපට හමුවන විවිධ දත්ත වර්ග (Numbers, Strings, Booleans) ගැන ඉතාමත් විස්තරාත්මකව ඉගෙන ගනිමු.

නිස්වන පරිච්ඡේදය: දත්ත වර්ග - දත්ත හඳුනා ගැනීම (Data Types)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

1. ජාවාස්ක්‍රිප්ට් වල දත්ත වර්ග (Data Types) යනු මොනවාදැයි අවබෝධ කර ගැනීම.
2. ප්‍රාථමික දත්ත වර්ග (Primitive Data Types) 7 හඳුනා ගැනීම.
3. ජාවාස්ක්‍රිප්ට් "Dynamic Typing" (නම්‍යශීලී දත්ත වර්ගීකරණය) සහිත භාෂාවක් ලෙස හැසිරෙන ආකාරය තේරුම් ගැනීම.
4. Number, String, Boolean, Undefined, සහ Null යන දත්ත වර්ග සවිස්තරාත්මකව ඉගෙන ගැනීම.
5. typeof මෙහෙයුම්කරු (Operator) භාවිතා කර දත්ත වර්ග පරීක්ෂා කිරීම.
6. විවිධ දත්ත වර්ග වැරදි ලෙස භාවිතා කිරීමෙන් සිදුවන දෝෂ හඳුනා ගැනීම.

30.1 දත්ත වර්ගයක් යනු කුමක්ද? (What is a Data Type?)

පෙර පරිච්ඡේදයේදී අප ඉගෙන ගත්තේ විචල්‍යයන් යනු දත්ත ගබඩා කරන "පෙට්ටි" බවයි. නමුත් එම පෙට්ටි ඇතුළත අප ගබඩා කරන්නේ කුමන වර්ගයේ දත්තද යන්න පරිගණකය දැනගත යුතුය.

උදාහරණයක් ලෙස:

- ඔබට අංක දෙකක් එකතු කිරීමට අවශ්‍ය නම්, එම දත්ත "අංක" (Numbers) විය යුතුය.
- ඔබට පුද්ගලයෙකුගේ නමක් පෙන්වීමට අවශ්‍ය නම්, එම දත්ත "අකුරු" (Strings) විය යුතුය.

පරිගණකය දත්ත වර්ගය හඳුනා ගන්නේ එම දත්තය මත කළ හැකි ක්‍රියාවන් මොනවාදැයි තීරණය කිරීමටයි. (අංක දෙකක් ගුණ කළ හැකි වුවද, නම් දෙකක් ගුණ කළ නොහැක).

30.2 ජාවාස්ක්‍රිප්ට් හි නම්‍යශීලී බව (Dynamic Typing)

ජාවාස්ක්‍රිප්ට් වල ඇති එක් විශේෂත්වයක් වන්නේ අප විචල්‍යයක් සාදන විට එහි දත්ත වර්ගය කුමක්දැයි කල්තියා පැවසිය යුතු නොවීමයි. ඔවුසරය විසින් එහි ඇති අගය දෙස බලා එය කුමන වර්ගයේ දත්තයක්දැයි ස්වයංක්‍රීයව තීරණය කරයි.

එසේම, එක් වර්ගයක දත්තයක් ඇති විචල්‍යයකට පසුව වෙනත් වර්ගයක දත්තයක් ලබා දීමටද හැකියාව ඇත.

```
let myData = 25;    // මෙහිදී මෙය Number (අංකයක්) වේ
```

```
myData = "කසුන්"; // දැන් එය String (අකුරු වැලක්) බවට ස්වයංක්‍රීයව මාරු වේ
```

30.3 ප්‍රාථමික දත්ත වර්ග 7 (The 7 Primitive Data Types)

ජාවාස්ක්‍රිප්ට් වල මූලිකවම දත්ත වර්ග 7ක් ඇත. ආරම්භකයෙකු ලෙස මෙහි මුල් 5 පිළිබඳව මනා දැනුමක් තිබීම අත්‍යවශ්‍ය වේ.

1. Number (අංක)

ජාවාස්ක්‍රිප්ට් වල පූර්ණ සංඛ්‍යා (Integers) සහ දශම සංඛ්‍යා (Decimals) යන දෙකම "Number" ලෙස හඳුන්වයි.

```
let age = 16;    // Integer
```

```
let price = 99.50; // Float / Decimal
```

විශේෂ අගයන්: Infinity (අනන්තය) සහ NaN (Not a Number - ගණිතමය වශයෙන් වැරදි දෙයක් කළ විට ලැබෙන අගය) ද Number වර්ගයට අයත් වේ.

2. String (අකුරු වැල්)

ඕනෑම අකුරු, වචන හෝ වාක්‍ය සමූහයක් මෙලෙස හැඳින්වේ. මේවා සැමවිටම උද්ධෘත ලකුණු තුළ ලිවිය යුතුය.

- තනි උද්ධෘත: 'Hello'
- ද්විත්ව උද්ධෘත: "Hello"
- Backticks: `Hello`

```
let name = "Kamal";
```

```
let message = 'සාදරයෙන් පිළිගනිමු!';
```

3. Boolean (බුලියන්)

මෙයට තිබිය හැක්කේ අගයන් දෙකක් පමණි. මෙය තීරණ ගැනීම සඳහා භාවිතා කරයි.

- true (සත්‍ය)
- false (අසත්‍ය)

```
let isStudent = true;
```

```
let isGraduated = false;
```

4. Undefined (අර්ථ නොදැක්වූ)

විචල්‍යයක් ප්‍රකාශ කර ඇති නමුත් එයට කිසිදු අගයක් ලබා දී නැති විට ලැබෙන අගයයි. "හිස් පෙට්ටියක්" වැනිය.

```
let x;
```

```
console.log(x); // ප්‍රතිඵලය: undefined
```

5. Null (ශුන්‍ය / හිස්)

විචල්‍යයක කිසිදු අගයක් නැති බව හිතාමතාම දැක්වීමට භාවිතා කරයි. මෙය "හිස්" බව පෙන්වන ලකුණකි.

```
let partnerName = null; // දැනට සහකරුවෙකු නැති බව හිතාමතාම පවසයි
```

6. BigInt (ඉතා විශාල අංක)

සාමාන්‍ය Number වර්ගයට දරාගත නොහැකි ඉතාමත් විශාල පූර්ණ සංඛ්‍යා සඳහා භාවිතා කරයි.

(අංකය අගට n අකුර යොදයි).

7. Symbol (සංකේත)

අනන්‍ය වූ (Unique) අගයන් සෑදීමට භාවිතා කරයි. (මෙය උසස් ජාවාස්ක්‍රිප්ට් වලදී ඉගෙන ගනිමු).

30.4 typeof මෙහෙයුම්කරු (The typeof Operator)

යම් විචල්‍යයක ඇති දත්ත වර්ගය කුමක්දැයි අපට සැකයක් ඇත්නම්, එය පරීක්ෂා කිරීමට typeof භාවිතා කළ හැක.

```
let count = 100;
```

```
console.log(typeof count); // ප්‍රතිඵලය: "number"
```

```
let city = "Colombo";
```

```
console.log(typeof city); // ප්‍රතිඵලය: "string"
```

```
let isActive = false;
```

```
console.log(typeof isActive); // ප්‍රතිඵලය: "boolean"
```

30.5 ASCII රූපසටහන: දත්ත වර්ග වල ස්වභාවය

දත්ත වර්ගය (Data Type)	උදාහරණය	ස්වභාවය
Number	10, 3.14	ගණනය කිරීම් සඳහා
String	"Hello"	තොරතුරු පෙන්වීමට
Boolean	true / false	නිරණ ගැනීමට
Undefined	(අගයක් නැත)	සලකුණු නොකළ පෙට්ටියක්
Null	null	හිතාමතා හිස් කළ පෙට්ටියක්

30.6 පොදු වැරදි සහ ව්‍යාකූලතා (Common Mistakes)

1. Numbers සහ Strings පටලවා ගැනීම: අංකයක් උද්ධෘත ලකුණු තුළ ලිවුවොත් එය අංකයක් නොව අකුරු වැලක් (String) බවට පත් වේ.

```
let x = 10; // මෙය Number එකකි
```

```
let y = "10"; // මෙය String එකකි
```

```
console.log(x + x); // පිළිතුර: 20
```

```
console.log(y + y); // පිළිතුර: "1010" (දෙක එකට ඇලවේ)
```

2. typeof null හි ඇති ගැටලුව: ජාවාස්ක්‍රිප්ට් වල ඇති පැරණි දෝෂයක් (Bug) නිසා typeof null පරීක්ෂා කළ විට එය "object" ලෙස පෙන්වයි. එය "null" ලෙස පෙන්විය යුතු වුවත්, තාක්ෂණික හේතූන් මත එය තවමත් "object" ලෙස පවතී. මෙය මතක තබා ගැනීම වැදගත්ය.

30.7 වෘත්තීය උපදෙස් (Professional Tips)

- String සඳහා Backticks භාවිතා කරන්න: නූතන වෙබ් සංවර්ධනයේදී ` (backticks) භාවිතා කිරීම ඉතා ජනප්‍රියයි. මන්දයත් ඒ තුළ විචල්‍යයන් ඇතුළත් කිරීම (Template Literals) ඉතා පහසු බැවිනි.

- Undefined vs Null: කිසිවිටකත් විචල්‍යයකට හිතාමතාම undefined ලබා නොදෙන්න. යමක් හිස් කිරීමට අවශ්‍ය නම් සැමවිටම null භාවිතා කරන්න. එවිට එය හිතාමතා කළ දෙයක් බව අනෙක් සංවර්ධකයන්ට වැටහේ.

30.8 සාරාංශය (Key Points)

- ජාවාස්ක්‍රිප්ට් වල දත්ත වර්ග 7ක් ඇත. (ප්‍රධාන වශයෙන්: Number, String, Boolean, Null, Undefined).
- ජාවාස්ක්‍රිප්ට් Dynamic Typing භාවිත කරයි (විචල්‍යයක දත්ත වර්ගය වෙනස් විය හැක).
- Strings සැමවිටම උද්ධෘත ලකුණු "" හෝ '' තුළ ලිවිය යුතුය.
- Booleans තීරණ ගැනීමට (true/false) භාවිත කරයි.
- typeof මගින් දත්ත වර්ගය සොයාගත හැක.

30.9 අභ්‍යාස (Exercises)

1. පහත දැක්වෙන විචල්‍යයන් වල දත්ත වර්ගය කුමක්දැයි ලියන්න:

- let a = "25";
- let b = 25;
- let c = true;
- let d;
- let e = null;

2. console.log("5" + 5); සහ console.log(5 + 5); යන දෙකෙහි ප්‍රතිඵලයන්ගේ වෙනස පැහැදිලි කරන්න.

3. ඔබේ වයස සහ නම විචල්‍යයන් දෙකක ගබඩා කර, typeof භාවිත කර ඒවායේ දත්ත වර්ග කන්සෝලයේ මුද්‍රණය කරන්න.

30.10 සම්මුඛ පරීක්ෂණ ප්‍රශ්න (Interview Questions)

- ප්‍රශ්නය: undefined සහ null අතර ඇති ප්‍රධාන වෙනස කුමක්ද?
- පිළිතුර: undefined යනු විචල්‍යයකට අගයක් ලැබී නැති බව ඉවුසරය පවසන ආකාරයයි. null යනු විචල්‍යයක අගයක් නැති බව සංවර්ධකයා විසින් හිතාමතාම පවසන ආකාරයයි.

මිලින පරිච්ඡේදයෙන් අපි ජාවාස්ක්‍රිප්ට් වල ගණිතමය සහ තර්කන ක්‍රියාවන් සිදු කිරීමට

භාවිත කරන "ක්‍රියාකරුවන්" (Operators) ගැන ඉතාමත් විස්තරාත්මකව ඉගෙන ගනිමු.

නිස්ඵල වන පරිච්ඡේදය:

ක්‍රියාකරුවන් - දත්ත සමඟ වැඩ කිරීම (Operators)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

1. ජාවාස්ක්‍රිප්ට් හි ක්‍රියාකරුවන් (Operators) යනු කුමක්දැයි තේරුම් ගැනීම.
2. ගණිතමය ක්‍රියාකරුවන් (Arithmetic Operators) භාවිත කර ගණනය කිරීම් සිදුකිරීම.
3. ආදේශන ක්‍රියාකරුවන් (Assignment Operators) මගින් විචල්‍යයන්ට අගයන් ලබා දීම.
4. සංසන්දනාත්මක ක්‍රියාකරුවන් (Comparison Operators) මගින් දත්ත දෙකක් අතර වෙනස හඳුනා ගැනීම.
5. තර්කන ක්‍රියාකරුවන් (Logical Operators) භාවිත කර සංකීර්ණ තීරණ ගැනීම.
6. තන්තුවල (Strings) එකතු කිරීම හෙවත් සන්යෝජනය (Concatenation) තේරුම් ගැනීම.
7. ක්‍රියාකරුවන් ක්‍රියාත්මක වීමේ ප්‍රමුඛතාවය (Operator Precedence) අවබෝධ කර ගැනීම.

31.1 ක්‍රියාකරුවෙකු යනු කුමක්ද? (What is an Operator?)

අප කලින් පරිච්ඡේදවලදී විචල්‍යයන් (පෙට්ටි) සහ දත්ත (පෙට්ටිය ඇතුළත ඇති දේ) ගැන ඉගෙන ගත්තෙමු. නමුත් මෙම දත්ත භාවිතයෙන් යම් කාර්යයක් කිරීමට අපට ක්‍රමයක් අවශ්‍ය වේ.

ක්‍රියාකරුවෙකු (Operator) යනු දත්ත එකක් හෝ කිහිපයක් මත යම් ක්‍රියාවක් (Action) සිදු කිරීමට භාවිත කරන විශේෂිත සලකුණකි.

සැබෑ ලෝකයේ උදාහරණයක්: වඩු කාර්මිකයෙකු ගැන සිතන්න.

- ලැලි සහ ඇණ යනු දත්ත (Data) වේ.
- මිටිය සහ කියත යනු ක්‍රියාකරුවන් (Operators) වේ. මිටිය භාවිත කර ඇණ ගැසීම හෝ

කියත භාවිත කර ලැල්ල කැපීම මගින් අලුත් දෙයක් නිර්මාණය වේ.

ක්‍රමලේඛනයේදී (Programming) අප දත්ත දෙකක් එකතු කිරීමට + ලකුණ භාවිත කරන විට,

එම+ ලකුණ ක්‍රියාකරුවෙකු (Operator) ලෙස හැඳින්වේ. එම ක්‍රියාවට භාජනය වන දත්ත

(උදා: 10 සහ 20) Operands ලෙස හැඳින්වේ.

31.2 ගණිතමය ක්‍රියාකරුවන් (Arithmetic Operators)

මූලික ගණිතමය ගණනය කිරීම් සඳහා මේවා භාවිතා කරයි.

ක්‍රියාකරු නම (English)	විස්තරය (Sinhala)	උදාහරණය
+:-----	:-----	:-----
'+' Addition	එකතු කිරීම	'10 + 5 = 15'
'-' Subtraction	අඩු කිරීම	'10 - 5 = 5'
'*' Multiplication	ගුණ කිරීම	'10 * 5 = 50'
'/' Division	බෙදීම	'10 / 2 = 5'
'%' Modulus	ඉතිරිය සෙවීම	'10 % 3 = 1'
'**' Exponentiation	බලය සෙවීම	'2 ** 3 = 8'
'++' Increment	එකකින් වැඩි කිරීම	'x++'
'--' Decrement	එකකින් අඩු කිරීම	'x--'

විශේෂ පැහැදිලි කිරීම - Modulus (%): බෙදීමකදී ඉතිරි වන අගය මෙයින් ලබා දෙයි.

උදාහරණයක් ලෙස: ඔබ පැන් 10ක් ළමුන් 3 දෙනෙකුට සමානව බෙදා දුන්නොත්, එක් අයෙකුට පැන් 3 බැගින් ලැබෙන අතර 1ක් ඉතිරි වේ. $10 \% 3$ හි පිළිතුර 1 වේ.

31.3 ආදේශන ක්‍රියාකරුවන් (Assignment Operators)

විචල්‍යයන්ට අගයන් ලබා දීමට සහ පවතින අගය වෙනස් කිරීමට මේවා භාවිතා කරයි.

- =** (සරල ආදේශනය): දකුණු පස ඇති අගය වම් පස ඇති විචල්‍යයට ලබා දෙයි. `let x = 10;`
- +=** (එකතු කර ආදේශන කිරීම): `x += 5;` යනු `x = x + 5;` යන්නෙහි කෙටි ක්‍රමයයි. (දැන් x හි අගය 15 වේ).
- =** (අඩු කර ආදේශන කිරීම): `x -= 2;` යනු `x = x - 2;` යන්නයි.

31.4 සංසන්දනාත්මක ක්‍රියාකරුවන් (Comparison Operators)

දත්ත දෙකක් එකිනෙක සසඳා බැලීමට මේවා භාවිතා කරයි. මෙහි ප්‍රතිඵලය සැමවිටම Boolean අගයක් (true හෝ false) වේ.

- ==** (සමානද?): අගයන් දෙක සමාන දැයි බලයි (දත්ත වර්ගය නොසලකා හරියි). `"5" == 5 -> true`
- ===** (සම්පූර්ණයෙන්ම සමානද? - Strict Equality): අගය සහ දත්ත වර්ගය (Data Type) යන දෙකම සමාන දැයි බලයි. (වෘත්තීය මට්ටමේදී සැමවිටම මෙය භාවිතා කරන්න). `"5"`

`=== 5 -> false` (මොකද එකක් String, අනෙක Number).

- !=** (නොසමානද?): අගයන් දෙක එකිනෙකට වෙනස් නම් true ලබා දෙයි.
- !==** (සම්පූර්ණයෙන්ම නොසමානද?): අගය හෝ දත්ත වර්ගය වෙනස් නම් true ලබා දෙයි.
- >** (වැඩිද?): වම් පස අගය දකුණු පස අගයට වඩා වැඩි නම් true.
- <** (අඩුද?): වම් පස අගය දකුණු පස අගයට වඩා අඩු නම් true.
- >=** (වැඩි හෝ සමානද?):
- <=** (අඩු හෝ සමානද?):

31.5 තර්කන ක්‍රියාකරුවන් (Logical Operators)

කොන්දේසි (Conditions) කිහිපයක් එකට සම්බන්ධ කර තීරණ ගැනීමට මේවා භාවිතා කරයි.

- &&** (AND - සහ): සියලුම කොන්දේසි සත්‍ය නම් පමණක් true ලබා දෙයි. (උදා: ඔබට සල්ලි තියෙනවා සහ කඩේ විවෘතයි නම් පමණක් ඔබට බඩු ගත හැක).
- ||** (OR - හෝ): අවම වශයෙන් එක් කොන්දේසියක් හෝ සත්‍ය නම් true ලබා දෙයි. (උදා: ඔබට තේ තිබේ හෝ කෝපි තිබේ නම් ඔබට පානයක් ගත හැක).
- !** (NOT - නොව): සත්‍ය අගයක් අසත්‍ය ලෙසත්, අසත්‍ය අගයක් සත්‍ය ලෙසත් වෙනස් කරයි. (ප්‍රතිවිරුද්ධ අගය).

31.6 තන්තු සන්යෝජනය (String Concatenation)

+ ක්‍රියාකරු අංක සමඟ භාවිතා කළ විට එකතු කිරීම සිදු කරන අතර, අකුරු (Strings) සමඟ

භාවිතා කළ විට ඒවා එකට ඇලවීම සිදු කරයි.

```
let firstName = "Kasun";
let lastName = "Kalhara";
let fullName = firstName + " " + lastName;

console.log(fullName); // ප්‍රතිඵලය: "Kasun Kalhara"
```

31.7 ක්‍රියාකරුවන්ගේ ප්‍රමුඛතාවය (Operator Precedence)

එකම කේත පේළියක ක්‍රියාකරුවන් කිහිපයක් ඇති විට, මූලික ක්‍රියාත්මක වන්නේ කුමක්ද යන්න මෙයින් තීරණය වේ. මෙය පාසලේදී අප ඉගෙන ගත් "කැවොගුබෙඑආ" (BODMAS) නීතියට සමානය.

1. වරහන් () තුළ ඇති දේ මූලිකම සිදුවේ.
2. ගුණ කිරීම * සහ බෙදීම / එකතු කිරීමට පෙර සිදුවේ.

උදාහරණයක්: let result = 10 + 5 * 2; මෙහි පිළිතුර 30 නොව 20 වේ. මන්ද මූලිකම $5 * 2 = 10$ සිදු වී, පසුව $10 + 10 = 20$ සිදුවන බැවිනි. ඔබට මූලික එකතු කිරීමට අවශ්‍ය නම් $(10 + 5) * 2$ ලෙස ලිවිය යුතුය.

31.8 ASCII රූපසටහන: තර්කන ක්‍රියාකරුවන් (Logical Table)

AND (&) Logic:	OR () Logic:
True && True = True	True True = True
True && False = False	True False = True
False && True = False	False True = True
False && False = False	False False = False

31.9 සම්පූර්ණ කේත උදාහරණය (Comprehensive Code Example)

පහත කේතය ඔබගේ script.js ගොනුවේ ලියා ප්‍රතිඵල පරීක්ෂා කරන්න.

```
// ගණිතමය ක්‍රියාවන්
let a = 10;
let b = 3;

console.log("එකතුව: ", a + b); // 13

console.log("ඉතිරිය: ", a % b); // 1 (10 බෙදීම 3 හි ඉතිරිය)

// සංසන්දනය
let myAge = 16;
let requiredAge = "16";

console.log("සමානද (==): ", myAge == requiredAge); // true

console.log("සම්පූර්ණ සමානද (===): ", myAge === requiredAge); // false (Number vs String)

// තර්කනය (Logical)
let hasTicket = true;
let hasMoney = false;

// ටිකට් එක තියෙන්න ඕනේ සහ සල්ලි තියෙන්න ඕනේ
console.log("ඇතුළු විය හැකිද: ", hasTicket && hasMoney); // false

// ටිකට් එක හෝ සල්ලි දෙකෙන් එකක් තිබීම සෑහේ
console.log("ඇතුළු විය හැකිද (OR): ", hasTicket || hasMoney); // true
```

31.10 පොදු වැරදි (Common Mistakes)

1. = සහ == පටලවා ගැනීම: = භාවිතා කරන්නේ අගයක් ලබා දීමටයි (Assignment). == භාවිතා කරන්නේ සංසන්දනය කිරීමටයි (Comparison).

2. NaN (Not a Number): අංකයක් නොවන දෙයක් (උදා: "Hello") අංකයකින් ගුණ කිරීමට හෝ අඩු කිරීමට උත්සාහ කළහොත් ඔබට NaN යනුවෙන් පිළිතුරක් ලැබේ.

3. Strict Equality මග හැරීම: නිතරම === භාවිතා කරන්න. == භාවිතා කිරීමේදී ජාවාස්ක්‍රිප්ට් විසින් අප නොසිතන අයුරින් දත්ත වර්ග මාරු කර වැරදි ප්‍රතිඵල ලබා දිය හැක.

31.11 වෘත්තීය උපදෙස් (Professional Tips)

- පැහැදිලි බව (Readability): සංකීර්ණ ගණනය කිරීම් වලදී සැමවිටම වරහන් () භාවිතා කරන්න. එය කේතය කියවන වෙනත් අයෙකුට වටහා ගැනීමට පහසු වේ.

- Increment/Decrement: $x = x + 1$ ලෙස ලියනවාට වඩා $x++$ ලෙස ලිවීම නූතන ක්‍රමයයි.

31.12 සාරාංශය (Key Points)

- ක්‍රියාකරුවන් දත්ත සමඟ ගනුදෙනු කිරීමට භාවිතා කරන සලකුණු වේ.

- +, -, *, /, % යනු මූලික ගණිතමය ක්‍රියාකරුවන්ය.

- === සහ !== මගින් වඩාත් ආරක්ෂිතව දත්ත සංසන්දනය කළ හැක.

- &&, ||, ! මගින් තර්කන ගොඩනැගිය හැක.

- + ලකුණ අකුරු (Strings) දෙකක් එකට ඇලවීමට ද භාවිතා කරයි.

31.13 අභ්‍යාස (Exercises)

1. පහත දැක්වෙන කේතයන්හි ප්‍රතිඵලය (true/false) කුමක්දැයි සොයන්න:

- $10 > 5 \ \&\& \ 3 < 1$
- $5 === "5"$
- $10 !== 10 \ || \ 2 === 2$
- $!(5 > 10)$

2. පහත ගණනය කිරීමේ පිළිතුර සොයන්න: $let \ val = 20 - 2 * 5 + (10 / 2);$

3. විචල්‍යයන් දෙකක් සාදා ඒවා එකතු කර, එහි පිළිතුර තවත් විචල්‍යයක ගබඩා කර කන්සෝලය මත පෙන්වන්න.

4. මොඩියුලස් (%) ක්‍රියාකරු භාවිතා වන සැබෑ ලෝකයේ අවස්ථාවක් ගැන සිතා බලන්න.

(ඉභිය: ඉරට්ටේ/ඔත්තේ සංඛ්‍යා සෙවීම).

31.14 සම්මුඛ පරීක්ෂණ ප්‍රශ්න (Interview Questions)

- ප්‍රශ්නය: == සහ === අතර ඇති වෙනස කුමක්ද?

- පිළිතුර: == (Loose Equality) මගින් සංසන්දනය කරන්නේ අගය පමණි. එය අවශ්‍ය නම් දත්ත වර්ගය ස්වයංක්‍රීයව වෙනස් කරයි (Type Coercion). නමුත් === (Strict Equality) මගින් අගය සහ දත්ත වර්ගය යන දෙකම සමාන දැයි පරීක්ෂා කරයි. වෘත්තීය මට්ටමේදී වඩාත් ආරක්ෂිත === භාවිතා කිරීම නිර්දේශ කෙරේ.

මිලභ පරිච්ඡේදයෙන් අපි ජාවාස්ක්‍රිප්ට් වල ඉතාමත් තීරණාත්මක කොටසක් වන "පාලන

ඉතාමත් විස්තරාත්මකව ඉගෙන ගනිමු.

නිස්දෙවන පරිච්ඡේදය: පාලන ව්‍යුහයන් - තීරණ ගැනීම (Control Flow - Decisions)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

1. පාලන ව්‍යුහයන් (Control Flow) යනු කුමක්දැයි අවබෝධ කර ගැනීම.
2. if, else if සහ else ප්‍රකාශන භාවිතා කර තීරණ ගන්නා ආකාරය.
3. ජාවාස්ක්‍රිප්ට් හි "සත්‍ය වැනි" (Truthy) සහ "අසත්‍ය වැනි" (Falsy) අගයන් හඳුනා ගැනීම.
4. switch ප්‍රකාශනය භාවිතා කර බොහෝ තේරීම් කළමනාකරණය කරන ආකාරය.
5. තෘතීයික ක්‍රියාකරු (Ternary Operator) හෙවත් කෙටි if ක්‍රමය ඉගෙන ගැනීම.
6. කොන්දේසි එකක් ඇතුළත තවත් එකක් ලිවීම (Nested Conditions).
7. සැබෑ ලෝකයේ උදාහරණයක් ලෙස "ATM යන්ත්‍රයක" තර්කනය ගොඩනැගීම.

32.1 හැඳින්වීම: පාලන ව්‍යුහයක් යනු කුමක්ද?

සාමාන්‍යයෙන් පරිගණකයක් කරන්නේ අප ලියන කේතය පේළියෙන් පේළිය කියවාගෙන යාමයි. නමුත්

සෑම අවස්ථාවකම අපට සියලුම ජේළි ක්‍රියාත්මක කිරීමට අවශ්‍ය වන්නේ නැත. සමහර විට යම්

"කොන්දේසියක්" (Condition) මත පදනම්ව කේතයේ යම් කොටසක් ක්‍රියාත්මක කිරීමටත්, තවත් කොටසක් මඟ හැරීමටත් අපට සිදු වේ.

මෙලෙස කේතය ක්‍රියාත්මක වන පිළිවෙළ පාලනය කිරීමට භාවිතා කරන ක්‍රමවේදයට Control

Flow (පාලන ප්‍රවාහය) යැයි කියනු ලැබේ.

සැබෑ ලෝකයේ උදාහරණයක්: උදෑසන අවදි වූ පසු ඔබ තීරණයක් ගනී:

- කොන්දේසිය: අද වැස්සද?
- තීරණය 1 (සත්‍ය නම්): ඔව්, වැස්ස නම් කුඩයක් අතට ගන්න.
- තීරණය 2 (අසත්‍ය නම්): නැහැ, වැස්ස නැත්නම් අව කණ්ණාඩියක් දමන්න.

පරිගණක වැඩසටහන් ද ක්‍රියා කරන්නේ මෙලෙස තීරණ ගැනීමෙනි.

32.2 if ප්‍රකාශනය (The if Statement)

මෙය තීරණ ගැනීම සඳහා ඇති මූලිකම ක්‍රමයයි. යම් කොන්දේසියක් සත්‍ය (true) නම් පමණක් වරහන් ඇතුළත ඇති කේතය ක්‍රියාත්මක වේ.

ව්‍යුහය (Syntax):

```
if (කොන්දේසිය) {
```

```
    // කොන්දේසිය සත්‍ය නම් මෙම කේතය
    ක්‍රියාත්මක වේ
```

```
}
```

උදාහරණය:

```
let age = 20;
```

```
if (age >= 18) {
```

```
    console.log("ඔබට ජන්දය ප්‍රකාශ කළ හැක.");
```

```
}
```

32.3 else ප්‍රකාශනය (The else Statement)

කොන්දේසිය අසත්‍ය (false) වන විට කුමක් සිදුවිය යුතුදැයි පැවසීමට else භාවිතා කරයි.

උදාහරණය:

```
let age = 15;
```

```
if (age >= 18) {
```

```
    console.log("ඔබ වැඩිහිටියෙකි.");
```

```
} else {
```

```
    console.log("ඔබ තවම දරුවෙකි.");
```

```
}
```

32.4 else if ප්‍රකාශනය (Multiple Conditions)

අපට පරීක්ෂා කිරීමට කොන්දේසි එකකට වඩා වැඩි ගණනක් ඇති විට else if භාවිතා කරයි.

සැබෑ ලෝකයේ උදාහරණය: රථවාහන සංඥා (Traffic Lights)

```
let light = "yellow";
```

```
if (light === "red") {
```

```
    console.log("තවතින්න!");
```

```
} else if (light === "yellow") {
```

```
    console.log("සුදානම් වන්න.");
```

```
} else if (light === "green") {
```

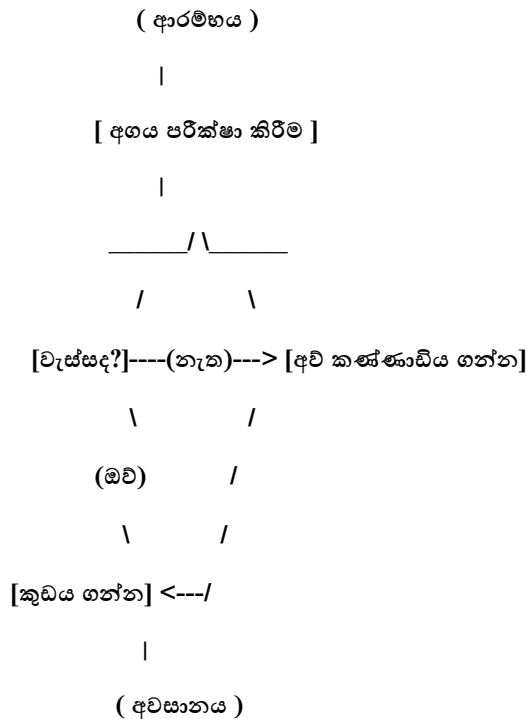
```
    console.log("ඉදිරියට යන්න.");
```

```
} else {
```

```
    console.log("සංඥා පද්ධතිය ක්‍රියා විරහිතයි.");
```

```
}
```


32.5 ප්‍රවාහ සටහන (ASCII Flowchart)



32.6 සත්‍ය වැනි සහ අසත්‍ය වැනි අගයන් (Truthy & Falsy)

ජාවාස්ක්‍රිප්ට් හි සෑම අගයකටම සහජයෙන්ම සත්‍ය හෝ අසත්‍ය බවක් ඇත. කොන්දේසියක් තුළ භාවිතා කළ විට එම අගය **true** ලෙස හෝ **false** ලෙස හැසිරේ.

Falsy Values (අසත්‍ය ලෙස සලකන අගයන්):

1. false
2. 0 (බිත්දුට)
3. "" (හිස් අකුරු වැල්)
4. null
5. undefined
6. NaN (Not a Number)

අනෙක් සෑම අගයක්ම (උදා: ඕනෑම අංකයක්, ඕනෑම අකුරක්) **Truthy** ලෙස සලකනු ලැබේ.

උදාහරණය:

```
let myName = ""; // හිස් බැවින් මෙය Falsy වේ
if (myName) {
```

```
    console.log("නමක් තිබේ.");
```

```
} else {
```

```
    console.log("නම හිස් ය."); // මෙය ක්‍රියාත්මක වේ
```

```
}
```

32.7 switch ප්‍රකාශනය (The switch Statement)

පරීක්ෂා කිරීමට බොහෝ අගයන් ඇති විට (උදා: සතියේ දින 7 පරීක්ෂා කිරීමට), **if...else if** ලිවීම වෙනසකරය. ඒ සඳහා වඩාත් පිරිසිදු ක්‍රමය **switch** වේ.

විද්‍යුතය:

```
let dayNumber = 2;
```

```
switch (dayNumber) {
```

```
    case 1:
```

```
        console.log("සඳුදා");
```

```
        break;
```

```
    case 2:
```

```
        console.log("අඟහරුවාදා");
```

```
        break;
```

```
    case 3:
```

```
        console.log("බදාදා");
```

```
        break;
```

```
    default:
```

```
        console.log("වැරදි අංකයකි.");
```

```
}
```

- **break:** මෙය ඉතා වැදගත් වේ. එක **case** එකක් නිවැරදි වූ පසු ඉතිරි ඒවා පරීක්ෂා නොකර පිටතට යාමට මෙය භාවිතා කරයි.

- **default:** කිසිම **case** එකක් සමාන නොවූණහොත් ක්‍රියාත්මක වන කොටසයි.

32.8 තෘතීයික ක්‍රියාකරු (Ternary Operator)

මෙය සරල if...else එකක් එක පේළියකින් ලිවීමට ඇති ක්‍රමයයි.

වියුහය: කොන්දේසිය ? සත්‍ය_නම් : අසත්‍ය_නම්;

උදාහරණය:

```
let age = 20;
```

```
let message = (age >= 18) ? "වැඩිහිටි" :  
"කුඩා";
```

```
console.log(message); // "වැඩිහිටි"
```

32.9 පොදු වැරදි (Common Mistakes)

- සමාන ලකුණ වැරදීම: if (x = 5) ලෙස ලිවීම. මෙහිදී x ට 5 ආදේශ වනවා මිස සංසන්දනයක් සිදු නොවේ. සැමවිටම if (x === 5) භාවිතා කරන්න.
- Switch එකේ Break අමතක වීම: break නොලියුවහොත්, එක් case එකක් නිවැරදි වුවද ඊළඟ case වල ඇති කේතයන් ද ක්‍රියාත්මක වේ.
- Curly Brackets { } අමතක වීම: if එකකින් පසු ක්‍රියාත්මක වීමට පේළි එකකට වඩා තිබේ නම් අනිවාර්යයෙන්ම { } භාවිතා කළ යුතුය.

32.10 වෘත්තීය උපදෙස් (Professional Tips)

- Guard Clauses: දිගු if...else ලියනවාට වඩා වැරදි අවස්ථාවන් මුලින්ම පරීක්ෂා කර කේතයෙන් ඉවත් වීම (Return) වෘත්තීය ක්‍රමයකි.

- Strict Equality: සැමවිටම === භාවිතා කරන්න. එය දත්ත වර්ගය ද පරීක්ෂා කරන බැවින් අනපේක්ෂිත දෝෂ වළක්වයි.

32.11 සාරාංශය (Key Points)

- if භාවිතා කර තීරණ ගනු ලැබේ.
- else භාවිතා කර අසත්‍ය අවස්ථාව හසුරුවනු ලැබේ.
- else if මගින් අතිරේක කොන්දේසි පරීක්ෂා කරයි.
- 0, "", null, undefined යනු Falsy අගයන්ය.
- switch බොහෝ තේරීම් කළමනාකරණය කිරීමට සුදුසුය.

- Ternary Operator එක පේළියේ තීරණ සඳහා භාවිතා වේ.

32.12 අභ්‍යාස (Exercises)

- අංකයක් ඉරට්ටේ ද (Even) ඔත්තේ ද (Odd) කියා පරීක්ෂා කරන කේතයක් ලියන්න. (ඉඟිය: % 2 === 0 භාවිතා කරන්න).
- සිසුවෙකුගේ ලකුණු මත පදනම්ව ශ්‍රේණිය (Grade) පෙන්වන කේතයක් ලියන්න:

- 75 ට වැඩි නම් "A"
- 65 ට වැඩි නම් "B"
- 50 ට වැඩි නම් "C"
- අනෙක් අයට "F"

- switch භාවිතා කර මාසයක අංකය (1-12) දුන් විට එම මාසයේ නම පෙන්වන කේතයක් ලියන්න.
- Ternary Operator එක භාවිතා කර පරිශීලකයා ලොගින් වී ඇත්නම් (isLoggedIn = true) "Welcome" ලෙසත්, නැත්නම් "Please Log in" ලෙසත් පෙන්වන කේතයක් ලියන්න.

32.13 ප්‍රායෝගික ව්‍යාපෘතිය (Mini Project)

ව්‍යාපෘතිය: ATM මුදල් ගැනීමේ තර්කනය පහත විවලයන් සාදා දත්ත පරීක්ෂා කරන්න:

- let balance = 5000; (ගිණුමේ ඇති මුදල)
- let withdrawalAmount = 2000; (ගැනීමට අවශ්‍ය මුදල)
- let pinEntered = 1234;
- const correctPin = 1234;

තර්කනය:

- මුලින්ම PIN එක නිවැරදි දැයි බලන්න.
- නිවැරදි නම්, ගැනීමට අවශ්‍ය මුදල ගිණුමේ ඇති මුදලට වඩා අඩු දැයි බලන්න.
- මුදල තිබේ නම්, "මුදල් ලබා ගැනීම සාර්ථකයි" සහ ඉතිරි මුදල පෙන්වන්න.
- මුදල මදි නම්, "ප්‍රමාණවත් මුදල් නොමැත" ලෙස පෙන්වන්න.
- PIN එක වැරදි නම්, "වැරදි PIN අංකයකි" ලෙස පෙන්වන්න.

නිස්කුන්වන පරිච්ඡේදය: පුඩු හෙවත් ලූප්ස් - නැවත නැවත සිදුවන ක්‍රියා (Loops)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

1. පුඩු (Loops) යනු කුමක්ද සහ ක්‍රමලේඛනයේදී ඒවායේ වැදගත්කම තේරුම් ගැනීම.
2. for ලූප් එකෙහි ව්‍යුහය සහ එය ක්‍රියා කරන ආකාරය පියවරෙන් පියවර වටහා ගැනීම.
3. while ලූප් එක සහ එය භාවිතා කළ යුතු අවස්ථා හඳුනා ගැනීම.
4. do...while ලූප් එක සහ අනෙකුත් ලූප් අතර ඇති වෙනස ඉගෙන ගැනීම.
5. අනන්ත පුඩු (Infinite Loops) යනු මොනවාද සහ ඒවායින් බේරෙන ආකාරය.
6. break සහ continue යන විධානයන් මගින් ලූප් එකක් පාලනය කරන ආකාරය.
7. සැබෑ ලෝකයේ උදාහරණයක් ලෙස "ගුණන වගුවක්" ලූප් එකක් හරහා නිර්මාණය කිරීම.

33.1 හැඳින්වීම: පුඩු (Loops) යනු මොනවාද?

සිතන්න ඔබේ ගුරුවරයා ඔබට දඩුවමක් ලෙස "මම නැවත පන්තියේ කතා නොකරමි" යන වැකිය 100 වතාවක් පොතේ ලියන්න යැයි පවසනවා කියා. මෙය මිනිසෙකුට ඉතාමත් කම්මැලි සහ වෙහෙසකර වැඩකි. නමුත් පරිගණකයකට මෙය තත්පරයකින් දහයෙන් පංගුවකදී කළ හැක.

ක්‍රමලේඛනයේදී (Programming), එකම කේත කොටසක් නිශ්චිත වාර ගණනක් හෝ යම් කොන්දේසියක් සත්‍ය වන තෙක් නැවත නැවත ක්‍රියාත්මක කිරීමට භාවිතා කරන ක්‍රමවේදය Loop (පුඩුව) ලෙස හැඳින්වේ.

ඇයි අපට ලූප්ස් අවශ්‍ය වන්නේ?

- කාලය ඉතිරි කර ගැනීමට: එකම දේ 100 වතාවක් ටයිප් කිරීමට අවශ්‍ය නැත.
- කේතය පිරිසිදුව තබා ගැනීමට (DRY - Don't Repeat Yourself): කේතය කෙටි වන තරමට වැරදි (Bugs) අඩු වේ.
- දත්ත ලැයිස්තු සමඟ වැඩ කිරීමට: වෙබ් අඩවියක ඇති දත්ත ගණනක් නිෂ්පාදන (Products) පෙන්වීමට ලූප්ස් අත්‍යවශ්‍ය වේ.

33.2 for ලූප් එක (The for Loop)

ජාවාස්ක්‍රිප්ට් වල බහුලවම භාවිතා වන ලූප් එක මෙයයි. යම් කාර්යයක් නිශ්චිත වාර ගණනක් (උදා: 10 වතාවක්) කිරීමට අවශ්‍ය විට මෙය භාවිතා කරයි.

ව්‍යුහය (Syntax):

```
for (ආරම්භය; කොන්දේසිය; වැඩිවීම/අඩුවීම) {  
    // නැවත නැවත ක්‍රියාත්මක විය යුතු කේතය  
}
```

කොටස් 3 පැහැදිලි කිරීම:

1. ආරම්භය (Initialization): මෙහිදී අපි ගණන් කිරීම ආරම්භ කරන විචල්‍යය සාදන්නෙමු. සාමාන්‍යයෙන් මෙය `let i = 0;` ලෙස ලියයි.
2. කොන්දේසිය (Condition): මෙම ලූප් එක දිගටම ක්‍රියාත්මක විය යුතුද නැද්ද යන්න තීරණය කරයි. මෙය සත්‍ය (true) වන තෙක් ලූප් එක කැරකෙයි.
3. වැඩිවීම/අඩුවීම (Increment/Decrement): සෑම වාරයක් (Iteration) අවසානයේදීම ගණන් කරන විචල්‍යය වෙනස් කරයි.

සරල උදාහරණයක්: 1 සිට 5 දක්වා අංක කන්සෝලයේ මුද්‍රණය කරමු.

```
for (let i = 1; i <= 5; i++) {  
    console.log("අංකය: " + i);  
}
```

පියවරෙන් පියවර ක්‍රියාවලිය:

- වාරය 1: i යනු 1 කි. `1 <= 5` ද? ඔව් (true). "අංකය: 1" මුද්‍රණය වේ. i එකකින් වැඩි වී 2 වේ.
- වාරය 2: i යනු 2 කි. `2 <= 5` ද? ඔව් (true). "අංකය: 2" මුද්‍රණය වේ. i එකකින් වැඩි වී 3 වේ.
- ...
- වාරය 6: i යනු 6 කි. `6 <= 5` ද? නැත (false). ලූප් එක නතර වේ.

33.3 while ලූප් එක (The while Loop)

යම් කාර්යයක් කී වතාවක් කළ යුතුදැයි අප හරියටම නොදන්නා නමුත්, යම් කොන්දේසියක් සත්‍ය වන

තෙක් එය කළ යුතු විට **while** ලූප් එක භාවිතා කරයි.

විද්‍යුතය:

```
while (කොන්දේසිය) {
```

```
    // කේතය
```

```
}
```

උදාහරණයක්:

```
let energy = 3;
```

```
while (energy > 0) {
```

```
    console.log("මම දුවනවා... ශක්තිය: " + energy);
```

```
    energy--; // ශක්තිය 1 බැගින් අඩු කරයි
```

```
}
```

```
console.log("මම වෙහෙසට පත් වුණා.");
```

වැදගත්: **while** ලූප් එකක් භාවිතා කරන විට, කොන්දේසිය කවදා හෝ අසත්‍ය (**false**) වන ලෙස කේතය සකස් කළ යුතුය. නැතිනම් එය කිසිදා නතර නොවන **Infinite Loop** එකක් බවට පත් වී ඔබේ පරිගණකය හිර විය හැක.

33.4 do...while ලූප් එක

මෙය **while** ලූප් එකට ඉතා සමානයි. නමුත් මෙහි ඇති විශේෂත්වය නම්, කොන්දේසිය පරීක්ෂා කිරීමට පෙර අවම වශයෙන් එක් වතාවක් හෝ කේතය ක්‍රියාත්මක වීමයි.

උදාහරණය:

```
let x = 10;
```

```
do {
```

```
    console.log("මෙය අවම වශයෙන් එක් වරක්වත් පෙනෙනු ඇත.");
```

```
    x++;
```

```
} while (x < 5);
```

මෙහි $x < 5$ යන්න අසත්‍ය වුවද, **do** කොටස මුලින්ම ඇති බැවින් එක් වරක් පරීක්ෂා වේ.

33.5 ලූප් පාලනය: break සහ continue

සමහර අවස්ථාවලදී අපට ලූප් එකක් නියමිත කාලයට පෙර නැවැත්වීමට හෝ යම් කොටසක් මඟ හැරීමට සිදු වේ.

1. **break:** ලූප් එකෙන් සම්පූර්ණයෙන්ම පිටතට පැමිණීමට භාවිතා කරයි.

```
for (let i = 1; i <= 10; i++) {
```

```
    if (i === 5) break; // 5 ට ආ විට ලූප් එක නවතී
```

```
    console.log(i);
```

```
}
```

2. **continue:** වත්මන් වාරය මඟ හැර ඊළඟ වාරයට යාමට භාවිතා කරයි.

```
for (let i = 1; i <= 5; i++) {
```

```
    if (i === 3) continue; // 3 මඟ හරියි
```

```
    console.log(i); // 1, 2, 4, 5 මුද්‍රණය වේ
```

```
}
```

33.6 ASCII රූපසටහන: Loop එකක ගමන

```
[ පටන්ගැන්ම ]
```

```
|
```

```
-----> [ කොන්දේසිය පරීක්ෂාව ] <-----
```

```
| |
```

```
| (සත්‍ය ද?) ---- (නැත) ----> [ අවසානය ]
```

```
| |
```

```
| (ඔව්)
```

```
| |
```

```
| [ කේතය ක්‍රියාත්මක වේ ]
```

```
| |
```

```
| [ අගය වෙනස් වේ (i++) ]
```

```
|_____|
```

33.7 පොදු වැරදි (Common Mistakes)

1. Infinite Loops (අනන්ත පුඩු): කොන්දේසිය කවදාවත් අසත්‍ය නොවීම. (උදා: $i++$ ලියන්න අමතක වීම). එවිට බ්‍රවුසරය හිර වේ (Crash).
2. Off-by-one Error: ලූප් එක එක් වාරයක් අඩුවෙන් හෝ වැඩියෙන් කැරකීම. (උදා: < 10 සහ ≤ 10 අතර වෙනස).
3. වැරදි කොන්දේසි: $i > 10$ වෙනුවට $i < 10$ ලෙස වැරදි දිශාවට කොන්දේසිය ලිවීම.

33.8 වෘත්තීය උපදෙස් (Professional Tips)

- අරාවන් (Arrays) සමඟ භාවිතය: සැබෑ වැඩබිම්කදී ලූප්ස් වැඩිපුරම භාවිතා වන්නේ අරාවක (Array) ඇති දත්ත කියවීමටයි. (මෙය **35 වන පරිච්ඡේදයේදී** ගැඹුරින් ඉගෙන ගනිමු).
- Let vs Var: ලූප් එකක් ඇතුළත විචල්‍යයන් සාදන විට සැමවිටම let භාවිතා කරන්න. එය වඩාත් ආරක්ෂිතයි.

33.9 සාරාංශය (Key Points)

- ලූප්ස් මගින් කේතයක් නැවත නැවත ක්‍රියාත්මක කරයි.
- for ලූප් එක වාර ගණන දන්නා විට භාවිතා කරයි.
- while ලූප් එක කොන්දේසියක් මත පදනම් වේ.
- break ලූප් එක නවත්වයි; continue වත්මන් පියවර මඟ හරියි.
- සැමවිටම Infinite loop ඇති නොවන සේ වගබලා ගන්න.

33.10 අභ්‍යාස (Exercises)

1. 1 සිට 20 දක්වා ඇති ඉරට්ටේ සංඛ්‍යා (Even Numbers) පමණක් මුද්‍රණය කරන for ලූප් එකක් ලියන්න.
2. 10 සිට 1 දක්වා පසුපසට (Reverse) ගණන් කරන for ලූප් එකක් ලියන්න.
3. while ලූප් එකක් භාවිතා කර 1 සිට 100 දක්වා සංඛ්‍යාවල එකතුව (Total Sum) සොයන කේතයක් ලියන්න.
4. "JavaScript" යන වචනය 10 වතාවක් මුද්‍රණය කරන ලූප් එකක් ලියන්න.

33.11 ප්‍රායෝගික ව්‍යාපෘතිය (Mini Project)

ව්‍යාපෘතිය: ඕනෑම අංකයක ගුණන වගුව සාදන්න (Multiplication Table)

පරිශීලකයා ලබා දෙන අංකයක (උදා: 5) ගුණන වගුව 1 සිට 10 දක්වා මුද්‍රණය කරන්න.

ප්‍රතිඵලය මෙලෙස දිස්විය යුතුය: $5 \times 1 = 5$ $5 \times 2 = 10$... 5×10

= 50

උපදෙස්:

- let num = 5; ලෙස විචල්‍යයක් සාදන්න.
- for ලූප් එකක් භාවිතා කර 1 සිට 10 දක්වා කැරකෙන්න.
- ලූප් එක තුළදී num * i ගණනය කර ලස්සනට පෙන්වන්න.

මිළඟ පරිච්ඡේදයෙන් අපි ජාවාස්ක්‍රිප්ට් වල ඉතාමත් වැදගත් සහ බලවත් කොටසක් වන "ශ්‍රිත හෙවත් ෆන්ක්ෂන්ස්" (Functions) ගැන ඉතාමත් විස්තරාත්මකව ඉගෙන ගනිමු.

නිස්භ්‍යතරවන පරිච්ඡේදය: ශ්‍රිත හෙවත් ෆන්ක්ෂන්ස් - කේතය නැවත භාවිතා කිරීම (Functions - Code Reusability)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

1. ශ්‍රිතයක් (Function) යනු කුමක්ද සහ එය ක්‍රමලේඛනයේදී අත්‍යවශ්‍ය වන්නේ ඇයිද යන්න තේරුම් ගැනීම.
2. DRY (Don't Repeat Yourself - ඔබම නැවත සිදු නොකරන්න) සංකල්පය වටහා ගැනීම.
3. ශ්‍රිතයක් ප්‍රකාශ කරන ආකාරය (Function Declaration) සහ එය ක්‍රියාත්මක කරන ආකාරය (Function Calling/Invoking).
4. පරාමිතීන් (Parameters) සහ තර්කයන් (Arguments) අතර වෙනස හඳුනා ගැනීම.
5. return ප්‍රකාශනය (Return Statement) භාවිතා කර ප්‍රතිඵලයක් ලබා ගන්නා ආකාරය.

6. ශ්‍රිත විෂය පථය (Function Scope) හෙවත් විචල්‍යයන්ගේ පාලන සීමාවන් තේරුම් ගැනීම.
7. සැබෑ ලෝකයේ උදාහරණයක් ඇසුරින් සරල "කෝපි යන්ත්‍රයක්" තර්කනය ගොඩනැගීම.

34.1 හැඳින්වීම: ශ්‍රිතයක් යනු කුමක්ද?

ක්‍රමලේඛනය කිරීමේදී අපට එකම කේත කොටස (Logic) වෙබ් අඩවියේ විවිධ ස්ථානවලදී භාවිතා කිරීමට සිදු වේ. උදාහරණයක් ලෙස,

පරිශීලකයෙකුගේ ලකුණු මත පදනම්ව ඔහු සමත්ද අසමත්ද කියා පරීක්ෂා කරන කේතය සිතන්න. මෙම කේතය සිසුන් 100 දෙනෙකු සඳහා පරීක්ෂා කිරීමට නම්, අප එම කේතය 100 වතාවක් ලිවිය යුතුද?

නැත. ඒ සඳහා අප භාවිතා කරන්නේ Function (ශ්‍රිතය) යි.

ශ්‍රිතයක් යනු යම් නිශ්චිත කාර්යයක් ඉටු කිරීම සඳහා වෙන් කරන ලද, නැවත නැවත භාවිතා කළ හැකි කේත සමූහයකි. ඔබ එය එක් වරක් ලියා තබා, අවශ්‍ය ඕනෑම වාර ගණනක් එහි නම භාවිතා කර ක්‍රියාත්මක කළ හැකිය.

34.2 සැබෑ ලෝකයේ උපමාව: තේ යන්ත්‍රය (The Tea Machine)

ශ්‍රිතයක් ක්‍රියා කරන ආකාරය තේරුම් ගැනීමට "තේ යන්ත්‍රයක්" ගැන සිතන්න.

1. Input (ආදානය/පරාමිතීන්): ඔබ යන්ත්‍රයට තේ කොළ, සීනි සහ වතුර ලබා දෙයි.
2. Process (සැකසීම/තර්කනය): යන්ත්‍රය ඇතුළත වතුර රත් වී, තේ කොළ සහ සීනි මිශ්‍ර වී තේ සෑදේ. (මේ කේතය යන්ත්‍රය තුළ ස්ථාවරව ඇත).
3. Output (ප්‍රතිදානය/Return): අවසානයේදී ඔබට උණුසුම් තේ කෝප්පයක් ලැබේ.

ඔබට තේ අවශ්‍ය සෑම වතාවකම යන්ත්‍රය අලුතින් සෑදීමට අවශ්‍ය නැත. ඔබ කළ යුත්තේ ද්‍රව්‍ය (Inputs) ලබා දී බොත්තම එබීම (Call) පමණි.

34.3 ශ්‍රිතයක ව්‍යුහය (Function Syntax)

ජාවාස්ක්‍රිප්ට් හි ශ්‍රිතයක් ලිවීමට පියවර දෙකක් ඇත:

පියවර 1: ශ්‍රිතය ප්‍රකාශ කිරීම (Defining/Declaring a Function)

මෙහිදී අපි ශ්‍රිතය සාදා එයට නමක් ලබා දෙන්නෙමු.

```
function makeTea() {  
  
    console.log("වතුර රත් කරනවා...");  
  
    console.log("තේ කොළ සහ සීනි දමනවා...");  
  
    console.log("තේ සුදානම්!");  
  
}
```

- function: මෙය ශ්‍රිතයක් බව පවසන මූලපදය (Keyword).

- makeTea: ශ්‍රිතයට අප ලබා දුන් නම (camelCase ක්‍රමයට තිබිය යුතුය).

- (): මෙයට වරහන් (Parentheses) යැයි කියනු ලැබේ. (දත්ත ඇතුළු කරන්නේ මෙතැනිනි).

- {}: කරලි වරහන් (Curly Braces) ඇතුළත ඇත්තේ ශ්‍රිතය ක්‍රියාත්මක වන විට සිදුවිය යුතු දේය (Body).

පියවර 2: ශ්‍රිතය කැඳවීම (Calling/Invoking a Function)

ශ්‍රිතය ලිවූ පමණින් එය ක්‍රියාත්මක නොවේ. එය ක්‍රියාත්මක කිරීමට එහි නම භාවිතා කළ යුතුය.

makeTea(); // දැන් කන්සෝලයේ ඉහත පණිවිඩ දිස්වේ.

34.4 පරාමිතීන් සහ තර්කයන් (Parameters and Arguments)

ශ්‍රිතයකට දත්ත ලබා දීමට අප භාවිතා කරන්නේ Parameters ය.

- Parameter (පරාමිතිය): ශ්‍රිතය සාදන විට වරහන් තුළ ලියන විචල්‍ය නාමයයි. (මෙය "පෙට්ටියක්" වැනිය).

- Argument (තර්කය/දත්තය): ශ්‍රිතය ක්‍රියාත්මක කරන විට ඇත්තටම ලබා දෙන අගයයි. උදාහරණය:

```
function sayHello(userName) { // userName යනු  
    පරාමිතියයි
```

```
    console.log("ආයුබෝවන් " + userName + "!");
```

```
}
```

sayHello("කසුන්"); // "කසුන්" යනු තර්කයයි (Argument)

sayHello("නිමල්"); // තවත් තර්කයක්

34.5 අගයක් ආපසු ලබා දීම (The return Statement)

බොහෝ විට අපට අවශ්‍ය වන්නේ ශ්‍රිතයක් තුළ යමක් ගණනය කර එම පිළිතුර ලබා ගැනීමයි. ඒ සඳහා **return** මූලපදය භාවිතා කරයි.

වැදගත්: **return** විධානය ක්‍රියාත්මක වූ සැණින් ශ්‍රිතය එතැනින් නතර වේ. ඊට පහළින් ඇති කිසිදු කේතයක් ක්‍රියාත්මක නොවේ.

උදාහරණය: අංක දෙකක එකතුව සෙවීම

```
function addNumbers(a, b) {
```

```
    let sum = a + b;
```

```
    return sum; // පිළිතුර ආපසු එවයි
```

```
}
```

```
let result = addNumbers(10, 20); // result  
තුළට 30 යන අගය ලැබේ
```

```
console.log("පිළිතුර: " + result);
```

34.6 ASCII රූපසටහන: ශ්‍රිතයක ගමන

පරිශීලකයා (User)	ශ්‍රිතය (Function)
-----	-----
[අගයන් යවයි] ----->	[පරාමිතීන් ලබා ගනී]
	[තර්කනය ක්‍රියාත්මක වේ]
[ප්‍රතිඵලය ලබයි] <-----	[Return කරයි]

34.7 ශ්‍රිත විෂය පථය (Function Scope)

ශ්‍රිතයක් ඇතුළත සාදන විචල්‍යයන් එම ශ්‍රිතයෙන් පිටතදී භාවිතා කළ නොහැක. මෙය **Local Scope** ලෙස හැඳින්වේ.

```
function myScope() {
```

```
    let message = "මම ඇතුළේ විතරයි ඉන්නේ";
```

```
    console.log(message); // මෙය වැඩ කරයි
```

```
}
```

```
// console.log(message); // මෙතැනදී ERROR  
එකක් ලැබේ.
```

34.8 පොදු වැරදි (Common Mistakes)

1. **Return** කිරීමට අමතක වීම: ඔබට ශ්‍රිතයකින් පිළිතුරක් අවශ්‍ය නම් **return** ලියන්න. නැතිනම් ප්‍රතිඵලය **undefined** ලෙස ලැබේ.
2. **වරහන් ()** අමතක වීම: ශ්‍රිතයක් කැඳවීමේදී නමට පසු වරහන් යෙදිය යුතුය. **makeTea**; ලෙස ලිවීමෙන් ශ්‍රිතය ක්‍රියාත්මක නොවේ.
3. නම පටලවා ගැනීම: ශ්‍රිතය සාදන විට දුන් නමම (**Capital/Simple** අකුරු සහිතව) කැඳවීමේදී භාවිතා කළ යුතුය.

34.9 වෘත්තීය උපදෙස් (Professional Tips)

- **DRY Principle:** ඔබ කේතයේ එකම දෙය දෙපාරකට වඩා ලියනවා නම්, වහාම එය ශ්‍රිතයක් බවට පත් කරන්න.
- **One Function, One Task:** එක් ශ්‍රිතයකින් එක වැඩක් පමණක් කිරීමට උත්සාහ කරන්න. (උදා: **sendEmailAndCalculateTax** ලෙස ලියනවාට වඩා ශ්‍රිත දෙකක් ලිවීම හොඳය).
- **Naming:** ශ්‍රිතවල නම සැමවිටම ක්‍රියා පදයකින් (**Verb**) ආරම්භ කරන්න. (උදා: **calculateTotal**, **fetchData**, **getName**).

34.10 සාරාංශය (Key Points)

- ශ්‍රිත යනු කේතය නැවත භාවිතා කිරීමේ ක්‍රමයයි.
- **function** keyword එකෙන් ආරම්භ වේ.
- **Parameters** යනු **Input** වන අතර **Return** යනු **Output** වේ.
- ශ්‍රිතයක් කැඳවීමට (**Call**) එහි නම සහ වරහන් () භාවිතා කළ යුතුය.

34.11 අභ්‍යාස (Exercises)

1. අංකයක් ලබා දුන් විට එහි වර්ගඵලය (Square) ගණනය කර ආපසු ලබා දෙන (return) ශ්‍රිතයක් ලියන්න.
2. පරිශීලකයාගේ නම සහ උපන් වර්ෂය පරාමිතීන් ලෙස ගෙන, ඔහුගේ වයස ගණනය කර "X වන ඔබගේ වයස Y කි" ලෙස පණිවිඩයක් මුද්‍රණය කරන ශ්‍රිතයක් ලියන්න.
3. මවුසය බොත්තමක් මතට ගෙන යන විට (onclick) alert එකක් පෙන්වන සරල HTML සහ JS ශ්‍රිතයක් සාදන්න.

34.12 ප්‍රායෝගික ව්‍යාපෘතිය (Mini Project)

ව්‍යාපෘතිය: සරල ඒකක පරිවර්තකය (Unit Converter)

පහත කාර්යයන් කරන ශ්‍රිත තුනක් ලියන්න:

1. සෙල්සියස් (Celsius) අගයක් ෆැරන්හයිට් (Fahrenheit) බවට හැරවීම.
2. කිලෝමීටර් අගයක් මීටර් බවට හැරවීම.
3. රූපියල් අගයක් ඩොලර් බවට හැරවීම (වර්තමාන අනුපාතය භාවිතා කරන්න).

එක් එක් ශ්‍රිතය කැඳවා විවිධ අගයන් ලබා දී ප්‍රතිඵලය කන්සෝලයේ පෙන්වන්න.

මූලික පරිච්ඡේදයෙන් අපි ජාවාස්ක්‍රිප්ට් වල දත්ත සමූහයක් එක තැනක ගබඩා කරන "අරාවන්" (Arrays) ගැන ඉතාමත් විස්තරාත්මකව ඉගෙන ගනිමු.

නිස්පස්ථ පරිච්ඡේදය: අරාවන් - දත්ත ලැයිස්තු කළමනාකරණය (Arrays)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

1. අරාවක් (Array) යනු කුමක්ද සහ එකම විචල්‍යයක දත්ත රාශියක් ගබඩා කිරීමේ වැදගත්කම වටහා ගැනීම.
2. අරාවක් ප්‍රකාශ කරන (Declare) ආකාරය සහ එයට දත්ත ඇතුළත් කරන ආකාරය.

3. දර්ශකය (Index) යනු කුමක්දැයි තේරුම් ගැනීම සහ 0 සිට ගණන් කිරීමේ වැදගත්කම.
4. අරාවක ඇති දත්ත ලබා ගැනීම (Accessing), වෙනස් කිරීම (Modifying) සහ පරීක්ෂා කිරීම.
5. අරාවක දිග (Length) සෙවීම.
6. මූලික අරා ක්‍රමවේද (Basic Array Methods) වන push, pop, shift, සහ unshift භාවිතය.
7. අරාවක් සහ ලූප් (Loops) එකක් එකට භාවිතා කරන ආකාරය.

35.1 හැඳින්වීම: අරාවක් යනු කුමක්ද? (What is an Array?)

අපි මෙතෙක් ඉගෙන ගත්තේ එක විචල්‍යයක ගබඩා කළ හැක්කේ එක අගයක් (Value) පමණක් බවයි.

උදාහරණයක් ලෙස:

```
let fruit1 = "Apple";
```

```
let fruit2 = "Banana";
```

```
let fruit3 = "Mango";
```

නමුත් සිතන්න ඔබට පලතුරු 100ක නම් ගබඩා කිරීමට අවශ්‍ය බව. ඒ සඳහා විචල්‍යයන් 100ක් සෑදීම ඉතාමත් අපහසු සහ කම්මැලි වැඩකි. එමෙන්ම එම දත්ත කළමනාකරණය කිරීම ද ඉතා සංකීර්ණ වේ.

මෙම ගැටලුවට විසඳුම වන්නේ Array (අරාව) යි. අරාවක් යනු එකම විචල්‍ය නාමයක් යටතේ අගයන් සමූහයක් ලැයිස්තුවක් ලෙස ගබඩා කළ හැකි විශේෂ දත්ත ව්‍යුහයකි.

සැබෑ ලෝකයේ උපමාව (බිත්තර පැකට්ටුව): සාමාන්‍ය විචල්‍යයක් යනු එක බිත්තරයක් තැබිය හැකි කුඩා කුප්පියක් වැනිය. නමුත් අරාවක් (Array) යනු බිත්තර 12ක් හෝ 30ක් එක පෙළට තැබිය හැකි බිත්තර ඇසුරුමකි (Egg Carton). එම ඇසුරුමට "බිත්තර" කියා එක නමක් තිබුණද, ඒ ඇතුළත බිත්තර රාශියක් ඇත. ඔබට අවශ්‍ය නම් 1 වෙනි බිත්තරය හෝ 5 වෙනි බිත්තරය වෙත වෙනම ගෙන භාවිතා කළ හැක.

35.2 අරාවක් සාදන ආකාරය (Creating an Array)

ජාවාස්ක්‍රිප්ට් වලදී අරාවක් සෑදීමට අප සම්මතවම වරහන් [] (Square Brackets) භාවිතා කරමු. එහි

ඇති අගයන් එකිනෙක වෙන් කරන්නේ කොමාව (,) මගිනි.

විෂූභය:

```
let ලැයිස්තුවේ_නම = [අගය1, අගය2, අගය3];
```

උදාහරණය:

```
let fruits = ["Apple", "Banana", "Mango", "Orange"];
```

35.3 දර්ශකය: අරාවක අගයන් ගණන් කිරීම (Array Indexing)

අරාවක ඇති සෑම අයිතමයකටම (Element) අදාළ අංකයක් ඇත. මෙයට Index (දර්ශකය) යැයි කියනු ලැබේ.

වැදගත්ම නීතිය: ජාවාස්ක්‍රිප්ට් වලදී (සහ බොහෝ ක්‍රමලේඛන භාෂාවලදී) ගණන් කිරීම ආරම්භ වන්නේ 0 (බිංදුව) සිටය. 1 සිට නොවේ.

ASCII රූපසටහන (Indexing):

```
අරාව: ["Apple", "Banana", "Mango", "Orange"]
```

```
| | | |
```

```
දර්ශකය: [0] [1] [2] [3]
```

අගයන් ලබා ගන්නා ආකාරය: අරාවක ඇති යම් අගයක් ලබා ගැනීමට, අරාවේ නමට පසුව සමචතුරස්‍ර වරහන් තුළ අදාළ දර්ශකය ලිවිය යුතුය.

```
let fruits = ["Apple", "Banana", "Mango"];
```

```
console.log(fruits[0]); // Apple
```

```
console.log(fruits[1]); // Banana
```

```
console.log(fruits[2]); // Mango
```

35.4 අරාවක අගයන් වෙනස් කිරීම (Modifying Elements)

අරාවක ඇති අගයක් අපට ඕනෑම වෙලාවක වෙනස් කළ හැක.

```
let cars = ["Toyota", "Nissan", "BMW"];
```

```
// Nissan වෙනුවට Mazda දමමු
```

```
cars[1] = "Mazda";
```

```
console.log(cars); // ["Toyota", "Mazda", "BMW"]
```

35.5 අරාවක දිග සෙවීම (Array Length)

අරාවක අයිතම කීයක් තිබේදැයි දැන ගැනීමට .length යන ගුණාංගය භාවිතා කරයි.

```
let students = ["Amara", "Kamal", "Nimal", "Sunil"];
```

```
console.log(students.length); // 4
```

වෘත්තීය ඉභිය: අරාවක ඇති අවසන් අයිතමය ලබා ගැනීමට නිතරම students[students.length

- 1] යන ක්‍රමය භාවිතා කරයි.

35.6 මූලික අරා ක්‍රමවේද (Basic Array Methods)

අරාවකට අලුතින් දත්ත එකතු කිරීමට හෝ ඉවත් කිරීමට ජාවාස්ක්‍රිප්ට් අපට විශේෂ "මෙවලම්" ලබා දෙයි.

1. push() - අගට එකතු කිරීම

අරාවක අවසානයට අලුත් අගයක් එකතු කරයි.

```
let colors = ["Red", "Green"];
```

```
colors.push("Blue");
```

```
// දැන්: ["Red", "Green", "Blue"]
```

2. pop() - අගින් ඉවත් කිරීම

අරාවක අවසානයට ඇති අගය ඉවත් කරයි.

```
colors.pop();
```

```
// දැන්: ["Red", "Green"]
```

3. unshift() - මුලට එකතු කිරීම

අරාවක ආරම්භයට අලුත් අගයක් එකතු කරයි.

```
colors.unshift("Yellow");
```

```
// දැන්: ["Yellow", "Red", "Green"]
```

4. shift() - මූලින් ඉවත් කිරීම

අරාචක මූලින්ම ඇති අගය ඉවත් කරයි.

```
colors.shift();
```

```
// දැන්: ["Red", "Green"]
```

35.7 අරාචන් සහ ලූප්ස් (Arrays and Loops)

අරාචක ඇති සියලුම දත්ත එකින් එක පරීක්ෂා කිරීමට (Iterate) ලූප් එකක් භාවිතා කිරීම ඉතාමත් සාමාන්‍ය දෙයකි.

```
let marks = [85, 92, 78, 65, 88];
```

```
for (let i = 0; i < marks.length; i++) {
```

```
    console.log("ශිෂ්‍යයාගේ ලකුණු: " + marks[i]);
```

```
}
```

මෙහිදී i අගය 0 සිට ලකුණු ලැයිස්තුවේ දිග දක්වා වැඩි වෙමින් සෑම අගයක්ම මුද්‍රණය කරයි.

35.8 වෘත්තීය පැහැදිලි කිරීම (Professional Insight)

සැබෑ ලෝකයේ වෙබ් අඩවි වල දත්ත ගබඩා කරන්නේ අරාචන් ලෙසය. උදාහරණයක් ලෙස, YouTube වෙබ් අඩවියේ ඔබ දකින වීඩියෝ ලැයිස්තුව, Facebook හි මිතුරන්ගේ ලැයිස්තුව, Amazon හි භාණ්ඩ ලැයිස්තුව යන සියල්ල ජාවාස්ක්‍රිප්ට් අරාචන් (Arrays) වේ. මෙම අරාචන් තුළ පවතින්නේ බොහෝ විට Objects (ඊළඟ පරිච්ඡේදයේදී ඉගෙන ගනිමු) වන අතර, ඒවා හැසිරවීමට අරාචන් පිළිබඳ දැනුම අත්‍යවශ්‍ය වේ.

35.9 පොදු වැරදි (Common Mistakes)

1. Index එක 1 සිට ආරම්භ කිරීම: මෙය ආරම්භකයන් කරන ප්‍රධානතම වැරද්දයි. නිතරම

මතක තබා ගන්න පළමු අගය ඇත්තේ [0] ස්ථානයේ බව.

2. Length එක සහ අවසාන Index එක පවලවා ගැනීම: අරාචක අයිතම 5ක් තිබේ නම්, එහි length එක 5 වේ. නමුත් අවසාන අයිතමය ඇත්තේ [4]

වෙනි ස්ථානයේය. [5] වෙනි ස්ථානය පරීක්ෂා කළහොත් ඔබට undefined ලෙස ලැබෙනු ඇත.

3. වැරදි වරහන් භාවිතා කිරීම: අරාචක් සඳහා [] භාවිතා කළ යුතුය. { } භාවිතා කරන්නේ Objects සඳහාය.

35.10 සාරාංශය (Key Points)

- Array එකක් යනු අගයන් සමූහයක් එක තැනක තබා ගන්නා ලැයිස්තුවකි.

- අරාචක ගණන් කිරීම 0 සිට ආරම්භ වේ (Index 0).

- .length මගින් අරාචේ ඇති මුළු අයිතම ගණන ලබා දෙයි.

- push/pop අගට වැඩ කරන අතර, unshift/shift මුලට වැඩ කරයි.

35.11 අභ්‍යාස (Exercises)

1. ඔබේ මිතුරන් පස් දෙනෙකුගේ නම් අඩංගු අරාචක් සාදන්න.

2. එම අරාචේ 3 වන ස්ථානයේ සිටින මිතුරාගේ නම කන්සෝලයේ මුද්‍රණය කරන්න.

3. අරාචට අලුතින් මිතුරෙකු push මගින් එකතු කරන්න.

4. ලූප් එකක් භාවිතා කර එම අරාචේ ඇති සියලුම නම් "මගේ මිතුරා: [නම]" ලෙස පෙන්වන්න.

5. පහත අරාචේ ඇති අංක අතරින් වැඩිම අංකය සෙවීමට ලූප් එකක් සහ if කොන්දේසියක් භාවිතා කර උත්සාහ කරන්න: [45, 89, 23, 12, 67]

35.12 සම්මුඛ පරීක්ෂණ ප්‍රශ්න (Interview Questions)

- ප්‍රශ්නය: අරාචක අවසානයට අගයක් එකතු කිරීමට සහ මූලින් අගයක් ඉවත් කිරීමට භාවිතා කරන Methods මොනවාද?

- පිළිතුර: අවසානයට එකතු කිරීමට push() ද, මූලින් ඉවත් කිරීමට shift() ද භාවිතා කරයි.

මිළඟ පරිච්ඡේදයෙන් අපි ජාවාස්ක්‍රිප්ට් වල සැබෑ ලෝකයේ වස්තූන් (මිනිසුන්, කාර්, දුරකථන) නිරූපණය කරන "වස්තූන්" (Objects) ගැන ඉතාමත් විස්තරාත්මකව ඉගෙන ගනිමු.

කිසිභයවන පරිච්ඡේදය: වස්තූන් - සැබෑ ලෝකයේ දත්ත නිරූපණය (Objects) ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

1. ජාවාස්ක්‍රිප්ට් වස්තුවක් (Object) යනු කුමක්ද සහ එහි වැදගත්කම තේරුම් ගැනීම.
2. අරායක් (Array) සහ වස්තුවක් (Object) අතර ඇති ප්‍රධාන වෙනස හඳුනා ගැනීම.
3. යතුරු-අගය යුගල (Key-Value Pairs) සංකල්පය වටහා ගැනීම.
4. වස්තුවක ඇති දත්ත ලබා ගැනීමට භාවිතා කරන ක්‍රම දෙක (Dot notation සහ Bracket notation) ඉගෙන ගැනීම.
5. වස්තුවක් තුළ ඇති ශ්‍රිත හෙවත් ක්‍රමවේද (Methods) භාවිතා කරන ආකාරය.
6. වස්තුවක් තුළ තවත් වස්තුවක් (Nested Objects) පවතින ආකාරය තේරුම් ගැනීම.
7. this යන විශේෂ මූලපදය (Keyword) සරලව තේරුම් ගැනීම.

36.1 හැඳින්වීම: වස්තුවක් යනු කුමක්ද? (What is an Object?)

පෙර පරිච්ඡේදයේදී අප ඉගෙන ගත්තේ එකම වර්ගයේ දත්ත ලැයිස්තුවක් ගබඩා කිරීමට අරායන් (Arrays) භාවිතා කරන ආකාරයයි. නමුත් සැබෑ ලෝකයේ දත්ත සැමවිටම ලැයිස්තුවක් ලෙස පමණක් පවතින්නේ නැත. සැබෑ ලෝකයේ ඕනෑම දෙයක් (මිනිසෙකු, මෝටර් රථයක්, ජංගම දුරකථනයක්) සලකා බැලුවහොත්, ඒ සෑම දෙයකටම විවිධ ගුණාංග රාශියක් ඇත.

උදාහරණයක් ලෙස "මෝටර් රථයක්" (A Car):

- ගුණාංග (Properties): එහි වර්ගය (Brand), වර්ණය (Color), නිෂ්පාදිත වර්ෂය (Year).
- ක්‍රියාකාරකම් (Methods/Actions): එයට වේගයෙන් යා හැක (Drive), එයට නැවැත්විය හැක (Stop).

ජාවාස්ක්‍රිප්ට් වලදී, මෙලෙස යම් කිසි "වස්තුවක්" පිළිබඳව ඇති සියලුම තොරතුරු සහ ක්‍රියාකාරකම් එකම ස්ථානයක ගබඩා කිරීමට භාවිතා කරන දත්ත වර්ගය Object (වස්තුව) ලෙස හැඳින්වේ.

36.2 Array සහ Object අතර වෙනස

- Array: දත්ත ගබඩා කරන්නේ අනුපිළිවෙලකටය. ඒවා ලබා ගැනීමට අංක (0, 1, 2...) භාවිතා කරයි.
- Object: දත්ත ගබඩා කරන්නේ නම් (Names/Keys) භාවිතා කරමිනි. මෙහිදී අනුපිළිවෙල වැදගත් නොවේ.

උපමාවක්:

- Array එකක් යනු සාමාන්‍ය පොතක් වැනිය. එහි තොරතුරු ඇත්තේ පිටු අංක පිළිවෙලටය.
- Object එකක් යනු දුරකථන නාමාවලියක් (Phonebook) වැනිය. එහි තොරතුරු සොයන්නේ නම (Key) භාවිතා කරමිනි.

36.3 වස්තුවක් සාදන ආකාරය (Creating an Object)

වස්තුවක් සෑදීමට අප කරලි වරහන් { } (Curly Braces) භාවිතා කරමු. එහි ඇතුළත තොරතුරු ලියන්නේ Key: Value (යතුර: අගය) යර වාටටය.

විෂ්‍රහය:

```
const objectName = {
    key1: value1,
    key2: value2,
    key3: value3
};

ප්‍රායෝගික උදාහරණයක්:

const student = {
    firstName: "Kasun",
    lastName: "Kalhara",
    age: 16,
    isStudent: true,
    subjects: ["Maths", "ICT", "Science"]
};
```

මෙහි firstName යනු Key එකක් වන අතර, "Kasun" යනු එහි Value එකයි.

36.4 වස්තුවක දත්ත ලබා ගන්නා ක්‍රම (Accessing Data)

වස්තුවක ඇති තොරතුරු ලබා ගැනීමට ප්‍රධාන ක්‍රම දෙකක් ඇත.

1. Dot Notation (නිත භාවිතා කිරීමේ ක්‍රමය) මෙය වඩාත් බහුලව භාවිතා වන සහ පහසුම ක්‍රමයයි.

```
console.log(student.firstName); // Kasun
```

```
console.log(student.age); // 16
```

2. Bracket Notation (වරහන් භාවිතා කිරීමේ ක්‍රමය) සමහර අවස්ථාවලදී Key එක විචල්‍යයක් ලෙස පවතින විට මෙය භාවිතා කරයි.

```
console.log(student["lastName"]); // Kalhara
```

36.5 වස්තු ක්‍රමවේද හෙවත් මෙතඩ්ස් (Object Methods)

වස්තුවක් තුළ දත්ත (Properties) පමණක් නොව, යම් ක්‍රියාවක් කළ හැකි ශ්‍රිත (Functions) ද තිබිය හැක. වස්තුවක් ඇතුළත ඇති ශ්‍රිතයක් Method එකක් ලෙස හැඳින්වේ.

උදාහරණය:

```
const laptop = {
  brand: "Dell",
  model: "Latitude",
  start: function() {
    console.log("ලැප්ටොප් එක පණ ගැන්වේ...");
  }
};
```

// ක්‍රියාත්මක කරන ආකාරය

```
laptop.start(); // ලැප්ටොප් එක පණ ගැන්වේ...
```

36.6 this මූලපදය (The this Keyword)

වස්තුවක් ඇතුළත ඇති ශ්‍රිතයකදී, එම වස්තුවේම වෙනත් ගුණාංග භාවිතා කිරීමට අවශ්‍ය නම්, අපි this යන වචනය භාවිතා කරමු. this යන්නෙන් අදහස් වන්නේ "මෙම වස්තුවම" යන්නයි.

```
const user = {
  name: "Nimal",
  age: 18,
  greet: function() {
    console.log("ආයුබෝවන්, මගේ නම " + this.name);
  }
};
```

```
user.greet(); // ආයුබෝවන්, මගේ නම Nimal
```

36.7 වස්තුවක් ඇතුළත තවත් වස්තුවක් (Nested Objects)

අපට වස්තුවක් ඇතුළත තවත් වස්තුවක් දත්තයක් ලෙස ගබඩා කළ හැක.

```
const employee = {
  id: 101,
  name: "Amara",
  address: {
    city: "Colombo",
    street: "Main Street",
    zip: 10100
  }
};
```

```
// නගරය ලබා ගන්නා ආකාරය
console.log(employee.address.city); // Colombo
```

36.8 ASCII රූපසටහන: වස්තුවක ව්‍යුහය

Object (වස්තුව)

/ | \

[Property] [Property] [Method]

| | |

Name: Age: Greet:

"Kasun" 16 function()

36.9 වෘත්තීය පැහැදිලි කිරීම (Professional Insight)

නූතන ජාවාස්ක්‍රිප්ට් වලදී, අප අන්තර්ජාලයෙන් (API හරහා) ලබා ගන්නා දත්ත සියල්ලම පාහේ ලැබෙන්නේ Objects ආකාරයටය. විශේෂයෙන්ම JSON (JavaScript Object Notation) නමැති දත්ත හුවමාරු ක්‍රමය සම්පූර්ණයෙන්ම පදනම් වී ඇත්තේ මෙම Object සංකල්පය මතය. එබැවින් වස්තූන් (Objects) හැසිරවීමට දැන සිටීම වෘත්තීය මට්ටමේ වෙබ් සංවර්ධකයෙකුට අත්‍යවශ්‍යම දක්ෂතාවයකි.

36.10 පොදු වැරදි (Common Mistakes)

1. කොමා (Comma) අමතක වීම: වස්තුවක එක ගුණාංගයක් අවසානයේ කොමාවක් යෙදීමට අමතක කිරීම නිසා Error එකක් ලැබිය හැක. (අවසන් ගුණාංගයට කොමාව අවශ්‍ය නැතත්, එය තැබීම දැන් වෘත්තීය පුරුද්දකි).
2. this අමතක වීම: වස්තුවක් තුළ සිටින විට එම වස්තුවේම දත්ත ගැනීමට this භාවිතා නොකිරීම.
3. Key එක String එකක් ලෙස වැරදි ලෙස ලිවීම: Key එක මැද Space එකක් ඇත්නම් (උදා: "first name") එය අනිවාර්යයෙන්ම උද්ධෘත ලකුණු තුළ ලිවිය යුතු අතර එය ලබා ගැනීමට Bracket notation භාවිතා කළ යුතුය.

36.11 සාරාංශය (Key Points)

- වස්තූන් යනු Key-Value යුගල ලෙස දත්ත ගබඩා කරන ව්‍යුහයකි.
- Properties මගින් වස්තුවේ දත්ත විස්තර කරන අතර, Methods මගින් එහි ක්‍රියාවන් පෙන්වයි.

- දත්ත ලබා ගැනීමට Dot (.) සහ Bracket ([]) භාවිතා කරයි.
- this මූලපදය වර්තමාන වස්තුව හැඳින්වීමට භාවිතා කරයි.

36.12 අභ්‍යාස (Exercises)

1. ඔබේ ජංගම දුරකථනය (Mobile Phone) පිළිබඳ තොරතුරු අඩංගු Object එකක් සාදන්න. (Brand, Model, Price, Storage).
2. එම Object එකට checkBattery නමින් Method එකක් එකතු කර එයින් "Battery level is 80%" ලෙස මුද්‍රණය කරන්න.
3. පහත දැක්වෙන Object එකෙන් "Science" යන අගය ලබා ගන්නේ කෙසේද?

```
const school = {
  name: "Central College",
  classes: ["Grade 10", "Grade 11"],
  subjects: {
    science: "Science",
    maths: "Maths"
  }
};
```

4. විචල්‍යයන් භාවිතා කර Object එකක ඇති දත්තයක් වෙනස් කරන්නේ කෙසේද?

36.13 ප්‍රායෝගික ව්‍යාපෘතිය (Mini Project)

ව්‍යාපෘතිය: සරල බැංකු ගිණුම් පද්ධතිය (Simple Bank Account)

bankAccount නමින් Object එකක් සාදන්න. එහි පහත දෑ තිබිය යුතුය:

1. accountHolder (ගිණුම් හිමියාගේ නම).
2. balance (දැනට ඇති මුදල).
3. deposit(amount) (මුදල් තැන්පත් කරන Method එකක් - මෙහිදී balance එක වැඩි විය යුතුය).

4. **withdraw(amount)** (මුදල් ලබා ගන්නා Method එකක් - මෙහිදී **balance** එක අඩු විය යුතුය).

5. **checkBalance()** (වත්මන් මුදල පෙන්වන Method එකක්).

මෙම බැංකු ගිණුම භාවිතා කර මුදල් තැන්පත් කර සහ ලබාගෙන (Withdraw) අවසාන මුදල පරීක්ෂා කර බලන්න.

මිලිග පරිච්ඡේදයෙන් අපි ජාවාස්ක්‍රිප්ට් මගින් වෙබ් පිටුවක ඇති HTML කොටස් පාලනය කරන ආකාරය හෙවත් "ඩොම්" (DOM Manipulation) ගැන ඉතාමත් විස්තරාත්මකව ඉගෙන ගනිමු.

නිස්භත්වන පරිච්ඡේදය: ඩොම් පාලනය - වෙබ් පිටුවට පණ ලබා දීම (DOM Manipulation - Giving Life to the Web Page)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

1. ඩොම් (DOM - Document Object Model) යනු කුමක්දැයි වෘත්තීය මට්ටමින් තේරුම් ගැනීම.
2. වෙබ් බ්‍රවුසරය HTML ගොනුවක් "වස්තූන්ගේ ගසක්" (Object Tree) ලෙස දකින ආකාරය වටහා ගැනීම.
3. ජාවාස්ක්‍රිප්ට් භාවිතා කර HTML මූලිකාංග තෝරා ගන්නා (Selecting Elements) ආකාරය.
4. තෝරාගත් මූලිකාංගවල අන්තර්ගතය (Content) සහ හැඩතල (Styles) වෙනස් කරන ආකාරය.
5. ජාවාස්ක්‍රිප්ට් මගින් අලුතින් HTML ටැග් නිර්මාණය කරන ආකාරය.
6. **document** වස්තුව (Object) යනු කුමක්ද සහ එහි ඇති වැදගත්කම තේරුම් ගැනීම.

37.1 ඩොම් (DOM) යනු කුමක්ද? (What is the DOM?)

අපි කලින් ඉගෙන ගත් පරිදි, HTML යනු වෙබ් අඩවියක සැකිල්ලයි. නමුත් ජාවාස්ක්‍රිප්ට් වලට මෙම සැකිල්ලේ ඇති අකුරු වෙනස් කිරීමට හෝ වර්ණ මාරු කිරීමට නම්, බ්‍රවුසරය විසින් එම HTML

කේතය ජාවාස්ක්‍රිප්ට් වලට තේරෙන භාෂාවකට පරිවර්තනය කළ යුතුය.

DOM (Document Object Model) යනු බ්‍රවුසරය විසින් වෙබ් පිටුව කියවා මතකය (Memory)

තුළ සාදා ගන්නා "සිතියමකි". මෙහිදී බ්‍රවුසරය සෑම HTML ටැග් එකක්ම ජාවාස්ක්‍රිප්ට් වස්තුවක් (Object) ලෙස සලකයි. සැබෑ ලෝකයේ උපමාව (රූපවාහිනිය සහ රිමෝට් එක):

- **HTML පිටුව:** රූපවාහිනී යන්ත්‍රය වැනිය. එය තිබෙන තැනම පවතී.

- **DOM:** රූපවාහිනියේ ඇතුළත ඇති සියලුම ක්‍රියාකාරකම් (වැනල් මාරු කිරීම, හඬ පාලනය) ජාවාස්ක්‍රිප්ට් වලට සම්බන්ධ කරන පද්ධතියයි.

- **JavaScript:** රිමෝට් එක (Remote Control) වැනිය. රිමෝට් එක මගින් ඩොම් එක හරහා අපට රූපවාහිනියේ ඕනෑම දෙයක් වෙනස් කළ හැක.

37.2 ඩොම් ගස (The DOM Tree)

බ්‍රවුසරය HTML කියවන විට එය "ගසක අතු බෙදී යන ආකාරයට" සකස් කරයි. මෙහි මූල (Root)වන්නේ **html** ටැගයයි.

ASCII රූපසටහන (DOM Tree):

```

Document (මුළු පිටුවම)
|
<html> (මූල)
|
/   \
<head> <body>
|       /   \
|       <title> <h1> <p>
|       |       |
|       |       |
"My Web" "Hello" "Welcome"
```

මෙම රූපසටහන දෙස බැලූ විට අපට පෙනෙන්නේ සෑම ටැග් එකක්ම එකිනෙකට සම්බන්ධ වී ඇති ආකාරයයි. ජාවාස්ක්‍රිප්ට් වලට මෙම "ගසේ" ඕනෑම අන්තකට (ටැග් එකකට) ගොස් එය වෙනස් කළ හැක.

37.3 document වස්තුව (The document Object)

ජාවාස්ක්‍රිප්ට් වලදී අපට වෙබ් පිටුවට පිවිසීමට ඇති ප්‍රධාන දොරටුව වන්නේ document වස්තුවයි. මෙය බ්‍රවුසරය විසින් අපට ලබා දෙන ඉතාමත් බලවත් උපකරණයකි. ඔබේ කන්සෝල්යේ (Console) console.log(document); ලෙස ටයිප් කළහොත්, මුළු HTML පිටුවම වස්තුවක් ලෙස එහි දිස්වනු ඇත.

37.4 මූලිකාංග තෝරා ගැනීම (Selecting Elements)

වෙබ් පිටුවේ ඇති යම් ටැග් එකක් වෙනස් කිරීමට පෙර, අප එම ටැග් එක "අල්ලා ගත" හෝ "තෝරා ගත" යුතුය. ඒ සඳහා ප්‍රධාන ක්‍රම කිහිපයක් ඇත.

1. getElementById (ID එක මගින් තේරීම)
වඩාත්ම කාර්යක්ෂම ක්‍රමය මෙයයි. HTML හි යම් ටැග් එකකට id එකක් ලබා දී ඇත්නම්, එම නම භාවිතා කර එය තෝරාගත හැක.

```
let myTitle =
document.getElementById("main-
heading");
```

2. getElementsByClassName (Class එක මගින් තේරීම) එකම class එක ඇති ටැග් කිහිපයක් ඇති විට ඒවා සියල්ල ලැයිස්තුවක් (Array වැනි) ලෙස ලබා ගැනීමට මෙය භාවිතා කරයි.

```
let items =
document.getElementsByClassName("list-
item");
```

3. querySelector (නූතන ක්‍රමය - ඉතා වැදගත්)
මෙය CSS Selectors භාවිතා කර ඕනෑම ටැග් එකක් තේරීමට ඇති දියුණුම ක්‍රමයයි.

```
let firstPara =
document.querySelector("p"); // පළමු
පේදය
```

```
let specialDiv =
document.querySelector(".box"); // box
නමැති class එක
```

```
let myBtn =
document.querySelector("#submit-btn"); //
ID එක
```

37.5 අන්තර්ගතය වෙනස් කිරීම (Changing Content)

අපි ටැග් එකක් තෝරා ගත් පසු එහි ඇති අකුරු වෙනස් කරන්නේ කෙසේද?

1. innerText: ටැග් එක ඇතුළත ඇති අකුරු (Text) පමණක් වෙනස් කිරීමට.
2. innerHTML: ටැග් එක ඇතුළත තවත් HTML ටැග් එකතු කිරීමට හෝ වෙනස් කිරීමට.

උදාහරණය:

```
let box = document.getElementById("info-
box");
```

```
box.innerText = "මම අලුත් අකුරු!";
```

```
box.innerHTML = "<strong>මම තද
අකුරු!</strong>";
```

37.6 හැඩතල වෙනස් කිරීම (Changing Styles)

ජාවාස්ක්‍රිප්ට් මගින් ඕනෑම CSS ගුණාංගයක් වෙනස් කළ හැක. ඒ සඳහා .style යන පදය භාවිතා කරයි.

වැදගත්: CSS වල background-color ලෙස ලියන වචන ජාවාස්ක්‍රිප්ට් වලදී backgroundColor (camelCase) ලෙස ලිවිය යුතුය.

```
let myDiv =
document.querySelector(".container");
```

```
myDiv.style.backgroundColor = "blue";
```

```
myDiv.style.color = "white";
```

```
myDiv.style.padding = "20px";
```

37.7 සම්පූර්ණ කේත උදාහරණය (Complete Example)

අපි දැන් බොත්තමක් එබූ විට වෙබ් පිටුවේ ඇති අකුරු සහ වර්ණ වෙනස් වන සරල පද්ධතියක් සාදමු.

HTML (index.html):

```
<h1 id="title">ආයුබෝවන්!</h1>
```



```
<button onclick="changeNow()">වෙනස් කරන්න</button>
```

JavaScript (script.js):

```
function changeNow() {

    // 1. මාතෘකාව තෝරා ගන්න

    let heading =
    document.getElementById("title");

    // 2. අකුරු වෙනස් කරන්න

    heading.innerHTML = "ඔබ සාර්ථකව වෙනස් කළා!"; // 3. වර්ණය වෙනස් කරන්න

    heading.style.color = "red";

    heading.style.fontSize = "50px";

}
```

37.8 වෘත්තීය උපදෙස් (Professional Tips)

- Caching Selectors: එකම ටැග් එක නැවත නැවත තෝරා ගන්නවා වෙනුවට, එය විචල්‍යයක (Variable) ගබඩා කර තබා ගන්න. එය වේගවත්ය.
- Script placement: මතක තබා ගන්න, ඔබගේ <script> ටැගය සැමවිටම HTML පිටුවේ පතුලේ (</body> ට පෙර) තිබිය යුතුය. නැතිනම් ජාවාස්ක්‍රිප්ට් ක්‍රියාත්මක වන විට බොම් එක තවම සෑදී නොතිබිය හැක (එවිට null error එකක් ලැබේ).

37.9 පොදු වැරදි (Common Mistakes)

1. ID එකේ අකුරු වැරදීම: HTML හි id="myText" ලෙස ලියා ජාවාස්ක්‍රිප්ට් හි getElementById("mytext") ලෙස ලිවීම (Case sensitivity).
2. innerHTML හි ආරක්ෂාව: පරිශීලකයන් ලබා දෙන දත්ත කෙලින්ම innerHTML තුළට ඇතුළත් නොකරන්න. එයින් වෙබ් අඩවියේ ආරක්ෂාවට (XSS attacks) හානි විය හැක. ඒ සඳහා innerText භාවිතා කරන්න.
3. අමතක වීම: document. යන පදය ලිවීමට අමතක වීම.

37.10 අභ්‍යාස (Exercises)

1. පහත දැක්වෙන HTML කොටසෙහි ඇති අකුරු නිල් පාට කිරීමට අවශ්‍ය ජාවාස්ක්‍රිප්ට් කේතය ලියන්න: <p class="msg">මම පණිවිඩයක්.</p>
2. innerText සහ innerHTML අතර ඇති ප්‍රධාන වෙනස කුමක්ද?
3. බොත්තමක් ක්ලික් කළ විට වෙබ් පිටුවේ පසුබිම් වර්ණය (Background Color) වෙනස් වන සරල ශ්‍රිතයක් (Function) ලියන්න.
4. document.querySelector() භාවිතා කර ID එකක් තෝරා ගන්නා ආකාරය පෙන්වන්න.

37.11 ප්‍රායෝගික ව්‍යාපෘතිය (Mini Project)

ව්‍යාපෘතිය: විදුලි බුබුල ක්‍රියාත්මක කිරීම (Bulb On/Off Simulation)

1. HTML පිටුවක "බුබුල නිවා ඇත" යනුවෙන් සඳහන් div එකක් සාදන්න. එයට අළු පැහැති පසුබිමක් ලබා දෙන්න.
2. "On" සහ "Off" ලෙස බොත්තම් (Buttons) දෙකක් සාදන්න.
3. "On" එබූ විට div එකේ පසුබිම කහ පැහැයට හැරී අකුරු "බුබුල දැල්වී ඇත" ලෙස වෙනස් විය යුතුය.
4. "Off" එබූ විට එය නැවත මුල් තත්වයට පත් විය යුතුය.

මිලහ පරිච්ඡේදයෙන් අපි පරිශීලකයා කරන ක්‍රියා (Click, Hover, Type) වලට ජාවාස්ක්‍රිප්ට් මගින් ප්‍රතිචාර දක්වන ආකාරය හෙවත් "සිදුවීම් පාලනය" (Events) ගැන ඉතාමත් විස්තරාත්මකව ඉගෙන ගනිමු.

නිස්අටවන පරිච්ඡේදය: සිදුවීම්

පාලනය - පරිශීලකයා සමඟ

ගනුදෙනු කිරීම (Event Handling - Interacting with the User)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

1. ජාවාස්ක්‍රිප්ට් හි සිදුවීමක් (Event) යනු කුමක්දැයි වටහා ගැනීම.

2. වෙබ් අඩවියක සිදුවන විවිධ වර්ගයේ සිදුවීම් (Click, Input, Submit, Mouseover හඳුනා ගැනීම.
3. Event Listeners භාවිතා කරමින් සිදුවීමකට ප්‍රතිචාර දක්වන (Response) ආකාරය.
4. addEventListener නමැති වෘත්තීය ක්‍රමවේදය භාවිතා කරන ආකාරය.
5. සිදුවීම් වස්තුව (Event Object - e) යනු කුමක්ද සහ එයින් ලබාගත හැකි තොරතුරු මොනවාදැයි තේරුම් ගැනීම.
6. සිදුවීම් ප්‍රචාරණය (Event Bubbling) යනු කුමක්දැයි මූලිකව හඳුනා ගැනීම.
7. සැබෑ ලෝකයේ භාවිත වන "බොත්තමක් එබූ විට ක්‍රියාත්මක වන පද්ධතියක්" නිර්මාණය කිරීම.

38.1 සිදුවීමක් යනු කුමක්ද? (What is an Event?)

පෙර පරිච්ඡේදයේදී අප ඩොම් (DOM) හරහා වෙබ් පිටුවේ ඇති දේවල් වෙනස් කිරීමට ඉගෙන ගත්තෙමු. නමුත් සැබෑ වෙබ් අඩවියකදී මෙම වෙනස්කම් ස්වයංක්‍රීයව සිදු නොවේ. ඒවා සිදුවන්නේ පරිශීලකයා යම් ක්‍රියාවක් කළ විටය.

සිදුවීමක් (Event) යනු ඔබ්‍රසරය තුළ සිදුවන යම් ක්‍රියාවක් හෝ අවස්ථාවකි. පරිශීලකයා මවුසය ක්ලික් කළහොත් හෝ යතුරුපුවරුවේ අකුරක් ටයිප් කළහොත්, ඔබ්‍රසරය එම අවස්ථාව "සිදුවීමක්" ලෙස හඳුනා ගනී. ජාවාස්ක්‍රිප්ට් වලට මෙම සිදුවීම් වලට ඇහුම්කන් දීමට (Listen) සහ ඒවාට ප්‍රතිචාර දැක්වීමට හැකියාව ඇත.

සැබෑ ලෝකයේ උපමාව (දොර සිනුව): ඔබේ නිවසේ දොර සිනුව (Doorbell) ගැන සිතන්න.

1. සිදුවීම (Event): අමුත්තෙකු පැමිණ සිනුව ඔබයි.
2. ඇහුම්කම් දීම (Listener): සිනුවට සම්බන්ධ කර ඇති විදුලි පද්ධතිය එම එබීම හඳුනා ගනී.
3. ප්‍රතිචාරය (Reaction/Action): නිවස ඇතුළත "ටිං ටොං" යන ශබ්දය ඇසේ.

වෙබ් අඩවියකදී:

1. Event: පරිශීලකයා බොත්තමක් මත ක්ලික් කරයි.
2. Event Listener: ජාවාස්ක්‍රිප්ට් එම ක්ලික් කිරීම හඳුනා ගනී.
3. Callback Function: එම ක්ලික් කිරීමට අදාළව අප ලියා ඇති කේතය ක්‍රියාත්මක වේ.

38.2 ප්‍රධාන සිදුවීම් වර්ග (Common Event Types)

ඔබ්‍රසරයක සිදුවීම් සිය ගණනක් ඇතත්, ආරම්භකයෙකු ලෙස පහත ඒවා හඳුනා ගැනීම ඉතා වැදගත් වේ:

1. Mouse Events (මවුසය ආශ්‍රිත සිදුවීම්):

- click: මවුසයෙන් එක් වරක් ක්ලික් කළ විට.
- dblclick: දෙවරක් ක්ලික් කළ විට.
- mouseover: මවුසය යම් මූලිකාංගයක් මතට ගෙන ගිය විට.
- mouseout: මවුසය මූලිකාංගයකින් ඉවතට ගත් විට.

2. Keyboard Events (යතුරුපුවරුව ආශ්‍රිත සිදුවීම්):

- keydown: යතුරක් එබූ සැණින්.
- keyup: එබූ යතුර අත්හළ සැණින්.

3. Form Events (පෝරම ආශ්‍රිත සිදුවීම්):

- submit: පෝරමයක් (Form) යැවූ විට.
- input: පෝරමයක කොටුවක් තුළ යමක් ටයිප් කරන සෑම වාරයකදීම.
- change: පෝරමයක තෝරා ගැනීමක් (Select box) වෙනස් කළ විට.

4. Document Events:

- DOMContentLoaded: මුළු වෙබ් පිටුවේම HTML කියවා අවසන් වූ විට.

38.3 සිදුවීම් හසුරුවන ආකාරය (How to Handle Events)

සිදුවීමකට ප්‍රතිචාර දැක්වීමට ක්‍රම කිහිපයක් ඇත.

1. Inline Event Handlers (නිර්දේශ නොකරන පැරණි ක්‍රමය)

HTML ටැගය තුළම onclick වැනි attribute එකක් භාවිතා කිරීම. <button onclick="alert('Hello')">මාව ඔබන්න</button>

- ඇයි මෙය භාවිතා නොකරන්නේ? මෙය භාවිතා කළ විට HTML සහ JavaScript එකිනෙක මිශ්‍ර

වේ. එවිට විශාල ව්‍යාපෘති කළමනාකරණය කිරීම ඉතා අපහසු වේ.

2. Event Listener භාවිතා කිරීම (වෘත්තීය නවීන ක්‍රමය)

මෙය ලොව පුරා වෘත්තීය සංවර්ධකයන් භාවිතා කරන හොඳම ක්‍රමයයි. මෙහිදී අප භාවිතා කරන්නේ `addEventListener()` නමැති ශ්‍රිතයයි.

ව්‍යුහය (Syntax):

```
element.addEventListener(event, function);
```

1. **element:** අප සිදුවීම බලාපොරොත්තු වන HTML වැගය (උදා: බොත්තම).
2. **event:** සිදුවීමේ නම (උදා: "click").
3. **function:** සිදුවීම සිදු වූ විට ක්‍රියාත්මක විය යුතු කේතය (ශ්‍රිතය).

38.4 ප්‍රායෝගික උදාහරණය: වර්ණ මාරු කරන බොත්තම

HTML:

```
<button id="colorBtn">පාට වෙනස් කරන්න</button>
```

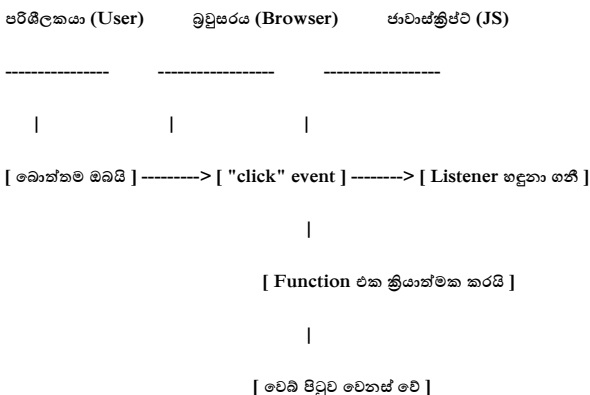
```
<div id="box" style="width: 100px; height: 100px; border: 1px solid black;"></div>
```

JavaScript:

```
// 1. මූලිකාංග තෝරා ගන්න
```

```
const btn = document.getElementById("colorBtn");
```

38.6 ASCII රූපසටහන: Event Listener ක්‍රියා කරන ආකාරය



```
const myBox = document.getElementById("box");
```

```
// 2. සිදුවීමට ඇහුම්කන් දෙන්න (Add Event Listener)
```

```
btn.addEventListener("click", function() {
```

```
// 3. කළ යුතු දේ පවසන්න
```

```
myBox.style.backgroundColor = "orange"; console.log("බොත්තම ක්ලික් කරන ලදී!");});
```

විස්තරය: මෙහිදී පරිශීලකයා `colorBtn` මත ක්ලික් කළ විට, බ්‍රවුසරය ස්වයංක්‍රීයව අප ලියා ඇති අභ්‍යන්තර ශ්‍රිතය ක්‍රියාත්මක කර කොටුවේ (box) වර්ණය තැඹිලි පාට කරයි.

38.5 සිදුවීම වස්තුව (The Event Object - e)

සිදුවීමක් සිදු වූ විට, බ්‍රවුසරය එම සිදුවීම පිළිබඳ සියලු තොරතුරු අඩංගු "වස්තුවක්" (Object) ස්වයංක්‍රීයව ශ්‍රිතයට ලබා දෙයි. සාමාන්‍යයෙන් අප මෙය `e` හෝ `event` යන නමින් හඳුන්වමු.

```
btn.addEventListener("click", function(e) {
```

```
console.log(e); // මෙම වස්තුව තුළ ක්ලික් කළ ස්ථානය, වේලාව ආදී දත්ත ඇත.
```

```
console.log(e.target); // ක්ලික් කළේ හරියටම කුමන වැගය මතදැයි පෙන්වයි.
```

```
});
```

38.7 පොදු වැරදි (Common Mistakes)

- සිදුවීමේ නම ඉදිරියට "on" යෙදීම:
addEventListener තුළ ලියන විට "onclick" ලෙස නොව "click" ලෙස ලිවිය යුතුය.
- ශ්‍රිතය කැඳවීම: addEventListener("click", myFunc()) ලෙස ලිවීම වැරදිය. ඔබ වරහන් () යෙදුවහොත්, ක්ලික් කිරීමටත් පෙර ශ්‍රිතය ක්‍රියාත්මක වේ. නිවැරදි ක්‍රමය වන්නේ වරහන් රහිතව නම පමණක් (myFunc) ලබා දීමයි.
- තෝරා ගැනීම වැරදීම: null අගයකට addEventListener එක් කිරීමට උත්සාහ කිරීම.

සැමවිටම වැගය නිවැරදිව තෝරාගෙන ඇත්දැයි console.log කර බලන්න.

38.8 වෘත්තීය උපදෙස් (Professional Tips)

- Arrow Functions: නූතන වෘත්තීය මට්ටමේදී අප ශ්‍රිත ලියන්නේ () => { ... } (Arrow Function) ආකාරයටය. මෙය කේතය කෙටි කරයි.
- Remove Event Listener: සමහර විට යම් අවස්ථාවකට පසු සිදුවීමට ඇහුම්කන් දී නැවැත්වීමට අවශ්‍ය නම් removeEventListener භාවිතා කළ හැක.
- Prevent Default: පෝරමයක් (Form) Submit කරන විට පිටුව Refresh වීම නැවැත්වීමට e.preventDefault() භාවිතා කිරීම වෘත්තීය මට්ටමේදී අත්‍යවශ්‍ය වේ.

38.9 සාරාංශය (Key Points)

- Event යනු පරිශීලකයා වෙබ් අඩවිය සමඟ කරන ගනුදෙනුවකි.
- addEventListener යනු සිදුවීම් හසුරුවන වෘත්තීය ක්‍රමයයි.
- ප්‍රධාන සිදුවීම් අතර click, input, submit, mouseover වැදගත් වේ.
- Event Object එක තුළ සිදුවීම පිළිබඳ සියලු රහස් තොරතුරු ගබඩා වී ඇත.

38.10 අභ්‍යාස (Exercises)

- වෙබ් පිටුවක ඇති පින්තූරයක් මතට මවුසය ගෙන ගිය විට එහි පළල (width) වැඩි වන සේ Event Listener එකක් ලියන්න.
- Input කොටුවක් සාදා, එහි යමක් ටයිප් කරන විට එම අකුරු තථ්‍ය කාලීනව (Real-time) ඊට පහළින් ඇති ඡේදයක (Paragraph) පෙන්වීමට

උත්සාහ කරන්න. (ඉභිය: input event එක භාවිතා කරන්න).

- බොත්තම් දෙකක් සාදන්න. එකකින් අංකයක් වැඩි විය යුතුය (Increment), අනෙකෙන් එම අංකය අඩු විය යුතුය (Decrement). එම අංකය තිරය මත පෙන්වන්න.

38.11 ප්‍රායෝගික ව්‍යාපෘතිය (Mini Project)

ව්‍යාපෘතිය: අඳුරු මාදිලිය (Dark Mode Toggle)

- HTML පිටුවක "අඳුරු මාදිලිය සක්‍රීය කරන්න" යනුවෙන් බොත්තමක් සාදන්න.
- ජාවාස්ක්‍රිප්ට් මගින් එම බොත්තම ක්ලික් කරන විට:

- මුළු වෙබ් පිටුවේම (body) පසුබිම් වර්ණය කළු පාටටත්, අකුරු සුදු පාටටත් වෙනස් කරන්න.
- බොත්තමේ ඇති අකුරු "සාමාන්‍ය මාදිලිය සක්‍රීය කරන්න" ලෙස මාරු කරන්න.

- නැවත ක්ලික් කළ විට එය සාමාන්‍ය තත්ත්වයට පත් වන සේ සකසන්න. (ඉභිය: if කොන්දේසියක් භාවිතා කර පසුබිම් වර්ණය දැනට කුමක්දැයි පරීක්ෂා කරන්න).

මිළඟ පරිච්ඡේදයෙන් අපි පරිමාව 3 හි අවසාන පාඩම ලෙස, මේ දක්වා ඉගෙන ගත් ජාවාස්ක්‍රිප්ට් මූලධර්ම සියල්ල භාවිතා කර ප්‍රායෝගික "වැඩ ලැයිස්තු පද්ධතියක්" (To-Do List App) මුල සිට අගට නිර්මාණය කරන ආකාරය ඉගෙන ගනිමු.

නිස්ථාවචන පරිච්ඡේදය: ප්‍රායෝගික ව්‍යාපෘතිය - වැඩ ලැයිස්තු පද්ධතිය (Capstone Project: To-Do List App)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

- පරිමාව 3 තුළ ඉගෙන ගත් සියලුම ජාවාස්ක්‍රිප්ට් සංකල්ප (Variables, Functions, Arrays, DOM, Events) එකට එකතු කර සැබෑ මෘදුකාංගයක් නිර්මාණය කිරීම.
- මෘදුකාංගයක් නිර්මාණය කිරීමට පෙර එහි තර්කනය (Logic) සැලසුම් කරන ආකාරය.

3. පරිශීලකයා ඇතුළත් කරන දත්ත (Input) ග්‍රහණය කරගෙන ඒවා තීරය මත ගතකව (Dynamically) පෙන්වන ආකාරය.
4. මූලිකාංග මකා දැමීම (Deleting elements) සහ තත්ත්වයන් වෙනස් කිරීම (Toggling status) කළමනාකරණය කරන ආකාරය.
5. වෘත්තීය මට්ටමේ කේත ව්‍යුහයක් (Code Structure) පවත්වා ගන්නා ආකාරය.

39.1 ව්‍යාපෘති හැඳින්වීම (Project Overview)

අපි අද නිර්මාණය කිරීමට යන්නේ "වැඩ ලැයිස්තු පද්ධතියක්" (To-Do List App) ය. මෙය ඕනෑම Frontend Developer කෙනෙකු තම මුල් කාලයේදී අනිවාර්යයෙන්ම නිර්මාණය කළ යුතු මූලික ව්‍යාපෘතියකි.

මෙම මෘදුකාංගයෙන් කළ හැකි දේ:

1. පරිශීලකයාට තමා කිරීමට ඇති වැඩක් (Task) ටයිප් කර එකතු කළ හැක.
2. එකතු කළ වැඩ සියල්ල ලැයිස්තුවක් ලෙස පෙන්වේ.
3. යම් වැඩක් කර අවසන් වූ පසු එය මත ක්ලික් කර එය "සම්පූර්ණයි" (Completed) ලෙස සලකුණු කළ හැක.
4. අවශ්‍ය නොවන වැඩ ලැයිස්තුවෙන් මකා දැමිය හැක.

39.2 පියවර 1: ව්‍යුහය සැලසුම් කිරීම (The Plan)

කේත ලිවීමට පෙර අප වැඩ කරන ආකාරය සිතිය යුතුය.

1. HTML: අපට දත්ත ටයිප් කිරීමට input එකක්, එකතු කිරීමට button එකක් සහ ලැයිස්තුව පෙන්වීමට ul (Unordered List) ටැගයක් අවශ්‍ය වේ.
2. JavaScript Logic:
 - බොත්තම එබූ විට input එකේ ඇති අකුරු කියවිය යුතුය.
 - එම අකුරු භාවිතා කර අලුත් li ටැගයක් සෑදිය යුතුය.
 - එම li ටැගය ul එක ඇතුළට ඇතුළත් කළ යුතුය.
 - සෑම වැඩක් අසලම "මකන්න" (Delete) බොත්තමක් තිබිය යුතුය.

39.3 පියවර 2: HTML සැකිල්ල (The HTML)

ඔබේ index.html ගොනුවේ පහත සැකිල්ල සකස් කරගන්න.

```
<!DOCTYPE html>

<html lang="si">

<head>

  <meta charset="UTF-8">

  <title>මගේ වැඩ ලැයිස්තුව</title>

  <link rel="stylesheet" href="style.css">

</head>

<body>

  <div class="container">

    <h1>මගේ වැඩ ලැයිස්තුව</h1>

    <div class="input-group">

      <input type="text" id="taskInput"
        placeholder="අලුත් වැඩක් ඇතුළත්
        කරන්න..."> <button id="addBtn">එකතු
        කරන්න</button>

    </div>

    <ul id="taskList">

      <!-- ඡායාස්තූපි මගින් වැඩ මෙතැනට
      එකතු වේ -->

    </ul>

  </div>

  <script src="script.js"></script>

</body>

</html>
```


39.4 පියවර 3: මූලික හැඩතල (The CSS)

මෙය ක්‍රියාත්මක වන විට ලස්සනට පෙනීමට සරල CSS කිහිපයක් එක් කරමු.

```
body { font-family: sans-serif; background-color: #f4f4f4; display: flex; justify-content: center; padding-top: 50px; }
```

```
.container { background: white; padding: 20px; border-radius: 8px; box-shadow: 0 4px 10px rgba(0,0,0,0.1); width: 350px; }
```

```
h1 { text-align: center; color: #333; }
```

```
.input-group { display: flex; gap: 10px; }
```

```
input { flex: 1; padding: 10px; border: 1px solid #ddd; border-radius: 4px; }
```

```
button { padding: 10px; background-color: #28a745; color: white; border: none; cursor: pointer; border-radius: 4px; }
```

```
ul { list-style: none; padding: 0; margin-top: 20px; }
```

```
li { background: #eee; margin-bottom: 5px; padding: 10px; display: flex; justify-content: space-between; align-items: center; border-radius: 4px; }
```

```
.delete-btn { background: #dc3545; padding: 5px 10px; font-size: 12px; }
```

```
.completed { text-decoration: line-through; opacity: 0.5; }
```

39.5 පියවර 4: ජාවාස්ක්‍රිප්ට් තර්කනය (The JavaScript)

දැන් අපි පද්ධතියේ "මොළය" නිර්මාණය කරමු. මෙය ඉතා අවධානයෙන් කියවන්න.

// 1. අවශ්‍ය මූලිකාංග තෝරා ගැනීම (Selecting Elements)

```
const taskInput = document.getElementById("taskInput");
```

```
const addBtn = document.getElementById("addBtn");
```

```
const taskList = document.getElementById("taskList");
```

// 2. බොත්තම එබූ විට සිදුවන දේ (Event Listener)

```
addBtn.addEventListener("click", function() {
```

```
    const taskValue = taskInput.value; // Input එකේ ඇති අකුරු ලබා ගැනීම
```

```
    if (taskValue === "") {
```

```
        alert("කරුණාකර වැඩක් ඇතුළත් කරන්න!");
```

```
        return; // හිස් නම් මින් ඉදිරියට කේතය ක්‍රියා නොකරයි
```

```
    }
```

// 3. අලුත් වැඩක් (li) නිර්මාණය කිරීම

```
const li = document.createElement("li");
```

```
li.innerText = taskValue;
```

// 4. මකා දැමීමේ බොත්තම සෑදීම

```
const deleteBtn = document.createElement("button");
```

```
deleteBtn.innerText = "මකන්න";
```

```
deleteBtn.classList.add("delete-btn");
```

// මකා දැමීමේ බොත්තම ක්ලික් කළ විට සිදුවන දේ

```
deleteBtn.addEventListener("click", function() {
```

```
    li.remove(); // එම පේළිය මකා දමන්න
});
```

// 5. සම්පූර්ණ කළ බව සලකුණු කිරීම (Toggle Complete)

```

li.addEventListener("click", function() {
li.classList.toggle("completed"); // CSS වල
ඇති .completed class එක එක් කරයි/ඉවත් කරයි

});

// 6. මකා දැමීමේ බොත්තම li එකට එකතු කිරීම

li.appendChild(deleteBtn);

// 7. li එක සම්පූර්ණ ලැයිස්තුවට (ul) එකතු
කිරීම taskList.appendChild(li);

// 8. Input එක හිස් කිරීම

taskInput.value = "";

});

```

39.6 කේතය පැහැදිලි කිරීම (Step-by-Step Code Explanation)

1. `document.createElement("li")`: මෙය ඉතා වැදගත් විධානයකි. මෙයින් කරන්නේ මතකය තුළ අලුතින් HTML වැග් එකක් නිර්මාණය කිරීමයි. එය තවම තිරය මත නොපෙන්වේ.
2. `li.appendChild(deleteBtn)`: මෙයින් අදහස් වන්නේ li වැගය ඇතුළතට deleteBtn වැගය ඇතුළු කරන බවයි.
3. `li.classList.toggle("completed")`: toggle යනු ස්විචයක් වැනි දෙයකි. එය අදාළ Class එක තිබේ නම් ඉවත් කරයි, නැතිනම් එක් කරයි. මෙය මෘදුකාංග වල "On/Off" ක්‍රියාවලීන් සඳහා නිතර භාවිතා වේ.
4. `li.remove()`: තිරයේ ඇති මූලිකාංගයක් සම්පූර්ණයෙන්ම ඉවත් කිරීමට මෙය භාවිතා කරයි.

39.7 පොදු වැරදි සහ විසඳුම් (Common Mistakes & Debugging)

- පිටුව Refresh වීම: ඔබ button එකක් වෙනුවට form එකක් භාවිතා කරන්නේ නම්, බොත්තම එබූ විට පිටුව Refresh විය හැක. එවිට `e.preventDefault()` භාවිතා කරන්න.
- හිස් වැඩ එකතු වීම: පරිශීලකයා කිසිවක් ටයිප් නොකර බොත්තම එබූ විට හිස් පේළි එකතු වීම වැළැක්වීමට `if (taskValue === "")` පරීක්ෂාව අත්‍යවශ්‍ය වේ.
- Events වැඩ නොකිරීම: ඔබ අලුතින් හඳුනා මූලිකාංග වලට (Dynamic elements) Event Listeners එකතු කළ යුත්තේ ඒවා නිර්මාණය කරන අවස්ථාවේදීමය.

39.8 වෘත්තීය උපදෙස් (Professional Tips)

- දත්ත ගබඩා කිරීම: දැනට ඔබ පිටුව Refresh කළහොත් ඔබ එකතු කළ වැඩ සියල්ල මැකී යයි. සැබෑ මෘදුකාංගයකදී මේවා Local Storage (පරිමාව 4 දී ඉගෙන ගනිමු) තුළ ගබඩා කළ යුතුය.
- Enter Key එක භාවිතය: බොත්තම ක්ලික් කරනවාට අමතරව, කීබෝඩ් එකේ Enter යතුර එබූ විටත් වැඩේ එකතු වන සේ සකස් කිරීම පරිශීලකයාට පහසුවකි.

```

taskInput.addEventListener("keypress",
function(e) {

    if (e.key === "Enter") {addBtn.click();
// බොත්තම එබූවා හා සමාන ක්‍රියාවක් කරයි

    }

});

```

39.9 සාරාංශය (Key Points)

- ව්‍යාපෘතියක් ආරම්භ කිරීමට පෙර එහි පියවරවල් සැලසුම් කළ යුතුය.
- `createElement` සහ `appendChild` මගින් වෙබ් පිටුවට ගතිකව කොටස් එක් කළ හැක.
- Event Listeners මගින් මෘදුකාංගය පරිශීලකයා සමඟ සම්බන්ධ කරයි.
- `classList.toggle` මගින් තත්ත්වයන් (Status) කළමනාකරණය කළ හැක.

39.10 වාර 3 විෂයන් සටහන් (Volume 3 Revision Notes)

ඔබ දැන් පරිමාව 3 සාර්ථකව අවසන් කර ඇත. මෙතෙක් අප ඉගෙන ගත් දේ කෙටියෙන්:

1. ජාවාස්ක්‍රිප්ට් යනු වෙබ් අඩවියේ මොළයයි.
2. විචල්‍යයන් (let, const) දත්ත මතක තබා ගනී.
3. කොන්දේසි (if/else) තීරණ ගනී.
4. පුඩු (loops) එකම වැඩේ නැවත කරයි.
5. ශ්‍රිත (functions) කේතය පිරිසිදුව තබා ගනී.
6. අරායන් (arrays) සහ වස්තුන් (objects) දත්ත සමූහ කළමනාකරණය කරයි.
7. DOM සහ Events මගින් HTML සමඟ ගනුදෙනු කරයි.

39.11 අභ්‍යාස (Exercises)

- අප සෑදූ වැඩ ලැයිස්තුවට "සියල්ල මකන්න" (Clear All) නමින් අලුත් බොත්තමක් එක් කරන්න. එය එබූ විට ලැයිස්තුවේ ඇති සියලුම වැඩ මැකී යා යුතුය. (ඉභිය: `taskList.innerHTML = ""`);).
- එක් එක් වැඩක් එකතු කරන විට, එය එකතු කළ වේලාව ද ඒ අසලින් පෙන්වන්න. (ඉභිය: `new Date()` භාවිතා කරන්න).
- වැඩක් "Completed" ලෙස සලකුණු කළ විට එහි පසුබිම් වර්ණය කොළ පාට වන සේ CSS වෙනස් කරන්න.

මෙම පරිච්ඡේදය හුදෙක් කේතය ලියන ආකාරය ගැන නොව, ජාවාස්ක්‍රිප්ට් එන්ජිම (Engine) ඇතුළත සිදුවන "විද්‍යාව" ගැන ඔබට ඉගැන්වීමට වෙන් කර ඇත. මෙය කියවීමෙන් පසු ඔබ සාමාන්‍ය කේත ලියන්නෙකු නොව, මෘදුකාංගයක අභ්‍යන්තරය හඳුනන ජාවාස්ක්‍රිප්ට් ඉංජිනේරුවෙකු බවට පත් වනු ඇත.

හතළිස්වන පරිච්ඡේදය: ජාවාස්ක්‍රිප්ට් ඉංජිනේරු විද්‍යාව - එන්ජිම ඇතුළත සිදුවන රහස් (JavaScript Engineering - Under the Hood)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව අතිශය ගැඹුරු වෘත්තීය දැනුමක්

ලැබෙනු ඇත:

- ජාවාස්ක්‍රිප්ට් එන්ජිම (Engine): V8 එන්ජිම කේතයක් කියවා එය ක්‍රියාත්මක කරන පියවර 3ක ක්‍රියාවලිය තේරුම් ගැනීම.
- ක්‍රියාත්මක වීමේ සන්දර්භය (Execution Context): ජාවාස්ක්‍රිප්ට් විසින් මතකය කළමනාකරණය කරන ආකාරය සහ "Global" සහ "Function" සන්දර්භයන් අතර වෙනස.
- කෝල් ස්ටැක් එක (The Call Stack): ජාවාස්ක්‍රිප්ට් එක වරකට එක වැඩක් පමණක් කරන්නේ කෙසේද (Single-threaded) යන්න විද්‍යාත්මකව වටහා ගැනීම.
- හෝස්ටින්ග් (Hoisting) රහස්: විචල්‍යයන් සහ ශ්‍රිත (Functions) ප්‍රකාශ කිරීමට පෙර ක්‍රියාත්මක විය හැකි ආකාරය පිටුපස ඇති සැබෑ රහස්.

5. විෂය පථ දාමය (Scope Chain): ජාවාස්ක්‍රිප්ට් විසින් විචල්‍යයක් සොයා ගන්නා තෙක් ඉහළට ගමන් කරන ආකාරය.

6. සිද්ධි පුළුට (The Event Loop): අසමකාලීන වැඩ (setTimeout වැනි) වෙබ් අඩවිය හිර නොකර ක්‍රියාත්මක වන්නේ කෙසේද යන්න.

40.1 ජාවාස්ක්‍රිප්ට් එන්ජිම යනු කුමක්ද? (What is JS Engine?)

පරිගණකයක ප්‍රොසෙසරයට (Processor) ඔබ ලියන `let x = 10` වැනි ජාවාස්ක්‍රිප්ට් කේත කෙලින්ම තේරෙන්නේ නැත. එයට තේරෙන්නේ 0 සහ 1 (Binary) පමණි. ඔබ ලියන ඉංග්‍රීසි වැනි වචන, පරිගණකයට තේරෙන භාෂාවට හරවන මහා පරිමාණ මෘදුකාංගය JavaScript Engine ලෙස හඳුන්වයි.

ලොව ප්‍රබලතම එන්ජිම වන්නේ Google Chrome හි ඇති V8 Engine එකයි. එය පියවර 3කින් ඔබේ කේතය ක්‍රියාත්මක කරයි:

1. Parsing (විග්‍රහ කිරීම): ඔබේ කේතය කියවා එහි අර්ථය තේරුම් ගෙන AST (Abstract Syntax Tree) නමැති ගසක හැඩයට සකස් කරයි.
2. Compilation (පරිවර්තනය): එම ගස කියවා ඉක්මනින් ක්‍රියාත්මක කළ හැකි Bytecode බවට හරවයි. මෙහිදී එය JIT (Just-In-Time) Compiler එකක් භාවිතා කරයි.
3. Execution (ක්‍රියාත්මක කිරීම): අවසානයේදී සකස් කළ උපදෙස් ප්‍රොසෙසරය වෙත යවයි.

40.2 ක්‍රියාත්මක වීමේ සන්දර්භය (Execution Context)

ජාවාස්ක්‍රිප්ට් වල වැදගත්ම සංකල්පය මෙයයි. ඔබ ජාවාස්ක්‍රිප්ට් ගොනුවක් පණ ගැන්වූ සැණින්, ඔවුසරය විසින් Global Execution Context (GEC) එකක් නිර්මාණය කරයි. මෙය හරියට ඔබ වැඩ කරන "කාමරයක්" වැනිය.

මෙම කාමරය (Context) තුළ අංශ දෙකක් ඇත:

1. Memory Component (මතක අංශය): මෙහිදී ඔබ සාදන සියලුම විචල්‍යයන් සහ ශ්‍රිත

"යතුර: අගය" (Key:Value) ලෙස ගබඩා වේ. (මෙයට Variable Environment යැයි ද කියනු ලැබේ).

2. Code Component (කේත අංශය): මෙහිදී ඔබේ කේතය පේළියෙන් පේළිය ක්‍රියාත්මක වේ. (මෙයට Thread of Execution යැයි ද කියනු ලැබේ).

40.3 හෝයිස්ටින්ග්: ඔබ නොදන්නා රහස (The Truth about Hoisting)

අපි 29 වන පරිච්ඡේදයේදී ඉගෙන ගත්තා විචල්‍යයක් මූලින් ප්‍රකාශ කළ යුතු බව. නමුත් ජාවාස්ක්‍රිප්ට් වල අමුතු දෙයක් සිදු වේ:

```
console.log(x); // පිළිතුර: undefined (Error එකක් නොවේ!)
```

```
var x = 5;
```

sayHello(); // පිළිතුර: "Hello!" (ශ්‍රීකය වැඩ කරයි!)

```
function sayHello() {  
    console.log("Hello!");  
}
```

ඇයි මෙහෙම වෙන්නේ? කේතය ක්‍රියාත්මක වීමට පෙර, JS එන්ජිම විසින් මුළු ලේඛනයම කියවා මතකය සුදානම් කරයි.

- var මගින් හඳුනා විචල්‍යයන් සඳහා මතකයේදී undefined යන අගය ලබා දෙයි.
- සම්පූර්ණ ශ්‍රිත (Functions) එලෙසම මතකයේ තැන්පත් කරයි. මෙය හරියට කේතයේ පහළ ඇති දේ ස්වයංක්‍රීයව ඉහළට ඔසවා තැබීමක් (Hoisting) වැනිය. ප්‍රෝ රහස: let සහ const වලදී ද Hoisting සිදු වේ, නමුත් ඒවාට පිවිසීමට ඉඩ ලබා නොදෙයි. මෙය Temporal Dead Zone ලෙස හැඳින්වේ. එම නිසා සැමවිටම let සහ const භාවිතා කිරීම වඩාත් ආරක්ෂිතය.

40.4 කෝල් ස්ටැක් එක: වැඩ පිළිවෙළ (The Call Stack)

පාඨාස්ත්‍රීයව යනු **Single-threaded** භාෂාවකි. එනම් එයට එක වරකට කළ හැක්කේ එක වැඩක් පමණි. මෙම වැඩයිළිවෙළ පාලනය කරන "පිහන් තට්ටුව" **Call Stack** එකයි.

උපමාවක් (පිහන් තට්ටුවක්): ඔබ පිහන් එක මත එක තබනවා යැයි සිතන්න.

1. මූලිකම පතුවලේ ඇත්තේ **Global Context** එකයි.
2. ඔබ ශ්‍රිතයක් කැඳවූ (**Call**) විට එය **Stack** එකේ උඩට පැමිණේ.
3. එම ශ්‍රිතය ඇතුළත තවත් ශ්‍රිතයක් තිබේ නම්, එය ඊටත් උඩින් පැමිණේ.
4. වැඩේ අවසන් වූ පසු උඩම ඇති පිහාන (**Function**) ඉවත් වී, නැවත යට ඇති වැඩට යයි.

40.5 සිද්ධි පුඩුව: වෙබ් අඩවිය නිර්මාණය කිරීමේදී (The Event Loop)

සිතන්න ඔබ `fetch()` භාවිතා කර අන්තර්ජාලයෙන් දත්ත ගෙනවනවා කියා. එයට තත්පර 5ක් ගත වේ නම්, ජාවාස්ක්රිප්ට් තනි ප්‍රවාහයක වැඩ කරන නිසා මුළු වෙබ් අඩවියම තත්පර 5කට හිර විය යුතුය (**Freeze**). නමුත් එසේ වන්නේ නැත.

මෙයට හේතුව Event Loop එකයි.

ASCII රූපසටහන: Event Loop ක්‍රියාවලිය

[JS Engine] [Web APIs (Browser)]

```
[ Call Stack ] <--- [ Timer / Fetch / DOM
]
```

```

|           |
|  [ Callback Queue ] <|
|           |
|<--- (Event Loop) <|

```

එය ක්‍රියා කරන ආකාරය:

1. කාලය ගත වන වැඩ (Asynchronous tasks) ඔවුසරය වෙත භාර දෙයි.
2. ඔවුසරය එම වැඩේ පසුබිමෙන් සිදු කරයි.
3. වැඩේ අවසන් වූ පසු එම ප්‍රතිඵලය Callback Queue නමැති පෝලිමට දමයි.
4. Event Loop එක සැමවිටම බලමින් සිටින්නේ Call Stack එක හිස්ද කියාය.
5. Stack එක හිස් වූ සැණින්, පෝලිමේ ඇති වැඩ උඩට ගෙනැවිත් ක්‍රියාත්මක කරයි.

40.6 ක්ලෝෂර්ස්: ජාවාස්ක්‍රිප්ට් හි මායා බලය (Closures)

මෙය වෘත්තීය ජාවාස්ක්‍රිප්ට් සංවර්ධකයෙකු සතු විය යුතු ඉහළම දැනුමයි. Closure යනු ශ්‍රිතයක් තමා නිර්මාණය වූ වටපිටාව (Scope) මතක තබා ගැනීමේ හැකියාවයි.

```
function outer() {
    let money = 100;

    function inner() {
        console.log("මම ළඟ රුපියල් " + money
        + " තියෙනවා.");
    }

    return inner;
}

const myWallet = outer();

// outer() දැන් අවසන් වී Stack එකෙන් ඉවත් වී ඇත.

// නමුත් myWallet() ක්‍රියාත්මක කළ විට තවමත් 100 අගය පෙන්වයි!

myWallet();

විස්තරය: inner ශ්‍රිතය පිටතට පැමිණෙන විට, තමාට අවශ්‍ය වූ money විචල්‍යය ද රහසින් මෙන් තමා සමඟ රැගෙන එයි. මෙය Closure ලෙස හඳුන්වයි.
```

40.7 වෘත්තීය උපදෙස් (Professional Engineering Tips)

- **Memory Leak වැළැක්වීම:** ඔබ භාවිත නොකරන විශාල අරාවන් (Arrays) හෝ වස්තුන් (Objects) විචල්‍යයන් ලෙස තබා නොගන්න. ඒවා වැඩ අවසන් වූ පසු null කරන්න. එවිට බ්‍රවුසරයේ Garbage Collector එක මගින් එම මතකය නිදහස් කරයි.
- **Blocking වැළැක්වීම:** කිසිවිටකත් ඉතා විශාල ගණනය කිරීම් Call Stack එක ඇතුළත (Synchronous ලෙස) සිදු නොකරන්න.

එවිට වෙබ් අඩවිය හිර වේ. ඒ සඳහා Web Workers භාවිතා කරන්න.

- **Strict Mode:** සැමවිටම ඔබේ JS ගොනුවේ ඉහළින්ම 'use strict'; ලෙස ලියන්න. එවිට වැරදිමකින් ඔබ අතින් සිදුවන තාක්ෂණික දෝෂ (උදා: නොදන්නා විචල්‍යයන් සෑදීම) බ්‍රවුසරය විසින් පෙන්වා දෙනු ඇත.

40.8 සාරාංශය (Key Points)

- ජාවාස්ක්‍රිප්ට් එන්ජිම (V8) කේතය Bytecode බවට හරවා ක්‍රියාත්මක කරයි.
- **Execution Context** මගින් දත්ත සහ ක්‍රියාවලිය පාලනය කරයි.
- **Hoisting** මගින් කේතය ක්‍රියාත්මක වීමට පෙර මතකය සකසයි.
- **Event Loop** මගින් අසමකාලීන වැඩ පාලනය කර වෙබ් අඩවිය වේගවත් කරයි.
- **Closures** මගින් ශ්‍රිතවලට තම උපන් වටපිටාව මතක තබා ගත හැක.

40.9 ප්‍රේම-මට්ටමේ අභ්‍යාසය (Advanced Challenge)

පහත කේතය ක්‍රියාත්මක කළ විට කන්සෝල්යේ පිළිතුරු ලැබෙන පිළිවෙළ කුමක්ද කියා අනුමාන කර, එයට හේතුව Call Stack සහ Event Loop ඇසුරින් පැහැදිලි කරන්න.

```
console.log("1. මුල");

setTimeout(() => {
    console.log("2. මැද");
}, 0);

console.log("3. අග");
```

(පිළිතුර: 1, 3, 2. තත්පර 0ක් වුවත් setTimeout එක පෝලිමට (Queue) යන බැවින් එය අවසානයට ක්‍රියාත්මක වේ).

මෙම පරිච්ඡේදය හරහා ඔබ ලබාගත් දැනුම සාමාන්‍ය ඉදිරිපෙළ සංවර්ධකයෙකුට වසර ගණනාවක අත්දැකීම් වලින් පසුව ලැබෙන දැනුමකි. ඔබ දැන් ජාවාස්ක්‍රිප්ට් ලිවීමට පමණක් නොව, එය ඇතුළතින් ක්‍රියා කරන ආකාරය පිළිබඳව ද මනා දැනුමක් ඇති ප්‍රවීණයෙකි.

හතළිස් එක්වන පරිච්ඡේදය: දත්ත හැසිරවීමේ රහස් සහ ඩොම් ගවේෂණය (String/Math Methods, Deep Scope, and DOM Traversal)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

- String Methods:** අකුරු වැල් (Strings) කපන, කොටන සහ වෙනස් කරන වෘත්තීය ක්‍රමවේද.
- Math Object:** ජාවාස්ක්‍රිප්ට් මගින් සංකීර්ණ ගණිතමය ගැටලු සහ අහඹු අංක (Random Numbers) නිපදවන ආකාරය.
- Deep Scope:** Global, Function සහ Block Scope අතර ඇති සියුම් වෙනස්කම් අවබෝධ කර ගැනීම.
- DOM Traversal:** ඩොම් ගස ඇතුළත ඉහළට, පහළට සහ දෙපැත්තට (Parent, Child, Sibling) ගමන් කරමින් ටැග් සොයා ගන්නා ආකාරය.
- Value vs Reference:** දත්ත මතකයේ ගබඩා වන ආකාරය අනුව සිදුවන අමුතු හැසිරීම් වටහා ගැනීම.

41.1 අකුරු වැල් හැසිරවීම (String Methods)

අකුරු වැලක් (String) යනු ජාවාස්ක්‍රිප්ට් වලදී හුදෙක් අකුරු පෙළක් පමණක් නොවේ. ඒ මත විවිධ "විච්ඡාදන" කිරීමට අපට ශ්‍රිත (Methods) රාශියක් ඇත.

- **length:** අකුරු වැලේ දිග සෙවීමට.
- **slice(start, end):** අකුරු වැලකින් කොටසක් කපා ගැනීමට.
- **replace():** යම් වචනයක් වෙනුවට වෙනත් වචනයක් ආදේශ කිරීමට.
- **toUpperCase() / toLowerCase():** කැපිටල් හෝ සිම්පල් කිරීමට.
- **trim():** අකුරු වැල දෙපැත්තේ ඇති අනවශ්‍ය හිස්තැන් (Spaces) ඉවත් කිරීමට.

ප්‍රායෝගික උදාහරණය:

```
let text = " JavaScript is Cool ";
```

```
let cleanText = text.trim().toUpperCase();
```

```
console.log(cleanText); // "JAVASCRIPT IS COOL"
```

```
let part = cleanText.slice(0, 10);
```

```
console.log(part); // "JAVASCRIPT"
```

41.2 ගණිතමය මැජික් (The Math Object)

වෙබ් අඩවි වලදී අපට නිතරම අංක සමඟ වැඩ කිරීමට සිදු වේ. ඒ සඳහා ජාවාස්ක්‍රිප්ට් සතුව Math නමැති උපකරණ පෙට්ටියක් ඇත.

- **Math.round():** ආසන්නතම පූර්ණ සංඛ්‍යාවට වටයයි.
- **Math.ceil():** ඉහළ අගයට වටයයි (3.1 -> 4).
- **Math.floor():** පහළ අගයට වටයයි (3.9 -> 3).
- **Math.random():** 0 සහ 1 අතර අහඹු දශම සංඛ්‍යාවක් ලබා දෙයි.

ප්‍රෝ රහස්: 1 සහ 10 අතර අහඹු අංකයක් සෑදීම

```
let randomNum = Math.floor(Math.random() * 10) + 1;
```

```
console.log(randomNum);
```

41.3 විෂය පථය ගැන ගැඹුරින් (Deep Dive into Scope)

අපි කලින් "Scope" ගැන මූලිකව කතා කළෙමු. නමුත් වෘත්තීය මට්ටමේදී ඔබ මෙලෙස වර්ගීකරණය කළ යුතුය:

- Global Scope:** මුළු ගොනුව පුරාම භාවිතා කළ හැකි විචල්‍යයන්.
- Function (Local) Scope:** ශ්‍රිතයක් ඇතුළත පමණක් භාවිතා කළ හැකි විචල්‍යයන්.
- Block Scope (let & const සඳහා):** { } වරහන් (උදා: if හෝ for ඇතුළත) පමණක් භාවිතා කළ හැකි විචල්‍යයන්.

ASCII සටහන: Scope Hierarchy

```
[ Global Scope ]
|
|-- [ Function Scope ]
|
|-- [ Block Scope (if/for) ]
```

දරුවෙකුට තම පියාගේ දත්ත බැලිය හැකි වුවත්, පියාට තම දරුවාගේ රහස් (Block scope)බැලිය නොහැක.

41.4 ඩොම් ගස ඇතුළත සංචාරය කිරීම (DOM Traversal)

අපි 37 වන පරිච්ඡේදයේදී getElementById ගැන ඉගෙන ගත්තෙමු. නමුත් සමහර විට අපට අවශ්‍ය වන්නේ අප සොයාගත් ටැගයට ඉහළින් හෝ පහළින් ඇති ටැගය සොයා ගැනීමටයි.

- **parentElement:** තෝරාගත් ටැගයේ පියා (Parent) සොයා ගැනීමට.
- **children:** ටැගයේ සිටින සියලුම දරුවන් ලැයිස්තුවක් ලෙස ලබා ගැනීමට.
- **nextElementSibling:** ඊළඟ සහෝදරයා සොයා ගැනීමට.
- **previousElementSibling:** කලින් සිටි සහෝදරයා සොයා ගැනීමට.

උදාහරණය:

```
const list = document.querySelector('ul');
```

```
const firstItem = list.children[0]; // පළමු දරුවා
```

```
const secondItem = firstItem.nextElementSibling; // ඊළඟ සහෝදරයා
```

```
const parentOfList = list.parentElement; // පියා
```

41.5 අගය සහ සබැඳිය (Value vs Reference)

මෙය ජාවාස්ක්‍රිප්ට් වල ඇති අතිශය වැදගත් සහ බොහෝ දෙනා පටලවා ගන්නා කරුණකි.

1. **Primitive Types (Number, String...):** මේවා මතකයේ ගබඩා වන්නේ අගයක් (Value) ලෙසය. එකක් වෙනස් කළ විට අනෙක වෙනස් නොවේ.
2. **Reference Types (Array, Object):** මේවා මතකයේ ගබඩා වන්නේ සබැඳියක් (Reference) ලෙසය.

උදාහරණය:

```
// Reference Type හැසිරීම
```

```
let arr1 = [1, 2];
```

```
let arr2 = arr1; // මෙහිදී arr1 හි ඇති දත්ත arr2 ට ලැබෙන්නේ නැත. ලැබෙන්නේ එම දත්ත ඇති ලිපිනයයි.
```

```
arr2.push(3);
```

```
console.log(arr1); // [1, 2, 3] <-- arr1 ද වෙනස් වී ඇත!
```

වෘත්තීය උපදෙස: අරාවක් පිටපත් කිරීමට අවශ්‍ය නම් සැමවිටම Spread Operator

```
(...arr1) භාවිතා කරන්න.
```

41.6 පොදු වැරදි (Common Mistakes)

1. **String එකක් අතින් වෙනස් කිරීමට උත්සාහ කිරීම:** let s = "Hello"; s[0] = "M"; කළ නොහැක. Strings යනු Immutable (වෙනස් කළ නොහැකි) දත්ත වර්ගයකි. ඔබට අවශ්‍ය නම් අලුත් එකක් සෑදිය යුතුය.
2. **Scope පටලවා ගැනීම:** if එකක් ඇතුළත var භාවිතා කළහොත් එය එම block එකෙන් පිටතට ද කාන්දු (Leak) වේ. සැමවිටම let හෝ const භාවිතා කරන්න.
3. **Math.random() භාවිතය:** Math.random() මගින් ලැබෙන්නේ 0-1 අතර අගයක් බව අමතක කිරීම.

41.7 සාරාංශය (Key Points)

- Strings සහ Math Objects මගින් දත්ත හැසිරවීම පහසු කරයි.
- Block Scope මගින් කේතය වඩාත් ආරක්ෂිත කරයි.
- DOM Traversal මගින් මුළු වෙබ් පිටුව පුරාම ගමන් කළ හැක.
- අරාවන් සහ වස්තූන් (Arrays & Objects) හැසිරෙන්නේ ලිපින (Reference) හරහාය.

41.8 අභ්‍යාස (Exercises)

1. පරිශීලකයා ලබා දෙන ඕනෑම වාක්‍යයක ඇති නිස්තැන් (Spaces) ඉවත් කර, සියලුම අකුරු සිම්පල් (Lowercase) කර පෙන්වන ශ්‍රිතයක් ලියන්න.

2. පහත අරාථ පිටපත් කර (Copy), මුල් අරාථට බලපෑමක් නොවන සේ අලුත් අරාථට දත්තයක් එක් කරන්න: `let colors = ["Red", "Green"];`
3. HTML ලැයිස්තුවක් සාදා, ජාවාස්ක්‍රිප්ට් මගින් දෙවන ලැයිස්තු අයිතමයේ සිට එහි පියා (ul) සොයාගෙන එහි පසුබිම් වර්ණය වෙනස් කරන්න.

41.9 පරිමාව 3 අවසන් පිරික්සුම් ලැයිස්තුව (Volume 3 Complete Checklist)

දැන් ඔබ පරිමාව 3 හි සියලුම පාඩම් අවසන් කර ඇති බැවින්, පහත දෑ ඔබට කළ හැකි දැයි බලන්න:

- [] මට JavaScript Syntax (විවර්ණයන්, කොන්දේසි, ලූප්ස්) 100% ක් නිවැරදිය.
- [] මම Arrays සහ Objects දත්ත හැසිරවීමට දනිමි.
- [] මම DOM පාලනය සහ Events ගැන ප්‍රවීණයෙකි.
- [] මට JavaScript එන්ජිම ක්‍රියා කරන ආකාරය සහ Hoisting, Scope, Closures පැහැදිලි කළ හැක.
- [] මට තනිවම සරල මෘදුකාංග (Calculator, To-Do List) නිර්මාණය කළ හැක.

හතළිස් දෙවන පරිච්ඡේදය:
ජාවාස්ක්‍රිප්ට් ගෘහ නිර්මාණ ශිල්පය
- ප්‍රෝටෝටයිප්, ක්‍රියාකාරී ප්‍රමුඛතාවය සහ උසස් හුක්ස් (JS Architecture, Prototypes, Microtasks & Context Binding)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබ සාමාන්‍ය සංවර්ධකයෙකුගේ මට්ටම ඉක්මවා ගොස් පහත සඳහන්

"Senior-Level" කරුණු පිළිබඳව පූර්ණ ප්‍රවීණත්වයක් ලබා ගනු ඇත:

1. ප්‍රෝටෝටයිප් උරුමය (Prototypal Inheritance): ජාවාස්ක්‍රිප්ට් වස්තූන් එකිනෙක සම්බන්ධ වී දත්ත හුවමාරු කර ගන්නා සැබෑ ආකාරය තේරුම් ගැනීම.

2. මයික්‍රො-ටාස්ක් පෝලිම (Microtasks vs Macrotasks): අසමකාලීන වැඩ (Promises vs setTimeout) අතර පවතින ප්‍රමුඛතාවය (Priority) විද්‍යාත්මකව වටහා ගැනීම.

3. Context Binding (this keyword): call, apply, සහ bind භාවිතා කරමින් ඕනෑම තැනක පවතින ශ්‍රිතයක් තමන්ට අවශ්‍ය පරිදි පාලනය කිරීම.

4. ඉහළ පෙළේ ශ්‍රිත (Higher-Order Functions): reduce, filter, සහ map භාවිතා කරමින් දත්ත හැසිරවීමේ ප්‍රවීණයෙකු වීම.

5. උසස් මොඩියුල රටා (Advanced Module Patterns): කේතය ආරක්ෂා කරන (Private/Public) ආකාරය තේරුම් ගැනීම.

42.1 ජාවාස්ක්‍රිප්ට් හි "පියා" - ප්‍රෝටෝටයිප් (Prototypes & Inheritance)

ජාවාස්ක්‍රිප්ට් යනු Prototypal භාෂාවකි. මෙහිදී එක් වස්තුවකට තවත් වස්තුවක ඇති ලක්ෂණ උරුම කර ගත හැක.

එය ක්‍රියා කරන්නේ කෙසේද? සෑම ජාවාස්ක්‍රිප්ට් වස්තුවකටම (Object) රහසිගත සබැඳියක් ඇත. එය [[Prototype]] ලෙස හැඳින්වේ. ඔබ වස්තුවක නැති දෙයක් සෙවීමට උත්සාහ කළහොත්, ජාවාස්ක්‍රිප්ට් එන්ජිම කරන්නේ එහි "පියා" හෙවත් Prototype එක වෙත ගොස් එය සෙවීමයි.

මෙයට Prototype Chain යැයි කියනු ලැබේ.

ප්‍රායෝගික උදාහරණය:

```
let animal = {
  eats: true,
  walk() { console.log("සතා ඇවිදිනවා..."); }
};

let rabbit = {
  jumps: true,
  __proto__: animal // rabbit දැන් animal
  ගේ දේපල උරුම කර ගනී
};
```

```
console.log(rabbit.eats); // true (මෙය animal ගෙන් ලැබුණි)
```

```
rabbit.walk(); // "සතා ඇවිදිනවා..."
```

වෘත්තීය රහස: ඔබ භාවිතා කරන `Array.push()` හෝ `String.toUpperCase()` වැනි සියලුම දෑ ලැබෙන්නේ මෙම **Prototype Chain** එක හරහාය.

42.2 මයික්‍රෝ-ටාස්ක් පෝලිම: අසමකාලීන ප්‍රමුඛතාවය (The Priority Secret)

අපි 40 වන පරිච්ඡේදයේදී "Event Loop" ගැන ඉගෙන ගත්තෙමු. නමුත් සියලුම අසමකාලීන (Asynchronous) වැඩ සමාන නොවේ. බ්‍රවුසරය තුළ පෝලිම (Queues) දෙකක් පවතී:

1. **Microtask Queue** (කුඩා කාර්යයන්): Promises සහ MutationObserver මෙයට අයත් වේ.
2. **Macrotask Queue** (විශාල කාර්යයන්): `setTimeout`, `setInterval`, සහ DOM Events මෙයට අයත් වේ.

රන් තීරණය: Event Loop එක සැමවිටම Microtask Queue එකට ප්‍රමුඛතාවය ලබා දෙයි. එනම් `setTimeout` එකට වඩා කලින් Promise එකක් ක්‍රියාත්මක වේ.

සංකීර්ණ පරීක්ෂණය:

```
console.log("1");
```

```
setTimeout(() => console.log("2"), 0); // Macrotask
```

```
Promise.resolve().then(() => console.log("3")); // Microtask
```

```
console.log("4");
```

පිළිතුර: 1, 4, 3, 2. (ඇයි 3 මුලින් ආවේ? මන්ද Promises වලට ප්‍රමුඛතාවය වැඩි බැවිනි).

42.3 this පාලනය කිරීම: Call, Apply, සහ Bind

ජාවාස්ක්‍රිප්ට් වල `this` යන වචනය ඉතාමත් අස්ථිර දෙයකි. එය කැඳවන ස්ථානය අනුව එහි අර්ථය

වෙනස් වේ. වෘත්තීය මට්ටමේදී අපට මෙය පාලනය කිරීමට සිදු වේ.

1. **call()**: ශ්‍රිතයක් කැඳවන අතරතුර `this` ලෙස කුමන වස්තුව ගත යුතුදැයි වහාම පවසයි.
2. **apply()**: `call` වලට සමානයයි, නමුත් දත්ත යවන්නේ අරාවක් (Array) ලෙසයි.
3. **bind()**: ශ්‍රිතය වහාම ක්‍රියාත්මක නොකර, අනාගතයේදී භාවිතා කිරීමට `this` අගය තහවුරු කළ (Bound) අලුත් ශ්‍රිතයක් ලබා දෙයි.

උදාහරණය:

```
const person = {
```

```
  name: "කසුන්",
```

```
  printName: function() {
    console.log(this.name);
  }
```

```
};
```

```
const anotherPerson = { name: "නිමල්" };
```

// person ගේ ශ්‍රිතය anotherPerson සඳහා භාවිතා කරමු

```
person.printName.call(anotherPerson); // "නිමල්"
```

42.4 උසස් දත්ත හැසිරවීම: reduce() ප්‍රවීණතාවය

වෘත්තීය සංවර්ධකයෙකු `for` ලූප්ස් භාවිතා කරන්නේ ඉතා අඩුවෙනි. ඔවුන් විශාල දත්ත හැසිරවීමට `reduce()` භාවිතා කරයි. මෙය අරාවක ඇති සියලුම දත්ත එකතු කර තනි අගයක් (Single value) ලබා දෙන බලවත් මෙවලමකි.

```
const numbers = [1, 2, 3, 4, 5];
```

// සියල්ලේ එකතුව සෙවීම

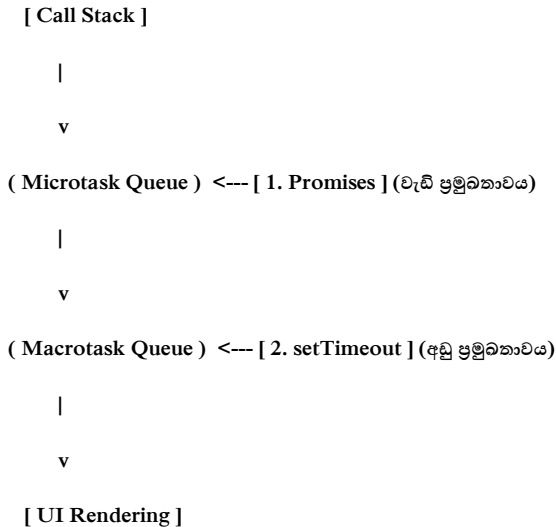
```
const sum = numbers.reduce((accumulator, current) => {
```

```
  return accumulator + current;
```

```
}, 0); // 0 යනු ආරම්භක අගයයි
```

```
console.log(sum); // 15
```

42.5 ASCII රූපසටහන: Event Loop එකේ ප්‍රමුඛතාවය (Priority)



42.6 ආරක්ෂාව සහ ගබඩා කිරීම (Security & Performance)

අපි 49 වන පරිච්ඡේදයේ "Local Storage" ගැන ඉගෙන ගත්තෙමු. නමුත් ප්‍රෝ-සංවර්ධකයෙකු එහි ඇති සීමාවන් දැන:

- XSS Protection: LocalStorage තුළ රහස්‍ය දත්ත (Passwords/Tokens) තැන්පත් කිරීම** අනතුරුදායක විය හැක. සයිබර් අපරාධකරුවෙකුට `document.cookie` හරහා දත්ත සොරාගත හැක. ඒ සඳහා **HttpOnly Cookies** භාවිතා කිරීම වෘත්තීය ක්‍රමයයි.
- IndexedDB: ඔබට 5MB ට වඩා විශාල දත්ත ප්‍රමාණයක් (උදා: 1GB) ඔබ්බරයේ සේවි කිරීමට** අවශ්‍ය නම් **LocalStorage** වෙනුවට **IndexedDB** භාවිතා කළ යුතුය.

42.7 වෘත්තීය උපදෙස් (Senior Developer Pro-Tips)

- **Avoid Global Variables:** කිසිවිටකත් **Global window object** එකේ විචල්‍යයන් ගොඩගසන්න එපා. එය මතකය පිරීමට (Memory pollution) හේතු වේ.
- **Use Functional Programming:** දත්ත වෙනස් නොකර (Immutability), අලුත් දත්ත

අරාමන් සෑදීමේ කලාව පුරුදු වන්න. (උදා: `const newArr = [...oldArr]`).

- **Debouncing & Throttling:** පරිශීලකයා එක දිගට බොත්තමක් ක්ලික් කරන විට (උදා: **Search bar**), සෑම අකුරකටම API එකට පණිවිඩ යවන්නේ නැතිව, කාල පරතරයක් තබා දත්ත ලබා ගැනීමේ (Optimization) තාක්ෂණයන් ඉගෙන ගන්න.

42.8 සාරාංශය (Key Points)

- ජාවාස්ක්‍රිප්ට් වස්තූන් **Prototypes** හරහා දත්ත උරුම කර ගනී.
- **Promises (Microtasks)** සැමවිටම **setTimeout (Macrotasks)** වලට වඩා කලින් සිදුවේ.
- **call, apply, bind** මගින් **this** පාලනය කළ හැක.
- **reduce** යනු දත්ත ගොනුවක් කළමනාකරණය කිරීමට ඇති බලවත්ම ශ්‍රිතයයි.
- ආරක්ෂාව සඳහා **LocalStorage** හි රහස්‍ය දත්ත තැබීමෙන් වළකින්න.

42.9 ප්‍රෝ-මට්ටමේ අභ්‍යාසය (The Ultimate Challenge)

අභියෝගය:

- අයිතම 5ක් ඇති අරාමක් සාදන්න.
- reduce** භාවිතා කර එම අරාමේ ඇති සියලුම ඉලක්කම්වල ගුණිතය (Multiplication) සොයන්න.
- setTimeout** එකක් සහ **Promise** එකක් එකම තැන ලියා, කුමන එක මුලින්ම වැඩ කරන්නේදැයි පරීක්ෂා කර බලා එයට හේතුව මෙම පරිච්ඡේදයේ කරුණු ඇසුරින් විස්තර කරන්න.

මෙම පරිච්ඡේදය හරහා ඔබ ජාවාස්ක්‍රිප්ට් පිළිබඳව ලබාගත් දැනුම සාමාන්‍ය

උපාධිධාරියෙකු සතු දැනුමට වඩා ඉහළ මට්ටමක පවතී. ඔබ දැන් සැබෑ ඉදිරිපෙළ ගෘහ නිර්මාණ

ශිල්පියෙකු (Frontend Architect) ලෙස සිතීමට සූදානම්.

උසස් ජාවාස්ක්‍රිප්ට්(Advanced JavaScript)

හතළිස්තුන්වන පරිච්ඡේදය: නූතන ජාවාස්ක්‍රිප්ට් - ES6+ හඳුන්වාදීම (Introduction to Modern JavaScript - ES6+)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

1. ECMAScript (ES) යනු කුමක්ද සහ එහි ඉතිහාසය කෙටියෙන් වටහා ගැනීම.
2. ES6 සංස්කරණය ජාවාස්ක්‍රිප්ට් ලෝකයේ පෙරළියක් ඇති කළේ ඇයි දැයි තේරුම් ගැනීම.
3. අකුරු වැල් (Strings) හැසිරවීමට භාවිතා කරන Template Literals (ආකෘතිමය අකුරු වැල්) ඉගෙන ගැනීම.
4. Arrow Functions (ඊතල ශ්‍රිත) සහ සාමාන්‍ය ශ්‍රිත අතර ඇති වෙනස හඳුනා ගැනීම.
5. Destructuring (ව්‍යුහගත දත්ත වෙන් කිරීම) මගින් කේතය කෙටි කරන ආකාරය.
6. Spread සහ Rest ක්‍රියාකරුවන් (...) භාවිතා කරන ආකාරය.
7. වෘත්තීය මට්ටමේ කේත ලිවීමේදී නූතන ක්‍රමවේද භාවිතා කිරීමේ වැදගත්කම.

43.1 ECMAScript යනු කුමක්ද? (What is ECMAScript?)

ජාවාස්ක්‍රිප්ට් ඉගෙන ගන්නා විට ඔබට නිතරම ECMAScript හෝ ES6 යන වචන ඇසෙනු ඇත. මෙය සරලව තේරුම් ගනිමු.

ජාවාස්ක්‍රිප්ට් යනු භාෂාවයි. ECMAScript යනු එම භාෂාව ක්‍රියා කළ යුතු ආකාරය තීරණය කරන "ප්‍රමිතිය" (Standard) හෝ "නීති මාලාව" වේ.

සැබෑ ලෝකයේ උදාහරණයක්: සිංහල භාෂාව ගැන සිතන්න. අප කතා කරන සිංහල භාෂාව "ජාවාස්ක්‍රිප්ට්" වැනිය. එම භාෂාව නිවැරදිව ලිවිය යුතු ආකාරය පවසන ව්‍යාකරණ පොත "ECMAScript" වැනිය. කාලයෙන් කාලයට ව්‍යාකරණ පොතට අලුත් වචන සහ නීති එකතු වන්නා සේ, ජාවාස්ක්‍රිප්ට් වලට ද අලුත් නීති එකතු වේ.

- ES5: 2009 දී පැමිණි පැරණි නීති මාලාව.
- ES6 (ES2015): 2015 දී පැමිණි ඉතාමත් වැදගත් නීති මාලාව. මෙතැන් සිට ජාවාස්ක්‍රිප්ට් ඉතාමත් බලවත් සහ නවීන භාෂාවක් බවට පත් විය.
- ES6+: මෙයින් අදහස් කරන්නේ 2015 සිට මේ දක්වා පැමිණි සියලුම අලුත් සංස්කරණයන්ය.

43.2 Template Literals: අකුරු වැල් සමඟ මැජික් පෙන්වීම

පැරණි ක්‍රමයට (ES5) අකුරු සහ විචල්‍යයන් එකට එකතු කිරීම ඉතා වෙහෙසකර විය.

පැරණි ක්‍රමය (ES5):

```
let name = "කසුන්";
let age = 16;
console.log("මගේ නම " + name + " වන අතර මගේ වයස " + age + " කි.");
```

මෙහිදී + ලකුණු සහ උද්ධෘත ලකුණු "" වැරදීමට ඇති ඉඩ වැඩිය.

නූතන ක්‍රමය (ES6 - Template Literals): මෙහිදී අපි තනි උද්ධෘත " වෙනුවට යතුරුපුවරුවේ 1 බොත්තමට වම් පැත්තේ ඇති Backticks (`) භාවිතා කරමු. විචල්‍යයන් ඇතුළත් කිරීමට \${ } භාවිතා කරයි.

```
let name = "කසුන්";
let age = 16;
// වඩාත් පිරිසිදු සහ පහසු ක්‍රමය
console.log(`මගේ නම ${name} වන අතර මගේ වයස ${age} කි.`);
```

ඇයි මෙය වැදගත්?

1. කේතය කියවීමට ඉතා පහසුය.
2. පේළි කිහිපයක පණිවිඩ (Multi-line strings) ලිවීම ඉතා පහසුය.

43.3 Arrow Functions: කෙටි සහ බලවත් ශ්‍රිත (ඊතල ශ්‍රිත)

ES6 මගින් ශ්‍රිත (Functions) ලිවීමට ඉතාමත් කෙටි සහ නවීන ක්‍රමයක් හඳුන්වා දෙන ලදී.

සාමාන්‍ය ශ්‍රිතය (Regular Function):

```
function double(number) {
  return number * 2;
}
```

ඊතල ශ්‍රිතය (Arrow Function):

```
const double = (number) => {
  return number * 2;
};
```

// කේතය ඉතා කෙටි නම් මෙලෙස ද ලිවිය හැක:

```
const doubleShort = number => number * 2;
```

විස්තරය:

- function යන වචනය අවශ්‍ය නොවේ.
- => ලකුණ මගින් එය ශ්‍රිතයක් බව හඳුන්වයි.
- වෘත්තීය කරුණක්: Arrow functions වල this යන පදය හැසිරෙන ආකාරය සාමාන්‍ය ශ්‍රිත වලට වඩා වෙනස්ය. මෙය React වැනි තාක්ෂණයන්හිදී ඉතා වැදගත් වේ.

43.4 Destructuring: දත්ත වෙන් කර ගැනීම

අරාමක (Array) හෝ වස්තුවක (Object) ඇති දත්ත ඉතා ඉක්මනින් විවලයයන් වලට ලබා ගැනීමට මෙය භාවිතා කරයි.

වස්තුවක් සඳහා (Object Destructuring):

```
const user = {
  firstName: "නිමල්",
  city: "කොළඹ",
  job: "Developer"
};
```

// පැරණි ක්‍රමය
// let firstName = user.firstName;
// let city = user.city;

// ES6 ක්‍රමය
const { firstName, city, job } = user;

console.log(firstName); // නිමල්

අරාමන් සඳහා (Array Destructuring):

```
const colors = ["රතු", "කොළ", "නිල්"];
```

```
const [first, second] = colors;
```

```
console.log(first); // රතු
console.log(second); // කොළ
```

43.5 Spread සහ Rest Operators (...)

මෙම තිත් තුනේ සලකුණ (...) ජාවාස්ක්‍රිප්ට් වල "හිතවතෙකු" වැනිය. එය අවස්ථාව අනුව වැඩ දෙකක් කරයි.

1. Spread Operator (පැතිරවීම)

පවතින අරාමක ඇති සියලුම දත්ත තවත් අරාමකට "පිටපත්" කිරීමට මෙය භාවිතා කරයි.

```
const numbers = [1, 2, 3];
const newNumbers = [...numbers, 4, 5];
```

```
console.log(newNumbers); // [1, 2, 3, 4, 5]
```

2. Rest Operator (ඉතිරි දේ)

ශ්‍රිතයකට එවන දත්ත සමූහයක් එකම අරාමකට ලබා ගැනීමට මෙය භාවිතා කරයි.

```
function sum(...allNumbers) {
  // allNumbers යනු අප එවන සියලුම අංක
  අඩංගු අරාමකි
  console.log(allNumbers);
}
```

```
sum(10, 20, 30, 40);
```

43.6 ASCII සංසන්දනය: පැරණි එදිරිව නවීන

ලක්ෂණය	පැරණි ක්‍රමය (ES5)	නවීන ක්‍රමය (ES6+)
විවලයයන්	var	let / const
අතුරු වැල්	"Hello " + name	`Hello \${name}`
ශ්‍රිත	function() { }	() => { }
පිටපත් කිරීම	looping strings	[...array]
දත්ත වෙන් කිරීම	obj.name	const { name } = obj

43.7 වෘත්තීය උපදෙස් (Professional Tips)

- සැමවිටම Const භාවිතා කරන්න: ඔබ විවලයයක අගය වෙනස් කිරීමට බලාපොරොත්තු නොවේ නම්, සැමවිටම const භාවිතා කරන්න. එය ඔබගේ කේතය වඩාත් ආරක්ෂිත කරයි.

- Arrow Functions වල සීමාව: සියලුම තැන්වලට Arrow functions සුදුසු නොවේ.

විශේෂයෙන්ම Object Methods සඳහා සාමාන්‍ය ශ්‍රිත භාවිතා කිරීම වඩාත් සුදුසුයි (මන්ද this පදය නිසා).

- Clean Code: Destructuring භාවිතා කිරීමෙන් ඔබ ලියන පේළි ගණන අඩු කර කේතය පිරිසිදු කරගත හැක.

43.8 පොදු වැරදි (Common Mistakes)

1. Backticks වෙනුවට Quotes: `{ }` ක්‍රියා කරන්නේ backticks (```) තුළ පමණි. `"` හෝ `'` තුළ එය සාමාන්‍ය අකුරු ලෙස පෙනේ.
2. Destructuring හි නම වැරදීම: Object එකක දත්ත වෙන් කරන විට, Key එකේ නම සහ ඔබ ලබා දෙන විචල්‍ය නාමය සමාන විය යුතුය. `const { name } = user;` නිවැරදි වුවත් `const { නම } = user;` ක්‍රියා නොකරයි.
3. Spread Operator එකේ තිත් ගණන: සැමවිටම තිත් 3ක් (...) තිබිය යුතුය. 2ක් හෝ 4ක් තැබීමෙන් Error එකක් ලැබේ.

43.9 සාරාංශය (Key Points)

- ES6 ජාවාස්ක්‍රිප්ට් භාෂාවට නව පණක් ලබා දුන්නේය.
- Template Literals මගින් අකුරු සහ විචල්‍යයන් පහසුවෙන් සම්බන්ධ කරයි.
- Arrow Functions මගින් ශ්‍රිත ලිවීම කෙටි සහ නවීන කරයි.
- Destructuring මගින් Object සහ Array වලින් දත්ත පහසුවෙන් ලබා ගනී.
- Spread Operator මගින් දත්ත පිටපත් කිරීම සහ එකතු කිරීම පහසු කරයි.

43.10 අභ්‍යාස (Exercises)

1. පහත කේතය Template Literals භාවිතා කර නැවත ලියන්න: `console.log("ඔබේ ලකුණු " + marks + " වන අතර ඔබ " + status + " වී ඇත.");`
2. අංක දෙකක් ගුණ කර පිළිතුර ලබා දෙන සරල Arrow Function එකක් ලියන්න.
3. පහත Object එකෙන් Destructuring භාවිතා කර brand සහ price යන අගයන් ලබා ගන්න: `const phone = { brand: "Samsung", model: "S21", price: 85000 };`
4. අරාවන් දෙකක් (Array 1 සහ Array 2) Spread Operator භාවිතා කර එකට එකතු කරන ආකාරය පෙන්වන්න.

43.11 සම්මුඛ පරීක්ෂණ ප්‍රශ්න (Interview Questions)

- ප්‍රශ්නය: Arrow Function එකක් සහ සාමාන්‍ය Function එකක් අතර ඇති ප්‍රධාන වෙනස කුමක්ද?
- පිළිතුර: ප්‍රධාන වෙනස්කම් දෙකකි. 1. Arrow functions ලිවීමට පහසු සහ කෙටිය. 2. Arrow functions වලට තමන්ගේම කියා this අගයක් නැත (එය තමාට ඉහළින් ඇති Scope එකෙන් this අගය ලබා ගනී).

මිළඟ පරිච්ඡේදයෙන් අපි ජාවාස්ක්‍රිප්ට් ගොනු කිහිපයක් එකිනෙක සම්බන්ධ කරන ආකාරය හෙවත් "මොඩියුල" (Modules - Export & Import) ගැන ඉතාමත් විස්තරාත්මකව ඉගෙන ගනිමු.

පරිමාව 4: උසස් ජාවාස්ක්‍රිප්ට් (Advanced JavaScript)

හතළිස් හතරවන පරිච්ඡේදය: මොඩියුල - කේතය ක්‍රමවත්ව බෙදා වෙන් කිරීම (Modules - Organizing Your Code)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

1. ජාවාස්ක්‍රිප්ට් මොඩියුල (Modules) යනු කුමක්දැයි වෘත්තීය මට්ටමින් තේරුම් ගැනීම.
2. විශාල කේත ගොනු කුඩා කොටස්වලට බෙදීමේ (Modularization) වැදගත්කම වටහා ගැනීම.
3. export මූලපදය භාවිතා කර දත්ත පිටතට යවන ආකාරය.
4. import මූලපදය භාවිතා කර වෙනත් ගොනුවක ඇති දත්ත ලබා ගන්නා ආකාරය.
5. නම් කළ අපනයනයන් (Named Exports) සහ පෙරනිම් අපනයනයන් (Default Exports) අතර වෙනස හඳුනා ගැනීම.
6. HTML ගොනුවක `type="module"` ගුණාංගය භාවිතා කරන ආකාරය.
7. මොඩියුල ක්‍රියාත්මක කිරීමට Live Server එකක් අවශ්‍ය වන්නේ ඇයි දැයි වටහා ගැනීම.

44.1 හැඳින්වීම: මොඩියුලයක් යනු කුමක්ද?

ඔබ මෙතෙක් ලියූ ජාවාස්ක්‍රිප්ට් කේත සියල්ලම තිබුණේ එකම එක script.js ගොනුවක් තුළය. කුඩා ව්‍යාපෘති සඳහා එය ගැටලුවක් නොවේ. නමුත් සැබෑ ලෝකයේ විශාල වෙබ් අඩවිවල කේත පේළි දහස් ගණනක් හෝ ලක්ෂ ගණනක් තිබිය හැක. මේ සියල්ල එකම ගොනුවක ලිව්වොත්:

1. කේතයේ වැරද්දක් සෙවීම ඉතා අපහසු වේ.
2. එකම නම ඇති විචල්‍යයන් පටලවා ගැනීමට ඉඩ ඇත.
3. කණ්ඩායමක් ලෙස වැඩ කිරීමේදී එකම ගොනුව සංස්කරණය කිරීම ගැටලුකාරී වේ.

Module (මොඩියුලය) යනු විශාල ජාවාස්ක්‍රිප්ට් ලේඛනයක් කුඩා, ස්වාධීන ගොනු කිහිපයකට බෙදා වෙන් කිරීමේ ක්‍රමයයි. සෑම ගොනුවක්ම එක් නිශ්චිත කාර්යයක් සඳහා වෙන් කරනු ලැබේ.

සැබෑ ලෝකයේ උදාහරණයක් (මෝටර් රථයක්): මෝටර් රථයක් යනු එකම යකඩ ගොඩක් නොවේ. එය ඇන්ජිම, රෝද, ලයිට් සහ ගියර් පද්ධතිය වැනි වෙන වෙනම "මොඩියුල" එකතු වී සැදුණු නිර්මාණයකි. ඇන්ජිමේ දෝෂයක් ආ විට අපට රෝද ගලවා පරීක්ෂා කිරීමට අවශ්‍ය නැත; අප පරීක්ෂා කරන්නේ ඇන්ජිම නමැති මොඩියුලය පමණි. ජාවාස්ක්‍රිප්ට් මොඩියුල ද ක්‍රියා කරන්නේ මෙලෙසමය.

44.2 Export සහ Import සංකල්පය

මොඩියුල ක්‍රියා කරන්නේ ප්‍රධාන පියවර දෙකක් හරහාය:

1. Export (අපනයනය): යම් ගොනුවක ඇති විචල්‍යයක්, ශ්‍රිතයක් හෝ වස්තුවක් වෙනත් ගොනුවකට භාවිතා කිරීමට ඉඩ ලබා දීම.
2. Import (ආනයනය): වෙනත් ගොනුවකින් එවන ලද (Export කළ) දත්ත තම ගොනුව තුළට ලබා ගැනීම.

44.3 නම් කළ අපනයනයන් (Named Exports)

එකම ගොනුවකින් දේවල් කිහිපයක් පිටතට යැවීමට අවශ්‍ය විට මෙය භාවිතා කරයි. මෙහිදී

අපනයනය කරන නමම ආනයනය කරන විට ද භාවිතා කළ යුතුය.

ගොනුව: mathUtils.js (ගණිතමය ශ්‍රිත මෙහි ලියමු)

```
// විචල්‍යයක් export කිරීම
export const PI = 3.14159;
```

```
// ශ්‍රිතයක් export කිරීම
export const add = (a, b) => a + b;
```

```
export const multiply = (a, b) => a * b;
```

ගොනුව: app.js (ප්‍රධාන ගොනුව)

```
// වෙනත් ගොනුවකින් ලබා ගැනීම (Import)
import { PI, add, multiply } from './mathUtils.js';
```

```
console.log(PI); // 3.14159
console.log(add(10, 5)); // 15
console.log(multiply(2, 4)); // 8
```

සටහන්: ආනයනය කරන විට නම් වරහන් { } ඇතුළත ලිවිය යුතුය.

44.4 පෙරනිමි අපනයනයන් (Default Exports)

යම් ගොනුවකින් ප්‍රධාන වශයෙන් එක් දෙයක් පමණක් පිටතට යැවීමට අවශ්‍ය විට මෙය භාවිතා කරයි. ගොනුවකට තිබිය හැක්කේ එක් default export එකක් පමණි.

ගොනුව: User.js

```
const user = {
  name: "කසුන්",
  age: 16
};
```

```
export default user; // මෙහිදී නමක් අවශ්‍ය නැත,
වස්තුවම යවයි
```

ගොනුව: main.js

```
import MyUser from './User.js'; // මෙහිදී කැමති
නමක් (MyUser) ලබා දිය හැක
```

```
console.log(MyUser.name); // කසුන්
```

විශේෂත්වය: Default export එකක් ලබා ගන්නා විට වරහන් { } අවශ්‍ය නොවන අතර, ඔබට අවශ්‍ය ඕනෑම නමක් ලබා දිය හැක.

44.5 HTML තුළ මොඩියුල ක්‍රියාත්මක කිරීම

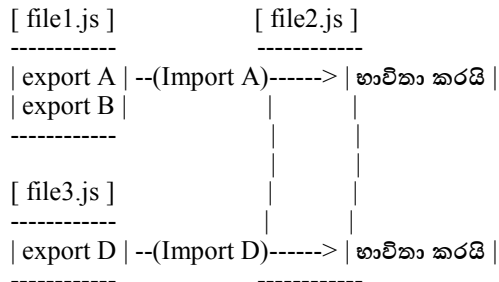
මොඩියුල භාවිතා කරන විට සාමාන්‍ය `<script>` ටැගය ප්‍රමාණවත් නොවේ. ඔබ අනිවාර්යයෙන්ම `type="module"` යන්න සඳහන් කළ යුතුය.

ගොනුව: index.html

```
<!-- වැදගත්: type="module" යෙදිය යුතුය -->
<script type="module" src="app.js"></script>
```

අවධානයට: ඔබ මොඩියුල භාවිතා කරන විට, ඔබේ පරිගණකයේ ඇති ගොනුව කෙලින්ම (Double click කර) බ්‍රවුසරයෙන් විවෘත කළ නොහැක. එය ආරක්ෂක හේතූන් මත (CORS policy) ක්‍රියා නොකරනු ඇත. ඔබ අනිවාර්යයෙන්ම VS Code හි Live Server වැනි ක්‍රමයක් භාවිතා කළ යුතුය.

44.6 ASCII රූපසටහන: මොඩියුල අතර ගමනාගමනය



44.7 වෘත්තීය පැහැදිලි කිරීම (Professional Explanation)

වෘත්තීය Frontend Architect කෙනෙකු ලෙස, කේතය මොඩියුල වලට බෙදීම "Separation of Concerns" (කාර්යයන් වෙන් කිරීම) ලෙස හඳුන්වනු ලැබේ. උදාහරණයක් ලෙස, වෙබ් අඩවියක ඇති API දත්ත ලබා ගන්නා කොටස් `api.js` ලෙසත්, හැඩතල පාලනය කරන කොටස් `ui.js` ලෙසත් වෙන් කිරීමෙන් මෘදුකාංගය නඩත්තු කිරීම (Maintenance) ඉතා පහසු වේ. නවීන Build Tools(Webpack, Vite) ක්‍රියා කරන්නේ ද මෙම මොඩියුල පද්ධතිය පදනම් කරගෙනය.

44.8 පොදු වැරදි (Common Mistakes)

1. File Path වැරදීම: `./` ලකුණ යෙදීමට අමතක වීම. `import {add} from 'math.js'` ලෙස ලිවීම වැරදිය. එය `import {add} from './math.js'` ලෙස තිබිය යුතුය.
2. Extension එක අමතක වීම: බ්‍රවුසරය තුළ මොඩියුල භාවිතා කරන විට ගොනු නාමය අගට `.js` යෙදීම අනිවාර්ය වේ.
3. Default Export දෙකක් සෑදීම: එකම ගොනුවක export default දෙකක් තිබිය නොහැක.
4. වරහන් පටලවා ගැනීම: Named exports සඳහා `{ }` අවශ්‍ය වන අතර Default exports සඳහා එය අවශ්‍ය නොවේ.

44.9 සාරාංශය (Key Points)

- Modules මගින් කේතය විවිධ ගොනු වලට බෙදා වෙන් කරයි.
- export මගින් දත්ත පිටතට ලබා දෙන අතර import මගින් දත්ත ලබා ගනී.
- Named exports කිහිපයක් තිබිය හැක; Default export ඇත්තේ එකක් පමණි.
- HTML හි `type="module"` භාවිතා කළ යුතුය.
- මොඩියුල ක්‍රියාත්මක කිරීමට Live Server එකක් අවශ්‍ය වේ.

44.10 අභ්‍යාස (Exercises)

1. Named Export සහ Default Export අතර ඇති ප්‍රධාන වෙනස්කම් 3ක් ලියන්න.
2. පහත කාර්යය සිදු කරන්න:
 - `greeting.js` නමින් ගොනුවක් සාදා එහි "Hello World" පෙන්වන ශ්‍රිතයක් සාදා එය export කරන්න.
 - `main.js` ගොනුව තුළ එම ශ්‍රිතය import කර ක්‍රියාත්මක කරන්න.
3. HTML ගොනුවක `type="module"` සඳහන් නොකළහොත් කුමක් සිදුවේද? ඔබම පරීක්ෂා කර බලන්න.
4. පහත කේතයේ ඇති වැරද්ද කුමක්ද? `import add from './math.js';` (මෙහි `math.js` තුළ ඇත්තේ `export const add = ...` නම්).

44.11 කුඩා ව්‍යාපෘතිය (Mini Project)

ව්‍යාපෘතිය: පුද්ගලික තොරතුරු පද්ධතිය (Personal Info System)

1. data.js: මෙහි ඔබේ නම, වයස, සහ ප්‍රියතම විෂයයන් අඩංගු වස්තුවක් (Object) export default කරන්න.
2. functions.js: මෙහි නමක් ලබා දුන් විට "සාදරයෙන් පිළිගනිමු [නම]" ලෙස පණිවිඩයක් මුද්‍රණය කරන ශ්‍රිතයක් export කරන්න.
3. app.js: ඉහත ගොනු දෙකම මෙයට import කර, ඔබගේ තොරතුරු කන්සෝලයේ පෙන්වන්න.

මිලිභ පරිච්ඡේදයෙන් අපි ජාවාස්ක්‍රිප්ට් වල කාලයන් සමඟ සිදුවන වැඩ පාලනය කරන ආකාරය හෙවත් "අසමකාලීන ක්‍රමලේඛනය" (Asynchronous JavaScript - Promises) ගැන ඉතාමත් විස්තරාත්මකව ඉගෙන ගනිමු.

හතළිස් පස්වන පරිච්ඡේදය: අසමකාලීන ජාවාස්ක්‍රිප්ට් - පොරොන්දු (Asynchronous JavaScript - Promises)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

1. සමකාලීන (Synchronous) සහ අසමකාලීන (Asynchronous) ක්‍රමලේඛනය අතර වෙනස වටහා ගැනීම.
2. ජාවාස්ක්‍රිප්ට් තනි ප්‍රවාහයක (Single-threaded) ක්‍රියා කරන ආකාරය තේරුම් ගැනීම.
3. අවහිර කිරීම (Blocking) යනු කුමක්දැයි හඳුනා ගැනීම.
4. පොරොන්දුවක් (Promise) යනු කුමක්ද සහ එහි ඇති වැදගත්කම වටහා ගැනීම.
5. පොරොන්දුවක පවතින අවස්ථා තුන (Pending, Resolved, Rejected) හඳුනා ගැනීම.
6. .then(), .catch() සහ .finally() භාවිතා කරමින් පොරොන්දු හැසිරවීම.
7. සැබෑ ලෝකයේ උදාහරණයක් ඇසුරින් පොරොන්දුවක් නිර්මාණය කිරීම.

45.1 හැඳින්වීම: සමකාලීන එදිරිව අසමකාලීන (Synchronous vs Asynchronous)

අපි මෙතෙක් ලියූ සියලුම ජාවාස්ක්‍රිප්ට් කේත ක්‍රියාත්මක වූයේ සමකාලීන

(Synchronous) ක්‍රමයටයි. එනම්, පරිගණකය පළමු ජේළිය කියවා අවසන් වූ පසු පමණක් දෙවන ජේළියට ගමන් කරයි.

සැබෑ ලෝකයේ උදාහරණයක් (පෝලිමක්): සිතන්න ඔබ බැංකුවක පෝලිමක ඉන්නවා කියා. එහි ඇත්තේ එකම එක කවුන්ටරයකි. ඉදිරියේ සිටින පුද්ගලයාගේ වැඩේ අවසන් වන තෙක් ඔබට ඊළඟ පියවරට යා නොහැක. මෙය Synchronous ක්‍රමයයි. නමුත් සැබෑ ලෝකයේ සමහර වැඩ නිම වීමට කාලය ගත වේ. උදාහරණයක් ලෙස:

- අන්තර්ජාලයෙන් විශාල පින්තූරයක් බාගත කිරීම.
- වෙනත් සේවාදායකයකින් (Server) දත්ත ලබා ගැනීම.

මෙවැනි වැඩක් සිදුවන තෙක් මුළු වෙබ් අඩවියම "නැවතී" (Freeze වී) තිබුණහොත් එය පරිශීලකයාට මහත් කරදරයකි. මෙන්න මේ ගැටලුව විසඳීමට අප අසමකාලීන (Asynchronous) ක්‍රමය භාවිතා කරමු.

අසමකාලීන උදාහරණය (ආපනශාලාවක්): ඔබ ආපනශාලාවකට ගොස් පීසා එකක් ඇණවුම් කරයි.

වේටර්වරයා ඔබට "බසර්" (Buzzer) එකක් ලබා දී වාඩි වෙන්නැයි පවසයි. දැන් ඔබ පීසා එක ලැබෙන තෙක් පෝලිමේ සිටිය යුතු නැත; ඔබට වෙනත් වැඩක් (මිතුරෙකු සමඟ කතා කිරීම) කළ හැක. පීසා එක සුදානම් වූ පසු බසර් එක ශබ්ද වේ. මෙය Asynchronous ක්‍රමයයි.

45.2 පොරොන්දුවක් යනු කුමක්ද? (What is a Promise?)

ජාවාස්ක්‍රිප්ට් හි Promise (පොරොන්දුව) යනු අනාගතයේදී සිදුවීමට නියමිත යම් ක්‍රියාවක ප්‍රතිඵලය නිරූපණය කරන වස්තුවකි.

සරලවම කිවහොත්, "මම දැන් වැඩක් පටන් ගන්නවා, ඒක ඉවර වුණාම ඔයාට ප්‍රතිඵලය (සතුටුදායක හෝ අසතුටුදායක) දෙනවා" කියා ජාවාස්ක්‍රිප්ට් අපට දෙන පොරොන්දුවකි.

45.3 පොරොන්දුවක අවස්ථා තුන (States of a Promise)

පොරොන්දුවක් සැමවිටම පහත සඳහන් තත්වයන් තුනෙන් එකක පවතී:

1. Pending (අපේක්ෂිත): පොරොන්දුව ලබා දුන් මොහොතේ සිට වැඩේ අවසන් වන තෙක් පවතින කාලය. (තවම ප්‍රතිඵලයක් නැත).
2. Fulfilled / Resolved (සාර්ථක): පොරොන්දු වූ වැඩේ සාර්ථකව අවසන් වී අපට අවශ්‍ය දත්ත ලැබුණු අවස්ථාව.
3. Rejected (අසාර්ථක): යම් වැරද්දක් නිසා (උදා: අන්තර්ජාලය නැති වීම) වැඩේ කරගත නොහැකි වූ අවස්ථාව.

45.4 පොරොන්දුවක් නිර්මාණය කරන ආකාරය (Creating a Promise)

අපි දැන් සරල පොරොන්දුවක් කේතය හරහා නිර්මාණය කරමු.

```
const myFirstPromise = new Promise((resolve, reject) => {
  // තත්පර 2ක කාලයක් ගත වන වැඩක් යැයි සිතන්න
  let isSuccess = true;

  setTimeout(() => {
    if (isSuccess) {
      resolve("වැඩේ සාර්ථකයි! මෙන්න දත්ත.");
    } else {
      reject("වැඩේ අසාර්ථකයි! වැරද්දක් වුණා.");
    }
  }, 2000); // මිලි තත්පර 2000 = තත්පර 2
});
```

කේතය පැහැදිලි කිරීම:

- new Promise(): අලුත් පොරොන්දුවක් සාදයි.
- resolve: වැඩේ හරියට වුණොත් අප අමතන ශ්‍රිතයයි.
- reject: වැඩේ වැරදුණොත් අප අමතන ශ්‍රිතයයි.
- setTimeout(): යම් කාලයක් පමා කිරීමට භාවිතා කරන ජාවාස්ක්‍රිප්ට් ශ්‍රිතයකි.

45.5 පොරොන්දුවක ප්‍රතිඵලය ලබා ගැනීම (Consuming a Promise)

පොරොන්දුවක් සෑදූ පමණින් වැඩක් නැත, එහි ප්‍රතිඵලය අප ලබා ගත යුතුය. ඒ සඳහා ප්‍රධාන ක්‍රමවේද තුනක් ඇත.

1. .then(): පොරොන්දුව සාර්ථක වූ විට (resolve වූ විට) ක්‍රියාත්මක වේ.
2. .catch(): පොරොන්දුව අසාර්ථක වූ විට (reject වූ විට) වැරද්ද හඳුනා ගැනීමට භාවිතා කරයි.
3. .finally(): සාර්ථක වුවත් අසාර්ථක වුවත් අවසානයේදී අනිවාර්යයෙන්ම සිදුවිය යුතු

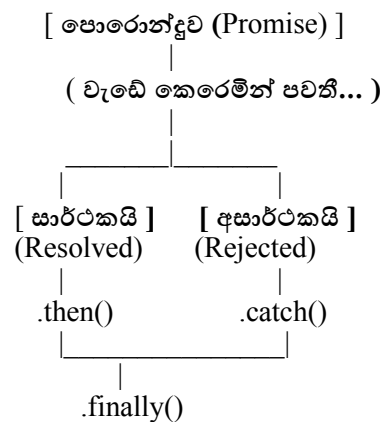
දෙයක් තිබේ නම් මෙහි ලියයි.

උදාහරණය:

```
myFirstPromise
  .then((data) => {
    console.log(data); // "වැඩේ සාර්ථකයි! මෙන්න දත්ත."
  })
  .catch((error) => {
    console.log(error); // වැරද්දක් වුණොත් මෙතැනට එයි
  })
  .finally(() => {
    console.log("ක්‍රියාවලිය අවසන්.");
  });
```

45.6 ASCII රූපසටහන:

පොරොන්දුවක ගමන



45.7 වෘත්තීය පැහැදිලි කිරීම (Professional Explanation)

සැබෑ මෘදුකාංග සංවර්ධනයේදී අප විසින් පොරොන්දු නිර්මාණය කිරීම (Creating Promises) කරන්නේ ඉතා අඩුවෙනි. ඒ වෙනුවට අප කරන්නේ ඔවුසරය හෝ වෙනත් පුස්තකාල (Libraries) මගින් අපට ලබා දෙන පොරොන්දු භාවිතා කිරීමයි (Consuming Promises). උදාහරණයක් ලෙස fetch() ශ්‍රිතය භාවිතා කර අන්තර්ජාලයෙන් දත්ත ලබා ගැනීමේදී ලැබෙන්නේ පොරොන්දුවකි.

මෙය Callback Hell (කේතය පාලනය කළ නොහැකි ලෙස සංකීර්ණ වීම) වළක්වා ගැනීමට හඳුන්වා දුන් ඉතා පිරිසිදු ක්‍රමයකි.

45.8 පොදු වැරදි (Common Mistakes)

1. `.catch()` අමතක වීම: පොරොන්දුවක් අසාර්ථක වූ විට එය හසුකර නොගන්නා බැවින්, ඔවුන්ගේ "Uncaught Error" නමැති රතු පැහැති වැරද්දක් දිස්වේ. මෙය වෙබ් අඩවිය අඩපණ වීමට හේතු විය හැක.
2. අසමකාලීන බව අමතක කිරීම: පොරොන්දුවක් ඇතුළත ඇති අගයක් ඊට පිටතදී වහාම භාවිතා කිරීමට උත්සාහ කිරීම. (වැඩේ අවසන් වීමට පෙර ප්‍රතිඵලය සෙවීම).

45.9 සාරාංශය (Key Points)

- Synchronous කේතය ජේළියෙන් ජේළිය ක්‍රියාත්මක වේ; Asynchronous කේතය පසුබිමෙන් ක්‍රියාත්මක වේ.
- Promise යනු අසමකාලීන වැඩක ප්‍රතිඵලය පෙන්වන වස්තුවකි.
- අවස්ථා 3: Pending, Resolved (Fulfilled), Rejected.
- ප්‍රතිඵලය හැසිරවීමට `.then()` සහ `.catch()` භාවිතා කරයි.

45.10 අභ්‍යාස (Exercises)

1. සැබෑ ලෝකයේ අසමකාලීන (Asynchronous) වැඩක් සිදුවන අවස්ථාවක් උදාහරණයකින් පැහැදිලි කරන්න.
2. පහත කාර්යය සඳහා පොරොන්දුවක් (Promise) ලියන්න:
 - අංකයක් පරාමිතියක් ලෙස ගෙන, එම අංකය 10 ට වඩා වැඩි නම් resolve කරන්න, 10 ට වඩා අඩු නම් reject කරන්න.
 - ඉන්පසු එම පොරොන්දුව ක්‍රියාත්මක කර `.then()` සහ `.catch()` හරහා පණිවිඩ පෙන්වන්න.
3. `.finally()` ප්‍රකාශනය භාවිතා කිරීම වැදගත් වන අවස්ථාවක් පවසන්න.

45.11 සම්මුඛ පරීක්ෂණ ප්‍රශ්න (Interview Questions)

- ප්‍රශ්නය: "ජාවාස්ක්‍රිප්ට් තනි ප්‍රවාහයක (Single-threaded) ක්‍රියා කරන භාෂාවක් වුවත්, එය අසමකාලීන වැඩ කරන්නේ කෙසේද?"
- පිළිතුර: ජාවාස්ක්‍රිප්ට් එන්ජිම තනි ප්‍රවාහයක වැඩ කළද, ඔවුන්ගේ සතුටු Web APIs (උදා: `setTimeout`, `fetch`) ඇත. මෙම අසමකාලීන වැඩ ඔවුන්ගේ පසුබිමෙන්

සිදුවන අතර, ඒවා අවසන් වූ පසු "Event Loop" එක හරහා නැවත ජාවාස්ක්‍රිප්ට් වෙත දත්ත එවනු ලැබේ.

මිලන පරිච්ඡේදයෙන් අපි පොරොන්දු හැසිරවීමට ඇති නවීනතම සහ පහසුම ක්‍රමය වන "Async/Await" ගැන ඉතාමත් විස්තරාත්මකව ඉගෙන ගනිමු.

හතළිස් හයවන පරිච්ඡේදය: අසින්ක් සහ අවේට් - අසමකාලීන කේතය සරල කිරීම (Async and Await - Modern Asynchronous JS)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

1. `async` සහ `await` යනු කුමක්දැයි වෘත්තීය මට්ටමින් තේරුම් ගැනීම.
2. පොරොන්දු (Promises) හැසිරවීමට `.then()` ක්‍රමයට වඩා මෙය දියුණු වන්නේ ඇයි දැයි වටහා ගැනීම.
3. `async` මූලපදය භාවිතා කර ශ්‍රිතයක් අසමකාලීන කරන ආකාරය.
4. `await` මූලපදය මගින් කේතය ක්‍රියාත්මක වීම තාවකාලිකව නවත්වන (Pause) ආකාරය.
5. අසමකාලීන කේතවල සිදුවන වැරදි හැසිරවීමට `try...catch` භාවිතා කරන ආකාරය.
6. සැබෑ ලෝකයේ උදාහරණයක් ලෙස දත්ත ලබා ගැනීමේ ක්‍රියාවලියක් (Data Fetching) සරලව ගොඩනැගීම.

46.1 හැඳින්වීම: ඇයි අපට `async/await` අවශ්‍ය වන්නේ?

පෙර පරිච්ඡේදයේදී අප ඉගෙන ගත්තේ පොරොන්දු (Promises) සහ ඒවායේ ප්‍රතිඵල ලබා ගැනීමට භාවිතා කරන `.then()` සහ `.catch()` ක්‍රමවේදයන්ය. පොරොන්දු ඉතා විශිෂ්ට වුවත්, අපට එකකට පසුව එකක් ලෙස සිදුවන අසමකාලීන වැඩ රාශියක් ඇති විට කේතය ඉතාමත් සංකීර්ණ සහ කියවීමට අපහසු තත්වයට පත් වේ. මෙයට "Promise Chaining" ගැටලුව යැයි කියනු ලැබේ.

2017 වසරේදී ජාවාස්ක්‍රිප්ට් වලට Async/Await හඳුන්වා දෙන ලදී. මෙහි ප්‍රධාන අරමුණ වන්නේ අසමකාලීන කේත (Asynchronous code), දෘශ්‍යමාන ලෙස සමකාලීන කේත (Synchronous code) ලෙස පෙන්වීමයි. එනම්, අසමකාලීන කේතයක් වුවද සාමාන්‍ය කේතයක් මෙන් ඉහළ සිට පහළට ඉතා පිරිසිදුව ලිවීමට මෙයින් ඉඩ ලබා දෙයි.

46.2 async මූලපදය (The async Keyword)

ඕනෑම ශ්‍රිතයක් (Function) ඉදිරියෙන් async යන වචනය යෙදූ විට, එම ශ්‍රිතය සැමවිටම පොරොන්දුවක් (Promise) ආපසු ලබා දෙයි. ඔබ එම ශ්‍රිතයෙන් සාමාන්‍ය අගයක් ලබා දුන්නද (Return කළද), ජාවාස්ක්‍රිප්ට් විසින් එය ස්වයංක්‍රීයව පොරොන්දුවක් තුළ අසුරා (Wrap කර) එවනු ලබයි.

උදාහරණය:

```
async function greet() {
  return "ආයුබෝවන්!";
}
```

```
// මෙය ක්‍රියාත්මක කළ විට ලැබෙන්නේ
"ආයුබෝවන්" යන අකුරු නොව,
පොරොන්දුවකි.
greet().then(res => console.log(res));
```

46.3 await මූලපදය (The await Keyword)

await යන්නෙහි තේරුම "ඉවසන්න" හෝ "පොරොන්දු වන්න" යන්නයි. මෙම මූලපදය භාවිතා කළ හැක්කේ async ශ්‍රිතයක් ඇතුළත පමණි.

යම් පොරොන්දුවක් ඉදිරියෙන් await යෙදූ විට, එම පොරොන්දුව සාර්ථක (Resolved) වන තෙක් ජාවාස්ක්‍රිප්ට් එන්ජින් එම ජේළියේදී නතර වී සිටියි. පොරොන්දුව අවසන් වූ පසු එහි ප්‍රතිඵලය ලබාගෙන ඊළඟ ජේළියට ගමන් කරයි.

වැදගත්: මෙය ඔවුසරය හිර කරන්නේ නැත. එය කරන්නේ එම විශේෂිත ශ්‍රිතයේ ක්‍රියාකාරිත්වය පමණක් තාවකාලිකව නැවැත්වීමයි.

46.4 පිරිසිදු කේත සංසන්දනය (Comparison)

අපි පොරොන්දුවක් හැසිරවීමට ඇති පැරණි ක්‍රමය සහ නවීන ක්‍රමය සසඳා බලමු.

පැරණි ක්‍රමය (Promises with .then):

```
function getData() {
  fetchData()
    .then(res => {
      console.log(res);
      return processData(res);
    })
    .then(processedRes => {
      console.log(processedRes);
    })
    .catch(err => {
      console.log(err);
    });
}
```

නවීන ක්‍රමය (Async/Await):

```
async function getData() {
  try {
    const res = await fetchData(); // වැඩේ
    ඉවර වෙනකම් මෙතන නතර වෙනවා
    console.log(res);

    const processedRes = await
    processData(res); // ඊළඟ වැඩේටත් ඉවසනවා
    console.log(processedRes);
  } catch (err) {
    console.log(err); // වැරදුණොත් මෙතැනට
    එනවා
  }
}
```

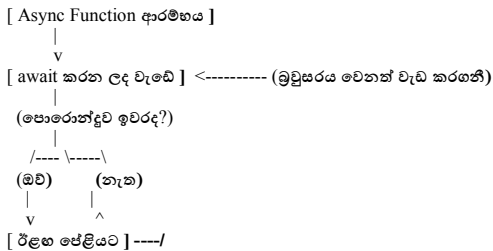
දැක්කාද? දෙවන ක්‍රමය සාමාන්‍ය කේතයක් මෙන් කියවීමට ඉතා පහසුයි.

46.5 වැරදි හැසිරවීම: try...catch (Error Handling)

.then() ක්‍රමයේදී අප වැරදි හසුකර ගැනීමට .catch() භාවිතා කළෙමු. async/await ක්‍රමයේදී ඒ සඳහා ජාවාස්ක්‍රිප්ට් හි ඇති සම්මත වැරදි හැසිරවීමේ ක්‍රමය වන try...catch භාවිතා කරයි.

- try { ... } : නිවැරදිව සිදුවිය යුතු කේතය මෙහි ලියයි.
 - catch (error) { ... } : කිසියම් පේළියක වැරද්දක් (Promise rejection) සිදු වුවහොත් වහාම මෙම කොටස ක්‍රියාත්මක වේ.

46.6 ASCII රූපසටහන: Async/Await ක්‍රියාවලිය



46.7 ප්‍රායෝගික උදාහරණය: කෝපි ඇණවුම් කිරීමේ පද්ධතිය

කේතය:

```
// 1. පොරොන්දුවක් සාදමු (කෝපි සෑදීමට කාලය ගත වේ)
const makeCoffee = () => {
  return new Promise((resolve, reject) => {
    let machineWorking = true;
    setTimeout(() => {
      if (machineWorking) {
        resolve("ඔබේ රසවත් කෝපි කෝප්පය සුදානම්! ☕");
      } else {
        reject("කෝපි යන්ත්‍රය ක්‍රියා විරහිතයි! ✖");
      }
    }, 3000);
  });
};
```

```
// 2. අයිනික් ශ්‍රිතය හරහා එය ලබා ගනිමු
async function orderCoffee() {
  console.log("ඇණවුම භාරගත්තා. කරුණාකර ඳි සිටින්න...");

  try {
    const status = await makeCoffee(); // තත්පර 3ක් මෙතුන නතර වේ
    console.log(status);
    console.log("ඔබට ස්තුතියි, නැවත එන්න!");
  } catch (error) {
```

```
    console.log(error);
  }
}

orderCoffee();
```

46.8 වෘත්තීය පැහැදිලි කිරීම (Professional Explanation)

async/await යනු හුදෙක් පොරොන්දු වලට උඩින් ඇති "Syntactic Sugar" (පහසුව සඳහා සාදන ලද ආවරණයක්) පමණි. මෙහි පසුබිමෙහි තවමත් ක්‍රියාත්මක වන්නේ Promises ය. වෘත්තීය මට්ටමේදී, API හරහා දත්ත ලබා ගැනීමේදී (Fetch API) අප 99% ක්ම භාවිතා කරන්නේ මෙම async/await ක්‍රමයයි. මන්දයත් එය "Callback Hell" වළක්වා ගැනීමටත්, කේතය පරීක්ෂා කිරීමට (Testing) සහ දෝෂ සෙවීමට (Debugging) ඉතා පහසු බැවිනි.

46.9 පොදු වැරදි (Common Mistakes)

1. Await තනිව භාවිතා කිරීම: await භාවිතා කළ හැක්කේ async ශ්‍රිතයක් ඇතුළත පමණි. නැතිනම් "SyntaxError" එකක් ලැබේ.
2. Try/Catch අමතක වීම: අසමකාලීන වැඩවලදී වැරදි සිදුවීමට ඇති ඉඩ වැඩිය (උදා: අන්තර්ජාලය විසන්ධි වීම). එම නිසා සැමවිටම try...catch භාවිතා කරන්න.
3. සමාන්තරව (Parallel) කළ හැකි වැඩ එකකට පසුව එකක් කිරීම: වැඩ දෙකක් එකිනෙකට සම්බන්ධ නැතිනම්, await දෙකක් භාවිතා කර එකිනෙක පමා නොකර Promise.all() වැනි ක්‍රම භාවිතා කළ යුතුය (මෙය පසුව ඉගෙන ගනිමු).

46.10 සාරාංශය (Key Points)

- async ශ්‍රිතයක් සැමවිටම Promise එකක් ලබා දෙයි.
- await මගින් Promise එකක් අවසන් වන තෙක් ශ්‍රිතය පාලනය කරයි.
- await භාවිතා කළ හැක්කේ async ශ්‍රිත තුළ පමණි.
- වැරදි හැසිරවීමට try...catch අත්‍යවශ්‍ය වේ.
- මෙය කේතය කියවීමට සහ නඩත්තු කිරීමට පහසු කරයි.

46.11 අභ්‍යාස (Exercises)

1. `.then()` භාවිතා කර ලියන ලද ඕනෑම පොරොන්දුවක් `async/await` ක්‍රමයට පරිවර්තනය කරන්න.
2. පහත පියවරවල් අසින්ක් ශ්‍රිතයක් හරහා සිදු කරන්න:
 - පියවර 1: "පරිශීලකයා සොයනවා..." (තත්පර 1ක් පමා කරන්න).
 - පියවර 2: "පරිශීලකයා හමු වුණා." (තත්පර 2ක් පමා කරන්න).
 - පියවර 3: අවසාන දත්ත මුද්‍රණය කරන්න.
3. `async` ශ්‍රිතයක් ඇතුළත `throw` දෙයක් භිතාමතාම සිදු කර (Throw error), එය `catch` කොටසින් හසුකර ගන්නා ආකාරය පෙන්වන්න.

46.12 සම්මුඛ පරීක්ෂණ ප්‍රශ්න (Interview Questions)

- ප්‍රශ්නය: `await` භාවිතා කිරීම නිසා මුළු JavaScript වැඩසටහනම නිරවද්‍ය වෙනවාද?
- පිළිතුර: නැත. `await` මගින් කරන්නේ එම `async` ශ්‍රිතයේ ක්‍රියාකාරිත්වය පමණක් අත්හිටුවීමයි. ජාවාස්ක්‍රිප්ට් එන්ජින්ට එම කාලය තුළ වෙනත් වැඩ (උදා: Click events, animations) කරගෙන යාමට ඉඩ සැලසීමට දෙයි.

මිලහ පරිච්ඡේදයෙන් අපි අන්තර්ජාලයේ ඇති සැබෑ සේවාදායකයන් (Servers) සමඟ ගනුදෙනු කරමින් දත්ත ගෙන එන ආකාරය හෙවත් "Fetch API" ගැන ඉතාමත් විස්තරාත්මකව ඉගෙන ගනිමු.

හතළිස් හත්වන පරිච්ඡේදය: ගෙවී ඒ.පී.අයි. සහ ජේසන් - අන්තර්ජාලයෙන් දත්ත ලබා ගැනීම (Fetch API & JSON)

ඉගෙනුම් අරමුණු (Learning Objectives)

- මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:
1. ඒ.පී.අයි. (API) යනු කුමක්ද සහ එහි වැදගත්කම වටහා ගැනීම.
 2. ජේසන් (JSON) දත්ත හැඩතලය හඳුනා ගැනීම සහ එහි ව්‍යුහය තේරුම් ගැනීම.

3. Fetch API භාවිතා කරමින් බාහිර සේවාදායකයකට (Server) ඉල්ලීමක් (Request) යවන ආකාරය.
4. ලැබෙන ප්‍රතිචාරය (Response) ජාවාස්ක්‍රිප්ට් වස්තුවක් බවට පරිවර්තනය කරන ආකාරය.
5. `async/await` සමඟ `fetch()` භාවිතා කර දත්ත ගනිමට (Dynamically) වෙබ් පිටුවක පෙන්වන ආකාරය.
6. ජාල දෝෂ (Network Errors) සහ HTTP වැරදි හසුකර ගන්නා ආකාරය.

47.1 ඒ.පී.අයි. (API) යනු කුමක්ද?

API යනු Application Programming Interface යන්නෙහි කෙටි නමයි. මෙය මෘදුකාංග දෙකක් අතර තොරතුරු හුවමාරු කර ගැනීමට භාවිතා කරන "පාලමක්" වැනිය.

සැබෑ ලෝකයේ උදාහරණයක් (ආපනශාලාව):

- ඔබ (Client): මේසයේ වාඩි වී සිටින පාරිභෝගිකයා.
- කුස්සිය (Server): ආහාර පිළියෙළ කරන ස්ථානය (දත්ත ගබඩාව).
- වේටර් (API): ඔබ ආහාරයක් ඇණවුම් කළ විට (Request), එම පණිවිඩය කුස්සියට රැගෙන ගොස්, ආහාරය සුදානම් වූ පසු එය නැවත ඔබ වෙත රැගෙන එන (Response) පුද්ගලයා.

වෙබ් සංවර්ධනයේදී, ඔබේ වෙබ් අඩවියට කාලගුණ තොරතුරු, ක්‍රිකට් ලකුණු, හෝ මුදල් පරිවර්තන අනුපාත අවශ්‍ය නම්, ඔබ කරන්නේ අදාළ සමාගමක API එකකට ඉල්ලීමක් යැවීමයි.

47.2 ජේසන් (JSON) යනු කුමක්ද?

JSON යනු JavaScript Object Notation යන්නයි. මෙය අන්තර්ජාලය හරහා දත්ත හුවමාරු කර ගැනීමට භාවිතා කරන ලෝක සම්මත "භාෂාව" හෝ හැඩතලයයි. මෙය ජාවාස්ක්‍රිප්ට් වල Objects වලට 99% ක්ම සමාන වේ.

JSON ව්‍යුහය:

```
{
  "name": "කසුන්",
  "age": 16,
  "isStudent": true,
  "subjects": ["Maths", "ICT"]
}
```

වැදගත්: JSON වලදී "Key" (යතුර) සැමවිටම ද්විත්ව උද්ධෘත ලකුණු (" ") ඇතුළත ලිවිය යුතුය.

47.3 ලෙච් ජී.පී.අයි. (Fetch API) හඳුන්වාදීම

Fetch API යනු බ්‍රවුසරය විසින් ලබා දෙන නවීනතම උපකරණයකි. මෙහි කාර්යය වන්නේ වෙබ් ලිපිනයකට (URL) ගොස් එතැන ඇති දත්ත අපේ ජාවාස්ක්‍රිප්ට් කේතය තුළට රැගෙන ඒමයි. fetch() ශ්‍රිතය ක්‍රියාත්මක වූ විට එය සැමවිටම පොරොන්දුවක් (Promise) ආපසු ලබා දෙයි.

47.4 දත්ත ලබා ගැනීමේ පියවර (Step-by-Step Data Fetching)

අපි සාමාන්‍යයෙන් අන්තර්ජාලයෙන් දත්ත ලබා ගන්නා විට පියවර දෙකක් අනුගමනය කරමු:

1. URL එකට ඉල්ලීමක් යවා "ප්‍රතිචාරය" (Response) ලබා ගැනීම.
2. එම ප්‍රතිචාරය අපට තේරෙන JSON (JavaScript Object) බවට හරවා ගැනීම.

47.5 ප්‍රායෝගික කේත උදාහරණය

අපි දැන් JSONPlaceholder නමැති නොමිලේ දත්ත ලබා දෙන සේවාවකින් පරිශීලක තොරතුරු ලබා ගනිමු.

```
async function getUserData() {
  try {
    // පියවර 1: දත්ත ලබා ගැනීමට ඉල්ලීමක් යැවීම
    const response = await
    fetch('https://jsonplaceholder.typicode.com/users/1');

    // පියවර 2: සර්වර් එකෙන් පිළිතුරක් ආවාදැයි පරීක්ෂා කිරීම
    if (!response.ok) {
      throw new Error("දත්ත ලබා ගැනීමට නොහැකි විය!");
    }

    // පියවර 3: දත්ත JSON ලෙස පරිවර්තනය කිරීම
    const data = await response.json();
```

```
// පියවර 4: දත්ත භාවිතා කිරීම
console.log("නම:", data.name);
console.log("විද්‍යුත් තැපෑල:", data.email);
console.log("නගරය:", data.address.city);
```

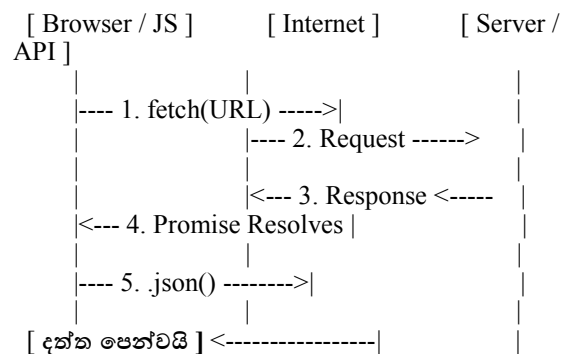
```
} catch (error) {
  console.error("වැරද්දක් සිදු විය:",
  error.message);
}
```

```
getUserData();
```

47.6 කේතය විස්තරාත්මකව පැහැදිලි කිරීම

1. fetch('URL'): මෙමගින් අදාළ ලිපිනයට දත්ත ඉල්ලා පණිවිඩයක් යවයි.
2. await: සර්වර් එකෙන් පිළිතුරක් ලැබෙන තෙක් කේතය තාවකාලිකව නතර කරයි.
3. response.ok: සර්වර් එක සාර්ථකව පිළිතුරක් දුන්නේ නම් (Status 200-299) මෙය true වේ.
4. response.json(): සර්වර් එකෙන් ලැබෙන්නේ අකුරු ගොඩකි. එය ජාවාස්ක්‍රිප්ට් වස්තුවක් (Object) බවට හැරවීමට මෙය භාවිතා කරයි. මෙයට ද සුළු කාලයක් ගත වන බැවින් ඒ ඉදිරියෙන් ද await යෙදිය යුතුය.

47.7 ASCII රූපසටහන: Fetch ක්‍රියාවලිය



47.8 පොදු වැරදි (Common Mistakes)

1. .json() ඉදිරියෙන් await අමතක වීම: දත්ත පරිවර්තනය ද අසමකාලීන වැඩක් බැවින් await අනිවාර්ය වේ.
2. වැරදි URL භාවිතය: URL එකේ එක අකුරක් හෝ වැරදුනහොත් දත්ත නොලැබේ.

3. CORS Error: සමහර වෙබ් අඩවි ආරක්ෂක හේතූන් මත වෙනත් අඩවි වලට දත්ත ලබා දීම තහනම් කර ඇත. එවිට CORS (Cross-Origin Resource Sharing) වැරද්දක් ලැබේ.

47.9 වෘත්තීය උපදෙස් (Professional Tips)

- Loading State: දත්ත ලැබෙන තෙක් වෙබ් පිටුවේ "දත්ත බාගත වෙමින් පවතී (Loading...)" යනුවෙන් පණිවිඩයක් පෙන්වීම හොඳ පරිශීලක අත්දැකීමකි (UX).
- Environment Variables: සැබෑ වැඩබිමකදී API ලිපින කෙලින්ම කේතය තුළ ලියන්නේ නැත. ඒවා රහස්‍ය ගොනු තුළ තැන්පත් කරයි.
- Error Messages: පරිශීලකයාට වැරද්දක් සිදු වූ විට එය සිංහලෙන් හෝ සරල ඉංග්‍රීසියෙන් පැහැදිලිව පෙන්වන්න.

47.10 සාරාංශය (Key Points)

- API යනු දත්ත ලබා ගැනීමට ඇති දොරටුවයි.
- JSON යනු දත්ත හුවමාරු වන සම්මත හැඩතලයි.
- fetch() මගින් දත්ත ලබා ගැනීමට ඉල්ලීම් යවයි.
- response.json() මගින් දත්ත භාවිතා කළ හැකි තත්ත්වයට පත් කරයි.
- try...catch මගින් අන්තර්ජාල සම්බන්ධතාවය බිඳ වැටීම වැනි දෝෂ හසුකර ගනී.

47.11 අභ්‍යාස (Exercises)

1. JSON සහ JavaScript Object අතර ඇති ප්‍රධාන වෙනස කුමක්ද?
2. පහත URL එක භාවිතා කර අහඹු උපදෙසක් (Advice) ලබා ගෙන එය කන්සෝලයේ පෙන්වන ශ්‍රිතයක් ලියන්න:
<https://api.advicejslip.com/advice>
3. fetch() එකකදී response.ok පරීක්ෂා කිරීම වැදගත් වන්නේ ඇයි?

47.12 කුඩා ව්‍යාපෘතිය (Mini Project)

ව්‍යාපෘතිය: පරිශීලක නාමාවලිය (User Profile Card)

1. HTML පිටුවක එක බොත්තමක් සහ හිස් div එකක් සාදන්න.

2. බොත්තම ක්ලික් කළ විට <https://jsonplaceholder.typicode.com/users/1> ලිපිනයෙන් දත්ත ලබා ගන්න.
3. ලැබෙන දත්ත (නම, විද්‍යුත් තැපෑල, දුරකථන අංකය) ලස්සන කාඩ්පතක් ලෙස HTML div එක ඇතුළත පෙන්වන්න.

මිළන පරිච්ඡේදයෙන් අපි ජාවාස්ක්‍රිප්ට් වල වැරදි සිදු වූ විට ඒවා හඳුනා ගන්නා සහ පාලනය කරන ආකාරය හෙවත් "දෝෂ හැසිරවීම" (Error Handling) ගැන ඉතාමත් විස්තරාත්මකව ඉගෙන ගනිමු.

හතළිස් අටවන පරිච්ඡේදය: දෝෂ හැසිරවීම - වැරදි පාලනය කරමු (Error Handling - try...catch)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

1. ජාවාස්ක්‍රිප්ට් හි දෝෂ (Errors) යනු කුමක්ද සහ ඒවා සිදුවන ආකාරය තේරුම් ගැනීම.
2. ප්‍රධාන දෝෂ වර්ග (Syntax, Reference, Type) හඳුනා ගැනීම.
3. try...catch ව්‍යුහය භාවිතා කරමින් කේතයක් බිඳ වැටීම (Crash වීම) වළක්වා ගන්නා ආකාරය.
4. finally කොටසෙහි ඇති වැදගත්කම වටහා ගැනීම.
5. throw මූලපදය භාවිතා කරමින් තමන්ගේම දෝෂ (Custom Errors) නිර්මාණය කරන ආකාරය.
6. දෝෂ වස්තුව (Error Object) තුළ ඇති තොරතුරු කියවන ආකාරය.
7. වෘත්තීය මට්ටමේ දෝෂ හැසිරවීමේ හොඳම පුරුදු (Best Practices) ඉගෙන ගැනීම.

48.1 හැඳින්වීම: දෝෂයක් (Error) යනු කුමක්ද?

ක්‍රමලේඛනයේදී (Programming) වැරදි සිදුවීම ඉතාමත් සාමාන්‍ය දෙයකි. ඔබ කෙතරම් දක්ෂ සංවර්ධකයෙකු වුවද, යම් දිනක ඔබේ කේතයේ වැරද්දක් සිදුවිය හැක.

ජාවාස්ක්‍රිප්ට් වලදී වැරද්දක් සිදු වූ විට, ඔවුසරය සාමාන්‍යයෙන් කරන්නේ කේතය ක්‍රියාත්මක කිරීම සම්පූර්ණයෙන්ම නතර කර රතු පැහැති පණිවිඩයක් කන්සෝලයේ පෙන්වීමයි.

මෙය වෙබ් අඩවිය භාවිතා කරන පරිශීලකයාට (User) ඉතා නරක අත්දැකීමකි, මන්ද වෙබ් අඩවිය හිර වී හෝ වැඩ නොකර පවතින බැවිනි. සැබෑ ලෝකයේ උදාහරණයක්: සිතන්න ඔබ වාහනයක් පදවනවා කියා. වාහනයේ එන්ජිමේ යම් දෝෂයක් ආ විට, වාහනය පාර මැදම නැවතී යනවාට වඩා, ඩෑෂ්බෝඩ් එකේ කුඩා ලාම්පුවක් දැල්වී "කරුණාකර කාර්මිකයෙකු හමුවන්න" යන පණිවිඩය පෙන්වමින් ආරක්ෂිතව වාහනය පසෙකට ගැනීමට හැකි වීම වඩා හොඳය.

Error Handling යනු ජාවාස්ක්‍රිප්ට් වලදී මෙලෙස වැරදි සිදුවන විට වැඩසටහන සම්පූර්ණයෙන් නතර වීමට ඉඩ නොදී, එම වැරද්ද "හසුකර ගෙන" (Catch කර), එයට සුදුසු පිළියමක් කිරීමයි.

48.2 ප්‍රධාන දෝෂ වර්ග (Types of Errors)

ජාවාස්ක්‍රිප්ට් වල අපට නිතරම හමුවන දෝෂ වර්ග 3ක් ඇත:

1. Syntax Error (ව්‍යාකරණ දෝෂ): භාෂාවේ නීති රීති වැරදි ලෙස ලිවීම. (උදා: වරහනක් වැසීමට අමතක වීම).
2. Reference Error (සබැඳි දෝෂ): ප්‍රකාශ කර නැති (Declare කර නැති) විචල්‍යයක් භාවිතා කිරීමට උත්සාහ කිරීම.
3. Type Error (වර්ගීකරණ දෝෂ): යම් දත්ත වර්ගයකට කළ නොහැකි දෙයක් කිරීමට උත්සාහ කිරීම. (උදා: අංකයක් අකුරු ලෙස සලකා toUpperCase() කිරීමට උත්සාහ කිරීම).

48.3 try...catch ව්‍යුහය

වැරදි පාලනය කිරීමට ජාවාස්ක්‍රිප්ට් අපට ලබා දෙන ප්‍රධානතම මෙවලම මෙයයි.

ව්‍යුහය (Syntax):

```
try {
    // වැරදීමට ඉඩ ඇති කේතය මෙහි ලියන්න
} catch (error) {
    // වැරද්දක් වුණොත් සිදු විය යුතු දේ මෙහි ලියන්න
}
```

එය ක්‍රියා කරන ආකාරය:

1. මූලිකම ඔවුසරය try කොටස ඇතුළත ඇති කේතය ක්‍රියාත්මක කිරීමට උත්සාහ කරයි.
2. එහි කිසිදු වැරද්දක් නැත්නම්, catch කොටස මඟ හැර කේතය දිගටම කරගෙන යයි.
3. යම් හෙයකින් try ඇතුළත වැරද්දක් සිදු වුවහොත්, වහාම එම පේළියේදී ක්‍රියාවලිය නතර වී පාලනය catch කොටසට මාරු වේ.

ප්‍රායෝගික උදාහරණය:

```
try {
    let x = 10;
    console.log(x + y); // y ප්‍රකාශ කර නැති නිසා මෙතැනදී වැරද්දක් සිදුවේ (Reference Error)
} catch (err) {
    console.log("සමාවන්න, යම් වැරද්දක් සිදු විය!");
    console.log("වැරද්දේ විස්තරය: " + err.message);
}
```

```
console.log("වැඩසටහන තවමත් ක්‍රියාත්මකයි...");
```

මෙහිදී වැරද්දක් සිදු වුවද, අවසාන පේළිය මුද්‍රණය වේ. එනම් වැඩසටහන නතර වූයේ නැත.

48.4 දෝෂ වස්තුව (The Error Object)

catch(error) යන වරහන තුළ ඇති error යනු ඔවුසරය විසින් අපට ලබා දෙන වස්තුවකි. එහි වැදගත් ගුණාංග දෙකක් ඇත:

- error.name: දෝෂයේ නම (උදා: ReferenceError).
- error.message: දෝෂය පිළිබඳ විස්තරය.

48.5 finally කොටස

වැරද්දක් සිදු වුවත්, සිදු නොවුවත් අනිවාර්යයෙන්ම අවසානයට සිදු විය යුතු දෙයක් තිබේ නම් එය finally තුළ ලියයි. උදාහරණය:

```
try {
    console.log("ගොනුවක් විවෘත කරනවා...");
    // යම් කේතයක්...
} catch (err) {
    console.log("ගොනුව කියවීමේදී වැරද්දක්!");
} finally {
    console.log("වැඩේ ඉවරයි, දැන් ගොනුව වසා දමනවා.");
}
```


48.6 තමන්ගේම දෝෂ නිර්මාණය කිරීම (throw)

සමහර විට ජාවාස්ක්‍රිප්ට් වලට අනුව වැරද්දක් නොවන නමුත්, අපේ වැඩසටහනේ තර්කනයට අනුව වැරදි අවස්ථා තිබිය හැක. (උදා: වයස ලබා දෙන කොටුවක සෘණ අංකයක් (Minus value) තිබීම). එවැනි විටකදී අපටම දෝෂයක් "විසිකළ" (Throw) හැක.

```
function checkAge(age) {
  if (age < 0) {
    // අප විසින්ම දෝෂයක් නිර්මාණය කරයි
    throw new Error("වයස සෘණ අංකයක් විය නොහැක!");
  }
  return "ඔබේ වයස: " + age;
}

try {
  console.log(checkAge(-5));
} catch (err) {
  console.error("වැරද්ද: " + err.message);
}
```

48.7 ASCII රූපසටහන: try...catch ගලායෑම

```
( ආරම්භය )
|
| [ try { } ] <--- කේතය ක්‍රියාත්මක කරයි
|
| (වැරද්දක් ද?)
| / \
| (නැත) (ඔව්)
| |
| | [ catch(err) { } ] <--- වැරද්ද පාලනය
| | කරයි
| |
| | [ finally { } ] <--- අනිවාර්යයෙන්ම සිදුවේ
| | ( දිගටම... )
```

48.8 වෘත්තීය උපදෙස් (Professional Tips)

- Silent Catch වළකින්න: වැරද්දක් හසුකරගෙන (Catch) කිසිවක් නොකර සිටීම ඉතා

නරක පුරුද්දකි. අවම වශයෙන් එම වැරද්ද කන්සෝලයේ හෝ මුද්‍රණය කරන්න. නැතිනම් පසුව වැරද්ද සෙවීම ඉතා අපහසු වේ.

- Fetch සමඟ භාවිතය: පෙර පරිච්ඡේදයේ අප උගත් fetch() ක්‍රමයේදී ජාල සම්බන්ධතාවය බිඳ වැටුණහොත් එය හසුකර ගැනීමට අනිවාර්යයෙන්ම try...catch භාවිතා කළ යුතුය.

- Specific Errors: හැකි සෑම විටම සාමාන්‍ය පණිවිඩ වෙනුවට පරිශීලකයාට තේරෙන පැහැදිලි පණිවිඩ පෙන්වන්න.

48.9 පොදු වැරදි (Common Mistakes)

1. Syntax Errors හසුකර ගැනීමට උත්සාහ කිරීම: try...catch මගින් හසුකර ගත හැක්කේ ක්‍රියාත්මක වන විට (Runtime) සිදුවන වැරදි පමණි. කේතය ලියන විට සිදුවන Syntax Errors මගින් කේතය ලෝඩ් වීමට වළක්වන බැවින් එය හසුකර ගත නොහැක.
2. අසමකාලීන කේත (Asynchronous code): සාමාන්‍ය setTimeout එකක් ඇතුළත ඇති වැරද්දක් ඊට පිටතින් ඇති try...catch එකකට හසු නොවේ. ඒ සඳහා async/await භාවිතා කළ යුතුය (අපි 46 වන පරිච්ඡේදයේදී මෙය කළෙමු).

48.10 සාරාංශය (Key Points)

- Error Handling මගින් මෘදුකාංගයක් Crash වීම වළක්වයි.
- try තුළ ක්‍රියාත්මක කළ යුතු කේතය ලියයි.
- catch තුළ වැරද්දකදී කළ යුතු දේ ලියයි.
- throw මගින් අපටම දෝෂ නිර්මාණය කළ හැක.
- finally සෑම අවස්ථාවකම ක්‍රියාත්මක වේ.

48.11 අභ්‍යාස (Exercises)

1. අංක දෙකක් බෙදන (Divide) ශ්‍රිතයක් ලියන්න. යම්කු අංකයක් බිංදුවෙන් (0) බෙදීමට උත්සාහ කළහොත් throw භාවිතා කර "බිංදුවෙන් බෙදිය නොහැක" යන දෝෂය පෙන්වන්න.

2. පහත කේතයේ ඇති වැරද්ද try...catch භාවිතා කර පාලනය කර කන්සෝලයේ පෙන්වන්න: let colors = ["red", "blue"]; console.log(colors[5].name);

3. පරිශීලකයෙකුගෙන් මුරපදයක් (Password) ලබා ගන්නා ශ්‍රිතයක් සාදන්න. මුරපදය අතුරු 8කට වඩා අඩු නම් දෝෂයක් විසිකරන්න.

48.12 සම්මුඛ පරීක්ෂණ ප්‍රශ්න (Interview Questions)

- ප්‍රශ්නය: finally කොටස භාවිතා කරන්නේ කුමන අවස්ථාවලදී?
- පිළිතුර: වැඩසටහන සාර්ථක වුවත් නැතත් අවසානයට සිදු විය යුතු පිරිසිදු කිරීමේ කටයුතු (උදා: සර්වර් සම්බන්ධතාවය විසන්ධි කිරීම, Loading spinner එක නතර කිරීම) සඳහා මෙය භාවිතා කරයි.

මිළහ පරිච්ඡේදයෙන් අපි වෙබ් ඔවුසරය තුළ දත්ත ස්ථීරව ගබඩා කර තබන ආකාරය හෙවත් "ලෝකල් ස්ටෝරේජ්" (Local Storage) ගැන ඉතාමත් විස්තරාත්මකව ඉගෙන ගනිමු.

හතළිස් නවවන පරිච්ඡේදය: ලෝකල් ස්ටෝරේජ් - ඔවුසරය තුළ දත්ත ස්ථීරව ගබඩා කිරීම (Local Storage)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

1. වෙබ් ඔවුසරයක දත්ත ගබඩා කිරීමේ අවශ්‍යතාවය තේරුම් ගැනීම.
2. Local Storage (ලෝකල් ස්ටෝරේජ්) සහ Session Storage (සෙෂන් ස්ටෝරේජ්) අතර වෙනස හඳුනා ගැනීම.
3. setItem, getItem, removeItem සහ clear යන ක්‍රමවේද භාවිතා කරන ආකාරය.
4. අරාවන් (Arrays) සහ වස්තූන් (Objects) ලෝකල් ස්ටෝරේජ් තුළ ගබඩා කිරීමට JSON.stringify සහ JSON.parse භාවිතා කරන ආකාරය.
5. වෙබ් පිටුව Refresh කළද දත්ත මැකී නොයන ලෙස තබා ගන්නා ආකාරය.
6. ලෝකල් ස්ටෝරේජ් හි ඇති සීමාවන් සහ ආරක්ෂක ගැටලු අවබෝධ කර ගැනීම.

49.1 හැඳින්වීම: දත්ත ගබඩා කිරීමේ අවශ්‍යතාවය

අපි මෙතෙක් නිර්මාණය කළ සියලුම ව්‍යාපෘතිවල (උදා: To-Do App) තිබූ ප්‍රධාන ගැටලුවක් වූයේ, වෙබ් පිටුව Refresh කළ විට හෝ ඔවුසරය වසා දැමූ විට අප ඇතුළත් කළ දත්ත සියල්ල මැකී යාමයි. මෙයට හේතුව ජාවාස්ක්‍රිප්ට් විචල්‍යයන් (Variables) පරිගණකයේ තාවකාලික මතකයේ (RAM) පමණක් රැඳී තිබීමයි.

සැබෑ ලෝකයේ මෘදුකාංගයකදී පරිශීලකයාගේ දත්ත ස්ථීරව ගබඩා විය යුතුය. ඒ සඳහා අපට සේවාදායකයක් (Database Server) භාවිතා කළ හැක. නමුත් කුඩා දත්ත ප්‍රමාණයන් (උදා: Dark mode සක්‍රියද නැද්ද යන්න, පරිශීලකයාගේ නම) ගබඩා කිරීමට ඔවුසරය තුළම අපට කුඩා ඉඩක් ලබා දී ඇත. මෙය Web Storage API ලෙස හැඳින්වේ.

49.2 ලෝකල් ස්ටෝරේජ් යනු කුමක්ද? (What is Local Storage?)

Local Storage යනු පරිශීලකයාගේ ඔවුසරය තුළ දත්ත "යතුර සහ අගය" (Key-Value pairs) ලෙස ගබඩා කළ හැකි පද්ධතියකි.

විශේෂ ලක්ෂණ:

- ස්ථීර බව: පරිශීලකයා විසින් හිතාමතාම මකා දමන තෙක් දත්ත එහි පවතී. ඔවුසරය වසා දැමුවද දත්ත මැකී නොයයි.
- ධාරිතාව: සාමාන්‍යයෙන් 5MB - 10MB පමණ දත්ත ප්‍රමාණයක් ගබඩා කළ හැක.
- ආරක්ෂාව: දත්ත ගබඩා වන්නේ පරිශීලකයාගේම පරිගණකයේ නිසා වෙනත් වෙබ් අඩවි වලට ඒවා කියවිය නොහැක.

49.3 මූලික ක්‍රමවේද 4 (The 4 Main Methods)

ලෝකල් ස්ටෝරේජ් සමඟ වැඩ කිරීමට අපට පහත සඳහන් ශ්‍රිත (Methods) භාවිතා කළ හැක.

1. localStorage.setItem(key, value) - දත්ත ගබඩා කිරීම

මෙහිදී අපි දත්තයට නමක් (Key) සහ අගයක් (Value) ලබා දිය යුතුය.

```
localStorage.setItem("username", "Kasun");
localStorage.setItem("theme", "dark");
```

2. `localStorage.getItem(key)` - දත්ත ලබා ගැනීම
ගබඩා කළ දත්තයක් නැවත ලබා ගැනීමට එහි
නම (Key) භාවිතා කරයි.

```
let name = localStorage.getItem("username");
console.log(name); // Kasun
```

3. `localStorage.removeItem(key)` - දත්තයක්
මකා දැමීම විශේෂිත එක් දත්තයක් පමණක් ඉවත්
කිරීමට.

```
localStorage.removeItem("theme");
```

4. `localStorage.clear()` - සියල්ල මකා දැමීම
එම වෙබ් අඩවියට අදාළව ලෝකල් ස්ටෝරේජ් හි
ඇති සියලුම දත්ත ඉවත් කිරීමට.

```
localStorage.clear();
```

49.4 වැදගත්ම නීතිය: Strings පමණි!

ලෝකල් ස්ටෝරේජ් තුළ ගබඩා කළ හැක්කේ
අකුරු වැල් (Strings) පමණි. ඔබ අංකයක්,
අරාමයක්(Array) හෝ වස්තුවක් (Object) ගබඩා
කළහොත්, ඔවුන් සරල එය බලහත්කාරයෙන් String
එකක් බවට හරවයි. මෙය ගැටලුවකි.
උදාහරණය:

```
let user = { name: "Nimal", age: 16 };
localStorage.setItem("myUser", user);
```

```
// දැන් ලබා ගනිමු
let result = localStorage.getItem("myUser");
console.log(result); // මෙහිදී "[object Object]"
ලෙස ලැබෙනු ඇත. එය භාවිතා කළ නොහැක.
```

49.5 JSON භාවිතා කර Objects/Arrays ගබඩා කිරීම

ඉහත ගැටලුව විසඳීමට අප පරිමාව 4, 47 වන
පරිච්ඡේදයේ උගත් JSON භාවිතා කරමු.
1. ගබඩා කිරීමට පෙර: `JSON.stringify()` භාවිතා
කර වස්තුව String එකක් බවට හරවන්න.
2. ලබා ගත් පසු: `JSON.parse()` භාවිතා කර එම
String එක නැවත ජාවාස්ක්‍රිප්ට් වස්තුවක් බවට
හරවන්න.

නිවැරදි ක්‍රමය:

```
const student = { name: "Amara", grade: 11 };
```

```
// 1. String එකක් බවට හරවා ගබඩා කිරීම
localStorage.setItem("studentData",
JSON.stringify(student));
```

```
// 2. ලබාගෙන නැවත Object එකක් බවට හැරවීම
const storedData =
localStorage.getItem("studentData");
const finalData = JSON.parse(storedData);
```

```
console.log(finalData.name); // Amara
```

49.6 ASCII රූපසටහන: දත්ත ගලායෑම

```
[ JS Object ] ---- JSON.stringify() ----> [
"String Data" ]
```

```
localStorage.setItem()
|
[ Browser Storage
]
```

```
localStorage.getItem()
|
[ JS Object ] <---- JSON.parse() ----- [
"String Data" ]
```

49.7 ප්‍රායෝගික භාවිතය: පෝට්ෆෝලියෝ තේමාව මතක තබා ගැනීම

බොත්තමක් ක්ලික් කර Dark Mode මාරු කරන
විට, පරිශීලකයා ඊළඟ වතාවේ එන විටත් එම
තේමාවම තිබීමට අපට ලෝකල් ස්ටෝරේජ්
භාවිතා කළ හැක.

```
// තේමාව වෙනස් කරන ශ්‍රිතය
function toggleTheme(selectedTheme) {
    document.body.className =
selectedTheme;
    // තේමාව ලෝකල් ස්ටෝරේජ් එකේ සේවා
කරන්න
    localStorage.setItem("userTheme",
selectedTheme);
}
```

```
// පිටුව මුලින්ම Load වන විට පරීක්ෂා කිරීම
window.onload = () => {
  const savedTheme =
    localStorage.getItem("userTheme");
  if (savedTheme) {
    document.body.className =
      savedTheme;
  }
};
```

49.8 ලෝකල් ස්ටෝරේජ් පරීක්ෂා කරන්නේ කොහේද? (Developer Tools)

ඔබට ඔබගේ වෙබ් අඩවියේ ගබඩා වී ඇති දත්ත බැලීමට:

1. බ්‍රවුසරයේ Right Click -> Inspect කරන්න.
2. ඉහළ ඇති ටැබ් අතරින් Application යන්න තෝරන්න.
3. වම් පස ඇති Storage යටතේ Local Storage ක්ලික් කර ඔබේ වෙබ් අඩවියේ නම තෝරන්න.

49.9 වෘත්තීය උපදෙස් (Professional Tips)

- මුරපද ගබඩා නොකරන්න: කිසිවිටකත් මුරපද (Passwords), ක්‍රෙඩිට් කාඩ් විස්තර වැනි රහස්‍ය තොරතුරු ලෝකල් ස්ටෝරේජ් හි ගබඩා නොකරන්න. මන්දයත් ඕනෑම කෙනෙකුට Inspect Element හරහා ඒවා බැලිය හැකි බැවිනි.

- Error Handling: ලෝකල් ස්ටෝරේජ් පිරී ගියහොත් (Quota Exceeded) කේතය බිඳ වැටිය හැක. එබැවින් try...catch භාවිතා කිරීම සුදුසුය.

- Session Storage: ඔබට දත්ත අවශ්‍ය වන්නේ එම ටැබ් එක වසා දමන තෙක් පමණක් නම් sessionStorage භාවිතා කරන්න. එහි ක්‍රමවේද සියල්ල ලෝකල් ස්ටෝරේජ් වලට සමානය.

49.10 පොදු වැරදි (Common Mistakes)

1. JSON.parse(null): ලෝකල් ස්ටෝරේජ් හි දත්තය නැති විට (null ලැබෙන විට) එය Parse කිරීමට උත්සාහ කළහොත් වැරද්දක් සිදුවේ. එබැවින් අගයක් තිබේදැයි සැමවිටම පරීක්ෂා කරන්න.

2. සීමාව ඉක්මවා යාම: රූප (Images) හෝ ඉතා විශාල දත්ත ලෝකල් ස්ටෝරේජ් තුළ ගබඩා කිරීමට උත්සාහ කිරීම.

49.11 සාරාංශය (Key Points)

- Local Storage මගින් බ්‍රවුසරය තුළ දත්ත ස්ථිරව තබා ගනී.
- එය Key-Value පද්ධතියකි.
- ගබඩා කළ හැක්කේ Strings පමණි.
- Objects/Arrays සඳහා JSON භාවිතා කළ යුතුය.
- පිටුව Refresh කළද දත්ත මැකී නොයයි.

49.12 අභ්‍යාස (Exercises)

1. ලෝකල් ස්ටෝරේජ් සහ සෙෂන් ස්ටෝරේජ් අතර ඇති ප්‍රධාන වෙනස කුමක්ද?
2. පහත අරා වල ලෝකල් ස්ටෝරේජ් තුළ ගබඩා කිරීමට අවශ්‍ය කේතය ලියන්න: let fruits = ["Apple", "Banana", "Orange"];
3. පරිශීලකයා වෙබ් අඩවියට පිවිසෙන සෑම වතාවකම ඔහු කලින් පිවිසුණු වාර ගණන (Visit Count) පෙන්වන සරල කේතයක් ලියන්න.
4. ලෝකල් ස්ටෝරේජ් එකේ "my_data" නමින් ඇති දත්තය මකා දැමීමට භාවිතා කරන විධානය කුමක්ද?

49.13 සම්මුඛ පරීක්ෂණ ප්‍රශ්න (Interview Questions)

- ප්‍රශ්නය: JSON.stringify() භාවිතා කරන්නේ ඇයි?
- පිළිතුර: ලෝකල් ස්ටෝරේජ් තුළ ගබඩා කළ හැක්කේ Strings පමණි. එබැවින් ජාවාස්ක්‍රිප්ට් වස්තූන් (Objects) හෝ අරාවන් (Arrays) ගබඩා කිරීමට පෙර ඒවා String එකක් බවට හැරවීමට මෙම ශ්‍රිතය භාවිතා කරයි.

49.14 පරිමාව 4 අවසන් ව්‍යාපෘතිය (Final Project Volume 4)

ව්‍යාපෘතිය: ස්ථිර දත්ත සහිත වැඩ ලැයිස්තුව (To-Do App with Persistence)

පරිමාව 3 හි ඔබ නිර්මාණය කළ To-Do App එක ගෙන එයට ලෝකල් ස්ටෝරේජ් සම්බන්ධ කරන්න.

- අලුත් වැඩක් එකතු කරන විට එය අරාවක (Array) ගබඩා කර ලෝකල් ස්ටෝරේජ් එකට setItem කරන්න.
- පිටුව Refresh කරන සෑම අවස්ථාවකම ලෝකල් ස්ටෝරේජ් එකෙන් දත්ත getItem කර තීරය මත පෙන්වන්න.
- වැඩක් මකා දැමූ විට ලෝකල් ස්ටෝරේජ් එකෙන් ද එය ඉවත් කිරීමට වගබලා ගන්න.

පනස්වන පරිච්ඡේදය: පන්ති සහ මතක කළමනාකරණය - වෘත්තීය මෘදුකාංග ව්‍යුහය (Classes, OOP, and Memory Management)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව පූර්ණ ප්‍රවීණත්වයක් ලැබෙනු ඇත:

- Classes (පන්ති): ජාවාස්ක්‍රිප්ට් වල "සැලසුම් නිර්මාණය" (Blueprints) කරන ආකාරය තේරුම් ගැනීම.
- Constructor (නිර්මාපකය): වස්තුවක් (Object) සෑදෙන මොහොතේම ක්‍රියාත්මක වන රහස්‍ය ශ්‍රිතය වටහා ගැනීම.
- Inheritance (උරුමය): පවතින පන්තියක ලක්ෂණ තවත් පන්තියකට ලබා දෙන (extends & super) ආකාරය.
- Static Methods: වස්තුවක් නොසාදා පන්තිය හරහා සෘජුවම වැඩ කරන මෙවලම් හඳුනා ගැනීම.
- Getters & Setters: දත්ත ආරක්ෂිතව ලබා ගන්නා සහ වෙනස් කරන (Encapsulation) ආකාරය.
- Memory Management: ඉවුසරය අනවශ්‍ය දත්ත මතකයෙන් ඉවත් කරන ආකාරය (Garbage Collection).

50.1 පන්ති යනු මොනවාද? (What are Classes?)

අපි කලින් වස්තූන් (Objects) ගැන ඉගෙන ගත්තෙමු. නමුත් ඔබට එක හා සමාන වස්තූන් 100ක් (උදා: පරිශීලකයන් 100ක්) සෑදීමට අවශ්‍ය නම්, එකින් එක Object ලිවීම අපහසුය. Class (පන්තිය) යනු වස්තූන් නිර්මාණය කිරීමට භාවිතා කරන "සැලසුම් සටහනක්" (Blueprint) වැනිය.

සැබෑ ලෝකයේ උපමාව (නිවාස සැලසුම): නිවසක් තැනීමට පෙර ඉංජිනේරුවෙකු විසින් නිවසේ සැලසුමක් (Plan) අඳිනු ලබයි.

- Class: යනු එම නිවාස සැලසුමයි. (එය තවම සැබෑ නිවසක් නොවේ).
- Object (Instance): යනු එම සැලසුම බලා සාදන ලද සැබෑ නිවසයි. එක් සැලසුමකින් නිවාස දහස් ගණනක් වුවද සෑදිය හැක.

50.2 පන්තියක් ලියන ආකාරය (Class Syntax)

```
class User {
  // 1. වස්තුව සෑදෙන විටම ක්‍රියාත්මක වන කොටස
  constructor(name, email) {
    this.name = name;
    this.email = email;
  }

  // 2. මෙයට අයිති ක්‍රියාකාරකම් (Methods)
  sayHello() {
    console.log(`ආයුබෝවන්, මම ${this.name}!`);
  }
}

// වස්තුවක් නිර්මාණය කිරීම (Creating an Instance)
const user1 = new User("කසුන්", "kasun@mail.com");
const user2 = new User("නිමල්", "nimal@mail.com");

user1.sayHello(); // "ආයුබෝවන්, මම කසුන්!"
```

50.3 උරුමය: කේතය නැවත භාවිතා කිරීම (Inheritance)

සමහර විට අපට එක් පන්තියක ඇති දේවල් තවත් පන්තියකට "උරුම කර දීමට" අවශ්‍ය වේ. ඒ සඳහා extends භාවිතා කරයි.

උදාහරණය (පරිශීලකයා සහ පරිපාලකයා - User and Admin):

```
class Admin extends User {
  constructor(name, email, role) {
    // super() මගින් පියාගේ (User) constructor එක කැඳවයි
  }
}
```



```

    super(name, email);
    this.role = role;
}

deleteUser() {
    console.log("පරිශීලකයා මකා දමන ලදී.");
}
}

const myAdmin = new Admin("අමරා",
"amara@mail.com", "SuperAdmin");
myAdmin.sayHello(); // පියාගෙන් ලැබුණු ශ්‍රිතය
(Inherited)
myAdmin.deleteUser(); // තමන්ගේම විශේෂ ශ්‍රිතය

```

50.4 දත්ත ආරක්ෂා කිරීම: Getters සහ Setters

වෘත්තීය මට්ටමේදී අපි වස්තුවක ඇති දත්ත කෙලින්ම වෙනස් කිරීමට ඉඩ දෙන්නේ නැත. අපි ඒ සඳහා විශේෂ "ගේට්ටු" සාදන්නෙමු.

- get: දත්තයක් ලබා ගැනීමට.
- set: දත්තයක් ඇතුළත් කිරීමට පෙර එය නිවැරදිදැයි පරීක්ෂා කිරීමට.

```
class Product {
  constructor(price) {
    this._price = price; // _ලකුණින් පවසන්නේ
    මෙය රහස්‍ය (private) බවයි
  }

  get price() {
    return `රුපියල් ${this._price}`;
  }

  set price(value) {
    if (value < 0) {
      console.log("මිල සෘණ අංකයක් විය
      නොහැක!");
    } else {
      this._price = value;
    }
  }
}
```

```
const item = new Product(100);
item.price = -50; // "මිල සෘණ අංකයක් විය නොහැක!"
console.log(item.price); // "රුපියල් 100"
```

50.5 මතක කළමනාකරණය (Memory Management & Garbage Collection)

වෘත්තීය සංවර්ධනයෙකු ලෙස ඔබ දැනගත යුතු රහස්‍යක් මෙන්න. ජාවාස්ක්‍රිප්ට් වල මතකය (Memory) කළමනාකරණය කරන්නේ කෙසේද?

1. Memory Heap: සියලුම වස්තුන් (Objects)
ගබඩා වන විශාල ඉඩ ප්‍රමාණය.

2. Garbage Collector (කුණු එකතු කරන්නා):
 ඔවුසරය තුළ රහසිගතව ක්‍රියාත්මක වන
 වැඩසටහනකි. යම් වස්තුවක් හෝ විචල්‍යයක්
 තවදුරටත් කේතයේ කොතැනකවත් සම්බන්ධ
 නැතිනම් (No references), මෙම වැඩසටහන
 විසින් එය ස්වයංක්‍රීයව මතකයෙන් ඉවත් කර ඉඩ
 නිදහස් කරයි.

මෙක කාන්දුවක් (Memory Leak) යනු කුමක්ද? ඔබ අනවශ්‍ය දත්ත විචල්‍යයකට අමුණා දිගටම තබා ගන්නේ නම්, Garbage Collector හට එය ඉවත් කළ නොහැක. එවිට වෙබ් අඩවිය මන්දගාමී වේ. මෙය වැළැක්වීමට වැඩ අවසන් වූ පසු විශාල විචල්‍යයන් null කිරීම සුදුසුය.

50.6 ASCII රූපසටහන: Class සහ Instance අතර සම්බන්ධය

```
[ Class Blueprint: User ]
| - property: name      |
| - method: sayHello()  |
```

```
(new User("Kasun"))
```

v	
Instance Object: user1	
- name: "Kasun"	
- [Link to sayHello()]	

50.7 වෘත්තීය රහස්: ඉමියුටබිලිටි (Immutability)

React වැනි තාක්ෂණයන් ඉගෙන ගැනීමට පෙර ඔබ මෙය දැනගත යුතුය. ප්‍රෝ-සංවර්ධකයන් පවතින දත්ත වෙනස් (Mutate) කිරීමට අකමැතිය.

- Mutation (ඡරකයි): user.name = "New Name";

- Immutability (භෞද්ධ): `const newUser = { ...user, name: "New Name" }; (පරණ එක එලෙසම තබා අලුත් පිටපතක් සාදීම).`

මෙය මතකයේ ස්ථායීතාවය (State Stability) සහ දෝෂ සෙවීම පහසු කරයි.

50.8 සාරාංශය (Key Points)

- Classes යනු වස්තූන් සෑදීමට භාවිතා කරන සැලසුම් (Blueprints) වේ.
- constructor මගින් මුලින්ම දත්ත ඇතුළත් කරයි.
- extends මගින් පියෙකුගේ ගුණාංග දරුවෙකුට ලබා දෙයි.
- Garbage Collection මගින් භාවිත නොවන දත්ත ස්වයංක්‍රීයව මකකයෙන් ඉවත් කරයි.
- වෘත්තීය මට්ටමේදී දත්ත සෘජුව වෙනස් නොකර පිටපත් (Clones) සෑදීම සිදු කරයි.

50.9 අභ්‍යාස (Exercises)

1. Shape නමින් පන්තියක් සාදා එයට වර්ණයක් (color) ලබා දෙන්න. ඉන්පසු Circle නමින් දරු පන්තියක් සාදා එයට අරය (radius) සහ වර්ගඵලය සෙවීමේ ශ්‍රිතයක් එක් කරන්න.
2. Bank නමින් පන්තියක් සාදා එහි මුදල් (balance) ප්‍රමාණය බාහිරින් වෙනස් කළ නොහැකි ලෙස set එකක් හරහා පාලනය කරන්න. (උදා: සෘණ අංක ඇතුළත් කළ නොහැකි ලෙස).
3. පන්තියක් (Class) සහ වස්තුවක් (Object) අතර ඇති ප්‍රධාන වෙනස ඔබේ වචනයෙන් පැහැදිලි කරන්න.

පරිමාව 4 - අවසාන පිරික්සුම් ලැයිස්තුව (Volume 4 Complete A-Z Checklist)

දැන් ඔබ පරිමාව 4 හි සියලුම පාඨම් අවසන් කර ඇති බැවින්, පහත දෑ ඔබට කළ හැකි දැයි බලන්න:

- [] මම සියලුම ES6+ විශේෂාංග (Arrow functions, Destructuring, Spread) දනිමි.
- [] මට Async/Await සහ Fetch API භාවිතා කර අන්තර්ජාලයෙන් ඕනෑම දත්තයක් ලබාගත හැක.
- [] මම JavaScript එන්ජිම ඇතුළත ක්‍රියා කරන ආකාරය (Prototypes, Event Loop) දනිමි.
- [] මට දත්ත LocalStorage තුළ ස්ථිරව ගබඩා කළ හැක.
- [] මම වෘත්තීය මට්ටමේ Classes සහ OOP සංකල්ප දනිමි.

පනස් එක්වන පරිච්ඡේදය: මහා ව්‍යාපෘතිය - "SkyCast Pro" කාලගුණ විශ්ලේෂණ පද්ධතිය (Capstone Project: SkyCast Pro Weather Dashboard)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම මහා ව්‍යාපෘතිය සාර්ථකව අවසන් කිරීමෙන් පසු ඔබට පහත සඳහන් සියලුම දක්ෂතාවන් එකට එකතු කර මෘදුකාංගයක් නිර්මාණය කිරීමේ හැකියාව ලැබෙනු ඇත:

1. සැබෑ ලෝකයේ API දත්ත හැසිරවීම: OpenWeatherMap වැනි බාහිර සේවාදායකයකට සම්බන්ධ වී දත්ත ලබා ගැනීම.
2. අසමකාලීන ක්‍රමලේඛනය (Async/Await): දත්ත ලැබෙන තෙක් මෘදුකාංගය හිර නොවී (Non-blocking) පවත්වා ගැනීම.
3. පන්ති මූලික සැලසුම්කරණය (Class-based Architecture): දත්ත ලබා ගැනීම සහ තීරය මත පෙන්වීම සඳහා වෘත්තීය මට්ටමේ Classes නිර්මාණය කිරීම.
4. LocalStorage කළමනාකරණය: පරිශීලකයාගේ සෙවුම් ඉතිහාසය ස්ථිරව ගබඩා කිරීම.
5. උසස් දෝෂ හැසිරවීම (Advanced Error Handling): අන්තර්ජාලය විසන්ධි වීම හෝ වැරදි දත්ත ඇතුළත් කිරීම් වෘත්තීය මට්ටමින් පාලනය කිරීම.
6. නූතන CSS සහ UI/UX: Glassmorphism සහ Responsive පිරිසැලසුම් හරහා උසස් නිමාවක් ලබා දීම.

51.1 ව්‍යාපෘති විස්තරය (Project Overview)

අපි අද නිර්මාණය කරන්නේ සාමාන්‍ය කාලගුණ ඇප් එකක් නොවේ. මෙය වෘත්තීය මට්ටමේ Dashboard එකකි. මෙහිදී අප පරිමාව 4 තුළ ඉගෙන ගත් සංකීර්ණ ජාවාස්ක්‍රිප්ට් සංකල්ප සියල්ල ප්‍රායෝගිකව භාවිතා කරනු ඇත.

මෘදුකාංගයේ ප්‍රධාන අංග:

- Weather Service Class: දත්ත ලබා ගැනීමට වෙන්වූ පන්තියක්.
- UI Engine Class: තීරය මත දත්ත පෙන්වීමට වෙන්වූ පන්තියක්.

- Real-time Search: ඕනෑම නගරයක දත්ත සෙවීම.
- Dynamic Theme: කාලගුණ තත්ත්වය අනුව පසුබිම් වර්ණ වෙනස් වීම.
- Search History: මීට පෙර සෙවූ නගර ලැයිස්තුව මතක තබා ගැනීම.

51.2 පියවර 1: වෘත්තීය HTML සැකිල්ල (Semantic Structure)

අපි මූලිකම පිරිසිදු HTML සැකිල්ලක් සාදා ගනිමු.

index.html

```
<!DOCTYPE html>
<html lang="si">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
content="width=device-width, initial-scale=1.0">
  <title>SkyCast Pro | කාලගුණ විශ්ලේෂණ
පද්ධතිය</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <div class="dashboard-container">
    <!-- 1. සෙවුම් පුවරුව -->
    <header class="search-header">
      <div class="logo">SkyCast
<span>Pro</span></div>
      <div class="search-bar">
        <input type="text" id="cityInput"
placeholder="නගරයේ නම ටයිප් කරන්න...">
        <button id="searchBtn">දත්ත ලබා
ගන්න</button>
      </div>
    </header>

    <!-- 2. ප්‍රධාන කාලගුණ පුවරුව -->
    <main class="main-stats">
      <section id="weatherCard" class="glass-
card hidden">
        <div class="city-info">
          <h2 id="cityName">නගරය</h2>
          <p id="dateTime">දිනය සහ
වේලාව</p>
        </div>
        <div class="temp-section">
          <img id="weatherIcon" src=""
alt="Icon">
          <h1 id="temperature">0°C</h1>
          <h3 id="description">තත්ත්වය</h3>
        </div>
      </section>
    </main>
  </div>
</body>
</html>
```

```
<!-- 3. අමතර විස්තර (Analytics) -->
<section id="detailsCard" class="details-
grid hidden">
  <div class="detail-box">
    <p>ආර්ද්‍රතාවය</p>
    <h4 id="humidity">0%</h4>
  </div>
  <div class="detail-box">
    <p>සුළඟේ වේගය</p>
    <h4 id="windSpeed">0 km/h</h4>
  </div>
  <div class="detail-box">
    <p>පීඩනය</p>
    <h4 id="pressure">0 hPa</h4>
  </div>
  <div class="detail-box">
    <p>දෘශ්‍යතාවය</p>
    <h4 id="visibility">0 km</h4>
  </div>
</section>
</main>

<!-- 4. සෙවුම් ඉතිහාසය -->
<footer class="history-footer">
  <h3>මෑතකදී සෙවූ නගර</h3>
  <div id="historyContainer" class="history-
chips">
    <!-- දත්ත මෙතැනට පැමිණේ -->
  </div>
</footer>

<!-- Error Popup -->
<div id="errorPopup" class="error-msg
hidden"></div>

<script src="script.js"></script>
</body>
</html>
```

51.3 පියවර 2: ප්‍රේෂ්-මට්ටමේ CSS හැඩතල (Advanced CSS)

අපි 68 වන පරිච්ඡේදයේ ඉගෙන ගත් Glassmorphism සහ CSS Variables භාවිතා කරමු.

style.css

```
:root {
  --primary-bg: #0f172a;
  --glass-bg: rgba(255, 255, 255, 0.1);
  --glass-border: rgba(255, 255, 255, 0.2);
  --accent-color: #38bdf8;
  --text-white: #f8fafc;
}
```

```

* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  background-color: var(--primary-bg);
  color: var(--text-white);
  font-family: 'Segoe UI', system-ui, sans-serif;
  min-height: 100vh;
  display: flex;
  justify-content: center;
  padding: 40px 20px;
  background-image: radial-gradient(circle at top right, #1e293b, #0f172a);
}

.dashboard-container {
  width: 100%;
  max-width: 900px;
}

/* Glass Effect */
.glass-card {
  background: var(--glass-bg);
  backdrop-filter: blur(12px);
  border: 1px solid var(--glass-border);
  border-radius: 24px;
  padding: 30px;
  margin-bottom: 20px;
  text-align: center;
}

.search-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 40px;
  flex-wrap: wrap;
  gap: 20px;
}

.logo { font-size: 24px; font-weight: 800; }
.logo span { color: var(--accent-color); }

.search-bar { display: flex; gap: 10px; flex: 1; min-width: 300px; }

input {
  flex: 1;
  background: var(--glass-bg);
  border: 1px solid var(--glass-border);
  padding: 12px 20px;
  border-radius: 12px;
  color: white;
  outline: none;
}

button {
  background: var(--accent-color);
  color: var(--primary-bg);
  border: none;
  padding: 12px 24px;
  border-radius: 12px;
  font-weight: 700;
  cursor: pointer;
  transition: 0.3s;
}

button:hover { opacity: 0.9; transform: translateY(-2px); }

.details-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(150px, 1fr));
  gap: 15px;
}

.detail-box {
  background: var(--glass-bg);
  padding: 20px;
  border-radius: 18px;
  text-align: center;
  border: 1px solid var(--glass-border);
}

.history-chips {
  display: flex;
  gap: 10px;
  margin-top: 15px;
  flex-wrap: wrap;
}

.chip {
  background: var(--glass-border);
  padding: 8px 16px;
  border-radius: 20px;
  font-size: 14px;
  cursor: pointer;
}

.hidden { display: none; }

```

```
.error-msg {
  position: fixed;
  bottom: 20px;
  background: #ef4444;
  padding: 15px 30px;
  border-radius: 10px;
  box-shadow: 0 10px 20px rgba(0,0,0,0.3);
}
```

51.4 පියවර 3: උසස් ජාවාස්ක්‍රිප්ට් තර්කනය (Class-based JS)

අපි දැන් 50 වන පරිච්ඡේදයේ ඉගෙන ගත් Classes සහ 45 වන පරිච්ඡේදයේ ඉගෙන ගත් Async/Await භාවිතා කරමු.

script.js

```
"use strict";

/**
 * 1. දත්ත ලබා ගැනීමේ පන්තිය (Weather Service)
 * වගකීම: API එක සමඟ ගනුදෙනු කිරීම
 */
class WeatherService {
  constructor(apiKey) {
    this.apiKey = apiKey;
    this.baseUrl =
      "https://api.openweathermap.org/data/2.5/weather";
  }

  async fetchWeather(city) {
    try {
      const response = await
        fetch(`${this.baseUrl}?q=${city}&units=metric&appid=${this.apiKey}`);
      if (!response.ok) {
        if (response.status === 404) throw new
          Error("නගරය සොයාගත නොහැකි විය!");
        throw new Error("සර්වර් එක සමඟ
          සම්බන්ධ විය නොහැක!");
      }

      const data = await response.json();
      return this.formatData(data);
    } catch (error) {
      throw error; // වැරද්ද පිටතට යවමු
    }
  }

  formatData(data) {
    // Destructuring භාවිතා කර දත්ත වෙන් කර
    ගැනීම (Chapter 34)
    const { name, main, weather, wind, visibility,
      dt } = data;
```

```
    return {
      city: name,
      temp: Math.round(main.temp),
      humidity: main.humidity,
      pressure: main.pressure,
      windSpeed: wind.speed,
      condition: weather[0].description,
      icon: weather[0].icon,
      visible: (visibility / 1000).toFixed(1),
      time: new Date(dt * 1000).toLocaleString()
    };
  }
}
```

```
/**
 * 2. ඉතිහාසය පාලනය කරන පන්තිය (History
  Manager)
 * වගකීම: LocalStorage කළමනාකරණය (Chapter
  45)
 */
```

```
class HistoryManager {
  constructor() {
    this.key = "skycast_history";
  }

  save(city) {
    let list = this.getAll();
    if (!list.includes(city)) {
      list.push(city);
      if (list.length > 5) list.shift(); // උපරිම 5ක්
      පමණක් තබා ගනිමු
      localStorage.setItem(this.key,
        JSON.stringify(list));
    }
  }
```

```
  getAll() {
    return
      JSON.parse(localStorage.getItem(this.key)) || [];
  }
}
```

```
/**
 * 3. ප්‍රධාන පාලක පන්තිය (App Controller)
 * වගකීම: UI සහ දත්ත එකිනෙක සම්බන්ධ කිරීම
 */
```

```
class SkyCastApp {
  constructor() {
    this.weatherService = new
      WeatherService("ඔබේ_API_KEY_එක_මෙතැනට"
    );
    this.historyManager = new HistoryManager();
    this.init();
  }

  init() {
    // DOM Elements තෝරා ගැනීම
```



```

    this.btn =
document.querySelector("#searchBtn");
    this.input =
document.querySelector("#cityInput");
    this.errorBox =
document.querySelector("#errorPopup");

    // බොත්තම එබූ විට සිදුවන දේ
    this.btn.addEventListener("click", () =>
this.searchCity());

    // Enter යතුර එබූ විට සිදුවන දේ
    this.input.addEventListener("keypress", (e) =>
{
    if (e.key === "Enter") this.searchCity();
    });

    this.renderHistory();
}

async searchCity(cityName = null) {
    const city = cityName || this.input.value.trim();
    if (!city) return;

    this.showLoading(true);

    try {
        const data = await
this.weatherService.fetchWeather(city);
        this.updateUI(data);
        this.historyManager.save(data.city);
        this.renderHistory();
        this.hideError();
    } catch (error) {
        this.showError(error.message);
    } finally {
        this.showLoading(false);
    }
}

updateUI(data) {

document.querySelector("#weatherCard").classList
.remove("hidden");

document.querySelector("#detailsCard").classList.r
emove("hidden");

document.querySelector("#cityName").innerText =
data.city;

document.querySelector("#temperature").innerText
= `${data.temp} °C`;

document.querySelector("#description").innerText
= data.condition.toUpperCase();

```

```

document.querySelector("#dateTime").innerText =
data.time;

document.querySelector("#humidity").innerText =
`${data.humidity}%`;

document.querySelector("#windSpeed").innerText
= `${data.windSpeed} m/s`;

document.querySelector("#pressure").innerText =
`${data.pressure} hPa`;

document.querySelector("#visibility").innerText =
`${data.visible} km`;
    document.querySelector("#weatherIcon").src
=
`https://openweathermap.org/img/wn/${data.icon}
@2x.png`;
}

renderHistory() {
    const container =
document.querySelector("#historyContainer");
    container.innerHTML = "";
    const list = this.historyManager.getAll();

    list.forEach(city => {
        const chip =
document.createElement("span");
        chip.className = "chip";
        chip.innerText = city;
        chip.onclick = () => this.searchCity(city);
        container.appendChild(chip);
    });

    showError(msg) {
        this.errorBox.innerText = msg;
        this.errorBox.classList.remove("hidden");
        setTimeout(() => this.hideError(), 3000);
    }

    hideError() {
        this.errorBox.classList.add("hidden");
    }

    showLoading(state) {
        this.btn.innerText = state ? "සොයමින්..." :
"දත්ත ලබා ගන්න";
        this.btn.disabled = state;
    }
}

// මෘදුකාංගය පණ ගන්වමු
new SkyCastApp();

```

51.5 කේතය පිළිබඳ ගැඹුරු තාක්ෂණික විග්‍රහය (Deep Analysis)

1. Classes භාවිතය (Architecture): අපි සියලුම කේතයන් එකම තැන ගොඩ ගැසුවේ නැත. දත්ත ලබා ගැනීමට එක පන්තියක් ද, ඉතිහාසය පාලනයට තව එකක් ද භාවිතා කළෙමු. මෙය වෘත්තීය මට්ටමේ Separation of Concerns සංකල්පයයි.
2. Try/Catch/Finally (Chapter 44): දත්ත ලබා ගන්නා විට වැරද්දක් වුවහොත් එය catch හරහා හසුකර ගත් අතර, දත්ත ලැබුණත් නැතත් finally හරහා Loading බොත්තම යථා තත්ත්වයට පත් කළෙමු.
3. Destructuring & Template Literals (Chapter 39): API ප්‍රතිචාරයෙන් (Response) දත්ත වෙන් කර ගැනීමට සහ URL එක සැකසීමට මෙම නවීන ක්‍රම භාවිතා කළෙමු.
4. LocalStorage (Chapter 45): JSON.stringify සහ JSON.parse භාවිතා කර සෙවුම් ඉතිහාසය ස්ථිරවම ඔවුසරයේ තැන්පත් කළෙමු.

51.6 වෘත්තීය උපදෙස් (Professional Engineering Tips)

- API Key ආරක්ෂාව: සැබෑ මෘදුකාංගයකදී API Key එක කේතය ඇතුළත ලියන්නේ නැත. ඒ වෙනුවට Environment Variables (.env) භාවිතා කරයි. (Vite වැනි මෙවලම් භාවිතා කරන විට මෙය පහසුය).
- Performance: එකම නගරය නැවත නැවත සොයන විට, සර්වර් එකට පණිවිඩ නොයවා Caching ක්‍රමවේදයක් (මතකයේ තබා ගැනීම) භාවිතා කළ හැක.
- Accessibility: පින්තූර සඳහා alt tags සහ මවුසය භාවිතා කළ නොහැකි අය සඳහා tab භාවිතය නිවැරදිව සකසා ඇත්දැයි Lighthouse හරහා පරීක්ෂා කරන්න.

51.7 සාරාංශය (Key Points)

- වෘත්තීය මෘදුකාංගයක් සැමවිටම කොටස් වලට බෙදා (Modular) නිර්මාණය කළ යුතුය.
- Async/Await මගින් අසමකාලීන දත්ත ලබා ගැනීම පාලනය කරයි.
- පන්ති (Classes) මගින් කේතය සංවිධානය කිරීම පහසු කරයි.
- පරිශීලක අත්දැකීම (UX) සඳහා දෝෂ පණිවිඩ සහ Loading තත්ත්වයන් ඉතා වැදගත් වේ.

51.8 පරිමාව 4 අවසන් අභ්‍යාසය (The Final Challenge)

මෙම ව්‍යාපෘතිය තවත් ඉහළ මට්ටමකට ගෙන ඒමට උත්සාහ කරන්න:

1. Metric/Imperial Toggle: සෙල්සියස් සහ ෆැරන්හයිට් අතර උෂ්ණත්වය මාරු කිරීමට බොත්තමක් එක් කරන්න.
2. Geolocation API: පරිශීලකයාගේ අවසරය ඇතිව ඔහු සිටින ස්ථානයේ කාලගුණය පිටුවට ආසැණින් පෙන්වන්න.
3. Dynamic Backgrounds: කාලගුණය "Rainy" නම් පසුබිමට වැස්ස පෙන්වන පින්තූරයක් ද, "Clear" නම් ඉර පායා ඇති පින්තූරයක් ද පෙන්වීමට උත්සාහ කරන්න.

සුභ පැතුම්! ඔබ දැන් "පරිමාව 4: උසස් ජාවාස්ක්‍රිප්ට්" සාර්ථකව අවසන් කර ඇත.

වෘත්තීය මෙවලම් සහ වෙබ් අඩවි ප්‍රසිද්ධ කිරීම

පහස් දෙවන පරිච්ඡේදය:

ටර්මිනල් - විධාන මගින්

පරිගණකය පාලනය කිරීම (The Command Line Interface / Terminal)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

1. ටර්මිනල් (Terminal) හෙවත් කමාන්ඩ් ලයින් (Command Line) යනු කුමක්දැයි වෘත්තීය මට්ටමින් තේරුම් ගැනීම.
2. GUI (Graphical User Interface) සහ CLI (Command Line Interface) අතර වෙනස හඳුනා ගැනීම.
3. වෘත්තීය මෘදුකාංග ඉංජිනේරුවෙකුට ටර්මිනල් දැනුම අත්‍යවශ්‍ය වන්නේ ඇයි දැයි වටහා ගැනීම.
4. මූලික ටර්මිනල් විධාන (Commands) වන ls, cd, mkdir, clear වැනි දෑ භාවිතා කරන ආකාරය.
5. ගොනු පද්ධතිය (File System) හරහා විධාන මගින් ගමන් කරන ආකාරය.
6. VS Code තුළ ඇති අභ්‍යන්තර ටර්මිනලය (Integrated Terminal) භාවිතා කරන ආකාරය.

52.1 හැඳින්වීම: GUI සහ CLI අතර වෙනස

පරිගණකයක් සමඟ ගනුදෙනු කිරීමට ප්‍රධාන ක්‍රම දෙකක් ඇත.

1. GUI (Graphical User Interface - ප්‍රස්ථාරික පරිශීලක අතුරුමුහුණත): මෙය ඔබ දැනටමත් භාවිතා කරන සාමාන්‍ය ක්‍රමයයි. මෙහිදී ඔබ අයිකන (Icons) දකිනවා, මවුසය (Mouse) භාවිතා කර ඒවා මත ක්ලික් කරනවා, Folder එකක් විවෘත කිරීමට Double Click කරනවා. මෙය ඉතා පහසු ක්‍රමයකි.
2. CLI (Command Line Interface - විධාන පේළි අතුරුමුහුණත): මෙහිදී කිසිදු අයිකනයක් හෝ රූපයක් නැත. ඇත්තේ කළු පැහැති තිරයක් පමණි. ඔබ පරිගණකයට සියලුම උපදෙස් ලබා දෙන්නේ අකුරු (Commands) ටයිප් කිරීම මගිනි. උදාහරණයක් ලෙස, Folder එකක් විවෘත

කිරීමට නම් ඔබ cd folder_name ලෙස ටයිප් කළ යුතුය.

සැබෑ ලෝකයේ උපමාව (රොබෝවරයෙක් සහ දුරස්ථ පාලකය): සිතන්න ඔබේ නිවසේ වැඩ කරන රොබෝවරයෙකු සිටිනවා කියා.

- ඔබ රොබෝවරයාගේ පිටේ ඇති "ඛන් උයන්න" නමැති බොත්තම එබුවහොත් එය GUI ක්‍රමයයි.

- ඔබ රොබෝවරයාට කටින් පවසනවා නම් "මුළුතැන්ගෙයට ගොස් සහල් සෝදා ඛන් උයන්න" කියා, එය CLI ක්‍රමයයි.

52.2 ඇයි වෘත්තීය මෘදුකාංග ඉංජිනේරුවෙකුට ටර්මිනල් අවශ්‍ය වන්නේ?

බොහෝ දෙනෙක් අසන ප්‍රශ්නයක් නම්, "මවුසයෙන් ක්ලික් කර වැඩේ කරගන්න පුළුවන් නම්, ඇයි අමාරුවෙන් අකුරු ටයිප් කරන්නේ?" යන්නයි. ඒ සඳහා හේතු කිහිපයක් ඇත:

1. වේගය (Speed): පළපුරුදු සංවර්ධකයෙකුට මවුසය භාවිතා කරනවාට වඩා වේගයෙන් විධාන ටයිප් කර වැඩ අවසන් කළ හැක.
2. ස්වයංක්‍රීයකරණය (Automation): විධාන මාලාවක් ලියා තැබූ විට, පරිගණකය විසින්ම එම වැඩ සියල්ල එකවර සිදු කරයි.
3. සේවාදායක පාලනය (Server Management): වෙබ් අඩවි ගබඩා කර ඇති සේවාදායක (Servers) වල බොහෝ විට GUI එකක් නැත. ඒවා පාලනය කළ හැක්කේ ටර්මිනලය හරහා පමණි.
4. වෘත්තීය මෙවලම්: ඉදිරියේදී අප ඉගෙන ගන්නා Git, GitHub, React වැනි සියලුම නවීන මෙවලම් ක්‍රියා කරන්නේ ටර්මිනලය හරහාය.

52.3 ටර්මිනලය විවෘත කරගන්නා ආකාරය

ඔබ භාවිතා කරන මෙහෙයුම් පද්ධතිය (OS) අනුව මෙය වෙනස් වේ:

- Windows: PowerShell හෝ Command Prompt (CMD) භාවිතා කරයි.
- Mac / Linux: Terminal භාවිතා කරයි.

වැදගත්: VS Code භාවිතා කරන විට, ඔබට වෙනම මෘදුකාංග විවෘත කිරීමට අවශ්‍ය නැත.

- VS Code හි ඉහළ මෙනුවේ ඇති Terminal -> New Terminal යන්න ක්ලික් කරන්න.
- එවිට VS Code හි පහළ කොටසේ කළු තිරය දිස්වනු ඇත.

52.4 මූලික විධාන (Basic Commands)

දැන් අපි ඉගෙන ගනිමු පරිගණකයට උපදෙස් දීමට භාවිතා කරන ප්‍රධාන වචන කිහිපයක්.

(A) දැනට සිටින ස්ථානයේ ඇති දේ බැලීම (ls හෝ dir) ඔබ සිටින Folder එක ඇතුළේ ඇති සියලුම Files සහ Folders ලැයිස්තුවක් පෙන්වීමට මෙය භාවිතා කරයි.

- Mac/Linux/PowerShell සඳහා: ls (List)
- Windows CMD සඳහා: dir (Directory)

(B) ස්ථානය මාරු කිරීම (cd) Change Directory යන්න මෙහි තේරුමයි. එක් Folder එකක සිට තවත් Folder එකකට යාමට මෙය භාවිතා කරයි.

- භාවිතය: cd folder_name
- උදාහරණ: cd Desktop (ඩෙස්ක්ටොප් එක වෙත යාමට)
- නැවත පස්සට යාමට (Back): cd .. (නිත් දෙකක් යෙදිය යුතුය)

(C) අලුත් Folder එකක් සෑදීම (mkdir) Make Directory යන්න මෙහි තේරුමයි.

- භාවිතය: mkdir my_new_project

(D) අලුත් File එකක් සෑදීම (touch හෝ ni)

- Mac/Linux සඳහා: touch index.html
- Windows PowerShell සඳහා: ni index.html (New Item)

(E) තිරය පිරිසිදු කිරීම (clear හෝ cls) ටර්මිනලය අතුරු වලින් පිරී ඇති විට ඒවා ඉවත් කර පිරිසිදු කිරීමට.

- Mac/Linux/PowerShell සඳහා: clear
- Windows CMD සඳහා: cls

52.5 මාර්ග පද්ධතිය (The Path)

ටර්මිනලය භාවිතා කරන විට ඔබ "කොහේද සිටින්නේ" යන්න දැන ගැනීම ඉතා වැදගත් වේ. පරිගණකය මෙය පෙන්වන්නේ Path (මග) ලෙසයි.

- උදාහරණ:
C:\Users\Kasun\Desktop\MyProject මෙහි තේරුම ඔබ MyProject නමැති ෆෝල්ඩරය තුළ සිටින බවයි.

ඔබ දැනට සිටින සම්පූර්ණ මාර්ගය බැලීමට pwd (Print Working Directory) විධානය භාවිතා කළ හැක.

52.6 ASCII රූපසටහන: Folder ව්‍යුහය හරහා ගමන් කිරීම

Root Folder (මූල)

```
|
|-- index.html
|-- style.css
|-- [ images ] <--- දැනට මෙතන ඉන්නවා නම්
(cd images)
|
|-- logo.png
|-- bg.jpg
```

නැවත මූලට යාමට නම්: ටයිප් කරන්න "cd .."

52.7 වෘත්තීය උපදෙස් (Professional Tips)

- Tab Key එක භාවිතා කරන්න: ඔබ Folder එකක නම ටයිප් කරන විට මුල් අකුරු දෙක තුන ටයිප් කර KeyBoard එකේ ඇති Tab බොත්තම ඔබන්න. එවිට පරිගණකය විසින් ඉතිරි අකුරු ස්වයංක්‍රීයව සම්පූර්ණ කරනු ඇත. මෙය ඉතාමත් වේගවත් ක්‍රමයකි.

- විධාන ඉතිහාසය: ඔබට කලින් ටයිප් කළ විධානයක් නැවත අවශ්‍ය නම්, KeyBoard එකේ ඇති Up Arrow (ඉහළට යන ඊතලය) ඔබන්න.

- නම මැද හිස්තැන්: ෆෝල්ඩර වල නම් වලට හිස්තැන් (Spaces) ලබා දීමෙන් වළකින්න.

(උදා: My Project වෙනුවට my-project ලෙස භාවිතා කරන්න). හිස්තැන් තිබේ නම් විධානයක් ලියන විට එය උද්ධෘත ලකුණු තුළ ලිවිය යුතුය: cd "My Project".

52.8 පොදු වැරදි (Common Mistakes)

1. වැරදි Folder එකක සිට විධාන ලබා දීම: ඔබ index.html සෑදීමට උත්සාහ කරන්නේ වැරදි තැනක සිට නම්, එය සොයා ගැනීමට අපහසු වනු ඇත. සැමවිටම pwd මගින් ඔබ සිටින තැන පරීක්ෂා කරන්න.
2. අකුරු වැරදීම (Case Sensitivity): සමහර පරිගණක (Mac/Linux) වල Project සහ project යනු දෙකකි. අකුරු වල හැඩය (Capital/Simple) ගැන සැලකිලිමත් වන්න.

52.9 සාරාංශය (Key Points)

- CLI යනු අකුරු මගින් පරිගණකය පාලනය කරන ක්‍රමයයි.
- VS Code තුළ ඇති Integrated Terminal එක සංවර්ධකයන්ට ඉතා පහසු වේ.
- ls (පෙන්වීම), cd (යාම), mkdir (සෑදීම) යනු මූලිකම විධාන වේ.
- Path එක මගින් අප සිටින ස්ථානය හඳුනා ගනී.

52.10 අභ්‍යාස (Exercises)

1. GUI සහ CLI අතර ඇති ප්‍රධාන වෙනස්කම් 2ක් ඔබේ වචනයෙන් ලියන්න.
2. ඔබේ පරිගණකයේ VS Code විවෘත කර ටර්මිනලය හරහා පහත කාර්යයන් සිදු කරන්න:
 - web-lab නමින් අලුත් Folder එකක් සාදන්න.
 - එම Folder එක තුළට පිවිසෙන්න (cd).
 - එහි ඇති දේ පෙන්වන්න (ls).
 - නැවත පස්සට (Back) පැමිණෙන්න.
3. ටර්මිනලය පිරිසිදු කිරීමට (Clear) භාවිතා කරන විධානය කුමක්ද?

මිලහ පරිච්ඡේදයෙන් අපි ලෝකයේ ඕනෑම වෘත්තීය මට්ටමේ මෘදුකාංග ඉංජිනේරුවෙකුට අත්‍යවශ්‍ය වන Git (ගිට්) නමැති සංස්කරණ පාලන පද්ධතිය (Version Control System) ගැන ඉතාමත් විස්තරාත්මකව ඉගෙන ගනිමු.

පහස් තුන්වන පරිච්ඡේදය: ගිට් - සංස්කරණ පාලන පද්ධතිය (Git - Version Control System)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

1. සංස්කරණ පාලනය (Version Control) යනු කුමක්ද සහ එහි වැදගත්කම තේරුම් ගැනීම.
2. ගිට් (Git) යනු කුමක්ද සහ එහි ඉතිහාසය කෙටියෙන් වටහා ගැනීම.
3. ගිට් සහ ගිට්හබ් (Git vs GitHub) අතර ඇති වෙනස පැහැදිලිව හඳුනා ගැනීම.
4. ගිට් හි ක්‍රියාකාරී ස්ථර තුන (Working Directory, Staging Area, Repository) අවබෝධ කර ගැනීම.
5. මූලික ගිට් විධාන (git init, git add, git commit, git status) ප්‍රායෝගිකව භාවිතා කරන ආකාරය.
6. "කමිට් පණිවිඩයක්" (Commit Message) නිවැරදිව ලියන ආකාරය ඉගෙන ගැනීම.

53.1 හැඳින්වීම: සංස්කරණ පාලනය යනු කුමක්ද? (What is Version Control?)

ඔබ පාසලේ ව්‍යාපෘතියක් සඳහා පරිගණකයේ වාර්තාවක් (Report) සකස් කරනවා යැයි සිතන්න. ඔබ එය සේවි කරන්නේ මෙලෙසයි:

- වාර්තාව.docx
- දින කිහිපයකට පසු අලුත් දේවල් එකතු කළ විට: වාර්තාව_අලුත්.docx
- තව වෙනස්කම් කළ පසු: වාර්තාව_අවසාන.docx
- නැවතත් වැරද්දක් හදා සේවි කළ විට: වාර්තාව_අවසාන_සැබෑ_එක.docx

අවසානයේදී ඔබ සතුව ගොනු රාශියක් ඇති අතර, මුල් ගොනුවේ නිවු දෙයක් නැවත බැලීමට අවශ්‍ය වුවහොත් එය සොයා ගැනීම ඉතා අපහසු වේ. මෘදුකාංග සංවර්ධනයේදී ද දහස් ගණනක් කේත ජේළි දිනපතා වෙනස් වේ. මෙම වෙනස්කම් ක්‍රමානුකූලව පාලනය කරන ක්‍රමවේදයට Version Control (සංස්කරණ පාලනය) යැයි කියනු ලැබේ.

සැබෑ ලෝකයේ උපමාව (වීඩියෝ ක්‍රීඩාවක Save Points): ඔබ වීඩියෝ ක්‍රීඩාවක් (Video Game) කරන විට, යම් අසීරු අවස්ථාවකට පෙර ඔබ ක්‍රීඩාව "Save" කරයි. ඔබ පරාජය වුවහොත්, මූල සිටම ආරම්භ නොකර අවසානයට සේවි කළ ස්ථානයෙන් (Checkpoint) නැවත ආරම්භ කළ හැක. Git යනු ඔබේ කේතය සඳහා ඇති එවැනි "Save Point" පද්ධතියකි.

53.2 ගිට යනු කුමක්ද? (What is Git?)

Git (ගිට) යනු ලොව ජනප්‍රියම සහ බලවත්ම විමධ්‍යගත සංස්කරණ පාලන පද්ධතියයි (Distributed Version Control System). මෙය 2005 වසරේදී ලිනස් ටෝර්වල්ඩ්ස් (Linus Torvalds) නමැති ඉංජිනේරුවා (Linux මෙහෙයුම් පද්ධතියේ නිර්මාතෘ) විසින් නිර්මාණය කරන ලදී.

Git මගින් කරන ප්‍රධාන කාර්යයන්:

1. කේතයට කරන ලද සෑම කුඩා වෙනසක්ම මතක තබා ගැනීම.
2. අවශ්‍ය ඕනෑම අවස්ථාවක පරණ සංස්කරණයකට (Version) ආපසු යාමට ඉඩ දීම.
3. එකම ව්‍යාපෘතියේ පුද්ගලයන් කිහිප දෙනෙකුට එකවර වැඩ කිරීමට ඉඩ ලබා දීම.
4. කවුරුන්, කවදා, කුමන වෙනසක් කළාද යන්න වාර්තා කිරීම.

53.3 වැදගත්: Git සහ GitHub අතර වෙනස

බොහෝ ආරම්භකයින් මෙම දෙක පටලවා ගනී. මෙය ඉතා හොඳින් තේරුම් ගන්න:

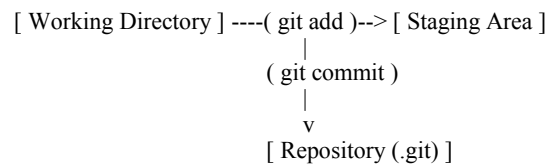
- Git: යනු ඔබේ පරිගණකයේ ස්ථාපනය කරන "මෙවලම" (Tool) වේ. එය ඔබේ පරිගණකය තුළ කේතය පාලනය කරයි. (මිටියක් හෝ කියතක් වැනිය).
- GitHub: යනු අන්තර්ජාලයේ ඇති "වෙබ් අඩවියකි" (Cloud Service). එහි කරන්නේ ඔබ සාදන ලද ගිට ගොනු ලෝකයට පෙනෙන සේ තැන්පත් කර තැබීමයි. (ගබඩාවක් හෝ ප්‍රදර්ශනාගාරයක් වැනිය).

53.4 ගිට වැඩබිමේ ස්ථර 3 (The Three Stages of Git)

Git ක්‍රියා කරන ආකාරය තේරුම් ගැනීමට මෙම කොටස් තුන දැන ගැනීම අත්‍යවශ්‍ය වේ.

1. Working Directory (වැඩ කරන ලෝල්ඩරය): ඔබ දැනට කේතය ලියමින් සිටින සාමාන්‍ය ලෝල්ඩරයයි. මෙහි කරන වෙනස්කම් තවම ගිට විසින් සේවි කර නැත.
2. Staging Area (සුදානම් වීමේ ප්‍රදේශය): ඔබ සේවි කිරීමට (Commit කිරීමට) බලාපොරොත්තු වන ගොනු තෝරාගෙන පෝලිමේ තබා ගන්නා ස්ථානයයි.
3. Repository / .git directory (ගබඩාව): ඔබ ස්ථිරවම "Save Point" එකක් සලකුණු කළ පසු එම තොරතුරු ගබඩා වන ආරක්ෂිත ස්ථානයයි.

ASCII රූපසටහන: Git Workflow



53.5 මූලික ගිට විධාන (Basic Git Commands)

දැන් අපි ප්‍රායෝගිකව ගිට භාවිතා කරන ආකාරය බලමු. ඔබේ පරිගණකයේ VS Code ටර්මිනලය විවෘත කරන්න.

1. ව්‍යාපෘතියක් ආරම්භ කිරීම (git init)

යම් ලෝල්ඩරයක් ඇතුළත ගිට පද්ධතිය සක්‍රීය කිරීමට මෙය භාවිතා කරයි.

git init

මෙය ටයිප් කළ පසු, ඔබේ ලෝල්ඩරය තුළ නොපෙනෙන .git නමැති ලෝල්ඩරයක් සෑදේ. දැන් ගිට ඔබේ කේතය දෙස බලා සිටීමට පටන් ගනී.

2. තත්ත්වය පරීක්ෂා කිරීම (git status)

දැනට කේතයේ සිදුවී ඇති වෙනස්කම් මොනවාදැයි බැලීමට මෙය නිතරම භාවිතා කරයි.

git status

3. ගොනු පෝලිමට එකතු කිරීම (git add)

සේවි කිරීමට පෙර ගොනු Staging area එකට දැමීමට මෙය භාවිතා කරයි.

- තනි ගොනුවක් සඳහා: git add index.html

- සියලුම වෙනස් වූ ගොනු සඳහා: `git add .` (නිතක් යෙදීමෙන් අදහස් වන්නේ "සියලුම" යන්නයි).

4. ස්ථිරවම සේවි කිරීම (git commit)

මෙය ඉතාමත් වැදගත් පියවරයි. මෙහිදී අප කරන්නේ "Save Point" එකක් සෑදීමයි. සෑම කමිටි එකකටම අර්ථවත් පණිවිඩයක් තිබිය යුතුය.

`git commit -m "වෙබ් අඩවියේ මෙනුව සාදා නිම කරන ලදී"`

- `-m` යන්නෙන් "Message" යන්න අදහස් වේ.

53.6 වෘත්තීය Commit Message එකක් ලියන ආකාරය

වෘත්තීය මට්ටමේදී "Modified", "Update", "Changes" වැනි තේරුමක් නැති වචන භාවිතා නොකරන්න. ඒ වෙනුවට:

- කෙටි සහ පැහැදිලි වන්න.
- වර්තමාන කාලයෙන් (Present Tense) ලියන්න. (උදා: "Add login form", "Fix header bug").

53.7 ASCII රූපසටහන: Git ගමන (Summary)

```

කේතය ලිවීම -> [ Working ]
                    |
                    | <--- පෝලිමට දමමු
                    v
පෝලිම (Staging) -> [ Area ]
                    |
                    | <--- Checkpoint එකක් ගනිමු
                    v
ගබඩාව (Repo) -> [ .git ] <--- දැන් ආරක්ෂිතයි!

```

53.8 පොදු වැරදි (Common Mistakes)

1. Git Add කිරීමට අමතක වීම: බොහෝ අය `commit` කළත් එහි වෙනසක් සිදු නොවේ. එයට හේතුව ඊට පෙර `add` විධානය භාවිතා කර ගොනු පෝලිමට නොදැමීමයි.
2. සෑම පේළියකටම කමිටි කිරීම: සෑම අකුරකටම වෙනස් කරන විට කමිටි කිරීම අවශ්‍ය නැත. යම් නිශ්චිත වැඩ කොටසක් (උදා: Navbar එක සාදා නිම කිරීම) අවසන් වූ පසු කමිටි කරන්න.

3. වැරදි Folder එකක Init කිරීම: ඔබ නොදැනුවත්ව ඔබේ Desktop එකේ හෝ මුළු පරිගණකයේම `git init` කළහොත් එය විශාල අවදානමක් විය හැක. සැමවිටම ව්‍යාපෘති ෆෝල්ඩරය තුළ පමණක් එය කරන්න.

53.9 වෘත්තීය උපදෙස් (Professional Tips)

- නිතර කමිටි කරන්න (Commit Often): දවස අවසානයේ එක විශාල කමිටි එකක් කරනවාට වඩා, කුඩා වැඩ අවසන් වන විට කමිටි 5-10ක් කිරීම වඩාත් සුදුසුය. එවිට වැරද්දක් වුවහොත් ආපසු යාමට පහසු වේ.
- ගිට් ලොග් පරීක්ෂාව (git log): ඔබේ ව්‍යාපෘතියේ ඉතිහාසය (History) බැලීමට `git log` විධානය භාවිතා කරන්න. එහි ඔබ මෙතෙක් කළ සියලුම කමිටි ලැයිස්තුවක් පෙන්වනු ඇත.

53.10 අභ්‍යාස (Exercises)

1. සංස්කරණ පාලන පද්ධතියක් (VCS) භාවිතා කිරීමේ ප්‍රධාන වාසි 3ක් ලියන්න.
2. පහත දැක්වෙන අවස්ථා වලදී භාවිතා කළ යුතු ගිට් විධානය ලියන්න:
 - අලුතින් ගිට් ව්‍යාපෘතියක් ආරම්භ කිරීමට.
 - වැඩ කරන ෆෝල්ඩරයේ ඇති සියලුම ගොනු පෝලිමට (Stage) එක් කිරීමට.
 - අර්ථවත් පණිවිඩයක් සමඟ වෙනස්කම් ස්ථිරවම සේවි කිරීමට.
3. Git හි "Staging Area" එකේ කාර්යභාරය ඔබේ වචනයෙන් පැහැදිලි කරන්න.

53.11 ප්‍රායෝගික ක්‍රියාකාරකම (Practical Task)

1. ඔබේ පරිගණකයේ `git-test` නමින් ෆෝල්ඩරයක් සාදන්න.
2. VS Code හරහා එම ෆෝල්ඩරය විවෘත කර ටර්මිනලයේ `git init` කරන්න.
3. `hello.txt` නමින් ගොනුවක් සාදා එහි යමක් ලියන්න.
4. `git status` මගින් ගොනුව රතු පාටින් පෙන්වන ආකාරය බලන්න.
5. `git add hello.txt` කර නැවත `git status` බලන්න. (දැන් එය කොළ පාට විය යුතුය).
6. `git commit -m "පළමු ගොනුව එක් කරන ලදී"` විධානය ලබා දෙන්න.

මිලහ පරිච්ඡේදයෙන් අපි ඔබගේ කේතය අන්තර්ජාලයේ ගබඩා කරන GitHub වෙබ් අඩවිය සහ ඔබගේ

පරිගණකයේ ඇති කේතය එයට යවන (Push) ආකාරය ඉතාමත් විස්තරාත්මකව ඉගෙන ගනිමු.

පහස් හතරවන පරිච්ඡේදය: ගිට්හබ් - ඔබේ කේතය වලාකුළේ තැන්පත් කරමු (GitHub - Hosting Your Code in the Cloud)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

1. ගිට්හබ් (GitHub) යනු කුමක්ද සහ එහි ඇති වෘත්තීය වැදගත්කම වටහා ගැනීම.
2. ගිට්හබ් ගිණුමක් (Account) නිවැරදිව සාදාගන්නා ආකාරය.
3. දුරස්ථ ගබඩාවක් (Remote Repository) යනු කුමක්දැයි හඳුනා ගැනීම.
4. ඔබේ පරිගණකයේ ඇති ගිට් ව්‍යාපෘතියක් ගිට්හබ් වෙත යවන (Push) ආකාරය.
5. git remote, git push, git pull සහ git clone යන විධානයන් ඉගෙන ගැනීම.
6. ගිට්හබ් ඔබේ වෘත්තීය ජීව දත්ත පත්‍රිකාවක් (Portfolio) ලෙස භාවිතා කරන ආකාරය.

54.1 හැඳින්වීම: ගිට්හබ් යනු කුමක්ද? (What is GitHub?)

පෙර පරිච්ඡේදයේදී අපි ඉගෙන ගත්තේ ගිට් (Git) භාවිතා කර අපේ පරිගණකය තුළම කේතයේ සංස්කරණ පාලනය කරන ආකාරයයි. නමුත් ඔබේ පරිගණකය හදිසියේම අක්‍රීය වුවහොත් හෝ ඔබට වෙනත් පරිගණකයක සිට වැඩ කිරීමට අවශ්‍ය වුවහොත් කුමක් සිදුවේද? එතැනදී අපට අපේ කේතය අන්තර්ජාලයේ ආරක්ෂිතව ගබඩා කිරීමට තැනක් අවශ්‍ය වේ.

GitHub (ගිට්හබ්) යනු ගිට් (Git) භාවිතා කර නිර්මාණය කරන ව්‍යාපෘති අන්තර්ජාලය තුළ තැන්පත් කර තැබීමට (Hosting) ඉඩ ලබා දෙන ලොව විශාලතම වෙබ් අඩවියයි. මෙය මෘදුකාංග සංවර්ධකයන් සඳහා ඇති "සමාජ ජාලය" (Social Network) ලෙසද හැඳින්විය හැක.

සැබෑ ලෝකයේ උපමාව (පුද්ගලික දිනපොත සහ පුස්තකාලය):

- Git: යනු ඔබේ නිවසේ ඔබ ලියන රහස් දිනපොත වැනිය. එය ඔබේ පාලනයේ පවතී.
- GitHub: යනු එම දිනපොතේ පිටපතක් ලෝකයේ ඕනෑම අයෙකුට කියවිය හැකි සේ (හෝ ඔබට පමණක් පෙනෙන සේ) තබා ඇති විශාල මහජන පුස්තකාලයක් වැනිය.

54.2 ගිට්හබ් ඇයි මෙතරම් වැදගත් වන්නේ? (Importance)

1. ආරක්ෂිත පිටපත (Backup): ඔබේ කේතය අන්තර්ජාලයේ සුරැකී පවතී.
2. සහයෝගීතාව (Collaboration): ලෝකයේ විවිධ රටවල සිටින සංවර්ධකයන්ට එකම ව්‍යාපෘතියකට දායක විය හැක.
3. වෘත්තීය ප්‍රදර්ශනාගාරය (Portfolio): අද කාලයේ ඔබ රැකියාවකට යන විට, ඔබ ලියූ කේතයන් බැලීමට සේවාදායකයන් (Clients) ඉල්ලා සිටින්නේ ඔබේ ගිට්හබ් ලිපික් එකයි.
4. විවෘත මූලාශ්‍ර (Open Source): ලෝක ප්‍රසිද්ධ මෘදුකාංග වල කේත නොමිලේ කියවීමට සහ ඒවාට උදව් කිරීමට ගිට්හබ් මග පාදයි.

54.3 මූලික පාරිභාෂික වචන (Terminology)

- Repository (Repo): ඔබේ ව්‍යාපෘතිය ගබඩා කර ඇති "පෙට්ටිය" හෝ ලෝල්ඩරයයි.
- Remote (දුරස්ථ): අන්තර්ජාලයේ (GitHub හි) ඇති ඔබේ ව්‍යාපෘතියේ පිටපතයි.
- Push (තල්ලු කිරීම/යැවීම): ඔබේ පරිගණකයේ ඇති වෙනස්කම් ගිට්හබ් වෙත යැවීම.
- Pull (ඇද ගැනීම/ලබා ගැනීම): ගිට්හබ් හි ඇති වෙනස්කම් ඔබේ පරිගණකයට ලබා ගැනීම.
- Clone (පිටපත් කිරීම): ගිට්හබ් හි ඇති ව්‍යාපෘතියක් මූලින්ම ඔබේ පරිගණකයට භාගත කර ගැනීම.

54.4 පියවරෙන් පියවර: කේතය ගිට්හබ් වෙත යවන ආකාරය

දැන් අපි ප්‍රායෝගිකව වැඩේ කරමු.

පියවර 1: ගිණුමක් සෑදීම

github.com වෙත ගොස් ඔබගේ විද්‍යුත් තැපෑල (Email) භාවිතා කර නොමිලේ ගිණුමක් සාදාගන්න.

පියවර 2: අලුත් ගබඩාවක් (Repository) සෑදීම

ගිට්හබ් හි ඉහළ දකුණු කෙළවරේ ඇති "+" ලකුණ ක්ලික් කර "New repository" තෝරන්න.

- Repository name: ඔබේ ව්‍යාපෘතියට නමක් දෙන්න (උදා: my-portfolio).
- Public/Private: ලෝකයටම පෙනෙන්න අවශ්‍ය නම් Public තෝරන්න.
- Create repository බොක්කම ඔබන්න.

දැන් ඔබට වෙබ් ලිපිනයක් (URL) ලැබෙනු ඇත. එය මෙවැනි වේ:
https://github.com/ඔබේ_නම/my-portfolio.git

පියවර 3: ඔබේ පරිගණකය සහ ගිට්හබ් සම්බන්ධ කිරීම

දැන් ඔබේ පරිගණකයේ VS Code ටර්මිනලය විවෘත කර (ව්‍යාපෘති ෆෝල්ඩරය තුළ සිට), පහත විධානයන් ටයිප් කරන්න.

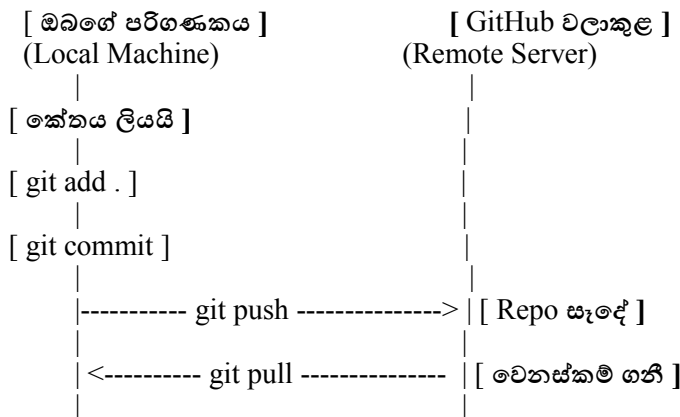
```
# 1. ඔබේ ගබඩාව කොහේදී තියෙන්නේ කියා පරිගණකයට පවසන්න
git remote add origin
https://github.com/ඔබේ_නම/my-portfolio.git
```

```
# 2. ප්‍රධාන අත්ත (Main Branch) නම් කරන්න
git branch -M main
```

```
# 3. කේතය ගිට්හබ් වෙත යවන්න (Push)
git push -u origin main
```

(පළමු වතාවේදී ඔබට GitHub වෙත ලොගින් වීමට පණිවිඩයක් ලැබෙනු ඇත. එයට එකඟ වන්න).

54.5 ASCII රූපසටහන: පරිගණකය සහ GitHub අතර සම්බන්ධය



54.6 නිතර භාවිතා වන විධාන (Daily Commands)

ඔබ ව්‍යාපෘතියක වැඩ කරන විට සාමාන්‍යයෙන් දිනපතා මෙම පිළිවෙළ අනුගමනය කරයි:

1. කේතයේ වෙනස්කම් සිදු කරයි.
2. git add . (පෝලිමට දමයි).
3. git commit -m "අඟවල් දේ වෙනස් කළා" (සේව් කරයි).
4. git push origin main (ගිට්හබ් එකට යවයි).

54.7 ක්ලෝන් කිරීම (Cloning)

ඔබට වෙනත් පරිගණකයක සිට ඔබේම ව්‍යාපෘතියක් හෝ වෙනත් අයෙකුගේ ව්‍යාපෘතියක්

ලබා ගැනීමට අවශ්‍ය නම් clone විධානය භාවිතා කරයි.

```
git clone https://github.com/නම/ව්‍යාපෘතිය.git
```

මෙමගින් එම ව්‍යාපෘතියේ සම්පූර්ණ පිටපතක් ඔබේ පරිගණකය තුළ සෑදේ.

54.8 පොදු වැරදි (Common Mistakes)

1. කමිට් (Commit) නොකර පුෂ් (Push) කිරීම: ඔබ වෙනස්කම් කමිට් කර නැතිනම්, පුෂ් කිරීමට කිසිවක් ඉතිරි නොවේ.
2. වැරදි URL එක භාවිතා කිරීම: URL එකේ සුළු වැරදීමක් වුවහොත් පරිගණකයට ගිට්හබ් සොයාගත නොහැක.
3. Conflict (ගැටුම්): ඔබ පරිගණකයෙන් යමක් වෙනස් කර පුෂ් කිරීමට පෙර, ගිට්හබ් වෙබ් අඩවිය

තුළම යමක් වෙනස් කර තිබුණහොත්, ගිට් විසින් "Conflict" එකක් පෙන්වනු ඇත. එවිට ඔබට මුලින්ම git pull කිරීමට සිදු වේ.

54.9 වෘත්තීය උපදෙස් (Professional Tips)

- README.md ගොනුව: සෑම ව්‍යාපෘතියකම README.md නමින් ගොනුවක් තබා ගන්න. එහි එම ව්‍යාපෘතිය කුමක්ද සහ එය භාවිතා කරන්නේ කෙසේදැයි විස්තර කරන්න. මෙය ඔබව වෘත්තීය මට්ටමේ සංවර්ධකයෙකු ලෙස පෙන්වීමට උදව් වේ.

- Star (ස්ටාර්): ඔබ කැමති ව්‍යාපෘති වලට ගිට්හබ් හි "Star" එකක් ලබා දෙන්න. එය එම සංවර්ධකයා දිරිමත් කිරීමකි.

- Contribution Graph: ඔබ දිනපතා කේත ලියා පූෂ් කරන විට, ඔබේ ගිට්හබ් ප්‍රෝෆයිලයේ ඇති කොටු කොළ පාට වේ. රැකියා ලබා දෙන ආයතන මෙම ග්‍රාෆ් එක දෙස බලා ඔබ කෙතරම් උනන්දුවෙන් වැඩ කරනවාදැයි තීරණය කරයි.

54.10 අභ්‍යාස (Exercises)

1. ඔබේ නමින් ගිට්හබ් ගිණුමක් සාදාගන්න.
2. html-css-practice නමින් අලුත් Repository එකක් සාදන්න.
3. ඔබ මෙතෙක් සාදන ලද ඕනෑම HTML පිටුවක් එම Repository එකට push කරන්න.
4. ඔබේ මිතුරෙකුගේ ගිට්හබ් ලින්ක් එකක් ලබාගෙන එය ඔබේ පරිගණකයට clone කර බලන්න.
5. git push සහ git pull අතර ඇති වෙනස පැහැදිලි කරන්න.

මිලහ පරිච්ඡේදයෙන් අපි ඔබේ වෙබ් අඩවිය ඕනෑම අයෙකුට අන්තර්ජාලය හරහා නැරඹිය හැකි සේ නොමිලේ ප්‍රසිද්ධ කරන ආකාරය හෙවත් Deployment (Netlify & Vercel) ගැන ඉතාමත් විස්තරාත්මකව ඉගෙන ගනිමු.

පහස් පස්වන පරිච්ඡේදය: වෙබ් අඩවි ප්‍රසිද්ධ කිරීම - ලෝකයටම පෙන්වමු (Deployment - Making Your Site Public)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

1. වෙබ් අඩවියක් ප්‍රසිද්ධ කිරීම (Deployment) සහ සත්කාරකත්වය (Hosting) යනු කුමක්දැයි තේරුම් ගැනීම.
2. ලෝකල් හොස්ට් (Localhost) සහ සජීවී සේවාදායකයක් (Live Server) අතර වෙනස හඳුනා ගැනීම.
3. නෙට්ලිෆයි (Netlify) සහ වර්සෙල් (Vercel) යන නවීන මෙවලම් හඳුනා ගැනීම.
4. GitHub සමඟ සම්බන්ධ වී සන්තතික ප්‍රසිද්ධ කිරීම (Continuous Deployment) සිදු කරන ආකාරය.
5. ඔබගේ වෙබ් අඩවිය සඳහා නොමිලේ වෙබ් ලිපිනයක් (URL) ලබා ගන්නා ආකාරය.
6. වෙබ් අඩවියක් ප්‍රසිද්ධ කිරීමට පෙර පරීක්ෂා කළ යුතු අවසාන පියවර (Final Checklist) අවබෝධ කර ගැනීම.

55.1 හැඳින්වීම: ප්‍රසිද්ධ කිරීම (Deployment) යනු කුමක්ද?

මෙතෙක් ඔබ වෙබ් අඩවි නිර්මාණය කළේ ඔබේ පරිගණකය තුළය. ඔබ ඒවා බ්‍රවුසරයෙන් නැරඹුවේ C:/Users/Kasun/Desktop/index.html වැනි ලිපිනයක් හරහාය. මෙම ලිපිනය ඔබ ඔබේ මිතුරෙකුට යැවුවහොත්, ඔහුට එය බැලීමට නොහැක. මන්ද එම ගොනු තිබෙන්නේ ඔබේ පරිගණකයේ දෘඩ තැටිය (Hard Drive) තුළ පමණක් වන බැවිනි.

Deployment (ඩිප්ලෝයිමන්ට්) යනු ඔබේ පරිගණකයේ ඇති වෙබ් අඩවි ගොනු (HTML, CSS, JS) අන්තර්ජාලයට සම්බන්ධ වූ බලවත් සේවාදායකයක් (Server) වෙත යැවීමේ ක්‍රියාවලියයි. එලෙස යැවූ පසු, ලෝකයේ ඕනෑම තැනක සිටින ඕනෑම අයෙකුට වෙබ් ලිපිනයක් (URL) හරහා ඔබේ නිර්මාණය නැරඹිය හැක.

සැබෑ ලෝකයේ උපමාව (පොතක් ලිවීම සහ මුද්‍රණය කිරීම): සිතන්න ඔබ ලස්සන කතාවක් ලියනවා කියා.

- ඔබ එය ඔබේ පොතේ ලියන තෙක් එය කියවිය හැක්කේ ඔබට පමණි. මෙය Local Development (ඔබේ පරිගණකයේ වැඩ කිරීම) වැනිය.

- ඔබ එම කතාව මුද්‍රණය කර ප්‍රකාශන ආයතනයක් හරහා පොත් කඩ වලට බෙදා හැරිය විට ඕනෑම අයෙකුට එය කියවිය හැක. මෙය Deployment (ප්‍රසිද්ධ කිරීම) වැනිය.

55.2 සත්කාරකත්වය (Web Hosting) යනු කුමක්ද?

ඔබේ වෙබ් අඩවියේ ගොනු අන්තර්ජාලයේ ගබඩා කර තබන "ස්ථානය" Hosting (හෝස්ටිං) ලෙස හැඳින්වේ. සාමාන්‍යයෙන් මේ සඳහා මාසිකව මුදලක් ගෙවිය යුතුය. නමුත් අද වන විට Frontend Developer වරුන් සඳහා Netlify සහ Vercel වැනි ආයතන විසින් ඉතා උසස් මට්ටමේ පහසුකම් නොමිලේ ලබා දෙයි.

55.3 නෙට්ලිෆි (Netlify) සහ වර්සෙල් (Vercel)

මෙම වෙබ් අඩවි දෙකම ඉතා ජනප්‍රියය. ඒවායේ විශේෂත්වය වන්නේ ඔබ GitHub වෙත කේතය යැවූ (Push කළ) සැණින්, ස්වයංක්‍රීයවම ඔබේ වෙබ් අඩවිය යාවත්කාලීන (Update) කිරීමයි.

- Netlify: ආරම්භකයින්ට ඉතා පහසුය. සරල HTML/CSS වෙබ් අඩවි සඳහා ඉතා සුදුසුය.
- Vercel: React වැනි දියුණු තාක්ෂණයන් සඳහා වැඩිපුර භාවිතා කරයි.

55.4 සන්තතික ප්‍රසිද්ධ කිරීම (Continuous Deployment - CD)

මෙය වෘත්තීය වෙබ් සංවර්ධනයේ ඇති ඉතාමත් පහසු ක්‍රමයකි.

1. ඔබ ඔබේ පරිගණකයේ කේතය වෙනස් කරයි.
2. ඔබ එය GitHub වෙත Push කරයි.
3. GitHub විසින් Netlify වෙත පණිවිඩයක් යවයි: "මෙන්න අලුත් කේතය ඇවිත්!".
4. Netlify ස්වයංක්‍රීයව එම අලුත් කේතය ගෙන වෙබ් අඩවිය අලුත් කරයි.

පරිශීලකයාට සැමවිටම පෙනෙන්නේ ඔබේ අලුත්ම සංස්කරණයයි.

55.5 පියවරෙන් පියවර: Netlify හරහා ප්‍රසිද්ධ කරන ආකාරය

අපි දැන් ඔබගේ පෝට්ෆෝලියෝ වෙබ් අඩවිය (පරිමාව 2 හි සෑදූ) අන්තර්ජාලයට මුදාහරිමු.

පූර්ව අවශ්‍යතාවය: ඔබගේ ව්‍යාපෘතිය GitHub හි Repository එකක් ලෙස තිබිය යුතුය (පෙර පරිච්ඡේදය බලන්න).

පියවර 1: Netlify ගිණුමක් සාදාගන්න

netlify.com වෙත ගොස් Sign up with GitHub යන්න තෝරන්න. (එවිට වෙනම ගිණුම් සෑදීමට අවශ්‍ය නැත).

පියවර 2: GitHub Repository එක සම්බන්ධ කරන්න

1. Netlify Dashboard එකේ ඇති "Add new site" බොත්තම ඔබන්න.
2. "Import an existing project" යන්න තෝරන්න.
3. "GitHub" බොත්තම ක්ලික් කරන්න. (Netlify හට ඔබේ GitHub ගිණුමට ඇතුළු වීමට අවසර දෙන්න).

පියවර 3: ව්‍යාපෘතිය තෝරාගන්න

ලැයිස්තුවේ පෙන්වන ඔබේ ව්‍යාපෘතිය (උදා: my-portfolio) මත ක්ලික් කරන්න.

පියවර 4: Deploy (ප්‍රසිද්ධ) කරන්න

මෙහිදී ඔබට විවිධ Settings කිහිපයක් පෙනෙනු ඇත. දැනට ඒවා වෙනස් නොකර පතුලේ ඇති "Deploy my-portfolio" බොත්තම ඔබන්න.

දැන් තත්පර කිහිපයක් රැඳී සිටින්න. Netlify විසින් ඔබේ කේතය කියවා වෙබ් අඩවිය සකස් කරනු ඇත.

පියවර 5: වෙබ් ලිපිනය (URL) ලබා ගැනීම

වැඩේ අවසන් වූ පසු ඔබට ලිපිනක් එකක් ලැබෙනු ඇත. උදාහරණයක් ලෙස: <https://super-creative-site-123.netlify.app>. දැන් ඔබට මෙම ලිපිනක් එක ඕනෑම අයෙකුට යැවිය හැක!

55.7 වෘත්තීය උපදෙස් (Professional Tips)

- Custom Domains (පුද්ගලික නාමයන්):
.netlify.app වෙනුවට www.yourname.com වැනි නමක් ඔබේ වෙබ් අඩවියට ලබා දීමට අවශ්‍ය නම්, ඔබට Netlify හරහාම එම නම මිලදී ගෙන සම්බන්ධ කළ හැක.
- Site Settings: Netlify හි ඇති "Domain Settings" වෙත ගොස් ඔබේ වෙබ් අඩවියේ ඇති අමුතු නම (උදා: super-creative-site) ඔබට කැමති නමකට (උදා: kasun-dev) වෙනස් කර ගත හැක.
- HTTPS (ආරක්ෂාව): Netlify සහ Vercel මගින් ඔබේ වෙබ් අඩවියට නොමිලේම SSL සහතිකය (අගුළු ලකුණ සහිත ආරක්ෂාව) ලබා දෙයි.

55.8 පොදු වැරදි (Common Mistakes)

1. Index.html නොමැති වීම: ඔබේ ප්‍රධාන ගොනුව අනිවාර්යයෙන්ම index.html විය යුතුය. ඔබ එය home.html හෝ main.html ලෙස නම් කළහොත්, සේවාදායකයට මුලින්ම පෙන්විය යුතු පිටුව සොයා ගැනීමට නොහැකි වී "404 Not Found" වැරද්ද පෙන්වනු ඇත.
2. ගොනු නාමවල හැඩය (Case Sensitivity): ඔබේ පරිගණකයේ (Windows) Image.jpg සහ image.jpg යනු එකම දෙයක් විය හැක. නමුත් අන්තර්ජාල සේවාදායකයන් (Linux) මේ දෙක පටලවා නොගනී. ඔබ HTML හි `src="image.jpg"` ලෙස ලියා සැබෑ ගොනුව Image.jpg ලෙස නිබුණහොත්, ප්‍රසිද්ධ කළ පසු පින්තූරය නොපෙනී යනු ඇත.
3. නොකළ කමිටි (Uncommitted Changes): ඔබ පරිගණකයේ වෙනසක් කර GitHub වෙත Push නොකළහොත්, වෙබ් අඩවිය වෙනස් නොවේ.

55.9 සාරාංශය (Key Points)

- Deployment යනු වෙබ් අඩවියක් අන්තර්ජාලයට මුදා හැරීමයි.
- Netlify සහ Vercel යනු Frontend Developers ලා සඳහා ඇති හොඳම නොමිලේ හෝ ස්වේච්ඡා සේවාවන්ය.
- GitHub හරහා සම්බන්ධ කළ විට, ඔබ Push කරන සෑම වාරයකම වෙබ් අඩවිය ස්වයංක්‍රීයව යාවත්කාලීන වේ.

- සෑම ව්‍යාපෘතියකටම index.html ගොනුවක් තිබීම අනිවාර්ය වේ.

55.10 අභ්‍යාස (Exercises)

1. ප්‍රසිද්ධ කිරීම (Deployment) සහ සත්කාරකත්වය (Hosting) අතර වෙනස ඔබේ වචනයෙන් පැහැදිලි කරන්න.
2. ඔබේ පරිගණකයේ ඇති ඕනෑම ව්‍යාපෘතියක් Netlify හරහා ප්‍රසිද්ධ කර එහි ලින්ක් එක ඔබේ දුරකථනයෙන් විවෘත කර බලන්න.
3. වෙබ් අඩවියක් ප්‍රසිද්ධ කළ පසු එහි ඇති අකුරක් වෙනස් කර GitHub වෙත Push කරන්න. දැන් නැවත වෙබ් ලිපිනයට (URL) ගොස් බලන්න; වෙනස ස්වයංක්‍රීයව සිදු වී තිබේද?
4. Netlify හි ඇති වෙබ් ලිපිනය (URL) ඔබේ නම සහිත ලින්ක් එකක් ලෙස වෙනස් කරන්න.

මිළඟ පරිච්ඡේදයෙන් අපි වෙබ් අඩවියක ඇති වැදගත්ම කොටසක් වන "සෙවුම් යන්ත්‍ර ප්‍රශස්තකරණය" (SEO) සහ Google හි ඔබේ වෙබ් අඩවිය මුලටම ගෙන එන රහස් ඉතාමත් විස්තරාත්මකව ඉගෙන ගනිමු.

පහස් හයවන පරිච්ඡේදය: සෙවුම් යන්ත්‍ර ප්‍රශස්තකරණය සහ කාර්යක්ෂමතාව (SEO and Web Performance)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

1. SEO (Search Engine Optimization) යනු කුමක්දැයි වෘත්තීය මට්ටමින් තේරුම් ගැනීම.
2. Google වැනි සෙවුම් යන්ත්‍ර වෙබ් අඩවියක් කියවන ආකාරය (Crawling & Indexing) වටහා ගැනීම.
3. On-Page SEO සඳහා අත්‍යවශ්‍ය Meta Tags භාවිතා කරන ආකාරය.
4. සෙවුම් ප්‍රතිඵලවල ඔබේ වෙබ් අඩවිය ඉහළට ගෙන ඒම සඳහා Semantic HTML වල බලපෑම.
5. වෙබ් අඩවියක වේගය (Performance) සහ SEO අතර ඇති සම්බන්ධය හඳුනා ගැනීම.
6. පින්තූර සහ කේත ප්‍රශස්තකරණය (Optimization) කරන ආකාරය.
7. Google Lighthouse මෙවලම භාවිතා කර වෙබ් අඩවියේ ගුණාත්මකභාවය පරීක්ෂා කිරීම.

56.1 SEO යනු කුමක්ද? (What is SEO?)

SEO යනු Search Engine Optimization යන්නෙහි කෙටි නමයි. සිංහලෙන් මෙයට "සෙවුම් යන්ත්‍ර ප්‍රශස්තකරණය" යැයි කියනු ලැබේ. සරලවම කිවහොත්, යමෙකු Google, Bing හෝ Yahoo වැනි සෙවුම් යන්ත්‍රයක යම් වචනයක් (Keyword) සෙවූ විට, ඔබේ වෙබ් අඩවිය එම ප්‍රතිඵල ලැයිස්තුවේ මුලටම ගෙන ඒමට කරනු ලබන තාක්ෂණික වෙනස්කම් සමූහය SEO ලෙස හැඳින්වේ.

සැබෑ ලෝකයේ උපමාව (පුස්තකාලය):
ලෝකයේ ඇති සියලුම වෙබ් අඩවි යනු මහා විශාල පුස්තකාලයක ඇති පොත් වැනිය. Google යනු එම පුස්තකාලයේ සිටින සේවකයා (Librarian) වැනිය. යමෙකු පැමිණ "මට වෙබ් නිර්මාණය ගැන පොතක් ඕනේ" කියා ඇසූ විට, එම සේවකයා මුලින්ම ලබා දෙන්නේ හොඳම මාතෘකාව ඇති, පිරිසිදුව ඇති සහ වැඩි දෙනෙක් නිර්දේශ කළ පොතයි. SEO මගින් කරන්නේ ඔබේ වෙබ් අඩවිය එම "හොඳම පොත" බවට පත් කිරීමයි.

56.2 Google ක්‍රියා කරන්නේ කෙසේද? (How Search Engines Work)

Google ඔබේ වෙබ් අඩවිය සොයා ගන්නේ ප්‍රධාන පියවර 3ක් හරහාය:

1. Crawling (සොයා යාම): Google සතු "Bots" හෝ "Spiders" නමැති මෘදුකාංග අන්තර්ජාලය පුරා සැරිසරමින් අලුත් වෙබ් අඩවි සොයයි.
2. Indexing (ලියාපදිංචි කිරීම): සොයාගත් වෙබ් අඩවියේ අන්තර්ගතය කියවා බලා, එය කුමක් ගැනදැයි තේරුම් ගෙන Google හි මහා දත්ත ගබඩාවේ (Index) තැන්පත් කරයි.
3. Ranking (පෙළගැස්වීම): යමෙකු යමක් සෙවූ විට, දත්ත ගබඩාවේ ඇති වෙබ් අඩවි අතරින් වඩාත් ගැලපෙනම ඒවා පිළිවෙළකට පෙන්වයි.

56.3 On-Page SEO: කේතය තුළ කළ යුතු වෙනස්කම්

Frontend Developer කෙනෙකු ලෙස ඔබට SEO සඳහා කළ හැකි වැදගත්ම දේ මෙයයි. මෙයට HTML හි <head> කොටසට එකතු කරන දත්ත (Metadata) අයිති වේ.

1. Title Tag (මාතෘකාව)

සෙවුම් ප්‍රතිඵලවල නිල් පාටින් පෙනෙන ප්‍රධාන ලිපි එක මෙයයි. එය අකුරු 50-60 අතර තිබීම සුදුසුය.

```
<title>වෙබ් නිර්මාණය ඉගෙන ගනිමු | සිංහලෙන් සම්පූර්ණ පාඨමාලාව</title>
```

2. Meta Description (විස්තරය)

පිටුව ගැන කෙටි හැඳින්වීමක්. මෙය අකුරු 150-160 අතර තිබිය යුතුය.

```
<meta name="description" content="මූල සිට සරලව සිංහලෙන් වෙබ් සංවර්ධනය ඉගෙන ගන්න. HTML, CSS සහ JavaScript පිළිබඳව සම්පූර්ණ මගපෙන්වීම මෙතැනින්.">
```

3. Semantic HTML (අර්ථවත් ටැග්)

Google බොට්ස්ලා ඔබේ වෙබ් අඩවිය කියවන විට ඔවුන්ට <div> වලට වඩා <header>, <main>, <article>, <footer> වැනි ටැග් තේරුම් ගැනීම පහසුය. අපි පරිමාව 1, 8 වන පරිච්ඡේදයේ ඉගෙන ගත් මෙම ටැග් භාවිතා කිරීම SEO වලට සෘජුවම බලපායි.

56.4 වෙබ් අඩවියේ වේගය සහ SEO (Web Performance)

Google විසින් මෑතකදී හඳුන්වා දුන් නීතියක් වන්නේ "වේගවත් වෙබ් අඩවි වලට මුල් තැන දීමයි." ඔබේ වෙබ් අඩවිය ලෝඩ් වීමට තත්පර 3කට වඩා ගත වන්නේ නම්, බොහෝ පරිශීලකයන් එයින් ඉවත් වී යයි. මෙය ඔබගේ Rank එක අඩු කිරීමට හේතු වේ.

වේගය වැඩි කිරීමට කළ හැකි දේ:

1. පින්තූර ප්‍රශස්තකරණය (Image Optimization)

විශාල පින්තූර (උදා: 5MB) වෙබ් අඩවියක තිබීම වේගය අඩු වීමට ප්‍රධාන හේතුවයි.

- WebP: JPEG හෝ PNG වෙනුවට WebP වැනි නවීන සහ සැහැල්ලු වර්ගයේ පින්තූර භාවිතා කරන්න.

- Compression: පින්තූරයේ ගුණාත්මකභාවයට හානි නොවී එහි ප්‍රමාණය (Size) අඩු කරන්න (TinyPNG වැනි වෙබ් අඩවි භාවිතා කළ හැක).

2. මිනිසිරිකේෂණ (Minification)

ඔබේ CSS සහ JS ගොනු වල ඇති හිස්තැන් (Spaces) සහ Comments ඉවත් කර ගොනුවේ ප්‍රමාණය කුඩා කිරීම මෙලෙස හැඳින්වේ. (මෙය ස්වයංක්‍රීයව කරන මෙවලම් තිබේ).

56.5 මූලික SEO පරීක්ෂුම් ලැයිස්තුව (Checklist)

ඔබ වෙබ් අඩවියක් ප්‍රසිද්ධ කිරීමට පෙර මේවා තිබේදැයි බලන්න:

- [] සෑම පින්තූරයකටම alt text එකක් තිබේද?
- [] සෑම පිටුවකටම අනන්‍ය වූ <title> එකක් තිබේද?
- [] වෙබ් අඩවිය ජංගම දුරකථන වලට ගැලපෙනවද (Mobile Friendly)?
- [] <meta name="viewport"> වැගය ඇතුළත් කර තිබේද?
- [] HTTPS (ආරක්ෂාව) සක්‍රියද?

56.6 Google Lighthouse භාවිතා කිරීම

මෙය ඔබගේ වෙබ් අඩවියේ ගුණාත්මකභාවය මැනීමට ඇති හොඳම වෘත්තීය මෙවලමයි.

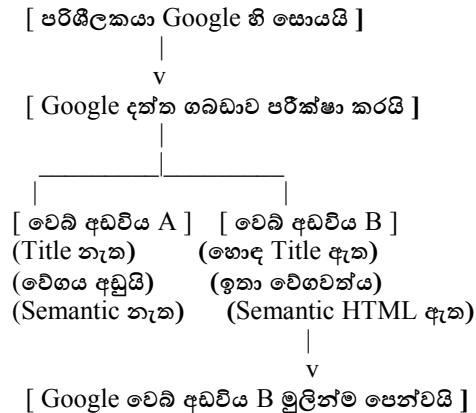
භාවිතා කරන ආකාරය:

1. ඔබේ වෙබ් අඩවිය Google Chrome බ්‍රවුසරයෙන් විවෘත කරන්න.
2. Right Click -> Inspect කරන්න.
3. ඉහළ ඇති වැඩි අතරින් Lighthouse යන්න තෝරන්න.
4. "Analyze page load" බොත්තම ඔබන්න.

එවිට Google විසින් ඔබට අංශ 4කින් ලකුණු 100න් ලබා දෙනු ඇත:

- Performance: වේගය.
- Accessibility: ආබාධිත අයට ඇති පහසුව.
- Best Practices: වෘත්තීය නීති පිළිපැදීම.
- SEO: සෙවුම් යන්ත්‍ර වලට ඇති ගැලපීම.

56.7 ASCII රූපසටහන: SEO බලපාන ආකාරය



56.8 වෘත්තීය පැහැදිලි කිරීම (Professional Explanation)

වෘත්තීය Frontend Developer කෙනෙකුගේ වගකීම වන්නේ හුදෙක් ලස්සන වෙබ් අඩවියක් සෑදීම පමණක් නොවේ. එය සෙවුම් යන්ත්‍ර වලට කියවිය හැකි (Machine Readable) එකක් විය යුතුය. අපි මෙයට Technical SEO යැයි කියමු. ඔබ ලියන කේතය පිරිසිදු වන තරමට සහ වෙගවත් වන තරමට, එම ව්‍යාපාරයේ අරමුණු ඉටු කර ගැනීමට සේවාදායකයාට පහසු වේ.

56.10 අභ්‍යාස (Exercises)

1. SEO යනු කුමක්ද සහ එය ව්‍යාපාරයකට වැදගත් වන්නේ ඇයි?
2. පහත සඳහන් වැග් වල කාර්යභාරය පැහැදිලි කරන්න:
 - <title>
 - <meta name="description">
 -
3. වෙබ් අඩවියක වෙගය වැඩි කිරීමට කළ හැකි තාක්ෂණික පියවර 2ක් ලියන්න.
4. ඔබ ප්‍රසිද්ධ කළ පෝට්ෆෝලියෝ වෙබ් අඩවිය Lighthouse හරහා පරීක්ෂා කර එහි ලකුණු ප්‍රමාණයන් සටහන් කරගන්න.

මිලහ පරිච්ඡේදයෙන් අපි පරිමාව 5 හි අවසාන කොටස ලෙස, වෘත්තීය Frontend Developer කෙනෙකු ලෙස ඔබේ ගමන ආරම්භ කිරීමට අවශ්‍ය ප්‍රායෝගික ව්‍යාපෘති සහ මගපෙන්වීම් ගැන ඉගෙන ගනිමු.

පනස් හත්වන පරිච්ඡේදය: වෘත්තීය වෙබ් අඩවි ප්‍රසිද්ධ කිරීමේ ව්‍යාපෘතිය සහ විගණනය (Professional Deployment Project and Audit)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

1. මෙතෙක් උගත් සියලුම වෘත්තීය මෙවලම් (Terminal, Git, GitHub, Netlify, SEO) එකට එකතු කර සැබෑ ව්‍යාපෘතියක් නිම කිරීම.
2. ව්‍යාපෘතියක් මුල සිට අගට වෘත්තීය මට්ටමින් සංවිධානය කරන ආකාරය.
3. සැබෑ ලෝකයේ භාවිත වන "Deployment Pipeline" එකක් ක්‍රියාත්මක කිරීම.
4. වෙබ් අඩවියක ගුණාත්මකභාවය මැනීමට "Lighthouse Audit" එකක් සිදු කර වැරදි නිවැරදි කිරීම.
5. වෘත්තීය Frontend Developer කෙනෙකු ලෙස ව්‍යාපෘතියක් අවසන් කරන ආකාරය.

57.1 ව්‍යාපෘති හැඳින්වීම (Project Overview)

පරිමාව 5 තුළදී ඔබ ඉගෙන ගත්තේ වෙබ් අඩවියක් සාදන ආකාරය පමණක් නොව, එය ලෝකයට පෙනෙන සේ ක්‍රමානුකූලව අන්තර්ජාලයට මුදාහරින (Deploy) ආකාරයයි. මෙම පරිච්ඡේදයේදී අපි ඔබ සතු හොඳම වෙබ් අඩවිය (උදා: ඔබගේ Portfolio එක) ගෙන එය සියලුම ප්‍රමිතීන්ට අනුකූලව ප්‍රසිද්ධ කරමු.

මෙම ව්‍යාපෘතියේ ප්‍රධාන අදියර 4කි:

1. සංවිධානය (Organization): ටර්මිනලය සහ ගිට් භාවිතා කර ව්‍යාපෘතිය සූදානම් කිරීම.
2. ප්‍රශස්තකරණය (SEO & Performance): සෙවුම් යන්ත්‍ර වලට සහ වෙබයට අදාළ වෙනස්කම් සිදු කිරීම.
3. ප්‍රසිද්ධ කිරීම (Deployment): GitHub සහ Netlify හරහා වෙබ් අඩවිය සජීවී (Live) කිරීම.
4. විගණනය (Audit): Lighthouse භාවිතා කර ලකුණු 100/100 ලබා ගැනීමට උත්සාහ කිරීම.

57.2 පියවර 1: වෘත්තීය ගොනු ව්‍යුහය (Professional Structure)

පළමුව, ටර්මිනලය (Terminal) භාවිතා කර ඔබේ ව්‍යාපෘති ෆෝල්ඩරය තුළට ගොස් එය ගිට් (Git) මගින් සූදානම් කරන්න.

විධාන (Commands):

```
# ව්‍යාපෘති ෆෝල්ඩරයට යන්න
cd my-professional-site
```

```
# ගිට් ආරම්භ කරන්න
git init
```

```
# ගොනු ව්‍යුහය පරීක්ෂා කරන්න
ls
```

වෘත්තීය ගොනු ව්‍යුහය (ASCII Tree):

```
my-professional-site/
├── index.html      (ප්‍රධාන පිටුව)
├── about.html     (වෙනත් පිටුව)
├── css/
│   └── style.css  (CSS ගොනු)
├── js/
│   └── script.js  (JS ගොනු)
├── images/
│   ├── logo.webp (WebP පින්තූර)
│   └── profile.webp
└── README.md      (ව්‍යාපෘතිය ගැන විස්තරය)
```

57.3 පියවර 2: SEO සහ ප්‍රවේශ්‍යතාව (SEO & Accessibility)

ඔබේ index.html ගොනුවේ <head> කොටස විවෘත කර පහත දෑ නිවැරදිව තිබේදැයි බලන්න.

SEO Tags:

```
<title>Kasun Kalhara | Professional Frontend
Developer</title>
<meta name="description" content="Kasun
Kalhara's professional portfolio showcasing web
development skills in HTML, CSS, and
JavaScript.">
<meta name="keywords" content="Frontend
Developer, Sri Lanka, Web Design, HTML, CSS,
JavaScript">
```

```
<!-- Open Graph Tags (Facebook/WhatsApp ලින්ක්
එක යවන විට ලස්සනට පෙනීමට) -->
```

```
<meta property="og:title" content="Kasun Kalhara
- Portfolio">
```

```
<meta property="og:description" content="Check
out my latest web development projects.">
```

```
<meta property="og:image" content="images/og-preview.jpg">
```

Accessibility (ප්‍රවේශ්‍යතාව): සෑම පින්තූරයකටම අර්ථවත් alt අගයක් ලබා දෙන්න.

```

```

57.4 පියවර 3: ගිට්හබ් වෙත යැවීම (Push to GitHub)

දැන් ටර්මිනලය හරහා ඔබ කළ සියලුම වෙනස්කම් ගිට්හබ් වෙත යවන්න.

1. සියලුම වෙනස්කම් පෝලිමට එක් කරන්න
git add .

2. අර්ථවත් පණිවිඩයක් සමඟ සේව් කරන්න
git commit -m "SEO metadata සහ WebP පින්තූර එක් කරන ලදී"

3. GitHub වෙත Push කරන්න (ඔබ දැනටමත් remote එකතු කර ඇත්නම්)
git push origin main

57.5 පියවර 4: Netlify හරහා විකාශනය කිරීම

1. netlify.com වෙත ගොස් ඔබගේ GitHub ව්‍යාපෘතිය තෝරන්න.
2. "Deploy Site" ක්ලික් කරන්න.
3. වෙබ් අඩවිය Live වූ පසු, එහි URL එක (උදා: kasun-dev.netlify.app) පිටපත් කරගන්න.

57.6 පියවර 5: Lighthouse විගණනය (The Audit)

වෙබ් අඩවිය සජීවී වූ පසු, එහි ගුණාත්මකභාවය පරීක්ෂා කරමු.

1. ඔබගේ ප්‍රසිද්ධ කළ වෙබ් අඩවිය (URL එක) Google Chrome මගින් විවෘත කරන්න.
2. F12 ඔබා Lighthouse ටෑබ් එකට යන්න.
3. "Analyze page load" ක්ලික් කරන්න.

ඔබේ ඉලක්කය (Target): සෑම අංශයකින්ම (Performance, Accessibility, Best Practices, SEO) ලකුණු 90 ට වඩා වැඩි අගයක් ලබා ගැනීම.

57.7 දෝෂ සෙවීම සහ නිවැරදි කිරීම (Debugging the Audit)

Lighthouse ලකුණු අඩු වුවහොත් කළ යුතු දේ:

- Performance අඩු නම්: පින්තූර තවත් කුඩා (Compress) කරන්න. අනවශ්‍ය CSS ඉවත් කරන්න.
- SEO අඩු නම්: <meta> tags හෝ alt tags අමතක වී ඇත්දැයි බලන්න.
- Accessibility අඩු නම්: අකුරු වල පාට සහ පසුබිම් පාට අතර වෙනස (Contrast) මදි විය හැක. නැතිනම් <h1>, <h2> අනුපිළිවෙළ වැරදි විය හැක.

57.8 වෘත්තීය පැහැදිලි කිරීම (Professional Explanation)

සැබෑ මෘදුකාංග සමාගමකදී, ඔබ ලියන කේතය (Code) පාරිභෝගිකයාට ලබා දීමට පෙර මෙවැනි විගණනයන් (Audits) කිහිපයක් සිදු කරනු ලැබේ. අපි මෙයට Quality Assurance (QA) යැයි කියමු. වෙබ් අඩවියක් ලස්සනට තිබීම පමණක් ප්‍රමාණවත් නොවේ; එය තාක්ෂණිකව ද නිවැරදිව තිබීම වෘත්තීය මට්ටමේ Frontend Developer කෙනෙකුගේ ලක්ෂණයකි.

57.9 සාරාංශය (Key Points)

- ව්‍යාපෘතියක් සංවිධානය කිරීමට ටර්මිනලය සහ ගිට් භාවිතා කරයි.
- SEO මගින් Google හි මූලට ඒමට උදව් කරයි.
- Netlify/Vercel හරහා සජීවී (Live) විකාශනය සිදු කරයි.
- Lighthouse මගින් වෙබ් අඩවියේ ප්‍රමිතිය සහතික කරයි.

57.10 පරිමාව 5 - අවසන් අභ්‍යාස (Final Volume 5 Exercises)

1. ඔබේ වෙබ් අඩවියේ පින්තූර WebP වර්ගයට හරවා ප්‍රමාණය (Size) 50% කින් අඩු කරන්න.
2. Lighthouse වාර්තාවේ ලකුණු 100/100 ලබා ගැනීමට උත්සාහ කරන්න. (ලකුණු 100 ම ලැබුණු පසු එහි Screen Shot එකක් සුරැක තබා ගන්න).
3. GitHub හි ඔබගේ ව්‍යාපෘතියේ About කොටසේ Netlify ලින්ක් එක ඇතුළත් කරන්න.

4. මෙතෙක් අප ඉගෙන ගත් සියලුම දෑ ඇතුළත් කර සම්පූර්ණ "වෘත්තීය පෝට්ෆෝලියෝවක්" සජීවීව අන්තර්ජාලයට එක් කරන්න.

සුභ පැතුම්! ඔබ දැන් පරිමාව 5 සාර්ථකව අවසන් කර ඇත. ඔබ සතුව දැන් වෙබ් අඩවි නිර්මාණය කිරීමට පමණක් නොව, ඒවා වෘත්තීය මට්ටමින් කළමනාකරණය කිරීමට සහ ලෝකයටම පෙන්වීමට අවශ්‍ය දැනුම ඇත.

පනස් අටවන පරිච්ඡේදය (විශේෂ ප්‍රෝ-මට්ටමේ): ගිට් අභ්‍යන්තරය, ස්වයංක්‍රීයකරණය සහ උසස් වෙබ් ඉංජිනේරු විද්‍යාව (Git Internals, CI/CD Automation, and Advanced Web Engineering)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම විශේෂිත පරිච්ඡේදය අධ්‍යයනය කිරීමෙන් පසු, ඔබ සාමාන්‍ය වෙබ් අඩවි ප්‍රසිද්ධ කරන්නෙකුගේ මට්ටම ඉක්මවා ගොස් "DevOps-aware Frontend Engineer" (සංවර්ධන සහ මෙහෙයුම් පිළිබඳ අවබෝධයක් ඇති ඉදිරිපෙළ ඉංජිනේරුවෙකු) බවට පත් වනු ඇත:

1. Git Internals (ගිට් අභ්‍යන්තරය): ගිට් පද්ධතිය ඇතුළත දත්ත ගබඩා කරගන්නා ආකාරය (Snapshots vs Diffs) සහ Hash අගයන්හි ක්‍රියාකාරිත්වය අවබෝධ කර ගැනීම.
2. Advanced Git Workflows: වෘත්තීය මට්ටමේ කණ්ඩායම් භාවිතා කරන GitFlow සහ Trunk-based Development ක්‍රමවේදයන් ප්‍රශ්න කිරීම.
3. Rebase vs Merge: කේතයන් එකතු කිරීමේදී භාවිතා කරන උසස් ක්‍රම දෙකෙහි වෙනස සහ ඒවා භාවිතා කළ යුතු අවස්ථා හඳුනා ගැනීම.
4. CI/CD සහ Automation: GitHub Actions භාවිතා කර කේතය ස්වයංක්‍රීයව පරීක්ෂා කිරීම (Testing) සහ ප්‍රසිද්ධ කිරීම (Deployment) සිදු කරන ආකාරය.
5. Advanced SEO & Technical Discovery: Robots.txt, Sitemap.xml සහ JSON-LD වැනි උසස් සෙවුම් යන්ත්‍ර ප්‍රශස්තකරණ මෙවලම් නිර්මාණය කිරීම.
6. Performance Optimization: Critical Rendering Path එක කළමනාකරණය කිරීම සහ

ප්‍රභවයන් කලින් ලබා ගැනීම (Resource Hinting - Preload/Prefetch).

1. ගිට් එන්ජිම ඇතුළත සිදුවන දේ (The Science of Git)

අපි 53 වන පරිච්ඡේදයේදී git commit කිරීමට ඉගෙන ගත්තෙමු. නමුත් ඇත්තටම .git ෆෝල්ඩරය ඇතුළත සිදුවන්නේ කුමක්ද?

1.1 Snapshot (ඡායාරූපය) එදිරිව Diff (වෙනස)

පැරණි සංස්කරණ පාලන පද්ධති (උදා: SVN) ගබඩා කරගන්නේ කේතයේ සිදුවූ "වෙනස" (Diff) පමණි. නමුත් Git ක්‍රියා කරන්නේ Snapshots පදනම මතය. ඔබ කමිට් (Commit) එකක් කරන සෑම වාරයකම, ගිට් විසින් එම මොහොතේ ඔබේ මුළු ව්‍යාපෘතියේම "ඡායාරූපයක්" ලබා ගනී. ගොනුවක් වෙනස් වී නැතිනම්, එය නැවත ගබඩා නොකර පරණ ගොනුවට සබැඳියක් (Link) පමණක් ලබා දෙයි. මෙය ගිට් ඉතා වේගවත් කිරීමට හේතුවයි.

1.2 SHA-1 සහ Content Addressable Storage

ගිට් විසින් සෑම ගොනුවකටම සහ සෑම කමිට් එකකටම අකුරු සහ ඉලක්කම් 40ක අනන්‍ය කේතයක් ලබා දෙයි. මෙයට Hash (හැෂ්) අගය යැයි කියනු ලැබේ.

- උදාහරණ: 5f7a2b3... ගොනුවේ එකම එක තිත්ක් වෙනස් කළද, මෙම හැෂ් අගය සම්පූර්ණයෙන්ම වෙනස් වේ. මේ නිසා ගිට් තුළ දත්ත වංචා කිරීම හෝ දත්ත නැතිවීම කළ නොහැක.

2. උසස් ගිට් ක්‍රමවේද (Professional Git Workflows)

විශාල මෘදුකාංග සමාගමක ඔබ තනිව වැඩ කරන්නේ නැත. සිය ගණනක් සංවර්ධකයන් සමඟ වැඩ කිරීමේදී පහත රටා භාවිතා කරයි.

2.1 GitFlow

මෙහිදී ප්‍රධාන වශයෙන් අතු (Branches) දෙකක් පවතී:

1. Main: සැමවිටම පාරිභෝගිකයාට පෙනෙන, දෝෂ රහිත කේතය.
2. Develop: සංවර්ධකයන් තම අලුත් කොටස් එකතු කරන ස්ථානය.

3. Feature Branches: එක් එක් අලුත් විශේෂාංගය (උදා: Login page) සඳහා සාදන තාවකාලික අතු.

2.2 Rebase එදිරිව Merge

- Merge: අතු දෙකක් එකතු කරන විට අලුතින් "Merge Commit" එකක් සාදයි. මෙය ඉතිහාසය (History) අවුල් සහගත වීමට බලපායි.

- Rebase: ඔබේ වෙනස්කම් වික තාවකාලිකව පසෙක තබා, ප්‍රධාන අත්තට (Main) අලුතින් පැමිණි දේවල් මුලට ගෙන, ඉන්පසු ඔබේ වෙනස්කම් ඊට උඩින් තැබීමයි. මෙයින් ඉතා පිරිසිදු, සරල රේඛීය ඉතිහාසයක් (Linear History) ලැබේ.

ASCII සටහන: Rebase ක්‍රියාවලිය

```
පරණ ඉතිහාසය: A --- B --- C (Main)
                \--- D --- E (Feature)
```

Rebase කළ පසු: A --- B --- C --- D' --- E' (Main + Feature එක පෙළට)

3. CI/CD සහ ස්වයංක්‍රීයකරණය (Automation with GitHub Actions)

ප්‍රෝ-සංවර්ධකයෙකු තම වෙබ් අඩවිය අතින් (Manually) ප්‍රසිද්ධ කරන්නේ නැත. ඒ සඳහා CI/CD (Continuous Integration / Continuous Deployment) පද්ධතියක් සකසයි.

GitHub Actions: යනු ඔබේ GitHub ගබඩාව (Repo) තුළම වැඩසටහන් ස්වයංක්‍රීයව ක්‍රියාත්මක කළ හැකි මෙවලමකි.

- Workflow: ඔබ කේතය Push කළ සැණින් ක්‍රියාත්මක වන නීති මාලාවකි.
- Runner: එම නීති ක්‍රියාත්මක කරන වලාකුළු පරිගණකය (Cloud Server).

ප්‍රායෝගික උදාහරණය: කේතය Push කළ සැණින් එය Build කර Netlify වෙත යැවීමට පෙර එහි වැරදි (Linting) තිබේදැයි පරීක්ෂා කිරීම.

4. උසස් සෙවුම් යන්ත්‍ර ප්‍රශස්තකරණය (Technical SEO Mastery)

අපි 56 වන පරිච්ඡේදයේදී මූලික SEO ඉගෙන ගත්තෙමු. ප්‍රෝ-මට්ටමේදී අපට මෙම ගොනු ද අවශ්‍ය වේ:

4.1 Robots.txt

Google බොට්ස්ලාට ඔබේ වෙබ් අඩවියේ ඇතුළු විය යුතු සහ නොවිය යුතු තැන් පවසන ලිපියයි.

```
User-agent: *
Disallow: /admin/
Sitemap: https://mysite.com/sitemap.xml
```

4.2 Sitemap.xml

ඔබේ වෙබ් අඩවියේ ඇති සියලුම පිටු ලැයිස්තුවක් මෙහි අඩංගු වේ. මෙය Google එකට ඔබේ පිටු ඉක්මනින් සොයා ගැනීමට උදව් වේ.

4.3 JSON-LD (Structured Data)

මෙය පරිමාව 1 හිදී අප මූලිකව සාකච්ඡා කළෙමු. වෘත්තීය මට්ටමේදී, සෑම නිෂ්පාදනයක් හෝ ලිපියක් සඳහාම මෙවැනි කේතයක් අන්තර්ගත කළ යුතුය.

```
<script type="application/ld+json">
{
  "@context": "https://schema.org",
  "@type": "SoftwareApplication",
  "name": "SkyCast Pro",
  "operatingSystem": "Web",
  "applicationCategory": "WeatherApplication"
}
</script>
```

5. උසස් කාර්යක්ෂමතා ඉංජිනේරු විද්‍යාව (Advanced Web Performance)

වෙබ් අඩවියක් ලෝඩ් වන තත්පරයෙන් දහයෙන් පංගුවක් පවා පරිශීලකයා රඳවා ගැනීමට වැදගත් වේ.

5.1 Resource Hinting (සම්පත් කලින් ලබා ගැනීම)

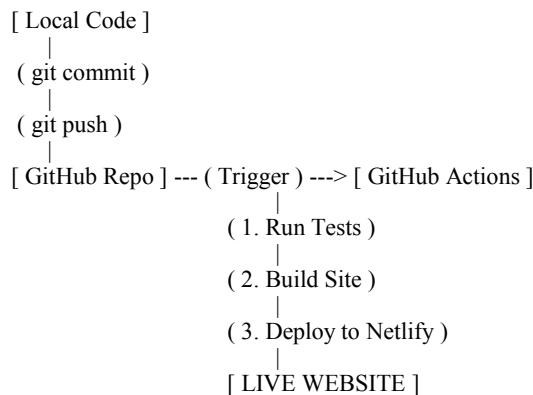
බ්‍රවුසරයට පවසන්න ඊළඟට අවශ්‍ය වන දේ ගැන කලින්ම සූදානම් වන්න කියා.

- <link rel="preload">: ඉතා වැදගත් ගොනු (උදා: ප්‍රධාන CSS හෝ Fonts) වහාම ලබා ගන්න.
- <link rel="preconnect">: වෙනත් සර්වර් එකකට (උදා: Google Fonts) සම්බන්ධතාවය කලින්ම සාදා ගන්න.

5.2 Tree Shaking

ඔබ විශාල ජාවාස්ක්‍රිප්ට් පුස්තකාලයක් භාවිතා කරන විට, එයින් ඔබ භාවිතා නොකරන කොටස් ඉවත් කර අවසාන ගොනුව (Bundle) කුඩා කිරීම මෙලෙස හැඳින්වේ. (Vite වැනි මෙවලම් මෙය ස්වයංක්‍රීයව කරයි).

6. ASCII රූපසටහන: වෘත්තීය ප්‍රසිද්ධ කිරීමේ පියවර (The Pro Pipeline)



7. වෘත්තීය මෙවලම් පිරික්සුම් ලැයිස්තුව (Professional Pro-Tools)

1. **GitLens: VS Code සඳහා ඇති ඉතාමත් බලවත් Extension එකකි. කේතයේ සෑම පේළියක්ම ලිවේ කවුද සහ කවදාද යන්න මෙයින් දැකගත හැක.**
2. **Sentry: ඔබේ වෙබ් අඩවිය ප්‍රසිද්ධ කළ පසු එහි සිදුවන වැරදි (Errors) එසැණින් ඔබට වාර්තා කරන මෘදුකාංගයකි.**
3. **Cloudflare: ඔබේ වෙබ් අඩවියේ වේගය සහ ආරක්ෂාව (DDoS protection) වැඩි කරන ලොව ප්‍රමුඛතම සේවාවයි.**

8. ප්‍රෝ-ච්චමේ අභ්‍යාසය (Advanced Challenge)

අනිගණ්‍යය:

1. ඔබේ පෝට්ෆෝලියෝ ව්‍යාපෘතිය සඳහා sitemap.xml සහ robots.txt ගොනු දෙකක් නිර්මාණය කරන්න.
2. GitHub Actions භාවිතා කර, ඔබ කේතය Push කරන සෑම වාරයකම "Build Successful" යනුවෙන් පණිවිඩයක් ලැබෙන පේ සරල Workflow එකක් සකසන්න.
3. Lighthouse හරහා ඔබේ වෙබ් අඩවියේ "Best Practices" ලකුණු 100 ලබා ගැනීමට උත්සාහ

කරන්න. (එහිදී HTTPS සහ පින්තූරවල Aspect Ratio ගැන සැලකිලිමත් වන්න).

මෙම පරිච්ඡේදය හරහා ඔබ ලබාගත් දැනුම සාමාන්‍ය ඉදිරිපෙළ සංවර්ධනයෙකුට වඩා බෙහෙවින් ඉහළ මට්ටමක පවතී. ඔබ දැන් හුදෙක් වෙබ් අඩවි සාදන්නෙකු නොව, මෘදුකාංග පද්ධතියක් (Software Ecosystem) කළමනාකරණය කළ හැකි Frontend Engineer කෙනෙකි.

පහස් නවවන පරිච්ඡේදය: උසස්
හිටි ශිල්පීය ක්‍රම, මාර්ක්ඩවුන් සහ
ආරක්ෂිත වෙබ් ප්‍රමිතීන්
(Advanced Git Techniques,
Markdown, and Secure Web
Standards)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව අතිශය ගැඹුරු වෘත්තීය දැනුමක් ලැබෙනු ඇත:

1. Git Branching (අතු බෙදීම): එකම ව්‍යාපෘතියක විවිධ විශේෂාංග (Features) එකවර නිර්මාණය කරන ආකාරය.
2. Undoing Mistakes (වැරදි නිවැරදි කිරීම): ගිට පද්ධතිය තුළ සිදුවූ වැරදි reset සහ revert හරහා නිවැරදි කරන ආකාරය.
3. SSH Keys: GitHub සමඟ වඩාත් ආරක්ෂිතව සහ වේගවත්ව සම්බන්ධ වන ආකාරය.
4. Markdown (.md): වෘත්තීය මට්ටමේ README ගොනු ලිවීමට භාවිතා කරන භාෂාව ඉගෙන ගැනීම.
5. Environment Variables (.env): රහස්‍ය දත්ත (Passwords/API Keys) අන්තර්ජාලයට නිරාවරණය නොවී ආරක්ෂා කරන ආකාරය.
6. Web Analytics & Monitoring: ඔබේ වෙබ් අඩවියට පැමිණෙන පිරිස මැන බලන ආකාරය.

59.1 ଶିବି ଅନୁ ଭେଦିତ (Git Branching - The Power of Parallel Work)

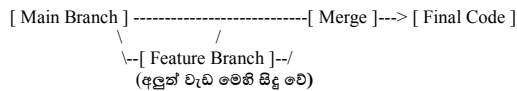
වෘක්ෂය මට්ටමේදී කිසිවිටකත් ප්‍රධාන කේතයට (main branch) සෘජුවම වෙනස්කම් සිදු නොකරයි. අප කරන්නේ අලුත් වැඩක් සඳහා ප්‍රධාන කේතයෙන් වෙනම "අත්තක්" (Branch) සාදා ගැනීමයි.

ඇයි මෙය කරන්නේ? සිතන්න ඔබ වෙබ් අඩවියකට "Login Page" එකක් හදනවා කියා. එය සාදන අතරතුර වෙබ් අඩවියේ ප්‍රධාන කොටසේ දෝෂයක් (Bug) ආවොත්, ඔබට පුළුවන් Login Page එක පැත්තකට දමා වහාම ප්‍රධාන කොටස නිවැරදි කරන්න. මෙය කළ හැක්කේ අතු බෙදීමෙනි.

විධාන (Commands):

- git branch: දැනට ඇති අතු ලැයිස්තුව බැලීමට.
- git checkout -b feature-login: feature-login නමින් අලුත් අත්‍යක් සාදා එයට මාරු වීමට.
- git merge feature-login: අලුත් අත්‍යේ වැඩ අවසන් වූ පසු එය ප්‍රධාන අත්‍යට (main) එකතු කිරීමට.

ASCII රූපසටහන: Branching



59.2 වැරදි නිවැරදි කිරීම (Undoing Mistakes)

ඔබ වැරදි කේතයක් Commit කළහොත් කුමක් කරන්නේද? ගිට් සතුව "කාල යන්ත්‍රයක්" ඇත.

1. git checkout <file_name>: ගොනුවක කළ වෙනස්කම් සේවි කිරීමට පෙර එය තිබූ තත්ත්වයට ගැනීමට.
2. git reset --soft HEAD~1: අවසානයට කළ Commit එක මකා දමා ගොනු ටික නැවත Staging area එකට ගැනීමට.
3. git revert <commit_id>: පරණ Commit එකක් නිසා ඇතිවූ වෙනසක් අවලංගු කිරීමට (මෙය පරණ දත්ත මකා නොදමයි, ඒ වෙනුවට ප්‍රතිවිරුද්ධ වෙනසක් සහිත අලුත් Commit එකක් සාදයි).

59.3 මාර්ක්ඩවුන් සහ රීඩ්මී (Markdown & README.md)

ඔබේ GitHub ප්‍රෝෆයිලය ලස්සන කරන්නේ README.md ගොනුවයි. මෙය ලිවීමට භාවිතා කරන භාෂාව Markdown ලෙස හැඳින්වේ. එය HTML වලට වඩා ඉතා සරල "සලකුණු භාෂාවකි".

මූලික සංකේත:

- # මාතෘකාව : මෙය <h1> වැනිය.
- ## උප මාතෘකාව : මෙය <h2> වැනිය.
- **තද අකුරු** : මෙය Bold අකුරු සඳහාය.
- [ලින්ක් එකේ නම](URL) : ලින්ක් එකක් සෑදීමට.
- - කරුණක් : ලැයිස්තුවක් (List) සෑදීමට.

වෘත්තීය උපදෙස: ඔබේ සෑම ව්‍යාපෘතියකම එම ව්‍යාපෘතිය ගැන විස්තරයක්, ස්ථාපනය කරන ආකාරය සහ ඡායාරූප (Screenshots) සහිත README ගොනුවක් අනිවාර්යයෙන්ම තිබිය යුතුය.

59.4 ආරක්ෂාව සහ රහස්‍යභාවය (Environment Variables - .env)

මෙය ප්‍රෝ-සංවර්ධකයෙකු සහ සාමාන්‍ය අයෙකු වෙන් කරන ඉතාමත් තීරණාත්මක පාඩමකි. ඔබ කාලගුණ API එකක යතුරු (API Key) හෝ Database එකක මුරපදයක් භාවිතා කරන්නේ නම්, එය කිසිවිටකත් ඔබේ සාමාන්‍ය කේතය (JS file) තුළ ලියන්න එපා.

ඔබ එසේ ලියා GitHub වෙත Push කළහොත්, ලෝකයේ ඕනෑම අයෙකුට ඔබේ රහස්‍ය යතුරු සොරාගෙන අසත්‍ය ප්‍රයෝජන ගත හැක.

විසඳුම: .env ගොනුව

1. ව්‍යාපෘති ෆෝල්ඩරයේ .env නමින් ගොනුවක් සාදන්න.
2. එහි රහස් දත්ත මෙලෙස ලියන්න:
WEATHER_API_KEY=12345abcde
3. ඔබගේ .gitignore ගොනුවට .env යන නම ඇතුළත් කරන්න. (එවිට එය GitHub වෙත යන්නේ නැත).
4. Netlify හෝ Vercel හි "Environment Variables" සැකසුම් (Settings) වලට ගොස් මෙම යතුර එහි වෙනම ඇතුළත් කරන්න.

59.5 එස්.එස්.එච්. සම්බන්ධතාවය (SSH Keys for GitHub)

බිබ් GitHub වෙත කේතය යවන සෑම විටම Username සහ Password ගැසීම කරදරකාරී වැඩකි. ඒ සඳහා වෘත්තීය සංවර්ධකයන් භාවිතා කරන්නේ SSH (Secure Shell) ය.

මෙය "අග්‍රලක්ෂ්‍ය සහ යතුරක්" වැනිය.

- ඔබේ පරිගණකයේ රහස්‍ය යතුරක් (Private Key) සාදා ගනී.
- එහි පොදු යතුර (Public Key) GitHub වෙත ලබා දෙයි.
- පසුව ඔබ කේතය යවන විට මුරපද අවශ්‍ය නොවේ, පරිගණක දෙක රහසින් එකිනෙකා හඳුනා ගනී.

59.6 වෙබ් අඩවිය නිරීක්ෂණය (Analytics and Google Search Console)

වෙබ් අඩවිය ප්‍රසිද්ධ කළාට පසු එයට මිනිසුන් එනවාදැයි දැනගත යුතුය.

1. Google Analytics: ඔබේ වෙබ් අඩවියට කී දෙනෙක් පැමිණියාද, ඔවුන් කුමන රටේ අයද යන්න මැන බලයි.
2. Google Search Console: Google සෙවුම් යන්ත්‍රය තුළ ඔබේ වෙබ් අඩවියේ සොබා තත්ත්වය පරීක්ෂා කිරීමට භාවිතා කරයි.
3. Favicon Generator: අපි 57 වන පරිච්ඡේදයේ ඉගෙන ගත් Favicon එක විවිධ දුරකථන (iPhone/Android) වලට ගැලපෙන සේ සැකසීමට realfavicongenerator.net වැනි වෙබ්ලම් භාවිතා කරන්න.

59.7 ASCII රූපසටහන: වෘත්තීය GitHub ගමන

```
[ කේතය ලියයි ] -> [ git add . ] -> [ git commit ]
      |
      ( git checkout -b feature )
      |
      [ අලුත් අත්තේ වැඩ ]
      |
      ( git push origin feature )
      |
      [ GitHub: Pull Request ]
      |
      [ Code Review & Merge ]
      |
      [ LIVE ON NETLIFY ]
```

59.8 පොදු වැරදි (Common Mistakes)

1. විශාල ගොනු Commit කිරීම: පින්තූර හෝ විඩියෝ දහස් ගණනක් ගිට් පද්ධතියට දැමීමෙන් ව්‍යාපෘතිය මන්දගාමී වේ. ඒ සඳහා LFS (Large File Storage) භාවිතා කරන්න.
2. අර්ථයක් නැති Commit Messages: "abc", "fixed", "update" ලෙස ලිවීම ඉතා නරක

පුරුද්දකි. සැමවිටම "Add login validation logic" වැනි පැහැදිලි වාක්‍යයක් ලියන්න.

3. Credential Leak: .env ගොනුව වැරදිමකින් හෝ GitHub වෙත යැවීම. මෙය සිදුවුවහොත් වහාම එම API Key එක අවලංගු කර අලුත් එකක් ලබා ගන්න.

59.9 සාරාංශය (Key Points)

- Branching මගින් එකම ව්‍යාපෘතියේ විවිධ කොටස් ආරක්ෂිතව නිර්මාණය කළ හැක.
- Markdown මගින් වෘත්තීය ලියකියවිලි (Documentation) සකස් කළ හැක.
- .env ගොනුව වෙබ් අඩවියේ රහස්‍යභාවය ආරක්ෂා කරයි.
- git reset සහ revert මගින් වැරදි පාලනය කළ හැක.
- Search Console මගින් Google සමඟ වෙබ් අඩවිය සම්බන්ධ කරයි.

59.10 අභ්‍යාස (Exercises)

1. ඔබේ ව්‍යාපෘතියට README.md ගොනුවක් සාදා එහි මාතෘකා, ලැයිස්තු සහ ඔබගේ GitHub ප්‍රෝෆයිලයට ලින්ක් එකක් ඇතුළත් කරන්න.
2. අලුත් අත්තක් (Branch) සාදා එහි යම් වෙනසක් කර, නැවත ප්‍රධාන අත්තට පැමිණ එය එකතු (Merge) කරන්න.
3. ඔබේ වෙබ් අඩවියේ පින්තූර Lighthouse හරහා පරීක්ෂා කර, ඒවායේ Alt text සහ ප්‍රමාණයන් නිවැරදි දැයි තහවුරු කරන්න.
4. .gitignore ගොනුවක් සාදා එහි node_modules සහ .env යන නම් ඇතුළත් කරන්න.

රියැක්ට් (React.js) - නූතන ඉදිරිපෙළ සංවර්ධනය

හැටවන පරිච්ඡේදය: රියැක්ට් හැඳින්වීම සහ අතරා ඩොම් (Introduction to React and the Virtual DOM)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අධ්‍යයනය කිරීමෙන් පසු ඔබට පහත කරුණු පිළිබඳව අතිශය ගැඹුරු සහ

වෘත්තීය අවබෝධයක් ලැබෙනු ඇත:

1. රියැක්ට් (React) යනු කුමක්ද සහ එහි ඓතිහාසික පසුබිම වටහා ගැනීම.
2. පුස්තකාලයක් (Library) සහ රාමුවක් (Framework) අතර ඇති තාක්ෂණික වෙනස හඳුනා ගැනීම.
3. නියෝගාත්මක (Imperative) සහ ප්‍රකාශනශීලී (Declarative) ක්‍රමලේඛන රටාවන් අතර වෙනස තේරුම් ගැනීම.
4. සාමාන්‍ය ජාවාස්ක්‍රිප්ට් (Vanilla JS) මගින් විශාල මෘදුකාංග සෑදීමේදී ඇතිවන ගැටලු හඳුනා ගැනීම.
5. අතරා ඩොම් (Virtual DOM) සංකල්පය සහ එය වෙබ් අඩවියක වේගය වැඩි කරන ආකාරය විද්‍යාත්මකව තේරුම් ගැනීම.
6. සංරචක (Components) පදනම් කරගත් නිර්මාණකරණය යනු කුමක්දැයි වටහා ගැනීම.
7. තනි පිටුවේ මෘදුකාංග (Single Page Applications - SPA) ක්‍රියා කරන ආකාරය අවබෝධ කර ගැනීම.

60.1 හැඳින්වීම (Introduction)

අපි මෙතෙක් ඉගෙන ගත් HTML, CSS සහ JavaScript භාවිතා කර ඔබට ඕනෑම වෙබ් අඩවියක්

නිර්මාණය කිරීමට දැන් හැකියාව ඇත. නමුත් සැබෑ ලෝකයේදී, උදාහරණයක් ලෙස Facebook, Instagram හෝ Netflix වැනි දිනකට දත්ත මිලියන ගණනක් වෙනස් වන වෙබ් අඩවි නිර්මාණය කිරීමේදී සාමාන්‍ය ජාවාස්ක්‍රිප්ට් (Vanilla JS) පමණක් භාවිතා කිරීම ඉතාමත් අපහසු සහ වෙහෙසකර කාර්යයකි.

වෙබ් අඩවියක් ඉතා විශාල සහ සංකීර්ණ වන විට, එහි ඇති දත්ත පාලනය කිරීමත්, ඒවා නිවැරදිව තීරය මත පෙන්වීමත් (DOM Manipulation) අතිශය අභියෝගාත්මක වේ. මෙම ගැටලුවට ලොව පවතින සාර්ථකම විසඳුම වන්නේ React (රියැක්ට්) තාක්ෂණයයි.

60.2 රියැක්ට් යනු කුමක්ද? (Definition of React)

රියැක්ට් (React) යනු වෙබ් අඩවි වල පරිශීලක අතුරුමුහුණත් (User Interfaces - UI) නිර්මාණය කිරීම සඳහා භාවිතා කරන, නිදහස් සහ විවෘත මූලාශ්‍ර (Open-source) ජාවාස්ක්‍රිප්ට් පුස්තකාලයකි (JavaScript Library).

පරිශීලක අතුරුමුහුණත (User Interface - UI): යනු වෙබ් අඩවියකට පිවිසෙන පුද්ගලයෙකුට පෙනෙන සහ ඔහුට හැසිරවිය හැකි සියලුම කොටස් (බොත්තම්, මෙනු, පින්තූර ආදිය) වේ.

තාක්ෂණික පැහැදිලි කිරීම: රියැක්ට් යනු "Declarative" (ප්‍රකාශනශීලී) සහ "Component-based" (සංරචක මත පදනම් වූ) පුස්තකාලයකි. මෙහි ඇති විශේෂත්වය වන්නේ වෙබ් අඩවියේ පෙනුම කුඩා කැබලි වලට කඩා නිර්මාණය කර, පසුව ඒවා එකිනෙක සම්බන්ධ කර මහා පරිමාණ මෘදුකාංග නිපදවීමට හැකි වීමයි.

60.3 රියැක්ට් හි ඉතිහාසය (History of React)

රියැක්ට් නිර්මාණය වීමට ප්‍රධානතම හේතුව වූයේ Facebook සමාගමට මුහුණ දීමට සිදු වූ තාක්ෂණික ගැටලුවකි.

1. ආරම්භය: 2011 වසරේදී Facebook සමාගමේ මෘදුකාංග ඉංජිනේරුවෙකු වූ ජෝර්ඩන් වෝක් (Jordan Walke) විසින් රියැක්ට් හි මූලික ආකෘතිය නිර්මාණය කරන ලදී.
2. පළමු භාවිතය: මෙය ප්‍රථම වරට Facebook හි "News Feed" කොටස සඳහා භාවිතා කරන ලදී. පසුව Instagram සමාගම විසින් ද මෙය වැළඳ ගන්නා ලදී.
3. ප්‍රසිද්ධ කිරීම: 2013 වසරේ මැයි මාසයේදී Facebook සමාගම විසින් රියැක්ට් ලෝකයටම විවෘත මෘදුකාංගයක් (Open Source) ලෙස මුදා හරින ලදී.

4. වර්තමානය: අද වන විට රියැක්ට් යනු ලොව ජනප්‍රියම ඉදිරිපෙළ (Frontend) තාක්ෂණයයි. මෙය නඩත්තු කරනු ලබන්නේ Meta (පෙර Facebook) සමාගම සහ ලොව පුරා විසිරී සිටින දක්ෂ සංවර්ධකයන් සමූහයක් විසිනි.

60.4 ඇයි රියැක්ට් අවශ්‍ය වන්නේ? (Purpose and Why React?)

සාමාන්‍ය ජාවාස්ක්‍රිප්ට් වලින් අපිට වැඩ කරගත හැකි නම් ඇයි රියැක්ට් ඉගෙන ගන්නේ? මෙයට හේතු කිහිපයක් ඇත.

1. DOM හැසිරවීමේ අපහසුව (The Problem with DOM Manipulation)

අපි 37 වන පරිච්ඡේදයේදී ඉගෙන ගත්තා JavaScript මගින් HTML වෙනස් කරන ආකාරය.

උදාහරණයක් ලෙස පණිවිඩයක් ලැබුණු විට එහි අංකය වෙනස් කිරීමට අපිට HTML වැගය සොයාගෙන (getElementById) එය වෙනස් කිරීමට සිදු වේ. නමුත් Facebook වැනි පිටුවක මෙවැනි වෙනස්කම් දහස් ගණනක් එකවර සිදු වේ. ඒ සැම එකකම අතින් (Manually) වෙනස් කිරීමේදී වැරදි සිදු වීමට ඇති ඉඩ ඉතා වැඩිය.

2. සංකීර්ණත්වය පාලනය කිරීම (Managing Complexity)

විශාල වෙබ් අඩවියක එක තැනක කරන වෙනසක් නිසා තවත් තැනක ඇති දෙයක් වෙනස් විය හැක.

රියැක්ට් මගින් මෙම සංකීර්ණ සම්බන්ධතා ඉතා සරලව කළමනාකරණය කරයි.

3. වේගය (Performance)

බ්‍රවුසරයේ ඇති සැබෑ DOM එක වෙනස් කිරීම ඉතා "මිල අධික" එනම් කාලය ගතවන වැඩකි.

රියැක්ට් විසින් මෙය ඉතාමත් වේගවත් කරන විශේෂ ක්‍රමවේදයක් (Virtual DOM) භාවිතා කරයි.

60.5 නියෝගාත්මක එදිරිව ප්‍රකාශනායිලී ක්‍රමය (Imperative vs Declarative)

මෙය රියැක්ට් තේරුම් ගැනීමට ඇති වැදගත්ම සංකල්පයකි.

- **Imperative (නියෝගාත්මක):** පරිගණකයට වැඩක් කරන ආකාරය පියවරෙන් පියවර පැවසීම. (සාමාන්‍ය ජාවාස්ක්‍රිප්ට් ක්‍රමය).

- **Declarative (ප්‍රකාශනායිලී):** පරිගණකයට වැඩේ කරන ආකාරය නොව, ඔබට අවසානයේදී අවශ්‍ය ප්‍රතිඵලය කුමක්දැයි පමණක් පැවසීම. (රියැක්ට් ක්‍රමය).

සැබෑ ලෝකයේ උපමාවක්: සිතන්න ඔබ ආපනශාලාවකට ගොස් වතුර වීදුරුවක් ඉල්ලනවා කියා.

- **Imperative ක්‍රමය:** "මේසයෙන් නැගිටින්න, කුස්සියට යන්න, වීදුරුවක් ගන්න, කරාමය හරවන්න, වතුර පුරවන්න, වීදුරුව මගේ මේසයට ගෙන එන්න." (ඔබ පියවරෙන් පියවර නියෝග දෙයි).

- **Declarative ක්‍රමය:** "මට වතුර වීදුරුවක් දෙන්න." (ඔබට අවශ්‍ය ප්‍රතිඵලය පමණක් පවසයි. එය කරන්නේ කෙසේද යන්න වෙබ් රේඛා බලා ගනී).

රියැක්ට් වලදී ඔබ කරන්නේ "දත්තය 5 නම්, තිරයේ '5' පෙන්වන්න" කියා පැවසීම පමණි.

දත්තය 6 වූ සැණින් රියැක්ට් විසින්ම තිරය වෙනස් කරයි.

60.6 අතථ්‍ය ඩොම් (The Virtual DOM)

මෙය රියැක්ට් හි ඇති සුවිශේෂීතා තාක්ෂණයයි.

සැබෑ ඩොම් (Real DOM): බ්‍රවුසරය සතු සැබෑ HTML සැකිල්ලයි. මෙය වෙනස් කිරීම මන්දගාමී වේ.

අතථ්‍ය ඩොම් (Virtual DOM): රියැක්ට් විසින් මතකය (Memory) තුළ තබා ගන්නා සැබෑ ඩොම් එකේ සැහැල්ලු පිටපතකි.

එය ක්‍රියා කරන්නේ මෙලෙසයි:

1. වෙබ් අඩවියේ දත්තයක් (State) වෙනස් වූ විට, රියැක්ට් මූලිකවම එම වෙනස කරන්නේ මේ Virtual DOM එකේය.
2. රියැක්ට් විසින් පරණ Virtual DOM එක සහ අලුත් Virtual DOM එක එකිනෙක සසඳා බලයි. මෙයට Diffing යැයි කියනු ලැබේ.
3. මෙම සැසඳීමෙන් පසු, වෙනස් විය යුතුම කොටස පමණක් සොයා ගනී.

4. අවසාන පියවර ලෙස, සැබෑ ඩොම් එකේ (Real DOM) එම විශේෂිත කුඩා කොටස පමණක් යාවත්කාලීන කරයි. මෙයට **Reconciliation** යැයි කියනු ලැබේ.

ASCII රූපසටහන: Virtual DOM ක්‍රියාවලිය

[දත්ත (State) වෙනස් වීම]

```
|
v
```

[අලුත් Virtual DOM එකක් සෑදේ]

```
|
v
```

[පරණ සහ අලුත් සසඳයි (Diffing)]

```
|
v
```

[වෙනස ඇති කොටස පමණක් සොයා ගනී]

```
|
v
```

[සැබෑ DOM එක යාවත්කාලීන කරයි] (Reconciliation)

60.7 සංරචක පදනම් කරගත් නිර්මාණකරණය (Component-Based Architecture)

රියාකට්ටු වලදී අප මුළු වෙබ් අඩවියම කුඩා, ස්වාධීන කැබලි වලට කඩා වෙන් කරමු. මෙම සැමකැබැල්ලක්ම **Component** (සංරචකයක්) ලෙස හැඳින්වේ.

උපමාවක් (Lego Bricks): සිතන්න ඔබ ලෙගෝ (Lego) කැලි වලින් නිවසක් හඳුනවා කියා. එහි ජෙන්නෙල, දොර, සහ වහලය යනු වෙන වෙනම කැලි වේ. රියාකට්ටු වලදී ද වෙබ් අඩවියක ඇති:

- Header (හිස)
- Sidebar (පැති පුවරුව)
- Search Bar (සෙවුම් කොටුව)
- Button (බොත්තම) යනු වෙන වෙනම සංරචක (Components) වේ.

වාසි:

1. නැවත භාවිතය (Reusability): එකම බොත්තම හෝ මෙනුව වෙබ් අඩවියේ ඕනෑම තැනක නැවත නැවත භාවිතා කළ හැක.
2. පහසු නඩත්තුව (Maintainability): එක් කොටසක වැරද්දක් ආ විට මුළු වෙබ් අඩවියම පරීක්ෂා කිරීමට අවශ්‍ය නැත. අදාළ Component එක පමණක් සකස් කිරීම ප්‍රමාණවත්ය.

60.8 තනි පිටුවේ මෘදුකාංග (Single Page Applications - SPA)

සාමාන්‍ය වෙබ් අඩවියක ඔබ "About Us" පිටුවට යන ලින්ක් එකක් ක්ලික් කළ විට, බ්‍රවුසරය සම්පූර්ණයෙන්ම Refresh වී අලුත් පිටුවක් සර්වර් එකෙන් ගෙන එයි. මෙය කාලය ගතවන වැඩකි. නමුත් රියාකට්ටු මගින් සාදන වෙබ් අඩවි වල ඇත්තේ එකම එක HTML පිටුවකි. ඔබ පිටු මාරු කරන විට බ්‍රවුසරය Refresh වන්නේ නැත. ඒ වෙනුවට ජාවාස්ක්‍රිප්ට් මගින් පිටුවේ මාරු විය යුතු කොටස පමණක් ක්ෂණිකව මාරු කරයි. මෙය ජංගම දුරකථන මෘදුකාංගයක් (Mobile App) භාවිතා කරනවා වැනි ඉතාමත් වේගවත් අත්දැකීමකි.

60.9 සැබෑ ලෝකයේ භාවිතය (Real-world Usage)

ලොව දැවැන්තම සමාගම් රියාකට්ටු භාවිතා කරන්නේ එහි ඇති විශ්වාසවන්තභාවය සහ වේගය නිසාය:

- Facebook: මුළු වෙබ් අඩවියම රියාකට්ටු මත පදනම් වේ.
- Instagram: සියලුම UI කොටස් රියාකට්ටු වලින් සාදා ඇත.
- Netflix: ඔවුන්ගේ ඉතාමත් සංකීර්ණ විඩියෝ ලැයිස්තු පාලනය කිරීමට රියාකට්ටු භාවිතා කරයි.
- WhatsApp Web: පණිවිඩ එසැණින් පෙන්වීමට මෙය භාවිතා කරයි.

60.10 වෘත්තීය පැහැදිලි කිරීම (Professional Explanation)

වෘත්තීය Frontend Architect කෙනෙකු ලෙස රියාකට්ටු දෙස බැලීමේදී, එය හුදෙක් UI එකක් සෑදීමේ මෙවලමක් පමණක් නොවේ. එය වෙබ් අඩවියක State (දත්තවල තත්ත්වය) සහ View (අපට පෙනෙන දේ) සැමවිටම එකිනෙකට ගැළපී පවතින බව සහතික කරන එන්ජිමකි. රියාකට්ටු හි ඇති "Unidirectional Data Flow" (දත්ත ගලා යන්නේ එක දිශාවකට පමණි) සංකල්පය නිසා දෝෂ (Bugs) හඳුනා ගැනීම ඉතා පහසු වේ.

60.11 පොදු වැරදි (Common Mistakes)

1. සෘජුවම DOM එක වෙනස් කිරීම: රියැක්ට් භාවිතා කරන අතරතුර

`document.getElementById` වැනි විධානයන් භාවිතා කර HTML වෙනස් කිරීමට උත්සාහ කිරීම. මෙය රියැක්ට් හි වැඩපිළිවෙළ අවුල් කරයි.

2. රියැක්ට් යනු Framework එකක් යැයි සිතීම: රියැක්ට් යනු පුස්තකාලයක් (Library) පමණි. රවුටින් (Routing) හෝ දත්ත ගබඩා කිරීම (State management) වැනි දේවල් සඳහා අපට අමතර පුස්තකාල අවශ්‍ය වේ.

3. පැරණි ජාවාස්ක්‍රිප්ට් දැනුම මදි වීම: ජාවාස්ක්‍රිප්ට් (පරිමා 3 සහ 4) නිසි ලෙස නොදැන රියැක්ට් ඉගෙනීමට උත්සාහ කිරීම.

60.12 සාරාංශය (Key Points)

- රියැක්ට් යනු Facebook මගින් නිර්මාණය කළ UI පුස්තකාලයකි.
- එය ප්‍රකාශනශීලී (Declarative) ක්‍රමයට වැඩ කරයි.
- Virtual DOM තාක්ෂණය නිසා වෙබ් අඩවිය ඉතා වේගවත් වේ.
- මුළු වෙබ් අඩවියම කුඩා Components (සංරචක) වල එකතුවකි.
- SPA සංකල්පය නිසා පිටුව Refresh නොවී වැඩ කරයි.

60.13 අභ්‍යාස (Exercises)

1. රියැක්ට් නිර්මාණය කළ පුද්ගලයා සහ සමාගම කවුද?
2. සැබෑ DOM එකට වඩා Virtual DOM එක වේගවත් වන්නේ කෙසේද? ඔබේ වචනයෙන් පැහැදිලි කරන්න.
3. "Declarative programming" යනු කුමක්ද? සරල උදාහරණයක් දෙන්න.
4. සංරචක (Component) පදනම් කරගත් නිර්මාණකරණයේ ඇති ප්‍රධාන වාසි 2ක් ලියන්න.

60.14 සම්මුඛ පරීක්ෂණ ප්‍රශ්න (Interview Questions)

- ප්‍රශ්නය: රියැක්ට් පුස්තකාලයක් (Library) මිස රාමුවක් (Framework) නොවන්නේ ඇයි?
- පිළිතුර: රාමුවක් (Framework) විසින් මෘදුකාංගයක් සෑදීමට අවශ්‍ය සියලුම නීති සහ

මෙවලම් (Routing, HTTP requests, etc.) ලබා දෙයි. නමුත් රියැක්ට් අවධානය යොමු කරන්නේ පරිශීලක අතුරුමුහුණත (UI) නිර්මාණය කිරීමට පමණි. අනෙකුත් කාර්යයන් සඳහා අපට වෙනත් මෙවලම් භාවිතා කිරීමට නිදහස ඇත.

මිළහ පරිච්ඡේදයෙන් අපි රියැක්ට් කේත ලිවීමට අවශ්‍ය මෘදුකාංග පද්ධතිය (Node.js සහ Vite) ඔබගේ පරිගණකයේ සකසා ගන්නා ආකාරය ඉතාමත් විස්තරාත්මකව ඉගෙන ගනිමු.

හැට එක්වන පරිච්ඡේදය: වැඩබිම් සකස් කිරීම - Node.js සහ Vite (Environment Setup)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අධ්‍යයනය කිරීමෙන් පසු ඔබට පහත කරුණු පිළිබඳව අතිශය ගැඹුරු සහ වෘත්තීය අවබෝධයක් ලැබෙනු ඇත:

1. රියැක්ට් මෘදුකාංගයක් නිර්මාණය කිරීමට අවශ්‍ය මූලික මෙවලම් හඳුනා ගැනීම.
2. Node.js යනු කුමක්ද සහ එය රියැක්ට් සංවර්ධනයට අත්‍යවශ්‍ය වන්නේ ඇයිද යන්න වටහා ගැනීම.
3. NPM (Node Package Manager) මගින් බාහිර පුස්තකාල කළමනාකරණය කරන ආකාරය තේරුම් ගැනීම.
4. Vite (වීට්) යනු කුමක්ද සහ පැරණි ක්‍රම (CRA) වෙනුවට එය භාවිතා කරන හේතු වටහා ගැනීම.
5. ඔබගේ පරිගණකයේ රියැක්ට් වැඩබිම් නිවැරදිව ස්ථාපනය කරන ආකාරය (Step-by-step).
6. රියැක්ට් ව්‍යාපෘතියක ඇති ගොනු ව්‍යුහය (Folder Structure) පිළිබඳව පූර්ණ අවබෝධයක් ලබා ගැනීම.

61.1 හැඳින්වීම: රියැක්ට් සඳහා සූදානම් වීම

අපි මෙතෙක් ලියූ ජාවාස්ක්‍රිප්ට් කේත ක්‍රියාත්මක වූයේ සෘජුවම බ්‍රවුසරය තුළය. නමුත් රියැක්ට් යනු නවීන තාක්ෂණයකි. එය ලිවීමේදී අපට අමතර මෙවලම් රාශියක් අවශ්‍යවේ. අප ලියන රියැක්ට් කේතය (JSX) බ්‍රවුසරයට කෙලින්ම තේරෙන්නේ නැත. එම නිසා අපගේ කේතය බ්‍රවුසරයට තේරෙන භාෂාවට පරිවර්තනය කිරීමට සහ අපගේ වැඩ කටයුතු වේගවත් කිරීමට විශේෂ "පරිසරයක්" (Environment) අපගේ පරිගණකයේ සකසා ගත යුතුය.

61.2 Node.js සහ NPM යනු මොනවාද?

රියැක්ට් සංවර්ධකයෙකුට තිබිය යුතු වැදගත්ම මෙවලම වන්නේ Node.js ය. Node.js (නෝඩ් ජේ.එස්)

ජාවාස්ක්‍රිප්ට් සාමාන්‍යයෙන් ක්‍රියා කරන්නේ බ්‍රවුසරය තුළ පමණි. නමුත් Node.js යනු ජාවාස්ක්‍රිප්ට් බ්‍රවුසරයෙන් පිටත, එනම් ඔබගේ පරිගණකයේ මෙහෙයුම් පද්ධතිය තුළ ක්‍රියාත්මක කිරීමට ඉඩ ලබා දෙන "ක්‍රියාකාරී පරිසරයකි" (Runtime Environment).

- රියැක්ට් වලට මෙය අවශ්‍ය ඇයි? රියැක්ට් ව්‍යාපෘතියක් ගොඩනැගීමට (Build කිරීමට) අවශ්‍ය මෙවලම් සියල්ල ක්‍රියා කරන්නේ Node.js මත පදනම්වය. NPM (Node Package Manager)

ඔබ Node.js ස්ථාපනය කරන විට ඒ සමඟම NPM නමැති මෘදුකාංගය ද ලැබේ. මෙය ලොව විශාලතම මෘදුකාංග ගබඩාව" වැනිය.

- භාවිතය: අපට අපේ වෙබ් අඩවියට අවශ්‍ය අමතර කොටස් (උදා: Animations සඳහා පුස්තකාල, Icons, හෝ රියැක්ට් මෘදුකාංගයම) අන්තර්ජාලයෙන් බාගත කර ගැනීමට NPM භාවිතා කරයි.

- උපමාවක්: ඔබ දුරකථනයට Apps බාගත කර ගැනීමට Play Store භාවිතා කරන්නා සේ, ජාවාස්ක්‍රිප්ට් ව්‍යාපෘති වලට අවශ්‍ය කොටස් බාගත කිරීමට NPM භාවිතා කරයි.

61.3 වීට් (Vite) යනු කුමක්ද?

අතීතයේදී රියැක්ට් ව්‍යාපෘතියක් ආරම්භ කිරීමට create-react-app (CRA) නමැති ක්‍රමය භාවිතා කරන ලදී. නමුත් එය දැන් යල් පැන ගිය සහ ඉතා මන්දගාමී ක්‍රමයකි. අද වන විට ලොව පුරා වෘත්තීය සංවර්ධකයන් භාවිතා කරන්නේ Vite (වීට්) නමැති මෙවලමයි.

- Vite යනු: අතිශය වේගවත් "Build Tool" (ව්‍යාපෘති ගොඩනගන මෙවලමකි).

- වාසි:

1. ව්‍යාපෘතිය ආරම්භ වීමට ගත වන්නේ තත්පරයකටත් අඩු කාලයකි.
2. ඔබ කේතයේ කරන වෙනස්කම් එසැණින් බ්‍රවුසරයේ පෙන්වයි (Hot Module Replacement).

3. නිපදවන අවසාන ගොනුවේ ප්‍රමාණය ඉතා කුඩාය (Optimized bundle).

61.4 පියවරෙන් පියවර ස්ථාපනය (Step-by-Step Installation)

අපි දැන් ඔබගේ පරිගණකය රියැක්ට් සඳහා සූදානම් කරමු.

පියවර 1: Node.js ස්ථාපනය කිරීම

1. ඔබගේ බ්‍රවුසරයෙන් nodejs.org වෙත යන්න.
2. එහි ඇති LTS (Long Term Support) අනුවාදය තෝරා බාගත (Download) කරගන්න. (LTS යනු වඩාත් ස්ථාවර සහ ආරක්ෂිත අනුවාදයයි).
3. බාගත වූ ගොනුව විවෘත කර සාමාන්‍ය මෘදුකාංගයක් ස්ථාපනය කරන ලෙස (Next, Next...) Install කරගන්න.

පියවර 2: ස්ථාපනය සාර්ථක දැයි පරීක්ෂා කිරීම

ටර්මිනලය (Terminal / Command Prompt) විවෘත කර පහත විධානයන් ටයිප් කර බලන්න:

```
node -v
```

```
npm -v
```

ඔබට අංකයක් (උදා: v20.10.0) දිස්වේ නම් ස්ථාපනය සාර්ථකයි.

61.5 ප්‍රථම රියැක්ට් ව්‍යාපෘතිය සෑදීම (Creating First Project)

දැන් අපි Vite භාවිතා කර අපේ මුල්ම රියැක්ට් ව්‍යාපෘතිය සාදමු.

1. ඔබ ව්‍යාපෘති කරන Folder එක වෙත ටර්මිනලය හරහා යන්න (cd භාවිතා කර).
2. පහත විධානය ටයිප් කර Enter කරන්න: `npm create vite@latest my-react-app`
3. දැන් ඔබට විකල්ප කිහිපයක් ලැබෙනු ඇත:

- Select a framework: React (රීතල යතුරු මගින් තෝරා Enter කරන්න).

- Select a variant: JavaScript (හෝ JavaScript + SWC).

4. දැන් ව්‍යාපෘතියේ ෆෝල්ඩරය තුළට පිවිසෙන්න: `cd my-react-app`

5. අවශ්‍ය සියලුම අභ්‍යන්තර කොටස් (Dependencies) බාගත කර ගැනීමට මෙය ටයිප් කරන්න:

```
npm install
```

6. අවසානයේදී ව්‍යාපෘතිය ක්‍රියාත්මක කිරීමට මෙය ටයිප් කරන්න:

```
npm run dev
```

දැන් ඔබට <http://localhost:5173> වැනි ලිපිනයේ එකක් ලැබෙනු ඇත. එය බ්‍රවුසරයේ විවෘත කර බලන්න. රියැක්ට් සාදරයෙන් පිළිගැනීමේ පිටුව ඔබට පෙනෙනු ඇත!

61.6 රියැක්ට් ගොනු ව්‍යුහය තේරුම් ගැනීම (Folder Structure)

VS Code මගින් my-react-app ෆෝල්ඩරය විවෘත කරන්න. එහි ඇති ප්‍රධාන කොටස් මෙසේය:

my-react-app/

```
├─ node_modules/ (NPM මගින් බාගත කළ සියලුම බාහිර පුස්තකාල මෙහි ඇත)
├─ public/ (පින්තූර වැනි පොදු ගොනු මෙහි තැන්පත් කරයි)
├─ src/ (අපගේ සියලුම කේතයන් ලියන වැදගත්ම ෆෝල්ඩරය මෙයයි)
│ └─ assets/ (කේතයට අවශ්‍ය පින්තූර)
│ └─ App.jsx (අපගේ ප්‍රධාන Component එක)
│ └─ main.jsx (මුළු රියැක්ට් මෘදුකාංගයම ආරම්භ වන ස්ථානය)
│ └─ index.css (ප්‍රධාන CSS ගොනුව)
└─ index.html (වෙබ් අඩවියේ ඇති එකම HTML පිටුව)
└─ package.json (ව්‍යාපෘතියේ විස්තර සහ භාවිතා කරන පුස්තකාල ලැයිස්තුව)
└─ vite.config.js (Vite මෙවලමේ සැකසුම්)
```

61.7 ASCII රූපසටහන: රියැක්ට් වැඩබිමේ සැකැස්ම

[ඔබගේ කේතය (JSX)]

v

[Vite Build Tool] <---- (Node.js මත ක්‍රියා කරයි)

v

[HTML] [CSS] [JS] <---- (බ්‍රවුසරයට තේරෙන ගොනු)

v

[වෙබ් බ්‍රවුසරය]

61.8 වෘත්තීය පැහැදිලි කිරීම (Professional Explanation)

සැබෑ මෘදුකාංග සමාගමකදී අප `node_modules` ෆෝල්ඩරය කිසිවිටකත් `GitHub` වෙත යවන්නේ නැත(එය ඉතා විශාල බැවිනි). ඒ වෙනුවට අප කරන්නේ `package.json` ගොනුව පමණක් යැවීමයි. තවත් සංවර්ධකයෙකු එම ව්‍යාපෘතිය ලබා ගත් පසු ඔහු `npm install` කළ සැණින්, `NPM` විසින් එම ලැයිස්තුව කියවා නැවතත් `node_modules` ෆෝල්ඩරය සාදා දෙයි. මෙයට **Dependency Management** යැයි කියනු ලැබේ.

61.9 පොදු වැරදි (Common Mistakes)

1. `npm install` කිරීමට අමතක වීම: ව්‍යාපෘතියක් සාදා හෝ `GitHub` එකෙන් ලබාගෙන කෙලින්ම `npm run dev` කළහොත් "command not found" හෝ වෙනත් error එකක් ලැබේ. සැමවිටම මුලින්ම `npm install` කළ යුතුය.
2. පැරණි `Node.js` අනුවාද: සමහර නවීන රියැක්ට් පුස්තකාල පැරණි `Node.js` වල වැඩ කරන්නේ නැත. සැමවිටම LTS අනුවාදය භාවිතා කරන්න.
3. **Terminal** එක වැරදීම: ව්‍යාපෘතියේ ෆෝල්ඩරය තුළට (`cd` මගින්) නොගොස් විධාන ලබා දීම.

61.10 සාරාංශය (Key Points)

- Node.js යනු ජාවාස්ක්‍රිප්ට් පරිගණකයේ ක්‍රියාත්මක කරන එන්ජිමයි.
- NPM මගින් අවශ්‍ය මෘදුකාංග කොටස් බාගත කර ගනී.
- Vite යනු නවීන සහ වේගවත් රියැක්ට් ව්‍යාපෘති සාදන මෙවලමයි.
- `npm run dev` මගින් ව්‍යාපෘතිය සජීවීව නැරඹිය හැක.
- `src` ෆෝල්ඩරය යනු අපේ කේත ලියන ප්‍රධාන ස්ථානයයි.

61.11 අභ්‍යාස (Exercises)

1. Node.js සහ NPM අතර ඇති සම්බන්ධය සරලව පැහැදිලි කරන්න.
2. ඔබේ පරිගණකයේ Node.js ස්ථාපනය කර `node -v` මගින් එහි අනුවාදය සටහන් කරගන්න.
3. Vite භාවිතා කර `my-first-react-app` නමින් අලුත් ව්‍යාපෘතියක් සාදන්න.
4. `package.json` ගොනුව විවෘත කර එහි `"dependencies"` යටතේ ඇති දෑ මොනවාදැයි බලන්න.

හැට දෙවන පරිච්ඡේදය: JSX - ජාවාස්ක්‍රිප්ට් තුළ HTML ලිවීම (Understanding JSX - JavaScript XML)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අධ්‍යයනය කිරීමෙන් පසු ඔබට පහත කරුණු පිළිබඳව අතිශය ගැඹුරු සහ වෘත්තීය අවබෝධයක් ලැබෙනු ඇත:

1. JSX යනු කුමක්ද සහ රියැක්ට් තුළ එහි ඇති වැදගත්කම වටහා ගැනීම.
2. ජාවාස්ක්‍රිප්ට් තුළ HTML ලිවීමේ වාසි සහ අනිකුත් ක්‍රමවේදයන් අතර වෙනස හඳුනා ගැනීම.
3. JSX කේතයක් බ්‍රවුසරයට තේරෙන සාමාන්‍ය ජාවාස්ක්‍රිප්ට් බවට පරිවර්තනය වන ආකාරය (Babel & Transpilation) තේරුම් ගැනීම.
4. JSX ලිවීමේදී අනුගමනය කළ යුතු අනිවාර්ය නීති රීති (The Rules of JSX) ප්‍රගුණ කිරීම.
5. JSX තුළ ජාවාස්ක්‍රිප්ට් ප්‍රකාශන (Expressions) සහ විචල්‍යයන් භාවිතා කරන ආකාරය.
6. HTML සහ JSX අතර ඇති සියුම් තාක්ෂණික වෙනස්කම් හඳුනා ගැනීම.

62.1 JSX යනු කුමක්ද? (What is JSX?)

JSX යනු JavaScript XML යන්නෙහි කෙටි නමයි. මෙය ජාවාස්ක්‍රිප්ට් භාෂාව සඳහා සාදන ලද "සින්ටැක්ස් දිගුවකි" (Syntax Extension). සරලවම පැවසුවහොත්, JSX මගින් අපට ජාවාස්ක්‍රිප්ට් ගොනුවක් ඇතුළත HTML වැනි කේත ලිවීමට ඉඩ ලබා දෙයි.

අතීතයේදී: වෙබ් සංවර්ධනයේදී "සැකිල්ල" (HTML) සහ "තර්කනය" (JavaScript) වෙන වෙනම තැබිය යුතු බව පැවසුණි (Separation of Concerns). රියැක්ට් සමඟ: රියැක්ට් පවසන්නේ වෙබ් අඩවියක ඇති එක් කොටසක (Component) පෙනුම සහ එහි ක්‍රියාකාරිත්වය එකට තිබීම වඩාත් කාර්යක්ෂම බවයි.

සැබෑ ලෝකයේ උපමාව: සිතන්න ඔබ පාන් සැන්ඩ්විච් එකක් සාදනවා කියා.

- පාන් යනු HTML (සැකිල්ල) ය.
- සැන්ඩ්විච් එක ඇතුළත ඇති දෑ ජාවාස්ක්‍රිප්ට් (තර්කනය) ය.
- JSX යනු එම දෙකම එකට එකතු වී සැකසුණු සම්පූර්ණ "සැන්ඩ්විච්" එකයි. ඔබට වෙන වෙනම පාන් කැලී සහ ඇතුළත දෑ කළමනාකරණය කරනවාට වඩා, සැන්ඩ්විච් එක ලෙසම කළමනාකරණය කිරීම පහසුය.

62.2 JSX ක්‍රියා කරන්නේ කෙසේද? (Behind the Scenes)

මතක තබා ගන්න, වෙබ් බ්‍රවුසරයකට (Chrome හෝ Firefox) JSX කේතය කෙලින්ම තේරෙන්නේ නැත. බ්‍රවුසරයට තේරෙන්නේ සාමාන්‍ය ජාවාස්ක්‍රිප්ට් (Vanilla JS) පමණි.

එබැවින් අප ලියන JSX කේතය බ්‍රවුසරය වෙත යැවීමට පෙර, එය සාමාන්‍ය ජාවාස්ක්‍රිප්ට් බවට "පරිවර්තනය" කළ යුතුය. මෙම ක්‍රියාවලිය Transpilation ලෙස හඳුන්වයි. මේ සඳහා Babel නමැති මෘදුකාංගය භාවිතා කරයි. (අප පෙර පරිච්ඡේදයේදී සැකසූ Vite මගින් මෙම කාර්යය පසුබිමින් සිදු කරයි).

ASCII රූපසටහන: JSX පරිවර්තනය

[ඔබ ලියන JSX]

```
const element = <h1>Hello</h1>;
```

v

[Babel / Vite පරිවර්තකය]

v

[ඔවුසරයට තේරෙන JS]

```
const element = React.createElement('h1', null, 'Hello');
```

62.3 JSX ලිවීමේ රන් නීති (The Rules of JSX)

JSX ලිවීමේදී ඔබ අනිවාර්යයෙන්ම පිළිපැදිය යුතු නීති 4ක් ඇත. මේවා වැරදුණහොත් ඔබේ රියැක්ට් මෘදුකාංගය ක්‍රියා විරහිත වේ.

නීතිය 1: එකම පියා (Single Parent Element)

ශ්‍රිතයකින් (Component) HTML කේතයක් ආපසු ලබා දෙන (Return) විට, එය එක් ප්‍රධාන මූලිකාංගයක් ඇතුළත තිබිය යුතුය. වැග් දෙකක් වෙන වෙනම තිබිය නොහැක.

- වැරදියි:

```
return (
  <h1>මාතෘකාව</h1>
  <p>පේදය</p>
);
```

- නිවැරදියි:

```
return (
  <div>
    <h1>මාතෘකාව</h1>
    <p>පේදය</p>
  </div>
);
```

ප්‍රෝ රහස: ඔබට අනවශ්‍ය ලෙස <div> වැග් එකතු කිරීමට අවශ්‍ය නැතිනම්, රියැක්ට් හි ඇති Fragment (<> </>) භාවිතා කළ හැක.

නීතිය 2: සෑම වැගයක්ම වැසිය යුතුය (Close All Tags)

සාමාන්‍ය HTML වලදී
 හෝ වැග් වැග් වැසීම අනිවාර්ය නැත. නමුත් JSX වලදී සෑම වැගයක්ම අනිවාර්යයෙන්ම වැසිය යුතුය.

- උදාහරණ:
, , <input type="text" />.

නීතිය 3: camelCase ගුණාංග (camelCase Attributes)

JSX අවසානයේදී ජාවාස්ක්‍රිප්ට් බවට පත්වන බැවින්, HTML ගුණාංග ලිවිය යුත්තේ camelCase ක්‍රමයටය.

- onclick වෙනුවට onClick
- onchange වෙනුවට onChange
- tabindex වෙනුවට tabIndex

නීතිය 4: වෙන්වූ වචන (Reserved Words)

ජාවාස්ක්‍රිප්ට් වල class සහ for යන වචන ඒ වන විටත් වෙනත් කාර්යයන් සඳහා වෙන් කර ඇති බැවින් JSX හිදී ඒවා වෙනස් වේ:

- class වෙනුවට className භාවිතා කළ යුතුය.
- <label> සඳහා භාවිතා කරන for වෙනුවට htmlFor භාවිතා කළ යුතුය.

62.4 JSX තුළ ජාවාස්ක්‍රිප්ට් භාවිතා කිරීම (JS Expressions)

JSX හි ඇති බලවත්ම ලක්ෂණය මෙයයි. ඔබට HTML කේතය ඇතුළත ඕනෑම ජාවාස්ක්‍රිප්ට් විචල්‍යයක් හෝ ගණිතමය ගණනය කිරීමක් සිදු කළ හැක. ඒ සඳහා ඔබ කළ යුත්තේ එම කේතය කරලි වරහන් { } ඇතුළත ලිවීමයි.

උදාහරණය:

```
function Welcome() {

  const name = "කසුන්";

  const age = 20;

  return (

    <div>

      <h1>ආයුබෝවන් {name}!</h1>

      <p>ලබන වසරේ ඔබේ වයස {age + 1} කි.</p>

    </div>

  );

}
```

62.5 HTML සහ JSX අතර වෙනස (Comparison Table)

ලක්ෂණය	සාමාන්‍ය HTML	React JSX
:	:	:
CSS Class	<code>`class="box"`</code>	<code>`className="box"`</code>
Inline Style	<code>`style="color: red"`</code>	<code>`style={{ color: 'red' }}`</code>
Scripting	HTML තුළ JS ලිවිය නොහැක	<code>`{}`</code> මගින් JS ලිවිය හැක
Self-closing	<code>`<img src="...">`</code>	<code>`<img src="..." />`</code>
JavaScript Event	<code>`onclick="run()"`</code>	<code>`onClick={run}`</code>

62.6 වෘත්තීය කේත උදාහරණය සහ විග්‍රහය

අපි දැන් Vite හරහා සකසූ App.jsx ගොනුව තුළ මෙවැනි කේතයක් ලියමු.

```
import React from 'react';

function App() {

  const user = {

    name: 'නිමල්',

    role: 'Frontend Developer',

    imgUrl: 'https://via.placeholder.com/150'

  };

  const statusStyle = {

    color: 'green',

    fontWeight: 'bold'

  };

  return (

    <div className="container">

      <h1 id="main-heading">පරිශීලක තොරතුරු</h1>

      <img src={user.imgUrl} alt={user.name} />

      <h2>නම: {user.name}</h2>

      <p>තනතුර: <span style={statusStyle}>{user.role}</span></p>

      <button onClick={() => alert('වැඩි විස්තර සොයමින්...')}>

        විස්තර බලන්න

      </button>

    </div>

  );
```

}

export default App;

පේළියෙන් පේළියට විග්‍රහය:

1. src={user.imgUrl}: පින්තූරයේ ලිපිනය (Source) ලබා ගැනීමට වස්තුවක (Object) දත්තයක් { } හරහා සම්බන්ධ කර ඇත.
2. style={statusStyle}: Inline CSS ලබා දීමේදී ජාවාස්ක්‍රිප්ට් Object එකක් භාවිතා කර ඇත.
3. onClick={() => ...}: බොත්තමක් ක්ලික් කළ විට සිදුවන ක්‍රියාව (Event) සෘජුවම ශ්‍රිතයක් ලෙස ලබා දී ඇත.

62.7 වෘත්තීය උපදෙස් (Professional Tips)

- පිරිසිදු බව (Cleanliness): JSX තුළ ඉතා දිගු ජාවාස්ක්‍රිප්ට් තර්කනයන් ලිවීමෙන් වළකින්න. එවැනි දේ return එකට ඉහළින් විවලායක ගබඩා කර පසුව { } හරහා පෙන්වන්න.
- Fragment භාවිතය: නිතරම අනවශ්‍ය <div> භාවිතා නොකර <> ... </> (Fragments) භාවිතා කරන්න. එය වෙබ් අඩවියේ වේගයට සහ DOM එකේ පිරිසිදු බවට උදව් වේ.
- Comments: JSX තුළ සටහන් ලිවීමට { /* මෙය සටහනකි */ } යන රටාව භාවිතා කළ යුතුය.

62.8 පොදු වැරදි (Common Mistakes)

1. className වෙනුවට class ලිවීම: මෙය ඉතාමත් පොදු වැරද්දකි. බ්‍රවුසරයේ Console එකේ Error එකක් මගින් මෙය පෙන්වා දෙනු ඇත.
2. වැග් එකක් වැසීමට අමතක වීම: <input> වැනි වැග් අගට / යොදා නොවැසීම නිසා මුළු ව්‍යාපෘතියම වැඩ නොකර පවතිනු ඇත.
3. Return වරහන්: return යන වචනයට පසුව අලුත් පේළියකින් JSX ආරම්භ කරන්නේ නම්, එය අනිවාර්යයෙන්ම සාමාන්‍ය වරහන් () ඇතුළත තිබිය යුතුය.

62.9 සාරාංශය (Key Points)

- JSX යනු ජාවාස්ක්‍රිප්ට් තුළ HTML වැනි කේත ලිවීමට භාවිතා කරන දිගුවකි.
- බ්‍රවුසරයට යැවීමට පෙර Babel මගින් JSX සාමාන්‍ය ජාවාස්ක්‍රිප්ට් බවට පරිවර්තනය කරයි.

- සෑම JSX කොටසකටම එකම "පියා" (Parent Element) සිටිය යුතුය.
- ජාවාස්ක්‍රිප්ට් භාවිතා කිරීමට කරලි වරහන් { } යොදා ගනී.
- HTML හි class වෙනුවට className භාවිතා කළ යුතුය.

62.10 අභ්‍යාස (Exercises)

1. පහත HTML කේතය නිවැරදි JSX කේතයක් බවට පරිවර්තනය කරන්න:

```
<div class="user">

  <h1 id="title">පරිශීලකයා</h1>

  <input type="text" placeholder="නම">

  <br>

  <label for="name">නම ඇතුළත්
  කරන්න</label>

</div>
```

2. ඔබේ ප්‍රියතම ආහාර 3ක නම් අරාචක (Array) ගබඩා කර, ඒවා JSX තුළ { } භාවිතා කර පෙන්වීමට උත්සාහ කරන්න.
3. JSX තුළ { } භාවිතා කළ නොහැකි අවස්ථාවක් පවසන්න. (ඉභිය: if ප්‍රකාශන හෝ for loops සෘජුවම ලිවිය හැකිද?).

62.11 සම්මුඛ පරීක්ෂණ ප්‍රශ්න (Interview Questions)

- ප්‍රශ්නය: JSX යනු ඔවුසරයට තේරෙන භාෂාවක්ද?
- පිළිතුර: නැත. JSX යනු සින්ටැක්ස් දිගුවක් පමණි. එය ඔවුසරය කියවීමට පෙර Babel වැනි Transpiler එකක් මගින් React.createElement() වැනි සාමාන්‍ය ජාවාස්ක්‍රිප්ට් විධානයන් බවට පත් කළ යුතුය.

මිලිභ පරිච්ඡේදයෙන් අපි රියැක්ට් හි අත්තිවාරම වන "සංරචක" (React Components) නිර්මාණය කරන ආකාරය ඉතාමත් විස්තරාත්මකව ඉගෙන ගනිමු.

හැට තුන්වන පරිච්ඡේදය: සංරචක - රියැක්ට් හි ගඩොල් සහ සිමෙන්ති (React Components)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අධ්‍යයනය කිරීමෙන් පසු ඔබට පහත කරුණු පිළිබඳව අතිශය ගැඹුරු සහ වෘත්තීය අවබෝධයක් ලැබෙනු ඇත:

1. සංරචක (Component) යනු කුමක්ද සහ රියැක්ට් හි එහි ඇති මූලික වැදගත්කම වටහා ගැනීම.
2. ක්‍රියාකාරී සංරචක (Functional Components) සහ පන්ති සංරචක (Class Components) අතර වෙනස හඳුනා ගැනීම.
3. සංරචකයක් නිර්මාණය කිරීමේදී අනුගමනය කළ යුතු අනිවාර්ය නාමකරණ නීති (Naming Conventions).
4. සංරචක එකක් ඇතුළත තවත් එකක් තැබීම (Nesting Components) සහ ව්‍යුහගත කිරීම.
5. සංරචක අපනයනය (Export) සහ ආනයනය (Import) කරන වෘත්තීය ක්‍රමවේද.
6. සංරචක මත පදනම් වූ නිර්මාණකරණයේ (Component-Based Architecture) සැබෑ වාසි අත්විඳීම.

63.1 හැඳින්වීම: සංරචකයක් යනු කුමක්ද? (What is a Component?)

අපි මෙතෙක් ඉගෙන ගත් සාමාන්‍ය වෙබ් අඩවිවල නිවුණේ ඉතා දිගු HTML ලිපිගොනු (Files) ය. නමුත් රියැක්ට් වලදී අප වෙබ් අඩවියක් දෙස බලන්නේ සම්පූර්ණ පිටුවක් ලෙස නොව, ස්වාධීන කැබලි (Independent Pieces) සමූහයක එකතුවක් ලෙසය. මෙම සෑම කුඩා කැබැල්ලක්ම Component (සංරචකයක්) ලෙස හැඳින්වේ.

නිල අර්ථ දැක්වීම: සංරචකයක් යනු තමන්ගේම පෙනුමක් (UI) සහ තර්කනයක් (Logic) සහිත, වෙබ් අඩවියේ ඕනෑම තැනක නැවත නැවත භාවිතා කළ හැකි (Reusable) ස්වාධීන කේත කොටසකි.

සැබෑ ලෝකයේ උපමාව (මෝටර් රථයක්): මෝටර් රථයක් ගැන සිතන්න. එය එකම යකඩ ගොඩක් නොව කොටස් රාශියක එකතුවකි:

- එන්ජිම (Engine Component)
- රෝදය (Wheel Component)

- ආසනය (Seat Component) රෝදය සෑදූ පසු එය කාර් එකේ ඕනෑම පැත්තකට සවි කළ හැක. රියැක්ට් වලදී ද අප කරන්නේ වෙබ් අඩවියේ ඇති බොත්තම්, මෙනු සහ පින්තූර මෙලෙස "රෝද" මෙන් වෙන වෙනම සාදා පසුව ඒවා එකට අමුණා වෙබ් අඩවිය නිර්මාණය කිරීමයි.

63.2 සංරචක වර්ග දෙක (Types of Components)

රියැක්ට් ඉතිහාසය තුළ සංරචක වර්ග දෙකක් පැවතුණි:

1. **Functional Components** (ක්‍රියාකාරී සංරචක): මේවා සාමාන්‍ය ජාවාස්ක්‍රිප්ට් ශ්‍රිත (Functions) වේ. මේවා සරලයි, වෙනවත්යි, සහ කියවීමට පහසුයි. නවීන රියැක්ට් ලෝකයේ 99% ක්ම භාවිතා කරන්නේ මේවාය.
2. **Class Components** (පන්ති සංරචක): මේවා ජාවාස්ක්‍රිප්ට් පන්ති (Classes) මත පදනම් වේ. මේවා පැරණි රියැක්ට් සංස්කරණවල භාවිතා වූ අතර වර්තමානයේ මේවා භාවිතය අත්හැර දමා ඇත.

අපගේ පාඨමාලාව පුරාම අප ඉගෙන ගන්නේ වෘත්තීය ප්‍රමිතිය වන Functional Components පිළිබඳවයි.

63.3 සංරචකයක් නිර්මාණය කිරීමේ නීති (Rules of Components)

සංරචකයක් ලිවීමේදී ඔබ අනිවාර්යයෙන්ම පිළිපැදිය යුතු එක් විශේෂ නීතියක් ඇත:

- නම සැමවිටම කැපිටල් (Capital) අකුරකින් ආරම්භ විය යුතුය.
- නිවැරදි: `function MyButton() { ... }`
- වැරදි: `function myButton() { ... }` (මෙය බ්‍රවුසරය විසින් සාමාන්‍ය HTML ටැගයක් ලෙස වරදවා වටහා ගනී).

63.4 ප්‍රථම සංරචකය නිර්මාණය කිරීම (Step-by-Step)

අපි දැන් Header.jsx නමින් අලුත් සංරචකයක් සාදා බලමු.

පියවර 1: අලුත් ගොනුවක් සෑදීම ඔබේ Vite ව්‍යාපෘතියේ src ෆෝල්ඩරය තුළ Header.jsx නමින් ගොනුවක් සාදන්න. (රියැක්ට් ගොනු සඳහා .jsx අගය භාවිතා කිරීම වෘත්තීය ක්‍රමයයි).

පියවර 2: කේතය ලිවීම

// Header.jsx

```
function Header() {  
  
  return (  
  
    <header style={{ backgroundColor: '#282c34', color: 'white', padding: '20px' }}>
```

```
      <h1>SkyCast කාලගුණ පද්ධතිය</h1>
```

```
      <p>අලුත්ම තොරතුරු එසැණින්</p>
```

```
    </header>
```

```
  );
```

```
}
```

`export default Header;` // මෙය වෙනත් ගොනු වලට භාවිතා කිරීමට ඉඩ ලබා දීම

63.5 සංරචක අපනයනය සහ ආනයනය (Export & Import)

සංරචකයක් එක් ගොනුවක සාදා තවත් ගොනුවක භාවිතා කිරීමට නම්, අපනයනය (Export) සහ ආනයනය (Import) යන සංකල්පය භාවිතා කළ යුතුය.

Export (අපනයනය)

පෙර උදාහරණයේ `export default Header;` මගින් පවසන්නේ: "මෙම ගොනුවේ ඇති ප්‍රධානම දේ Header ශ්‍රිතයයි, ඕනෑම කෙනෙකුට මෙය රැගෙන යා හැක" යන්නයි.

Import (ආනයනය)

දැන් අපට මෙම Header එක App.jsx තුළ පෙන්වීමට අවශ්‍ය නම්:

App.jsx

```
import Header from './Header'; // Header එක ගෙන්වා ගැනීම
```

```
function App() {
```

```
  return (
```



```

<div>

  <Header /> {/* මෙන්න Header එක භාවිතා
  කළ ආකාරය */}

  <main>

    <p>වෙබ් අඩවියේ ප්‍රධාන අන්තර්ගතය
    මෙතැනට එයි.</p>

  </main>

</div>

);
}

```

export default App;

63.6 සංරචක ව්‍යුහය (Component Tree)

වෙබ් අඩවියක් වැඩෙන විට සංරචක එකක් ඇතුළත තවත් බොහෝ සංරචක තිබිය හැක.

ASCII රූපසටහන: Component Tree

[App Component] (ප්‍රධාන පියා)

/ | \

[Header] [Main] [Footer] (දරුවන්)

|

[WeatherCard] (මුණුබුරා)

මෙම ක්‍රමය නිසා මුළු වෙබ් අඩවියම ඉතාමත් පිළිවෙළකට කළමනාකරණය කළ හැක.

63.7 වෘත්තීය පැහැදිලි කිරීම (Professional Explanation)

සැබෑ මෘදුකාංග ඉංජිනේරු ක්ෂේත්‍රයේදී අප සංරචක වර්ග දෙකකට බෙදා වෙන් කරමු:

1. **Presentational Components:** මේවායේ කරන්නේ පෙනුම (UI) පමණක් පෙන්වීමයි. (උදා: Button, Navbar).

2. **Container Components:** මේවායේ කරන්නේ දත්ත ලබා ගැනීම සහ අනෙකුත් සංරචක පාලනය කිරීමයි. මෙසේ වෙන් කිරීමෙන් කේතය පරීක්ෂා කිරීම (Testing) සහ නඩත්තු කිරීම (Maintenance) ඉතා පහසු වේ.

63.8 පොදු වැරදි (Common Mistakes)

1. නම සිම්පල් (Small) අකුරින් ආරම්භ කිරීම: function header() ලෙස ලිවීම නිසා රියැක්ට එය හඳුනා නොගනී. සැමවිටම Header ලෙස ලියන්න.
2. Return කිරීමට අමතක වීම: ශ්‍රිතය ඇතුළත return විධානය නොලියුවහොත් තිරය මත කිසිවක් දිස් නොවේ.
3. Default Export පටලවා ගැනීම: export default කළ සංරචකයක් ආනයනය කරන විට වරහන් { } භාවිතා නොකළ යුතුය.

63.9 පිරිසිදු කේතනය සඳහා හොඳම පුරුදු (Best Practices)

- එක් ගොනුවකට එක් සංරචකයක්: එක් .jsx ගොනුවක් තුළ ප්‍රධාන වශයෙන් එක් Component එකක් පමණක් තබන්න.
- ගොනු නාමය සහ සංරචක නාමය සමාන කිරීම: සංරචකයේ නම Navbar නම්, ගොනුවේ නම ද Navbar.jsx ලෙස තැබීම වෘත්තීය පුරුද්දකි.
- Arrow Functions භාවිතය: නවීන රියැක්ට් වලදී සංරචක ලිවීමට Arrow Functions භාවිතා කිරීම ඉතා ජනප්‍රියයි.

```

const Navbar = () => {

  return <nav>මෙතුව</nav>;

};

```

63.10 අභ්‍යාස (Exercises)

1. ඔබේ ව්‍යාපෘතියේ Footer.jsx නමින් අලුත් සංරචකයක් සාදා එහි ඔබේ නම සහ හිමිකම් (Copyright) පණිවිඩයක් ඇතුළත් කරන්න. එය App.jsx තුළට ආනයනය (Import) කර පෙන්වන්න.
2. පහත කේතයේ ඇති වැරදි 3ක් සොයා නිවැරදි කරන්න:

```

function mycomponent() {

  <div>

```

```

<h1>Hello</h1>

<p>World</p>

</div>

}

```

3. සංරචකයක් (Component) සහ සාමාන්‍ය HTML ටැගයක් අතර ඇති වෙනස ඔබේ වචනයෙන් පැහැදිලි කරන්න.

63.11 කුඩා ව්‍යාපෘතිය (Mini Project)

ව්‍යාපෘතිය: සරල වෙබ් අඩවි සැකිල්ල (Simple Layout)

1. පහත සංරචක 4 වෙන වෙනම ගොනු ලෙස සාදන්න:

- Navbar.jsx
- Sidebar.jsx
- Content.jsx
- Footer.jsx

2. මේ සියල්ල App.jsx තුළට ගෙන්වා ගෙන ලස්සන සැකැස්මක් නිර්මාණය කරන්න.

3. එක් එක් සංරචකයට හඳුනා ගැනීමට විවිධ පසුබිම් වර්ණ CSS මගින් ලබා දෙන්න.

මිලිභ පරිච්ඡේදයෙන් අපි සංරචක අතර දත්ත හුවමාරු කරන ආකාරය හෙවත් ඉතාමත් වැදගත්

"ප්‍රොප්ස්" (Props) ගැන ඉතාමත් විස්තරාත්මකව ඉගෙන ගනිමු.

හැට හතරවන පරිච්ඡේදය: ප්‍රොප්ස් - සංරචක අතර දත්ත හුවමාරුව (Props - Passing Data to Components)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අධ්‍යයනය කිරීමෙන් පසු ඔබට පහත කරුණු පිළිබඳව අතිශය ගැඹුරු සහ වෘත්තීය අවබෝධයක් ලැබෙනු ඇත:

1. Props (ප්‍රොප්ස්) යනු කුමක්ද සහ රියැක්ට් හි දත්ත ගලා යන ආකාරය වටහා ගැනීම.

2. දෙමාපිය සංරචකයක (Parent Component) සිට දරු සංරචකයකට (Child Component) දත්ත යවන ආකාරය.

3. ප්‍රොප්ස් යනු "කියවීමට පමණක් හැකි" (Read-only / Immutable) දත්ත බව තේරුම් ගැනීම.

4. Destructuring තාක්ෂණය භාවිතා කර පිරිසිදුව ප්‍රොප්ස් ලබා ගන්නා ආකාරය.

5. සංරචකයක් තුළට ඕනෑම දත්ත වර්ගයක් (Strings, Numbers, Arrays, Objects, Functions) ප්‍රොප්ස් ලෙස යවන ආකාරය.

6. සංරචක නැවත නැවත භාවිතා කිරීමේදී (Reusability) ප්‍රොප්ස් වල ඇති ප්‍රබල දායකත්වය අත්විඳීම.

64.1 හැඳින්වීම: ප්‍රොප්ස් යනු මොනවාද? (What are Props?)

පෙර පරිච්ඡේදයේදී අපි ඉගෙන ගත්තා සංරචක (Components) යනු වෙබ් අඩවියක ඇති කුඩා කැබලි බව. නමුත් සිතන්න ඔබ "බොත්තමක්" (Button) සඳහා සංරචකයක් සෑදුවා කියා. එම බොත්තම එක් තැනක "Login" ලෙසත්, තවත් තැනක "Signup" ලෙසත් දිස්විය යුතුය. එකම සංරචකය භාවිතා කරමින් විවිධ දත්ත පෙන්වන්නේ කෙසේද? මෙන්න මෙතැනදී අපට Props උදව් වේ.

Props යනු Properties (ගුණාංග) යන්නෙහි කෙටි නමයි. මෙය දෙමාපිය සංරචකයක සිට දරු සංරචකයකට තොරතුරු යැවීමට භාවිතා කරන ක්‍රමයයි.

සැබෑ ලෝකයේ උපමාව (තැපැල් පාර්සලයක්): ඔබේ අම්මා (Parent) ඔබට (Child) පාර්සලයක් (Props) ලබා දෙනවා යැයි සිතන්න.

- අම්මා තීරණය කරනවා පාර්සලය ඇතුළත තිබිය යුත්තේ කුමක්ද කියා.
- ඔබට කළ හැක්කේ එම පාර්සලය ලබාගෙන එහි ඇති දේ භාවිතා කිරීම පමණි.
- ඔබට එම පාර්සලය ඇතුළත ඇති දේ වෙනස් කිරීමට බලයක් නැත (එය අම්මා එවන ලද දෙයකි). රියැක්ට් වලදී ද පියා විසින් දරුවාට ලබා දෙන මෙම දත්ත "ප්‍රොප්ස්" ලෙස හැඳින්වේ.

64.2 ප්‍රොප්ස් භාවිතා කරන ආකාරය (Step-by-Step)

ප්‍රොප්ස් ක්‍රියාවලිය පියවර දෙකකින් සිදු වේ.

පියවර 1: දත්ත යැවීම (Passing Props)

දෙමාපිය සංරචකය තුළදී, HTML Attribute එකක් ලියන ආකාරයටම දත්තය ඇතුළත් කරයි.

```
// App.jsx (පියා)
```

```
import Greet from './Greeting';
```

```
function App() {
```

```
  return (
    <div>
      {/* name සහ age යනු අප යවන Props වේ */}
      <Greet name="කසුන්" age="16" />
      <Greet name="නිමල්" age="20" />
    </div>
  );
}
```

පියවර 2: දත්ත ලබා ගැනීම (Receiving Props)

දරු සංරචකයේ ශ්‍රිතය (Function) වෙත පළමු පරාමිතිය (Parameter) ලෙස මෙම සියලුම දත්ත "වස්තුවක්" (Object) ආකාරයෙන් ලැබේ.

```
// Greeting.jsx (දරුවා)
```

```
function Greeting(props) {
```

```
  // props යනු { name: "කසුන්", age: "16" } වැනි Object එකකි

  return (
    <div>
      <h2>ආයුබෝවන් {props.name}!</h2>
      <p>ඔබේ වයස අවුරුදු {props.age} කි.</p>
    </div>
  );
}

export default Greeting;
```

64.3 ASCII රූපසටහන: දත්ත ගලායාම (Data Flow)

රියාකට් හි දත්ත ගලා යන්නේ සැමවිටම ඉහළ සිට පහළටය. මෙයට Unidirectional Data Flow(එක් දිශාවකට ගලන දත්ත) යැයි කියනු ලැබේ.

```
[ Parent: App ]
```

```
|
|--- (name="Ali") ---> [ Child: Greeting ]
|
|--- (UI පෙන්වයි: "Hello Ali")
```

64.4 ප්‍රොප්ස් ලබා ගැනීමේ වෘත්තීය ක්‍රමය: Destructuring

වෘත්තීය සංරචක ලියන විට අප props.name, props.age ලෙස හැම තැනම ලියන්නේ නැත. ඒ වෙනුවට අපි පරිමාව 4 හි ඉගෙන ගත් Destructuring තාක්ෂණය භාවිතා කරමු.

ප්‍රෝ-මට්ටමේ කේතය:

```
// props වෙනුවට කෙලින්ම { } ඇතුළත නම් ලියන්න
```

```
function Greeting({ name, age }) {
```

```
  return (
    <div>
      <h2>ආයුබෝවන් {name}!</h2>
      <p>වයස: {age}</p>
    </div>
  );
}
```

මෙය කියවීමට ඉතා පහසු සහ පිරිසිදු ක්‍රමයයි.

64.5 ඕනෑම දත්ත වර්ගයක් යැවීම (Passing Any Data Type)

ප්‍රොප්ස් ලෙස අපට ඕනෑම දෙයක් යැවිය හැක. අංක හෝ Boolean අගයන් යැවීමේදී ඒවා { } ඇතුළත ලිවිය යුතුය.

<UserCard

```
name="අමරා"           // String
age={25}               // Number
isMember={true}        // Boolean
hobbies=[["Reading", "Coding"]] // Array
address={{ city: "Galle" }} // Object
```

/>

64.6 ප්‍රොප්ස් වල ඇති වැදගත්ම නීතිය: Read-only (Immutable)

මෙය ඔබ කිසිදා අමතක නොකළ යුතු නීතියකි. දරු සංරචකයකට තමාට ලැබුණු ප්‍රොප්ස් වෙනස් කළ නොහැක.

- වැරදියි:

```
function Greeting(props) {
  props.name = "අලුත් නම"; // ✗ මෙලෙස
  කිරීම තහනම්! Error එකක් ලැබේ.

  return <h1>{props.name}</h1>;
}
```

- හේතුව: ප්‍රොප්ස් අයිති වන්නේ දෙමාපිය සංරචකයටය. එය වෙනස් කළ හැක්කේ දෙමාපිය සංරචකයට පමණි. දරුවා කළ යුත්තේ එය පෙන්වීම පමණි.

64.7 වෘත්තීය පැහැදිලි කිරීම (Professional Explanation)

සැබෑ මෘදුකාංග ඉංජිනේරු ක්ෂේත්‍රයේදී අප ප්‍රොප්ස් භාවිතා කරන්නේ සංරචක "ගොළු"(Dumb Components / Presentational Components) බවට පත් කිරීමටයි. මෙයින් අදහස් වන්නේ සංරචකය දන්නේ නැත එය පෙන්වන්නේ කාගේ දත්තද කියා. එයට දත්ත ලැබෙන අතර එය UI එක සාදයි. මෙය මෘදුකාංගය පරීක්ෂා කිරීම (Testing) ඉතා පහසු කරයි.

64.8 පොදු වැරදි (Common Mistakes)

1. කරලි වරහන් { } අමතක වීම: දත්ත ලබා ගන්නා විට { name } වෙනුවට name ලෙස ලිවීම. එවිට ඔබට ලැබෙන්නේ මුළු props object එකමය.
2. ප්‍රොප්ස් වෙනස් කිරීමට උත්සාහ කිරීම: දරු සංරචකය ඇතුළතින් ප්‍රොප්ස් අගයන් වෙනස් කිරීමට යාම.
3. නම් පටලවා ගැනීම: දෙමාපිය ගොනුවේ යවන නම (උදා: userName) සහ දරුවා ලබා ගන්නා නම (උදා: user) වෙනස් වීම. මේවා සමාන විය යුතුය.

64.9 සාරාංශය (Key Points)

- Props මගින් දෙමාපිය සංරචකයේ සිට දරු සංරචකයට දත්ත යවයි.
- දත්ත ගලා යන්නේ එක් දිශාවකට පමණි (Top to Down).
- ප්‍රොප්ස් වෙනස් කළ නොහැක (Immutable).
- Destructuring මගින් කේතය පිරිසිදු කරගත හැක.
- ඕනෑම ජාවාස්ක්‍රිප්ට් දත්තයක් ප්‍රොප්ස් ලෙස යැවිය හැක.

64.10 අභ්‍යාස (Exercises)

1. Product.jsx නමින් සංරචකයක් සාදන්න. එයට name, price, සහ description යන ප්‍රොප්ස් යවා ලස්සන කාඩ්පතක් ලෙස තිරයේ පෙන්වන්න.
2. පහත කේතයේ ඇති වැරද්ද කුමක්ද?

```
// Parent
```

```
<Avatar img="user.png" />
```

```
// Child
```

```
function Avatar(props) {
```

```
  return <img src={props.url} />;
```

```
}
```

3. අරාට්ක් (Array) ප්‍රොප් එකක් ලෙස යවා දරු සංරචකය තුළ map() ශ්‍රිතය භාවිතා කර ලැයිස්තුවක් පෙන්වීමට උත්සාහ කරන්න.

64.11 කුඩා ව්‍යාපෘතිය (Mini Project)

ව්‍යාපෘතිය: ශිෂ්‍ය තොරතුරු පද්ධතිය (Student Info Cards)

1. StudentCard.jsx නමින් සංරචකයක් සාදා එයට ශිෂ්‍යයාගේ නම, ලකුණු සහ ඔහු සමත්ද අසමත්ද (isPassed) යන දත්ත යවන්න.
2. සමත් නම් කොළ පාටින් ද, අසමත් නම් රතු පාටින් ද පෙනෙන සේ සැකසුම් (Conditional Styling) කරන්න.
3. App.jsx තුළ විවිධ සිසුන් 5 දෙනෙකු සඳහා මෙම සංරචකය භාවිතා කරන්න.

මිලින පරිච්ඡේදයෙන් අපි රියැක්ට් හි දත්ත පාලනය කරන සැබෑ මැජික් එක හෙවත් "ස්ටේට්"(State - useState Hook) ගැන ඉතාමත් විස්තරාත්මකව ඉගෙන ගනිමු.

හැට පස්වන පරිච්ඡේදය: ස්ටේට් - සංරචකයක අභ්‍යන්තර දත්ත පාලනය (State - Internal Data Management and useState Hook)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අධ්‍යයනය කිරීමෙන් පසු ඔබට පහත කරුණු පිළිබඳව අතිශය ගැඹුරු සහ වෘත්තීය අවබෝධයක් ලැබෙනු ඇත:

1. State (ස්ටේට්) යනු කුමක්ද සහ රියැක්ට් හි එහි ඇති තීරණාත්මක වැදගත්කම වටහා ගැනීම.
2. ප්‍රොප්ස් (Props) සහ ස්ටේට් (State) අතර ඇති මූලික වෙනස හඳුනා ගැනීම.
3. useState නමැති රියැක්ට් හූක් (Hook) එකෙහි ව්‍යුහය සහ එය ක්‍රියා කරන ආකාරය තේරුම් ගැනීම.
4. දත්ත වෙනස් වන විට රියැක්ට් විසින් ස්වයංක්‍රීයව UI එක යාවත්කාලීන කරන ආකාරය (Re rendering) වටහා ගැනීම.
5. ස්ටේට් එකක් සෘජුව වෙනස් නොකිරීමේ (Immutability) වැදගත්කම ඉගෙන ගැනීම.
6. සැබෑ ලෝකයේ උදාහරණයක් ඇසුරින් දත්තවෙනස් වන මෘදුකාංගයක් නිර්මාණය කිරීම.

65.1 හැඳින්වීම: ස්ටේට් යනු කුමක්ද? (What is State?)

පෙර පරිච්ඡේදයේදී අප ඉගෙන ගත්තේ ප්‍රොප්ස් (Props) ගැනයි. ප්‍රොප්ස් යනු සංරචකයකට පිටතින් ලැබෙන දත්ත වේ. නමුත් සැබෑ වෙබ් අඩවියකදී

සංරචකයක් ඇතුළත දත්ත නිරන්තරයෙන් වෙනස් වීමට සිදු වේ.

උදාහරණයක් ලෙස:

- ක්‍රිකට් ලකුණු පුවරුවක ලකුණු වැඩි වීම.
- පරිශීලකයෙකු "Like" බොත්තම එබූ විට එහි වර්ණය සහ අංකය වෙනස් වීම.
- Input කොටුවක අකුරු ටයිප් කරන විට ඒවා තිරයේ දිස්වීම.

මෙලෙස කාලයත් සමඟ වෙනස් වන, සංරචකයකටම ආවේණික වූ අභ්‍යන්තර දත්ත State (ස්ටේට්) ලෙස හැඳින්වේ.

සැබෑ ලෝකයේ උපමාව (මිනිස් සිරුර):

- Props: යනු ඔබ අඳින ඇඳුම් හෝ ඔබ අතේ ඇති පාර්සලයක් වැනිය. ඒවා පිටතින් ලැබෙන දේවල්ය.
- State: යනු ඔබේ හද ගැස්ම හෝ ඔබේ ශරීරයේ උෂ්ණත්වය වැනිය. ඒවා ඔබේ සිරුර ඇතුළතින්ම පාලනය වන, නිරන්තරයෙන් වෙනස් වන දත්ත වේ. ඔබේ උෂ්ණත්වය වෙනස් වූ විට ඔබේ ශරීරය එයට ප්‍රතිචාර දක්වයි (උදා: දහඩිය දැමීම). රියැක්ට් වලදී ද State එක වෙනස් වූ විට සංරචකය එයට ප්‍රතිචාර දක්වයි (එනම් UI එක අලුත් කරයි).

65.2 ඇයි අපට ස්ටේට් අවශ්‍ය වන්නේ? (Why State?)

සාමාන්‍ය ජාවාස්ක්‍රිප්ට් වලදී අප විචල්‍යයක් (let x = 10) සාදා එය වෙනස් කළ විට, එම වෙනස ඔවුසරයේ තිරය මත පෙන්නීමට අපට නැවතත් DOM එක අතින් වෙනස් කිරීමට සිදු වේ. නමුත් රියැක්ට් වලදී අප ස්ටේට් එකක් භාවිතා කළ විට:

1. අපට තිබෙන්නේ ස්ටේට් අගය වෙනස් කිරීම පමණි.
2. එම වෙනස සිදු වූ සැණින්, රියැක්ට් විසින් ස්වයංක්‍රීයව එම සංරචකය නැවත අඳියි (Re-render).
3. අපට DOM එක වෙනස් කිරීමට එකදු කේත පේළියක්වත් ලිවීමට අවශ්‍ය නැත.

65.3 හූක්ස් (Hooks) යනු මොනවාද?

ස්ටේට් ගැන ඉගෙන ගැනීමට පෙර අප Hooks යනු කුමක්දැයි දැනගත යුතුය. Hooks (හූක්ස්) යනු රියැක්ට් වල ඇති විශේෂ ශ්‍රිත (Special Functions) සමූහයකි. මේවා මගින් අපට රියැක්ට් හි ඇති ඉතා

බලවත් විශේෂාංග (ස්ටේට් වැනි) අපගේ Functional Components වලට සම්බන්ධ කර ගැනීමට (Hook කර ගැනීමට) ඉඩ ලබා දෙයි. රියැක්ට් හි සෑම හුක් එකක්ම පටන් ගන්නේ "use" යන වචනයෙනි (උදා: useState, useEffect).

65.4 useState හුක් එක භාවිතා කරන ආකාරය (Syntax)

ස්ටේට් එකක් සෑදීමට අප භාවිතා කරන්නේ useState නමැති හුක් එකයි.

භාවිතා කරන ආකාරය: මූලික මෙය රියැක්ට් වලින් ලබා ගත (Import) යුතුය.

```
import { useState } from 'react';
```

ඉන්පසු සංරචකය ඇතුළත මෙලෙස ප්‍රකාශ කරයි:

```
const [data, setData] =
  useState(initialValue);
```

කොටස් 4 පැහැදිලි කිරීම (Array Destructuring):

1. data: වත්මන් ස්ටේට් අගය (The current state value).
2. setData: මෙම අගය වෙනස් කිරීමට භාවිතා කරන විශේෂිත ශ්‍රිතය (The setter function).
3. useState: ස්ටේට් එකක් සාදන හුක් එක.
4. initialValue: ස්ටේට් එක පටන් ගන්නා විට තිබිය යුතු මූලික අගය (උදා: 0 හෝ "").

65.5 ප්‍රායෝගික උදාහරණය: අංක ගණන් කිරීමේ මෘදුකාංගය (Counter App)

අපි දැන් බොත්තමක් එබූ විට අංකය වැඩි වන සරල සංරචකයක් සාදමු.

```
import { useState } from 'react';
```

```
function Counter() {
```

```
  // 'count' යනු අගයයි, 'setCount' යනු එය
  වෙනස් කරන ශ්‍රිතයයි
```

5. අලුත් count අගය සමඟ Virtual DOM එක අලුත් වේ.

6. කිරය මත අපට අලුත් අංකය දිස්වේ. ASCII රූපසටහන: ස්ටේට් වෙනස් වීම

```
// ආරම්භක අගය 0 වේ.
```

```
const [count, setCount] = useState(0);
```

```
function increment() {
```

```
  // ස්ටේට් එක වෙනස් කිරීමේ නිවැරදි ක්‍රමය
```

```
  setCount(count + 1);
```

```
}
```

```
return (
```

```
  <div style={{ textAlign: 'center',
  marginTop: '50px' }}>
```

```
    <h1>ගණනය: {count}</h1>
```

```
    <button onClick={increment}>අංකය
    වැඩි කරන්න</button>
```

```
    <button onClick={() => setCount(count
    - 1)}>අංකය අඩු කරන්න</button>
```

```
  </div>
```

```
);
```

```
}
```

```
export default Counter;
```

65.6 රී-රෙන්ඩරින් ක්‍රියාවලිය (The Rendering Flow)

ස්ටේට් එකක් වෙනස් වූ විට රියැක්ට් ඇතුළත සිදුවන දේ මෙන්න:

1. පරිශීලකයා බොත්තම ඔබයි.
2. setCount ශ්‍රිතය ක්‍රියාත්මක වේ.
3. රියැක්ට් විසින් count අගය 1 කින් වැඩි කර මතකයේ තබා ගනී.
4. රියැක්ට් විසින් එම සංරචකය මුල සිට අගට නැවත කියවයි (Re-render).

[User Action] ---> [Setter Function (setCount)]

v

[State Updates in Memory]

v

[React triggers Re-render]

v

[UI Displays New Value]

65.7 ස්ටේට් සහ ප්‍රොප්ස් අතර වෙනස (State vs Props)

මෙය සම්මුඛ පරීක්ෂණ වලදී නිතර අසන ප්‍රශ්නයකි.

ලක්ෂණය	Props (ප්‍රොප්ස්)	State (ස්ටේට්)
:-	:-	:-
මූලාශ්‍රය	පියවරින් (Parent component එකෙන්) ලැබේ.	සංරචකය ඇතුළතම සාදනු ලැබේ.
වෙනස් කිරීම	සංරචකය ඇතුළතින් වෙනස් කළ නොහැක (Read-only).	Setter function එක මගින් වෙනස් කළ හැක.
අරමුණ	දත්ත හුවමාරු කිරීමට.	කාලයත් සමඟ වෙනස් වන දත්ත පාලනයට.

65.8 වැදගත්ම නීතිය: ස්ටේට් සාප්පුව වෙනස් නොකරන්න!

ඔබ කිසිම විටක මෙලෙස කේතය නොලිය යුතුය:

- වැරදියි: `count = count + 1;`
- හේතුව: ඔබ අගය වෙනස් කළද, රියැක්ට එය දැනගන්නේ නැත. එවිට Re-render එකක් සිදු නොවන අතර තිරය මත වෙනසක් පෙනෙන්නේ නැත.
- නිවැරදියි: සැමවිටම `setCount(newValue)` භාවිතා කරන්න. රියැක්ට හට පණිවිඩය ලැබෙන්නේ එවිට පමණි.

65.9 වෘත්තීය පැහැදිලි කිරීම (Professional Explanation)

වෘත්තීය මෘදුකාංග ඉංජිනේරුවෙකු ස්ටේට් එකක් භාවිතා කරන්නේ අවශ්‍යම අවස්ථාවලදී පමණි. ඕනෑවට වඩා ස්ටේට් භාවිතය මෘදුකාංගයේ වේගය අඩු වීමට (Performance issues) හේතු විය හැක. එසේම, රියැක්ට හි ස්ටේට් අප්ටේට් කිරීම අසමකාලීන (Asynchronous) ලෙස සිදුවන

බැවින්, එකම ශ්‍රිතයක් තුළ එකවර ස්ටේට් කිහිපයක් වෙනස් කිරීමේදී සැලකිලිමත් විය යුතුය.

65.10 පොදු වැරදි (Common Mistakes)

1. සංරචකයෙන් පිටත `useState` ලිවීම: `useState` ලිවිය හැක්කේ සංරචක ශ්‍රිතය ඇතුළත පමණි.
2. කොන්දේසි (If statements) ඇතුළත `useState` ලිවීම: මෙය ඉතාමත් වැදගත්ය. Hooks සැමවිටම සංරචකයේ ඉහළින්ම ලිවිය යුතු අතර ඒවා `if` හෝ `for loops` ඇතුළත ලිවිය නොහැක.
3. ආරම්භක අගය අමතක වීම: `useState()` ලෙස හිස්ව තැබුවහොත් ආරම්භක අගය `undefined` වේ. මෙය පසුව ගණිතමය දෝෂ ඇති කළ හැක.

65.11 සාරාංශය (Key Points)

- State යනු සංරචකයක් සතු වෙනස් වන දත්තයි.
- `useState` හුක් එක ස්ටේට් එකක් නිර්මාණය කිරීමට භාවිතා කරයි.
- ස්ටේට් එක වෙනස් වූ විට රියැක්ට විසින් සංරචකය නැවත අඳියි (Re-render).

- ප්‍රොපර්ටි යනු බාහිර දත්ත වන අතර ස්ටේට් යනු අභ්‍යන්තර දත්ත වේ.
- සැමවිටම අගය වෙනස් කිරීමට **Setter function** එක භාවිතා කළ යුතුය.

65.12 අභ්‍යාස (Exercises)

1. **useState** භාවිතා කර බොත්තමක් එබූ විට වෙබ් පිටුවේ පසුබිම් වර්ණය රතු සහ නිල් ලෙස මාරු වන (**Toggle**) සංරචකයක් සාදන්න.
2. පහත කේතයේ ඇති වැරද්ද කුමක්දැයි සොයා නිවැරදි කරන්න:

```
function MyComponent() {
  let name = "කසුන්";

  return <button onClick={() => name = "නිමල්"}>{name}</button>;
}
```

3. ආරම්භක අගය ලෙස **Object** එකක් (`{ name: "", age: 0 }`) ඇති ස්ටේට් එකක් ප්‍රකාශ කරන ආකාරය පෙන්වන්න.

65.13 කුඩා ව්‍යාපෘතිය (Mini Project)

ව්‍යාපෘතිය: සරල "පෙළ පෙන්වන/සහවන" පද්ධතිය (**Show/Hide Text**)

1. **VisibilityToggle.jsx** නමින් සංරචකයක් සාදන්න.
2. එහි **Boolean** ස්ටේට් එකක් සාදන්න (උදා: **isVisible**). ආරම්භක අගය **false** ලෙස තබන්න.
3. එක් බොත්තමක් සාදන්න. එය එබූ විට ස්ටේට් එක විරුද්ධ පැත්තට හැරවිය යුතුය (**Toggle**).
4. ස්ටේට් එක **true** නම් පමණක් යම් රහස්‍ය පණිවිඩයක් තිරයේ දිස්විය යුතුය. (ඉභිය:

```
{isVisible && <p>පණිවිඩය</p>} භාවිතා කරන්න).
```

මිලිභ පරිච්ඡේදයෙන් අපි රියැක්ට් හි සිදුවීම් පාලනය (**Events**) සහ ස්ටේට් එකට දත්ත ඇතුළත් කරන ආකාරය ඉතාමත් විස්තරාත්මකව ඉගෙන ගනිමු.

හැට් හයවන පරිච්ඡේදය: සිදුවීම් පාලනය - පරිශීලක ක්‍රියාකාරකම්වලට ප්‍රතිචාර දැක්වීම (**Handling Events in React**)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අධ්‍යයනය කිරීමෙන් පසු ඔබට පහත කරුණු පිළිබඳව අතිශය ගැඹුරු සහ වෘත්තීය අවබෝධයක් ලැබෙනු ඇත:

1. රියැක්ට් හි සිදුවීම් පාලනය (**Event Handling**) සහ සාමාන්‍ය **HTML** හි සිදුවීම් පාලනය අතර ඇති වෙනස හඳුනා ගැනීම.
2. සින්තෙටික් ඉවෙන්ට් (**Synthetic Events**) යනු කුමක්ද සහ රියැක්ට් ඒවා කළමනාකරණය කරන ආකාරය වටහා ගැනීම.
3. බොත්තම් ක්ලික් කිරීම (**onClick**) සහ පෙරරමවල දත්ත වෙනස්වීම් (**onChange**) හසුරුවන ආකාරය.
4. සිදුවීම් හසුරුවන ශ්‍රිත (**Event Handlers**) වෙත අමතර දත්ත (**Arguments**) යවන ආකාරය.
5. ඉවෙන්ට් ඔබ්ජෙක්ට් (**Event Object**) එක භාවිතා කර ඉන්පුට් කොටුවල අගයන් ලබා ගන්නා ආකාරය.
6. සිදුවීම් සහ ස්ටේට් (**State**) එකිනෙක සම්බන්ධ කර අන්තර්ක්‍රියාකාරී අතුරුමුහුණත් නිර්මාණය කිරීම.

66.1 හැඳින්වීම: රියැක්ට් හි සිදුවීම් (Events in React)

අපි සාමාන්‍ය ජාවාස්ක්‍රිප්ට් වලදී (පරිමාව 3, පරිච්ඡේදය 38) ඉගෙන ගන්නා වෙබ් පිටුවක සිදුවන ඕනෑම ක්‍රියාවක් (**Click, Hover, Type**) **Event** එකක් ලෙස හඳුන්වන බව. රියැක්ට් වලදී ද මෙම සිදුවීම් පාලනය කිරීම ඉතාමත් වැදගත් වේ. මන්දයත් පරිශීලකයා කරන ක්‍රියා මත පදනම්ව අපට වෙබ් අඩවියේ ඇති ස්ටේට් (**State**) එක වෙනස් කිරීමට සිදු වනබැවිනි.

රියැක්ට් හි සිදුවීම් පාලනය කිරීම සාමාන්‍ය **HTML** වලට ඉතා සමාන වුවත්, එහි ප්‍රධාන තාක්ෂණික වෙනස්කම් දෙකක් ඇත:

1. **camelCase** නාමකරණය: රියැක්ට් හි සිදුවීම් ලියන්නේ **camelCase** ක්‍රමයටය. (උදා: **HTML** හි **onclick** වෙනුවට රියැක්ට් හි **onClick**).
2. ශ්‍රිතයක් ලබා දීම: රියැක්ට් හිදී අප සිදුවීමකට ලබා දෙන්නේ **String** එකක් නොව, සැබෑ ශ්‍රිතයක් (**Function**) වේ.

සංසන්දනය:

- **HTML:** `<button onclick="greet()">Say Hello</button>`

- React: <button onClick={greet}>Say Hello</button>

66.2 සින්තටික් ඉවෙන්ට් යනු කුමක්ද? (What is SyntheticEvent?)

වෘත්තීය මට්ටමේදී ඔබ දැනගත යුතු රියැක්ට් රහස්‍යක් මෙන්න. ඔබ රියැක්ට් හි බොත්තමක් ක්ලික් කළ විට, බ්‍රවුසරයෙන් ලැබෙන සාමාන්‍ය Event එක රියැක්ට් විසින් ලබාගෙන එය SyntheticEvent නමැති විශේෂ ආවරණයකින් (Wrapper) ඔතනු ලබයි.

ඇයි රියැක්ට් මෙසේ කරන්නේ? විවිධ බ්‍රවුසර (Chrome, Safari, Firefox) වල සිදුවීම්

හැසිරෙන ආකාරය සුළු වශයෙන් වෙනස් විය හැක. රියැක්ට් විසින් මෙම සියලුම බ්‍රවුසර වල පවතින වෙනස්කම් ඉවත් කර, අපට සියලුම බ්‍රවුසර වල එක හා සමානව වැඩ කරන "පොදු ඉවෙන්ට් එකක්" ලබා දෙයි. මෙයට Cross-browser Wrapper එකක් යැයි කියනු ලැබේ.

66.3 මූලික සිදුවීම් පාලනය (Basic Event Handling)

අපි දැන් බොත්තමක් ක්ලික් කළ විට පැවරිය යුතු දිස්වන ආකාරය බලමු.

```
function ActionButton() {
  // සිදුවීම පාලනය කරන ශ්‍රිතය (Event Handler)

  const handleClick = () => {
    alert("බොත්තම ක්ලික් කරන ලදී!");
  };

  return (
    <button onClick={handleClick}>
      මාව ඔබන්න
    </button>
  );
}
```

වැදගත්: onClick={handleClick()} ලෙස වරහන් සහිතව ලියන්න එපා. ඔබ වරහන් යෙදවහොත්, බොත්තම එබීමටත් පෙර එම ශ්‍රිතය ස්වයංක්‍රීයව ක්‍රියාත්මක වේ. අපට අවශ්‍ය වන්නේ එය එබූ විට පමණක් ක්‍රියාත්මක වීමට බැවින්, වරහන් රහිතව නම පමණක් ලබා දිය යුතුය.

66.4 ඉවෙන්ට් ඔබ්ජෙක්ට් එක භාවිතා කිරීම (The Event Object)

පරිශීලකයා ඉන්පුට් කොටුවක යමක් ටයිප් කරන විට, එම අකුරු ලබා ගැනීමට අපට Event Object එක අවශ්‍ය වේ. සාමාන්‍යයෙන් අප මෙය e ලෙස හඳුන්වමු.

```
import { useState } from 'react';

function TextInput() {

  const [text, setText] = useState("");

  const handleChange = (e) => {

    // e.target යනු අදාළ input කොටුවයි

    // e.target.value යනු එහි ඇති අකුරු ප්‍රමාණයයි

    setText(e.target.value);

  };

  return (
    <div>

      <input type="text"
        onChange={handleChange}
        placeholder="යමක් ලියන්න..." />

      <p>ඔබ ටයිප් කළේ: {text}</p>

    </div>

  );
}
```

පැහැදිලි කිරීම: මෙහිදී onChange සිදුවීම නිසා පරිශීලකයා අකුරක් ටයිප් කරන සෑම වාරයකදීම handleChange ශ්‍රිතය ක්‍රියාත්මක වී ස්ටේට් එක යාවත්කාලීන කරයි.

66.5 ශ්‍රිතයකට අමතර දත්ත යැවීම (Passing Arguments)

සමහර විට අපට බොක්කමක් එබූ විට ශ්‍රිතය වෙත අමතර දත්තයක් (උදා: ID එකක් හෝ නමක්) යැවීමට අවශ්‍ය වේ. එවැනි විටකදී අප Arrow Function එකක් භාවිතා කළ යුතුය.

```
function UserList() {

  const deleteUser = (id) => {

    console.log("මකා දමන පරිශීලකයාගේ අංකය: " + id);

  };

  return (

    <div>

      {/* වැරදියි: onClick={deleteUser(1)} */}

      {/* නිවැරදියි: Arrow function එකක් ඇතුළත ලියන්න */}

      <button onClick={() => deleteUser(1)}>User 1 මකන්න</button>

      <button onClick={() => deleteUser(2)}>User 2 මකන්න</button>

    </div>

  );

}
```

66.6 පෝරම යැවීම පාලනය කිරීම (Handling Form Submission)

පෝරමයක් (Form) Submit කරන විට බ්‍රවුසරය ස්වයංක්‍රීයව පිටුව Refresh කරයි. රියැක්ට් වලදී මෙය වැළැක්වීමට e.preventDefault() භාවිතා කරයි.

```
function SimpleForm() {

  const handleSubmit = (e) => {

    e.preventDefault(); // පිටුව Refresh වීම වළක්වයි
```

```
    alert("දත්ත සාර්ථකව යවන ලදී!");

  };

  return (

    <form onSubmit={handleSubmit}>

      <input type="text" />

      <button type="submit">යවන්න</button>

    </form>

  );

}
```

66.7 ASCII රූපසටහන: සිදුවීම සහ ස්ටේට් අතර සම්බන්ධය

```
[ User Action ] ----> ( onClick / onChange )

v

[ Event Handler Function ]

v

[ setState() is called ]

v

[ React triggers Re-render ]

v

[ UI Updates on Screen ]
```

66.8 වෘත්තීය පැහැදිලි කිරීම (Professional Explanation)

වෘත්තීය මෘදුකාංග ඉංජිනේරුවෙකු සිදුවීම් පාලනය කිරීමේදී "Inline functions" (JSX එක ඇතුළතම ශ්‍රිත ලිවීම) භාවිතය අවම කරයි. මන්දයත් සැම රෙන්ඩර් (Render) වාරයකදීම එම ශ්‍රිතය අලුතින් මතකයේ නිර්මාණය වන බැවිනි. ඉතා සංකීර්ණ මෘදුකාංගවලදී මෙය වේගය අඩු වීමට (Performance issues) හේතු විය හැක. එබැවින් සැමවිටම ශ්‍රිතය වෙනම අර්ථ දක්වා පසුව එය සම්බන්ධ කිරීම හොඳ පුරුද්දකි.

66.9 පොදු වැරදි (Common Mistakes)

1. නම වැරදීම: onclick ලෙස ලිවීම (රියැක්ට් වලදී මෙය ක්‍රියා නොකරයි, onClick විය යුතුය).
2. ශ්‍රිතය වහාම ක්‍රියාත්මක කිරීම:
onClick={myFunc()} ලෙස ලිවීම. මෙය ශ්‍රිතය වහාම ක්‍රියාත්මක කර එහි ප්‍රතිඵලය (Return value) පමණක් බොත්තමට ලබා දෙයි. සැමවිටම නම පමණක් (myFunc) හෝ arrow function එකක් () => myFunc() භාවිතා කරන්න.
3. Prevent Default අමතක වීම: පෝරමයක් යැවීමේදී පිටුව Refresh වී ස්ටේට් එකේ ඇති සියලුම දත්ත මැකී යාම.

66.10 සාරාංශය (Key Points)

- රියැක්ට් සිදුවීම් camelCase ක්‍රමයට ලියනු ලැබේ.
- සිදුවීමකට ලබා දෙන්නේ ශ්‍රිතයකි (Function reference).
- SyntheticEvent මගින් බ්‍රවුසර අතර පවතින වෙනස්කම් මකා දමයි.
- e.target.value මගින් ඉන්පුට් දත්ත ලබා ගත හැක.
- e.preventDefault() මගින් පිටුව Refresh වීම නවත්වයි.

66.11 අභ්‍යාස (Exercises)

1. පින්තූරයක් මතට මවුසය ගෙන ගිය විට (onMouseEnter) කන්සෝලයේ පණිවිඩයක් මුද්‍රණය වන සරල සංරචකයක් සාදන්න.
2. පරිශීලකයාට නම සහ වයස ඇතුළත් කිරීමට input කොටු දෙකක් සාදා, "Show" බොත්තම එබූ විට එම දත්ත alert එකක් මගින් පෙන්වන පද්ධතියක් සකසන්න.
3. පහත කේතයේ වැරද්ද කුමක්ද?

```
<button
onClick={console.log("Clicked!")}>Click
Me</button>
```

66.12 කුඩා ව්‍යාපෘතිය (Mini Project)

ව්‍යාපෘතිය: වර්ණ මාරු කරන "Color Picker" පද්ධතිය

1. පරිශීලකයාට වර්ණයක නමක් ටයිප් කිරීමට Input කොටුවක් සාදන්න.

2. "Update Background" ලෙස බොත්තමක් සාදන්න.
3. බොත්තම එබූ විට, පරිශීලකයා ටයිප් කළ වර්ණයට අනුව වෙබ් පිටුවේ මැද ඇති විශාල කොටුවක (div) පසුබිම් වර්ණය වෙනස් විය යුතුය.
4. වැරදි වර්ණ නමක් ටයිප් කළහොත් දෝෂයක් නොවී එය කළමනාකරණය කිරීමට උත්සාහ කරන්න.

මිලන පරිච්ඡේදයෙන් අපි යම් කොන්දේසියක් මත පදනම්ව වෙබ් පිටුවේ කොටස් පෙන්වන හෝ සඟවන ආකාරය හෙවත් "කොන්දේසි මත පදනම් වූ නිරූපණය" (Conditional Rendering) ගැන ඉතාමත් විස්තරාත්මකව ඉගෙන ගනිමු.

හැට හත්වන පරිච්ඡේදය: කොන්දේසි මත පදනම් වූ නිරූපණය (Conditional Rendering in React)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අධ්‍යයනය කිරීමෙන් පසු ඔබට පහත කරුණු පිළිබඳව අතිශය ගැඹුරු සහ වෘත්තීය අවබෝධයක් ලැබෙනු ඇත:

1. රියැක්ට් හි Conditional Rendering (කොන්දේසි මත පදනම් වූ නිරූපණය) යනු කුමක්දැයි තේරුම් ගැනීම.
2. යම් දත්තයක හෝ ස්ටේට් (State) එකක පවතින අගය අනුව වෙබ් පිටුවේ පෙනුම වෙනස් කරන ආකාරය.
3. if ප්‍රකාශන (if Statements) භාවිතා කර සංරචක පාලනය කරන ආකාරය.
4. Ternary Operator (තෘතීයක ක්‍රියාකරු) භාවිතා කර JSX තුළ කෙටි කොන්දේසි ලිවීම.
5. Logical AND (&&) ක්‍රියාකරු භාවිතා කර යමක් පෙන්වීමට හෝ සැඟවීමට (Short-circuiting) ඉගෙන ගැනීම.
6. සංරචකයක් සම්පූර්ණයෙන්ම නොපෙනී යාමට සැලැස්වීම (Returning null).
7. සැබෑ ලෝකයේ උදාහරණයක් ලෙස "Login/Logout" පද්ධතියක තර්කනය ගොඩනැගීම.

67.1 හැඳින්වීම: කොන්දේසි මත පදනම් වූ නිරූපණය යනු කුමක්ද?

වෙබ් අඩවියක් යනු සෑම විටම එකම දේ පෙන්වන නිශ්චල පුවරුවක් නොවේ. පරිශීලකයාගේ තත්වය අනුව වෙබ් අඩවියේ පෙනුම වෙනස් විය යුතුය.

උදාහරණයක් ලෙස:

- පරිශීලකයා ලොගින් (Login) වී ඇත්නම් "සාදරයෙන් පිළිගනිමු" කියාත්, ලොගින් වී නැත්නම් "Login වන්න" බොත්තමකුත් පෙන්විය යුතුය.
- දත්ත සර්වර් එකෙන් ලැබෙන තෙක් "Loading..." යනුවෙන් පණිවිඩයක් පෙන්විය යුතුය.
- ඔබේ Notification (දැනුම්දීම්) තිබේ නම් පමණක් රතු පැහැති තීතක් පෙන්විය යුතුය.

ජාවාස්ක්‍රිප්ට් හි අප ඉගෙන ගත් if වැනි කොන්දේසි භාවිතා කර, යම් අවස්ථාවක් සත්‍ය ද අසත්‍ය ද යන්න මත පදනම්ව UI (පරිශීලක අතුරුමුහුණත) කොටස් පෙන්වීමට රියැක්ට් වලදී Conditional Rendering යැයි කියනු ලැබේ.

සැබෑ ලෝකයේ උපමාව (ස්වයංක්‍රීය දොරක්): සිතන්න සුපිරි වෙළඳසැලක ඇති ස්වයංක්‍රීය දොරක් ගැන.

- කොන්දේසිය: දොර ඉදිරිපිට කෙනෙකු සිටීද?
- ප්‍රතිඵලය 1 (සත්‍ය නම්): දොර විවෘත වේ (Show Open Door UI).
- ප්‍රතිඵලය 2 (අසත්‍ය නම්): දොර වැසී පවතී (Show Closed Door UI).

67.2 ක්‍රමය 1: if ප්‍රකාශනය භාවිතා කිරීම (Using if Statements)

සාමාන්‍ය if ප්‍රකාශනයක් JSX ඇතුළත ලිවිය නොහැක. එය ලිවිය යුත්තේ සංරචක ශ්‍රිතය ඇතුළත, නමුත් return විධානයට ඉහළින් ය.

උදාහරණය: ලොගින් තත්වය අනුව පණිවිඩය වෙනස් කිරීම

```
function Greeting({ isLoggedIn }) {

  // 1. කොන්දේසිය පරීක්ෂා කිරීම

  if (isLoggedIn) {

    return <h1>නැවතත් සාදරයෙන් පිළිගනිමු!</h1>;
```

```
}
```

```
// 2. කොන්දේසිය අසත්‍ය නම්
```

```
return <h1>කරුණාකර ලොගින් වන්න.</h1>;
```

```
}
```

විස්තරය: මෙහිදී isLoggedIn නමැති ප්‍රොප් (Prop) එක true නම් පළමු return එක ක්‍රියාත්මක වේ. එය false නම් දෙවන return එක ක්‍රියාත්මක වේ.

67.3 ක්‍රමය 2: තෘතීයක ක්‍රියාකරු (Ternary Operator - ? :)

JSX ඇතුළතම (එනම් HTML කොටස් මැදම) කොන්දේසියක් ලිවීමට අවශ්‍ය වූ විට අප භාවිතා කරන්නේ මෙයයි. මෙය වෘත්තීය මට්ටමේදී ඉතා ඉහළව භාවිතා වේ.

ව්‍යුහය (Syntax): කොන්දේසිය ? සත්‍ය_නම්_මෙය_පෙන්වන්න : අසත්‍ය_නම්_මෙය_පෙන්වන්න

ප්‍රායෝගික කේතය:

```
import { useState } from 'react';
```

```
function AuthButton() {
```

```
  const [isLoggedIn, setIsLoggedIn] = useState(false);
```

```
  return (
```

```
    <div>
```

```
      {/* Ternary Operator භාවිතය */}
```

```
      {isLoggedIn ? (
```

```
        <button onClick={() =>
          setIsLoggedIn(false)}>Logout
        වන්න</button>
```

```
      ) : (
```

```
        <button onClick={() =>
          setIsLoggedIn(true)}>Login වන්න</button>
```

```
      )})
```

```
<p>දැනට තත්වය: {isLoggedIn ? "Logged  
In" : "Logged Out"}</p>
```

```
</div>
```

```
);
```

```
}
```

67.4 ක්‍රමය 3: තර්කන AND ක්‍රියාකරු (Logical AND Operator - &&)

සමහර අවස්ථාවලදී අපට අවශ්‍ය වන්නේ යම් කොන්දේසියක් සත්‍ය නම් පමණක් යමක් පෙන්වීමයි. අසත්‍ය වූ විට කිසිවක් පෙන්වීමට අවශ්‍ය නැත. එවැනි විටකදී && භාවිතා කිරීම ඉතා පිරිසිදු ක්‍රමයකි.

එය ක්‍රියා කරන්නේ මෙලෙසයි: ජාවාස්ක්‍රිප්ට් වලදී true && expression ක්‍රියාත්මක වූ විට එය සැමවිටම expression අගය ලබා දෙයි. false && expression ක්‍රියාත්මක වූ විට එය false ලබා දෙයි (රියාත්මක වීසින් false යන්න තීරය මත නොපෙන්වයි).

උදාහරණය:

```
function Mailbox({ unreadMessages }) {
```

```
  return (
```

```
    <div>
```

```
      <h1>ආයුබෝවන්!</h1>
```

```
      /* පණිවිඩ තිබේ නම් පමණක් මෙය  
      පෙන්වන්න */
```

```
      {unreadMessages.length > 0 && (
```

```
        <h2 style={{ color: 'red' }}>
```

```
          ඔබට නොකියවූ පණිවිඩ  
{unreadMessages.length} ක් ඇත!
```

```
        </h2>
```

```
      )}
```

```
    </div>
```

```
  );
```

```
}
```

67.5 ක්‍රමය 4: සංරචකයක් සැහවීම (Returning null)

සමහර විට අපට අවශ්‍ය වන්නේ යම් කොන්දේසියක් මත මුළු සංරචකයම (Component) නොපෙන්වා සිටීමයි. එවැනි විටකදී අප ශ්‍රිතයෙන් null අගය ආපසු ලබා දිය යුතුය (Return null).

```
function WarningBanner({ showWarning })  
{
```

```
  // පෙන්විය යුතු නැතිනම් null එවන්න
```

```
  if (!showWarning) {
```

```
    return null;
```

```
  }
```

```
  return (
```

```
    <div className="warning">
```

```
      අවධානය! මෙය අනතුරු ඇඟවීමකි.
```

```
    </div>
```

```
  );
```

```
}
```

67.6 ASCII රූපසටහන: තීරණ ගැනීමේ ගලායුම

[Component Starts]

```
|
```

```
{ Condition ? }
```

```
/ \
```

```
(Yes) (No)
```

```
| |
```

[Show UI A] [Show UI B]

```
\_____/
```

```
|
```

[Screen Render]

67.7 වෘත්තීය පැහැදිලි කිරීම (Professional Explanation)

වෘත්තීය Frontend Architect කෙනෙකු ලෙස, ඔබ "Nested Ternaries" (එකක් ඇතුළත තවත් එකක් ඇති Ternary operators) භාවිතයෙන් වැළකී සිටිය යුතුය. එය කේතය කියවීමට ඉතා අපහසු කරයි. කොන්දේසි 3කට වඩා වැඩි නම්, සංරචකයෙන් පිටත වෙනම ශ්‍රිතයක් සාදා හෝ switch ප්‍රකාශනයක් භාවිතා කර පිරිසිදු බව (Readability) පවත්වා ගැනීම වෘත්තීය ලක්ෂණයකි.

67.8 පොදු වැරදි (Common Mistakes)

1. බිංදුව (0) පිළිබඳ ගැටලුව: `&&` භාවිතා කරන විට පරීක්ෂා කරන අගය 0 නම්, රියැක්ට් විසින් තීරය මත 0 අංකය මුද්‍රණය කරයි. මෙය වළක්වා ගැනීමට සැමවිටම එය Boolean අගයක් බවට හරවන්න (උදා: `count > 0 && ...`).
2. කරලි වරහන් `{ }` අමතක වීම: JSX තුළ ජාවාස්ක්‍රිප්ට් තර්කනයක් ලියන විට සැමවිටම එය `{ }` ඇතුළත තිබිය යුතුය.
3. Complex Logic in JSX: ඉතා සංකීර්ණ තර්කනයන් JSX එක ඇතුළත ලිවීම. සැමවිටම ඒවා වෙනම විචල්‍යයකට ගෙන `{variableName}` ලෙස භාවිතා කරන්න.

67.9 සාරාංශය (Key Points)

- Conditional Rendering මගින් තර්කනය මත පදනම්ව UI පෙන්වයි.
- if ප්‍රකාශන return එකට ඉහළින් භාවිතා කරයි.
- Ternary Operator (`? :`) JSX තුළ කෙටි තීරණ සඳහා සුදුසුය.
- Logical AND (`&&`) යමක් තිබේ නම් පමණක් පෙන්වීමට භාවිතා කරයි.
- null මගින් සංරචකයක් සම්පූර්ණයෙන්ම සභවා තැබිය හැක.

67.10 අභ්‍යාස (Exercises)

1. පරිශීලකයාගේ වයස (age) ප්‍රොප් එකක් ලෙස ගෙන, වයස 18 ට වැඩි නම් "ඇතුළුවීමට අවසර ඇත" ලෙසත්, අඩු නම් "අවසර නැත" ලෙසත් පෙන්වන සංරචකයක් සාදන්න.
2. isLoading නමැති ස්ටේට් එකක් සාදා, එය true නම් "පිටුව ලෝඩ් වෙමින් පවතී..." ලෙසත්, false

නම් සැබෑ අන්තර්ගතයන් පෙන්වන පද්ධතියක් සකසන්න.

3. පහත කේතයේ ඇති දෝෂය කුමක්ද?

```
{messages.length && <p>You have
messages</p>}
```

(ඉඟිය: `messages.length` බිංදුව වුවහොත් කුමක් සිදුවේද?)

67.11 කුඩා ව්‍යාපෘතිය (Mini Project)

ව්‍යාපෘතිය: බහු-අවස්ථා "Traffic Light" පද්ධතිය

1. TrafficLight.jsx නමින් සංරචකයක් සාදන්න.
2. එහි status නමින් ස්ටේට් එකක් සාදන්න. (අගයන්: "red", "yellow", "green").
3. බොත්තම් තුනක් සාදන්න. එක් එක් බොත්තම එබූ විට ස්ටේට් එක අදාළ වර්ණයට මාරු විය යුතුය.
4. තීරයේ මැද විශාල වටකුරු කොටුවක් (div) සාදන්න.
5. ස්ටේට් එක අනුව එම කොටුවේ වර්ණය සහ ඇතුළත ඇති පණිවිඩය (උදා: "තවතින්න", "සුදානම් වන්න", "යන්න") ගනිකව වෙනස් විය යුතුය.

මිලිභ පරිච්ඡේදයෙන් අපි අරාමක (Array) ඇති දත්ත සමූහයක් එකවර තීරය මත පෙන්වන ආකාරය හෙවත් "ලැයිස්තු සහ කීස්" (Lists and Keys) ගැන ඉතාමත් විස්තරාත්මකව ඉගෙන ගනිමු.

හැට අටවන පරිච්ඡේදය: ලැයිස්තු සහ කීස් - දත්ත සමූහයක් පෙන්වීම (Lists and Keys in React)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අධ්‍යයනය කිරීමෙන් පසු ඔබට පහත කරුණු පිළිබඳව අතිශය ගැඹුරු සහ වෘත්තීය අවබෝධයක් ලැබෙනු ඇත:

1. රියැක්ට් හි අරාමක (Array) ඇති දත්ත සමූහයක් ලැයිස්තුවක් ලෙස පෙන්වන්නේ කෙසේදැයි වටහා ගැනීම.
2. ජාවාස්ක්‍රිප්ට් හි `map()` ශ්‍රිතය JSX තුළ භාවිතා කර දත්ත UI (පරිශීලක අතුරුමුහුණත) බවට පරිවර්තනය කරන ආකාරය ඉගෙන ගැනීම.
3. key (කී) නමැති විශේෂ ගුණාංගය යනු කුමක්ද සහ එහි ඇති අතිශය වැදගත්කම තේරුම් ගැනීම.

4. රියැක්ට් විසින් ලැයිස්තු යාවත්කාලීන කිරීමේදී (Update) කීප් භාවිතා කර වේගය සහ ස්ථායීතාවය පවත්වා ගන්නා ආකාරය (Reconciliation) වටහා ගැනීම.
5. වස්තූන්ගෙන් සමන්විත සංකීර්ණ අරාවන් (Array of Objects) පාලනය කරන ආකාරය ප්‍රගුණ කිරීම.
6. සැබෑ ලෝකයේ උදාහරණයක් ලෙස "භාණ්ඩ ලැයිස්තුවක්" (Product List) නිර්මාණය කිරීම.

68.1 හැඳින්වීම: ලැයිස්තු පෙන්වීමේ අවශ්‍යතාවය

අපි මෙතෙක් ඉගෙන ගත් සංරචකවල පෙන්වූයේ තනි දත්තයන්ය (උදා: එක් නමක් හෝ එක් අංකයක්). නමුත් සැබෑ ලෝකයේ මෘදුකාංගවල අපට දත්ත සමූහයක් එකවර පෙන්වීමට සිදු වේ.

සැබෑ ලෝකයේ උදාහරණ:

- Facebook: ඔබේ මිතුරන්ගේ ලැයිස්තුව.
- WhatsApp: ඔබේ වැට් (Chat) ලැයිස්තුව.
- Uber Eats: පෙන්වීමට ඇති ආහාර වර්ග ලැයිස්තුව.

සාමාන්‍ය HTML වලදී අපට මේ සඳහා වැග් එකින් එක ලිවීමට සිදු වේ. නමුත් රියැක්ට් වලදී, අප සතුව දත්ත අරාවක් (Array) තිබේ නම්, අපට එම මුළු අරාවම ස්වයංක්‍රීයව ලස්සන ලැයිස්තුවක් බවට පත් කළ හැක.

68.2 map() ශ්‍රිතය භාවිතා කිරීම (The map() Method)

රියැක්ට් හි ලැයිස්තු පෙන්වීමට භාවිතා කරන ප්‍රධානතම ආයුධය වන්නේ ජාවාස්ක්‍රිප්ට් හි map() ශ්‍රිතයයි. (අපි මෙය පරිමාව 3 හි මූලිකව ඉගෙන ගත්තෙමු).

එය ක්‍රියා කරන්නේ මෙලෙසයි: map() ශ්‍රිතය මගින් අරාවක ඇති සෑම අයිතමයක්ම (Item) එකින් එක ගෙන, එයට යම් වෙනසක් සිදු කර (උදා: වැග් එකක් ඇතුළත එනීම), අලුත් අරාවක් අපට ලබා දෙයි.

සරල උදාහරණයක්:

```
function FruitList() {
  const fruits = ["Apple", "Banana",
    "Orange", "Mango"];
```

```
return (
  <ul>
    {fruits.map((fruit, index) => (
      <li key={index}>{fruit}</li>
    ))}
  </ul>
);
}
```

පැහැදිලි කිරීම:

1. { } මගින් JSX තුළ ජාවාස්ක්‍රිප්ට් ලියන බව පවසයි.
2. fruits.map(...) මගින් අරාව කියවීමට පටන් ගනී.
3. එක් එක් පලතුර ගෙන එය වැගයක් ඇතුළත තැන්පත් කර ආපසු එවයි.
4. අවසානයේදී අපට ලැබෙන්නේ වැග් 4ක් ඇති ලැයිස්තුවකි.

68.3 කීප් යනු මොනවාද? (What are Keys?)

ඉහත කේතයේ ඔබ දුටුවා ඇති <li key={index}> ලෙස ලියා තිබෙනවා. මෙය ඉතාමත් වැදගත් කොටසකි.

Key (කී) යනු ලැයිස්තුවක ඇති මූලිකාංග එකිනෙක වෙන් කර හඳුනා ගැනීමට රියැක්ට් භාවිතා කරන අනන්‍ය හැඳුනුම්කාරකයකි (Unique Identifier).

ඇයි කීප් අවශ්‍ය වන්නේ? ලැයිස්තුවක යම් අයිතමයක් මකා දැමූ විට, අලුතින් එක් කළ විට හෝ ඒවායේ පිළිවෙළ මාරු කළ විට, රියැක්ට් හට හරියටම දැනගත යුතුය වෙනස් වූයේ කුමන අයිතමයද කියා. එවිට රියැක්ට් විසින් මුළු ලැයිස්තුවම නැවත අඳින්නේ නැතිව, වෙනස් වූ එක පමණක් යාවත්කාලීන කරයි. මෙය වෙබ් අඩවියේ වේගයට ඉතා වැදගත් වේ.

සැබෑ ලෝකයේ උපමාව (පන්තියක සිසුන්): පන්තියක සිසුන් 40ක් සිටිනවා යැයි සිතන්න. ගුරුවරයා ඔවුන්ව හඳුනා ගන්නේ ඔවුන් වාඩි වී සිටින පුටු අංකයෙන් නම්, එක් අයෙකු අස් වී ගිය විට අනෙක් සියලු දෙනාගේ අංක මාරු වේ. මෙය අවුල් සහගතය. නමුත් සෑම සිසුවෙකුටම "ඇතුළත්

විමේ අංකයක්" (Index Number) තිබේ නම්, ඔහු කොහේ වාඩි වී සිටියත් ගුරුවරයාට ඔහුව නිවැරදිව හඳුනාගත හැක. රියැක්ට් හි Key එක යනු එම ඇතුළත් විමේ අංකයයි.

68.4 වෘත්තීය මට්ටමේ ලැයිස්තු හැසිරවීම (Array of Objects)

සැබෑ මෘදුකාංගවල අරාවක ඇත්තේ සරල අකුරු පමණක් නොවේ. ඒවායේ ඇත්තේ දත්ත රාශියක් ඇති වස්තූන් (Objects) ය.

```
const products = [
  { id: 101, name: "Laptop", price: 150000 },
  { id: 102, name: "Phone", price: 85000 },
  { id: 103, name: "Tablet", price: 45000 }
];
```

```
function Shop() {
  return (
    <div className="product-list">
      {products.map((product) => (
        <div className="card"
          key={product.id}>
            <h3>{product.name}</h3>
            <p>🏷️: ₺. {product.price}</p>
          </div>
        )))}
    </div>
  );
}
```

වෘත්තීය රහස්: සැමවිටම කී එක ලෙස දත්ත ගබඩාවෙන් ලැබෙන ID එක භාවිතා කරන්න. අරාවේ අංකය (index) කී එක ලෙස භාවිතා කිරීම වෘත්තීය මට්ටමේදී අනුමත නොකරයි (පසුව දත්ත පිළිවෙළ මාරු වුවහොත් වැරදි සිදු විය හැකි බැවිනි).

68.5 ASCII රූපසටහන: දත්ත UI බවට පත්වන ආකාරය (Transformation)

[Data Array] [React Logic (map)] [Final UI]

1. {id:1, n:"A"}	-----	Takes Item 1, wraps in <i>	----->
<i>A</i>			
2. {id:2, n:"B"}	-----	Takes Item 2, wraps in <i>	----->
<i>B</i>			
3. {id:3, n:"C"}	-----	Takes Item 3, wraps in <i>	----->
<i>C</i>			

(Browser renders 3 items)

68.6 වෘත්තීය පැහැදිලි කිරීම (Professional Explanation)

මෘදුකාංග ඉංජිනේරු විද්‍යාවේදී මෙය "Data-driven UI" ලෙස හඳුන්වයි. රියැක්ට් විසින් ලැයිස්තු පාලනය කිරීමට භාවිතා කරන ක්‍රියාවලිය Reconciliation ලෙස හැඳින්වේ. මෙහිදී key එක මගින් Virtual DOM එකට පවසන්නේ "මම පරණ කෙනාමයි, මගේ දත්ත විතරයි වෙනස් වුණේ" කියාය. මෙය වෙබ් අඩවිය අතිශය වේගවත් වීමට උපකාරී වේ.

68.7 བཅུག་པའི་འཕྲིམ་ཆུང་ (Common Mistakes)

1. Key එක ලබා දීමට අමතක වීම: එවිට ඔවුසරයේ Console එකේ "Each child in a list should have a unique key prop" යනුවෙන් රතු පැහැති අනතුරු ඇඟවීමක් දිස්වේ.
2. Key එක වැරදි තැනක ලිවීම: ඔබ සංරචකයක් (Custom Component) භාවිතා කරන්නේ නම්, key එක ලබා දිය යුත්තේ එම සංරචකයට මිස එය ඇතුළත ඇති HTML වැගයට නොවේ.

- නිමැවීම: <UserCard key={user.id} data={user} />

3. අන්‍යන්‍ය නොවන කීස්: එකම අංකය කීස් දෙකකට ලබා දීම. (උදා: සිසුන් දෙදෙනෙකුට එකම index අංකය තිබීම වැනිය).

68.8 සාරාංශය (Key Points)

- `map()` මගින් අරාවක ඇති දත්ත JSX ලැයිස්තුවක් බවට හරවයි.
- සෑම ලැයිස්තු අයිතමයකටම අනන්‍ය වූ `key` එකක් තිබිය යුතුය.
- `key` එක රියැක්ට් හට වෙනස්කම් හඳුනා ගැනීමට උදව් වේ.
- හොඳම `key` අගය වන්නේ දත්තවල ඇති `id` එකයි.

68.9 අභ්‍යාස (Exercises)

1. ඔබේ මිතුරන් පස් දෙනෙකුගේ නම් සහ ඔවුන්ගේ දුරකථන අංක අඩංගු අරාවක් සාදා, එය ලස්සන වගුවක් (Table) ලෙස පෙන්වන සංරචකයක් සාදන්න.
2. පහත කේතයේ ඇති දෝෂය කුමක්ද?

```
{items.map(item => {
  <li>{item}</li>
})}
```

(ඉභිය: Arrow function එකෙන් පසුව { } භාවිතා කරන්නේ නම් `return` අවශ්‍ය වේ).

3. පරිශීලකයා ලැයිස්තුවේ ඇති අයිතමයක් ක්ලික් කළ විට, එම අයිතමය පමණක් ඉවත් වන (Delete) පද්ධතියක් සාදන්න. (ඉභිය: `filter()` සහ `useState` භාවිතා කරන්න).

68.10 කුඩා ව්‍යාපෘතිය (Mini Project)

ව්‍යාපෘතිය: සරල "මිලදී ගත යුතු දෑ" ලැයිස්තුව (Shopping List)

1. `shoppingItems` නමින් අරාවක් සාදන්න. එහි භාණ්ඩයේ නම සහ ප්‍රමාණය (Quantity) තිබිය යුතුය.
2. `map()` භාවිතා කර ඒවා තිරයේ පෙන්වන්න.
3. සෑම භාණ්ඩයක් අසලම "ලබා ගන්නා" (Done) නමින් බොත්තමක් සාදන්න.
4. එම බොත්තම එබූ විට, අදාළ භාණ්ඩයේ අකුරු මැදින් ඉරක් වැටෙන සේ සකසන්න. (පරිච්ඡේදය 67 හි ඉගෙන ගත් Conditional Styling භාවිතා කරන්න).

මිළඟ පරිච්ඡේදයෙන් අපි රියැක්ට් හි ඉතාමත් වැදගත් සහ සංකීර්ණ කොටසක් වන "ඉලෙක්ට්ස්"

(`useEffect` Hook) සහ සංරචකයක ජීවන චක්‍රය (Lifecycle) ගැන ඉතාමත් විස්තරාත්මකව ඉගෙන ගනිමු.

හැට නවවන පරිච්ඡේදය: ඉලෙක්ට්ස් - බාහිර ක්‍රියාකාරකම් සහ ජීවන චක්‍රය (Side Effects and `useEffect` Hook)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අධ්‍යයනය කිරීමෙන් පසු ඔබට පහත කරුණු පිළිබඳව අතිශය ගැඹුරු සහ වෘත්තීය අවබෝධයක් ලැබෙනු ඇත:

1. රියැක්ට් හි Side Effects (බාහිර ක්‍රියාකාරකම්) යනු මොනවාදැයි තේරුම් ගැනීම.
2. `useEffect` හුක් එකෙහි අවශ්‍යතාවය සහ එය ක්‍රියා කරන ආකාරය වටහා ගැනීම.
3. සංරචකයක ජීවන චක්‍රය (Component Lifecycle) හෙවත් ස්ථාපනය (Mounting), යාවත්කාලීන කිරීම (Updating) සහ ඉවත් කිරීම (Unmounting) යන අවස්ථා හඳුනා ගැනීම.
4. Dependency Array (යැපීම් අරා) භාවිතා කර `useEffect` ක්‍රියාත්මක වන වාර ගණන පාලනය කිරීම.
5. Cleanup Function (පිරිසිදු කිරීමේ ශ්‍රිතය) මගින් මතකය කළමනාකරණය කරන ආකාරය.
6. සැබෑ ලෝකයේ උදාහරණයක් ලෙස අන්තර්ජාලයෙන් දත්ත ගෙන්වා ගැනීම (API Data Fetching) මූලිකව අත්හදා බැලීම.

69.1 හැදින්වීම: Side Effects (බාහිර ක්‍රියාකාරකම්) යනු මොනවාද?

අපි මෙතෙක් ඉගෙන ගත් රියැක්ට් සංරචකවල ප්‍රධාන කාර්යය වූයේ ස්ටේට් (State) සහ ප්‍රොප්ස් (Props) කියවා බලා ඒවාට ගැලපෙන UI (පෙනුම) එකක් සාදා දීමයි. නමුත් වෙබ් අඩවියක පෙනුම සැකසීමට අමතරව තවත් "බාහිර" වැඩ කිරීමට අපට සිදු වේ.

සැබෑ ලෝකයේ උදාහරණ:

- අන්තර්ජාලයෙන් (API) දත්ත ලබා ගැනීම.
- ඔවුසරයේ මාතෘකාව (`document.title`) වෙනස් කිරීම.
- Timer එකක් හෝ ඔරලෝසුවක් ක්‍රියාත්මක කිරීම.

- සංරචකය තිරයට පැමිණි සැණින් යම් පණිවිඩයක් පෙන්වීම.

මෙලෙස සංරචකයක පෙනුම (Rendering) සැකසීමට අමතරව සිදුවන ඕනෑම ක්‍රියාවක් රියැක්ට් වලදී Side Effect එකක් ලෙස හැඳින්වේ. මෙම ක්‍රියාවන් පාලනය කිරීමට රියැක්ට් අපට ලබා දෙන ප්‍රබලතම අවිය වන්නේ useEffect හුක් එකයි.

69.2 සංරචකයක ජීවන චක්‍රය (Component Lifecycle)

සෑම රියැක්ට් සංරචකයක්ම මනුෂ්‍යයෙකු මෙන් ජීවන චක්‍රයක් පසු කරයි. වෘත්තීය මට්ටමේදී මෙම අවස්ථා තේරුම් ගැනීම අත්‍යවශ්‍ය වේ.

1. Mounting (ස්ථාපනය): සංරචකය ප්‍රථම වරට තිරය මත දිස්වන අවස්ථාව. (උපත වැනිය).
2. Updating (යාවත්කාලීන වීම): ස්ටේට් එකක් හෝ ප්‍රොප් එකක් වෙනස් වීම නිසා සංරචකය නැවත රෙන්ඩර් (Re-render) වන අවස්ථාව. (ජීවිතයේ සිදුවන වෙනස්කම් වැනිය).
3. Unmounting (ඉවත් කිරීම): සංරචකය තිරයෙන් සම්පූර්ණයෙන්ම ඉවත් වන අවස්ථාව. (මරණය වැනිය).

useEffect මගින් අපට පවසන්න පුළුවන් මෙම එක් එක් අවස්ථාවකදී කළ යුතු දේ කුමක්ද කියා.

69.3 useEffect හුක් එකෙහි ව්‍යුහය (Syntax)

useEffect භාවිතා කිරීමට නම් එය මූලික වශයෙන් රියැක්ට් වලින් ලබාගත යුතුය:

import { useEffect } from 'react'; එහි මූලික ව්‍යුහය කොටස් දෙකකින් සමන්විත වේ:

```
useEffect(() => {
```

```
  // 1. සිදුවිය යුතු ක්‍රියාව (The Effect)
```

```
}, [/* 2. යැපීම් අරාම (Dependency Array) */]);
```

69.4 යැපීම් අරාම පාලනය කිරීම (Dependency Array Scenarios)

මෙම හුක් එක ක්‍රියාත්මක වන වාර ගණන තීරණය වන්නේ දෙවනුවට ඇති [] වරහන් මතය. මෙය ඉතාමත් අවධානයෙන් තේරුම් ගත යුතු කොටසකි.

අවස්ථාව 1: අරාම නොමැති විට (No Array)

```
useEffect(() => {
```

```
  console.log("සංරචකය වෙනස් වන සෑම වාරයකදීම මම වැඩ කරනවා!");
```

```
});
```

- සංරචකය මූලිකව පෙන්වන විටත්, ඉන්පසු ස්ටේට් එකක් වෙනස් වන සෑම වතාවකම මෙය ක්‍රියාත්මක වේ. (මෙය බොහෝ විට භාවිතා නොකරයි).

අවස්ථාව 2: හිස් අරාමක් ඇති විට (Empty Array [])

```
useEffect(() => {
```

```
  console.log("මම වැඩ කරන්නේ එකම එක පාරයි (Mounting)!");
```

```
}, []);
```

- මෙය ක්‍රියාත්මක වන්නේ සංරචකය මූලිකව තිරයට පැමිණි විට පමණි. දත්ත ලබා ගැනීමට (API calls) වැඩිපුරම භාවිතා කරන්නේ මෙයයි.

අවස්ථාව 3: අරාම තුළ විචල්‍යයන් ඇති විට ([data])

```
useEffect(() => {
```

```
  console.log("count එක වෙනස් වුණා, මම වැඩ කරනවා!");
```

```
}, [count]);
```

- මූලිකව වැඩ කරන අතර, ඉන්පසු count යන අගය වෙනස් වන සෑම වතාවකම මෙය නැවත ක්‍රියාත්මක වේ.

69.5 පිරිසිදු කිරීමේ ශ්‍රිතය (Cleanup Function)

සමහර ඉලෙක්ට්ස් (Effects) නිසා පරිගණකයේ මතකය අනවශ්‍ය ලෙස වැය විය හැක. උදාහරණයක් ලෙස සංරචකයක් ඉවත් වූ පසුත් ඔරලෝසුවක් ක්‍රියාත්මක වෙමින් තිබීම මතකය පිරීමට

(Memory Leak) හේතු වේ. මෙය නැවැත්වීමට අප Cleanup Function භාවිතා කරමු.

```
useEffect(() => {
    const timer = setInterval(() => {
        console.log("තත්පරයක් ගත වුණා");
    }, 1000);
    // පිරිසිදු කිරීමේ ශ්‍රිතය
    return () => {
        clearInterval(timer); // සංරචකය ඉවත් වන විට ඔරලෝසුව නවත්වන්න
        console.log("මම වැඩේ නතර කළා");
    };
}, []);
```

69.6 ප්‍රායෝගික උදාහරණය: ඔරලෝසුවක් සහ මාතෘකාව වෙනස් කිරීම

```
import { useState, useEffect } from 'react';

function EffectDemo() {
    const [count, setCount] = useState(0);

    // මාතෘකාව වෙනස් කිරීමේ ඉලෙක්ට් එක
    useEffect(() => {
        document.title = `ක්ලික් කිරීම්: ${count}`;

        console.log("බ්‍රවුසර මාතෘකාව වෙනස් කළා");

    }, [count]); // count වෙනස් වන විට පමණක් වැඩ කරයි

    return (
        <div style={{ textAlign: 'center' }}>
            <h1>ඔබ {count} වාරයක් බොත්තම එබුවා</h1>
            <button onClick={() => setCount(count + 1)}>එකතු කරන්න</button>
        </div>
```

69.7 ASCII රූපසටහන: useEffect ජීවන චක්‍රය

```
[ Render Start ]
|
[ UI Displays ]
|
[ useEffect Runs ] <--- ( Mount / Update )
|
[ Cleanup Runs ] <--- ( Before next effect or Unmount )
```

69.8 වෘත්තීය පැහැදිලි කිරීම (Professional Explanation)

මෘදුකාංග ඉංජිනේරු විද්‍යාවේදී `useEffect` යනු "සම්මුහුර්තකරණ මෙවලමකි" (Synchronization Tool). එය ඔබේ සංරචකයේ ඇති ස්ථාවර එක බාහිර ලෝකය සමඟ (උදා: Local Storage, සර්වර් දත්ත) ගැලපීමට උදව් වේ. වෘත්තීය මට්ටමේදී, `useEffect` තුළ අනවශ්‍ය ලෙස ස්ථාවර වෙනස් කිරීමෙන් වළකින්න, මන්ද එය "අනන්ත පුඩු" (Infinite Loops) ඇති කිරීමට ඉඩ ඇති බැවිනි.

69.9 පොදු වැරදි (Common Mistakes)

- Infinite Loops (අනන්ත පුඩු):** `useEffect` එකක් තුළ ස්ථාවර එකක් වෙනස් කිරීම සහ එම ස්ථාවර එකම `Dependency array` එකට ඇතුළත් කිරීම. (Effect එක ස්ථාවර එක වෙනස් කරයි -> ස්ථාවර එක Effect එක නැවත කරයි -> දිගටම සිදුවේ).
- Dependency Array අමතක වීම:** දෙවන පරාමිතිය ලබා නොදුන්නොත් එය සෑම රෙන්ඩර් එකකදීම වැඩ කර වෙබ් අඩවියේ වේගය අඩු කරයි.
- Cleanup අමතක වීම:** `Timer` එකක් හෝ `Event listener` එකක් ඉවත් නොකිරීම නිසා පසුව වෙබ් අඩවිය හිර වීම.

69.10 සාරාංශය (Key Points)

- `useEffect` මගින් බාහිර ක්‍රියාකාරකම් (Side Effects) පාලනය කරයි.
- හිස් අරාවක් `[]` දුන් විට වැඩ කරන්නේ ස්ථාපනය (Mount) වන විට පමණි.
- අරාව තුළ අගයන් දුන් විට ඒවා වෙනස් වන විට `Effect` එක නැවත වැඩ කරයි.
- `return` මගින් පිරිසිදු කිරීමේ (Cleanup) කාර්යයන් සිදු කරයි.
- සංරචකයක ජීවන චක්‍රය Mount, Update සහ Unmount ලෙස කොටස් තුනකි.

69.11 අභ්‍යාස (Exercises)

1. සංරචකය තිරය මත දිස්වන විට (Mount) "සාදරයෙන් පිළිගනිමු" කියාත්, ඉවත් වන විට (Unmount) "නැවත එන්න" කියාත් කන්සෝලයේ මුද්‍රණය වන සංරචකයක් සාදන්න.
2. Input කොටුවක් සාදා, එහි අකුරු වයිජ් කරන විට බ්‍රවුසරයේ මාතෘකාව (Document Title) එම අකුරු බවට පත්වන සේ `useEffect` භාවිතා කරන්න.
3. පහත කේතයේ ඇති දෝෂය කුමක්ද?

```
useEffect(() => {
    setCount(count + 1);
}, [count]);
```

69.12 කුඩා ව්‍යාපෘතිය (Mini Project)

ව්‍යාපෘතිය: සරල තත්පර ගණන් කරන ඔරලෝසුව (Simple Seconds Timer)

1. `Timer.jsx` නමින් සංරචකයක් සාදන්න.
2. එහි `seconds` නමින් ස්ටේට් එකක් සාදා ආරම්භක අගය 0 ලෙස දෙන්න.
3. `useEffect` භාවිතා කර සෑම තත්පරයකට වරක් එම අගය 1 කින් වැඩි කරන්න (`setInterval` භාවිතා කරන්න).
4. සංරචකය ඉවත් වන විට එම කාල ගණකය නැවැත්වීමට (Cleanup) වගබලා ගන්න.
5. තිරයේ එම තත්පර ගණන පෙන්වන්න.

මිලිභ පරිච්ඡේදයෙන් අපි රියැක්ට් හි දත්ත ඇතුළත් කරන පෝරම (Forms) සහ ඒවා වඩාත් කාර්යක්ෂමව පාලනය කරන ආකාරය (Controlled Components) ගැන ඉතාමත් විස්තරාත්මකව ඉගෙන ගනිමු.

හැත්තෑවන පරිච්ඡේදය: පෝරම හැසිරවීම - පාලිත සංරචක (Handling Forms - Controlled Components)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අධ්‍යයනය කිරීමෙන් පසු ඔබට පහත කරුණු පිළිබඳව අතිශය ගැඹුරු සහ

වෘත්තීය අවබෝධයක් ලැබෙනු ඇත:

1. සාමාන්‍ය HTML පෝරම (Forms) සහ රියැක්ට් පෝරම අතර ඇති දාර්ශනික වෙනස වටහා ගැනීම.
2. පාලිත සංරචක (Controlled Components) යනු මොනවාදැයි තේරුම් ගැනීම.
3. ස්ටේට් (State) භාවිතා කරමින් ඉන්පුට් (Input) කොටුවල අගයන් තත්පර කාලීනව (Real-time) පාලනය කරන ආකාරය.
4. පෝරමයක ඇති විවිධ දත්ත වර්ග (Text, Textarea, Select, Checkbox) හැසිරවීම.
5. තනි ශ්‍රිතයක් (Single Handler) මගින් ඉන්පුට් කොටු කිහිපයක් පාලනය කරන වෘත්තීය ක්‍රමය ඉගෙන ගැනීම.
6. පෝරමයක් යැවීමේදී (Submit) දත්ත ලබා ගන්නා සහ මූලික වැරදි පරීක්ෂා කරන (Validation) ආකාරය.

70.1 හැඳින්වීම: රියැක්ට් පෝරමවල විශේෂත්වය

අපි 39 වන පරිච්ඡේදයේදී (පරිමාව 3)

සාමාන්‍ය ජාවාස්ක්‍රිප්ට් මගින් පෝරම පාලනය කරන ආකාරය ඉගෙන ගත්තෙමු. එහිදී අප කළේ පෝරමය යවන මොහොතේදී (Submit) පමණක් එහි ඇති දත්ත කියවා බැලීමයි.

නමුත් රියැක්ට් වලදී අප අනුගමනය කරන්නේ මීට වඩා දියුණු ක්‍රමයකි. අපට අවශ්‍ය වන්නේ පරිශීලකයා අකුරක් ටයිප් කරන සෑම මොහොතකම එම දත්තය රියැක්ට් හි ස්ටේට් (State) එක තුළ තබා ගැනීමටයි. මෙයට **Controlled Components** (පාලිත සංරචක) යැයි කියනු ලැබේ.

සැබෑ ලෝකයේ උපමාව (අශ්වයෙකු පැදවීම):

- සාමාන්‍ය HTML පෝරමය: අශ්වයාට තනිවම යාමට ඉඩ දී, ගමන අවසානයේදී ඔහු සිටින තැන පරීක්ෂා කිරීම වැනිය.

- රියැක්ට් පාලිත පෝරමය: අශ්වයාගේ ලඟුව (State) ඔබේ අතේ තබාගෙන, ඔහු තබන සෑම අඩියක්ම (Every keystroke) ඔබ පාලනය කිරීම වැනිය. අශ්වයා කොහේ සිටිය යුතුද යන්න තීරණය කරන්නේ ඔබයි.

70.2 පාලිත සංරචක (Controlled Components) යනු කුමක්ද?

පාලිත සංරචකයකදී, HTML ඉන්පුට් කොටුවේ අගය (value) තීරණය වන්නේ රියැක්ට් ස්ටේට් එක මගිනි. පරිශීලකයා යමක් ටයිප් කළ විට:

1. onChange සිදුවීම (Event) ක්‍රියාත්මක වේ.
2. එමගින් ස්ටේට් (State) එක යාවත්කාලීන වේ.
3. ස්ටේට් එක වෙනස් වූ නිසා සංරචකය නැවත රෙන්ඩර් වේ.
4. අලුත් අගය ඉන්පුට් කොටුවේ දිස්වේ.

මෙමගින් දත්තවල "එකම සත්‍ය මූලාශ්‍රය" (Single Source of Truth) වන්නේ රියැක්ට්

ස්ටේට් එකයි.

70.3 මූලික පාලිත ඉන්පුට් එකක් නිර්මාණය කිරීම

අපි දැන් නමක් ඇතුළත් කරන සරල පෝරමයක් සාදමු.

```
import { useState } from 'react';

function NameForm() {

  const [name, setName] = useState("");

  const handleChange = (e) => {

    // පරිශීලකයා ටයිප් කරන අකුරු ස්ටේට් එකට දමමු

    setName(e.target.value);

  };

  const handleSubmit = (e) => {

    e.preventDefault(); // පිටුව Refresh වීම නවත්වමු

    alert("ඇතුළත් කළ නම: " + name);
```

```
  };

  return (

    <form onSubmit={handleSubmit}>

      <label>

        නම:

        <input

          type="text"

          value={name}      // ස්ටේට් එකෙන් අගය ගනී

          onChange={handleChange} // ටයිප් කරන විට ස්ටේට් එක වෙනස් කරයි

        />

      </label>

      <button type="submit">යවන්න</button>

      <p>ඔබ ටයිප් කරන්නේ: {name}</p>

    </div>

  );

}
```

වැදගත්: මෙහි value={name} සහ onChange={handleChange} යන දෙකම තිබීම අනිවාර්ය වේ.

value පමණක් තිබේ නම් ඔබට කිසිවක් ටයිප් කළ නොහැක (එය Read-only වේ).

70.4 විවිධ වර්ගයේ පෝරම කොටස් (Textarea, Select, Checkbox)

රියැක්ට් මෙම සියලුම කොටස් පාලනය කරන්නේ එකම ආකාරයකටය.

1. Textarea (දිගු විස්තර)

```
const [bio, setBio] = useState("");

<textarea value={bio} onChange={e => setBio(e.target.value)} />
```

2. Select (Dropdown මෙනු)

```
const [city, setCity] = useState("colombo");

<select value={city} onChange={(e) =>
  setCity(e.target.value)}>

  <option value="colombo">කොළඹ</option>

  <option value="kandy">මහනුවර</option>

  <option value="galle">ගාල්ල</option>

</select>
```

3. Checkbox (තේරීම)

වෙබ් බොක්ස් එකකදී අප භාවිතා කරන්නේ value නොව checked යන ගුණාංගයයි.

```
const [isAgreed, setIsAgreed] =
  useState(false);

<input

  type="checkbox"

  checked={isAgreed}

  onChange={(e) =>
    setIsAgreed(e.target.checked)} // value
    නොව checked බලන්න

/>
```

70.5 වෘත්තීය රහස්: ඉන්පුට් කොටු කිහිපයක් එකවර පාලනය කිරීම

ඔබේ පෙරමයේ කොටු 10ක් තිබේ නම්, useState 10ක් සහ handleChange ශ්‍රිත 10ක් ලිවීම ඉතා අපහසුය. වෘත්තීය මට්ටමේදී අප කරන්නේ එක් Object එකක් සහ එක් ශ්‍රිතයක් භාවිතා කිරීමයි.

ඒ සඳහා එක් එක් ඉන්පුට් එකට අනිවාර්යයෙන්ම name attribute එකක් තිබිය යුතුය.

```
import { useState } from 'react';

function RegistrationForm() {

  // සියලුම දත්ත එකම Object එකක ගබඩා කරමු

  const [formData, setFormData] =
    useState({
```

```
  username: "",

  email: "",

  password: ""

});

const handleInputChange = (e) => {

  // e.target.name මගින් කුමන කොටුවද කියා
  හඳුනා ගනී

  // e.target.value මගින් එහි අගය ලබා ගනී

  const { name, value } = e.target;

  setFormData({

    ...formData, // පරණ දත්ත සියල්ල මෙහි
    තබා ගන්න (Spread Operator)

    [name]: value // වෙනස් වූ කොටුව පමණක්
    අලුත් කරන්න

  });

};

return (

  <form>

    <input name="username"
      placeholder="Name"
      onChange={handleInputChange} />

    <input name="email"
      placeholder="Email"
      onChange={handleInputChange} />

    <input name="password"
      type="password" placeholder="Pass"
      onChange={handleInputChange} />

    <button onClick={() =>
      console.log(formData)}>Log
      Data</button>

  </form>

);
}
```

70.6 ASCII රූපසටහන: පාලිත සංරචකයක දත්ත වක්‍රය

[පරිශීලකයා ටයිප් කරයි]

v

[onChange Event]

v

[setState(e.target.value)]

v

[React State Updates]

v

[Component Re-renders]

v

[Input Value Updates UI]

70.7 පෝරම වලංගුතාවය (Basic Validation)

දත්ත යැවීමට පෙර ඒවා නිවැරදිදැයි පරීක්ෂා කිරීම වෘත්තීය ලක්ෂණයකි.

```
const handleSubmit = (e) => {
  e.preventDefault();

  if (formData.username.length < 3) {
    alert("නම අකුරු 3කට වඩා දිග විය යුතුය!");
    return;
  }

  if (!formData.email.includes("@")) {
    alert("නිවැරදි Email එකක් ඇතුළත් කරන්න!");
    return;
  }

  // සියලුම නිවැරදි නම් සර්වර් එකට යවන්න
```

```
console.log("Success!", formData);
};
```

70.8 වෘත්තීය පැහැදිලි කිරීම (Professional Explanation)

මහා පරිමාණ රියැක්ට් මෘදුකාංග වලදී (උදා: කොටු 50ක් ඇති පෝරමයක්), අප ඉහත කී `useState` ක්‍රමය භාවිතා කිරීම නිසා වෙබ් අඩවියේ වේගය මදක් අඩු විය හැක (මන්ද සෑම අකුරකටම මුළු පිටුවම Re-render වන බැවිනි). එවැනි අවස්ථාවලදී වෘත්තීය සංවර්ධකයන් `React Hook Form` හෝ `Formik` වැනි බාහිර පුස්තකාල භාවිතා කරයි. නමුත් මූලික මට්ටමේදී ඉහත ඉගෙන ගත් "Controlled Components" සංකල්පය 100% ක් දැන සිටීම අත්‍යවශ්‍ය වේ.

70.9 පොදු වැරදි (Common Mistakes)

1. Spread Operator (...) අමතක වීම: `setFormData({ name: value })` ලෙස ලියුවහොත්, පෝරමයේ තිබූ අනෙකුත් සියලුම දත්ත (Email, Password) මැකී යයි. පරණ දත්ත තබා ගැනීමට සැමවිටම `...formData` භාවිතා කරන්න.
2. Initial Value එකක් ලබා නොදීම: `useState()` ලෙස හිස්ව තැබුවහොත්, පළමු අකුර ටයිප් කරන තෙක් එය "Uncontrolled" ලෙස සලකා බුද්ධිමත් Error එකක් පෙන්වනු ඇත. සැමවිටම `useState("")` ලෙස හිස් අකුරු වැලක් ලබා දෙන්න.
3. Name Attribute එක වැරදීම: HTML හි ඇති `name` එක සහ `handleInputChange` තුළ භාවිතා කරන නම එකම විය යුතුය.

70.10 සාරාංශය (Key Points)

- Controlled Components වලදී ස්ටේට් එක මගින් ඉන්පුට් කොටුව පාලනය කරයි.
- value සහ onChange යනු පාලිත පෝරමයක ප්‍රධාන කොටස් දෙකයි.
- Spread Operator එක මගින් කොටු කිහිපයක දත්ත කළමනාකරණය පහසු කරයි.
- `e.preventDefault()` මගින් පිටුව Refresh වීම වළක්වයි.
- Checkbox සඳහා value වෙනුවට checked භාවිතා කරයි.

70.11 අභ්‍යාස (Exercises)

1. පරිශීලකයාට නම, වයස සහ පදිංචි නගරය ඇතුළත් කර "Register" බොත්තම එබූ විට එම දත්ත ලේඛනයට තීරය මත දිස්වන පද්ධතියක් සාදන්න.
2. මුරපද (Password) කොටු දෙකක් සාදන්න. ඒවා දෙකෙහිම ටයිප් කර ඇති දෑ සමාන දැයි (Confirm Password) තර්ක කාලීනව පරීක්ෂා කර පරීක්ෂණයක් පෙන්වන්න.
3. Checkbox එකක් සාදා එය Click කළ විට පමණක් "Submit" බොත්තම සක්‍රීය (Enable) වන ලෙස සකස්න්න. (ඉතිරි: <button disabled={!isAgreed}>).

70.12 කුඩා ව්‍යාපෘතිය (Mini Project)

ව්‍යාපෘතිය: ප්‍රතිපෝෂණ පද්ධතිය (Feedback Form)

1. පරිශීලකයාගේ නම (Text Input).
2. ලකුණු ලබා දීම (Select: 1 සිට 5 දක්වා).
3. අදහස් දැක්වීම (Textarea).
4. "නැවත සම්බන්ධ කර ගන්න" (Checkbox).
5. Submit කළ පසු එම දත්ත සියල්ල ස්ථාවර එකෙන් ඉවත් කර (Reset) සාර්ථක පණිවිඩයක් පෙන්වන්න.

මූලික පරිච්ඡේදයෙන් අපි රියැක්ට් වෙබ් අඩවියක් බහු-පිටු (Multi-page) මෘදුකාංගයක් බවට පත් කරන ආකාරය හෙවත් "රවුටින්" (React Router) ගැන ඉතාමත් විස්තරාත්මකව ඉගෙන ගනිමු.

හැත්තෑ එක්වන පරිච්ඡේදය: රියැක්ට් රවුටින් - බහු-පිටු මෘදුකාංග නිර්මාණය (React Router - Multi-page Applications)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අධ්‍යයනය කිරීමෙන් පසු ඔබට පහත කරුණු පිළිබඳව අතිශය ගැඹුරු සහ වෘත්තීය අවබෝධයක් ලැබෙනු ඇත:

1. තනි පිටුවේ මෘදුකාංග (Single Page Applications - SPA) සහ බහු-පිටු මෘදුකාංග (Multi-page Applications) අතර වෙනස වටහා ගැනීම.
2. රියැක්ට් රවුටර් (React Router) යනු කුමක්ද සහ එය ස්ථාපනය කරන ආකාරය.

3. රවුටින් සඳහා අවශ්‍ය මූලික සංරචක (BrowserRouter, Routes, Route) හඳුනා ගැනීම.
4. පිටුව Refresh නොවී එක් තැනක සිට තවත් තැනකට ගමන් කිරීමට Link සහ NavLink භාවිතා කරන ආකාරය.
5. වෙබ් ලිපිනය (URL) වැරද්දක් වීම "404 Not Found" පිටුවක් පෙන්වන ආකාරය.
6. වෙබ් ලිපිනය හරහා දත්ත යවන ආකාරය (URL Parameters) මූලිකව හඳුනා ගැනීම.

71.1 හැඳින්වීම: රවුටින් (Routing) යනු කුමක්ද?

සාමාන්‍ය වෙබ් අඩවියක ඔබ "About" පිටුවට යාමට ලින්ක් එකක් ක්ලික් කළ විට, බ්‍රවුසරය විසින් සර්වර් එකෙන් අලුත්ම HTML ගොනුවක් ඉල්ලා සිටියි. එවිට මුළු වෙබ් පිටුවම "Refresh" වී අලුතින් ලෝඩ් වේ.

නමුත් රියැක්ට් යනු Single Page Application (SPA) තාක්ෂණයකි. මෙහි ඇත්තේ එකම එක HTML ගොනුවක් පමණි. රවුටින් මගින් සිදු කරන්නේ, පරිශීලකයා වෙබ් ලිපිනය වෙනස් කරන විට (උදා: /about ලෙස), පිටුව Refresh කරන්නේ නැතිව, පවතින සංරචකය (Component) ඉවත් කර ඒ වෙනුවට අලුත් සංරචකයක් තීරයට ගෙන ඒමයි.

සැබෑ ලෝකයේ උපමාව (වේදිකාවක්): සිතන්න ඔබ වේදිකා නාට්‍යයක් නරඹනවා කියා.

- පැරණි ක්‍රමය: දර්ශනයක් අවසන් වූ පසු ඔබ නාට්‍ය ශාලාවෙන් පිටතට ගොස් නැවත ඇතුළු වීම වැනිය (Page Refresh).

- රියැක්ට් රවුටින් ක්‍රමය: ඔබ ශාලාව ඇතුළතම වාඩි වී සිටියදී, තීරය වසා වේදිකාවේ ඇති බඩු බාහිරාදිය (Components) පමණක් මාරු කිරීම වැනිය. ඔබට දැනෙන්නේ ඔබ අලුත් තැනකට පැමිණි බවයි, නමුත් ඔබ සිටින්නේ එකම තැනකයි.

71.2 රියැක්ට් රවුටර් ස්ථාපනය කිරීම (Installation)

රවුටින් යනු රියැක්ට් හි අභ්‍යන්තරයේම එන දෙයක් නොවේ. ඒ සඳහා අප react-router-dom නමැති බාහිර පුස්තකාලය ස්ථාපනය කළ යුතුය. ඔබේ ව්‍යාපෘතියේ ටර්මිනලය (Terminal) විවෘත කර පහත විධානය ලබා දෙන්න:

```
npm install react-router-dom
```

71.3 මූලික සංරචක 4 (The Big 4 Components)

රවුටින් ක්‍රියාවලිය සඳහා අපට ප්‍රධාන සංරචක 4ක් අවශ්‍ය වේ:

1. **BrowserRouter:** මෙය මුළු වෙබ් අඩවියම වටා ඇති "ආවරණය" වැනිය. වෙබ් ලිපිනයේ සිදුවන වෙනස්කම් නිරීක්ෂණය කරන්නේ මෙයයි.
2. **Routes:** මෙය අපේ සියලුම පාරවල් (Routes) අඩංගු වන පෙට්ටියයි.
3. **Route:** මෙයින් පවසන්නේ එක් විශේෂිත වෙබ් ලිපිනයකට අදාළව පෙන්විය යුතු සංරචකය කුමක්ද කියාය.
4. **Link:** සාමාන්‍ය HTML හි `<a>` ටැගය වෙනුවට භාවිතා කරන රියැක්ට් ලින්ක් එකයි.

71.4 පියවරෙන් පියවර: බහු-පිටු වෙබ් අඩවියක් සාදමු

අපි දැන් Home, About සහ Contact පිටු සහිත සරල රියැක්ට් මෘදුකාංගයක් සාදමු.

පියවර 1: මූලික සංරචක සාදා ගැනීම පළමුව පිටු සඳහා වෙන වෙනම ගොනු සාදා ගන්න

(Home.jsx, About.jsx, Contact.jsx).

පියවර 2: App.jsx තුළ පාරවල් සැකසීම

```
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';
```

```
import Home from './Home';
```

```
import About from './About';
```

```
import Contact from './Contact';
```

```
function App() {
```

```
  return (
```

```
    <BrowserRouter>
```

```
      /* 1. මෙහි එක (Navigation) */
```

```
      <nav>
```

```
        <ul>
```

```
          <li><Link to="/">මුල්  
පිටුව</Link></li>
```

```
          <li><Link to="/about">මා  
ගැන</Link></li>
```

```
          <li><Link to="/contact">සම්බන්ධ  
වන්න</Link></li>
```

```
        </ul>
```

```
      </nav>
```

```
      /* 2. පිටු මාරු වන ස්ථානය */
```

```
      <Routes>
```

```
        <Route path="/" element={<Home />}  
      />
```

```
        <Route path="/about"  
      element={<About />} />
```

```
        <Route path="/contact"  
      element={<Contact />} />
```

```
      </Routes>
```

```
    </BrowserRouter>
```

```
  );
```

```
}
```

```
export default App;
```

කොටස් පැහැදිලි කිරීම:

- `path="/about"`: ඔවුසරයේ ලිපිනය `mysite.com/about` වූ විට...
- `element={<About />}`: ...About සංරචකය පෙන්වන්න.
- `to="/contact"`: ලින්ක් එක ක්ලික් කළ විට `/contact` ලිපිනයට යන්න.

71.5 ඇයි `<a>` ටැගය භාවිතා නොකරන්නේ?

මෙය ඉතා වැදගත් ප්‍රශ්නයකි. සාමාන්‍ය HTML හි ලින්ක් එකක් සෑදීමට අප භාවිතා කරන්නේ

 වැගයයි. නමුත් රියැක්ට් වලදී එය භාවිතා කළහොත්:

1. බ්‍රවුසරය මුළු පිටුවම Refresh කරයි.
2. රියැක්ට් හි ඇති සියලුම ස්ටේට් (State) දත්ත මැකී යයි.
3. වෙබ් අඩවිය මන්දගාමී වේ.

Link භාවිතා කළ විට: රියැක්ට් රවුටර් විසින් බ්‍රවුසරය Refresh වීම නවත්වා, පිටුවේ මාරු විය යුතු කොටස පමණක් මාරු කරයි. මෙය ඉතා වේගවත්ය.

71.6 වැරදි ලිපින පාලනය (404 Not Found Page)

යමෙකු අපේ වෙබ් අඩවියේ නැති ලිපිනයක් (උදා: /abc) ටයිප් කළහොත් පෙන්වීමට පිටුවක් අප සැකසිය යුතුය. ඒ සඳහා path="*" භාවිතා කරයි.

```
<Routes>
```

```
  <Route path="/" element={<Home />} />
```

```
  <Route path="/about" element={<About />} />
```

```
  {/* ඉහත කිසිවකට නොගැලපෙන ඕනෑම ලිපිනයක් සඳහා */}
```

```
  <Route path="*" element={<h1>සමාවන්න, මෙම පිටුව සොයාගත නොහැක! (404)</h1>} />
```

```
</Routes>
```

71.7 ASCII රූපසටහන: රවුටින් ක්‍රියාවලිය

```
[ බ්‍රවුසරයේ URL එක ] ---- ( /about )
```

```
v
```

```
[ BrowserRouter ] <--- (ලිපිනය වෙනස් වූ බව දැන ගනී)
```

```
|
```

```
[ Routes Box ]
```

```
|
```

```
(මෙය /about ද?) --(ඔව්)--> [ About Component එක පෙන්වයි ]
```

```
|
```

```
(මෙය /contact ද?) --(නැත)--> (රිළඟට යයි)
```

71.8 උසස් විශේෂාංගය: NavLink (Active Links)

ඔබ දැනට ඉන්නේ කුමන පිටුවකද යන්න මෙනුවේ පෙන්වීමට (උදා: ඉන්නේ Home පිටුවේ නම් එහි අකුරු තදින් පෙන්වීමට) Link වෙනුවට NavLink භාවිතා කළ හැක. එය ස්වයංක්‍රීයව අදාළ ලින්ක් එකට active නමැති Class එක ලබා දෙයි.

```
<NavLink
```

```
  to="/about"
```

```
  style={({ isActive }) => ({ color: isActive ? 'red' : 'black' })}
```

```
>
```

```
  මා ගැන
```

```
</NavLink>
```

71.9 වෘත්තීය පැහැදිලි කිරීම (Professional Explanation)

මෘදුකාංග ඉංජිනේරු විද්‍යාවේදී මෙය Client-side Routing ලෙස හඳුන්වයි. රියැක්ට් රවුටර් විසින් බ්‍රවුසරයේ History API එක භාවිතා කරමින් වෙබ් ලිපිනය පාලනය කරයි. වෘත්තීය මට්ටමේදී අප "Nested Routes" (එකක් ඇතුළත තවත් එකක් ඇති පාරවල්) සහ "Protected Routes" (ලොගින් වූ අයට පමණක් පෙනෙන පාරවල්) වැනි දියුණු ක්‍රමවේද භාවිතා කරමු.

71.10 පොදු වැරදි (Common Mistakes)

1. Routes වැගයෙන් පිටත Route ලිවීම: සියලුම Route වැග් අනිවාර්යයෙන්ම Routes වැගය ඇතුළත තිබිය යුතුය.
2. BrowserRouter අමතක වීම: ඔබේ මුළු ඇප් එකම BrowserRouter එකකින් වට කර නැතිනම් රවුටින් ක්‍රියා නොකරයි.
3. Path මුලට ඇල ඉර (/) අමතක වීම: path="about" නොව path="/about" ලෙස ලිවිය යුතුය.

71.11 සාරාංශය (Key Points)

- රවුටින් මගින් SPA වෙබ් අඩවියක් බහු-පිටු මෘදුකාංගයක් බවට පත් කරයි.
- react-router-dom යනු ඒ සඳහා භාවිතා කරන ප්‍රධාන පුස්තකාලයයි.
- පිටුව Refresh වීම වැළැක්වීමට Link හෝ NavLink භාවිතා කළ යුතුය.
- path="*" මගින් 404 වැරදි පාලනය කළ හැක.
- සියලුම පාරවල් Routes ඇතුළත අර්ථ දැක්විය යුතුය.

71.12 අභ්‍යාස (Exercises)

1. Navbar නමින් වෙනම සංරචකයක් සාදා එය App.jsx හි BrowserRouter තුළ ඇතුළත් කර පෙන්වන්න.
2. පිටු පහළට යන විට පෙනෙන සේ ලස්සන Footer එකක් සාදා එය සියලුම පිටු වලට පොදුවේ පෙනෙන සේ සැකසුම් කරන්න. (ඉභිය: එය Routes වැගයට පිටතින් ලිවිය යුතුය).
3. ලොගින් පිටුවක් (Login.jsx) සාදා, එහි ඇති බොත්තමක් එබූ විට Dashboard එක වෙත රැගෙන යන ලෙස සැකසීමට උත්සාහ කරන්න. (ඉභිය: useNavigate හුක් එක ගැන අන්තර්ජාලයෙන් සොයා බලන්න).

71.13 කුඩා ව්‍යාපෘතිය (Mini Project)

ව්‍යාපෘතිය: පුද්ගලික පෝට්ෆෝලියෝව (Multi-page Portfolio)

1. පහත පිටු 4 සාදන්න: Home, Projects, Skills, Contact.
2. රියැක්ට් රවුටර් භාවිතා කර ඒවා එකිනෙක සම්බන්ධ කරන්න.
3. ලස්සන Navigation Bar එකක් සාදා NavLink භාවිතා කර ඔබ සිටින පිටුව කැපී පෙනෙන සේ (Highlight) සකසන්න.
4. වැරදි ලිපිනයක් ගැසූ විට "සමාවන්න, ඔබ වැරදි තැනකට පැමිණ ඇත" යනුවෙන් පණිවිඩයක් සහ "නැවත මුල් පිටුවට" යන ලින්ක් එක සහිත 404 පිටුවක් සාදන්න.

මිලිභ පරිච්ඡේදයෙන් අපි රියැක්ට් හි මුළු වෙබ් අඩවියටම බලපාන දත්ත පාලනය කරන ආකාරය හෙවත් "කොන්ටෙක්ස්ට් ඒ.පී.අයි" (Context API) ගැන ඉතාමත් විස්තරාත්මකව ඉගෙන ගනිමු.

හැත්තෑ දෙනෙහි පරිච්ඡේදය:

කොන්ටෙක්ස්ට් ඒ.පී.අයි - ගෝලීය දත්ත පාලනය (Context API - Global State Management)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අධ්‍යයනය කිරීමෙන් පසු ඔබට පහත කරුණු පිළිබඳව අතිශය ගැඹුරු සහ වෘත්තීය අවබෝධයක් ලැබෙනු ඇත:

1. Prop Drilling (ප්‍රොප් ඩ්‍රිලින්) යනු කුමක්ද සහ විශාල මෘදුකාංගවලදී එය ගැටලුවක් වන්නේ ඇයි දැයි තේරුම් ගැනීම.
2. Context API යනු කුමක්ද සහ එය රියැක්ට් හි දත්ත කළමනාකරණය පහසු කරන්නේ කෙසේදැයි වටහා ගැනීම.
3. createContext භාවිතා කර නව සන්දර්භයක් (Context) නිර්මාණය කරන ආකාරය.
4. Provider (සපයන්නා) මගින් දත්ත මුළු මෘදුකාංගයටම බෙදා හරින ආකාරය.
5. useContext හුක් එක භාවිතා කර ඕනෑම සංරචකයක සිට දත්ත ලබා ගන්නා ආකාරය.
6. සැබෑ ලෝකයේ උදාහරණයක් ලෙස "වර්ණ තේමාව" (Theme - Dark/Light Mode) පාලනය කරන පද්ධතියක් ගොඩනැගීම.

72.1 හැඳින්වීම: ප්‍රොප් ඩ්‍රිලින් ගැටලුව (The Problem of Prop Drilling)

අපි 64 වන පරිච්ඡේදයේදී ඉගෙන ගත්තා ප්‍රොප්ස් (Props) භාවිතා කරමින් දෙමාපිය සංරචකයේ සිට දරු සංරචකයට දත්ත යවන ආකාරය. මෙය කුඩා වෙබ් අඩවිවලට ඉතා හොඳ ක්‍රමයකි.

නමුත් සංරචක ගස (Component Tree) ඉතා ගැඹුරු වන විට කුමක් සිදුවේද?

සිතන්න ඔබේ App සංරචකයේ පරිශීලකයාගේ නම (userName) තිබෙනවා කියා. මෙම නම පෙන්විය යුත්තේ ඉතාමත් පහළින් ඇති Profile සංරචකයේය. නමුත් එම Profile එකට යාමට මැදින් තවත් සංරචක 5ක් පසු කළ යුතුය.

Prop Drilling: App -> Header -> Navbar -> UserMenu -> Settings -> Profile

මෙහිදී Header, Navbar, UserMenu, සහ Settings යන සංරචකවලට මෙම userName දත්තයෙන් කිසිදු වැඩක් නැත. නමුත් ඒවා කළ

යුත්තේ ඉහළින් එන දත්තය අල්ලාගෙන ඊළඟ දරුවාට ලබා දීම පමණි. මෙය ඉතාමත් වෙනස්කර සහ වැරදි සිදු වීමට ඉඩ ඇති ක්‍රමයකි. මෙයට Prop Drilling යැයි කියනු ලැබේ.

72.2 සැබෑ ලෝකයේ උපමාව: ගුවන්විදුලි විකාශනය (Radio Station)

- Prop Drilling ක්‍රමය: ලියුමක් එක් අයෙකුගෙන් තවත් අයෙකුට අතින් අත යවා ගමනාන්තයට රැගෙන යාම වැනිය. මැද සිටින සියලු දෙනා එම ලියුම අතේ තබා ගත යුතුය.

- Context API ක්‍රමය: ගුවන්විදුලි මධ්‍යස්ථානයක් වැනිය. මධ්‍යස්ථානය (Provider) විසින් දත්ත වාතයට මුදා හරියි (Broadcast). ඕනෑම කෙනෙකුට (Consumer) තමා සතු රේඩියෝව ක්‍රියාත්මක කර (useContext) එම දත්තයට සවන් දිය හැක. මැද සිටින අය ඒ ගැන කරදර විය යුතු නැත.

72.3 කොන්ටෙක්ස්ට් ඒ.පී.අයි යනු කුමක්ද? (What is Context API?)

Context API යනු රියැක්ට් විසින්ම ලබා දෙන විශේෂ තාක්ෂණයකි. මෙහි අරමුණ වන්නේ වෙබ් අඩවියේ ඇති ඕනෑම සංරචකයකට, ප්‍රොප්ස් (Props) හරහා දත්ත යවන්නේ නැතිව, සෘජුවම දත්ත ලබා ගැනීමට ඉඩ සැලසීමයි. මෙය Global State Management (ගෝලීය දත්ත පාලනය) ලෙස හැඳින්වේ.

ප්‍රධාන භාවිතයන්:

- වර්ණ තේමාවන් (Dark/Light mode).
- පරිශීලකයාගේ ලොගින් තොරතුරු (User Authentication).
- භාෂාව තේරීම (Language/Localization).
- සාපේක්ෂ කරත්තයක ඇති බඩු ලැයිස්තුව (Shopping Cart).

72.4 කොන්ටෙක්ස්ට් ක්‍රියාවලියේ පියවර 3 (The 3 Steps)

රියැක්ට් හි Context භාවිතා කිරීමට ප්‍රධාන පියවර 3ක් අනුගමනය කළ යුතුය.

පියවර 1: Context එක නිර්මාණය කිරීම (createContext)

මුලින්ම අප දත්ත ගබඩා කිරීමට "නාලිකාවක්" හෝ සන්දර්භයක් නිර්මාණය කළ යුතුය.

```
import { createContext } from 'react';
```

```
// 'ThemeContext' නමින් අලුත් සන්දර්භයක්  
සාදමු
```

```
export const ThemeContext =  
createContext();
```

පියවර 2: Provider මගින් දත්ත බෙදා හැරීම (The Provider)

දැන් අපට මෙම දත්තය අවශ්‍ය වන සියලුම සංරචක වටා "සපයන්නා" (Provider) තැබිය යුතුය.

```
function App() {
```

```
  const theme = "dark"; // අපට බෙදා හැරීමට  
  අවශ්‍ය දත්තය
```

```
  return (
```

```
    <ThemeContext.Provider  
      value={theme}>
```

```
      <Header />
```

```
      <MainContent />
```

```
      <Footer />
```

```
    </ThemeContext.Provider>
```

```
  );
```

```
}
```

මෙහි value ගුණාංගය හරහා අපට ඕනෑම දත්තයක් (Strings, Numbers, Objects, Functions) යැවිය හැක.

පියවර 3: දත්ත ලබා ගැනීම (useContext Hook)

ඕනෑම දරු සංරචකයක සිට ප්‍රොප්ස් භාවිතා නොකර මෙම දත්ත ලබා ගන්නේ මෙලෙසයි.

```
import { useContext } from 'react';
```

```
import { ThemeContext } from './App'; //  
මුලින් සෑදූ Context එක ගෙන්වා ගන්න
```

```
function Profile() {
```

```
const currentTheme =
useContext(ThemeContext); // දත්තය සාප්තවම
ලබා ගැනීම

return <div>දැනට පවතින තේමාව:
{currentTheme}</div>;

}
```

72.5 ප්‍රායෝගික ව්‍යාපෘතිය: තේමාවන් මාරු කරන පද්ධතිය (Theme Switcher)

අපි දැන් බොත්තමක් එබූ විට මුළු වෙබ් අඩවියේම වර්ණය මාරු වන සැබෑ පද්ධතියක් සාදමු.

ගොනුව: ThemeContext.jsx

```
import { createContext, useState } from
'react';

export const MyTheme = createContext();

export const ThemeProvider = ({ children })
=> {

  const [dark, setDark] = useState(false);

  const toggleTheme = () => setDark(!dark);

  return (

    <MyTheme.Provider value={{ dark,
toggleTheme }}>

      {children}

    </MyTheme.Provider>

  );

};
```

ගොනුව: App.jsx

```
import { ThemeProvider } from
'./ThemeContext';

import Content from './Content';

function App() {
```

```
  return (

    <ThemeProvider>

      <div className="App">

        <h1>Context API ඉගෙන ගනිමු</h1>

        <Content />

      </div>

    </ThemeProvider>

  );

}
```

ගොනුව: Content.jsx

```
import { useContext } from 'react';

import { MyTheme } from
'./ThemeContext';

function Content() {

  const { dark, toggleTheme } =
useContext(MyTheme);

  const styles = {

    backgroundColor: dark ? '#333' : '#eee',

    color: dark ? '#fff' : '#000',

    padding: '50px'

  };

  return (

    <div style={styles}>

      <h2>මෙය අන්තර්ගතයයි</h2>

      <button onClick={toggleTheme}>

        {dark ? "Light Mode එකට මාරු වන්න" :
"Dark Mode එකට මාරු වන්න"}

      </button>

    </div>

  );

}
```

```

</div>

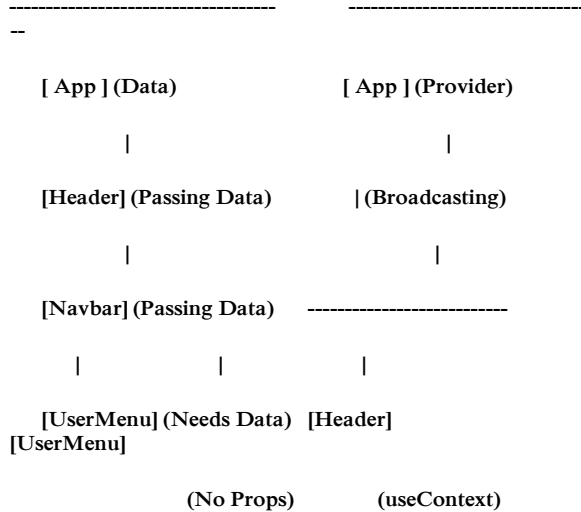
);

}

```

72.6 ASCII රූපසටහන: Prop Drilling එදිරිව Context

Prop Drilling (පැරණි ක්‍රමය): Context API (නවීන ක්‍රමය):



72.7 වෘත්තීය පැහැදිලි කිරීම (Professional Explanation)

සැබෑ මෘදුකාංග ඉංජිනේරු ක්ෂේත්‍රයේදී Context API භාවිතා කරන්නේ "Static" හෝ ඉතා අඩුවෙන් වෙනස් වන දත්ත සඳහාය (උදා: Locale, Theme, User Profile). ඉතා වේගයෙන් වෙනස් වන දත්ත (උදා: තරඟ කාලීන ස්ටොක් මාර්කට් දත්ත) සඳහා Context භාවිතා කිරීම වෙබ් අඩවියේ වේගය අඩු කිරීමට හේතු විය හැක. මන්දයත් Provider එකේ අගය වෙනස් වන සෑම වතාවකම එය යටතේ ඇති සියලුම සංරචක නැවත රෙන්ඩර් (Re-render) වන බැවිනි. එවැනි අවස්ථාවලදී Redux හෝ Zustand වැනි දියුණු පුස්තකාල භාවිතා කරයි.

72.8 පොදු වැරදි (Common Mistakes)

1. Provider එකට පිටතින් useContext භාවිතා කිරීම: ඔබ Provider එක ඇතුළත නැති සංරචකයක සිට දත්ත ගැනීමට උත්සාහ කළහොත් ඔබට undefined අගයක් ලැබෙනු ඇත.

2. සෑම දෙයකටම Context භාවිතා කිරීම: දත්තය යථාතේ එකම එක දරුවෙකුට නම්, නිකම්ම Props භාවිතා කරන්න. Context භාවිතා කළ යුත්තේ දත්තය තැන් කිහිපයකටම අවශ්‍ය වූ විට පමණි.

3. Context එක Export කිරීමට අමතක වීම: වෙනත් ගොනුවක සිට useContext කිරීමට නම් අනිවාර්යයෙන්ම මුල් ගොනුවෙන් Context එක export කළ යුතුය.

72.9 සාරාංශය (Key Points)

- Prop Drilling යනු දත්ත අනවශ්‍ය ලෙස සංරචක හරහා පහළට යැවීමයි.
- Context API මගින් ගෝලීය දත්ත පද්ධතියක් (Global Store) සාදයි.
- createContext මගින් සන්දර්භය සාදයි.
- Provider මගින් දත්ත බෙදා හරියි.
- useContext හුක් එක මගින් ඕනෑම තැනක සිට දත්ත ලබා ගනී.

72.10 අභ්‍යාස (Exercises)

1. useContext නමින් සන්දර්භයක් සාදා, පරිශීලකයාගේ නම මුළු ඇප් එක පුරාම පෙන්වන පද්ධතියක් සකසන්න.
2. Context භාවිතා කර සාප්පු කරත්තයක (Shopping Cart) ඇති භාණ්ඩ සංඛ්‍යාව පෙන්වන සරල පද්ධතියක් සාදන්න.
3. useContext භාවිතා නොකර Context එකක දත්ත ලබා ගැනීමට ඇති පැරණි ක්‍රමය (Consumer) අන්තර්ජාලයෙන් සොයා බලන්න. (මෙය දැන සිටීම සම්මුඛ පරීක්ෂණ වලට වැදගත් වේ).

72.11 කුඩා ව්‍යාපෘතිය (Mini Project)

ව්‍යාපෘතිය: භාෂා මාරු කරන පද්ධතිය (Multi-language Support)

1. LanguageContext.jsx නමින් සන්දර්භයක් සාදන්න.
2. එහි language නමින් ස්ටේට් එකක් සාදන්න (සම්මතය: "සිංහල").
3. බොත්තමක් එබූ විට එය "English" බවට මාරු විය යුතුය.
4. වෙබ් අඩවියේ ඇති අකුරු (උදා: "ආයුබෝවන්" / "Hello") තෝරාගත් භාෂාව අනුව ස්වයංක්‍රීයව වෙනස් විය යුතුය.

මිලිභ පරිච්ඡේදයෙන් අපි රියැක්ට් වලදී අන්තර්ජාලය හරහා සැබෑ සේවාදායකයන්ගෙන්

දත්ත ලබා ගන්නා ආකාරය (Fetching API in React) ඉතාමත් විස්තරාත්මකව ඉගෙන ගනිමු.

හැත්තෑ තුන්වන පරිච්ඡේදය: ඒ.පී.අයි දත්ත ලබා ගැනීම - සැබෑ ලෝකයේ දත්ත සමඟ ගනුදෙනු කිරීම (API Integration and Data Fetching in React)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අධ්‍යයනය කිරීමෙන් පසු ඔබට රියැක්ට් මෘදුකාංගයක් තුළ බාහිර දත්ත හැසිරවීම පිළිබඳව අතිශය ගැඹුරු සහ වෘත්තීය අවබෝධයක් ලැබෙනු ඇත:

1. රියැක්ට් සංරචකයක් තුළ බාහිර ඒ.පී.අයි (API) දත්ත ලබා ගැනීමේ අවශ්‍යතාවය තේරුම් ගැනීම.
2. `useEffect` හුක් එක සහ `fetch` විධානය එකිනෙක සම්බන්ධ කර දත්ත ගෙන්වා ගන්නා ආකාරය.
3. දත්ත බාගත වන තෙක් පවතින "ප්‍රවේශන තත්ත්වය" (Loading State) කළමනාකරණය කරන ආකාරය.
4. අන්තර්ජාලය විසන්ධි වීම වැනි දෝෂ අවස්ථාවන් (Error Handling) වෘත්තීය ලෙස හැසිරවීම.
5. අසමකාලීන ශ්‍රිත (Async/Await) රියැක්ට් තුළ නිවැරදිව භාවිතා කරන ආකාරය.
6. සංරචකය ඉවත් වන විට (Unmount) සිදුවන අනවශ්‍ය දත්ත ඉල්ලීම් නැවැත්වීම (Cleanup).

73.1 හැඳින්වීම: ඇයි අපිට API දත්ත අවශ්‍ය වන්නේ?

අපි මෙතෙක් නිර්මාණය කළ රියැක්ට් සංරචකවල නිඛුණේ අප විසින්ම කේතය තුළ ලියන ලද දත්ත (Static Data) වේ. නමුත් සැබෑ ලෝකයේ මෘදුකාංගයක දත්ත නිරන්තරයෙන් වෙනස් වේ.

සැබෑ ලෝකයේ උදාහරණ:

- ප්‍රවෘත්ති වෙබ් අඩවියක්: මොහොතින් මොහොත අලුත් වන පුවත් සර්වර් එකකින් ලබා ගත යුතුය.
- ලේස්ට්: ඔබේ මිතුරන් දමන අලුත්ම පෝස්ට් (Posts) සර්වර් එකකින් ගෙන පෙන්විය යුතුය.
- කාලගුණ ඇප් එකක්: වත්මන් උෂ්ණත්වය ලබා ගැනීමට කාලගුණ සේවාදායකයකට (Weather API) සම්බන්ධ විය යුතුය.

පරිමාව 4 හිදී අප Fetch API ගැන ඉගෙන ගත්තෙමු. දැන් අපි බලමු එම දැනුම රියැක්ට් හි State සහ Lifecycle සමඟ එකාබද්ධ කර වෘත්තීය මට්ටමින් භාවිතා කරන්නේ කෙසේද කියා.

73.2 රියැක්ට් හි දත්ත ලබා ගැනීමේ මූලික පියවර 5 (The 5 Steps)

වෘත්තීය මට්ටමේ දත්ත ලබා ගැනීමේ ක්‍රියාවලියක අනිවාර්යයෙන්ම මෙම පියවර 5 තිබිය යුතුය:

1. **State නිර්මාණය කිරීම:** ලැබෙන දත්ත ගබඩා කිරීමට ස්ථාවර එකක් අවශ්‍ය වේ.
2. **Loading State:** දත්ත ලැබෙන තෙක් පරිශීලකයාට "Loading..." වැනි පණිවිඩයක් පෙන්වීමට ස්ථාවර එකක් අවශ්‍ය වේ.
3. **Error State:** යම් හෙයකින් දත්ත ලබා ගැනීමට නොහැකි වුවහොත් (උදා: Internet නැති වීම) එය පෙන්වීමට ස්ථාවර එකක් අවශ්‍ය වේ.
4. **useEffect භාවිතය:** සංරචකය තිරය මත දිස්වන විට (Mount) පමණක් දත්ත ගෙන්වා ගැනීමට `useEffect` භාවිතා කළ යුතුය.
5. **Conditional Rendering:** දත්තවල තත්ත්වය අනුව (Loading ද? Error ද? දත්ත ලැබුණාද?) තිරය මත පෙන්වන දේ වෙනස් කළ යුතුය.

73.3 ප්‍රායෝගික උදාහරණය: පරිශීලක ලැයිස්තුවක් ලබා ගැනීම

අපි JSONPlaceholder නමැති නොමිලේ දත්ත ලබා දෙන සේවාවෙන් පරිශීලකයන් 10 දෙනෙකුගේ තොරතුරු ලබා ගෙන පෙන්වන සංරචකයක් නිර්මාණය කරමු.

සම්පූර්ණ කේතය සහ විග්‍රහය:

```
import { useState, useEffect } from 'react';
```

```
function UserList() {
```

```
  // 1. දත්ත ගබඩා කිරීමට ස්ථාවර එක (ආරම්භක අගය හිස් අරාවකි)
```

```
  const [users, setUsers] = useState([]);
```

```
  // 2. දත්ත බාගත වෙමින් පවතින බව පෙන්වීමට ස්ථාවර එක
```

```
  const [isLoading, setIsLoading] = useState(true);
```

```
  // 3. දෝෂ පණිවිඩ ගබඩා කිරීමට ස්ථාවර එක
```

```
  const [error, setError] = useState(null);
```

```
// 4. දත්ත ලබා ගැනීමේ අසමකාලීන ශ්‍රිතය

useEffect(() => {

  const fetchUsers = async () => {

    try {

      const response = await
fetch('https://jsonplaceholder.typicode.com/users');

      // සර්වර් එකෙන් පිළිතුරක් ආවාදැයි බලමු

      if (!response.ok) {

        throw new Error("දත්ත ලබා ගැනීමට නොහැකි
විය!");

      }

      const data = await response.json();

      setUsers(data); // දත්ත ස්ටේට් එකට දමමු

      setIsLoading(false); // බාගත වී අවසන් නිසා
Loading නතර කරමු

    } catch (err) {

      setError(err.message); // වැරද්දක් වූණෙත් එය
පෙන්වමු

      setIsLoading(false);

    }

  };

  fetchUsers();

}, []); // හිස් අරාට් [] නිසා මෙය ක්‍රියාත්මක වන්නේ එක් වරක්
පමණි
```

```
// 5. තත්ත්වය අනුව තීරය වෙනස් කිරීම (Conditional
Rendering)
```

```
if (isLoading) return <h2>දත්ත බාගත වෙමින්
පවතී...</h2>;

if (error) return <h2 style={{ color: 'red' }}>වැරද්දක්
සිදු විය: {error}</h2>;

return (

  <div>

    <h1>පරිශීලක ලැයිස්තුව</h1>

    <ul>
```

```
{users.map(user => (

  <li key={user.id}>

    <strong>{user.name}</strong> - {user.email}

  </li>

))}

</ul>

</div>

);

}

export default UserList;
```

73.4 කේතය පිළිබඳ ගැඹුරු තාක්ෂණික විග්‍රහය

1. ඇයි `useEffect` භාවිතා කළේ? අපි සෘජුවම සංරචක ශ්‍රිතය ඇතුළත `fetch` කළහොත්, ස්ටේට් එක වෙනස් වන සෑම වාරයකදීම (Re-render) නැවත නැවතත් දත්ත ඉල්ලා සර්වර් එකට පණිවිඩ යවයි. මෙය සර්වර් එක "හිර වීමට" හේතු වේ. `useEffect` හි ඇති `[]` මගින් එය එක් වරක් පමණක් සිදුවන බව සහතික කරයි.
2. `async/await` භාවිතය: අන්තර්ජාලයෙන් දත්ත ලැබීමට කාලයක් ගත වේ. එම කාලය තුළ වෙබ් අඩවිය හිර නොවී තබා ගැනීමට මෙම අසමකාලීන ක්‍රමය භාවිතා කරයි.
3. `isLoading` හි වැදගත්කම: දත්ත ලැබෙන තුරු `users` අරාට් හිස්ව පවතී. එම කාලය තුළ හිස් තීරයක් පෙන්වනවා වෙනුවට පරිශීලකයාට යම් පණිවිඩයක් පෙන්වීම වෘත්තීය ලක්ෂණයකි.

73.5 ASCII රූපසටහන: දත්ත ගලායෑමේ ක්‍රියාවලිය

```
[ සංරචකය පණ ගැන්වේ (Mount) ]

|

(isLoading = true) <--- මූලිකම පෙන්වයි

|

[ useEffect ක්‍රියාත්මක වේ ]

|

---- { fetch() Request } ----> [ සර්වර් එක ]

|

<--- { JSON Response } <-----|

|

[ setUsers(data) ]

[ setIsLoading(false) ]

|

[ රියැක්ට් Re-render කරයි ]

|

[ ලැයිස්තුව නිරයේ දිස්වේ ]
```

73.6 වෘත්තීය පැහැදිලි කිරීම (Professional Explanation)

සැබෑ මහා පරිමාණ ව්‍යාපෘතිවලදී අප නිකමිම `fetch()` භාවිතා කරන්නේ නැත. ඒ වෙනුවට `Axios` වැනි දියුණු පුස්තකාල හෝ `React Query` (`TanStack Query`) වැනි දත්ත කළමනාකරණ මෙවලම් භාවිතා කරයි. මන්දයත් ඒවා මගින් දත්ත මතක තබා ගැනීම (`Caching`) සහ ස්වයංක්‍රීයව නැවත දත්ත ලබා ගැනීම (`Refetching`) වැනි සංකීර්ණ වැඩ ස්වයංක්‍රීයව සිදු කරන බැවිනි. නමුත්, ඕනෑම දියුණු මෙවලමක් ඉගෙන ගැනීමට පෙර ඉහත සඳහන් කළ මූලික `useEffect` + `fetch` ක්‍රමය 100% ක් දැන සිටීම අත්‍යවශ්‍ය වේ.

73.7 පොදු වැරදි (Common Mistakes)

1. **Dependency Array අමතක වීම:** `useEffect` අවසානයේ `[]` නොයෙදීම නිසා දත්ත ලබා

ගැනීමේ "අනන්ත පුඩුවක්" (Infinite loop) ඇති වීම.

2. **`await` අමතක වීම:** `response.json()` ඉදිරියෙන් `await` නොයෙදීම නිසා දත්ත වෙනුවට `Promise` එකක් ලැබීම.

3. **Cleanup නොකිරීම:** දත්ත ලැබෙන්නට පෙර පරිශීලකයා වෙනත් පිටුවකට ගියහොත්, එම දත්ත ඉල්ලීම පසුබිමෙන් තවමත් ක්‍රියාත්මක වේ. මෙය මතක අවහිරතා (Memory Leaks) ඇති කරයි. (මෙය වැළැක්වීමට `AbortController` භාවිතා කරයි).

73.8 සාරාංශය (Key Points)

- රියැක්ට් හි දත්ත ලබා ගැනීමට `useEffect` අත්‍යවශ්‍ය වේ.
- **Loading** සහ **Error** තත්ත්වයන් පාලනය කිරීම වෘත්තීය මට්ටමේදී අනිවාර්ය වේ.
- `[]` මගින් දත්ත ලබා ගැනීම ස්ථාපනය (Mount) වන විට පමණක් සිදු කරයි.
- `map()` මගින් ලැබුණු දත්ත අරාම UI එකක් බවට හරවයි.
- දෝෂ පාලනයට `try...catch` භාවිතා කළ යුතුය.

73.9 අභ්‍යාස (Exercises)

1. <https://api.adviceslip.com/advice> යන ලිපිනයෙන් අහඹු උපදෙසක් (Random Advice) ලබා ගෙන එය තිරය මත පෙන්වන සංරචකයක් සාදන්න.
2. පරිශීලකයා බොත්තමක් එබූ විට පමණක් දත්ත ලබා ගන්නා පද්ධතියක් සකසන්න. (ඉඟිය: `useEffect` වෙනුවට සාමාන්‍ය බොත්තම් ශ්‍රිතයක් භාවිතා කරන්න).
3. පහත කේතයේ ඇති දෝෂය කුමක්ද?

```
useEffect(() => {

    const data = fetch('url').json();

    setItems(data);

}, []);
```

73.10 කුඩා ව්‍යාපෘතිය (Mini Project)

ව්‍යාපෘතිය: කාලගුණ තොරතුරු පුවරුව (Simple Weather App)

1. <https://api.openweathermap.org> හෝ වෙනත් නොමිලේ කාලගුණ ඒ.පී.අයි එකක් සොයා ගන්න.

2. පරිශීලකයාට නගරයක නම ටයිප් කිරීමට Input කොටුවක් ලබා දෙන්න.
3. "සොයන්න" බොත්තම එබූ විට අදාළ නගරයේ උෂ්ණත්වය (Temperature) සහ කාලගුණ තත්ත්වය (Description) ලබා ගන්න.
4. දත්ත ලබා ගන්නා අතරතුර "කාලගුණ දත්ත සොයමින් පවතී..." යනුවෙන් පණිවිඩයක් පෙන්වන්න.
5. වැරදි නගර නාමයක් දුන්නොත් "නගරය හමු නොවීය" කියා පෙන්වන්න.

මිලින පරිච්ඡේදයෙන් අපි පරිමාව 6 හි අවසාන කොටසට පිවිසෙමින්, අප නිර්මාණය කළ රියැක්ට් ව්‍යාපෘති ලෝකයටම පෙනෙන සේ අන්තර්ජාලයට මුදාහරින (Deployment) ආකාරය ඉතාමත් විස්තරාත්මකව ඉගෙන ගනිමු.

හැත්තෑ හතරවන පරිච්ඡේදය: රියැක්ට් ව්‍යාපෘති ප්‍රසිද්ධ කිරීම - ලෝකයටම පෙන්වමු (Deploying React Applications)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අධ්‍යයනය කිරීමෙන් පසු ඔබට රියැක්ට් මෘදුකාංගයක් ඔබගේ පරිගණකයෙන් පිටතට ගෙන ලෝකයටම විවෘත වන සේ අන්තර්ජාලයට මුදා හැරීම පිළිබඳව අතිශය ගැඹුරු සහ වෘත්තීය අවබෝධයක් ලැබෙනු ඇත:

1. සංවර්ධන මට්ටම (Development) සහ නිෂ්පාදන මට්ටම (Production) අතර ඇති තාක්ෂණික වෙනස හඳුනා ගැනීම.
2. රියැක්ට් ව්‍යාපෘතියක් "ගොඩනැගීම" (Building) යනු කුමක්ද සහ එය කරන්නේ ඇයි දැයි වටහා ගැනීම.
3. `npm run build` විධානය මගින් ප්‍රශස්ත කරන ලද (Optimized) ගොනු නිර්මාණය කරන ආකාරය.
4. නෙට්ලිෆයි (Netlify) සහ වර්සෙල් (Vercel) වැනි සේවාවන් හරහා සජීවී විකාශනය සිදු කරන ආකාරය.
5. රවුටින් (Routing) භාවිතා කරන විට නිෂ්පාදන මට්ටමේදී ඇතිවන "404 Error" ගැටලුව සහ එය විසඳන ආකාරය.
6. සන්තතික ප්‍රසිද්ධ කිරීම (Continuous Deployment - CD) සහ එහි ඇති වෘත්තීය වාසි තේරුම් ගැනීම.

74.1 හැඳින්වීම: ප්‍රසිද්ධ කිරීම යනු කුමක්ද? (What is Deployment?)

අපි මෙතෙක් රියැක්ට් කේත ලියූ අතර ඒවා ඔවුසරයෙන් නැරඹුවේ

`http://localhost:5173` වැනි

ලිපිනයක් හරහාය. මෙම ලිපිනය ක්‍රියා කරන්නේ ඔබගේ පරිගණකය ඇතුළත පමණි. එය ඔබගේ මිතුරෙකුට යැවූවහොත් ඔහුට එය නැරඹිය නොහැක.

Deployment (ප්‍රසිද්ධ කිරීම) යනු ඔබ ලියන ලද රියැක්ට් කේතය අන්තර්ජාලය සමඟ සම්බන්ධ වූ ප්‍රබල සේවාදායකයක් (Server) වෙත යවා, ලෝකයේ ඕනෑම තැනක සිටින අයෙකුට වෙබ් ලිපිනයක් (URL) හරහා එය බැලීමට සැලැස්වීමයි.

රියැක්ට් හි විශේෂත්වය වන්නේ, අප ලියන සංරචක (Components) සහ JSX කේත ඔවුසරයට කෙලින්ම තේරුම් ගත නොහැකි වීමයි. එබැවින් ප්‍රසිද්ධ කිරීමට පෙර අපට විශේෂ පියවරක් අනුගමනය කිරීමට සිදු වේ.

74.2 සංවර්ධන මට්ටම එදිරිව නිෂ්පාදන මට්ටම (Dev vs Production)

වෘත්තීය මෘදුකාංග ඉංජිනේරුවෙකු මෙම වෙනස ඉතා හොඳින් දැන සිටිය යුතුය.

1. Development Mode (සංවර්ධන මට්ටම):

- ඔබ කේත ලියන අතරතුර භාවිතා කරන අවස්ථාවයි.
- මෙහිදී වැරදි (Errors) පෙන්වීමට වැඩි ඉඩක් ලබා දී ඇත.
- කේතයේ ප්‍රමාණය විශාලය (එය වේගවත් නැත).
- Vite විසින් මෙය ඔබ වෙනුවෙන් පාලනය කරයි.

2. Production Mode (නිෂ්පාදන මට්ටම):

- සැබෑ පරිශීලකයන් වෙබ් අඩවිය භාවිතා කරන අවස්ථාවයි.
- මෙහිදී කේතය ඉතාමත් කුඩා වන සේ පිරිසිදු කර (Minified) ඇත.
- වේගය ඉතා ඉහළය.
- අනවශ්‍ය සටහන් (Comments) සහ අමතර දත්ත ඉවත් කර ඇත.

74.3 ව්‍යාපෘතිය ගොඩනැගීම (Building the Project)

ප්‍රසිද්ධ කිරීමට ප්‍රථම, ඔබගේ සියලුම JSX සහ CSS කේත ඔවුසරයට තේරෙන සාමාන්‍ය HTML, CSS සහ JavaScript බවට පරිවර්තනය කළ යුතුය. මෙයට Building (ගොඩනැගීම) යැයි කියනු ලැබේ.

උපමාවක්: ඔබේ කේතය යනු කේක් එකක් සෑදීමට ගන්නා පිටි, බිත්තර සහ සීනි වැනිය. ඔවුසරයට මේවා නිකම්ම "කෑමට" නොහැක. ඔබ මුලින්ම මේවා එකතු කර පෝරණුවක දමා පුළුස්සා කේක් එකක් සාදා ගත යුතුය. Build ක්‍රියාවලිය යනු එම "පිළිස්සීමයි".

විධානය: ටර්මිනලය (Terminal) විවෘත කර ඔබගේ ව්‍යාපෘති ෆෝල්ඩරය තුළ පහත විධානය ටයිප් කරන්න:

npm run build

මෙම විධානයෙන් පසු, ඔබගේ ව්‍යාපෘති ෆෝල්ඩරය තුළ අලුතින් dist (හෝ build) නමින් ෆෝල්ඩරයක් සෑදේ. ඔබ අන්තර්ජාලයට මුදා හැරිය යුත්තේ මෙම dist ෆෝල්ඩරයේ ඇති ගොනු පමණි.

74.4 නෙට්ලිෆයි (Netlify) හරහා ප්‍රසිද්ධ කිරීමේ පියවර

නෙට්ලිෆයි යනු රියැක්ට් මෘදුකාංග ප්‍රසිද්ධ කිරීමට ඇති ජනප්‍රියම සේවාවකි. මෙය පියවර දෙකකට කළ හැක:

ක්‍රමය 1: අතින් ප්‍රසිද්ධ කිරීම (Manual Deploy)

1. **npm run build** කර dist ෆෝල්ඩරය සාදා ගන්න.
2. **netlify.com** වෙත ගොස් ලොගින් වන්න.
3. එහි ඇති "Drag and Drop" පෙට්ටිය වෙත ඔබේ dist ෆෝල්ඩරය ඇද දමන්න.
4. තත්පර කිහිපයකින් ඔබට වෙබ් ලිපිනයක් ලැබෙනු ඇත.

ක්‍රමය 2: ගිට්හබ් හරහා ප්‍රසිද්ධ කිරීම (GitHub Integration - වෘත්තීය ක්‍රමය)

මෙය අප පරිමාව 5 හි ඉගෙන ගත් Continuous Deployment ක්‍රමයයි.

1. ඔබේ ව්‍යාපෘතිය GitHub වෙත යවන්න (Push).
2. Netlify හි ඇති "Add new site" ක්ලික් කර "Import an existing project" තෝරන්න.
3. GitHub තෝරා ඔබේ ව්‍යාපෘතිය (Repository) තෝරන්න.
4. "Build Command" එක ලෙස **npm run build** සහ "Publish directory" එක ලෙස **dist** නිවේදනය බලන්න.
5. "Deploy Site" බොත්තම ඔබන්න.

දැන් ඔබ මින් ඉදිරියට කරන සෑම වෙනසක්ම GitHub වෙත යවන විට, Netlify විසින් ස්වයංක්‍රීයව වෙබ් අඩවිය යාවත්කාලීන කරනු ලබයි.

74.5 රවුටින් සහ 404 ගැටලුව (The Routing Fix)

ඔබ **react-router-dom** භාවිතා කරන මෘදුකාංගයක් ප්‍රසිද්ධ කළහොත්, එක් විශේෂ ගැටලුවකට මුහුණ දීමට සිදු වේ. ඔබ "Home" පිටුවේ සිට "About" පිටුවට ගියහොත් එය හොඳින් වැඩ කරයි. නමුත් "About" පිටුවේ සිටියදී ඔවුසරය Refresh කළහොත්, ඔබට "404 Not Found" වැද්දේද ලැබෙනු ඇත.

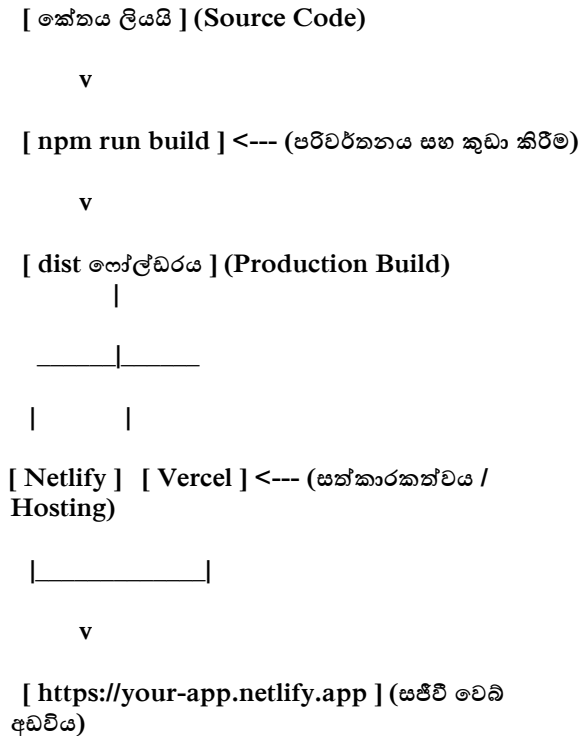
හේතුව: ඔවුසරය සිතන්නේ /about නමින් සැබෑ HTML ගොනුවක් සර්වර් එකේ ඇති බවයි. නමුත් රියැක්ට් හි ඇත්තේ එකම එක **index.html** ගොනුවක් පමණි.

විසඳුම: නෙට්ලිෆයි (Netlify) සඳහා, ඔබගේ **public** ෆෝල්ඩරය තුළ **_redirects** (නම ඉදිරියෙන් යට ඉරක් සහිතව) නමින් ගොනුවක් සාදා එහි පහත ජේෂිය ඇතුළත් කරන්න:

```
/* /index.html 200
```

මෙයින් සර්වර් එකට පවසන්නේ, කුමන ලිපිනයකට ආවත් එය **index.html** වෙත යොමු කරන ලෙසයි. එවිට රියැක්ට් රවුටර් එක වැඩ භාරගෙන අදාළ පිටුව පෙන්වනු ඇත.

74.6 ASCII රූපසටහන: නිෂ්පාදන ප්‍රවාහය (Deployment Pipeline)



74.7 වෘත්තීය පැහැදිලි කිරීම (Professional Explanation)

සැබෑ මෘදුකාංග නිෂ්පාදන වටපිටාවක (Enterprise Environment), අපි මෙම ක්‍රියාවලියට CI/CD (Continuous Integration / Continuous Deployment) යැයි පවසමු. මෙහිදී සංවර්ධකයා කරන්නේ කේතය ලියා පරීක්ෂා කර GitHub වෙත යැවීම පමණි. ඉතිරි සියලුම Build කටයුතු සහ ප්‍රසිද්ධ කිරීම් ස්වයංක්‍රීයව සිදු වේ. මෘදුකාංග ඉංජිනේරුවෙකු ලෙස ඔබගේ අරමුණ විය යුත්තේ හැකි සෑම විටම මානුෂීය මැදිහත්වීම අවම කර වැඩ කටයුතු ස්වයංක්‍රීය කිරීමයි.

74.8 පොදු වැරදි (Common Mistakes)

1. Environment Variables අමතක වීම: ඔබ API Keys භාවිතා කරන්නේ නම්, ඒවා .env ගොනුවේ පමණක් තැබීම ප්‍රමාණවත් නැත. Netlify හෝ Vercel හි "Site Settings" වෙත ගොස් එම Keys එහි ද ඇතුළත් කළ යුතුය.
2. පැරණි Node Version එකක් භාවිතය: සමහර විට ඔබේ පරිගණකයේ වැඩ කරන කේතය

නොවලිමයි එකේ Build නොවීමට හේතුව වන්නේ එහි ඇති Node.js අනුවාදය පැරණි වීමයි.

3. Large Assets: ඉතා විශාල පින්තූර හෝ විඩියෝ src තුළ තැබීම නිසා Build එකේ ප්‍රමාණය අනවශ්‍ය ලෙස වැඩි වීම.

74.9 සාරාංශය (Key Points)

- npm run build මගින් බෙදාහැරීමට සුදුසු ගොනු (Production files) සාදයි.
- මෙම ගොනු dist නමැති ෆෝල්ඩරයක තැන්පත් වේ.
- GitHub හරහා Netlify සම්බන්ධ කිරීම වෘත්තීය ක්‍රමයයි.
- SPA Routing ගැටලුව විසඳීමට _redirects ගොනුව භාවිතා කරයි.
- ප්‍රසිද්ධ කළ පසු වෙබ් අඩවියේ වේගය සහ ආරක්ෂාව ඉහළ මට්ටමක පවතී.

74.10 අභ්‍යාස (Exercises)

1. ඔබ මේ දක්වා සාදන ලද ඕනෑම රියැක්ට් ව්‍යාපෘතියක් ගෙන npm run build කර එහි ඇතුළත ඇති ගොනු පරීක්ෂා කර බලන්න. (පොඩ් JavaScript ගොනුවක් ඇතුළත විශාල කේත රාශියක් තිබේදැයි බලන්න).
2. නොවලිමයි ගිණුමක් සාදා ඔබගේ පළමු රියැක්ට් මෘදුකාංගය සජීවීව අන්තර්ජාලයට මුදා හරින්න.
3. _redirects ගොනුව නොමැතිව රවුටින් සහිත වෙබ් අඩවියක් පරීක්ෂා කරන්න. එය Refresh කළ විට කුමක් සිදුවේදැයි අත්දැකීමෙන් වටහා ගන්න.

74.11 කුඩා ව්‍යාපෘතිය (Mini Project)

ව්‍යාපෘතිය: "SkyCast" කාලගුණ පද්ධතිය සජීවීව ප්‍රසිද්ධ කිරීම

1. පසුගිය පරිච්ඡේදවලදී ඔබ නිර්මාණය කළ කාලගුණ පද්ධතිය (Weather App) ගෙන එයට අවශ්‍ය සියලුම මූලික හැඩතල සහ දෝෂ පාලනයන් එක් කරන්න.
2. එය GitHub වෙත සාර්ථකව Push කරන්න.
3. Vercel හෝ Netlify භාවිතා කර එය සජීවී (Live) කරන්න.
4. ලැබෙන ලින්ක් එක මිතුරන් අතර බෙදාගෙන විවිධ උපාංග (Mobile, Tablet) මගින් පරීක්ෂා කර බලන්න.

සුභ පැතුම්! ඔබ දැන් රියැක්ට් හි මූලික මට්ටමේ සිට වෘත්තීය මට්ටම දක්වා සියලුම අත්‍යවශ්‍ය කරුණු ආවරණය කර අවසානය.

හැත්තෑ පස්වන පරිච්ඡේදය: මහා ව්‍යාපෘතිය - "TaskMaster" වෘත්තීය ව්‍යාපෘති කළමනාකරණ පද්ධතිය (Capstone Project: TaskMaster Pro)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිමාවේ අවසාන සහ වැදගත්ම පරිච්ඡේදය වන මෙයින්, ඔබ මෙතෙක් උගත් සියලුම රියැක්ට් (React) සංකල්ප එකට එකතු කර සැබෑ ලෝකයේ භාවිත කළ හැකි මට්ටමේ පූර්ණ මෘදුකාංගයක් (Complete Web Application) නිර්මාණය කරනු ඇත. මෙම ව්‍යාපෘතිය අවසානයේදී ඔබට පහත දෑ පිළිබඳව පූර්ණ නිපුණතාවයක් ලැබෙනු ඇත:

1. සංකීර්ණ සංරචක ව්‍යුහය (Advanced Component Architecture): මෘදුකාංගයක් කුඩා කොටස්වලට බෙදා නිවැරදිව කළමනාකරණය කිරීම.
2. ගෝලීය ස්ටේට් පාලනය (Context API): මුළු ඇප් එක පුරාම දත්ත (Tasks) පාලනය කිරීමට Context භාවිතා කිරීම.
3. බහු-පිටු සංවලනය (React Router): Home, Active Tasks, සහ Completed Tasks ලෙස පිටු කිහිපයක් සැකසීම.
4. දත්ත ස්ථාවරත්වය (LocalStorage Persistence): බ්‍රවුසරය වසා දැමුවද දත්ත මැකී නොයන ලෙස සකස් කිරීම.
5. ප්‍රෝ-මට්ටමේ CRUD ක්‍රියාවලිය: දත්ත ඇතුළත් කිරීම (Create), කියවීම (Read), යාවත්කාලීන කිරීම (Update) සහ මකා දැමීම (Delete).
6. අතුරුමුහුණත් අලංකරණය (Styling): රියැක්ට් සමඟ CSS සහ Animations එකතුවීම කිරීම.

75.1 ව්‍යාපෘතියේ සැලැස්ම (The Master Plan)

අපි නිර්මාණය කරන "TaskMaster Pro" මෘදුකාංගයේ අඩංගු විය යුතු ප්‍රධාන අංග මෙසේය:

- Global Context: සියලුම වැඩ (Tasks) සහ ඒවායේ තත්ත්වයන් (Status) මෙහි ගබඩා වේ.
- Routing:
 - / : සියලුම වැඩ පෙන්වන ප්‍රධාන පිටුව.
 - /active : තවමත් නිම නොකළ වැඩ.
 - /completed : නිම කරන ලද වැඩ.

- Functionality:

- වැඩක් එකතු කිරීම.
- වැඩක් නිම කළ බව සලකුණු කිරීම (Toggle).
- වැඩක් මකා දැමීම.
- සියලුම දත්ත LocalStorage හි සේව කිරීම.

75.2 පියවර 1: වෘත්තීය ගොනු ව්‍යුහය සකස් කිරීම

ඔබගේ src ෆෝල්ඩරය තුළ පහත ආකාරයට ෆෝල්ඩර සහ ගොනු සකස් කරගන්න. මෙය වෘත්තීය මට්ටමේ (Standard) පිළිවෙළයි.

src/

```
├── context/
│   └── TaskContext.jsx  (දත්ත පාලනය කරන මොළය)
├── components/
│   ├── Navbar.jsx      (මෙහි එක)
│   ├── TaskForm.jsx     (වැඩ එකතු කරන පෝරමය)
│   ├── TaskItem.jsx     (තනි වැඩක් පෙන්වන කොටස)
│   └── TaskList.jsx     (සියලුම වැඩ ලැයිස්තුව)
├── pages/
│   ├── Home.jsx        (සියලු වැඩ පෙන්වන පිටුව)
│   ├── Active.jsx       (නිම නොකළ වැඩ පිටුව)
│   └── Completed.jsx    (නිම කළ වැඩ පිටුව)
├── App.jsx              (ප්‍රධාන ගොනුව)
└── main.jsx              (ආරම්භක ස්ථානය)
```

75.3 පියවර 2: දත්ත ගබඩාව නිර්මාණය (Context API)

මුලින්ම අපි දත්ත කළමනාකරණය කරන "නාලිකාව" (Context) සාදමු.

src/context/TaskContext.jsx

```
import { createContext, useState, useEffect } from 'react';
```

```
// 1. Context එක සාදමු
```

```

export const TaskContext =
  createContext();

export const TaskProvider = ({ children })
=> {

  // 2. LocalStorage එකෙන් පරණ දත්ත
  නිබේද්‍යීය බලා ලබා ගනිමු

  const savedTasks =
    JSON.parse(localStorage.getItem("myTasks")) || [];

  const [tasks, setTasks] =
    useState(savedTasks);

  // 3. දත්ත වෙනස් වන සෑම වාරයකම
  LocalStorage එක අප්ඩේට් කරමු

  useEffect(() => {

    localStorage.setItem("myTasks",
      JSON.stringify(tasks));

  }, [tasks]);

  // 4. අලුත් වැඩක් එකතු කිරීමේ ශ්‍රිතය

  const addTask = (title) => {

    const newTask = { id: Date.now(), title,
      isDone: false };

    setTasks([...tasks, newTask]);

  };

  // 5. වැඩක් නිම කළ බව මාරු කිරීම (Toggle)

  const toggleTask = (id) => {

    setTasks(tasks.map(t => t.id === id ? {
      ...t, isDone: !t.isDone } : t));

  };

  // 6. වැඩක් මකා දැමීම

  const deleteTask = (id) => {

    setTasks(tasks.filter(t => t.id !== id));

```

```

  };

  // 7. සියලු දත්ත සහ ශ්‍රිත දරුවන්ට ලබා දෙමු

  return (

    <TaskContext.Provider value={{ tasks,
      addTask, toggleTask, deleteTask }}>

      {children}

    </TaskContext.Provider>

  );

};

```

විස්තරය: මෙහිදී tasks අරාම සහ එය වෙනස් කරන සියලුම ශ්‍රිත (Functions) අපි එකම ස්ථානයක (Central Store) තබා ගත්තෙමු.

75.4 පියවර 3: සංරචක නිර්මාණය (Components)

දැන් අපි UI එකේ ඇති කොටස් එකින් එක සාදමු.

src/components/TaskForm.jsx

```

import { useState, useContext } from
'react';

import { TaskContext } from
'../context/TaskContext';

const TaskForm = () => {

  const [text, setText] = useState("");

  const { addTask } =
    useContext(TaskContext);

  const handleSubmit = (e) => {

    e.preventDefault();

    if (text.trim()) {

      addTask(text);

      setText(""); // කොටුව හිස් කරමු

    }

```

```

    };

    return (

      <form onSubmit={handleSubmit}
        className="task-form">

        <input

          value={text}

          onChange={(e) =>
            setText(e.target.value)}

          placeholder="අලුත් වැඩක් ලියන්න..."
        />

        <button type="submit">එකතු
        කරන්න</button>

      </form>

    );
  };

  export default TaskForm;

  src/components/TaskItem.jsx

  import { useContext } from 'react';

  import { TaskContext } from
    '../context/TaskContext';

  const TaskItem = ({ task }) => {

    const { toggleTask, deleteTask } =
      useContext(TaskContext);

    return (

      <div className={`task-item
        ${task.isDone ? 'completed' : ''}`>

        <span onClick={() =>
          toggleTask(task.id)}>{task.title}</span>

        <button onClick={() =>
          deleteTask(task.id)}>මකන්න</button>

      </div>

```

```

    );

  };

  export default TaskItem;

  75.5 පියවර 4: පිටු නිර්මාණය (Pages
  & Routing)

  අපි දැන් විවිධ පිටුවලදී දත්ත පෙරීම (Filtering) සිදු
  කරමු.

  src/pages/Home.jsx

  import { useContext } from 'react';

  import { TaskContext } from
    '../context/TaskContext';

  import TaskForm from
    '../components/TaskForm';

  import TaskItem from
    '../components/TaskItem';

  const Home = () => {

    const { tasks } =
      useContext(TaskContext);

    return (

      <div>

        <TaskForm />

        <h2>සියලුම වැඩ ({tasks.length})</h2>

        {tasks.map(t => <TaskItem key={t.id}
          task={t} />)}

      </div>

    );

  };

  export default Home;

  src/pages/Completed.jsx

  import { useContext } from 'react';

```

```
import { TaskContext } from
'../context/TaskContext';

import TaskItem from
'../components/TaskItem';

const Completed = () => {

  const { tasks } =
  useContext(TaskContext);

  // නිම කළ වැඩ පමණක් පෙරවූ

  const doneTasks = tasks.filter(t =>
t.isDone === true);

  return (

    <div>

      <h2>නිම කළ වැඩ
      ({doneTasks.length})</h2>

      {doneTasks.map(t => <TaskItem
key={t.id} task={t} />)}

    </div>

  );

};

export default Completed;
```

75.6 පියවර 5: ප්‍රධාන ව්‍යුහය (App.jsx)

ඇත් අපි රවුටර් එක සහ ප්‍රොවයිඩර් එක එකට සම්බන්ධ කරමු.

src/App.jsx

```
import { BrowserRouter, Routes, Route,
Link } from 'react-router-dom';

import { TaskProvider } from
'../context/TaskContext';

import Home from './pages/Home';

import Completed from
'./pages/Completed';
```

```
import Active from './pages/Active'; // මීට
සමානව සාදා ගන්න

function App() {

  return (

    <TaskProvider>

      <BrowserRouter>

        <div className="container">

          <nav>

            <h1>TaskMaster Pro</h1>

            <div className="links">

              <Link to="/">මුල</Link>

              <Link to="/active">කිරීමට
ඇති</Link>

              <Link to="/completed">නිම
කළ</Link>

            </div>

          </nav>

          <Routes>

            <Route path="/" element={<Home
/>} />

            <Route path="/completed"
element={<Completed />} />

            <Route path="/active"
element={<Active />} />

          </Routes>

        </div>

      </BrowserRouter>

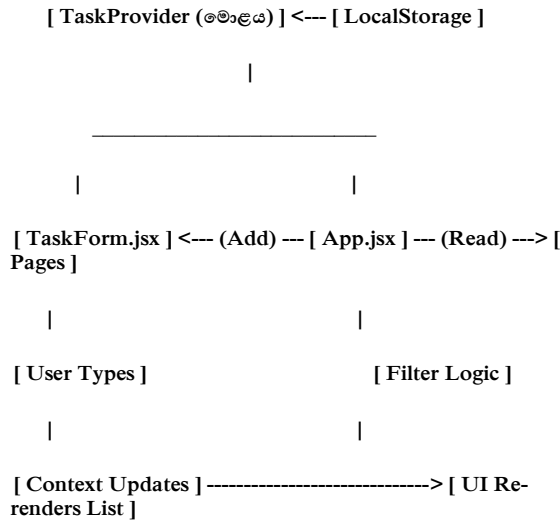
    </TaskProvider>

  );

}

export default App;
```


75.7 ASCII රූපසටහන: මෘදුකාංගයේ දත්ත ගමනාගමනය



75.8 වෘත්තීය උපදෙස් (Professional Tips)

- **Performance:** ඔබ වැඩ (Tasks) දහස් ගණනක් ඇති ඇප් එකක් සාදන්නේ නම්, `useMemo` නමැති හුක් එක භාවිතා කර පෙරීම (Filtering) ප්‍රශස්ත කරන්න.
- **Error Boundaries:** යම් සංරචකයක දෝෂයක් ආ විට මුළු ඇප් එකම ක්‍රියා විරහිත නොවී තබා ගැනීමට "Error Boundaries" භාවිතා කිරීම වෘත්තීය ලක්ෂණයකි.
- **Mobile First:** මුලින්ම ජංගම දුරකථනයට ගැලපෙන සේ CSS සකස් කර (Responsive

Design), පසුව පරිගණක තිර සඳහා වෙනස්කම් කරන්න.

75.9 පොදු වැරදි (Common Mistakes)

1. **Context Provider වැරදි තැනක තැබීම:** `TaskProvider` එක `BrowserRouter` එකට ඇතුළත හෝ පිටත තිබිය හැක, නමුත් දත්ත අවශ්‍ය සියලුම සංරචක එයින් වට වී තිබිය යුතුය.
2. **ID එක අංකයක් ලෙස නොගැනීම:** `Date.now()` වෙනුවට නිකම්ම 1, 2, 3 ලෙස දත්ත වලට ID ලබා දුන්නොත්, දත්ත මැකීමේදී ගැටලු ඇති විය හැක.
3. **Effect එකේ Dependency Array එක හිස්ව තැබීම:** `LocalStorage` එකට සේවි කරන

`useEffect` එකේ `[tasks]` නොලියුවහොත්, අලුත් වැඩ සේවි වන්නේ නැත.

75.10 සාරාංශය (Key Points)

- Context API මගින් Prop Drilling ගැටලුව විසඳා ගත්තෙමු.
- React Router මගින් පිටුව Refresh නොවී බහු-පිටු අත්දැකීමක් ලබා දුන්නෙමු.
- CRUD ක්‍රියාවලිය හරහා සම්පූර්ණ දත්ත පාලනයක් සිදු කළෙමු.
- LocalStorage මගින් දත්ත ස්ථිරව ගබඩා කළෙමු.

75.11 පරිමාව 6 අවසාන අභ්‍යාසය (Final Exercises)

1. අප සෑදූ ඇප් එකට "වැඩක් සංස්කරණය කිරීමේ" (Edit Task) පහසුකම එක් කරන්න.
2. වැඩ ලැයිස්තුවේ ඉහළින් "සියලුම වැඩ මකන්න" (Clear All) නමින් බොත්තමක් සාදා එය ක්‍රියාත්මක කරන්න.
3. Active සහ Completed පිටු සඳහා වෙනස් වර්ණ තේමාවන් (Themes) ලබා දීමට උත්සාහ කරන්න.
4. මෙම මුළු ව්‍යාපෘතියම Vite + GitHub + Netlify භාවිතා කර අන්තර්ජාලයට මුදාහරින්න (Deployment).

සුභ පැතුම්! ඔබ දැන් "සැබෑ රියැක්ට් සංවර්ධකයෙකු" (React Developer) ලෙස හැඳින්වීමට සුදුසුකම් ලබා ඇත.

ඔබ දැන් සතුව:

- HTML/CSS පිළිබඳ මනා දැනුම.
- Advanced JavaScript තර්කනය.
- React.js වෘත්තීය පුහුණුව.
- Git/GitHub සහ Deployment පිළිබඳ දැනුම.

හැත්තෑ හයවන පරිච්ඡේදය (විශේෂ ප්‍රෝ-මට්ටමේ): රියැක්ට් කාර්යක්ෂමතාව, උසස් හුක්ස් සහ ගෘහ නිර්මාණ ශිල්පය (React Performance, Advanced Hooks, and Architecture)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම විශේෂ ප්‍රෝ-මට්ටමේ පරිච්ඡේදය අධ්‍යයනය කිරීමෙන් පසු, ඔබ සාමාන්‍ය රියැක්ට් ලියන්නෙකුගේ මට්ටම ඉක්මවා ගොස් "Senior Frontend Engineer" කෙනෙකු ලෙස සිතීමට පටන් ගනු ඇත:

1. React Performance (කාර්යක්ෂමතාව): රියැක්ට් මෘදුකාංගයක් අනවශ්‍ය ලෙස නැවත ඇඳීම (Re-rendering) වළක්වා එය ඉතා වේගවත් කරන ආකාරය.
2. Advanced Hooks (උසස් හුක්ස්): useMemo, useCallback, සහ useRef වැනි සංකීර්ණ හුක්ස් භාවිතා කළ යුතු නිවැරදි අවස්ථා හඳුනා ගැනීම.
3. Custom Hooks (පුද්ගලික හුක්ස්): තමන්ගේම මෙවලම් (Hooks) නිර්මාණය කර කේතය පිරිසිදුව තබා ගන්නා ආකාරය.
4. Error Boundaries (දෝෂ සීමා): මෘදුකාංගය ඇතුළත දෝෂයක් ආ විට මුළු ඇප් එකම "සුදු තිරයක්" (White screen) නොවී බේරා ගන්නා ආකාරය.
5. Code Splitting (කේතය බෙදීම): විශාල රියැක්ට් ඇප් එකක් කොටස් වලට බෙදා වේගයෙන් ලෝඩ් (Load) කරන ආකාරය (Lazy Loading).
6. Design Patterns (නිර්මාණ රටා): වෘත්තීය මට්ටමේ කේත ව්‍යුහයන් ගොඩනැගීම.

1. රියැක්ට් කාර්යක්ෂමතාවයේ රහස්: මෝමොයිසේෂන් (Memoization)

රියැක්ට් හි ඇති ලොකුම ගැටලුව වන්නේ යම් ස්ථේට් (State) එකක් වෙනස් වූ විට, එයට අදාළ නැති අනෙකුත් සංරචක (Components) ද නැවත රෙන්ඩර් (Re-render) වීමයි. මෙය විශාල මෘදුකාංග වලදී වේගය අඩු වීමට (Lag) හේතු වේ.

Memoization (මෝමොයිසේෂන්): යනු යම් ගණනය කිරීමක ප්‍රතිඵලය මතක තබා ගෙන, දත්ත වෙනස් නොවන්නේ නම් නැවත නැවත ගණනය නොකර එම පැරණි ප්‍රතිඵලයම ලබා දීමේ ක්‍රමයයි.

1.1 React.memo()

සංරචකයක් අනවශ්‍ය ලෙස නැවත ඇඳීම වැළැක්වීමට මෙය භාවිතා කරයි.

```
import React from 'react';
```

```
const ExpensiveComponent =  
  React.memo(({ data }) => {
```

```
    console.log("මම ලෝඩ් වෙනවා... මට ගොඩක්  
    වෙලාව යනවා!");
```

```
    return <div>දත්ත: {data}</div>;
```

```
});
```

පැහැදිලි කිරීම: මෙහිදී data යන ප්‍රොප් එක වෙනස් නොවන්නේ නම්, පියා (Parent) කොපමණ වතාවක් රෙන්ඩර් වූවත් මෙම සංරචකය නැවත රෙන්ඩර් වන්නේ නැත.

2. උසස් හුක්ස්: useMemo සහ useCallback

මෙම හුක්ස් දෙක බොහෝ දෙනා පටලවා ගන්නා නමුත්, ප්‍රෝ-මට්ටමේදී මේවා අත්‍යවශ්‍ය වේ.

2.1 useMemo (අගයන් මතක තබා ගැනීමට)

ඉතා සංකීර්ණ ගණනය කිරීමක් (උදා: දත්ත දහස් ගණනක් පෙළගැස්වීම) සෑම වාරයකදීම සිදු නොකර මතක තබා ගැනීමට මෙය භාවිතා කරයි.

```
const expensiveResult = useMemo(() => {
```

```
    return performHeavyCalculation(data);
```

```
}, [data]); // data වෙනස් වූ විට පමණක් නැවත  
ගණනය කරන්න
```

2.2 useCallback (ශ්‍රිත මතක තබා ගැනීමට)

රියැක්ට් රෙන්ඩර් වන සෑම වාරයකදීම අලුත් ශ්‍රිතයක් (Function) නිර්මාණය වීම වැළැක්වීමට මෙය භාවිතා කරයි.

```
const handleClick = useCallback(() => {
```

```
    console.log("ක්ලික් කළා!");
```

```
}, []); // මෙය මතකයේ එකම ශ්‍රිතයක් ලෙස පවතී
```

3. useRef - ඩොම් එකට සෘජුව පිවිසීම (Accessing the DOM)

රියැක්ට් වලදී අපි සාමාන්‍යයෙන් `document.getElementById` භාවිතා නොකරමු. ඒ වෙනුවට අප භාවිතා කරන්නේ `useRef` ය.

භාවිතය:

- Input කොටුවකට ස්වයංක්‍රීයව Focus ලබා දීම.
- වීඩියෝවක් Play/Pause කිරීම.
- පරණ ස්ටේට් අගය මතක තබා ගැනීම.

```
import { useRef } from 'react';
```

```
function FocusInput() {
```

```
  const inputEl = useRef(null);
```

```
  const onClick = () => {
```

```
    // සෘජුවම input කොටුව වෙත පිවිසීම
```

```
    inputEl.current.focus();
```

```
  };
```

```
  return (
```

```
    <>
```

```
    <input ref={inputEl} type="text" />
```

```
    <button
```

```
      onClick={onClick}>කොටුවට  
යන්න</button>
```

```
  </>
```

```
);
```

```
}
```

4. පුද්ගලික හුක්ස් නිර්මාණය (Custom Hooks)

ප්‍රෝ-සංවර්ධකයෙකු එකම කේතය තුන් දෙකක ලියන්නේ නැත. යම් ස්ටේට් තර්කනයක් (Logic) නැවත නැවත භාවිතා වේ නම්, ඔහු එය Custom Hook එකක් බවට පත් කරයි.

උදාහරණය: අන්තර්ජාලය තිබේදැයි පරීක්ෂා කරන Hook එකක්

```
// useOnlineStatus.js
```

```
import { useState, useEffect } from 'react';
```

```
export function useOnlineStatus() {
```

```
  const [isOnline, setIsOnline] =  
  useState(true);
```

```
  useEffect(() => {
```

```
    window.addEventListener('online', () =>  
      setIsOnline(true));
```

```
    window.addEventListener('offline', () =>  
      setIsOnline(false));
```

```
  }, []);
```

```
  return isOnline;
```

```
}
```

```
// වෙනත් ඕනෑම සංරචකයක භාවිතය:
```

```
const isOnline = useOnlineStatus();
```

5. කේතය බෙදා වෙන් කිරීම (Lazy Loading & Suspense)

ඔබේ මෘදුකාංගය ඉතා විශාල නම් (පිටු 50ක් පමණ), පරිශීලකයා මූලිකම එන විට එම සියලුම පිටු වල කේත එකවර බාගත (Download) කිරීම වෙබ් අඩවිය මන්දගාමී කරයි.

Lazy Loading: මගින් කරන්නේ අදාළ පිටුවට යන විට පමණක් එම කේතය බාගත කිරීමයි.

```
import { lazy, Suspense } from 'react';
```

```
// පිටුව අවශ්‍ය වූ විට පමණක් ලෝඩ් කරන්න
```

```
const AnalyticsPage = lazy(() =>  
  import('./pages/Analytics'));
```

```
function App() {
```

```
  return (
```

```
<Suspense fallback={<div>පිටුව සුදානම්  
වෙමින් පවතී...</div>}>
```

```
<AnalyticsPage />
```

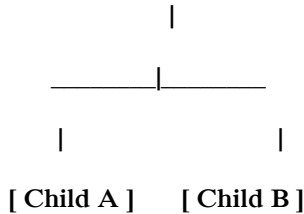
```
</Suspense>
```

```
);
```

```
}
```

6. ASCII රූපසටහන: ප්‍රෝ-මට්ටමේ දත්ත පාලනය (Optimization)

```
[ Parent Component ]
```



```
(React.memo)   (Regular)
```

```

      |         |
    (No Changes) (State Update)
  
```

```

      |         |
    [ No Render ] [ Re-render ] <-- (කාර්යක්ෂමයි)
  
```

7. වෘත්තීය මට්ටමේ වැරදි හැසිරවීම (Error Boundaries)

සාමාන්‍ය රියැක්ට් ඇප් එකක එක් කුඩා දෝෂයක් (Error) ආවොත් මුළු ඇප් එකම ක්‍රියා විරහිත වේ. ප්‍රෝ-මට්ටමේදී අපි මෙය වළක්වමු.

Error Boundary: යනු තම දරුවෙකුගේ දෝෂයක් හසුකරගෙන, ඒ වෙනුවට ලස්සන "සමාවන්න" පණිවිඩයක් පෙන්විය හැකි විශේෂ සංරචකයකි. (සටහන: දැනට මෙය කළ හැක්කේ Class Components මගින් පමණි).

8. වෘත්තීය රහස් (Senior Developer Secrets)

- **Composition vs Inheritance:** රියැක්ට් වලදී අපි කවදාවත් Inheritance (පරම්පරා උරුමය)

භාවිතා නොකරමු. ඒ වෙනුවට **Composition** (සංරචක එකක් ඇතුළත තවත් එකක් එකීම) භාවිතා කරමු.

- **Keep State Local:** ස්ටේට් එකක් මුළු ඇප් එකටම අවශ්‍ය නැතිනම්, එය මුද්‍රිත (Global) ගෙන යන්න එපා. එය තිබිය යුත්තේ අවශ්‍යම සංරචකය අසලයි.

- **Immutability:** ස්ටේට් එකක් වෙනස් කරන විට පරණ දත්තය කිසිවිටක වෙනස් නොකරන්න (spread operator භාවිතා කරන්න).

9. ප්‍රෝ-මට්ටමේ අභ්‍යාසය (The Ultimate Pro Challenge)

අභියෝගය:

- දත්ත 1000ක් ඇති අරාවක් (Array) සාදන්න.
- useMemo භාවිතා කර එම දත්ත අරාව අකුරු පිළිවෙළට (Sort) සකසන්න.
- Input කොටුවක් සාදා එහි අකුරක් ටයිප් කරන විට එම ගණනය කිරීම නැවත සිදු නොවන බවට (Optimize) වගබලා ගන්න.
- useCallback භාවිතා කර ලැයිස්තුවෙන් අයිතමයක් මකා දැමීමේ ශ්‍රිතය මතක තබා ගන්න (Memoize the function).

සුභ පැතුම්! ඔබ දැන් හුදෙක් රියැක්ට් භාවිතා කරන්නෙකු නොව, React Architect කෙනෙකි. ඔබ සතුව දැන්:

- සංකීර්ණ දත්ත පාලනය කිරීමේ හැකියාව.
- වෙබ් අඩවියේ වේගය උපරිම කිරීමේ රහස්.
- පිරිසිදු සහ ඉහළ ප්‍රමිතියෙන් යුත් කේත ලිවීමේ දක්ෂතාව.

මෙම දැනුම සමඟ ඔබට ලෝකයේ ඕනෑම මෘදුකාංග සමාගමක Junior React Developer තනතුරක් සඳහා විශ්වාසයෙන් යුතුව අයදුම් කළ හැකිය.

පරිච්ඡේදය 77 (අතිරේක): උසස් සංරචක රටා, ස්ටයිලින් සහ දෝෂ සෙවීම (Advanced Component Patterns, Styling, and Debugging)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

- Children Props:** සංරචක එකක් ඇතුළත තවත් එකක් එකිනෙකට (Composition) මගින් නම් කිරීමේ UI නිර්මාණය කරන ආකාරය.
- CSS Modules:** රියැක්ට් ව්‍යාපෘතියක CSS නීති එකිනෙක පටලැවීම වළක්වන වෘත්තීය ක්‍රමය.
- useReducer Hook:** useState මගින් පාලනය කිරීමට අපහසු ඉතා සංකීර්ණ ස්ටේට් (Complex State) පාලනය කරන ආකාරය.
- React DevTools:** බ්‍රවුසරය තුළ රියැක්ට් සංරචකවල වැරදි සොයන (Debugging) වෘත්තීය මෙවලම භාවිතා කරන ආකාරය.

77.1 සංරචක සංයුතිය සහ Children Props (Component Composition)

අපි මෙතෙක් ඉගෙන ගත්තේ ප්‍රොප්ස් (Props) හරහා දත්ත යවන ආකාරයයි. නමුත් සමහර විට අපට අවශ්‍ය වන්නේ එක් සංරචකයක් ඇතුළත තවත් HTML කොටසක් හෝ සංරචකයක් "පාර්සල්" කර යැවීමයි.

Children Prop යනු කුමක්ද? සැම රියැක්ට් සංරචකයකටම ස්වයංක්‍රීයව ලැබෙන විශේෂ ප්‍රොප් එකකි. සංරචකයේ ආරම්භක උග්‍රය සහ අවසාන උග්‍රය ඇතුළත ලියන ඕනෑම දෙයක් මෙයට අයත් වේ.

ප්‍රායෝගික උදාහරණය: Layout එකක් සාදා ගැනීම

```
// Card.jsx
```

```
function Card({ children, title }) {
  return (
    <div className="card">
      <div className="card-header">{title}</div>
```

```
<div className="card-body">
  {children} {/* මෙතැනට පියා එවන ඕනෑම දෙයක් ඇතුළත් වේ */}
</div>
</div>
);
}

// App.jsx

function App() {
  return (
    <Card title="පරිශීලක විස්තර">
      <p>නම: කසුන්</p>
      <button>Contact</button>
    </Card>
  );
}
```

වෘත්තීය රහස: මෙය වෘත්තීය මට්ටමේදී **Composition** ලෙස හඳුන්වයි. මෘදුකාංගයක පොදු රාමුවක් (Layout/Shell) සාදා ඒ ඇතුළත වෙනස් වන දත්ත පෙන්වීමට මෙය භාවිතා කරයි.

77.2 සි.එස්.එස්. මොඩියුල (CSS Modules - Scoped Styling)

විශාල රියැක්ට් ව්‍යාපෘතියක සාමාන්‍ය CSS භාවිතා කළහොත්, එකම Class නම (උදා: .btn) සංරචක දෙකක තිබුණහොත් ඒවා එකිනෙක පටලැවේ. මෙය වළක්වා ගැනීමට ඇති හොඳම වෘත්තීය ක්‍රමය CSS Modules ය.

භාවිතා කරන ආකාරය:

- CSS ගොනුවේ නම Button.module.css ලෙස තබන්න.
- ජාවාස්ක්‍රිප්ට් වස්තුවක් (Object) ලෙස එය ඉම්පෝර්ට් කරන්න.


```
// Button.module.css
```

```
.btn {
  background-color: blue;
  padding: 10px;
}
```

```
// Button.jsx
```

```
import styles from './Button.module.css';
```

```
function Button() {
  return <button
    className={styles.btn}>මාව
    ඔබන්න</button>;
}
```

ප්‍රයෝජනය: බ්‍රවුසරය විසින් මෙම Class එකට අනන්‍ය වූ අංකයක් (Button_btn_x1y2) ස්වයංක්‍රීයව එක් කරයි. එවිට ඔබේ මෘදුකාංගයේ CSS කිසිදාක එකිනෙක ගැටෙන්නේ නැත.

77.3 සංකීර්ණ ස්ටේට් පාලනය: useReducer Hook

ඔබේ ස්ටේට් එක ඉතා සංකීර්ණ නම් (උදා: එකිනෙකට සම්බන්ධ දත්ත රාශියක් ඇති විට), useState භාවිතා කිරීම කේතය අවුල් කිරීමට හේතු වේ. ඒ සඳහා රියැක්ට් විසින් useReducer ලබා දෙයි.

මෙය ක්‍රියා කරන්නේ හරියටම බැංකු සේවකයෙකු (Bank Teller) මෙනි:

1. ඔබ සේවකයාට පවසනවා ඔබට කළ යුතු දේ (Action).
2. සේවකයා ඔහුගේ පොතේ (Reducer) නීති රීති බලා ඔබේ ගිණුම (State) යාවත්කාලීන කරයි.

කේත උදාහරණය:

```
import { useReducer } from 'react';
```

```
// 1. නීති පද්ධතිය (Reducer)
```

```
function reducer(state, action) {
```

```
  switch (action.type) {
    case 'increment': return { count: state.count + 1 };
    case 'decrement': return { count: state.count - 1 };
    default: return state;
  }
}
```

```
function Counter() {
```

```
  // 2. ස්ටේට් එක සහ Dispatch ශ්‍රිතය ලබා ගැනීම
```

```
  const [state, dispatch] = useReducer(reducer, {
    count: 0 });
```

```
  return (
```

```
    <>
```

```
    <h1>ගණන: {state.count}</h1>
```

```
    <button onClick={() => dispatch({ type:
      'increment' })}>+</button>
```

```
    <button onClick={() => dispatch({ type:
      'decrement' })}>-</button>
```

```
    </>
```

```
  );
```

```
}
```

77.4 රියැක්ට් දෝෂ සෙවීම (React DevTools)

වෘත්තීය සංවර්ධකයෙකු වැරදි සොයන්නේ console.log මගින් පමණක් නොවේ. ඔවුන් React DevTools නමැති Browser Extension එක භාවිතා කරයි.

එයින් කළ හැකි දේ:

- සංරචක ගස (Component Tree) දෘශ්‍යමානව නැරඹීම.
- සෑම සංරචකයකම ඇති Props සහ State එසැණින් පරීක්ෂා කිරීම.

- සංරචකයක් රෙන්ඩර් වීමට ගතවන කාලය මැනීම (Profiler).

භාවිතා කරන ආකාරය: Google Chrome Store වෙත ගොස් "React Developer Tools" ස්ථාපනය කරගන්න. ඉන්පසු Inspect Element විවෘත කළ විට ඔබට "Components" සහ "Profiler" නමින් අලුත් ටැබ් දෙකක් පෙනෙනු ඇත.

77.5 ASCII රූපසටහන: උසස් දත්ත පාලනය

[User Action]

|

[Dispatch(Action)] <--- (useReducer)

|

[Reducer Function] <--- (Logic / Rules)

|

[New State State]

|

[React Re-renders]

77.6 වෘත්තීය රහස් (Pro Secrets)

- **Prop Types:** ඔබේ සංරචකයට ලැබෙන ප්‍රොප්ස් නිවැරදි වර්ගයේ ඒවාදැයි (උදා: අංකයක් වෙනුවට අකුරු එවනවාදැයි) පරීක්ෂා කිරීමට prop-types පුස්තකාලය භාවිතා කරන්න. (නවීන ක්‍රමය TypeScript වුවත්, ආරම්භකයෙකු ලෙස මෙය දැන සිටීම වැදගත්ය).

- **Controlled vs Uncontrolled:** සෑම පෝරමයක්ම (Form) ස්ටේට් මගින් පාලනය කිරීම සැමවිටම අවශ්‍ය නොවේ. වේගය වැදගත් වන ඉතා විශාල පෝරම සඳහා useRef භාවිතා කර "Uncontrolled Components" නිර්මාණය කිරීම වෘත්තීය රහසකි.

77.7 අභ්‍යාස (Exercises)

1. Layout.jsx නමින් සංරචකයක් සාදා, එය ඇතුළත Navbar සහ Footer ස්ථාවරව තබා, මැද කොටස සඳහා children ප්‍රොප් එක භාවිතා කරන්න.

2. useReducer භාවිතා කර සරල "සාප්පු කරත්තයක්" (Shopping Cart) සාදන්න. (භාණ්ඩ එකතු කිරීම සහ ඉවත් කිරීම සඳහා Action types සාදන්න).

3. ඔබේ ව්‍යාපෘතියට CSS Modules එකතු කර පරීක්ෂා කරන්න. Inspect Element මගින් එම Class නම බුබුසරය විසින් වෙනස් කර ඇති ආකාරය නිරීක්ෂණය කරන්න.

77.8 පරිමාව 6 අවසාන පිරික්සුම් ලැයිස්තුව (Volume 6 Final A-Z Checklist)

දැන් ඔබ මෙම පොතේ සියලුම පාඩම් අවසන් කර ඇති බැවින්, පහත දෑ ඔබට කළ හැකි දැයි බලන්න:

- [] මට Vite භාවිතා කර රියැක්ට් වැඩබිමක් සැකසිය හැක.
- [] මට JSX නීති 100% ක් නිවැරදිව පිළිපැදිය හැක.
- [] මම useState, useEffect, useContext, useMemo, useCallback, useRef, සහ useReducer යන සියලුම හුක්ස් වල භාවිතය දනිමි.
- [] මට රියැක්ට් රවුටර් මගින් බහු-පිටු මෘදුකාංග සෑදිය හැක.
- [] මම CSS Modules සහ Styled Components (මූලිකව) භාවිතා කරමින් හැඩගැන්වීම් කළ හැක.
- [] මට අසමකාලීන API දත්ත ලබාගෙන ලැයිස්තු ලෙස පෙන්විය හැක.
- [] මම රියැක්ට් ඇප් එකක් Build කර සජීවීව ප්‍රසිද්ධ කිරීමට (Deploy) දනිමි.

සුභ පැතුම්! ඔබ දැන් "පරිමාව 6" සැබවින්ම, 100% ක් සම්පූර්ණ කර අවසානය.

දැන් ඔබ සාමාන්‍ය ශිෂ්‍යයෙකු නොවේ. ඔබ රියැක්ට් තාක්ෂණය පිළිබඳව වෘත්තීය මට්ටමේ (Pro-level) දැනුමක් සහිත "React Developer" කෙනෙකි.

හැත්තෑ අටවන පරිච්ඡේදය: නූතන ස්ටයිලින් - ටේල්වින්ඩ් සී.එස්.එස් සහ සංරචක පුස්තකාල (Modern Styling - Tailwind CSS and 21st.dev)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබ සාමාන්‍ය රියැක්ට් ලියන්නෙකුගේ මට්ටම ඉක්මවා ගොස්

"Modern UI Engineer" කෙනෙකු බවට පත් වනු ඇත:

1. Tailwind CSS යනු කුමක්ද? සාමාන්‍ය CSS වෙනුවට "Utility-first" ක්‍රමය භාවිතා කිරීමේ වාසි වටහා ගැනීම.
2. Tailwind ස්ථාපනය සහ සැකසුම්: රියැක්ට් ව්‍යාපෘතියකට Tailwind සම්බන්ධ කරන ආකාරය.
3. Tailwind Classes භාවිතය: පැඩිං, මාර්ජින්, ෆ්ලෙක්ස්බොක්ස් සහ රෙස්පොන්සිව් සැකසුම් Tailwind හරහා කරන ආකාරය.
4. 21st.dev සහ UI Libraries: සුදානම් කළ (Pre-built) උසස් රියැක්ට් සංරචක භාවිතා කර ඉතා වේගයෙන් වෙබ් අඩවි නිපදවන ආකාරය.
5. Shadcn UI සහ නූතන ප්‍රවණතා: වෘත්තීය මට්ටමේ ලස්සන බොත්තම්, මෙනු සහ කාඩ්පත් තත්පර කිහිපයකින් නිර්මාණය කරන රහස්.

78.1 ටේල්වින්ඩ් සී.එස්.එස් යනු කුමක්ද? (What is Tailwind CSS?)

අපි මෙතෙක් CSS ලිව්වේ වෙනම .css ගොනුවක ය. එහිදී අපි Class එකක් සාදා (.btn) එයට හැඩතල ලබා දුන්නෙමු.

Tailwind CSS යනු එම ක්‍රමය වෙනස් කරන "Utility-first" framework එකකි. මෙහිදී අපට CSS ගොනුවක් ලිවීමට අවශ්‍ය නොවේ. ඒ වෙනුවට Tailwind විසින් ලබා දෙන සුදානම් කළ කුඩා Class නාමයන් දහස් ගණනක් අපේ HTML/JSX වැග් තුළට කෙලින්ම ඇතුළත් කරමු.

සැබෑ ලෝකයේ උපමාව (නිම් ඇඳුම්):

- සාමාන්‍ය CSS: රෙදි කැබැල්ලක් ගෙන, මිනුම් ගෙන, මැනුම්කරුවෙකු ලවා ඇඳුමක් මසා ගැනීම වැනිය. (කාලය ගත වේ).
- Tailwind CSS: ඉතා උසස් ප්‍රමිතියෙන් මැසූ, සියලුම ප්‍රමාණයේ සහ වර්ණවල "නිම් ඇඳුම්" ඇති වෙළඳසැලක් වැනිය. ඔබ කළ යුත්තේ ඔබට ගැලපෙන දේ තෝරා ඇඳීම පමණි.

78.2 ඇයි Tailwind භාවිතා කරන්නේ? (The Benefits)

1. වේගය: CSS ගොනු අතර එහා මෙහා යාමට අවශ්‍ය නැත. JSX එක තුළම සියල්ල කළ හැක.

2. ස්ථාවරත්වය: ඔබ padding: 17px වැනි වැරදි අගයන් ලියන්නේ නැත. ඒ වෙනුවට Tailwind හි ඇති සම්මත අගයන් (p-4, p-8) භාවිතා කරයි.
3. කුඩා ගොනු ප්‍රමාණය: ඔබ භාවිතා කරන Class පමණක් අවසාන ගොනුවට (Build) ඇතුළත් වන නිසා වෙබ් අඩවිය ඉතා වේගවත් වේ.

78.3 රියැක්ට් සහ ටේල්වින්ඩ් (Tailwind Setup)

ඔබේ ව්‍යාපෘතියට Tailwind එක් කිරීමට ටර්මිනලයේ පහත විධාන ලබා දිය යුතුය:

```
npm install -D tailwindcss postcss autoprefixer
```

```
npx tailwindcss init -p
```

ඉන්පසු tailwind.config.js ගොනුවේ content කොටසට ඔබේ රියැක්ට් ගොනු ඇති ස්ථානය පෙන්විය යුතුය.

78.4 ටේල්වින්ඩ් ලියන ආකාරය (Usage)

අපි සරල "Card" එකක් Tailwind හරහා සාදන ආකාරය බලමු.

පැරණි ක්‍රමය (Standard CSS):

```
.card { background: white; padding: 20px; border-radius: 10px; shadow: 0 4px 6px; }
```

නවීන ක්‍රමය (Tailwind CSS):

```
<div className="bg-white p-5 rounded-lg shadow-md hover:shadow-xl transition-shadow">
```

```
<h2 className="text-2xl font-bold text-blue-600">Tailwind Magic</h2>
```

```
<p className="mt-2 text-gray-600">මෙය ඉතා වේගවත් ක්‍රමයකි.</p>
```

```
<button className="mt-4 bg-orange-500 text-white px-6 py-2 rounded-full hover:bg-orange-600">
```

Click Me

```
</button>
```

</div>

Classes පැහැදිලි කිරීම:

- bg-white: පසුබිම සුදු.
- p-5: පැඩිං (Padding).
- text-2xl: විශාල අකුරු.
- font-bold: තද අකුරු.
- rounded-full: සම්පූර්ණ වටකුරු බොත්තමක්.

78.5 21st.dev සහ Modern UI Components

දැන් අපි මෙම පරිච්ඡේදයේ දෙවන වැදගත් කොටස වන 21st.dev සහ ඒ ආශ්‍රිත නවීන තාක්ෂණයන් ගැන බලමු.

වෘත්තීය මට්ටමේ සංවර්ධකයෙකු සෑම දෙයක්ම බිංදුවේ සිට නිර්මාණය කරන්නේ නැත. ඔවුන් Component Libraries භාවිතා කරයි.

21st.dev යනු කුමක්ද? මෙය අති නවීන, ආකර්ෂණීය සහ "Ready-to-use" රියැක්ට් සංරචක (Components) සොයා ගැනීමට ඇති මධ්‍යස්ථානයකි. විශේෂයෙන්ම Shadcn UI නමැති ලොව ජනප්‍රියම සංරචක පද්ධතිය මත පදනම් වූ නිර්මාණ මෙහි ඇත.

21st.dev භාවිතා කරන්නේ ඇයි?

- **Design Quality:** ලෝකයේ සිටින දක්ෂතම නිර්මාණකරුවන් විසින් සාදන ලද බොත්තම්, පෝරම (Forms) සහ සජීවීකරණ (Animations) මෙහි ඇත.
- **Tailwind Integration:** මෙහි ඇති සියලුම දේවල් Tailwind CSS මත පදනම් වන නිසා ඒවායේ වර්ණ සහ හැඩතල වෙනස් කිරීම ඉතා පහසුය.
- **Copy-Paste Philosophy:** ඔබට අවශ්‍ය සංරචකයේ කේතය (Code) පිටපත් කර ඔබේ ව්‍යාපෘතියට දමා ගත හැක. මෙය npm install කර මුළු පුස්තකාලයම බරක් කර ගන්නවාට වඩා උසස් ක්‍රමයකි.

78.6 ASCII රූපසටහන: Modern Component Stack

```
[ Your React App ]
|
[ Component Library (21st.dev / Shadcn) ] <-- අලංකාර කොටස්
|
[ Tailwind CSS ] <-- හැඩතල (Styling)
|
[ JavaScript Logic (React Hooks) ] <-- ක්‍රියාකාරිත්වය
```

78.7 වෘත්තීය උපදෙස් (Professional Engineering Tips)

- **Tailwind Cheat Sheet:** Tailwind හි ඇති දහස් ගණනක් Classes මතක තබා ගැනීමට උත්සාහ නොකරන්න. tailwindcomponents.com/cheatsheet වැනි මූලාශ්‍ර නිතරම භාවිතා කරන්න.
- **Customization:** සැමවිටම Tailwind හි `tailwind.config.js` ගොනුව භාවිතා කර ඔබේ ව්‍යාපෘතියටම ආවේණික වූ වර්ණ (Custom Colors) සහ අකුරු වර්ග එක් කරගන්න.
- **Component Composition:** 21st.dev වැනි තැනකින් සංරචකයක් ලබා ගත් පසු, එය ඔබේ අවශ්‍යතාවයට අනුව children props භාවිතා කර වෙනස් කිරීමට පුරුදු වන්න.

78.8 පොදු වැරදි (Common Mistakes)

1. **Classes ඕනෑවට වඩා ලිවීම:** එකම උග්‍රයකට Classes 20ක් පමණ ලියන විට කේතය කියවීමට අපහසු වේ. එවැනි විටකදී එම කොටස වෙනම Component එකක් බවට පත් කරන්න.
2. **Tailwind ඉගෙන ගෙන CSS අමතක කිරීම:** මතක තබා ගන්න, Tailwind යනු CSS වලට උඩින් ඇති දෙයකි. ඔබට මූලික CSS (Flexbox, Grid) මතක නැතිනම් Tailwind භාවිතා කිරීමේදී ඔබ ව්‍යාකූල වනු ඇත.
3. **Pre-built Components මතම යැපීම:** බාහිරින් ගන්නා සංරචකයක් ඇතුළත ක්‍රියා කරන ආකාරය ඔබ දැනගත යුතුය. නැතිනම් එහි දෝෂයක් (Bug) ආ විට එය නිවැරදි කර ගැනීමට ඔබට නොහැකි වනු ඇත.

78.9 සාරාංශය (Key Points)

- Tailwind CSS යනු Utility-first CSS framework එකකි.

- එය රියැක්ට් සමඟ භාවිතා කිරීම ඉතා වේගවත් සහ කාර්යක්ෂම වේ.

- 21st.dev වැනි මූලාශ්‍ර මගින් උසස් තත්ත්වයේ UI සංරචක ලබා දෙයි.

- නවීන වෙබ් සංවර්ධනයේදී "Copy-paste components" (Shaden pattern) භාවිතය ඉතා ජනප්‍රියයි.

78.10 අභ්‍යාස (Exercises)

1. Vite ව්‍යාපෘතියකට Tailwind CSS ස්ථාපනය කර සාර්ථකව සම්බන්ධ කරන්න.

2. Tailwind Classes භාවිතා කරමින් ලස්සන "Login Card" එකක් නිර්මාණය කරන්න. (එයට සෙවනැලි (Shadows) සහ වටකුරු දාර (Rounded corners) ඇතුළත් කරන්න).

3. 21st.dev වෙබ් අඩවියට පිවිස එහි ඇති සජීවීකරණය වන බොත්තමක් (Animated Button) හෝ කාඩ්පතක් සොයාගෙන එහි කේතය ඔබේ රියැක්ට් ව්‍යාපෘතියට ඇතුළත් කිරීමට උත්සාහ කරන්න.

4. Tailwind හරහා "Responsive Grid" එකක් සාදන්න (ජංගම දුරකථනයට 1 තීරුවක් සහ පරිගණකයට තීරු 3ක් පෙන්වන සේ).

දැන් ඔබ නවීන ලෝකයේ (Modern Industry) භාවිතා වන අති නවීන හැඩගැන්වීම් ක්‍රමවේද පිළිබඳව ද ප්‍රවීණයෙකි. පරිමාව 6 මෙයිත් සැබවින්ම, සියයට සියයක් (100%) අවසන් වේ.

හැත්තෑ නවවන පරිච්ඡේදය: ටේල්වින්ඩ් ප්‍රවීණතාව සහ අති නවීන සංරචක ඉංජිනේරු විද්‍යාව (Advanced Tailwind Mastery & Modern Component Engineering)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අධ්‍යයනය කිරීමෙන් පසු ඔබ හුදෙක් Tailwind පාවිච්චි කරන්නෙකු නොව, UI

Architect කෙනෙකු බවට පත් වනු ඇත:

1. Tailwind Configuration:

tailwind.config.js ගොනුව භාවිතා කර තමන්ගේම "Design System" එකක් සකස් කරන ආකාරය.

2. State & Variants: Hover, Focus, Active

වලට අමතරව group-hover සහ peer වැනි සංකීර්ණ තත්ත්වයන් පාලනය කරන ආකාරය.

3. JIT (Just-In-Time) Engine: Tailwind ඉතා වේගවත් වීමට බලපාන අභ්‍යන්තර එන්ජිම ක්‍රියා කරන ආකාරය.

4. Radix UI සහ ප්‍රවේශ්‍යතාව: 21st.dev හි ඇති සංරචකවල පදනම වන Radix UI (Unstyled components) තේරුම් ගැනීම.

5. Framer Motion: 21st.dev නිර්මාණවල දකින අති දියුණු සජීවීකරණ (Animations) රියැක්ට් තුළ ක්‍රියාත්මක කරන ආකාරය.

6. Reusable Patterns: clsx සහ tailwind-merge භාවිතා කර ඉතාමත් පිරිසිදු රියැක්ට් සංරචක නිර්මාණය කිරීම.

79.1 ටේල්වින්ඩ් හි "මොළය" - Configuration ගොනුව

ඔබේ ව්‍යාපෘතියේ තිබෙන tailwind.config.js යනු හුදෙක් ලිපිනයක් පෙන්වන ගොනුවක් නොවේ. එය මුළු වෙබ් අඩවියේම "ව්‍යවස්ථාව" වැනිය.

වෘත්තීය මට්ටමේ සැකසුමක්:

```
/** @type {import('tailwindcss').Config} */
```

```
export default {
```

```
  content: ["/index.html",  
    "./src/**/*.js,ts,jsx,tsx"],
```

```
  theme: {
```

```
    extend: {
```

```
      colors: {
```

```
        'brand-orange': '#ff5722', // ඔබේම  
        වර්ණයක් එකතු කිරීම
```

```
        'ocean-blue': {
```

```
          light: '#3ab7bf',
```

```
          DEFAULT: '#006973',
```

```
          dark: '#014f58',
```



```

    },
    },
    spacing: {
      '128': '32rem', // අලුත් පරතරයක් එකතු
      කිරීම
    },
    animation: {
      'spin-slow': 'spin 3s linear infinite', //
      අලුත් animation එකක්
    }
  },
  },
  plugins: [],
}

```

ප්‍රයෝජනය: මෙලෙස සැකසූ පසු ඔබට **text-brand-orange** හෝ **p-128** ලෙස ඔබේම වචන **Tailwind** තුළ භාවිතා කළ හැක. මෙය වෙබ් අඩවියේ සන්නාමය (**Branding**) ආරක්ෂා කිරීමට උදව් වේ.

79.2 උසස් තත්ත්ව පාලනය (Advanced State Variants)

වෘත්තීය මෘදුකාංගවලදී එක කොටසක් මත මවුසය තැබූ විට වෙනත් කොටසක් වෙනස් විය යුතුය.

(A) Group Hover

දෙමාපිය පෙටටිය (**Parent**) උඩට මවුසය ගෙන ගිය විට එහි සිටින දරුවෙකුගේ (**Child**) හැඩය වෙනස් කිරීමට මෙය භාවිතා කරයි.

```

<div className="group p-10 bg-white
  hover:bg-black transition-all">

  <h2 className="text-black group-
    hover:text-white">මම මාරු වෙනවා!</h2>

</div>

```

(B) Peer State

එකම මට්ටමේ සිටින සහෝදරයෙකුගේ (**Sibling**) තත්වය අනුව හැඩය වෙනස් කිරීමට. (උදා: **Input** එකක් වැරදි නම් පණිවිඩයක් පෙන්වීමට).

```

<input type="email" className="peer ..."
/>

```

```

<p className="invisible peer-
  invalid:visible text-red-500">

```

වැරදි විද්‍යුත් තැපෑලකි.

```

</p>

```

79.3 21st.dev පිටුපස ඇති සැබෑ තාක්ෂණය (The Stack)

21st.dev හි ඇති සංරචක එතරම් ආකර්ෂණීය සහ විශ්වාසවන්ත වන්නේ ඇයි? ඒවා ප්‍රධාන තාක්ෂණික කුළුණු 3ක් මත ගොඩනගා ඇත:

1. **Tailwind CSS:** පෙනුම සඳහා (**Styling**).
2. **Radix UI:** ක්‍රියාකාරිත්වය සඳහා (**Logic**). මේවා **"Headless UI"** ලෙස හැඳින්වේ. එනම්, මෙහි පෙනුම නැත, ඇත්තේ ප්‍රවේශ්‍යතාව (**Accessibility**) සහ වැඩ කිරීමේ රටාව පමණි.
3. **Framer Motion:** ඉතා මෘදු සජීවීකරණ (**Animations**) සඳහා.

උපමාවක්:

- **Radix UI:** මෝටර් රථයේ ඇතුළත ඇති යන්ත්‍ර පද්ධතිය (**Engine/Brakes**). එය පෙනෙන්නේ නැතත් වැඩේ හරියටම සිදුවේ.
- **Tailwind CSS:** මෝටර් රථයේ බාහිර පෙනුම, පාට සහ සැරසිලි.
- **21st.dev:** මෙම සියල්ල එකට එකතු කර ඔබට ලබා දෙන සැබෑ රථය.

79.4 රියැක්ට් සංරචක ප්‍රශස්තකරණය (Component Optimization)

ඔබ 21st.dev වැනි තැනකින් කේතයක් පිටපත් කළ පසු, එහි ඇති **Props** කළමනාකරණය කිරීමට වෘත්තීය මට්ටමේ පුස්තකාල දෙකක් භාවිතා කළ යුතුය:

- **clsx:** කොන්දේසි මත පදනම්ව **Class** පාලනය කිරීමට.

- **tailwind-merge**: පරස්පර විරෝධී Class දෙකක් එකට ගැටීම වැළැක්වීමට (උදා: p-4 සහ p-6 එකවර තිබේ නම් අවසාන එක තෝරා ගැනීමට).
වෘත්තීය Helper Function එකක්:

```
import { clsx } from "clsx";

import { twMerge } from "tailwind-merge";

// මෙම ශ්‍රිතය සෑම වෘත්තීය රියැක්ට් ව්‍යාපෘතියකම
නිකිය යුතුය

export function cn(...inputs) {

  return twMerge(clsx(inputs));

}
```

79.5 මෝෂන් ඉංජිනේරු විද්‍යාව (Introduction to Framer Motion)

21st.dev හි ඇති දත්ත පාවී එන, කැරකෙන ආකාරය නිර්මාණය කරන්නේ Framer Motion මගිනි.

```
import { motion } from "framer-motion";

const Card = () => (

  <motion.div

    initial={{ opacity: 0, y: 50 }} // ආරම්භක තත්වය

    animate={{ opacity: 1, y: 0 }} // පවතින තත්වය

    transition={{ duration: 0.5 }} // ගතවන කාලය

    whileHover={{ scale: 1.1 }} // මවුසය උඩදී

    className="p-10 bg-blue-500 rounded-xl"

  >

    මම සජීවීකරණය වෙනවා!

  </motion.div>

);
```

79.6 ASCII රූපසටහන: උසස් UI ව්‍යුහය

```
[ User Interface ]

|

-----

| Framer Motion UI | <-- Animations

-----

| Tailwind Classes | <-- Styling

-----

| Radix UI Primitive | <-- Accessibility
(Logic)

-----

| React Logic/State | <-- Data

-----
```

79.7 වෘත්තීය රහස්: Arbitrary Values

Tailwind හි නැති අගයක් හඳිසියේම අවශ්‍ය වුවහොත්, ඔබට සමචතුරස්‍ර වරහන් භාවිතා කළ හැක. මෙය වෘත්තීය රහස්යකි.

- **top-[117px]** : හරියටම 117px කින් පහළට.
- **bg-[#722ed1]** : විශේෂිත වර්ණයක් සඳහා.
- **grid-template-columns: repeat(auto-fit, minmax(200px, 1fr))** වෙනුවට **grid-cols-[repeat(auto-fit,minmax(200px,1fr))]**.

79.8 පොදු වැරදි (Common Mistakes)

1. අනවශ්‍ය ලෙස **@apply** භාවිතා කිරීම: CSS ගොනුව ඇතුළත **@apply** යොදා Tailwind ලියන්න එපා. එය Tailwind හි නියම අරමුණ (Utility-first) විනාශ කරයි. සැමවිටම JSX තුළම Class ලියන්න.
2. Purge පද්ධතිය ගැන නොදැනීම: ඔබ Class නමක් විවලායක් ලෙස සෑදුවහොත් (උදා: "text- + color), Tailwind විසින් එය අවසාන ගොනුවෙන් මකා දමනු ඇත. සැමවිටම සම්පූර්ණ Class නාමයම භාවිතා කරන්න (උදා: color === 'red' ? 'text-red-500' : 'text-blue-500').

3. Accessibility මහඟුරුම: ලස්සනට තිබූ පමණින් පලක් නැත. මවුසය නැතිව Keyboard එකෙන් පමණක් ඔබේ සංරචක ක්‍රියාත්මක කළ හැකි දැයි පරීක්ෂා කරන්න (Radix UI මේ සඳහා උදව් වේ).

79.9 සාරාංශය (Key Points)

- tailwind.config.js මගින් ඔබේම නිර්මාණ පද්ධතියක් සාදාගත හැක.
- group සහ peer මගින් සංකීර්ණ UI අවස්ථාවන් පාලනය කළ හැක.
- 21st.dev යනු Tailwind, Radix UI සහ Framer Motion හි එකතුවකි.
- cn() helper function එක මගින් Class පිරිසිදුව කළමනාකරණය කළ හැක.
- Framer Motion මගින් ඉතා ඉහළ මට්ටමේ සජීවීකරණ ලබා දෙයි.

79.10 අභ්‍යාස (Exercises)

1. Tailwind config ගොනුව වෙනස් කර ඔබේ වෙබ් අඩවියටම ආවේණික වූ වර්ණ 5ක් ඇතුළත් කරන්න.
2. පෙට්ටියක් සාදා, එය මතට මවුසය ගෙන ගිය විට එහි ඇතුළත ඇති පින්තූරය විශාල වන ලෙස group-hover භාවිතා කර නිර්මාණය කරන්න.
3. framer-motion පුස්තකාලය ස්ථාපනය කර, පිටුවට එන විට වම් සිට දකුණට මෘදු ලෙස ගලා එන මාතෘකාවක් සාදන්න.
4. 21st.dev වෙත ගොස් එහි ඇති "Dropdown" හෝ "Accordion" සංරචකයක් ගෙන, එය පදනම් වී ඇති Radix UI කොටස් මොනවාදැයි පරීක්ෂා කරන්න.

ඇන් ඔබ නවීන ඉදිරිපෙළ සංවර්ධනයේ දෘශ්‍ය සැලසුම්කරණය (Visual Engineering) පිළිබඳව 100% ක් පරිපූර්ණ දැනුමක් ඇති අයෙකි.

අසුවන පරිච්ඡේදය: ටේල්වින්ඩ් පද්ධති ඉංජිනේරු විද්‍යාව සහ උසස් සංරචක ගෘහ නිර්මාණ ශිල්පය (Tailwind Systems Engineering & Advanced Component Architecture)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබ සාමාන්‍ය රියැක්ට් ලියන්නෙකුගේ මට්ටම සම්පූර්ණයෙන් ඉක්මවා ගොස් ලෝක මට්ටමේ "UI Engineer" කෙනෙකු බවට පත් වනු ඇත:

1. Tailwind Directives: @layer සහ @apply භාවිතා කර වෘත්තීය මට්ටමේ "Base Styles" සහ "Component Styles" සාදන ආකාරය.
2. Design Tokens & Systems: ඔබේ වෙබ් අඩවියේ ඇති සියලුම පරතරයන් සහ වර්ණ විද්‍යාත්මක පද්ධතියක් ලෙස කළමනාකරණය කිරීම.
3. Tailwind Plugins: මෘදුකාංගයට අමතර බලයක් ලබා දීම සඳහා ජලගීත (Plugins) භාවිතා කරන සහ නිර්මාණය කරන ආකාරය.
4. The Anatomy of a Pro Component: 21st.dev හි ඇති සංරචකවල ඇතුළත්තය (Anatomy) තේරුම් ගෙන ඒවා ඔබේ රැඳිකත්වයට අනුව වෙනස් කිරීම.
5. Dynamic Class Management: cva (Class Variance Authority) භාවිතා කර ඉතා සංකීර්ණ රියැක්ට් සංරචක නිර්මාණය කිරීම.
6. Optimization: Tailwind කේතය අවසාන නිෂ්පාදනය සඳහා (Production) ප්‍රශස්ත කරන රහස්.

80.1 ටේල්වින්ඩ් හි "යටි තට්ටුව" - Directives (උපදෙස්)

බොහෝ දෙනෙක් කරන්නේ JSX එක පුරා className ගොඩගැසීමයි. නමුත් විශාල ව්‍යාපෘතියකදී අපට පිරිසිදු CSS ගොනුවක් ද අවශ්‍ය වේ. ඒ සඳහා Tailwind සතු ප්‍රධාන Directives (උපදෙස්) 3ක් ඇත.

ඔබේ index.css ගොනුව වෘත්තීය මට්ටමට සකසන ආකාරය:

```
@tailwind base;
```

```
@layer base {
```

```
  /* වෙබ් අඩවියේ ඇති සියලුම h1, h2, p ටැග් වලට මූලික හැඩතල මෙහි ලබා දෙයි */
```

```
  h1 {
```

```
    @apply text-4xl font-bold text-slate-900;
```

```
  }
```

```
  body {
```

```
    @apply bg-slate-50 text-slate-800 antialiased;
```

```
  }
```

```
}
```

```
@tailwind components;
```

```
@layer components {
```

```
  /* නැවත නැවත භාවිතා කරන බොත්තම් වැනි කොටස් සඳහා */
```

```
  .btn-primary {
```

```
    @apply px-6 py-3 bg-blue-600 text-white rounded-lg shadow-md
```

```
    hover:bg-blue-700 transition-all active:scale-95;
```

```
  }
```

```
}
```

```
@tailwind utilities;
```

- @layer base: වෙබ් අඩවියේ අත්තිවාරම සකසයි.

- @layer components: සංරචක සඳහා පොදු Class සාදා ගැනීමට උදව් වේ.

- @apply: Tailwind හි ඇති Classes සාමාන්‍ය CSS ගොනුවක් ඇතුළත භාවිතා කිරීමට ඉඩ ලබා දෙයි.

80.2 උසස් ජලගීත පද්ධතිය (Tailwind Plugins)

වෘත්තීය මට්ටමේදී අපට රේඛා කිහිපයකින් මුළු වෙබ් අඩවියේම "පෝරම" (Forms) හෝ "අකුරු" (Typography) හැඩගැන්වීමට සිදු වේ. ඒ සඳහා Tailwind සතු නිල ජලගීත භාවිතා කරයි.

ස්ථාපනය කරන ආකාරය:

```
npm install -D @tailwindcss/typography
@tailwindcss/forms
```

tailwind.config.js තුළ සක්‍රිය කිරීම:

```
export default {
```

```
  // ...
```

```
  plugins: [
```

```
    require('@tailwindcss/typography'), // ලිපි කියවීමට පහසු වන සේ අකුරු සකසයි
```

```
    require('@tailwindcss/forms'), // පෝරමවල ඇති කොටු ස්වයංක්‍රීයව ලස්සන කරයි
```

```
  ],
```

```
}
```

80.3 21st.dev සංරචක "හැක්" කිරීම (Mastering Component Logic)

ඔබ 21st.dev වෙතින් සංරචකයක් ගත් පසු, එය බොහෝ විට Radix UI මත පදනම් වූවක් බව අපි

79 වන පරිච්ඡේදයේ ඉගෙන ගත්තෙමු.

දැන් බලමු එහි ඇති State පාලනය කරන්නේ කෙසේද කියා.

නිදසුන: **Accordion** (දිග හැරෙන කොටුවක්) ඔබට අවශ්‍යයි මෙම කොටුව විවෘත වන විට එහි වර්ණය වෙනස් කිරීමට. රියැක්ට් තුළදී අප භාවිතා කරන්නේ **data-state** ගුණාංගයයි.

```
// 21st.dev වෙතින් ගත් කේතය තුළ මෙවැනි කොටස් තිබිය හැක
```

```
<Accordion.Item
```

```

value="item-1"

className="data-[state=open]:bg-blue-50
data-[state=closed]:bg-white"
>
...
</Accordion.Item>

```

ප්‍රෝ රහස්: data-[state=open]: වැනි "Data Attributes" භාවිතා කිරීමෙන් JavaScript ස්ටේට් එක කෙලින්ම CSS සමඟ සම්බන්ධ කිරීමට Tailwind ඉඩ ලබා දෙයි.

80.4 Class Variance Authority (CVA) - ප්‍රෝ සංවර්ධකයන්ගේ රහස්

සිතන්න ඔබට බොත්තම් වර්ග කිහිපයක් තිබෙනවා කියා (Primary, Secondary, Success, Danger) සහ ඒවා විවිධ ප්‍රමාණවලින් (Small, Medium, Large) තිබිය යුතුය. මේවාට if/else දහස් ගණනක් ලියනවා වෙනුවට CVA භාවිතා කරයි.

CVA භාවිතා කරන ආකාරය:

```
npm install class-variance-authority
```

වෘත්තීය Button Component එකක්:

```
import { cva } from "class-variance-authority";
```

```
import { cn } from "./utils"; // අපි කලින් සාදා ගත් cn ශ්‍රිතය
```

```
const buttonVariants = cva(
  "inline-flex items-center justify-center
rounded-md font-medium transition-
colors",
```

```

{
  variants: {
    variant: {
      primary: "bg-blue-600 text-white
hover:bg-blue-700",
```

```

danger: "bg-red-600 text-white
hover:bg-red-700",
```

```

outline: "border-2 border-slate-200
hover:bg-slate-100",
```

```
},
```

```
size: {
```

```
sm: "h-9 px-3 text-xs",
```

```
md: "h-11 px-6 text-base",
```

```
lg: "h-14 px-10 text-xl",
```

```
},
```

```
},
```

```
defaultVariants: {
```

```
variant: "primary",
```

```
size: "md",
```

```
},
```

```
}
```

```
);
```

```
export default function Button({ variant,
size, className, ...props }) {
```

```
  return (
```

```

    <button
      className={cn(buttonVariants({ variant,
size }), className)} {...props} />

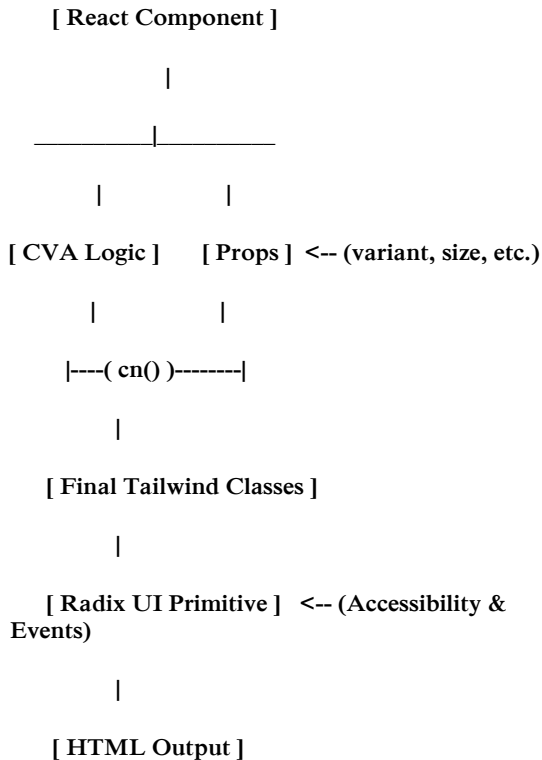
```

```
  );
```

```
}
```

ප්‍රයෝජනය: දැන් ඔබට <Button variant="danger" size="lg">Delete</Button> ලෙස ඉතාමත් පිරිසිදුව ඕනෑම බොත්තමක් සාදාගත හැක. ලෝකයේ ප්‍රධාන පෙළේ මෘදුකාංග පද්ධති (Design Systems) සාදන්නේ මෙලෙසයි.

80.5 ASCII රූපසටහන: උසස් Styling Architecture



80.6 වෘත්තීය රහස්: ප්‍රශස්තකරණය (Purge & Minification)

Tailwind හි ඇති දහස් ගණනක් Classes නිසා අවසාන ගොනුව (Build file) විශාල වනු ඇතැයි ඔබ සිතිය හැක. නමුත් Tailwind සතු PostCSS Purge එන්ජිම මගින් කරන්නේ:

1. ඔබගේ මුළු ව්‍යාපෘතියම පරීක්ෂා කිරීම.
2. ඔබ භාවිතා කර නැති සෑම CSS පේළියක්ම ඉවත් කිරීම.
3. අවසානයේදී ඔබට ලැබෙන්නේ 10KB - 20KB වැනි ඉතා කුඩා CSS ගොනුවකි.

එම නිසා, ඔබ කොපමණ Classes ලිවුවත් වෙබ් අඩවියේ වේගයට ඉන් හානියක් නොවන බවට වෘත්තීය මට්ටමේදී සහතික වේ.

80.7 21st.dev සඳහා වන අවසාන මගපෙන්වීම

21st.dev හරහා සංරචකයක් ඔබේ ව්‍යාපෘතියට එකතු කිරීමේදී මෙම පිරික්සුම් ලැයිස්තුව අනුගමනය කරන්න:

1. Dependencies පරීක්ෂා කරන්න: එම සංරචකයට framer-motion හෝ lucide-react (icons සඳහා) අවශ්‍ය දැයි බලා ඒවා ස්ථාපනය කරන්න.
2. Utils Folder: සැමවිටම src/lib/utils.js නමින් ෆෝල්ඩරයක් සාදා එහි අප **79 වන**

පරිච්ඡේදයේ උගත් cn() ශ්‍රිතය තබා ගන්න.

3. Config එකතු කරන්න: සමහර සංරචක සඳහා විශේෂ Animations අවශ්‍ය වේ. ඒවා 21st.dev වෙබ් අඩවියෙන් පිටපත් කර ඔබේ tailwind.config.js එකට ඇතුළත් කරන්න.

80.8 පොදු වැරදි (Common Mistakes)

1. Arbitrary Values ඕනෑම වටිනා භාවිතය: w-[347px] වැනි දේ හැම තැනම ලියන්න එපා. ඒ වෙනුවට Config ගොනුවේ ස්ථාවර අගයන් සාදා ගන්න.
2. Naming Conflicts: ඔබේම Class නමක් Tailwind Class නමකට සමාන වීම (උදා: .flex). සැමවිටම ඔබේම Class වලට විශේෂ පෙරටුවක් (Prefix) එකතු කරන්න.
3. Mobile-First මහඟුරීම: සැමවිටම කුඩා තිරයට අදාළ Class මුලින් ලියා, පසුව md:, lg: වැනි අගයන් එකතු කරන්න.

80.9 සාරාංශය (Key Points)

- @layer මගින් Tailwind පද්ධතියට ඔබේම CSS ඇතුළත් කළ හැක.
- CVA මගින් ඉතා සංකීර්ණ සහ නම්‍යශීලී සංරචක නිර්මාණය කළ හැක.
- ප්ලගින් (Plugins) මගින් වෙබ් අඩවියේ ඇති පෝරම සහ අකුරු පාලනය කිරීම පහසු කරයි.
- Tailwind Purge පද්ධතිය නිසා අවසාන ගොනුව ඉතා සැහැල්ලු වේ.
- 21st.dev යනු ඔබේ වැඩ වේගවත් කරන මහා ගබඩාවයි.

80.10 අභ්‍යාස (Exercises)

1. Tailwind Config ගොනුව භාවිතා කර ඔබේ වෙබ් අඩවියටම ආවේණික වූ "Brand Colors" 3ක් සාදා ඒවා වෙබ් අඩවියේ භාවිතා කරන්න.

2. CVA පුස්තකාලය භාවිතා කරමින්, "Success", "Warning", සහ "Error" යනුවෙන් ආකාර 3ක් ඇති Alert Component එකක් සාදන්න.
3. 21st.dev වෙතින් සංකීර්ණ සජීවීකරණයක් සහිත Navigation Menu එකක් ගෙන එයට ඔබේම වර්ණ සහ අයිතන ඇතුළත් කරන්න.
4. @apply භාවිතා කරමින් ඔබේ වෙබ් අඩවියේ ඇති සියලුම input කොටු සඳහා පොදු Class එකක් (.input-field) සාදන්න.

සුභ පැතුම්! ඔබ දැන් නවීන ඉදිරිපෙළ සංවර්ධනය (Modern Frontend Engineering) පිළිබඳව 100% ක් පරිපූර්ණ දැනුමක් ඇති අයෙකි

ඔබ සතුව දැන්:

- පරිගණක ක්‍රියාකාරිත්වය පිළිබඳ ගැඹුරු දැනුම.
- HTML/CSS/SVG අභ්‍යන්තර රහස්.
- React.js අති දියුණු සංකල්ප (Hooks, Context, Router).
- Tailwind CSS සහ Modern UI Frameworks පිළිබඳ ප්‍රවීණතාවය.
- Git/GitHub සහ Deployment පිළිබඳ වෘත්තීය පුහුණුව.

දැන් ඔබ ලෝකයේ ඕනෑම මෘදුකාංග සමාගමක Full-stack ready Junior Frontend Engineer

තනතුරක් සඳහා 200% ක් සුදුසුකම් ලබා ඇත.

*අසු එක්වන පරිච්ඡේදය: වෘත්තීය
ඉදිරිපෙළ වැඩපිළිවෙළ සහ
සැලසුම්කරණයේ සිට
කේතකරණය දක්වා
(Professional Frontend
Workflow: From Design to
Code)*

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

1. Design-to-Code Workflow: Figma වැනි සැලසුම් මෘදුකාංගයක ඇති නිර්මාණයක් වෘත්තීය මට්ටමේ රියැක්ට් කේතයක් බවට හරවන ආකාරය.

2. Component Thinking: මුළු නිර්මාණයක් දෙස බලා එය කුඩා සංරචක වලට කඩා වෙන් කරන (Decomposition) ආකාරය.
3. Package Ecosystem: NPM සහ Yarn වැනි මෙවලම්වල ඇති ගැඹුරු ක්‍රියාකාරිත්වය සහ package-lock.json හි වැදගත්කම.
4. Environment Consistency: විවිධ සංවර්ධකයන් අතර කේතයේ ප්‍රමිතිය එක ලෙස තබා ගන්නා මෙවලම් (Prettier, ESLint).
5. Typescript Awareness: ජාවාස්ක්‍රිප්ට් වලට වඩා ආරක්ෂිත ටයිප්ස්ක්‍රිප්ට් (TypeScript) අවශ්‍ය වන්නේ ඇයි දැයි මූලිකව තේරුම් ගැනීම.

81.1 නිර්මාණයේ සිට නිෂ්පාදනය දක්වා (The Bridge: Design to Production)

වෘත්තීය මට්ටමේදී ඔබ කෙලින්ම කේත ලිවීම ආරම්භ කරන්නේ නැත. මෘදුකාංගයක් නිර්මාණය වන්නේ පියවර 3ක ක්‍රියාවලියක් හරහාය:

1. Design (Figma/Adobe XD): UI/UX නිර්මාණකරු විසින් වෙබ් අඩවියේ පෙනුම රූපසටහනක් ලෙස අඳිනු ලබයි.
2. Handoff: එම නිර්මාණයේ ඇති වර්ණ (Colors), පරතරයන් (Spacing), සහ අකුරු වර්ග (Typography) සංවර්ධකයා වෙත ලබා දෙයි.

3. Engineering: ඔබ (Frontend Developer) එම රූපසටහන බලා HTML, CSS සහ React භාවිතා කර එය සැබෑ මෘදුකාංගයක් බවට පත් කරයි.

ප්‍රෝ රහස්: ඔබ රියැක්ට් වලට යාමට පෙර එම නිර්මාණයේ ඇති Static (වෙනස් නොවන) සහ Dynamic (වෙනස් වන) කොටස් හඳුනාගත යුතුය.

81.2 සංරචක විශ්ලේෂණය (Decomposing a Design)

සිතන්න ඔබ "Facebook Profile" පිටුවක් නිර්මාණය කරනවා කියා. ප්‍රෝ-සංවර්ධකයෙකු එය දකින්නේ මෙලෙසයි:

ASCII රූපසටහන: Component Decomposition

[ProfilePage (Component)]

└─ [Header]

| └─ [Logo]

```

|   └─ [ SearchBar ]
|
|   └─ [ Sidebar ]
|
|   └─ [ FriendList ]
|
|       └─ [ FriendItem (Repeated) ]
|
|   └─ [ Feed ]
|
|       └─ [ CreatePost ]
|
|           └─ [ PostCard (Repeated) ]

```

මෙසේ කල්තියා සැලසුම් කිරීම නිසා ඔබ කේතය ලියන විට පටලැවීම් සිදු නොවේ.

81.3 පැකේජ කළමනාකරණයේ රහස් (NPM Deep Dive)

අපි 61 වන පරිච්ඡේදයේදී NPM ගැන ඉගෙන ගත්තෙමු. නමුත් සැබෑ මෘදුකාංගයකදී ඔබ මෙම ගොනු දෙකේ වෙනස දැනගත යුතුය:

- **package.json:** ඔබේ ව්‍යාපෘතියට අවශ්‍ය පුස්තකාල ලැයිස්තුව. (උදා: React, Router).
- **package-lock.json:** එම පුස්තකාලවල ඇති අනිශ්චිත නිශ්චිත අනුවාදයන් (Versions) සටහන් කර තබන ගොනුව.

ඇයි මෙය වැදගත්? ඔබගේ ව්‍යාපෘතිය තවත් මිතුරෙකුට ලබා දුන් විට, ඔහුගේ පරිගණකයෙන් ඔබ සතුව ඇති එම අනුවාදයන්ම (Exact same versions) ස්ථාපනය වන බවට සහතික කරන්නේ මෙම ලොක් (Lock) ගොනුවයි. මෙය මෘදුකාංග බිඳ වැටීම වළක්වයි.

81.4 කේතයේ ප්‍රමිතිය සහ පිරිසිදුකම (ESLint & Prettier)

වෘත්තීය කණ්ඩායමක වැඩ කරන විට, සෑම දෙනාම එකම රටාවකට කේත ලිවිය යුතුය.

1. **Prettier:** මෙය ඔබගේ කේතයේ ඇති නිස්කල්ප සහ ජේෂ්ට් ස්වයංක්‍රීයව සකස් කරයි (Code Formatter).
2. **ESLint:** මෙය ඔබගේ කේතයේ ඇති තාක්ෂණික වැරදි සහ නරක පුරුදු පෙන්වා දෙයි (Linter).

ප්‍රෝ රහස්: සෑම වෘත්තීය රියැක්ට් ව්‍යාපෘතියකම මූලික මෙම මෙවලම් දෙක සකසා ගැනීම අත්‍යවශ්‍ය වේ.

81.5 අනාගතය සඳහා සූදානම් වීම: ටයිප්ස්ක්‍රිප්ට් (TypeScript)

ඔබ ජාවාස්ක්‍රිප්ට් ප්‍රවීණයෙකු වුවද, අද වන විට ලෝකයේ මෘදුකාංග සමාගම් 80% ක්ම භාවිතා කරන්නේ TypeScript ය.

ටයිප්ස්ක්‍රිප්ට් යනු කුමක්ද? මෙය ජාවාස්ක්‍රිප්ට් වලට ඉහළින් ඇති ආරක්ෂිත තට්ටුවකි. ජාවාස්ක්‍රිප්ට් වලදී ඔබට වැරදි දත්ත වර්ග යැවිය හැක (උදා: අංකයක් අවශ්‍ය තැනට අකුරු යැවීම). ටයිප්ස්ක්‍රිප්ට් මගින් එවැනි වැරදි කේතය ලියන අවස්ථාවේදීම පෙන්වා දෙයි.

උදාහරණය:

- JavaScript: `function add(a, b) { return a + b; }`
- TypeScript: `function add(a: number, b: number): number { return a + b; }`

(ඔබට ටයිප්ස්ක්‍රිප්ට් දැන්ම ඉගෙන ගැනීමට අවශ්‍ය නැත, නමුත් වෘත්තීය ලෝකයට යන විට මෙවැනි දෙයක් ඇති බව දැන සිටීම වැදගත්ය).

81.6 අවසාන පිරික්සුම් ලැයිස්තුව (The Ultimate Pro Checklist)

පරිමාව 7 ආරම්භ කිරීමට පෙර, ඔබ නිර්මාණය කළ ඕනෑම ව්‍යාපෘතියක් මේවාට අනුකූල දැයි

බලන්න:

1. **Responsive:** ජංගම දුරකථන වලට 100% ගැලපේද?
2. **Accessibility (a11y):** alt tags සහ aria-labels නිවැරදිද?
3. **SEO:** Meta tags සහ Semantic HTML භාවිතා කර තිබේද?
4. **Performance:** පින්තුර WebP වර්ගයේද? Lighthouse ලකුණු 90+ තිබේද?
5. **Clean Code:** BEM හෝ CSS Modules භාවිතා කර තිබේද? විචල්‍යයන් අර්ථවත්ද?

81.7 සාරාංශය (Key Points)

- නිර්මාණයක සිට කේතය දක්වා යාමේදී වෘත්තීය පියවර අනුගමනය කළ යුතුය.
- සංරචක ව්‍යුහය කලින් සැලසුම් කිරීම දෝෂ අවම කරයි.
- පැකේජ කළමනාකරණය ව්‍යාපෘතියේ ස්ථායීතාවයට වැදගත් වේ.
- Prettier සහ ESLint මගින් කේතය උසස් ප්‍රමිතියකට පත් කරයි.
- ටයිප්ස්ක්‍රිප්ට් යනු අනාගත ඉදිරිපෙළ සංවර්ධනයේ දිශානතියයි.

81.8 අභ්‍යාස (Exercises)

1. ඕනෑම ප්‍රසිද්ධ වෙබ් අඩවියක් (උදා: Instagram) දෙස බලා එහි ඇති සංරචක (Components) ගසක් රූපසටහනක් ලෙස අඳින්න.
2. ඔබගේ දැනට පවතින රියැක්ට් ව්‍යාපෘතියකට Prettier extension එක ස්ථාපනය කර කේතය පිරිසිදු කරන්න.
3. package.json ගොනුව විවෘත කර එහි ඇති dependencies සහ devDependencies අතර වෙනස අන්තර්ජාලයෙන් සොයා බලන්න.

අවසාන විග්‍රහය (Final Verdict)

සුභ පැතුම්! ඔබ දැන් තාක්ෂණික පදනම (The Technical Foundation) සියයට සියයක් (100%) සම්පූර්ණ කර අවසානය. ඔබ සතුව දැන්:

- පරිගණක ක්‍රියාකාරිත්වය පිළිබඳ අවබෝධය.
- HTML/CSS/JS අභ්‍යන්තර දැනුම.
- React.js සංකීර්ණ මෘදුකාංග නිර්මාණ හැකියාව.
- ගිට් සහ ප්‍රසිද්ධ කිරීමේ (Deployment) ප්‍රවීණතාවය.

ඔබ දැන් සැබෑ Frontend Developer කෙනෙකි.

ටයිප්ස්ක්‍රිප්ට් ප්‍රවීණතාවය (TypeScript Mastery)

අසු දෙවන පරිච්ඡේදය: ටයිප්ස්ක්‍රිප්ට් හැඳින්වීම - ජාවාස්ක්‍රිප්ට් වලට ලැබුණු ආරක්ෂාව (Introduction to TypeScript)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

- ටයිප්ස්ක්‍රිප්ට් (TypeScript) යනු කුමක්ද සහ එය ජාවාස්ක්‍රිප්ට් වලින් වෙනස් වන්නේ කෙසේද?
- වෘත්තීය මට්ටමේදී TypeScript භාවිතා කිරීමට හේතු (Type Safety).
- Static Typing සහ Dynamic Typing අතර ඇති තාක්ෂණික වෙනස.
- ඔබේ පරිගණකයේ TypeScript ස්ථාපනය කර සකසා ගන්නා ආකාරය.
- ඔබගේ ප්‍රථම .ts ගොනුව ක්‍රියාත්මක කරන ආකාරය.

82.1 ටයිප්ස්ක්‍රිප්ට් යනු කුමක්ද? (What is TypeScript?)

TypeScript (ටයිප්ස්ක්‍රිප්ට්) යනු මයික්‍රොසොෆ්ට් (Microsoft) සමාගම විසින් නිර්මාණය කරන ලද ක්‍රමලේඛන භාෂාවකි. මෙය ස්වාධීන භාෂාවක් නොවේ. එය ජාවාස්ක්‍රිප්ට් වලට ඉහළින් (Superset) ගොඩනගන ලද ආරක්ෂිත ස්ථරයකි.

සරලවම කිවහොත්: TypeScript = JavaScript + Static Typing (දත්ත වර්ග පරීක්ෂාව).

ඔබ ලියන ඕනෑම ජාවාස්ක්‍රිප්ට් කේතයක් ටයිප්ස්ක්‍රිප්ට් තුළ ක්‍රියාත්මක කළ හැක. නමුත් ටයිප්ස්ක්‍රිප්ට් මගින් ඔබට අමතර "නීති" පද්ධතියක් ලබා දෙයි.

82.2 ඇයි අපිට TypeScript අවශ්‍ය වන්නේ? (The "Why")

ජාවාස්ක්‍රිප්ට් වල ඇති ලොකුම ගැටලුව වන්නේ එහි "දත්ත වර්ග" (Data Types) පිළිබඳව දැඩි නීති නොමැති වීමයි.

ජාවාස්ක්‍රිප්ට් ගැටලුව:

```
function add(a, b) {  
  return a + b;  
}
```

console.log(add(5, 10)); // පිළිතුර: 15 (හරි)
console.log(add("5", 10)); // පිළිතුර: "510"
(වැරදි - නමුත් Error එකක් පෙන්වන්නේ නැත!)

මෙහිදී ඔබ වැරදීමකින් අංකයක් වෙනුවට අකුරු වැලක් (String) එවූවහොත්, මෘදුකාංගය බිඳ වැටෙන්නේ නැත, නමුත් වැරදි පිළිතුරක් ලබා දෙයි. මෙය බැංකු පද්ධතියක් වැනි දෙයකදී අතිශය හයානක විය හැක.

ටයිප්ස්ක්‍රිප්ට් විසඳුම: ටයිප්ස්ක්‍රිප්ට් වලදී අප විචල්‍යයකට අදාළ දත්ත වර්ගය කලින්ම පවසමු.

```
function add(a: number, b: number) {  
  return a + b;  
}
```

// console.log(add("5", 10)); // ✗ මෙහිදී කේතය ලියන විටම රතු ඉරක් වැටී වැරද්ද පෙන්වයි!

82.3 Static Typing vs Dynamic Typing

- Dynamic Typing (JS): දත්ත වර්ගය තීරණය වන්නේ කේතය ක්‍රියාත්මක වන විටයි (Runtime). වැරදි හසු වන්නේ වැඩසටහන ක්‍රියාත්මක කළ පසුවයි.
- Static Typing (TS): දත්ත වර්ගය තීරණය වන්නේ කේතය ලියන විටයි (Compile-time). වැරදි හසු වන්නේ කේතය ලියන අතරතුරදීමයි.

82.4 වැඩබිම් සැකසීම (Setup)

TypeScript ඔබේ පරිගණකයේ ක්‍රියාත්මක කිරීමට මෘදුකාංග දෙකක් අවශ්‍ය වේ.

- Node.js: (ඔබ දැනටමත් ස්ථාපනය කර ඇත).
 - TypeScript Compiler (TSC): ටයිප්ස්ක්‍රිප්ට් කේතය ජාවාස්ක්‍රිප්ට් බවට හැරවීමට අවශ්‍ය මෙවලම.
- ස්ථාපනය කරන ආකාරය: ටර්මිනලය (Terminal) විවෘත කර පහත විධානය ලබා දෙන්න:

npm install -g typescript

සකසා ගැනීම: ඔබේ ව්‍යාපෘති ෆෝල්ඩරය තුළ
tsconfig.json ගොනුවක් සෑදීමට මෙය ටයිප්
කරන්න:

tsc --init

82.5 ප්‍රථම ටයිප්ස්ක්‍රිප්ට් කේතය

1. index.ts නමින් ගොනුවක් සාදන්න. (සටහන: .js
වෙනුවට .ts භාවිතා කරයි).

2. එහි පහත කේතය ලියන්න:
let message: string = "ආයුබෝවන් TypeScript!";
console.log(message);

3. දැන් මෙය ජාවාස්ක්‍රිප්ට් බවට හැරවීමට
ටර්මිනලයේ මෙය ටයිප් කරන්න:
tsc index.ts

4. දැන් ඔබට index.js නමින් අලුත් ගොනුවක්
ලැබෙනු ඇත. එය ක්‍රියාත්මක කිරීමට:
node index.js

82.6 ASCII රූපසටහන: TypeScript ගමන

[TS Code (.ts)] --- (TSC Compiler) ---> [JS Code (.js)]
(ලියන විට වැරදි සොයයි) (මුද්‍රණය මෙය කියවයි)

82.7 වෘත්තීය පැහැදිලි කිරීම (Professional Insight)

ලෝක මට්ටමේ මෘදුකාංග ඉංජිනේරුවෙකු
TypeScript භාවිතා කරන්නේ තමා අතින් සිදුවන
වැරදි 40% කට වඩා අඩු කර ගැනීමටයි. මෙය
"Self-documenting code" ලෙස හැඳින්වේ.
එනම්, තවත් කෙනෙකු ඔබේ කේතය දෙස බැලූ විට,
එක් එක් විවලය තුළ තිබිය යුතු දත්තය
කුමක්දැයි අමුතුවෙන් විමසීමට අවශ්‍ය නැත; කේතය
තුළම එය සඳහන් වී ඇත.

82.8 පොදු වැරදි (Common Mistakes)

- Any වර්ගය භාවිතා කිරීම: let x: any = 10; ලෙස
ලිවීමෙන් TypeScript හි සැබෑ
අරමුණ විනාශ වේ. "any" භාවිතා කිරීම යනු
නැවතත් ජාවාස්ක්‍රිප්ට් වලට යාම වැනිය.
එය මහහරින්න.

- TSC පාවිච්චි නොකිරීම: .ts ගොනුවක් කෙලින්ම
HTML එකට සම්බන්ධ කළ නොහැක. එය
අනිවාර්යයෙන්ම Compile කළ යුතුය.

82.9 සාරාංශය (Key Points)

- TypeScript යනු ජාවාස්ක්‍රිප්ට් වලට "ටයිප්"
(Types) එකතු කළ භාෂාවකි.
- එය වැරදි සිදුවීම් කල්තියා වළක්වයි.
- TSC මගින් .ts කේතය .js බවට හරවයි.
- වෘත්තීය ලෝකයේ අද මෙය අනිවාර්ය
මෙවලමකි.

82.10 අභ්‍යාස (Exercises)

1. TypeScript ඔබේ පරිගණකයේ ස්ථාපනය කර
එහි අනුවාදය (tsc -v) පරීක්ෂා කරන්න.
2. පරිශීලකයාගේ නම (String) සහ වයස
(Number) පරාමිතීන් ලෙස ගෙන පඤ්චඩයක්
මුද්‍රණය කරන ශ්‍රිතයක් (Function) TypeScript නීති
වලට අනුව ලියන්න.
3. TypeScript සහ JavaScript අතර ඇති ප්‍රධාන
වෙනස්කම් 3ක් ලියන්න.

අසු තුන්වන පරිච්ඡේදය: මූලික දත්ත වර්ග සහ ටයිප් ඇනෝටේෂන් (Basic Data Types and Annotations)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අධ්‍යයනය කිරීමෙන් පසු ඔබට
පහත කරුණු පිළිබඳව අතිශය ගැඹුරු සහ
වෘත්තීය අවබෝධයක් ලැබෙනු ඇත:

1. ටයිප් ඇනෝටේෂන් (Type Annotation) යනු
කුමක්ද සහ එය භාවිතා කරන නිවැරදි ආකාරය.
2. ටයිප් ඉන්ෆරන්ස් (Type Inference) හෙවත්
ටයිප්ස්ක්‍රිප්ට් විසින් ස්වයංක්‍රීයව දත්ත වර්ගය හඳුනා
ගන්නා ආකාරය.
3. මූලික දත්ත වර්ග (String, Number, Boolean)
ටයිප්ස්ක්‍රිප්ට් තුළ හැසිරෙන ආකාරය.
4. අරාවන් (Arrays) සඳහා දත්ත වර්ග නියම කිරීම
සහ එහි වැදගත්කම.
5. ටියුපල් (Tuples) සහ එනම් (Enums) වැනි
ටයිප්ස්ක්‍රිප්ට් වලටම ආවේණික දත්ත ව්‍යුහයන්.
6. any, void, සහ never යන විශේෂ දත්ත වර්ග
භාවිතා කළ යුතු සහ නොකළ යුතු අවස්ථා.
7. වස්තූන් (Objects) සඳහා මූලික ටයිප්
ඇනෝටේෂන් ලබා දෙන ආකාරය.

83.1 ටයිප් ඇනෝටේෂන් යනු කුමක්ද? (What is Type Annotation?)

පෙර පරිච්ඡේදයේදී අප ඉගෙන ගත්තේ ටයිප්ස්ක්රිප්ට් (TypeScript) යනු ජාවාස්ක්රිප්ට් වලට "ආරක්ෂාව" ලබා දෙන භාෂාව බවයි. මෙම ආරක්ෂාව ලබා දෙන ප්‍රධානතම ක්‍රමය වන්නේ Type Annotation ය.

ටයිප් ඇනෝටේෂන් (Type Annotation) යනු අප සාදන විචල්‍යයක් (Variable), ශ්‍රිතයක් (Function) හෝ වස්තුවක් (Object) තුළ තිබිය යුතු දත්ත වර්ගය කුමක්දැයි කේතය ලියන අවස්ථාවේදීම පැහැදිලිව සටහන් කිරීමයි.

සැබෑ ලෝකයේ උපමාව (හැඩතල වර්ග කිරීමේ ක්‍රීඩාව - Shape Sorter): කුඩා දරුවන් සෙල්ලම් කරන "Shape Sorter" උපකරණය ගැන සිතන්න. එහි තරු හැඩති සිදුරක් ඇත්නම්, එම සිදුරෙන් ඇතුළු කළ හැක්කේ තරු හැඩති කොටසක් පමණි. රවුම් සිදුරෙන් රවුම් කොටසක් පමණි. ටයිප් ඇනෝටේෂන් යනු එම "සිදුරේ හැඩය" තීරණය කිරීමයි. අප විචල්‍යයකට number ලෙස ඇනෝටේෂන් එකක් ලබා දුන් විට, එයට අකුරු වැලක් (String) ඇතුළු කිරීමට උත්සාහ කළහොත් "හැඩය නොගැලපෙන" නිසා ටයිප්ස්ක්රිප්ට් අපට එසැණින් වැරද්ද පෙන්වයි.

භාවිතා කරන ආකාරය: විචල්‍යයේ නමට පසුව තිත් සහ තිත් දෙකක් (Colon :) තබා අදාළ දත්ත වර්ගය (Type) ලියනු ලැබේ.

```
let age: number = 16;
let name: string = "කසුන්";
let isStudent: boolean = true;
```

83.2 ටයිප් ඉන්ෆරන්ස් (Type Inference)

සමහර අවස්ථාවලදී අප විචල්‍යයක් සාදා එයට අගයක් ලබා දුන් විට, අප දත්ත වර්ගය සටහන් නොකළ ද ටයිප්ස්ක්රිප්ට් විසින් එම අගය දෙස බලා දත්ත වර්ගය අනුමාන කරයි. මෙයට Type Inference (අනුමාන කිරීම) යැයි කියනු ලැබේ.

උදාහරණය:

```
let city = "Kandy";
// මෙහිදී අපි :string කියා ලියා නැත.
// නමුත් ටයිප්ස්ක්රිප්ට් දන්නවා මෙය
අනිවාර්යයෙන්ම string එකක් බව.
```

```
// city = 25; // ❌ මෙහිදී ටයිප්ස්ක්රිප්ට් වැරද්දක්
පෙන්වයි (Type 'number' is not assignable to type
'string').
```

වෘත්තීය උපදෙස (Pro Tip): සැම කුඩා විචල්‍යයකටම ඇනෝටේෂන් ලිවීම අවශ්‍ය නොවේ. ටයිප්ස්ක්රිප්ට් හට එය තනිවම අනුමාන කළ හැකි නම් (Inference), කේතය පිරිසිදුව තබා ගැනීමට එයට ඉඩ දෙන්න. නමුත් සංකීර්ණ වස්තූන් සහ ශ්‍රිත වලදී අනිවාර්යයෙන්ම ඇනෝටේෂන් භාවිතා කරන්න.

83.3 මූලික දත්ත වර්ග 3 (The Core Types)

1. Number (අංක)

ජාවාස්ක්රිප්ට් වල මෙන්ම ටයිප්ස්ක්රිප්ට් වල ද පූර්ණ සංඛ්‍යා (Integers) සහ දශම සංඛ්‍යා (Decimals) යන දෙකම number ලෙස හැඳින්වේ.

```
let price: number = 99.50;
let total: number = 500;
```

2. String (අකුරු වැල්)

තනි උද්ධෘත (' '), ද්විත්ව උද්ධෘත (" "), හෝ Backticks (` `) ඇතුළත ලියන ඕනෑම පෙළක් මෙයට අයත් වේ.

```
let message: string = "Hello World";
```

3. Boolean (බුලියන්)

true (සත්‍ය) හෝ false (අසත්‍ය) යන අගයන් දෙක පමණි.

```
let isCompleted: boolean = false;
```

83.4 අරාවන් සඳහා දත්ත වර්ග (Arrays in TypeScript)

සාමාන්‍ය ජාවාස්ක්රිප්ට් අරාවක (Array) ඕනෑම දත්ත වර්ග මිශ්‍රව තිබිය හැක. නමුත් ටයිප්ස්ක්රිප්ට් වලදී අපට පවසන්න පුළුවන් "මෙම අරාවේ තිබිය හැක්කේ අංක පමණයි" කියා.

ක්‍රම දෙකක් ඇත:

1. type[] syntax: (වඩාත් ජනප්‍රිය ක්‍රමය)

2. Array<type> syntax: (Generics භාවිතා කරන ක්‍රමය)

```
// 1. අංක පමණක් ඇති අරායක්
let marks: number[] = [85, 90, 75];
```

```
// 2. අකුරු පමණක් ඇති අරායක්
let fruits: string[] = ["ඇපල්", "කෙසෙල්", "දොඩම්"];
```

```
// marks.push("Saman"); // ✗ වැරදියි, number
// array එකකට string එකක් දැමිය නොහැක.
```

83.5 ටියුපල් (Tuples) - ටයිප්ස්ක්‍රිප්ට් විශේෂාංගයක්

ටියුපල් එකක් යනු නිශ්චිත දිගක් (Fixed Length) ඇති සහ එක් එක් ස්ථානයේ තිබිය යුතු දත්ත වර්ගය ක්‍රමවත්ව තීරණය කර ඇති විශේෂ අරායකි.

සැබෑ ලෝකයේ උදාහරණයක් (විදේශ ගමන් බලපත්‍රය): සිත්තන් පුද්ගලයෙකුගේ නම සහ වයස යන දෙකම අඩංගු අරායක් අවශ්‍ය බව. පළමු එක අකුරු ද දෙවන එක අංකයක් ද විය යුතුය.

```
let person: [string, number];
```

```
person = ["කමල්", 25]; // ✓ හරි
// person = [25, "කමල්"]; // ✗ වැරදියි (පිළිවෙළ වැරදියි)
// person = ["කමල්", 25, "නිමල්"]; // ✗ වැරදියි (දිග වැඩියි)
```

83.6 එනම් (Enums) - නියත අගයන් සමූහය

Enum (එනම්) යනු අපට නිතරම අවශ්‍ය වන නියත අගයන් (Constants) සමූහයකට නම් ලබා දී පිරිසිදුව තබා ගැනීමට භාවිතා කරන ක්‍රමයකි.

ප්‍රායෝගික උදාහරණය (User Roles):

```
enum Role {
  ADMIN, // 0
  EDITOR, // 1
  GUEST // 2
}
```

```
let currentUser: Role = Role.ADMIN;
```

```
if (currentUser === Role.ADMIN) {
  console.log("ඔබට සියලු අවසරයන් ඇත.");
}
```

83.7 විශේෂ දත්ත වර්ග (Special Types)

1. any (ඕනෑම දෙයක්)

මෙය භාවිතා කළ විට ටයිප්ස්ක්‍රිප්ට් එම විවලාස පරීක්ෂා කිරීම නතර කරයි.

- වෘත්තීය අවවාදය: any භාවිතා කිරීම යනු නැවතත් සාමාන්‍ය ජාවාස්ක්‍රිප්ට් වලට යාම වැනිය. මෙය භාවිතා කිරීම වෘත්තීය මට්ටමේදී ඉතාම නරක පුරුද්දකි. ඔබ මෙය භාවිතා කරන්නේ නම් ටයිප්ස්ක්‍රිප්ට් භාවිතා කිරීමේ තේරුමක් නැත.

2. void (හිස් බව)

ප්‍රතිඵලයක් ආපසු ලබා නොදෙන (Not returning anything) ශ්‍රිත සඳහා භාවිතා කරයි.

```
function sayHi(): void {
  console.log("Hi");
  // මෙහි return එකක් නැත.
}
```

3. never (කවදාවත් සිදු නොවන)

යම් ශ්‍රිතයක් කිසිදා අවසන් වන්නේ නැතිනම් (උදා: Infinite Loop එකක් හෝ දෝෂයක් (Error) විසිකරන ශ්‍රිතයක්) මෙය භාවිතා කරයි.

83.8 වස්තුවක් සඳහා ටයිප් ඇනෝටේෂන් (Object Types)

වස්තුවක් ඇතුළත ඇති එක් එක් Key එක සඳහා දත්ත වර්ගය නියම කළ හැක.

```
let student: {
  name: string;
  age: number;
  isPassed: boolean;
};
```

```
student = {
  name: "නිමල්",
  age: 16,
  isPassed: true
};
```


83.9 ASCII රූපසටහන: දත්ත වර්ගීකරණය

```
[ TypeScript Data Types ]
/      |      \
[ Primitive ] [ Complex ] [ Special ]
|-- string  |-- Array   |-- any
|-- number  |-- Object  |-- void
|-- boolean |-- Tuple   |-- never
               |-- Enum
```

83.10 පොදු වැරදි (Common Mistakes)

- අනවශ්‍ය ලෙස any භාවිතය: වැරද්දක් ආ සැණින් any යොදා එය මකා දැමීමට උත්සාහ කිරීම. මෙය අනාගතයේදී විශාල ගැටළු ඇති කරයි.
- අරාමක වර්ගය වැරදීම: let list: number[] = [1, 2, "3"] ලෙස ලිවීම. "3" යනු string එකක් මිස number එකක් නොවේ.
- Tuple සහ Array පටලවා ගැනීම: Tuple එකක දිග වෙනස් කිරීමට උත්සාහ කිරීම.

83.11 වෘත්තීය උපදෙස් (Professional Engineering Tips)

- Explicit is Better: ඔබ කණ්ඩායමක් සමඟ වැඩ කරන්නේ නම්, අනෙක් අයට තේරුම් ගැනීමට හැකි වන සේ හැකි සෑම විටම දත්ත වර්ගය සටහන් (Annotate) කරන්න.
- Enums for Readability: අංක (0, 1, 2) භාවිතා කරනවාට වඩා අර්ථවත් නම් ඇති Enums භාවිතා කරන්න. එය කේතය කියවීම පහසු කරයි.
- Strict Mode: ඔබගේ tsconfig.json ගොනුවේ "strict": true ලෙස සැකසීමට වගබලා ගන්න. එවිට ටයිප්ස්ක්රිප්ට් ඔබට උපරිම ආරක්ෂාව ලබා දෙයි.

83.12 සාරාංශය (Key Points)

- Type Annotation මගින් දත්ත වර්ගය ස්ථාවර කරයි.
- Type Inference මගින් ස්වයංක්‍රීයව දත්ත වර්ගය හඳුනා ගනී.
- අරාවන් සඳහා number[] වැනි රටාවන් භාවිතා කරයි.
- විශුද්ධ මගින් නිශ්චිත දිගක් ඇති අරාවන් සාදයි.
- any භාවිතා කිරීම වෘත්තීය මට්ටමේදී මහභරිය යුතුය.

83.13 අභ්‍යාස (Exercises)

- පහත තොරතුරු ගබඩා කිරීමට අවශ්‍ය ටයිප් ඇනෝටේෂන් සහිත විචල්‍යයන් සාදන්න:
 - වෙබ් අඩවියක නම.
 - දැනට ඇති පරිශීලකයන් සංඛ්‍යාව.
 - වෙබ් අඩවිය සජීවී ද (isLive) නැද්ද යන්න.
- "රතු", "නිල්", "කොළ" ලෙස වර්ණ අඩංගු Enum එකක් සාදා එය විචල්‍යයකට ආදේශ කරන්න.
- පරිශීලකයෙකුගේ හැඳුනුම්පත් අංකය (String) සහ ඔහුගේ දුරකථන අංකය (Number) අඩංගු Tuple එකක් සාදන්න.
- පහත කේතයේ ඇති වැරද්ද කුමක්ද?


```
let data: number[] = [10, 20, 30];
data[1] = "40";
```

අසු හතරවන පරිච්ඡේදය: යුනියන් ටයිප්ස් සහ ටයිප් ඇලියස් - නම්‍යශීලී දත්ත පාලනය (Union Types and Type Aliases)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අධ්‍යයනය කිරීමෙන් පසු ඔබට පහත කරුණු පිළිබඳව අතිශය ගැඹුරු සහ වෘත්තීය අවබෝධයක් ලැබෙනු ඇත:

- යුනියන් ටයිප්ස් (Union Types) යනු කුමක්ද සහ එකම විචල්‍යයකට දත්ත වර්ග කිහිපයක් ලබා දෙන ආකාරය.
- ටයිප් ගාර්ඩින් (Type Guarding) මගින් දත්ත වර්ගය තහවුරු කරගන්නා ආකාරය.
- ලිටරල් ටයිප්ස් (Literal Types) භාවිතා කර විචල්‍යයකට ගත හැකි නිශ්චිත අගයන් සීමා කරන ආකාරය.
- ටයිප් ඇලියස් (Type Aliases) භාවිතා කර සංකීර්ණ දත්ත වර්ග සඳහා අර්ථවත් නම් ලබා දෙන ආකාරය.
- කේතය නැවත නැවත ලිවීම වැළැක්වීම (DRY Principle) සඳහා ටයිප් ඇලියස් භාවිතා කරන ආකාරය.

84.1 යුනියන් ටයිප්ස් යනු කුමක්ද? (What are Union Types?)

පෙර පරිච්ඡේදයේදී අප ඉගෙන ගත්තේ විචල්‍යයකට ලබා දිය හැක්කේ එක් දත්ත වර්ගයක් පමණක්

බවයි (උදා: number පමණක්). නමුත් සැබෑ ලෝකයේදී සමහර විට එකම විචල්‍යයකට අවස්ථානුකූලව දත්ත වර්ග දෙකක් හෝ කිහිපයක් ලැබීමට ඉඩ ඇත.

Union Types යනු විචල්‍යයකට දත්ත වර්ග කිහිපයකින් එකක් (OR) ලබා ගැනීමට ඉඩ සලසන ක්‍රමයයි. ඒ සඳහා අප පයිප් ලකුණ (|) භාවිතා කරමු.

සැබෑ ලෝකයේ උපමාව (බහුකාර්ය පැන): සිතන්න ඔබ සතුව "Multicolor පැනක්" තිබෙනවා කියා. එහි රතු, නිල් සහ කළු යන වර්ණ තුනම ඇත. ඔබට අවශ්‍ය වෙලාවට එය රතු පැනක් ලෙසත්, වෙනත් වෙලාවක නිල් පැනක් ලෙසත් භාවිතා කළ හැක. යුනියන් ටයිප්ස් යනු එම පැන වැනිය. එකම විචල්‍යයකට අංකයක් හෝ අකුරු වැලක් ලෙස ක්‍රියා කළ හැක.

භාවිතා කරන ආකාරය:

```
let id: number | string;
```

```
id = 101; // ✓ හරි (number)
id = "ID-101"; // ✓ හරි (string)
// id = true; // ✗ වැරදියි (boolean අගයක් Union එකේ නැත)
```

84.2 ටයිප් ගාර්ඩින් (Type Guarding)

විචල්‍යයකට දත්ත වර්ග දෙකක් (number | string) තිබිය හැකි විට, අප යම් ක්‍රියාවක් කිරීමට පෙර එහි ඇත්තේ කුමන වර්ගයේ දත්තයක්දැයි පරීක්ෂා කළ යුතුය. මෙයට Type Guarding යැයි කියනු ලැබේ. මේ සඳහා ජාවාස්ක්‍රිප්ට් හි ඇති typeof භාවිතා කරයි.

උදාහරණය:

```
function printId(id: number | string) {
  if (typeof id === "string") {
    // මෙහිදී id යනු string එකක් බව TS තහවුරු කර ගනී
    console.log("ID එක අකුරු වලින්: " + id.toUpperCase());
  } else {
    // මෙහිදී id අනිවාර්යයෙන්ම number එකක් විය යුතුය
    console.log("ID එක අංක වලින්: " + id);
  }
}
```

84.3 ලිටරල් ටයිප්ස් (Literal Types)

ලිටරල් ටයිප් එකක් මගින් විචල්‍යයකට ගත හැකි අගය ඉතාමත් නිශ්චිතව සීමා කරයි. නිකම්ම string කියා පවසනවා වෙනුවට අපට පවසන්න පුළුවන් "මෙම විචල්‍යයට තිබිය හැක්කේ 'yes' හෝ 'no' යන වචන දෙක පමණයි" කියා.

ප්‍රායෝගික උදාහරණය (වෙබ් අඩවියක තේමාව):

```
let theme: "light" | "dark";
```

```
theme = "light"; // ✓ හරි
theme = "dark"; // ✓ හරි
// theme = "red"; // ✗ වැරදියි (red යන අගය අප නියම කර නැත)
```

84.4 ටයිප් ඇලියස් (Type Aliases)

සමහර අවස්ථාවලදී අප ලියන ටයිප් ඇනෝටේෂන් ඉතා දිගු සහ සංකීර්ණ විය හැක (විශේෂයෙන් Objects සමඟ). එම සංකීර්ණ ටයිප් එකට අර්ථවත් නමක් ලබා දී එය නැවත නැවත භාවිතා කිරීමට ඉඩ සලසන ක්‍රමය Type Alias ලෙස හැඳින්වේ.

මේ සඳහා type යන මූලපදය භාවිතා කරයි.

විෂූභය:

```
type MyName = string | number; // අලුත් ටයිප් එකක් සාදා ගැනීම
let userId: MyName = 123;
```

84.5 වස්තූන් සඳහා ටයිප් ඇලියස් භාවිතය (Object Aliases)

වෘත්තීය මට්ටමේදී අපි සැම Object එකක් සඳහාම ටයිප් ඇලියස් එකක් සාදන්නෙමු. මෙය කේතය පිරිසිදුව තබා ගැනීමට ඉතා වැදගත් වේ.

```
// 1. ටයිප් එක සාදා ගැනීම (මෙහි නම කැපිටල් අකුරින් ආරම්භ කරන්න)
type User = {
  id: number;
  name: string;
  email: string;
  isAdmin: boolean;
};
```

```
// 2. එය භාවිතා කිරීම
```

```
let user1: User = {
  id: 1,
  name: "Kasun",
  email: "kasun@mail.com",
  isAdmin: true
};
```

```
let user2: User = {
  id: 2,
  name: "Nimal",
  email: "nimal@mail.com",
  isAdmin: false
};
```

වාසිය: ඔබට පසුව User ටයිප් එකට අලුත් දත්තයක් (උදා: phone) එකතු කිරීමට අවශ්‍ය වුවහොත්, එක් තැනක පමණක් වෙනස් කිරීම ප්‍රමාණවත්ය.

84.6 ASCII රූපසටහන: Union සහ Alias ක්‍රියාවලිය

[Union Type ()] (type)]	[Type Alias -----
let val: string number; string number;	type UserID =
(අගය string එකක් හෝ නිකියට UserID	
number එකක් විය හැක)	(මෙම සංකීර්ණ
නම් ලබා දෙයි)	නමැති පොදු
	let id1: UserID = 10;
	let id2: UserID = "A1";

84.7 වෘත්තීය රහස්: Optional Properties

ටයිප් ඇලියස් එකක ඇති යම් දත්තයක් අනිවාර්ය නොවේ නම්, ඒ සඳහා ප්‍රශ්නාර්ථ ලකුණ (?) භාවිතා කළ හැක.

```
type Employee = {
  id: number;
  name: string;
  phone?: string; // මෙය තිබුණත් නැතත් කමක්
  නැත
};
```

```
const emp1: Employee = { id: 1, name: "Ali" };
// ✓ හරි (phone නැත)
```

84.8 පොදු වැරදි (Common Mistakes)

1. Type Guarding අමතක වීම: Union type එකක ඇති විචල්‍යයකට සෘජුවම string Methods (උදා: toUpperCase) ලබා දීම. මූලිකම එය string එකක් දැයි පරීක්ෂා කළ යුතුය.
2. Alias නම සමාන වීම: පවතින JavaScript වචන (උදා: String, Number) ටයිප් ඇලියස් නාම ලෙස භාවිතා කිරීම. සැමවිටම අලුත් නමක් ලබා දෙන්න.
3. Literal Types සහ Union පටලවා ගැනීම: Literal types වලදී නිශ්චිත අගයන් ("admin" | "user") ලබා දෙන අතර Union වලදී දත්ත වර්ග (string | number) ලබා දෙයි.

84.9 වෘත්තීය උපදෙස් (Professional Engineering Tips)

- DRY (Don't Repeat Yourself): එකම Object ව්‍යුහය තුන් කිහිපයක ලියනවා වෙනුවට සැමවිටම Type Alias එකක් සාදා ගන්න.
- Predictable Values: ඔබේ විචල්‍යයකට ගත හැකි අගයන් ඔබ දන්නේ නම්, නිකම්ම string භාවිතා නොකර Literal Union එකක් ("success" | "error" | "loading") භාවිතා කරන්න. එය දෝෂ අවම කරයි.

84.10 සාරාංශය (Key Points)

- Union Types () මගින් එකකට වඩා වැඩි දත්ත වර්ග වලට ඉඩ ලබා දෙයි.
- Type Guarding (typeof) මගින් Union අගයන් ආරක්ෂිතව හසුරුවයි.
- Literal Types මගින් විචල්‍යයකට ගත හැකි නිශ්චිත අගයන් සීමා කරයි.
- Type Aliases මගින් සංකීර්ණ දත්ත වර්ග වලට අර්ථවත් නම් ලබා දෙයි.
- ? ලකුණ මගින් දත්තයක් අත්‍යවශ්‍ය නොවන (Optional) බව පවසයි.

84.11 අභ්‍යාස (Exercises)

1. විචල්‍යයක් සාදා එයට අංකයක් (Number) හෝ බූලියන් (Boolean) අගයක් ගත හැකි වන සේ Union Type එකක් ලබා දෙන්න.
2. Product නමින් ටයිප් ඇලියස් එකක් සාදන්න. එහි නම (String), මිල (Number) සහ

විස්තරය (String - මෙය අනිවාර්ය නොවිය යුතුය) අඩංගු කරන්න.

3. ශ්‍රිතයක් ලියන්න, එයට අගයක් ලෙස "small", "medium", හෝ "large" යන වචන තුනෙන්

එකක් පමණක් ලැබිය යුතුය. වෙනත් වචනයක් දුන්නොත් ටයිප්ස්ක්‍රිප්ට් වැරද්දක්

පෙන්විය යුතුය.

4. පහත කේතය නිවැරදි කර ලියන්න:

```
type Car = { brand: string, year: number };
```

```
let myCar: Car = { brand: "Toyota" };
```

// වැරද්ද: year අගය ලබා දී නැත. එය Optional කිරීමට කුමක් කළ යුතුද?

84.12 ප්‍රායෝගික ප්‍රෝ-ව්‍යාපෘතිය (Pro Mini Project)

ව්‍යාපෘතිය: සරල බැංකු ගනුදෙනු පද්ධතිය (Transaction System)

1. TransactionStatus නමින් ලිටරල් යූනියන් එකක් සාදන්න (අගයන්: "pending", "success", "failed").

2. Transaction නමින් ටයිප් ඇලියස් එකක් සාදන්න. එහි ID (Number හෝ String), මුදල (Number), සහ ඉහත සෑදූ තත්ත්වය (Status) තිබිය යුතුය.

3. ගනුදෙනුවල ලැයිස්තුවක් (Array) සාදා එයට දත්ත ඇතුළත් කරන්න.

4. එම ලැයිස්තුව කියවා (Map), තත්ත්වය "success" නම් පමණක් මුදල මුද්‍රණය කරන ශ්‍රිතයක් ලියන්න.

අසු පස්වන පරිච්ඡේදය: ශ්‍රිත සහ ටයිප්ස්ක්‍රිප්ට් - ආරක්ෂිතව තර්කනය ගොඩනැගීම (Functions in TypeScript)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අධ්‍යයනය කිරීමෙන් පසු ඔබට පහත කරුණු පිළිබඳව අතිශය ගැඹුරු සහ වෘත්තීය අවබෝධයක් ලැබෙනු ඇත:

- ශ්‍රිතවල පරාමිතීන් (Parameters) සඳහා ටයිප් ඇනෝටේෂන් ලබා දෙන ආකාරය.
- ශ්‍රිතයකින් ලැබෙන ප්‍රතිඵලය (Return Type) නිවැරදිව නිර්වචනය කරන ආකාරය.
- අනිවාර්ය නොවන පරාමිතීන් (Optional Parameters) සහ පෙරනිමි පරාමිතීන් (Default Parameters) භාවිතා කරන ආකාරය.

4. void සහ never යන දත්ත වර්ග ශ්‍රිත සමඟ ගැලපෙන ආකාරය තේරුම් ගැනීම.

5. රියැක්ට් (React) වැනි නවීන තාක්ෂණයන්හිදී ඊතල ශ්‍රිත (Arrow Functions)

ටයිප්ස්ක්‍රිප්ට් සමඟ භාවිතා කරන ආකාරය.

6. ශ්‍රිතයක් තවත් ශ්‍රිතයකට පරාමිතියක් ලෙස යවන ආකාරය (Function Types).

85.1 හැඳින්වීම: ඇයි අපි ශ්‍රිත ටයිප් (Type) කරන්නේ?

ජාවාස්ක්‍රිප්ට් වලදී අප ශ්‍රිතයක් ලියන විට, ඕනෑම දත්තයක් එයට යැවීමට ඉඩ ලබා දෙයි.

මෙය බොහෝ විට දෝෂ (Bugs) ඇතිවීමට හේතු වේ.

උදාහරණයක්: සිතන්න ඔබ උපන් වර්ෂය ලබා දුන් විට වයස ගණනය කරන ශ්‍රිතයක් ලිව්වා කියා.

යමෙකු අංකයක් වෙනුවට අකුරු වැලක් (String)

එවැනිවහොත් ජාවාස්ක්‍රිප්ට් එය පරීක්ෂා

කරන්නේ නැත. නමුත් ටයිප්ස්ක්‍රිප්ට් වලදී අපට

ශ්‍රිතයක "ගේට්ටුව" අසල ආරක්ෂකයෙකු

තැබිය හැක. එම ආරක්ෂකයා පරීක්ෂා කරනවා:

1. ශ්‍රිතයට ඇතුළු වන දත්ත නිවැරදි වර්ගයේ ඒවාද? (Parameter Types)

2. ශ්‍රිතය අවසානයේ ලබා දෙන පිළිතුර නිවැරදි වර්ගයේ එකක්ද? (Return Types)

85.2 පරාමිතීන් සහ ප්‍රතිලාභ අගයන් (Parameters and Return Types)

ටයිප්ස්ක්‍රිප්ට් වලදී ශ්‍රිතයක පරාමිතීන් අසලම එහි දත්ත වර්ගය ලියනු ලැබේ. එමෙන්ම

වරහන අවසානයේදී පිළිතුරේ දත්ත වර්ගය ලියනු ලැබේ.

විද්‍යුතය:

```
function ග්‍රිතයේ_නම(පරාමිතිය: දත්ත_වර්ගය):
  ප්‍රතිලාභ_වර්ගය {
    return පිළිතුර;
  }
```

ප්‍රායෝගික උදාහරණය:

```
function calculateTax(price: number): number {
  return price * 0.15;
}
```

```
const tax = calculateTax(100); // ✓ හරි
```

```
// const wrongTax = calculateTax("100"); // ✗
වැරදියි (string එකක් දිය නොහැක)
```

85.3 අනිවාර්ය නොවන සහ පෙරනිමි පරාමිතීන්

සමහර විට ශ්‍රිතයකට දත්තයක් එවීම අත්‍යවශ්‍ය නොවිය හැක. ඒ සඳහා අප ක්‍රම දෙකක් භාවිතා කරමු.

1. Optional Parameters (අනිවාර්ය නොවන) - ?

පරාමිතියේ නමට පසුව ප්‍රශ්නාර්ථ ලකුණ (?) යෙදීමෙන් එය අනිවාර්ය නොවන බව පවසයි.

```
function greet(name: string, message?: string): string {
  if (message) {
    return `ආයුබෝවන් ${name}, ${message}`;
  }
  return `ආයුබෝවන් ${name}`;
}
```

```
console.log(greet("කසුන්", "ඔබට සුභ දවසක්!"));
// දත්ත දෙකම ඇත
console.log(greet("නිමල්")); // පණිවිඩය
(message) නැත, නමුත් වැඩ කරයි
```

2. Default Parameters (පෙරනිමි) - =

පරිශීලකයා අගයක් ලබා නොදුන්නහොත් භාවිතා කළ යුතු අගය කලින්ම ලබා දීමයි.

```
function power(base: number, exponent: number = 2): number {
  return base ** exponent;
}
```

```
console.log(power(5)); // 5^2 = 25 (පෙරනිමි අගය 2 ගනී)
console.log(power(5, 3)); // 5^3 = 125 (අප ලබා දුන් අගය 3 ගනී)
```

85.4 Void සහ Never ටයිප්ස්

Void (හිස් බව)

ශ්‍රිතයකින් කිසිදු අගයක් return නොකරන්නේ නම්, එහි ප්‍රතිලාභ වර්ගය ලෙස void භාවිතා කරයි. (උදාහරණයක් ලෙස කන්සෝලයේ යමක් පෙන්වීමට පමණක් භාවිතා කරන ශ්‍රිත).

```
function logError(error: string): void {
```

```
  console.error("දෝෂයක් සිදු විය: " + error);
  // මෙහි return විධානයක් නැත
}
Never (කිසිදා අවසන් නොවන)
```

යම් ශ්‍රිතයක් කිසිදා අවසන් වන්නේ නැතිනම් හෝ එය සැමවිටම දෝෂයක් (Exception/Error) විසිකරන්නේ නම් never භාවිතා කරයි.

```
function throwError(msg: string): never {
  throw new Error(msg);
}
```

85.5 ඊතල ශ්‍රිත සඳහා ටයිප් ඇතොටේෂන් (Arrow Functions)

නූතන වෙබ් සංවර්ධනයේදී අප බහුලවම භාවිතා කරන්නේ ඊතල ශ්‍රිතයි. ඒවා ටයිප් කරන්නේ මෙලෙසයි:

```
const multiply = (a: number, b: number): number => {
  return a * b;
};
```

// වඩාත් කෙටි ක්‍රමය (Type Inference හරහා)
const add = (a: number, b: number) => a + b;
// මෙහිදී TS තනිවම හඳුනා ගන්නවා return අගය number එකක් බව.

85.6 ASCII රූපසටහන: ශ්‍රිතයක දත්ත ගලායෑම

Input (ආදානය)	Process (සැකසීම)	Output (ප්‍රතිදානය)
(param: string)	----> { Internal Logic }	----> (return: number)
* TS පරීක්ෂා කරයි දත්තය string ද?	* TS පරීක්ෂා කරයි විචල්‍යයන් වල types	* TS පරීක්ෂා කරයි පිළිතුර number ද?

85.7 වෘත්තීය රහස්: Function Type Expressions

අපට ශ්‍රිතයක් සඳහාම වෙනම ටයිප් එකක් (Type Alias) සාදාගත හැක. මෙය පසුව ඉතා සංකීර්ණ වැඩ වලදී ප්‍රයෝජනවත් වේ.

```
type MathFunction = (x: number, y: number) => number;
```

```
const addition: MathFunction = (a, b) => a + b;
const subtraction: MathFunction = (a, b) => a - b;
```


වාසිය: එකම රටාවේ ශ්‍රිත රාශියක් ඇති විට, ඒ සැමවිටම ටයිප් ලියනවා වෙනුවට එකම පොදු ටයිප් එකක් භාවිතා කළ හැක.

85.8 පොදු වැරදි (Common Mistakes)

1. Return Type වැරදීම: number එකක් ලබා දෙනවා යැයි පවසා string එකක් return කිරීමට උත්සාහ කිරීම.
2. Optional Parameters වල පිළිවෙළ: Optional parameter එකක් සැමවිටම තිබිය යුත්තේ අනිවාර්ය පරාමිතීන්ට (Required parameters) පසුවය.
 - function(a?, b) ✗ වැරදියි.
 - function(a, b?) ✓ නිවැරදියි.
3. Inference මඟහැරීම: ටයිප්ස්ක්‍රිප්ට් විසින්ම return type එක හඳුනා ගත හැකි සරල අවස්ථාවලදී පවා අතින් ටයිප් ලිවීමට උත්සාහ කර කේතය අවුල් කිරීම.

85.9 වෘත්තීය උපදෙස් (Professional Engineering Tips)

- Document with Types: ඔබේ ශ්‍රිතයේ පරාමිතීන්ට අර්ථවත් නම් සහ ටයිප්ස් ලබා දෙන්න. එවිට වෙනත් කෙනෙකුට ඔබේ කේතය කියවන විට එය "ස්වයං-ලේඛනයක්" (Self-documenting) ලෙස ක්‍රියා කරයි.
- Avoid 'any' in Functions: ශ්‍රිතවල පරාමිතීන් සඳහා කිසිවිටකත් any භාවිතා නොකරන්න. එයින් ටයිප්ස්ක්‍රිප්ට් හි සම්පූර්ණ ආරක්ෂාව නැති වී යයි.

85.10 සාරාංශය (Key Points)

- ශ්‍රිත පරාමිතීන් සහ return අගයන් අනිවාර්යයෙන්ම ටයිප් කළ යුතුය.
- ? මගින් පරාමිතියක් optional කළ හැක.
- = මගින් default අගයක් ලබා දිය හැක.
- අගයක් නැති විට void ද, කිසිදා අවසන් නොවන විට never ද භාවිතා කරයි.
- ඊතල ශ්‍රිත (Arrow functions) සඳහා ද ටයිප්ස් ලබා දිය හැක.

85.11 අභ්‍යාස (Exercises)

1. අංක දෙකක් ලබා දුන් විට ඒවායින් විශාලතම අංකය (Maximum) ලබා දෙන ශ්‍රිතයක් ටයිප්ස්ක්‍රිප්ට් නීති වලට අනුව ලියන්න.
2. පරිශීලකයාගේ නම සහ ඔහුට සුභ පැතීමට අවශ්‍ය භාෂාව ("si" හෝ "en") ලබා ගෙන පණිවිඩයක් මුද්‍රණය කරන ශ්‍රිතයක් ලියන්න. භාෂාව

ලබා නොදුන්නහොත් එය "en" ලෙස පවතින සේ (Default parameter) සකසන්න.

3. පහත කේතයේ ඇති වැරද්ද කුමක්ද?

```
function process(id?: number, name: string): void {
    console.log(id, name);
}
```

4. පිළිතුරක් ලෙස boolean අගයක් ලබා දෙන ඊතල ශ්‍රිතයක් (Arrow function) ලියන්න.

85.12 ප්‍රායෝගික ප්‍රොජෙක්ට්-ව්‍යාපෘතිය (Pro Mini Project)

ව්‍යාපෘතිය: ලකුණු පුවරු කළමනාකරණ පද්ධතිය (Scoreboard Manager)

1. Player නමින් ටයිප් ඇලියස් එකක් සාදන්න (නම, වත්මන් ලකුණු).
2. පහත ශ්‍රිත තුන ලියන්න:
 - addPoints(player: Player, points: number): Player - ලකුණු එකතු කර අලුත් වස්තුවක් (New Object) ලබා දිය යුතුය.
 - resetScore(player: Player): Player - ලකුණු 0 කර අලුත් වස්තුවක් ලබා දිය යුතුය.
 - displayStats(player: Player): void - ක්‍රීඩකයාගේ විස්තර කන්සෝලයේ පෙන්විය යුතුය.
3. මෙම ශ්‍රිත ක්‍රියාත්මක කර ප්‍රතිඵල පරීක්ෂා කරන්න.

අසු භයවන පරිච්ඡේදය:

ඉන්ටර්ලේස් - දත්ත ව්‍යුහයන් සැලසුම් කිරීම (Interfaces in TypeScript)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අධ්‍යයනය කිරීමෙන් පසු ඔබට පහත කරුණු පිළිබඳව අතිශය ගැඹුරු සහ වෘත්තීය අවබෝධයක් ලැබෙනු ඇත:

1. Interface (ඉන්ටර්ලේස්) යනු කුමක්ද සහ වස්තූන් (Objects) සඳහා එය භාවිතා කරන ආකාරය.
2. ඉන්ටර්ලේස් සහ ටයිප් ඇලියස් (Type Alias) අතර ඇති තාක්ෂණික සමානකම් සහ වෙනස්කම්.
3. ඉන්ටර්ලේස් එකක් තවත් එකක් සමඟ සම්බන්ධ කරන ආකාරය (Extending Interfaces).
4. කියවීමට පමණක් හැකි (Read-only) සහ අනිවාර්ය නොවන (Optional) ගුණාංග පාලනය කිරීම.

5. ඉන්ටර්ෆේස් භාවිතා කරමින් වෘත්තීය මට්ටමේ දත්ත ආකෘති (Data Models) නිර්මාණය කරන ආකාරය.

86.1 ඉන්ටර්ෆේස් යනු කුමක්ද? (What is an Interface?)

පෙර පරිච්ඡේදවලදී අපි "Type Alias" භාවිතා කර වස්තුවක (Object) හැඩය තීරණය කිරීමට ඉගෙන ගත්තෙමු. Interface යනු ද ඒ හා සමානම, නමුත් වස්තූන් (Objects) සඳහාම විශේෂයෙන් නිර්මාණය කරන ලද, වඩාත් බලවත් ක්‍රමවේදයකි.

සරල පැහැදිලි කිරීම: ඉන්ටර්ෆේස් යනු යම් වස්තුවක් සතු විය යුතු "ගිවිසුමක්" (Contract) වැනිය. ඔබ යම් වස්තුවකට ඉන්ටර්ෆේස් එකක් ලබා දුන් විට, එම වස්තුව අනිවාර්යයෙන්ම එම ගිවිසුමේ ඇති නීති රීති (Properties & Methods) පිළිපැදිය යුතුය.

සැබෑ ලෝකයේ උපමාව (රියදුරු බලපත්‍රය): රියදුරු බලපත්‍රයක් ලබා ගැනීමට නම් ඔබ යම් සුදුසුකම් සපුරාලිය යුතුය (ඇස් පෙනීම තිබීම, මාර්ග නීති දැන සිටීම). රියදුරු බලපත්‍රය යනු Interface එකකි. ඕනෑම පුද්ගලයෙකු (Object) එම බලපත්‍රය ලබා ගැනීමට නම් එම සියලුම "ටයිප්ස්" සපුරාලිය යුතුය.

86.2 ඉන්ටර්ෆේස් එකක් ලියන ආකාරය (Syntax)

ඉන්ටර්ෆේස් එකක් සෑදීමට interface යන මූලපදය භාවිතා කරයි. මෙහිදී සමාන ලකුණ (=) අවශ්‍ය නොවේ.

```
// 1. ඉන්ටර්ෆේස් එක නිර්වචනය කිරීම
interface Student {
  id: number;
  name: string;
  grade: number;
}
```

```
// 2. එය භාවිතා කිරීම
const student1: Student = {
  id: 501,
  name: "කසුන්",
  grade: 11
};
```

86.3 Optional සහ Read-only ගුණාංග

වෘත්තීය මෘදුකාංගවලදී සමහර දත්ත වෙනස් කළ නොහැකි ලෙසත්, සමහර ඒවා අනිවාර්ය නොවන ලෙසත් සැකසීමට සිදු වේ.

1. Optional (?): දත්තය තිබුණත් නැතත් කමක් නැති වීම.
2. Readonly (readonly): වරක් ඇතුළත් කළ පසු නැවත වෙනස් කළ නොහැකි දත්ත සඳහා.

```
interface Car {
  readonly vin: string; // මෙය වෙනස් කළ
  නොහැක (Vehicle ID)
  brand: string;
  model: string;
  color?: string; // පාට අනිවාර්ය නැත
}
```

```
const myCar: Car = {
  vin: "XYZ123",
  brand: "Toyota",
  model: "Corolla"
};
```

```
myCar.model = "Camry"; // ✓ හරි
// myCar.vin = "ABC789"; // ✗ වැරදියි
(readonly නිසා වෙනස් කළ නොහැක)
```

86.4 ඉන්ටර්ෆේස් සම්බන්ධ කිරීම (Extending Interfaces)

ඉන්ටර්ෆේස් වල ඇති ප්‍රබලම ලක්ෂණය මෙයයි. අපට එක ඉන්ටර්ෆේස් එකක ඇති ලක්ෂණ තවත් එකකට උරුම කර දිය හැක. මේ සඳහා extends භාවිතා කරයි.

උදාහරණය:

```
interface User {
  name: string;
  email: string;
}
```

```
// User හි ඇති ලක්ෂණ ද සහිතව Admin සාදමු
interface Admin extends User {
  adminLevel: number;
  deleteUser: () => void;
}
```

```
const headAdmin: Admin = {
  name: "නිමල්",
  email: "nimal@pro.com",
  adminLevel: 1,
```

```
deleteUser: () => console.log("මකා දමන ලදී")    };
```

86.5 Interface සහ Type Alias අතර වෙනස (Comparison)

මෙය ඔබ දැනගත යුතුම තාක්ෂණික රහසකි. මේ දෙකම බොහෝ දුරට සමාන වුවත්, ප්‍රධාන වෙනස්කම් කිහිපයක් ඇත.

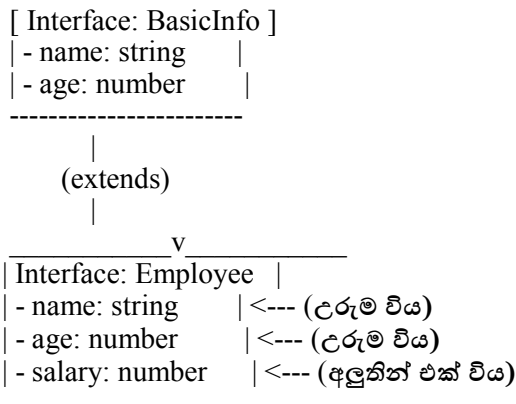
ලක්ෂණය	Interface	Type Alias
ප්‍රධාන අරමුණ	වස්තූන් (Objects) සඳහා පමණි.	ඕනෑම දත්ත වර්ගයක් සඳහා (Unions, Primitives).
Extend කිරීම	`extends` භාවිතා කරයි (පහසුයි).	Intersection (`&`) භාවිතා කරයි.
Re-opening	එකම නමින් ඉන්ටර්ෆේස් දෙකක් සාදා ඒවා එකතු කළ හැක.	එකම නමින් දෙකක් සෑදිය නොහැක.

වෘත්තීය උපදෙස: වස්තූන් සහ පන්ති (Classes)

සඳහා සැමවිටම Interface භාවිතා කරන්න.

Unions සහ සරල දත්ත වර්ග සඳහා Type Alias භාවිතා කරන්න.

86.6 ASCII රූපසටහන: Interface උරුමය (Inheritance)



86.7 වෘත්තීය රහස: Declaration Merging

ඔබ ඉන්ටර්ෆේස් එකක් එකම නමින් දෙවරක් ලියුවහොත්, ටයිප්ස්ක්‍රිප්ට් විසින් ඒවා දෙක එකට එකතු කරයි. මෙය බාහිර පුස්තකාල (External Libraries) භාවිතා කරන විට ඉතා ප්‍රයෝජනවත් වේ.

```
interface Person { name: string; }
interface Person { age: number; }
```

```
// Person දැන් { name, age } දෙකම සහිත එකකි
const me: Person = { name: "Ali", age: 20 };
```

86.8 පොදු වැරදි (Common Mistakes)

1. Interface එකට = ලකුණ යෙදීම: `interface User = { ... }` ලෙස ලිවීම වැරදිය. සමාන ලකුණ භාවිතා කරන්නේ type සඳහා පමණි.
2. Readonly අගයන් වෙනස් කිරීම: `readonly` ලෙස නම් කළ දත්ත පසුව වෙනස් කිරීමට උත්සාහ කිරීම.
3. Extends මහහැරීම: එකම දත්ත ව්‍යුහය නැවත නැවතත් ලිවීම (Don't Repeat Yourself - DRY නීතිය කඩ කිරීම).

86.9 සාරාංශය (Key Points)

- interface වස්තූන්ගේ හැඩය තීරණය කිරීමට භාවිතා කරයි.
- extends මගින් ඉන්ටර්ෆේස් එකිනෙක සම්බන්ධ කළ හැක.
- readonly මගින් දත්ත ආරක්ෂා කරයි.
- ? මගින් දත්ත අත්‍යවශ්‍ය නොවන බව පවසයි.
- වෘත්තීය මට්ටමේදී Objects සඳහා Interfaces භාවිතා කිරීම නිර්දේශ කෙරේ.

86.10 අභ්‍යාස (Exercises)

1. Computer නමින් ඉන්ටර්ෆේස් එකක් සාදන්න (RAM, Processor, Storage). RAM අගය පසුව වෙනස් කළ නොහැකි සේ (readonly) සකසන්න.
2. Animal නමින් ඉන්ටර්ෆේස් එකක් සාදා, එය Dog නමැති අලුත් ඉන්ටර්ෆේස් එකක් හරහා extends කරන්න. Dog සඳහා breed නමැති අලුත් දත්තයක් එක් කරන්න.
3. Type Alias සහ Interface අතර ඇති ප්‍රධාන වෙනස්කම් 3ක් ඔබේ වචනයෙන් ලියන්න.
4. පහත කේතය නිවැරදි කර ලියන්න:

```
interface User { name: string }
interface User { age: number }
```

```
const bob: User = { name: "Bob" }; // වැරද්දක්ද?
කුමක්ද?
```

86.11 ප්‍රායෝගික ප්‍රො-ව්‍යාපෘතිය (Pro Mini Project)

ව්‍යාපෘතිය: ආයතනික සේවක දත්ත කළමනාකරණ පද්ධතිය (Employee Management Model)

1. Address නමින් ඉන්ටර්ෆේස් එකක් සාදන්න (City, Street, Zip).
2. Person නමින් ඉන්ටර්ෆේස් එකක් සාදන්න (Name, Email).
3. Staff නමින් ඉන්ටර්ෆේස් එකක් සාදා එය Person වෙතින් extends කරන්න. එයට Address ඉන්ටර්ෆේස් එක දත්තයක් ලෙස ඇතුළත් කරන්න.
4. සේවකයන් 3 දෙනෙකුගේ දත්ත ඇතුළත් අරාමක් (Array) මෙම Staff ඉන්ටර්ෆේස් එකට අනුකූලව සාදන්න.

අසු හත්වන පරිච්ඡේදය: ජෙනරික්ස් - නම්‍යශීලී සහ නැවත භාවිතා කළ හැකි කේතකරණය (Generics in TypeScript)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අධ්‍යයනය කිරීමෙන් පසු ඔබට පහත කරුණු පිළිබඳව අතිශය ගැඹුරු සහ වෘත්තීය අවබෝධයක් ලැබෙනු ඇත:

1. ජෙනරික්ස් (Generics) යනු කුමක්ද සහ ටයිප්ස්ක්‍රිප්ට් වලදී එහි ඇති අත්‍යවශ්‍යතාවය වටහා ගැනීම.
2. any දත්ත වර්ගය භාවිතා කරනවාට වඩා Generics භාවිතා කිරීමේ ඇති තාක්ෂණික වාසි (Type Safety) හඳුනා ගැනීම.
3. ජෙනරික් ශ්‍රිත (Generic Functions) නිර්මාණය කරන ආකාරය සහ භාවිතා කරන ආකාරය.
4. ජෙනරික් ඉන්ටර්ෆේස් (Generic Interfaces) සහ පන්ති (Generic Classes) ගොඩනැගීම.
5. Constraints (සීමාවන්) භාවිතා කර ජෙනරික්ස් පාලනය කරන ආකාරය.
6. සැබෑ ලෝකයේ මෘදුකාංග නිර්මාණයේදී (API Responses, Data Storage) ජෙනරික්ස් භාවිතා වන අවස්ථා ප්‍රශ්න කිරීම.

87.1 හැඳින්වීම: ජෙනරික්ස් යනු කුමක්ද? (What are Generics?)

මෘදුකාංග ඉංජිනේරු විද්‍යාවේදී අප සැමවිටම උත්සාහ කරන්නේ එකම කේතය විවිධ අවස්ථාවලදී නැවත නැවත භාවිතා කිරීමටයි (Reusability).

ජෙනරික්ස් (Generics) යනු එක් විශේෂිත දත්ත වර්ගයකට (Data Type) පමණක් සීමා නොවී, විවිධ දත්ත වර්ග සමඟ එක හා සමානව වැඩ කළ හැකි සංරචක (Components/Functions) නිර්මාණය කිරීමට ටයිප්ස්ක්‍රිප්ට් ලබා දෙන ප්‍රබලතම මෙවලමයි.

සැබෑ ලෝකයේ උපමාව (වෙළඳ යන්ත්‍රයක් - Vending Machine): සිතන්න වෙළඳ යන්ත්‍රයක් ගැන. එහි ව්‍යුහය එකමය. නමුත් ඔබ එයට මුදල් දමා:

- බීම වර්ගයක් තෝරාගත හැක (ලැබෙන්නේ Drink වර්ගයේ දත්තයකි).
- බිස්කට් පැකට්ටුවක් තෝරාගත හැක (ලැබෙන්නේ Snack වර්ගයේ දත්තයකි). යන්ත්‍රයේ ක්‍රියාකාරිත්වය එකම වුවත්, එය ක්‍රියා කරන දත්ත වර්ගය (අයිතමය) ඔබ තෝරා ගන්නා දේ අනුව වෙනස් වේ. ජෙනරික්ස් යනු එම "යන්ත්‍රය" වැනිය.

87.2 ඇයි අපිට ජෙනරික්ස් අවශ්‍ය වන්නේ? (The "Why")

අපට ඕනෑම දත්ත වර්ගයක් සමඟ වැඩ කළ හැකි ශ්‍රිතයක් අවශ්‍ය නම්, අපට any භාවිතා කළ හැක. නමුත් any භාවිතා කළ විට ටයිප්ස්ක්‍රිප්ට් හි ඇති "ආරක්ෂාව" (Type Safety) නැති වී යයි.

ගැටලුව (Using any):

```
function getItem(arr: any[]): any {
    return arr[0];
}
```

```
const n = getItem([1, 2, 3]); // n දැන් 'any' වේ.
```

// n.toUpperCase(); // ❌ මෙය වැරද්දක් ලෙස TS පෙන්වන්නේ නැත, නමුත් Runtime එකේදී Crash වේ.

විසඳුම (Using Generics): ජෙනරික්ස් මගින් අපට පවසන්න පුළුවන් "මයාට ඕනෑම වර්ගයක අරාමක් ගන්න පුළුවන්, හැබැයි මයා ආපසු දෙන්න ඕනෙන් ඒ වර්ගයේම දත්තයක්" කියා.

87.3 ජෙනරික් ශ්‍රිත (Generic Functions)

ජෙනරික් ශ්‍රිතයක් ලිවීමේදී අප දත්ත වර්ගය වෙනුවට "ටයිප් පරාමිතියක්" (Type Parameter) භාවිතා කරමු. සාමාන්‍යයෙන් මේ සඳහා <T> යන ලකුණ භාවිතා කරයි (T යනු Type යන්නයි).

ව්‍යුහය (Syntax):

```
function identity<T>(arg: T): T {
    return arg;
}
```

භාවිතා කරන ආකාරය:

```
// 1. අංකයක් සමඟ
let output1 = identity<number>(100);

// 2. අකුරු වැලක් සමඟ
let output2 = identity<string>("Hello");

// 3. ටයිප්ස්ක්‍රිප්ට් ස්වයංක්‍රීයව හඳුනා ගැනීම
(Inference)
let output3 = identity(true); // TS දන්නවා මෙය
boolean බව.
```

87.4 ජෙනරික් ඉන්ටර්ෆේස් (Generic Interfaces)

වස්තූන් (Objects) සඳහා ද අපට ජෙනරික්ස් භාවිතා කළ හැක. උදාහරණයක් ලෙස, සර්වර් එකකින් ලැබෙන ප්‍රතිචාරයක් (API Response) ඕනෑම දත්තයක් විය හැක.

```
interface ApiResponse<T> {
    data: T;
    status: number;
    message: string;
}
```

```
// පරිශීලක දත්ත සඳහා
interface User {
    name: string;
    id: number;
}
```

```
const userResponse: ApiResponse<User> = {
    data: { name: "Kasun", id: 1 },
    status: 200,
    message: "Success"
};
```

```
// භාණ්ඩ ලැයිස්තුවක් සඳහා
const productResponse: ApiResponse<string[]>
= {
    data: ["Apple", "Orange"],
    status: 200,
    message: "Success"
};
```

87.5 ජෙනරික් පන්ති (Generic Classes)

විවිධ දත්ත වර්ග ගබඩා කළ හැකි පන්තියක් සාදන ආකාරය බලමු.

```
class StorageBox<T> {
    private contents: T[] = [];

    addItem(item: T) {
        this.contents.push(item);
    }

    getItems(): T[] {
        return this.contents;
    }
}
```

```
const numberBox = new
StorageBox<number>();
numberBox.addItem(10);
// numberBox.addItem("Hello"); // ✗ වැරද්දක්
පෙන්වයි.
```

```
const stringBox = new StorageBox<string>();
stringBox.addItem("TypeScript");
```

87.6 ජෙනරික් සීමාවන් (Generic Constraints - extends)

සමහර විට අපට අවශ්‍ය වෙනවා "ඕනෑම දත්ත වර්ගයක්" යන්න මඳක් සීමා කරන්න. උදාහරණයක් ලෙස, අප ලබා දෙන දත්තයට අනිවාර්යයෙන්ම .length යන ගුණාංගය තිබිය යුතු යැයි අපට පැවසිය හැක.

```
interface HasLength {
    length: number;
}
```

```
function logLength<T extends
HasLength>(item: T): void {
  console.log(`දිග ප්‍රමාණය: ${item.length}`);
}
```

logLength("Hello"); // ✓ String එකට length ඇත.
logLength([1, 2, 3]); // ✓ Array එකට length ඇත.
// logLength(123); // ✗ අංකයකට length එකක් නැති නිසා වැරදිදක් පෙන්වයි.

87.7 ASCII රූපසටහන: Generics ක්‍රියා කරන ආකාරය

[කේතය (Code)]	[වයිස් පරාමිතිය <T>]	[අවසාන ප්‍රතිඵලය]
function<T>(arg:T) <-- <string> යෙදූ විට -->	arg අනිවාර්යයෙන්ම string විය යුතුය.	
function<T>(arg:T) <-- <number> යෙදූ විට -->	arg අනිවාර්යයෙන්ම number විය යුතුය.	

87.8 පොදු වැරදි (Common Mistakes)

- Arrow Functions සමඟ භාවිතය: ඊතල ශ්‍රිත ලිවීමේදී JSX සමඟ පටලැවිලි ඇති විය හැක. ඒ සඳහා <T> හෝ <T extends {}> ලෙස ලිවීම වෘත්තීය ක්‍රමයයි.
- const wrap = <T>(arg: T) => arg; (React වලදී සමහර විට ගැටළු දිය හැක).
- අනවශ්‍ය ලෙස Generics භාවිතය: සරලව කළ හැකි දේටත් Generics යොදා කේතය සංකීර්ණ කිරීම. (අවශ්‍ය තැනට පමණක් භාවිතා කරන්න).

87.9 සාරාංශය (Key Points)

- ජෙනරික්ස් මගින් විවිධ දත්ත වර්ග සමඟ වැඩ කළ හැකි නැවත භාවිතා කළ හැකි කේත සාදයි.
- <T> යනු දත්ත වර්ගය සඳහා භාවිතා කරන විචල්‍යයකි.
- any වලට වඩා ජෙනරික්ස් ආරක්ෂිතය (Type Safety).
- extends මගින් ජෙනරික්ස් වලට සීමාවන් පැනවිය හැක.

87.10 අභ්‍යාස (Exercises)

- අරාවක (Array) ඇති අයිතම මාරු කරන (Reverse) ජෙනරික් ශ්‍රිතයක් ලියන්න.
- KeyValuePair<K, V> නමින් ජෙනරික් ඉන්ටර්ෆේස් එකක් සාදා එයට යතුරක් (Key) සහ අගයක්

(Value) ලබා දෙන්න. (උදා: id: number, name: string).

- අංක සහ අකුරු (string | number) පමණක් භාර ගන්නා ජෙනරික් සීමාවක් සහිත ශ්‍රිතයක් නිර්මාණය කරන්න.

අසු අටවන පරිච්ඡේදය (විශේෂ ප්‍රෝ-මට්ටමේ): උසස් ටයිප් ඉංජිනේරු විද්‍යාව සහ කර්මාන්ත සම්මත රටා (Advanced Type Engineering and Industrial Patterns)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම විශේෂ ප්‍රෝ-මට්ටමේ පරිච්ඡේදය අධ්‍යයනය කිරීමෙන් පසු, ඔබ සාමාන්‍ය ටයිප්ස්ක්‍රිප්ට් ලියන්නෙකුගේ මට්ටම සම්පූර්ණයෙන් ඉක්මවා ගොස් "Senior Type Engineer" කෙනෙකු ලෙස සිතීමට පටන් ගනු ඇත:

- Discriminated Unions: දෝෂ රහිතව සංකීර්ණ දත්ත ව්‍යුහයන් (උදා: API Responses) කළමනාකරණය කරන ආකාරය.
- Utility Types: ටයිප්ස්ක්‍රිප්ට් හි ඇති Partial, Pick, Omit, සහ Readonly වැනි ප්‍රබල මෙවලම් භාවිතා කර කේතය කාර්යක්ෂම කිරීම.
- Type Guarding & Narrowing: අති දියුණු ආකාරයට දත්ත වර්ග තහවුරු කරගැනීමේ තාක්ෂණයන්.
- Type Assertions: as මූලපදය භාවිතා කළ යුතු නිවැරදි සහ ආරක්ෂිත අවස්ථා හඳුනා ගැනීම.
- Index Signatures: ගතිකව දත්ත ඇතුළත් වන වස්තූන් (Dynamic Objects) සඳහා ටයිප්ස් නිර්මාණය කිරීම.
- Mapped Types: පවතින ටයිප් එකක් ඇසුරින් තවත් ටයිප් එකක් ස්වයංක්‍රීයව නිර්මාණය කිරීමේ රහස්.

1. තීරණාත්මක යුනියන් ටයිප්ස් (Discriminated Unions)

මෙය වෘත්තීය මට්ටමේ මෘදුකාංග නිර්මාණයේදී (විශේෂයෙන් React සහ Backend අතර දත්ත හුවමාරුවේදී) භාවිතා කරන අතිශය බලවත් රටාවකි.

ගැටලුව: සමහර විට වස්තුවක ඇති එක් අගයක් මත පදනම්ව එහි ඇති අනෙක් දත්ත වෙනස් වේ.

විසඳුම: සෑම ටයිප් එකකටම පොදු "ලේඛනයක්" (Literal type) ලබා දීම.

ප්‍රෝ-මටටමේ කේත උදාහරණය (API තත්ත්ව පාලනය):

```
type SuccessResponse = {
  status: "success"; // මෙය ලේඛනයයි
  data: string[];
};
```

```
type ErrorResponse = {
  status: "error"; // මෙය ලේඛනයයි
  message: string;
};
```

```
type ApiResponse = SuccessResponse |
  ErrorResponse;
```

```
function handleResponse(response:
  ApiResponse) {
  if (response.status === "success") {
    // ටයිප්ස්ක්‍රිප්ට් දැන් දන්නවා මෙහි ඇත්තේ
    'data' මිස 'message' නොවන බව
    console.log(response.data.map(d =>
      d.toUpperCase()));
  } else {
    // ටයිප්ස්ක්‍රිප්ට් දැන් දන්නවා මෙහි ඇත්තේ
    'message' බව
    console.error(response.message);
  }
}
```

ඇයි මෙය වැදගත්? මෙය භාවිතා කිරීමෙන් ඔබ කිසිදාක සාර්ථක ප්‍රතිචාරයක (Success) ඇති දත්ත අසාර්ථක ප්‍රතිචාරයක (Error) ඇති පණිවිඩ සමඟ පටලවා නොගන්නා බවට ටයිප්ස්ක්‍රිප්ට් සහතික වේ.

2. භාවිත උපයෝගීතා ටයිප්ස් (Utility Types)

ටයිප්ස්ක්‍රිප්ට් සතුව පවතින ටයිප්ස් ඉතා ඉක්මනින් වෙනස් කර ගැනීමට හැකි "සූදානම් කළ මෙවලම්" (Built-in tools) කිහිපයක් ඇත.

88.1 Partial (අර්ධ වශයෙන් ලබා ගැනීම)

පන්තියක හෝ ටයිප් එකක ඇති සියලුම දත්ත අත්‍යවශ්‍ය නොවන (Optional) ඒවා බවට පත් කරයි.

- භාවිතය: දත්තයක් යාවත්කාලීන (Update) කරන අවස්ථාවලදී.

```
interface User {
  id: number;
  name: string;
  email: string;
}
```

```
// පරිශීලකයාගේ නම පමණක් වෙනස් කිරීමට
අවශ්‍ය නම්:
function updateUser(id: number,
  fieldsToUpdate: Partial<User>) {
  // fieldsToUpdate තුළ name, email හෝ දෙකම
  තිබිය හැක
}
```

88.2 Pick<T, K> (තෝරා ගැනීම)

විශාල ටයිප් එකකින් අපට අවශ්‍ය කොටස් කිහිපයක් පමණක් තෝරාගෙන අලුත් ටයිප් එකක් සාදයි.

```
type UserPreview = Pick<User, "name" |
  "email">;
// UserPreview තුළ ඇත්තේ නම සහ ඊමේල්
පමණි. id නැත.
```

88.3 Omit<T, K> (ඉවත් කිරීම)

විශාල ටයිප් එකකින් අපට අවශ්‍ය නොවන කොටසක් ඉවත් කර ඉතිරිය ලබා ගනී.

```
type UserWithoutEmail = Omit<User, "email">;
// මෙහි id සහ name පමණක් ඉතිරි වේ.
```

3. දර්ශක අත්සන් (Index Signatures)

සමහර විට අප සාදන වස්තුවක (Object) තිබිය හැකි Key නම් මොනවාදැයි අප කලින්ම නොදනී (උදා: දත්ත ගබඩාවකින් ලැබෙන ඕනෑම ලැයිස්තුවක්). එවැනි විටකදී අප දර්ශක අත්සන් භාවිතා කරමු.

```
interface Dictionary {
  [key: string]: string; // ඕනෑම string key
  එකකට string value එකක් තිබිය හැක
}
```

```
const myDict: Dictionary = {
  "apple": "ඇපල්",
  "banana": "කෙසෙල්",
  "orange": "දොඩම්"
}
```

$$\};$$

4. ටයිප් තහවුරු කිරීම (Type Assertions - as)

සමහර අවස්ථාවලදී බ්‍රවුසරයෙන් ලැබෙන දත්තයක වර්ගය ටයිප්ස්ක්‍රිප්ට් වලට වඩා හොඳින් ඔබ දන්නා අවස්ථා ඇත. එවිට "මට විශ්වාසයි මෙය අනවල් වර්ගයේ දත්තයක්" කියා ටයිප්ස්ක්‍රිප්ට් හට පැවසීමට as භාවිතා කරයි.

```
const myInput =
document.getElementById("user-input") as
HTMLInputElement;
```

```
// දැන් අපට සෘජුවම .value භාවිතා කළ හැක
console.log(myInput.value);
```

ප්‍රෝ අවවාදය: මෙය භාවිතා කළ යුත්තේ ඔබට 100% ක් විශ්වාස නම් පමණි. නැතිනම් වැඩසටහන ක්‍රියාත්මක වන විට (Runtime) බිද වැටිය හැක.

5. ASCII රූපසටහන: උසස් වයිඒ පරිවර්තනය

```

[ Original Type: User ]
{ id, name, email, role }

      |
      |-----|
      |         |         |
(Pick<User, "id">) (Omit<User, "role">)
(Partial<User>)

      |         |         |
      { id }   { id, name, email }   { id?,
name?, email?, role? }

```

6. වෘත්තීය රහස්: මෝඩ්‍යුලර් ටයිප්ස් (Mapped Types)

Mapped types මගින් අපට පවතින ටයිප් එකක ඇති සෑම Key එකක්ම එකවර වෙනස් කිරීමට ඉඩ ලබා දෙයි. මෙය බොහෝ විට වෘත්තීය මට්ටමේ පුස්තකාල (Libraries) නිර්මාණය කිරීමේදී භාවිතා කරයි.

```
type FeatureFlags = {
  darkMode: boolean;
  newUserFlow: boolean;
};
```

```
// FeatureFlags හි ඇති සියලුම boolean අගයන්
string බවට හරවමු
```

```
type OptionsFlags<T> = {
    [Property in keyof T]: string;
};
```

```
type NewFlags = OptionsFlags<FeatureFlags>;
/* NewFlags දැන් මෙලෙස දිස්වේ:
   { darkMode: string, newUserFlow: string }
*/
```

7. වෘත්තීය ඉංජිනේරු උපදෙස් (Senior Engineering Tips)

- Discriminated Unions over Optional Props:
වස්තු වක දත්ත ? ලකුණු දමා Optional කරනවාට
වඩා Discriminated Unions භාවිතා කරන්න. එයින්
කේතය වඩාත් පැහැදිලි සහ ආරක්ෂිත වේ.

- Utility Types for Cleaner Code: එක හා සමාන ටයිප්ස් දෙකක් තිබේ නම්, දෙවන එක ලිවීම වෙනුවට පළමු එකෙන් Pick හෝ Omit භාවිතා කරන්න. එය DRY (Don't Repeat Yourself) මූලධර්මය ආරක්ෂා කරයි.

- Branded Types: ඉතා සංකීර්ණ වැඩ වලදී, number වැනි සාමාන්‍ය අගයන් පවා විශේෂිත නම් (උදා: USD_Currency vs LKR_Currency) ලබා දී වෙන් කිරීමට උත්සාහ කරන්න.

8. සාරාංශය (Key Points)

- Discriminated Unions මගින් සංකීර්ණ දත්ත ආරක්ෂිතව පාලනය කරයි.

- Utility Types (Partial, Pick, Omit) මගින් කේතය කෙටි කරයි.

- Index Signatures මගින් ගතික දත්ත ව්‍යුහයන් පාලනය කරයි.

- as මගින් ටයිප්ස් කළේ භි තීති තාවකාලිකව මහගුරුය හැක (පරෙස්සමිත් භාවිතා කරන්න).

- Mapped Types මගින් ටයිප්ස් ස්වයංක්‍රීයව වෙනස් කළ හැක.

9. ප්‍රෝ-මට්ටමේ අභ්‍යාසය (Advanced Pro Challenge)

අනිශ්චයාගය:

1. Vehicle නමින් වයිස් ඇලියස් එකක් සාදන්න.
එය Car (door count ඇති) සහ Bike (hasCarrier ඇති) යන දත්ත වර්ග දෙකේ Union එකක් විය යුතුය.

2. type: "car" සහ type: "bike" ලෙස Discriminant ලේඛනයක් භාවිතා කරන්න.

3. Partial සහ Omit භාවිතා කර Vehicle ටයිප් එකෙන් පින්තූරයක් පමණක් වෙනස් කළ හැකි (imageUpdate) අලුත් ටයිප් එකක් නිර්මාණය කරන්න.

4. වෘත්තීය මට්ටමේ ශ්‍රීතයක් ලියන්න, එයට Vehicle එකක් ලබා දුන් විට එහි වර්ගය අනුව නිවැරදි දත්ත පමණක් පෙන්විය යුතුය.

අසු නවවන පරිච්ඡේදය: ඉන්ටර්ලේස්, දියුණු පන්ති සහ ප්‍රකාශන ගොනු (Interfaces, Advanced Classes, and Declaration Files)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබ පහත සඳහන් සංකීර්ණ කරුණු පිළිබඳව පූර්ණ ප්‍රවීණත්වයක් ලබා ගනු ඇත:

1. Interfaces (ඉන්ටර්ලේස්): වස්තුවක ව්‍යුහය තීරණය කිරීමට භාවිතා කරන වෘත්තීය ක්‍රමය සහ එය Type Alias වලින් වෙනස් වන ආකාරය.
2. Access Modifiers: පන්තියක ඇති දත්ත ආරක්ෂා කිරීමට භාවිතා කරන public, private, සහ protected යන රහස් පාලක.
3. Abstract Classes: සෘජුවම වස්තූන් සෑදිය නොහැකි, සැලසුම් සඳහා පමණක් භාවිතා කරන "අමූර්ත පන්ති".
4. Implementing Interfaces: පන්තියක් අනිවාර්යයෙන්ම පිළිපැදිය යුතු නීති මාලාවක් පැනවීම.
5. Declaration Files (.d.ts): ජාවාස්ක්‍රිප්ට් පුස්තකාල සඳහා ටයිප්ස්ක්‍රිප්ට් සහය ලබා දෙන ආකාරය සහ @types භාවිතය.

89.1 ඉන්ටර්ලේස් යනු කුමක්ද? (What is an Interface?)

Interface (ඉන්ටර්ලේස්) යනු වස්තුවක් (Object) සතු විය යුතු හැඩය හෝ ව්‍යුහය විස්තර කරන "ගිවිසුමක්" වැනිය. මෙය Type Alias (type) එකකට ඉතා සමාන වුවත්, වෘත්තීය මට්ටමේදී වස්තූන් සඳහා වැඩිපුරම භාවිතා කරන්නේ මෙයයි.

ඇයි ඉන්ටර්ලේස් භාවිතා කරන්නේ? මෙහි ඇති විශේෂත්වය වන්නේ Extending (දිගු කිරීමේ)

හැකියාවයි. එනම්, එක ඉන්ටර්ලේස් එකක ඇති ලක්ෂණ තවත් එකකට ඉතා පහසුවෙන් උරුම කර දිය හැක.

```
interface Person {
  name: string;
  age: number;
}
```

```
// උරුම කර ගැනීම (Inheritance)
interface Employee extends Person {
  employeeId: number;
  department: string;
}
```

```
const newStaff: Employee = {
  name: "කසුන්",
  age: 30,
  employeeId: 501,
  department: "IT"
};
```

89.2 Interface vs Type Alias (ප්‍රධාන වෙනස)

බොහෝ දෙනෙක් අසන ප්‍රශ්නයක් නම් "මම පාවිච්චි කරන්න ඕනේ type ද නැත්නම් interface ද?" යන්නයි.

- Type Alias (type): ඕනෑම දෙයක් සඳහා භාවිතා කළ හැක (Union, Primitives, Tuples). මෙය සෑදූ පසු පසුව වෙනස් කළ නොහැක (Static).
- Interface: වස්තූන් (Objects) සහ පන්ති (Classes) සඳහා පමණක් භාවිතා කරයි. පසුව අලුත් ගුණාංග එකතු කළ හැක (Declaration Merging).

වෘත්තීය උපදෙස: සැමවිටම වස්තුවක ව්‍යුහය (Shape) සඳහා Interface භාවිතා කරන්න. Union හෝ Tuple වැනි විශේෂ වැඩ සඳහා පමණක් Type භාවිතා කරන්න.

89.3 පන්ති සහ ප්‍රවේශ පාලක (Access Modifiers in Classes)

අපි පරිමාව 4 හිදී ජාවාස්ක්‍රිප්ට් පන්ති (Classes) ගැන ඉගෙන ගත්තෙමු. නමුත් ටයිප්ස්ක්‍රිප්ට් වලදී අපට එම පන්තියක ඇති දත්ත වලට පිටතින් පිවිසිය හැකි සීමාවන්

(Security) තීරණය කළ හැක.

1. public (සම්මතය): ඕනෑම තැනක සිට මෙම දත්තය කියවිය හැක.
2. private: මෙම දත්තය භාවිතා කළ හැක්කේ එම පන්තිය (Class) ඇතුළතදී පමණි. පිටත සිට එය වෙනස් කළ නොහැක.
3. protected: එම පන්තිය ඇතුළත සහ එයින් උරුම වූ (Child classes) පන්ති ඇතුළත පමණක් භාවිතා කළ හැක.

ප්‍රායෝගික උදාහරණය:

```
class BankAccount {
    public holderName: string;
    private balance: number; // පිටතට රහස්‍ය

    constructor(name: string, initialBalance:
number) {
        this.holderName = name;
        this.balance = initialBalance;
    }

    public getBalance() {
        return `ඔබේ ඉතිරි මුදල: ${this.balance}`;
    }
}
```

```
const myAcc = new BankAccount("නිමල්",
5000);
console.log(myAcc.holderName); // ✓ වැඩ
කරයි
// console.log(myAcc.balance); // ✗ වැරද්දක්
පෙන්වයි (Private බැවින්)
```

89.4 පන්තියක් ඉන්ටර්ෆේස් එකකට අනුගත කිරීම (Implements)

අපට පුළුවන් පන්තියක් සෑදීමට පෙර එය අනුගතමය කළ යුතු නීති මාලාවක් (Interface) පැනවීමට.

```
interface Runnable {
    start(): void;
    stop(): void;
}
```

```
class Car implements Runnable {
    start() { console.log("කාර් එක පණ
ගැන්වුණා"); }
    stop() { console.log("කාර් එක නැවතුණා"); }
```

```
// start සහ stop ශ්‍රිත දෙකම මෙහි තිබීම අනිවාර්ය
වේ
}
```

89.5 ප්‍රකාශන ගොනු සහ බාහිර පුස්තකාල (.d.ts files)

සමහර විට ඔබ පරණ ජාවාස්ක්‍රිප්ට් පුස්තකාලයක් ඔබේ ටයිප්ස්ක්‍රිප්ට් ව්‍යාපෘතියට එකතු කළහොත්, ටයිප්ස්ක්‍රිප්ට් එය හඳුනා නොගෙන වැරදි පෙන්වනු ඇත. එයට හේතුව එම පැරණි කේතවල "Types" නොමැති වීමයි.

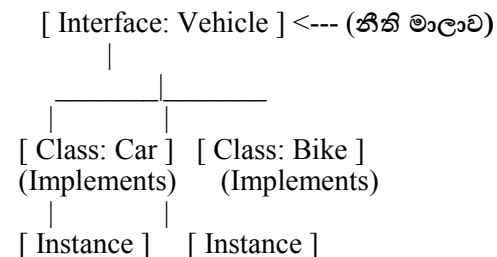
මෙය විසඳීමට ලෝකයේ සංවර්ධකයන් විසින් DefinitelyTyped නමැති මහා දත්ත ගබඩාවක එම පුස්තකාල සඳහා වන ටයිප්ස් (Declarations) ලියා තබා ඇත.

භාවිතා කරන ආකාරය: ඔබ lodash නමැති පුස්තකාලය ස්ථාපනය කරන්නේ නම්, ඒ සමඟම එහි ටයිප්ස් ද මෙලෙස ස්ථාපනය කළ යුතුය:

```
npm install lodash
npm install -D @types/lodash
```

දැන් ටයිප්ස්ක්‍රිප්ට් විසින් එම පුස්තකාලය තුළ ඇති සියලුම ශ්‍රිතවල දත්ත වර්ග හඳුනා ගනී.

89.6 ASCII රූපසටහන: උසස් ටයිප් සම්බන්ධතා



89.7 වෘත්තීය රහස් (Senior Developer Secrets)

- Readonly Properties: ඉන්ටර්ෆේස් එකක දත්තයක් කිසිදා වෙනස් කළ නොහැකි ලෙස සකස් කිරීමට readonly භාවිතා කරන්න.
- interface User { readonly id: number; }

- Abstract Classes vs Interfaces: ඔබට පොදු ක්‍රියාකාරිත්වයක් (Implementation) ලබා දීමට අවශ්‍ය නම් Abstract Class භාවිතා කරන්න. ඔබට හුදෙක් හැඩය (Shape) පමණක් ලබා දීමට අවශ්‍ය නම් Interface භාවිතා කරන්න.

89.8 සාරාංශය (Key Points)

- Interface යනු වස්තූන් සඳහා භාවිතා කරන නම්‍යශීලී ටයිප් ක්‍රමයකි.
- Access Modifiers (private, protected) මගින් දත්ත ආරක්ෂා කරයි.
- implements මගින් පන්ති (Classes) සඳහා නීති පනවයි.
- @types මගින් ජාවාස්ක්‍රිප්ට් පුස්තකාල ටයිප්ස්ක්‍රිප්ට් සමඟ සම්බන්ධ කරයි.

89.9 අභ්‍යාස (Exercises)

1. Animal නමින් ඉන්ටර්ලේස් එකක් සාදා එය Dog සහ Cat සඳහා extends කරන්න.
2. Employee නමින් පන්තියක් සාදා එහි මාසික වැටුප (Salary) private ලෙස තබා, එය බාහිරින් කියවීමට සහ වෙනස් කිරීමට Getters සහ Setters සාදන්න.
3. tsc --init මගින් සැකසෙන tsconfig.json ගොනුව විවෘත කර එහි ඇති target සහ strict යන කොටස්වලින් සිදුවන දේ අන්තර්ජාලයෙන් සොයා බලන්න.

89.10 පරිමාව 8 අවසන් නිගමනය (Final Verdict)

සුභ පැතුම්! ඔබ දැන් පරිමාව 8 (TypeScript) සැබවින්ම, 100% ක් සම්පූර්ණ කර අවසානය. ඔබ සතුව දැන්:

- ජාවාස්ක්‍රිප්ට් වලට ආරක්ෂාව ලබා දීමේ දක්ෂතාවය.
- සංකීර්ණ දත්ත ව්‍යුහයන් (Generics, Interfaces) කළමනාකරණය කිරීමේ හැකියාව.
- වෘත්තීය මට්ටමේ මෘදුකාංග සැලසුම් කිරීමේ (OOP) දැනුම.

ඔබ දැන් සැබෑ TypeScript Architect කෙනෙකි.

අපි දැන් අපගේ අවසාන මහා පිම්ම පැනීමට සූදානම්. ඒ තමයි නූතන වෙබ් සංවර්ධනයේ මහා බලකොටුව වන පරිමාව 9: Next.js.

නෙක්ස්ට් ජේ.එස් (Next.js) - අති නවීන වෙබ් රාමුව

අනුවන පරිච්ඡේදය: නෙක්ස්ට් ජේ.එස් හැඳින්වීම - රියැක්ට් තාක්ෂණයේ ඊළඟ පියවර (Introduction to Next.js - The Next Step of React)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අධ්‍යයනය කිරීමෙන් පසු ඔබට පහත කරුණු පිළිබඳව අතිශය ගැඹුරු සහ වෘත්තීය අවබෝධයක් ලැබෙනු ඇත:

1. Next.js (නෙක්ස්ට් ජේ.එස්) යනු කුමක්ද සහ එය රියැක්ට් (React) වලින් වෙනස් වන්නේ කෙසේද යන්න වටහා ගැනීම.
2. පුස්තකාලයක් (Library) සහ රාමුවක් (Framework) අතර ඇති තාක්ෂණික වෙනස නැවත විමසා බැලීම.
3. Server-Side Rendering (SSR) සහ Static Site Generation (SSG) යන සංකල්ප තේරුම් ගැනීම.
4. Next.js භාවිතා කිරීමේ ව්‍යාපාරික සහ තාක්ෂණික වාසි (SEO, Speed, DX) හඳුනා ගැනීම.
5. නූතන වෙබ් අඩවි නිර්මාණය සඳහා Next.js ලොව අංක 1 තේරීම වී ඇත්තේ ඇයිද යන්න වටහා ගැනීම.

90.1 නෙක්ස්ට් ජේ.එස් යනු කුමක්ද? (What is Next.js?)

Next.js යනු රියැක්ට් (React) මත පදනම් වූ, වෙබ් මෘදුකාංග නිර්මාණය කිරීමට භාවිතා කරන "පූර්ණ වෙබ් රාමුවකි" (Full-stack Web Framework). මෙය Vercel සමාගම විසින් නිර්මාණය කරන ලද්දකි.

අපි පරිමාව 6 දී ඉගෙන ගන්නා රියැක්ට් යනු පරිශීලක අතුරුමුහුණත (UI) සෑදීමට භාවිතා කරන පුස්තකාලයක් (Library) පමණක් බව. නමුත් සැබෑ ලෝකයේ මහා පරිමාණ වෙබ් අඩවියක් නිර්මාණය කිරීමට රියැක්ට් වලට අමතරව තවත් දේවල් අවශ්‍ය වේ:

- රවුටින් (Routing - පිටු අතර ගමනය).
- පින්තූර ප්‍රශස්තකරණය (Image Optimization).
- සෙවුම් යන්ත්‍ර ප්‍රශස්තකරණය (SEO).
- දත්ත ලබා ගැනීමේ ක්‍රමවේද (Data Fetching).

සාමාන්‍ය රියැක්ට් ව්‍යාපෘතියකදී මේවා අප විසින්ම එකින් එක සකසා ගත යුතුය. නමුත් Next.js මේ සියල්ල එකම පැකේජයක් ලෙස අපට ලබා දෙයි.

සරල සිංහල පැහැදිලි කිරීම: රියැක්ට් යනු ඉතා හොඳ "එන්ජිමක්" වැනිය. නමුත් වාහනයක් පැදවීමට නම් අපට එන්ජිමට අමතරව රෝද, බොඩි එක, ලයිට් සහ සීට් අවශ්‍ය වේ. Next.js යනු එම සියලුම කොටස් එකතු කර සාදන ලද, අපට කෙලින්ම පැදවිය හැකි "සම්පූර්ණ මෝටර් රථයකි".

90.3 සර්වර් සයිඩ් රෙන්ඩරින් (Server-Side Rendering - SSR) යනු කුමක්ද?

මෙය Next.js හි ඇති බලවත්ම ලක්ෂණයයි. සාමාන්‍ය රියැක්ට් වලදී සිදුවන්නේ කුමක්දැයි බලමු:

1. පරිශීලකයා වෙබ් අඩවියට එයි.
2. බ්‍රවුසරයට මුලින්ම ලැබෙන්නේ හිස් HTML පිටුවකි.
3. බ්‍රවුසරය විසින් විශාල JavaScript ගොනුවක් බාගත කර එය ක්‍රියාත්මක කරයි.
4. ඉන්පසු තිරය මත රූප දිස්වේ.

- ගැටලුව: අන්තර්ජාලය වේගවත් නැතිනම් පරිශීලකයාට තත්පර කිහිපයක් හිස් තිරයක් පෙන්වේ. Google සෙවුම් යන්ත්‍රයට ද මෙහි ඇති දත්ත කියවීමට අපහසු වේ.

Next.js (SSR) ක්‍රමය:

1. පරිශීලකයා වෙබ් අඩවියට එයි.
2. සේවාදායකය (Server) විසින්ම රියැක්ට් කේතය කියවා, එය සම්පූර්ණ HTML පිටුවක් බවට පත් කරයි.
3. බ්‍රවුසරයට ලැබෙන්නේ දැනටමත් දත්ත පිරවූ සම්පූර්ණ පිටුවකි.
4. තිරය මත එසැණින් රූප දිස්වේ.
 - වාසිය: වේගය අතිශය ඉහළය. Google සෙවුම් යන්ත්‍රයට සියල්ල හොඳින් පෙන්වේ.

90.2 රියැක්ට් සහ නෙක්ස්ට් ජේ.එස් අතර තාක්ෂණික වෙනස (React vs Next.js)

වෘත්තීය මට්ටමේදී මෙම වෙනස හඳුනා ගැනීම ඉතා වැදගත් වේ.

ලක්ෂණය	සාමාන්‍ය රියැක්ට් (CRA/Vite)	නෙක්ස්ට් ජේ.එස් (Next.js)
වර්ගීකරණය	Library (පුස්තකාලයකි)	Framework (රාමුවකි)
රෙන්ඩරින් ක්‍රමය	Client-Side Rendering (CSR)	SSR, SSG, සහ ISR සහය දක්වයි
රවුටින්	`react-router-dom` අමතරව අවශ්‍යයි	අභ්‍යන්තරයේම පවතී (File-based)
SEO	දුර්වලයි (Google බොටස්ට්‍රාට් කියවීමට අපහසුයි)	විශිෂ්ටයි (පිටුව කලින්ම සූදානම් කර ඇත)
වේගය	විශාල JS ගොනු නිසා ආරම්භයේදී මන්දගාමීයි	ඉතා වේගවත්ය (Automatic Code Splitting)

90.4 ස්ටැටික් සයිට් ජෙනරේෂන් (Static Site Generation - SSG)

සමහර පිටුවල දත්ත නිතර වෙනස් වන්නේ නැත (උදා: "මා ගැන" පිටුව හෝ බ්ලොග් ලිපියක්). එවැනි පිටු දහස් ගණනක් වුවද, වෙබ් අඩවිය සාදන මොහොතේදීම (Build time) HTML ගොනු ලෙස සකසා තැබීමට Next.js සමත් වේ. පරිශීලකයා එම පිටුවට එන විට, ලෝකයේ වේගවත්ම ආකාරයට එය ඔහුට පෙනෙන්නට සලස්වයි.

90.5 ඇයි වෘත්තීය මට්ටමේදී Next.js තෝරා ගන්නේ? (DX and SEO)

1. Search Engine Optimization (SEO): වෙබ් අඩවිය Google හි මූලධර්ම ගෙන ඒමට Next.js තරම් බලවත් වෙනත් රාමුවක් නැත.
2. Image Optimization: පින්තූරවල ප්‍රමාණය ස්වයංක්‍රීයව අඩු කර වේගයෙන් ලෝඩ් කිරීමට විශේෂ <Image /> ටැගයක් මෙහි ඇත.
3. File-based Routing: App ෆෝල්ඩරය ඇතුළත ෆෝල්ඩරයක් සෑදූ සෑම විටම එය වෙබ් පිටුවක පාරක් (Route) බවට පත් වේ.
4. Full-stack Capability: ඔබට වෙබ් අඩවියේ පෙණුම (Frontend) සහ දත්ත ගබඩා කිරීම (Backend API) යන දෙකම Next.js ඇතුළතම කළ හැක.

90.6 ASCII රූපසටහන: CSR එදිරිව SSR

සාමාන්‍ය රියැක්ට් (CSR):
[User] --(Request)--> [Empty HTML] --(Downloads JS)--> [React Renders] --> [Content Visible]

නෙක්ස්ට් ජේ.එස් (SSR):
[User] --(Request)--> [Server Renders HTML] --(Sends Ready Page)--> [Content Visible]

90.7 වෘත්තීය පැහැදිලි කිරීම (Professional Explanation)

මෘදුකාංග ඉංජිනේරු විද්‍යාවේදී අපි Next.js හඳුන්වන්නේ "The SDK for the Web" ලෙසයි. එය සංවර්ධක අත්දැකීම (Developer Experience - DX) උපරිම කරන අතරම, අවසාන පරිශීලකයාට (End User) අතිශය වේගවත් මෘදුකාංගයක් ලබා දෙයි. නූතන ඉදිරිපෙළ සංවර්ධකයෙකුට (Frontend Developer) React පිළිබඳව පමණක් දැනුම තිබීම ප්‍රමාණවත් නැත; වෘත්තීය ලෝකය ඔහුගෙන් බලාපොරොත්තු වන්නේ Next.js පිළිබඳ ප්‍රවීණතාවයයි.

90.8 සාරාංශය (Key Points)

- Next.js යනු React මත ගොඩනැගූ බලවත් මෘදුකාංග රාමුවකි (Framework).
- එය SSR සහ SSG තාක්ෂණයන් හරහා වෙබ් අඩවියේ වේගය සහ SEO ඉතා ඉහළ නංවයි.
- අභ්‍යන්තරයේම පවතින රවුටින් සහ පින්තූර ප්‍රශස්තකරණය නිසා වැඩ කටයුතු පහසු වේ.
- මෙය Frontend සහ Backend යන දෙකම හැසිරවිය හැකි (Full-stack) රාමුවකි.

90.9 අභ්‍යාස (Exercises)

1. පුස්තකාලයක් (Library) සහ රාමුවක් (Framework) අතර ඇති ප්‍රධාන වෙනස්කම් ඔබේ වචනයෙන් පැහැදිලි කරන්න.
2. Client-Side Rendering වලට වඩා Server-Side Rendering උසස් වන්නේ ඇයි දැයි කරුණු 2ක් ලියන්න.
3. Next.js නිර්මාණය කළ සමාගමේ නම සහ එයට සහය දක්වන ප්‍රධාන ක්ලවුඩ් (Cloud) පද්ධතිය කුමක්දැයි අන්තර්ජාලයෙන් සොයා බලන්න.
4. ඔබ මේ දක්වා නිර්මාණය කළ රියැක්ට් ව්‍යාපෘතියක් Next.js වලට මාරු කළහොත් ලැබෙන ප්‍රධාන වාසි 3ක් නම් කරන්න.

අනු ඵක්වන පරිච්ඡේදය: පද්ධතිය සැකසීම සහ ඇප් රවුටර් ව්‍යුහය (Environment Setup and the App Router Architecture)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

1. Next.js ව්‍යාපෘතියක් ආරම්භ කිරීමට අවශ්‍ය පූර්ව අවශ්‍යතාවයන් හඳුනා ගැනීම.
2. create-next-app මෙවලම භාවිතා කර ස්වයංක්‍රීයව ව්‍යාපෘතියක් නිර්මාණය කරන ආකාරය.
3. ස්ථාපනය කිරීමේදී අසනු ලබන විවිධ තාක්ෂණික ප්‍රශ්න (Prompts) වල තේරුම් වටහා ගැනීම.
4. නූතන Next.js හි හඳවන බඳු App Router ව්‍යුහය සහ එහි ඇති ලෝල්ඩර් පද්ධතිය තේරුම් ගැනීම.
5. ප්‍රධාන ගොනු වර්ග වන page.js, layout.js, සහ globals.css වල කාර්යභාරය වටහා ගැනීම.
6. සංවර්ධන සේවාදායකය (Development Server) ක්‍රියාත්මක කර ප්‍රථම Next.js පිටුව නැරඹීම.

91.1 පූර්ව අවශ්‍යතාවයන් (Prerequisites)

Next.js ස්ථාපනය කිරීමට පෙර ඔබගේ පරිගණකයේ Node.js තිබීම අත්‍යවශ්‍ය වේ.

- අවම අනුවාදය: Node.js 18.17.0 හෝ ඊට වැඩි.
- ඔබ දැනටමත් පරිමාව 6 හිදී මෙය ස්ථාපනය කර ඇත. නැවත පරීක්ෂා කිරීමට ටර්මිනලයේ `node -v` ටයිප් කරන්න.

91.2 ව්‍යාපෘතිය ආරම්භ කිරීම (Creating a New Project)

Next.js ව්‍යාපෘතියක් සෑදීමට ඇති පහසුම සහ වෘත්තීයම ක්‍රමය වන්නේ create-next-app විධානය (Command) භාවිතා කිරීමයි.

පියවර 1: ඔබගේ ව්‍යාපෘති ගබඩා කරන ලෝල්ඩරය තුළ ටර්මිනලය (Terminal) විවෘත කරන්න.

පියවර 2: පහත විධානය ටයිප් කර Enter කරන්න:

```
npx create-next-app@latest my-next-app
```

91.3 ස්ථාපන සැකසුම් තේරුම් ගැනීම (Understanding Installation Prompts)

ඔබ ඉහත විධානය ලබා දුන් පසු, Next.js විසින් ඔබෙන් ප්‍රශ්න කිහිපයක් අසනු ඇත. වෘත්තීය මට්ටමේදී ඔබ තෝරාගත යුතු පිළිතුරු මෙන්න:

1. Would you like to use TypeScript? → Yes (අපි පරිමාව 8 හි TypeScript ඉගෙන ගත් නිසා මෙය භාවිතා කිරීම අත්‍යවශ්‍යයි).
2. Would you like to use ESLint? → Yes (කේතයේ වැරදි සෙවීමට මෙය උදව් වේ).
3. Would you like to use Tailwind CSS? → Yes (අපි පරිමාව 7 හි ඉගෙන ගත් Tailwind CSS මෙයට ඉතා හොඳින් ගැලපේ).
4. Would you like to use src/ directory? → Yes (කේතය පිළිවෙළකට තබා ගැනීමට මෙය උදව් වේ).
5. Would you like to use App Router? (recommended) → Yes (මෙය Next.js හි ඇති නවීනතම සහ බලවත්ම රවුටින් පද්ධතියයි).
6. Would you like to customize the default import alias (@/*)? → No (සාමාන්‍යයෙන් අපට මෙහි වෙනසක් අවශ්‍ය නොවේ).

දැන් තත්පර කිහිපයක් ධූරී සිටින්න. Next.js විසින් අන්තර්ජාලයෙන් අවශ්‍ය සියලුම පුස්තකාල බාගත කර ඔබගේ ව්‍යාපෘතිය සූදානම් කරනු ඇත.

91.4 ඇප් රවුටර් ව්‍යුහය (The App Router Architecture)

Next.js හි නවතම සංකල්පය වන්නේ App Router පද්ධතියයි. මෙහිදී අප රවුටින් (පිටු අතර ගමනාගමනය) පාලනය කරන්නේ ලෝල්ඩර් සැකසීම මගිනි.

VS Code මගින් ඔබ සෑදූ ලෝල්ඩරය විවෘත කර බලන්න. එහි src/app නමින් ලෝල්ඩරයක් ඇති බව ඔබට පෙනෙනු ඇත. මෙය අපේ මුළු මෘදුකාංගයේම පාලන මධ්‍යස්ථානයයි.

ප්‍රධාන ගොනු 3:

1. page.tsx (හෝ page.js): මෙය වෙබ් පිටුවක අන්තර්ගතය අඩංගු ගොනුවකි. app ලෝල්ඩරය තුළ ඇති page.tsx යනු ඔබේ මුල් පිටුවයි (Homepage).
2. layout.tsx (හෝ layout.js): මෙය වෙබ් අඩවියේ පොදු සැකිල්ලයි. (උදා: Navbar සහ Footer සියලුම පිටු වලට පොදුවේ පෙන්වීමට මෙය භාවිතා කරයි).

3. `globals.css`: මුළු වෙබ් අඩවියටම බලපාන පොදු CSS හැඩතල (Tailwind ද ඇතුළුව) මෙහි ඇත.

91.5 මෘදුකාංගය ක්‍රියාත්මක කිරීම (Running the App)

දැන් අපි අපේ පළමු Next.js ඇප් එක ක්‍රියාත්මක කරමු.

1. ටර්මිනලය හරහා ව්‍යාපෘති ෆෝල්ඩරය තුළට යන්න: `cd my-next-app`
2. සංවර්ධන සේවාදායකය (Development Server) පණ ගන්වන්න: `npm run dev`
3. දැන් ඔබගේ බ්‍රවුසරයේ `http://localhost:3000` යන ලිපිනයට යන්න. ඔබට Next.js සාදරයෙන් පිළිගැනීමේ පිටුව දැකගත හැකි වනු ඇත.

91.6 ASCII රූපසටහන: ෆෝල්ඩර ව්‍යුහය (Folder Tree)

```

my-next-app/
├── node_modules/  (බාහිර පුස්තකාල)
├── public/         (පින්තූර සහ ෆොන්ට් වර්ග)
├── src/
│   ├── app/       (ප්‍රධාන කේත ප්‍රදේශය)
│   │   ├── layout.tsx  (පොදු සැකිල්ල - Root Layout)
│   │   ├── page.tsx    (මුල් පිටුව - Homepage)
│   │   └── globals.css  (පොදු CSS)
│   ├── package.json  (ව්‍යාපෘති තොරතුරු)
│   ├── tsconfig.json (TypeScript සැකසුම්)
│   └── tailwind.config.ts (Tailwind සැකසුම්)

```

91.7 වෘත්තීය පැහැදිලි කිරීම (Professional Insight)

වෘත්තීය Next.js සංවර්ධකයෙකු සැමවිටම App Router භාවිතා කරයි. මෙහි ඇති විශේෂත්වය වන්නේ Server Components සංකල්පයයි. සාමාන්‍ය රියැක්ට් වලදී සියලුම කේත බ්‍රවුසරයේ ක්‍රියාත්මක වන විට, Next.js වලදී බොහෝ සංරචක සර්වර් එක තුළම ක්‍රියාත්මක වේ. මෙය වෙබ් අඩවියේ ආරක්ෂාව සහ වේගය අතිශය ඉහළ මට්ටමකට රැගෙන එයි.

91.8 පොදු වැරදි (Common Mistakes)

- `page.tsx` ගොනුවේ නම වැරදීම: Next.js හඳුනා ගන්නේ `page.tsx` යන නම පමණි. ඔබ එය

`index.tsx` හෝ `home.tsx` ලෙස නම් කළහොත්, එම පිටුව බ්‍රවුසරයේ දිස් නොවනු ඇත.

- Node.js අනුවාදය පැරණි වීම: පැරණි Node.js භාවිතා කළහොත් `create-next-app` විධානය ක්‍රියාත්මක කිරීමේදී දෝෂ (Errors) ඇති විය හැක.

91.9 සාරාංශය (Key Points)

- `npx create-next-app` මගින් ව්‍යාපෘතිය ආරම්භ කරයි.
- App Router යනු නූතන Next.js හි සම්මතයයි.
- `src/app/page.tsx` යනු වෙබ් අඩවියේ මුල් පිටුවයි.
- `layout.tsx` මගින් පොදු ව්‍යුහය පාලනය කරයි.
- `npm run dev` මගින් කේතය සජීවීව පරීක්ෂා කළ හැක.

91.10 අභ්‍යාස (Exercises)

1. ඔබේ පරිගණකයේ `next-masterclass` නමින් අලුත් Next.js ව්‍යාපෘතියක් සාදන්න.
2. `src/app/page.tsx` ගොනුව විවෘත කර, එහි ඇති සියලුම කේත මකා දමා, සරල "ආයුබෝවන් Next.js!" යන පණිවිඩය පෙන්වන සංරචකයක් (Component) ලියන්න.
3. `globals.css` ගොනුව විවෘත කර එහි අඩංගු වී ඇති Tailwind රේඛා 3 හඳුනා ගන්න.
4. Next.js ඇප් එකක් ක්‍රියාත්මක වන සාමාන්‍ය පෝට් (Port) අංකය කුමක්ද?

අනු දෙවන පරිච්ඡේදය: ගොනු පද්ධතිය මත පදනම් වූ රවුටින් (File-based Routing in Next.js)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අධ්‍යයනය කිරීමෙන් පසු ඔබට නෙක්ස්ට් ජේ.එස්.එස් හි පිටු පද්ධතිය කළමනාකරණය කිරීම පිළිබඳව අතිශය ගැඹුරු වෘත්තීය දැනුමක් ලැබෙනු ඇත:

1. File-based Routing (ගොනු පද්ධතිය මත පදනම් වූ රවුටින්) යනු කුමක්දැයි වටහා ගැනීම.
2. සාමාන්‍ය රියැක්ට් රවුටින් (React Router) සහ නෙක්ස්ට් ජේ.එස්.එස් රවුටින් අතර වෙනස හඳුනා ගැනීම.
3. App Router (`app` directory) ව්‍යුහය තුළ පිටු නිර්මාණය කරන ආකාරය.
4. Special Files (විශේෂ ගොනු) වන `page.tsx`, `layout.tsx`, `loading.tsx` සහ `error.tsx` වල කාර්යභාරය තේරුම් ගැනීම.

5. Nested Routes (අනුපිටු) සහ Dynamic Routes (ගතික පිටු) නිර්මාණය කිරීමේ ප්‍රෝ-මටටමේ ක්‍රමවේද.

6. නෙක්ස්ට් ජේ.එස් හි වේගවත් සංවලනය සඳහා <Link> සංරචකය භාවිතා කරන ආකාරය.

92.1 රවුටින් යනු කුමක්ද? (What is Routing?)

Routing (රවුටින්) යනු පරිශීලකයෙකු වෙබ් අඩවියක විවිධ ලිපිනය (URL) වලට යන විට (උදා: /about, /contact), ඔවුසරය විසින් අදාළ පිටුව පෙන්වීමේ ක්‍රියාවලියයි.

සාමාන්‍ය රියැක්ට් ව්‍යාපෘතියකදී අපට රවුටින් සඳහා react-router-dom වැනි බාහිර පුස්තකාල අවශ්‍ය වූ අතර, ඒ සඳහා අපට බොහෝ කේතයන් ලිවීමට සිදු විය. නමුත් Next.js වලදී රවුටින් සිදු වන්නේ ස්වයංක්‍රීයවය. ඔබ කරන්නේ ව්‍යාපෘතියේ ෆෝල්ඩර් (Folders) සෑදීම පමණි. එම ෆෝල්ඩර්වල නම් අනුව වෙබ් ලිපිනය (URL) ස්වයංක්‍රීයව නිර්මාණය වේ. මෙයට File-based Routing යැයි කියනු ලැබේ.

92.2 නෙක්ස්ට් ජේ.එස් App Router ව්‍යුහය

නවතම නෙක්ස්ට් ජේ.එස් සංස්කරණවල (Version 13+) රවුටින් සඳහා භාවිතා කරන්නේ app නමැති ෆෝල්ඩරයයි. මෙහි ඇති ප්‍රධාන නීති දෙකක් නිබේ:

1. Folder = Path: app ෆෝල්ඩරය ඇතුළත ඔබ සාදන සෑම ෆෝල්ඩරයක්ම වෙබ් අඩවියේ පාරක් (URL Path) බවට පත් වේ.
2. page.tsx හෝ page.js: යම් ලිපිනයක් වෙබ් අඩවියේ පිටුවක් ලෙස පෙන්වීමට නම්, අදාළ ෆෝල්ඩරය ඇතුළත අනිවාර්යයෙන්ම page.tsx නමින් ගොනුවක් තිබිය යුතුය.

ASCII රූපසටහන: ෆෝල්ඩර ව්‍යුහය සහ URL

```

app/
├── page.tsx      <-- මෙය මුල් පිටුවයි
(example.com/)
├── about/
│   └── page.tsx  <-- මෙය "About" පිටුවයි
(example.com/about)
└── contact/
    └── page.tsx  <-- මෙය "Contact" පිටුවයි
(example.com/contact)
  
```

92.3 විශේෂ ගොනු වර්ග (The Special Files)

නෙක්ස්ට් ජේ.එස් වලදී ෆෝල්ඩරයක් ඇතුළත තිබිය හැකි විශේෂ නම් සහිත ගොනු කිහිපයක් තිබේ. මේවා මෘදුකාංගයේ ක්‍රියාකාරිත්වය ස්වයංක්‍රීයව පාලනය කරයි.

1. page.tsx (පිටුව)

මෙය වෙබ් පිටුවේ අන්තර්ගතය පෙන්වන ප්‍රධාන සංරචකයයි (Component). මෙය නොමැතිව එම URL එක වැඩ කරන්නේ නැත.

2. layout.tsx (සැකිල්ල)

වෙබ් අඩවියේ පිටු කිහිපයකට පොදු වූ කොටස් (උදා: Navbar සහ Footer) මෙහි අඩංගු වේ. ඔබ යම් ෆෝල්ඩරයක Layout එකක් සෑදූ විට, එයට යටින් ඇති සියලුම අනුපිටු වලට එම සැකිල්ල ස්වයංක්‍රීයව ලැබෙනු ඇත.

3. loading.tsx (ප්‍රවේශන පණිවිඩය)

දත්ත සර්වර් එකෙන් ලැබෙන තෙක් පෙන්විය යුතු Loading Spinner එක හෝ "Loading..." පණිවිඩය මෙහි ලියයි. Next.js මෙය ඉතාමත් බුද්ධිමත් ලෙස ස්වයංක්‍රීයව පෙන්වනු ලබයි.

4. not-found.tsx (404 පිටුව)

යමෙකු වැරදි ලිපිනයක් ගැසූ විට පෙන්විය යුතු පිටුවයි.

92.4 අනුපිටු සැකසීම (Nested Routing)

ෆෝල්ඩරයක් ඇතුළත තවත් ෆෝල්ඩරයක් සෑදීම මගින් අපට අනුපිටු (Sub-pages) සෑදිය හැක.

උදාහරණයක්: ඔබට

example.com/dashboard/settings යන ලිපිනය අවශ්‍ය නම්:

1. app තුළ dashboard නමින් ෆෝල්ඩරයක් සාදන්න.
2. dashboard තුළ settings නමින් තවත් ෆෝල්ඩරයක් සාදන්න.
3. settings තුළ page.tsx ගොනුවක් සාදා එහි කේතය ලියන්න.

92.5 ගතික රවුටින් (Dynamic Routes)

බ්ලොග් අඩවියක ඇති ලිපි සිය ගණනක් සඳහා වෙන වෙනම ෆෝල්ඩර් සෑදීම අපහසුය. ඒ සඳහා අප Dynamic Routing භාවිතා කරමු. මෙහිදී ෆෝල්ඩරයේ නම සම්බන්ධ වරහන් [] ඇතුළත ලියනු ලැබේ.

උදාහරණයක්: ඔබට `example.com/blog/1`, `example.com/blog/2` වැනි ලිපි අවශ්‍ය නම්:

- ෆෝල්ඩරයේ නම: `app/blog/[id]/page.tsx`

කේතය ලබා ගන්නා ආකාරය:

```
// app/blog/[id]/page.tsx
```

```
export default function BlogPost({ params }: {
  params: { id: string } }) {
  return (
    <div>
      <h1>ලිපියේ අංකය: {params.id}</h1>
    </div>
  );
}
```

92.6 පිටු අතර ගමනය (Navigation with <Link>)

නෙක්ස්ට් ජේ.එස් වලදී පිටු මාරු කිරීමට සාමාන්‍ය `<a>` ටැගය භාවිතා නොකරයි. ඒ වෙනුවට අප භාවිතා කරන්නේ `next/link` වෙනත් ලැබෙන `<Link>` සංරචකයයි.

ඇයි මෙය භාවිතා කරන්නේ?

- No Page Refresh: පිටුව Refresh නොවී ඉතා වේගයෙන් පිටු මාරු වේ.
- Prefetching: ඔබ ලින්ක් එකක් දකින විටම, Next.js විසින් පසුබිමෙන් එම පිටුවට අදාළ දත්ත කලින්ම බාගත (Download) කර ගනී.

භාවිතා කරන ආකාරය:

```
import Link from 'next/link';

export default function Navbar() {
  return (
    <nav>
      <Link href="/">මුල</Link>
      <Link href="/about">මා ගැන</Link>
    </nav>
  );
}
```

92.7 වෘත්තීය පැහැදිලි කිරීම (Professional Explanation)

නෙක්ස්ට් ජේ.එස් හි App Router ව්‍යුහය පදනම් වී ඇත්තේ React Server Components (RSC) මතය. මෙහිදී රවුටින් සිදු වන්නේ සේවාදායකයේ (Server) සිටය. මෙලෙස ගොනු පද්ධතිය මත රවුටින් පදනම් වීම නිසා සංවර්ධකයාට කේතයේ සංකීර්ණත්වය පාලනය කිරීම ඉතා පහසු වන අතර, වෙබ් අඩවියේ SEO තත්ත්වය ද ඉතා ඉහළ මට්ටමක පවතී.

92.8 පොදු වැරදි (Common Mistakes)

1. ෆෝල්ඩරය ඇතුළත `page.tsx` අමතක වීම: ඔබ ෆෝල්ඩරයක් සෑදුවද එහි `page.tsx` නැතිනම්, එම URL එකට පිවිසෙන විට "404 Not Found" වැරද්ද ලැබේ.
2. `<a>` ටැගය භාවිතා කිරීම: රියැක්ට් රවුටින් වලදී මෙන් මෙහිදී ද `<a>` භාවිතා කළහොත් මුළු පිටුවම Refresh වී වෙබ් අඩවියේ වේගය අඩු වේ.
3. නම වැරදීම: විශේෂ ගොනු වල නම් (උදා: `layout.tsx`, `page.tsx`) අක්ෂර වින්‍යාසය වැරදුනහොත් නෙක්ස්ට් ජේ.එස් ඒවා හඳුනා නොගනී.

92.9 සාරාංශය (Key Points)

- නෙක්ස්ට් ජේ.එස් හි රවුටින් සිදු වන්නේ ෆෝල්ඩර් ව්‍යුහය අනුවය.
- `app` ෆෝල්ඩරය ප්‍රධාන රවුටරය ලෙස ක්‍රියා කරයි.
- `page.tsx` යනු පිටුවක් පෙන්වීමට අත්‍යවශ්‍යම ගොනුවයි.
- `layout.tsx` මගින් පිටු කිහිපයකට පොදු සැකිලි සාදයි.
- ගතික ලිපි සඳහා `[id]` වැනි වරහන් සහිත ෆෝල්ඩර් භාවිතා කරයි.
- වේගවත් සංවර්ධනය සඳහා `<Link>` භාවිතා කළ යුතුය.

92.10 අභ්‍යාස (Exercises)

1. ඔබේ ව්‍යාපෘතිය තුළ `services` සහ `services/web-design` යන පිටු දෙක නිර්මාණය කර ඒවා බ්‍රවුසරයෙන් පරීක්ෂා කරන්න.
2. සියලුම පිටු වලට පෙනෙන සේ ලස්සන Navigation Menu එකක් `layout.tsx` තුළ නිර්මාණය කරන්න.
3. Dynamic Route එකක් සාදා (උදා: `user/[name]`), බ්‍රවුසරයේ ලිපිනයට ඔබේ නම ඇතුළත් කළ විට එය පිටුවේ දිස්වන සේ සකසන්න.

4. loading.tsx ගොනුවක් සාදා, එහි සරල Loading ඇතිමේෂනයක් ඇතුළත් කර එය ක්‍රියා කරන ආකාරය නිරීක්ෂණය කරන්න.

අනු තුන්වන පරිච්ඡේදය: සර්වර් සහ ක්ලයන්ට් සංරචක - නවීන රෙන්ඩරින් කලාව (Server vs. Client Components)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අධ්‍යයනය කිරීමෙන් පසු ඔබට නෙක්ස්ට් ජේ.එස්.ඒ පවතින "සංරචක දර්ශනය" (Component Paradigm) පිළිබඳව අතිශය ගැඹුරු වෘත්තීය දැනුමක් ලැබෙනු ඇත:

1. React Server Components (RSC) හෙවත් සර්වර් සංරචක යනු මොනවාදැයි තේරුම් ගැනීම.
2. Client Components හෙවත් ක්ලයන්ට් සංරචකවල කාර්යභාරය හඳුනා ගැනීම.
3. Next.js හි සැම සංරචකයක්ම ස්වයංක්‍රීයව සර්වර් සංරචකයක් ලෙස ක්‍රියා කරන්නේ ඇයි දැයි වටහා ගැනීම.
4. "use client" විධානය (Directive) භාවිතා කළ යුතු නිවැරදි අවස්ථා හඳුනා ගැනීම.
5. සර්වර් සහ ක්ලයන්ට් සංරචක අතර පවතින තාක්ෂණික වෙනස්කම් සහ ඒවායේ වාසි (Performance/SEO) සංසන්දනය කිරීම.
6. සංරචක ගසක් (Component Tree) ඇතුළත මෙම දෙවර්ගයම එකිනෙක මුසු කරන ආකාරය ඉගෙන ගැනීම.

93.1 හැඳින්වීම: රෙන්ඩරින් විචල්‍යය

සාමාන්‍ය රියැක්ට් (Vite/CRA) වලදී සියලුම සංරචක ක්‍රියාත්මක වූයේ පරිශීලකයාගේ බ්‍රවුසරය (Client) තුළය. මෙහිදී බ්‍රවුසරයට විශාල JavaScript ගොනු ප්‍රමාණයක් බාගත කිරීමට සිදු විය. නමුත් Next.js 13 සමඟ හඳුන්වා දුන් App Router ව්‍යුහය මෙම ක්‍රමය සම්පූර්ණයෙන්ම වෙනස් කළේය.

දැන් අපට සංරචක ක්‍රියාත්මක කළ හැකි ස්ථාන දෙකක් තිබේ:

1. Server (සේවාදායකය): වෙබ් අඩවිය ගබඩා කර ඇති ප්‍රබල පරිගණකය තුළ.
2. Client (සේවාලාභියා): පරිශීලකයාගේ බ්‍රවුසරය (Chrome/Safari) තුළ.

වෘත්තීය මට්ටමේ Next.js සංවර්ධකයෙකු විමට නම්, ඔබ මූලිකවම තේරුම් ගත යුත්තේ කුමන සංරචකය සර්වර් එකේ තැබිය යුතුද සහ කුමන සංරචකය ක්ලයන්ට් එකේ තැබිය යුතුද යන්නයි.

93.2 සර්වර් සංරචක (Server Components - RSC)

නෙක්ස්ට් ජේ.එස්.ඒ හි app ෆෝල්ඩරය තුළ ඔබ සාදන සෑම සංරචකයක්ම ස්වයංක්‍රීයව (By default) Server Component එකක් වේ.

එය ක්‍රියා කරන්නේ මෙලෙසයි: සර්වර් සංරචකයක් ක්‍රියාත්මක වන්නේ සේවාදායකයේ (Server) පමණි. එය බ්‍රවුසරය වෙත කිසිදු JavaScript කේතයක් එවන්නේ නැත. ඒ වෙනුවට එය කරන්නේ කේතය කියවා සුදානම් කළ HTML පමණක් බ්‍රවුසරය වෙත එවීමේ ක්‍රියාවලියයි.

වාසි:

- Data Fetching: දත්ත ගබඩාවලට (Databases) සෘජුවම සම්බන්ධ වී ඉතා වේගයෙන් දත්ත ලබා ගත හැක.
- Security: API Keys වැනි රහස්‍ය දත්ත බ්‍රවුසරයට නොපෙනී ආරක්ෂිතව සර්වර් එක තුළ තබා ගත හැක.
- Performance: බ්‍රවුසරයට බාගත කිරීමට සිදුවන JavaScript ප්‍රමාණය ඉතා කුඩා වේ (Zero Bundle Size).
- SEO: සෙවුම් යන්ත්‍රවලට පිටුව කියවීමට ඉතා පහසුය.

93.3 ක්ලයන්ට් සංරචක (Client Components)

ක්ලයන්ට් සංරචක යනු පරිශීලකයාගේ බ්‍රවුසරය තුළ ක්‍රියාත්මක වන සංරචක වේ. රියැක්ට් හි අප ඉගෙන ගත් අන්තර්ක්‍රියාකාරී (Interactive) දේවල් සිදු කිරීමට නම් අප ක්ලයන්ට් සංරචක භාවිතා කළ යුතුය.

යම් සංරචකයක් ක්ලයන්ට් සංරචකයක් බවට පත් කිරීමට නම්, එම ගොනුවේ ඉහළින්ම "use client" යනුවෙන් සඳහන් කළ යුතුය.

ක්ලයන්ට් සංරචක අවශ්‍ය වන අවස්ථා:

- Interactivity: බොත්තම් ක්ලික් කිරීම් (onClick), පෙරලම දත්ත (onChange) හැසිරවීමට.

- Hooks භාවිතය: useState, useEffect, useContext වැනි හුක්ස් භාවිතා කිරීමට.
- Browser APIs: window, localStorage, document වැනි බ්‍රවුසරයටම ආවේණික දේවල් භාවිතා කිරීමට.

93.4 සැබෑ ලෝකයේ උපමාව (ආපනශාලා උදාහරණය)

- Server Component (කුස්සිය): ආහාරය (HTML) පිසින ස්ථානයයි. ඔබ පිසා එකක් ඇණවුම් කළ විට, කුස්සියේ සිටින්නන් සියලුම අමුද්‍රව්‍ය එකතු කර, පිළිස්සීම සිදු කරයි. ඔබට ලැබෙන්නේ සම්පූර්ණයෙන්ම සූදානම් වූ ආහාරයකි. කුස්සියේ ඇති සංකීර්ණ උපකරණ (JavaScript Logic) ඔබ දකින්නේ නැත.
- Client Component (මේසය): පිසා එක ලැබුණු පසු ඔබ එයට ලුණු හෝ සෝස් එකතු කරයි. මේවා පාරිභෝගිකයා වන ඔබ විසින් සිදු කරන "අන්තර්ක්‍රියාකාරී" (Interactions) වැඩ වේ.

93.5 කවදාද කුමක් භාවිතා කරන්නේ? (The Decision Matrix)

වෘත්තීය මට්ටමේදී ඔබ තීරණ ගත යුත්තේ මෙලෙසයි:

අවශ්‍යතාවය	Server Component	Client Component
දත්ත ලබා ගැනීම (Fetching Data)	✓ නිර්දේශිතයි	✗ භාවිතා නොකරන්න
Backend සම්පත්වලට සෘජුවම පිවිසීම	✓ ඔව්	✗ බැහැ
රහස්‍ය තොරතුරු (Tokens/Keys) තබා ගැනීම	✓ ඔව්	✗ බැහැ
Hooks (useState, useEffect) භාවිතය	✗ බැහැ	✓ අනිවාර්යයි
Browser APIs (window, etc.) භාවිතය	✗ බැහැ	✓ අනිවාර්යයි
onClick, onChange වැනි සිදුවීම් පාලනය	✗ බැහැ	✓ අනිවාර්යයි

93.6 ප්‍රායෝගික කේත උදාහරණය

අපි දැන් සර්වර් සංරචකයක් ඇතුළත ක්ලයන්ට් සංරචකයක් භාවිතා කරන ආකාරය බලමු.

ගොනුව: Counter.tsx (ක්ලයන්ට් සංරචකය)

"use client"; // මෙය ක්ලයන්ට් එකේ වැඩ කරන බව පවසයි

```
import { useState } from 'react';
```

```
export default function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count + 1)}>
      ඔබ මාව {count} වාරයක් එබුවා
    </button>
  );
}
```

```
</button>
);
}
```

ගොනුව: page.tsx (සර්වර් සංරචකය - මූලික පෙන්වන පිටුව)

```
// මෙහි "use client" නැති නිසා මෙය සර්වර් සංරචකයකි
import Counter from './Counter';
```

```
export default async function HomePage() {
  // සර්වර් එකේ සිටම දත්ත ලබා ගනිමු (Fetch data on server)
  const res = await
  fetch('https://api.example.com/data');
  const data = await res.json();
```

```
  return (
    <main>
      <h1>පිළිගැනීමේ පිටුව</h1>
      <p>සර්වර් එකෙන් ලැබුණු දත්ත: {data.message}</p>
    </main>
  );
}
```

```
/* ක්ලයන්ට් සංරචකය ඇතුළත් කිරීම */
<Counter />
</main>
);
}
```

93.7 හයිඩ්‍රේෂන් (Hydration) යනු කුමක්ද?

ක්ලයන්ට් සංරචකයක් සර්වර් එකේ සිට එවන විට මූලිකව එය "වියළි" HTML එකක් ලෙස පැමිණේ. ඉන්පසුව බ්‍රවුසරය විසින් එම සංරචකයට අවශ්‍ය JavaScript කේතය ලබා දී එය ක්‍රියාකාරී තත්ත්වයට පත් කරයි. මෙම ක්‍රියාවලියට Hydration (හයිඩ්‍රේෂන්) යැයි කියනු ලැබේ. (එය හරියට වියළි ශාකයකට වතුර දමා පණ ලබා දීම වැනිය).

93.8 ASCII රූපසටහන: සංරචක ගස (Composition)

```
[ Layout.tsx (Server) ]
|
-----
[ NavBar (Server) ] [ Page.tsx (Server) ]
|
-----
[ Text (Server) ] [ SearchBar (Client) ] <--- "use client"
```


වෘත්තීය රීතිය: හැකි සෑම විටම සංරචකය සර්වර් එකේ තබා ගන්න (Server-first approach).
අන්තර්ක්‍රියාකාරී බවක් (Interactivity) අවශ්‍ය වුවහොත් පමණක් එම කොටස පමණක් ක්ලයන්ට් සංරචකයක් බවට පත් කරන්න.

93.9 පොදු වැරදි (Common Mistakes)

1. සැම තැනම "use client" ලිවීම: මෙය ඉතාමත් තරක පුරුද්දකි. මෙය සිදු කළහොත් නෙක්ස්ට් ජේ.එස්.ඒ සැබෑ වාසිය (Speed/SEO) නැති වී සාමාන්‍ය රියැක්ට් මෘදුකාංගයක් බවට පත් වේ.
2. සර්වර් සංරචකයක් තුළ Hooks භාවිතා කිරීම: useState වැනි දෙයක් සර්වර් සංරචකයක ලිවුවොත් වහාම Error එකක් ලැබේ.
3. ක්ලයන්ට් සංරචකයක් ඇතුළත සර්වර් සංරචකයක් Import කිරීම: මෙය තාක්ෂණිකව කළ නොහැක. සැමවිටම සර්වර් සංරචක ඇත්තේ "පිටත" වන අතර ක්ලයන්ට් සංරචක ඇත්තේ "ඇතුළත" ය.

93.10 සාරාංශය (Key Points)

- Next.js හි සියලුම සංරචක සම්මතයෙන් (Default) සර්වර් සංරචක වේ.
- සර්වර් සංරචක වේගවත්ය, ආරක්ෂිතය සහ SEO සඳහා විශිෂ්ටය.
- අන්තර්ක්‍රියාකාරී බව සඳහා "use client" විධානය භාවිතා කළ යුතුය.
- හයිඩ්‍රේෂන් ක්‍රියාවලිය මගින් ක්ලයන්ට් සංරචක පණ ගන්වයි.
- හොඳම ක්‍රමය වන්නේ මූලිකව සර්වර් සංරචක භාවිතා කර පසුව අවශ්‍ය කොටස පමණක් ක්ලයන්ට් සංරචකයකට ගැනීමයි.

93.11 අභ්‍යාස (Exercises)

1. සර්වර් සංරචක සහ ක්ලයන්ට් සංරචක අතර ඇති ප්‍රධාන වෙනස්කම් 3ක් ලියන්න.
2. පහත අවස්ථා සඳහා ඔබ තෝරා ගන්නේ සර්වර් සංරචකයක්ද නැත්නම් ක්ලයන්ට් සංරචකයක්ද?
 - සර්වර් එකක ඇති දත්ත ගබඩාවකින් (Database) මිල ලැයිස්තුවක් ලබා ගැනීම.
 - සෙවුම් කොටුවක (Search Bar) අකුරු ටයිප් කරන විට ප්‍රතිඵල මාරු වීම.
 - වෙබ් අඩවියේ ඇති "පින්තූර ගැලරියක්" (Gallery) පෙන්වීම.
 - "Login" බොත්තම එබූ විට alert එකක් පෙන්වීම.
3. ඔබේ ව්‍යාපෘතියේ page.tsx ගොනුවක් සාදා, එහි useState භාවිතා කිරීමට උත්සාහ කරන්න. ලැබෙන Error එක කියවා බලන්න.

අනු භතරවන පරිච්ඡේදය: උසස් සැකිලි සහ ටෙම්ප්ලේට්ස් - වෙබ් අඩවියේ ව්‍යුහය සැලසුම් කිරීම (Advanced Layouts and Templating)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අධ්‍යයනය කිරීමෙන් පසු ඔබට නෙක්ස්ට් ජේ.එස්.ඒ වෘත්තීය මට්ටමේ පිරිසැලසුම් (Layouts) නිර්මාණය කිරීම පිළිබඳව අතිශය ගැඹුරු දැනුමක් ලැබෙනු ඇත:

1. Root Layout (මූලික සැකිල්ල) යනු කුමක්ද සහ එහි ඇති අනිවාර්ය ටැග් හඳුනා ගැනීම.
2. Nested Layouts (අනු-සැකිලි) භාවිතා කර වෙබ් අඩවියේ විවිධ කොටස්වලට විවිධ සැකසුම් ලබා දෙන ආකාරය.
3. layout.tsx සහ template.tsx අතර ඇති සියුම් තාක්ෂණික වෙනස වටහා ගැනීම.
4. සංරචක ගස හරහා children ප්‍රෝපර්ටි එක ගලා යන ආකාරය තේරුම් ගැනීම.
5. Route Groups (folder) භාවිතා කර URL එකට බලපෑමක් නොකර ගොනු සංවිධානය කරන ආකාරය.
6. Metadata API භාවිතා කර සෑම පිටුවකටම අනන්‍ය වූ සෙවුම් යන්ත්‍ර ප්‍රශස්තකරණ (SEO) සැකසුම් ලබා දීම.

94.1 සැකිල්ලක් (Layout) යනු කුමක්ද?

වෙබ් අඩවියක පිටු කිහිපයක් හරහා පොදුවේ පවතින කොටස් (උදා: Navigation Bar, Footer, Sidebar) සෑම පිටුවකම නැවත නැවත ලිවීම අපහසුය. Layout යනු මෙම පොදු කොටස් එක් වරක් ලියා, ඒ ඇතුළත විවිධ පිටු ප්‍රදර්ශනය කිරීමට භාවිතා කරන "රාමුව" වැනිය.

සැබෑ ලෝකයේ උපමාව (ඡායාරූප රාමුවක්): සිතන්න ඔබ සතුව ලස්සන ලී රාමුවක් (Layout) තිබෙනවා කියා. ඔබට පුළුවන් එම රාමුව ඇතුළත ඇති පින්තූරය (Page Content) ඕනෑම වෙලාවක මාරු කරන්න. නමුත් පිටත ඇති රාමුව එලෙසම පවතී. නෙක්ස්ට් ජේ.එස්.ඒ Layouts ක්‍රියා කරන්නේ ද මෙලෙසමය.

94.2 මූලික සැකිල්ල (Root Layout)

නෙක්ස්ට් ජේ.එස් ව්‍යාපෘතියක app ෆෝල්ඩරය තුළ අනිවාර්යයෙන්ම තිබිය යුතු ගොනුව වන්නේ app/layout.tsx ය. මෙය මුළු වෙබ් අඩවියටම බලපාන ප්‍රධාන රාමුවයි.

විශේෂත්වය:

- මෙය අනිවාර්යයෙන්ම Server Component එකක් විය යුතුය.
- මෙය ඇතුළත අනිවාර්යයෙන්ම <html> සහ <body> යන ටැග් තිබිය යුතුය.
- අනෙකුත් සියලුම පිටු (page.tsx) මෙහි ඇති children නමැති කොටස තුළ ප්‍රදර්ශනය වේ.

කේතය (Basic Root Layout):

// app/layout.tsx

```
export default function RootLayout({
  children,
}): {
  children: React.ReactNode;
}) {
  return (
    <html lang="si">
      <body>
        <nav>මගේ මෙනුව</nav>

        {/* මෙතැනට තමයි page.tsx එකේ දත්ත
        පැමිණෙන්නේ */}
        <main>{children}</main>

        <footer>මගේ පාදය</footer>
      </body>
    </html>
  );
}
```

94.3 අනු-සැකිලි (Nested Layouts)

නෙක්ස්ට් ජේ.එස් හි ඇති විශේෂත්වය වන්නේ අපට එක් එක් ෆෝල්ඩරය (Route) ඇතුළත වෙන වෙනම Layouts සෑදීමට හැකි වීමයි. මෙම අනු-සැකිලි ඒවායේ මව්-සැකිල්ල (Parent Layout) ඇතුළත ස්වයංක්‍රීයව අසුරනු ලැබේ (Wrap).

උදාහරණයක්: ඔබේ වෙබ් අඩවියේ "Dashboard" කොටසට පමණක් පෙනෙන Sidebar එකක් අවශ්‍ය නම්:

- app/layout.tsx (මුළු අඩවියටම)
- app/dashboard/layout.tsx (ඩැෂ්බෝර්ඩ් එකට පමණයි)

ASCII රූපසටහන: Layout Nesting

[Root Layout (Navbar/Footer)]

```

|
└── [ Dashboard Layout (Sidebar) ]
    |
    └── [ Dashboard Page Content ]
  
```

94.4 ලේඛන (Layout) එදිරිව ටෙම්ප්ලේට් (Template)

නෙක්ස්ට් ජේ.එස් හි layout.tsx වලට අමතරව template.tsx නමින් ගොනුවක් ද තිබිය හැක. මේවා දෙකම එක හා සමාන ලෙස පෙනුනත් ක්‍රියා කරන ආකාරය වෙනස්ය.

1. Layout (layout.tsx): ඔබ පිටු අතර ගමන් කරන විට Layout එකේ ඇති ස්ටේට් (State) මැකී යන්නේ නැත. එය නොවෙනස්ව පවතී (Persistent). වේගය වැඩිය.
2. Template (template.tsx): ඔබ පිටු අතර ගමන් කරන විට සෑම වතාවකම මෙය අලුතින්ම නිර්මාණය වේ (Re-mounts). මෙහි ස්ටේට් මතක තබා ගන්නේ නැත.

වෘත්තීය රීතිය: 99% ක්ම භාවිතා කරන්නේ layout.tsx ය. යම් පිටුවක Entry Animation එකක් (පිටුවට එන විට සිදුවන ඇතිමේෂනයක්) සෑම පිටු මාරුවකදීම අවශ්‍ය නම් පමණක් template.tsx භාවිතා කරන්න.

94.5 රවුට් ගෘහප්ප් (Route Groups - (folder))

සමහර විට අපට ව්‍යාපෘතියේ ගොනු පිළිවෙළකට තැබීමට ෆෝල්ඩර අවශ්‍ය වේ, නමුත් එම ෆෝල්ඩරයේ නම වෙබ් ලිපිනයට (URL) ඇතුළත් වීමට අප අකමැති විය හැක. ඒ සඳහා ෆෝල්ඩරයේ නම වරහන් ඇතුළත (name) ලියනු ලැබේ.

ප්‍රයෝජනය: ඔබට ලොගින් පිටු (Login/Register) සඳහා වෙනම Layout එකක් අවශ්‍ය නම්:

- ෆෝල්ඩරය: app/(auth)/layout.tsx
- පිටුව 1: app/(auth)/login/page.tsx (ලිපිනය: /login)

4. Revalidation (යාවත්කාලීන කිරීම): පැරණි දත්ත ඉවත් කර අලුත් දත්ත ලබා ගන්නා කාල පරාසයන් තීරණය කරන ආකාරය.

5. Static vs. Dynamic Fetching: ස්ථාවර දත්ත සහ නිතර වෙනස් වන දත්ත හැසිරවිය යුතු නිවැරදි ක්‍රමවේද හඳුනා ගැනීම.

6. Error Handling & Suspense: දත්ත ලැබෙන තෙක් සහ වැරදි සිදු වන විට ඒවා කළමනාකරණය කරන ආකාරය.

95.1 හැඳින්වීම: දත්ත ලබා ගැනීමේ නව පරම්පරාව

අපි පරිමාව 6, පරිච්ඡේදය 65 හිදී ඉගෙන ගන්නා රියැක්ට් වලදී දත්ත ලබා ගැනීමට useEffect සහ fetch භාවිතා කරන ආකාරය. එහිදී සිදු වූයේ පරිශීලකයාගේ බ්‍රවුසරය (Client) විසින් දත්ත ඉල්ලා සිටීමයි.

නමුත් Next.js App Router තුළ, අපට දත්ත ලබා ගැනීමේ හැකියාව සේවාදායකය (Server) වෙත රැගෙන ගොස් ඇත. මෙහිදී සංරචක ශ්‍රිතයම අසමකාලීන ශ්‍රිතයක් (async function) බවට පත් කළ හැකි අතර, ඒ ඇතුළත සාප්‍රචම await fetch() භාවිතා කළ හැක.

සැබෑ ලෝකයේ උපමාව (ළිඳ සහ නළ ජලය):

- පැරණි රියැක්ට් ක්‍රමය (Client-side): ඔබේ නිවසට (Browser) වතුර නැත. ඔබට වතුර අවශ්‍ය වූ විට ඔබම බාල්දියක් ගෙන ළිඳ (Server API) වෙත ගොස් වතුර රැගෙන ආ යුතුය. මෙයට කාලය සහ වෙහෙස වැය වේ.

- නෙක්ස්ට් ජේ.එස්.එස් ක්‍රමය (Server-side): නිවසටම නළ ජලය ලැබෙනවා වැනිය. ඔබ කරාමය හරවන විටම වතුර සුදානම්ව ඇත. සේවාදායකය විසින්ම වතුර (Data) ලබාගෙන, පිරිසිදු කර (Process), සුදානම් කළ ජලය පමණක් ඔබ වෙත එවනු ලබයි.

95.2 සර්වර් සංරචක තුළ දත්ත ලබා ගැනීම (Fetching in Server Components)

නෙක්ස්ට් ජේ.එස්.එස් හි සර්වර් සංරචක ඇතුළත දත්ත ලබා ගැනීම අතිශය සරල සහ පිරිසිදු ක්‍රියාවලියකි. අපට useState හෝ useEffect අවශ්‍ය නොවේ.

ප්‍රායෝගික කේතය:

```
// app/posts/page.tsx
```

```
async function getPosts() {
  const res = await
  fetch('https://api.example.com/posts');

  if (!res.ok) {
    // වැරද්දක් වූණොත් ළඟම ඇති error.tsx
    ගොනුව ක්‍රියාත්මක වේ
    throw new Error('දත්ත ලබා ගැනීමට නොහැකි
    විය');
  }
}
```

```
return res.json();
}
```

```
export default async function PostsPage() {
  // සාප්‍රචම ශ්‍රිතය කැඳවා දත්ත ලබා ගනිමු
  const posts = await getPosts();
```

```
  return (
    <main>
      <h1>අලුත්ම පුවත්</h1>
      <ul>
        {posts.map((post: any) => (
          <li key={post.id}>{post.title}</li>
        ))}
      </ul>
    </main>
  );
}
```

95.3 කැෂින් (Caching) යනු කුමක්ද?

නෙක්ස්ට් ජේ.එස්.එස් හි ඇති බලවත්ම විශේෂාංගය වන්නේ Caching (කැෂින්) හෙවත් දත්ත මතකයේ තබා ගැනීමයි.

සාමාන්‍යයෙන් ඔබ fetch විධානය භාවිතා කරන විට, Next.js විසින් එම දත්ත සර්වර් එකේ විශේෂ මතකයක (Data Cache) තැන්පත් කරයි. ඊළඟ වතාවේ කවුරුන් හෝ එම පිටුවටම පිවිසෙන විට, Next.js නැවතත් API එකට පණිවිඩ යවන්නේ නැත. ඒ වෙනුවට මතකයේ ඇති පැරණි දත්තය ක්ෂණිකව පෙන්වයි.

වාසි:

- Speed: වෙබ් අඩවිය අතිශය වේගවත් වේ.
- Cost: බාහිර API වලට යන ඉල්ලීම් (Requests) ගණන අඩු වන නිසා මුදල් ඉතිරි වේ.
- Reliability: බාහිර API එක තාවකාලිකව ක්‍රියා විරහිත වුවද, මතකයේ ඇති දත්ත පෙන්වීමට හැකි වේ.

95.4 දත්ත යාවත්කාලීන කිරීම (Data Revalidation)

දත්ත සැමවිටම මතකයේ තිබීම ගැටලුවකි. මන්දයත් සැබෑ දත්ත වෙනස් වූ විට වෙබ් අඩවියේ පරණ දත්තම පෙනෙන බැවිනි. මෙය පාලනය කිරීමට ක්‍රම දෙකක් ඇත.

1. කාලය මත පදනම් වූ යාවත්කාලීන කිරීම (Time-based Revalidation)

යම් කාලයකට පසු (උදා: තත්පර 60කට පසු) දත්ත ස්වයංක්‍රීයව අලුත් කරන ලෙස පවසා සිටීමයි.

```
fetch('https://api.example.com/data', {
  next: { revalidate: 60 } // තත්පර 60කට වරක්
  අලුත් කරන්න
});
```

2. ඉල්ලීම මත යාවත්කාලීන කිරීම (On-demand Revalidation)

යම් සිදුවීමක් වූ විට (උදා: අලුත් පෝස්ට් එකක් සේව කළ විට) පමණක් දත්ත අලුත් කරන ලෙස පැවසීමයි. (මෙය `revalidateTag` හරහා සිදු කරයි).

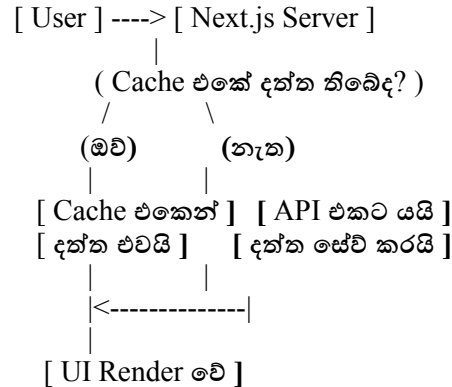
95.5 ස්ථාවර සහ ගතික දත්ත (Force Cache vs. No Store)

ඔබට අවශ්‍ය නම් කැමින් ක්‍රියාවලිය පාලනය කළ හැක.

- Force Cache (Default): දත්ත සැමවිටම මතකයේ තබා ගනී. (SSG සඳහා සුදුසුයි).
- No Store: දත්ත කිසිවිටකත් මතකයේ තබා නොගනී. සැම වතාවකම අලුත් දත්ත ලබා ගනී. (SSR සඳහා සුදුසුයි).

```
// සැම වතාවකම අලුත් දත්ත අවශ්‍ය නම් (උදා:
තොග ප්‍රමාණයන්)
fetch('https://api.example.com/stock', { cache:
'no-store' });
```

95.6 ASCII රූපසටහන: Data Cache ක්‍රියාවලිය



95.7 වෘත්තීය පැහැදිලි කිරීම (Professional Explanation)

මෘදුකාංග ඉංජිනේරු විද්‍යාවේදී මෙය "Memoization at the Network Level" ලෙස හඳුන්වයි. නෙක්ස්ට් ජේ.එස්.වීසින් පවතින `fetch` API එක පුළුල් කර (Extended) එයට කැමින් සහ රිවැලියුඩේෂන් හැකියාවන් එක් කර ඇත. වෘත්තීය මට්ටමේදී අප සැමවිටම උත්සාහ කරන්නේ හැකි තරම් දත්ත Cache කර තබා ගැනීමටයි. එමගින් වෙබ් අඩවියේ TTFB (Time to First Byte) අගය ඉතා පහළ මට්ටමක තබා ගත හැක.

95.8 පොදු වැරදි (Common Mistakes)

1. Client Component එකක Fetch කිරීම: use client භාවිතා කර ඇති සංරචකයක `async/await` භාවිතා කර සෘජුවම Fetch කිරීමට උත්සාහ කිරීම. (එය කළ හැක්කේ සර්වර් සංරචකවල පමණි).
2. සංවේදී දත්ත Cache කිරීම: පරිශීලකයාගේ පුද්ගලික තොරතුරු (උදා: බැංකු විස්තර) `force-cache` කිරීම ඉතා භයානක වැරද්දකි. එවැනි දත්ත සඳහා සැමවිටම `no-store` භාවිතා කරන්න.
3. Error Handling අමතක වීම: සර්වර් එකෙන් වැරදි පිළිතුරක් ආ විට (උදා: 500 Error) එය පරීක්ෂා නොකිරීම.

95.9 සාරාංශය (Key Points)

- Next.js හි දත්ත ලබා ගැනීම සර්වර් මට්ටමේදී (Server Components) සිදු වේ.
- සංරචක ශ්‍රිතයම `async` ලෙස ලිවිය හැක.
- `fetch` විධානය ස්වයංක්‍රීයව දත්ත මතකයේ තබා ගනී (Caching).

- revalidate අගය මගින් දත්ත අලුත් වන කාලය තීරණය කරයි.
- no-store මගින් සැම විටම සජීවී (Live) දත්ත ලබා ගැනීම සහතික කරයි.

95.10 අභ්‍යාස (Exercises)

1. JSONPlaceholder API එක භාවිතා කර බ්ලොග් ලිපි 10ක් ලබා ගෙන ඒවා පෙන්වන සර්වර් සංරචකයක් සාදන්න.
2. එම ලිපි ලැයිස්තුව සැම තත්පර 30කට වරක් අලුත් වන ලෙස (Revalidate) සකසන්න.
3. කාලගුණ දත්ත පෙන්වන පිටුවක් සඳහා ඔබ තෝරා ගන්නේ force-cache ද නැත්නම් no-store ද? හේතුව පැහැදිලි කරන්න.
4. වැරදි URL එකක් ලබා දී throw new Error පේළිය ක්‍රියාත්මක වන ආකාරය සහ බ්‍රවුසරය එයට ප්‍රතිචාර දක්වන ආකාරය නිරීක්ෂණය කරන්න.

අනු භයවන පරිච්ඡේදය: දත්ත ප්‍රවාහය සහ සස්පෙන්ස් - පරිශීලකයාට ක්ෂණික ප්‍රතිචාර ලබා දීම (Streaming and Suspense)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අධ්‍යයනය කිරීමෙන් පසු ඔබට නෙක්ස්ට් ජේ.එස්.එස්.ඒ අති නවීන පිරිසැලසුම් තාක්ෂණයන් පිළිබඳව අතිශය ගැඹුරු වෘත්තීය දැනුමක් ලැබෙනු ඇත:

1. Streaming (දත්ත ප්‍රවාහය) යනු කුමක්ද සහ එය වෙබ් අඩවියක වේගය කෙරෙහි බලපාන ආකාරය තේරුම් ගැනීම.
2. මුළු පිටුවම ලෝඩ් වන තෙක් බලා නොසිට කොටස් වශයෙන් පෙන්වීමේ (Progressive Rendering) වැදගත්කම වටහා ගැනීම.
3. loading.tsx ගොනුව භාවිතා කරමින් ස්වයංක්‍රීයව ප්‍රවේශන තීර (Loading States) නිර්මාණය කිරීම.
4. React Suspense භාවිතා කර නිශ්චිත සංරචකයක් (Component) සඳහා පමණක් ප්‍රවේශන පණිවිඩ ලබා දෙන ආකාරය.
5. Skeleton Screens (සැකිලි තිර) යනු මොනවාද සහ ඒවා නිර්මාණය කරන ආකාරය ඉගෙන ගැනීම.
6. පරිශීලක අත්දැකීම (UX) වැඩි දියුණු කිරීම සඳහා "බාධාවකින් තොර සංචලනය" (Instant Loading States) පාලනය කිරීම.

96.1 හැඳින්වීම: දත්ත ලැබෙන තෙක් බලා සිටීමේ ගැටලුව

පෙර පරිච්ඡේදයේදී අප ඉගෙන ගත්තේ සර්වර් සංරචක (Server Components) තුළ await fetch() භාවිතා කර දත්ත ලබා ගන්නා ආකාරයයි. නමුත් මෙහිදී එක් ප්‍රධාන ගැටලුවක් මතු විය හැක.

සිතන්න ඔබ දත්ත ලබා ගන්නා API එක ඉතාමත් මන්දගාමී බව. එවිට සර්වර් එක එම දත්ත ලැබෙන තෙක් await කරමින් සිටියි. දත්ත ලැබී අවසන් වන තෙක් බ්‍රවුසරයට කිසිදු HTML එකක් ලැබෙන්නේ නැත. එවිට පරිශීලකයාට පෙනෙන්නේ සුදු පැහැති හිස් තිරයක් පමණි. මෙය ඉතා නරක පරිශීලක අත්දැකීමකි.

Next.js මෙම ගැටලුව විසඳන්නේ Streaming නමැති තාක්ෂණය හරහාය.

96.2 දත්ත ප්‍රවාහය (Streaming) යනු කුමක්ද?

Streaming (ස්ට්‍රීම්) යනු වෙබ් පිටුවක HTML කේතය කුඩා කැබලි (Chunks) වලට කඩා, ඒවා සුදානම් වූ සැණින් බ්‍රවුසරය වෙත එවන ක්‍රමයයි.

මෙහිදී දත්ත වැඩිපුර අවශ්‍ය නොවන කොටස් (උදා: Navbar, Sidebar) වහාම බ්‍රවුසරයට යවනු ලබයි. දත්ත ලැබීමට කාලය ගතවන කොටස් (උදා: Product List) සඳහා කුඩා පණිවිඩයක් (Loading Spinner) පෙන්වා පසුව දත්ත ලැබුණු විට එම කොටස පමණක් යාවත්කාලීන කරයි.

සැබෑ ලෝකයේ උපමාව (ආපනශාලා සේවාව):

- සාමාන්‍ය ක්‍රමය (No Streaming): ඔබ ඇණවුම් කළ සියලුම ආහාර (සුප්, ප්‍රධාන කෑම, අතුරුපස) සුදානම් වන තෙක් වේටර්වරයා කිසිවක් ගෙන එන්නේ නැත. ඔබ බොහෝ වෙලාවක් කුසගින්නේ බලා සිටිය යුතුය.

- ස්ට්‍රීම් ක්‍රමය: වේටර්වරයා මුලින්ම වතුර වීදුරුව ගෙන එයි. ඉන්පසු සුප් එක ගෙන එයි. අවසානයේදී ප්‍රධාන කෑම ගෙන එයි. මෙහිදී ඔබට එක් එක් කොටස ලැබෙන නිසා බලා සිටීම අපහසු නොවේ.

96.3 ක්‍රමය 1: loading.tsx භාවිතා කිරීම (Route-level Streaming)

තෙක්ස්ට් ජේ.එස්.ඒ ඇති පහසුම ක්‍රමය මෙයයි. ඔබ යම් රවුට් (Route) ලෝලේඩරයක් තුළ loading.tsx නමින් ගොනුවක් සෑදූ විට, එම ලෝලේඩරයේ ඇති page.tsx ගොනුවට දත්ත ලැබෙනතෙක් Next.js ස්වයංක්‍රීයව මෙම Loading පිටුව පෙන්වයි.

ප්‍රායෝගික උදාහරණය: app/dashboard/loading.tsx

```
export default function Loading() {
  return <p>Dashboard දත්ත ගණනය කරමින් පවතී. කරුණාකර රැඳී සිටින්න...</p>
}
```

එය ක්‍රියා කරන ආකාරය: Next.js විසින් ස්වයංක්‍රීයව ඔබගේ page.tsx එක වටා React Suspense රාමුවක් නිර්මාණය කරයි. දත්ත ලැබුණු සැණින් Loading පණිවිඩය ඉවත් වී සැබෑ පිටුව දිස්වේ.

96.4 ක්‍රමය 2: React Suspense භාවිතා කිරීම (Component-level Streaming)

සමහර විට අපට මුළු පිටුවම Loading කර පෙන්වීම අවශ්‍ය නොවේ. අපට අවශ්‍ය වන්නේ පිටුවේ එක් විශේෂිත කොටසක් (උදා: Comment section) පමණක් දත්ත ලැබෙන තෙක් පාලනය කිරීමටයි. ඒ සඳහා අප <Suspense> සංරචකය භාවිතා කරමු.

ප්‍රායෝගික කේතය:

```
import { Suspense } from 'react';
import SlowComponent from './SlowComponent';
import FastComponent from './FastComponent';

export default function Page() {
  return (
    <main>
      <h1>මගේ වෙබ් පිටුව</h1>

      { /* මෙම කොටස වහාම දිස්වේ */ }
      <FastComponent />

      { /* දත්ත ලැබීමට කාලය ගතවන කොටස Suspense එකකින් වට කරමු */ }
      <Suspense fallback={<p>විස්තර ලෝඩ් වෙමින් පවතී...</p>}>
        <SlowComponent />
      </Suspense>
    </main>
  );
}
```

විස්තරය: මෙහිදී FastComponent එක වහාම නිරයේ දිස්වේ. SlowComponent එක ඇතුළත ඇති fetch විධානය අවසන් වන තෙක් fallback එකේ ඇති පණිවිඩය පෙන්වේ.

96.5 සැකිලි තිර (Skeleton Screens)

වෘත්තීය මට්ටමේ වෙබ් අඩවිවල "Loading..." යනුවෙන් අකුරු පෙන්වනවා වෙනුවට, සැබෑ දත්තවල හැඩය ගත් අළු පැහැති පෙට්ටි (Gradients) පෙන්වනු ලබයි. මේවා Skeleton Screens ලෙස හැඳින්වේ.

මෙය පරිශීලකයාට හැඟවීමට උදව් කරන්නේ දත්ත ඉතා ඉක්මනින් ලැබෙන බවත්, පිටුවේ සැකැස්ම කෙටිදුරු යන්නත් ය.

96.6 ASCII රූපසටහන: Loading ක්‍රියාවලිය

සාමාන්‍ය රියැක්ට් (Sequential):
[Navbar] -> [Sidebar] -> [Wait for Data..] -> [Content]
(පරිශීලකයාට කිසිවක් නොපෙන්වේ)

තෙක්ස්ට් ජේ.එස් (Streaming with Suspense):
[Navbar] (Visible)
[Sidebar] (Visible)
[Skeleton/Spinner] (Visible) <---- Suspense Fallback
|
v (Data ready)
[Real Content] (Visible)

96.7 වෘත්තීය පැහැදිලි කිරීම (Professional Explanation)

මෘදුකාංග ඉංජිනේරු විද්‍යාවේදී මෙය "Time to First Contentful Paint (TFCP)" සහ "Time to Interactive (TTI)" අගයන් වැඩි දියුණු කිරීමේ ක්‍රමයකි. Streaming භාවිතා කිරීමෙන් සේවාදායකයේ (Server) දත්ත සැකසෙන අතරතුරම බ්‍රවුසරය (Client) විසින් ලැබුණු HTML කොටස් ඇඳීම (Parsing) ආරම්භ කරයි. මෙය මහා පරිමාණ දත්ත පද්ධති සඳහා අත්‍යවශ්‍යම උපාය මාර්ගයකි.

96.8 පොදු වැරදි (Common Mistakes)

- අනවශ්‍ය ලෙස සැම තැනම Suspense දැමීම: ඕනෑවට වඩා Loading Spinners පිටුව පුරා තිබීම පරිශීලකයා ව්‍යාකූල කරයි. වැදගත් විශාල කොටස් සඳහා පමණක් මෙය භාවිතා කරන්න.
- Fallback එක හිස්ව තැබීම: fallback එකක් ලබා නොදුන්නොත්, Next.js විසින් සම්මතයෙන් මුළු රවුට් එකම ලෝඩ් වන තෙක් බලා සිටිනු ඇත.

3. Data Fetching වැරදි තැනක කිරීම: await විධානය page.tsx හි මුද්‍රණේම තිබේ නම්, එය මුළු පිටුවම බ්ලොක් (Block) කරයි. එම await එක අදාළ සංරචකය ඇතුළතට ගෙන යාම (Component-level fetching) Streaming සඳහා අත්‍යවශ්‍ය වේ.

96.9 සාරාංශය (Key Points)

- Streaming මගින් පිටුව කොටස් වශයෙන් ඔවුසරයට එවයි.
- loading.tsx මගින් මුළු පිටුවටම පොදු ප්‍රවේශන තීරයක් සාදයි.
- <Suspense> මගින් නිශ්චිත කොටස් වල දත්ත ප්‍රමාදය කළමනාකරණය කරයි.
- Fallback මගින් දත්ත ලැබෙන තෙක් පෙන්විය යුතු දේ පාලනය කරයි.
- මෙය වෙබ් අඩවියේ වේගය සහ UX ඉතා ඉහළ නංවයි.

96.10 අභ්‍යාස (Exercises)

1. ඔබේ ව්‍යාපෘතියේ dashboard රටුව එක සඳහා loading.tsx ගොනුවක් සාදා එහි ලස්සන සැකැස්මක් (UI) නිර්මාණය කරන්න.
2. එකම පිටුවක සංරචක දෙකක් සාදන්න. එකක් තත්පර 2කින් සහ අනෙක තත්පර 5කින් දත්ත ලබා ගත යුතුය. <Suspense> භාවිතා කර මේවා වෙනම ලෝඩ් වන ආකාරය පරීක්ෂා කරන්න. (ඉභිය: await new Promise(r => setTimeout(r, 2000))) භාවිතා කර පමාව ඇති කරන්න).
3. Skeleton Screen එකක් සාදා එය fallback අගය ලෙස ලබා දෙන්න.

අනු හත්වන පරිච්ඡේදය: පින්තූර ප්‍රශස්තකරණය සහ අකුරු පාලනය - කාර්යක්ෂමතාවයේ රහස් (Image Optimization and Fonts in Next.js)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අධ්‍යයනය කිරීමෙන් පසු ඔබට වෙබ් අඩවියක වේගය සහ ගුණාත්මකභාවය තීරණය කරන ප්‍රධාන සාධක දෙකක් පිළිබඳව අතිශය ගැඹුරු වෘත්තීය දැනුමක් ලැබෙනු ඇත:

1. සාමාන්‍ය HTML ටැගය වෙනුවට නෙක්ස්ට් ජේ.එස් හි <Image /> සංරචකය භාවිතා කළ යුතු හේතු වටහා ගැනීම.

2. Cumulative Layout Shift (CLS) හෙවත් පිටුව ලෝඩ් වන විට කොටස් එහා මෙහා පැනීම වැළැක්වීමට නෙක්ස්ට් ජේ.එස් උදව් වන ආකාරය තේරුම් ගැනීම.

3. පින්තූර ස්වයංක්‍රීයව WebP වැනි නවීන වර්ගවලට හරවන සහ ප්‍රමාණය අඩු කරන (Resizing) ආකාරය ඉගෙන ගැනීම.

4. Lazy Loading සහ Priority ගුණාංග භාවිතා කර පින්තූර ලෝඩ් වන පිළිවෙළ පාලනය කිරීම.

5. next/font භාවිතා කර Google Fonts වෙබ් අඩවියේ වේගයට හානි නොවන ලෙස සම්බන්ධ කරන ආකාරය.

6. පින්තූර සහ අකුරු නිසා සිදුවන වේගය අඩුවීම (Performance bottlenecks) මඟහරවා ගන්නා ආකාරය.

97.1 හැඳින්වීම: පින්තූර සහ අකුරු වෙබ් අඩවියට බලපාන ආකාරය

වෙබ් අඩවියක් මන්දගාමී වීමට බලපාන ප්‍රධානතම හේතුව වන්නේ විශාල පින්තූර (Images) සහ බාහිරින් ලබා ගන්නා අකුරු වර්ග (Fonts) ය. සාමාන්‍ය වෙබ් අඩවියක පින්තූරයක් ලෝඩ් වන විට එය බොහෝ විට මුල් ප්‍රමාණයෙන් (උදා: 5MB) බාගත වේ. මෙය පරිශීලකයාගේ දත්ත (Data) සහ කාලය නාස්ති කිරීමකි.

එසේම, අකුරු (Fonts) ලෝඩ් වීමට පෙර ඔවුසරය සාමාන්‍ය අකුරු පෙන්වා, පසුව අලුත් Font එක පැමිණි විට අකුරුවල හැඩය වෙනස් වේ. එවිට පිටුවේ ඇති දේවල් මඳක් එහා මෙහා වේ. මෙය Cumulative Layout Shift (CLS) ලෙස හඳුන්වන අතර, එය Google සෙවුම් යන්ත්‍රය මගින් ඔබේ වෙබ් අඩවියේ අගය (Ranking) අඩු කිරීමට හේතුවකි.

Next.js මෙම ගැටලු සියල්ල ස්වයංක්‍රීයව විසඳීමට ඉතා බලවත් මෙවලම් දෙකක් අපට ලබා දෙයි.

97.2 නෙක්ස්ට් ජේ.එස් ඉමේජ් සංරචකය (The <Image /> Component)

නෙක්ස්ට් ජේ.එස් විසින් ලබා දෙන <Image /> සංරචකය සාමාන්‍ය ටැගයට වඩා ඉතා දියුණුය.

එහි ඇති විශේෂාංග:

1. Size Optimization: පරිශීලකයාගේ තිරයේ ප්‍රමාණය අනුව පින්තූරයේ ප්‍රමාණය ස්වයංක්‍රීයව සකසයි (දුරකථනයකට කුඩා පින්තූරයක් ද, පරිගණකයට විශාල පින්තූරයක් ද යවයි).

2. Format Conversion: JPEG හෝ PNG පින්තූර ස්වයංක්‍රීයව ඉතා සැහැල්ලු WebP හෝ AVIF වර්ගවලට හරවයි.
3. Visual Stability: පින්තූරය ලෝඩ වීමට පෙර ඒ සඳහා අවශ්‍ය ඉඩ වෙන් කර තබන බැවින් Layout Shift ඇති නොවේ.
4. Lazy Loading: පින්තූරය තිරය මතට (Viewport) පැමිණෙන තෙක් එය බාගත නොකර සිටියි.

ප්‍රායෝගික කේතය:

```
import Image from 'next/image';
```

```
export default function ProfilePage() {
  return (
    <div>
      <h1>මගේ ප්‍රෝෆයිලය</h1>
      <Image
        src="/profile.jpg" // පින්තූරයේ ලිපිනය
        alt="කසුන් කල්හාරගේ ඡායාරූපය"
        width={500} // පළල (අනිවාර්ය වේ)
        height={500} // උස (අනිවාර්ය වේ)
        className="rounded-full"
      />
    </div>
  );
}
```

97.3 ප්‍රමුඛතාවය ලබා දීම (The priority Property)

වෙබ් අඩවියක ඉහළින්ම පෙනෙන පින්තූරය (උදා: Hero image හෝ Logo) ඉතා ඉක්මනින් ලෝඩ විය යුතුය. ඒ සඳහා අප priority නමැති ගුණාංගය භාවිතා කරමු. එවිට Next.js එම පින්තූරය Lazy Loading කරන්නේ නැතිව වහාම බාගත කරයි.

```
<Image
  src="/hero.png"
  alt="Main Banner"
  width={1200}
  height={600}
  priority // මෙය වැදගත් පින්තූරයක් බව පවසයි
/>
```

97.4 බාහිර පින්තූර භාවිතා කිරීම (Remote Images)

ඔබ පින්තූරයක් වෙනත් වෙබ් අඩවියකින් (උදා: Unsplash හෝ Facebook) ලබා ගන්නේ නම්,

ආරක්ෂක හේතූන් මත ඔබ එම වෙබ් අඩවියේ නම next.config.js ගොනුව තුළ ලියාපදිංචි කළ යුතුය.

next.config.js සැකසීම:

```
const nextConfig = {
  images: {
    remotePatterns: [
      {
        protocol: 'https',
        hostname: 'images.unsplash.com',
      },
    ],
  },
};
```

97.5 නෙක්ස්ට් ජේ.එස් ෆොන්ට් පාලනය (Next.js Fonts)

next/font නමැති පද්ධතිය මගින් Google Fonts හෝ ඔබේ පරිගණකයේ ඇති Fonts වෙබ් අඩවියට ඉතා කාර්යක්ෂමව සම්බන්ධ කරයි.

එහි ඇති වාසිය: සාමාන්‍යයෙන් Google Fonts සම්බන්ධ කළ විට, පරිශීලකයාගේ බ්‍රවුසරය Google සර්වර් එකට සම්බන්ධ වී අකුරු බාගත කළ යුතුය. මෙය වෙබ් අඩවිය මන්දගාමී කරයි. නමුත් Next.js විසින් කරන්නේ එම අකුරු අපේ සර්වර් එකටම බාගත කර තබා ගැනීමයි (Self-hosting). එවිට කිසිදු බාහිර සම්බන්ධතාවයක් අවශ්‍ය නොවේ.

ප්‍රායෝගික කේතය (Google Fonts භාවිතය):
app/layout.tsx ගොනුවේ මෙය සැකසීම සුදුසුය.

```
import { Inter } from 'next/font/google';
```

```
// අකුරු වර්ගය සාදා ගැනීම
const inter = Inter({
  subsets: ['latin'],
  display: 'swap', // CLS වැළැක්වීමට මෙය උදව් වේ
});
```

```
export default function RootLayout({ children }: {
  children: React.ReactNode }) {
  return (
    <html lang="si"
      className={inter.className}>
      <body>{children}</body>
    </html>
  );
}
```

97.6 ASCII රූපසටහන: Cumulative Layout Shift (CLS) වැළැක්වීම

සාමාන්‍ය (No Width/Height):

```
[ අකුරු පේළිය 1 ]
[ (පින්තූරය තවම ලෝඩ් වේ - ඉඩක් නැත) ]
[ අකුරු පේළිය 2 ]
|
v (පින්තූරය පැමිණි පසු)
[ අකුරු පේළිය 1 ]
[ ===== ]
[ | IMAGE | ] <--- අකුරු පහළට පනයි (Bad UX)
[ ===== ]
[ අකුරු පේළිය 2 ]
```

Next.js <Image /> (With Width/Height):

```
[ අකුරු පේළිය 1 ]
[ ..... ]
[ : RESERVED : ] <--- ඉඩ සරය ඉඩ වෙන් කර තබයි
[ : SPACE : ]
[ ..... ]
[ අකුරු පේළිය 2 ]
|
v (පින්තූරය පැමිණි පසු)
[ අකුරු පේළිය 1 ]
[ ===== ]
[ | IMAGE | ] <--- කිසිවක් එහා මෙහා නොවේ
(Good UX)
[ ===== ]
[ අකුරු පේළිය 2 ]
```

97.7 වෘත්තීය පැහැදිලි කිරීම (Professional Explanation)

මෘදුකාංග ඉංජිනේරු විද්‍යාවේදී මෙය "Zero-Runtime Image Optimization" ලෙස හඳුන්වයි. නෙක්ස්ට් ජේ.එස්.එස්.ඒස් පින්තූර සකසන්නේ සර්වර් එක තුළදීය (Server-side). මෙහිදී අප භාවිතා කරන්නේ Sharp නමැති ඉතා වේගවත් පින්තූර සැකසුම් පුස්තකාලයකි. වෘත්තීය මට්ටමේදී, LCP (Largest Contentful Paint) ලකුණු වැඩි කර ගැනීමට මෙම <Image/> සංරචකය සහ next/font භාවිතය අනිවාර්ය වේ.

97.8 පොදු වැරදි (Common Mistakes)

1. Width සහ Height ලබා නොදීම: පින්තූරයේ ප්‍රමාණය ලබා නොදුන්නොත් Next.js හට ඉඩ වෙන් කළ නොහැකි නිසා Error එකක් පෙන්වයි. (පින්තූරයේ ප්‍රමාණය නොදන්නේ නම් fill ගුණාංගය භාවිතා කරන්න).
2. Priority පාවිච්චි නොකිරීම: වෙබ් අඩවියේ ලාංඡනය (Logo) සඳහා Lazy Loading ක්‍රියාත්මක

වීම නිසා එය ප්‍රමාද වී දිස්විය හැක. එයට සැමවිටම priority ලබා දෙන්න.

3. බාහිර URL ලියාපදිංචි නොකිරීම: වෙනත් අඩවියක පින්තූරයක් පෙන්වීමට උත්සාහ කර next.config.js සැකසීමට අමතක වීම.

97.9 සාරාංශය (Key Points)

- <Image /> සංරචකය ස්වයංක්‍රීයව පින්තූර ප්‍රශස්ත කරයි.
- පින්තූර සඳහා width සහ height ලබා දීම CLS වැළැක්වීමට අත්‍යවශ්‍යය.
- priority මගින් වැදගත් පින්තූර ඉක්මනින් ලෝඩ් කරයි.
- next/font මගින් අකුරු වර්ග සර්වර් එක තුළම තැන්පත් කර වේගය වැඩි කරයි.
- මේ සියල්ල වෙබ් අඩවියේ Core Web Vitals ලකුණු වැඩි කිරීමට උදව් වේ.

97.10 අභ්‍යාස (Exercises)

1. ඔබේ ව්‍යාපෘතියට public ෆෝල්ඩරය හරහා පින්තූරයක් එක් කර එය <Image /> සංරචකය භාවිතා කර තිරය මත පෙන්වන්න.
2. Google Fonts වෙතින් "Roboto" හෝ "Poppins" අකුරු වර්ගය ගෙන එය ඔබේ මුළු වෙබ් අඩවියටම බලපාන සේ layout.tsx තුළ සකසන්න.
3. පින්තූරයක් මතට මවුසය ගෙන ගිය විට එය බොදා වන (Blur) සේ Tailwind CSS සහ Next.js Image එකාබද්ධ කරන්න.
4. පින්තූරයකට width සහ height වෙනුවට fill ගුණාංගය භාවිතා කරන්නේ කෙසේදැයි අන්තර්ජාලයෙන් සොයා බලන්න. (ඉභිය: එහිදී දෙමාපිය ධූරය position: relative විය යුතුය).

අනු අටවන පරිච්ඡේදය: සර්වර් ඇක්ෂන්ස් - පෝරම සහ දත්ත හැසිරවීමේ නව මාවත (Server Actions in Next.js)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අධ්‍යයනය කිරීමෙන් පසු ඔබට නෙක්ස්ට් ජේ.එස්.එස්.ඒස් ඇති බලවත්ම විශේෂාංගයක් වන සර්වර් ඇක්ෂන්ස් (Server Actions) පිළිබඳව අතිශය ගැඹුරු වෘත්තීය දැනුමක් ලැබෙනු ඇත:

1. Server Actions යනු කුමක්ද සහ ඒවායේ ඇති විචල්‍ය වැදගත්කම වටහා ගැනීම.

2. පිටතින් වෙනම Backend එකක් (API Route) නොමැතිව සෘජුවම දත්ත ගබඩාවකට (Database) දත්ත යවන ආකාරය තේරුම් ගැනීම.
3. "use server" විධානය භාවිතා කර අසමකාලීන ශ්‍රිත (Async Functions) සර්වර් මට්ටමේ ක්‍රියාත්මක කරන ආකාරය ඉගෙන ගැනීම.
4. පෝරම (Forms) සහ සර්වර් ඇක්ෂන්ස් එකිනෙක සම්බන්ධ කරන ආකාරය.
5. දත්ත ඇතුළත් කළ පසු වෙබ් පිටුව යාවත්කාලීන කිරීමට `revalidatePath` භාවිතා කරන ආකාරය.
6. පෝරමයක් යවන අතරතුර පවතින තත්ත්වය (Pending States) කළමනාකරණය කිරීමට `useFormStatus` හුක් එක භාවිතා කිරීම.

98.1 හැඳින්වීම: සර්වර් ඇක්ෂන්ස් යනු කුමක්ද?

පෙර පරිච්ඡේදවලදී (පරිමාව 6, පරිච්ඡේදය 62) අප ඉගෙන ගත්තේ රියැක්ට් පෝරම (Forms) පාලනය කිරීමට `useState` සහ `useEffect` භාවිතා කරමින් සර්වර් එකකට (API) දත්ත යවන ආකාරයයි. එහිදී අපට Frontend එකේ සිට Backend එකට දත්ත යැවීමට ලොකු කේතයක් ලිවීමට සිදු විය.

Server Actions යනු මෙම ක්‍රියාවලිය අතිශය සරල කරන Next.js හි අලුත්ම තාක්ෂණයයි. මෙහිදී අප කරන්නේ සාමාන්‍ය ජාවාස්ක්‍රිප්ට් ශ්‍රිතයක් (Function) ලියා, එය සර්වර් එක ඇතුළත පමණක් ක්‍රියාත්මක වන බවට නියෝග කිරීමයි. පෝරමයක් Submit කළ විට, එම ශ්‍රිතය කෙලින්ම සර්වර් එකේදී ක්‍රියාත්මක වී දත්ත ගබඩා කරයි.

සැබෑ ලෝකයේ උපමාව (තැපැල් ලියුම සහ සෘජු සංවාදය):

- පැරණි ක්‍රමය (API Routes): ඔබ ඔබේ මිතුරෙකුට යමක් පැවසීමට ලියුමක් ලියා (JSON Data), එය තැපැල් කර (HTTP Request), ඔහුට ලැබෙන තෙක් බලා සිටීම වැනිය.
- සර්වර් ඇක්ෂන්ස් ක්‍රමය: ඔබ මිතුරා අසලම හිඳගෙන සෘජුවම ඔහුට කතා කිරීම වැනිය. පණිවිඩය අතරමග කොහේවත් යන්නේ නැත. ඔබ පවසන දේ ඔහුට එසැණින් ඇසේ.

98.2 සර්වර් ඇක්ෂන් එකක මූලික ව්‍යුහය

සර්වර් ඇක්ෂන් එකක් සෑදීමට නම්, ශ්‍රිතයක් ඇතුළත හෝ ගොනුවක මුදුනේ "use server" යනුවෙන් සඳහන් කළ යුතුය.

මූලික කේතය:

```
// සර්වර් ඇක්ෂන් එකක් අර්ථ දැක්වීම
async function createInvoice(formData:
FormData) {
  'use server'; // මෙය සර්වර් එකේ පමණක්
  ක්‍රියාත්මක වන බව පවසයි
```

```
  const amount = formData.get('amount');
  const status = formData.get('status');
```

```
  // මෙතැනදී අපට දත්ත ගබඩාවට (Database) දත්ත
  යැවිය හැක
  console.log("දත්ත ලැබුණා:", amount, status);
}
```

98.3 පෝරමයක් සමඟ සම්බන්ධ කිරීම (Form Integration)

සාමාන්‍යයෙන් අප පෝරමයක `onSubmit` භාවිතා කළද, සර්වර් ඇක්ෂන්ස් සමඟ අප භාවිතා කරන්නේ `action` නමැති ගුණාංගයයි.

ප්‍රායෝගික කේතය:

```
// app/add-item/page.tsx
```

```
export default function AddItemPage() {
```

```
  // සර්වර් ඇක්ෂන් ශ්‍රිතය
  async function addItem(formData: FormData) {
    'use server';
    const itemName = formData.get('itemName');
    console.log(`${itemName} භාණ්ඩය සර්වර්
    එකට ලැබුණා!`);
  }
```

```
  return (
    <form action={addItem}>
      <input type="text" name="itemName"
        placeholder="භාණ්ඩයේ නම" required />
      <button type="submit">එකතු
        කරන්න</button>
    </form>
  );
}
```

විශේෂත්වය: මෙහිදී අපට කිසිදු `useState` එකක් හෝ `onChange` එකක් අවශ්‍ය නැත. පෝරමයේ ඇති `name` ගුණාංගය භාවිතා කර Next.js විසින්ම දත්ත ලබා ගනී.

98.4 පිටුව යාවත්කාලීන කිරීම (Revalidation)

සර්වර් ඇක්ෂන් එකක් මගින් දත්ත ඇතුළත් කළ පසු, ඔවුසරයේ ඇති පැරණි දත්ත මකා දමා අලුත් දත්ත පෙන්වීමට අපට `revalidatePath` අවශ්‍ය වේ.

```
import { revalidatePath } from 'next/cache';

async function addItem(formData: FormData) {
  'use server';
  // 1. දත්ත සේවි කරන්න
  await saveToDatabase(formData.get('name'));

  // 2. අදාළ පිටුව අලුත් (Refresh) කරන්නැයි සර්වර්
  එකට පවසන්න
  revalidatePath('/dashboard');
}
```

98.5 ප්‍රවේශන තත්ත්ව පාලනය (useFormStatus)

පෝරමය Submit කළ පසු දත්ත සේවි වීමට තත්පර කිහිපයක් ගත විය හැක. එම කාලය තුළ බොත්තම අක්‍රිය (Disable) කිරීමට හෝ "සේවි වෙමින් පවතී..." යනුවෙන් පෙන්වීමට `useFormStatus` හුක් එක භාවිතා කරයි.

වැදගත්: මෙය භාවිතා කළ හැක්කේ Client Component එකක් ඇතුළත පමණි.

ප්‍රායෝගික උදාහරණය:

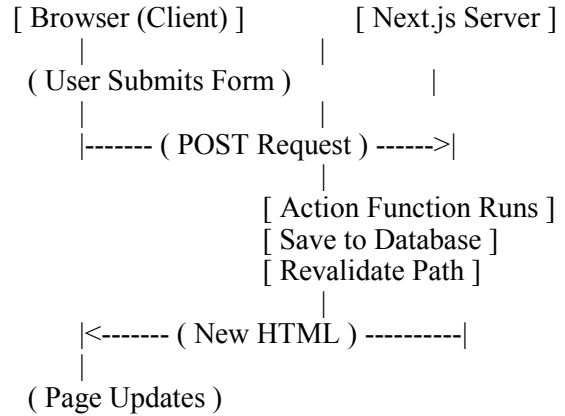
```
// components/SubmitButton.tsx
'use client';

import { useFormStatus } from 'react-dom';

export function SubmitButton() {
  const { pending } = useFormStatus();

  return (
    <button type="submit" disabled={pending}>
      {pending ? 'සේවි වෙමින් පවතී...' : 'සේවි
      කරන්න'}
    </button>
  );
}
```

98.6 ASCII රූපසටහන: Server Action ක්‍රියාවලිය



98.7 වෘත්තීය පැහැදිලි කිරීම (Professional Explanation)

මෘදුකාංග ඉංජිනේරු විද්‍යාවේදී මෙය "Progressive Enhancement" සංකල්පයට අනුකූල වේ. සර්වර් ඇක්ෂන්ස් හි ඇති විශේෂත්වය වන්නේ පරිශීලකයාගේ ඔවුසරයේ JavaScript ක්‍රියා විරහිත කර තිබුණත් (සමහර පැරණි දුරකථන වල), මෙම පෝරමය තවමත් සාර්ථකව ක්‍රියාත්මක වීමයි. මෙය ඉතාමත් ආරක්ෂිත ක්‍රමයකි (Security), මන්ද දත්ත හැසිරවීමේ තර්කනය (Business Logic) කිසිවිටකත් ඔවුසරය වෙත නොයන බැවිනි.

98.8 පොදු වැරදි (Common Mistakes)

1. "use server" වැරදි තැනක ලිවීම: ඔබ වෙනම ගොනුවක ඇක්ෂන්ස් ලියන්නේ නම්, එම ගොනුවේ ඉහළින්ම මෙය තිබිය යුතුය. නැතිනම් එය සාමාන්‍ය ශ්‍රිතයක් ලෙස සලකා Error එකක් පෙන්වනු ඇත.
2. Redirect වැරදි ලෙස භාවිතය: දත්ත සේවි කළ පසු පරිශීලකයා වෙතත් පිටුවකට රැගෙන යාමට `redirect()` භාවිතා කළ යුතුය. එය සැමවිටම `try...catch` කොටසෙන් පිටත ලිවීමට වගබලා ගන්න.
3. Client-side Validation නොමැති වීම: සර්වර් එකට දත්ත යැවීමට පෙර ඔවුසරයේදීම HTML `required` හෝ JavaScript මගින් දත්ත පරීක්ෂා කිරීම වේගය වැඩි කිරීමට උදව් වේ.

98.9 සාරාංශය (Key Points)

- Server Actions මගින් සර්වර් එක තුළ සෘජුවම කේත ක්‍රියාත්මක කරයි.
- "use server" විධානය මගින් මෙය සක්‍රීය කරයි.
- පෝරමවල action ගුණාංගය හරහා දත්ත යවනු ලැබේ.
- revalidatePath මගින් පිටුවේ දත්ත අලුත් කරයි.
- useFormStatus මගින් පෝරමයේ ප්‍රගතිය පෙන්වයි.

98.10 අභ්‍යාස (Exercises)

1. පරිශීලකයාගේ නම සහ විද්‍යුත් තැපෑල (Email) ලබා ගන්නා සරල පෝරමයක් සාදා, එය Server Action එකක් හරහා සර්වර් එකේ කන්සෝලයේ මුද්‍රණය කරන්න.
2. පෝරමය Submit කරන විට බොත්තම "යවමින් පවතී..." ලෙස වෙනස් වන සේ useFormStatus භාවිතා කර සකසන්න.
3. දත්ත ඇතුළත් කළ පසු පරිශීලකයා /success පිටුව වෙත රැගෙන යන ලෙස redirect විධානය භාවිතා කරන්න.
4. සර්වර් ඇත්මන් එකක් ඇතුළත try...catch භාවිතා කර වැරදි පාලනය කරන ආකාරය පෙන්වන්න.

අනු නවවන පරිච්ඡේදය: මැදිහත්කරු සහ ආරක්ෂාව - වෙබ් අඩවියේ ගේට්ටු පාලකයා (Middleware and Security in Next.js)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අධ්‍යයනය කිරීමෙන් පසු ඔබට නෙක්ස්ට් ජේ.එස්.ඒ ආරක්ෂක පද්ධති සහ දත්ත ගලායාම පාලනය කිරීම පිළිබඳව අතිශය ගැඹුරු වෘත්තීය දැනුමක් ලැබෙනු ඇත:

1. Middleware (මැදිහත්කරු) යනු කුමක්ද සහ එය වෙබ් අඩවියක ආරක්ෂාවට බලපාන ආකාරය තේරුම් ගැනීම.
2. පරිශීලකයෙකු පිටුවකට ඇතුළු වීමට පෙර ඔහුගේ අවසරය (Authentication) පරීක්ෂා කරන ආකාරය.
3. middleware.ts ගොනුව භාවිතා කර වෙබ් අඩවියේ පාරවල් (Routes) ආරක්ෂා කරන ආකාරය.

4. Authentication (සත්‍යාපනය) සහ Authorization (අවසරය) අතර ඇති තාක්ෂණික වෙනස වටහා ගැනීම.

5. Edge Runtime යනු කුමක්ද සහ මැදිහත්කරු එතරම් වේගවත් වන්නේ ඇයි දැයි ඉගෙන ගැනීම.

6. පොදු වෙබ් ප්‍රහාර (XSS, CSRF) වලින් වෙබ් අඩවිය ආරක්ෂා කරගන්නා මූලධර්ම හඳුනා ගැනීම.

99.1 හැඳින්වීම: මැදිහත්කරු (Middleware) යනු කවුද?

සිතන්න ඔබ බැංකුවකට ඇතුළු වීමට යනවා කියා. ඔබ බැංකුව ඇතුළට යාමට පෙර, දොර අසල සිටින ආරක්ෂක නිලධාරියා (Security Guard) ඔබව පරීක්ෂා කරයි. ඔබ සතුව හැඳුනුම්පත තිබේ නම් පමණක් ඔහු ඔබට ඇතුළු වීමට ඉඩ දෙයි. නැතිනම් ඔබව නැවත හරවා යවයි.

නෙක්ස්ට් ජේ.එස්.ඒ Middleware යනු එම ආරක්ෂක නිලධාරියා වැනිය. පරිශීලකයෙකු යම් වෙබ් පිටුවකට (උදා: /dashboard) පිවිසීමට ඉල්ලීමක් කළ විට, එම ඉල්ලීම සපුරාලීමට පෙර මැදිහත්කරු ක්‍රියාත්මක වේ. ඔහුට පුළුවන්:

- පරිශීලකයා ලොගින් වී ඇත්දැයි බැලීමට.
- ඔහුට එම පිටුව බැලීමට අවසර තිබේදැයි බැලීමට.
- අවසර නැතිනම් ඔහුව වෙනත් පිටුවකට (උදා: /login) යොමු කිරීමට (Redirect).

99.2 මැදිහත්කරු ස්ථාපනය කරන ආකාරය

නෙක්ස්ට් ජේ.එස්.ඒ වලදී මැදිහත්කරු සඳහා වෙන්වූ විශේෂ ගොනුවක් ඇත. එය ඔබගේ ව්‍යාපෘතියේ Root ෆෝල්ඩරය තුළ (එනම් app ෆෝල්ඩරයට සමාන්තරව) middleware.ts (හෝ .js) නමින් නිර්මාණය කළ යුතුය.

මූලික ව්‍යුහය:

```
// middleware.ts
import { NextResponse } from 'next/server'
import type { NextRequest } from 'next/server'
```

```
export function middleware(request:
NextRequest) {
  // 1. පරිශීලකයා ලොගින් වී ඇත්දැයි බලමු
  (Cookie එකක් හරහා)
  const currentUser =
request.cookies.get('session')
```

```
// 2. ලොගින් වී නැතිනම් ඔහුව login පිටුවට යවමු
```

```

if (!currentUser &&
request.nextUrl.pathname.startsWith('/dashboard
')) {
  return NextResponse.redirect(new
URL('/login', request.url))
}
}

```

```

// 3. මෙම මැදිහත්කරු ක්‍රියාත්මක විය යුතු පාරවල
(Routes) නම් කරන්න
export const config = {
  matcher: '/dashboard/:path*',
}

```

99.3 මැටර් (Matcher) යනු කුමක්ද?

මැදිහත්කරු වෙබ් අඩවියේ ඇති සෑම ගොනුවකටම (පින්තූර, CSS පවා) ක්‍රියාත්මක වීම වෙබ් අඩවිය මන්දගාමී වීමට හේතු විය හැක. එම නිසා අප Matcher භාවිතා කර පවසනවා මැදිහත්කරු ක්‍රියාත්මක විය යුත්තේ කුමන ලිපින සඳහා පමණක්ද කියා.

- matcher: '/dashboard/:path*' : මෙයින් අදහස් වන්නේ /dashboard සහ ඒ යටතේ ඇති ඕනෑම පිටුවකට යන විට මැදිහත්කරු ක්‍රියාත්මක විය යුතු බවයි.

99.4 සත්‍යාපනය (Auth) පිළිබඳ මූලික පාඩම

වෙබ් ආරක්ෂාව ප්‍රධාන කොටස් දෙකකි:

1. Authentication (සත්‍යාපනය): "ඔබ කවුද?" (Who are you?). පරිශීලකයා තමා පවසන පුද්ගලයාට බව තහවුරු කිරීම (Login වීම).
2. Authorization (අවසරය පරීක්ෂා කිරීම): "ඔබට කුමක් කළ හැකිද?" (What can you do?). ලොගින් වූ පුද්ගලයාට එම පිටුව බැලීමට බලය තිබේදැයි බැලීම (උදා: සාමාන්‍ය පරිශීලකයෙකුට Admin පිටුව බැලිය නොහැක).

99.5 නෙක්ස්ට්-ඕත් (Auth.js) හඳුන්වාදීම

වෘත්තීය මට්ටමේදී අප මේ සියලු ආරක්ෂක කේතයන් අතින් ලියන්නේ නැත. ඒ සඳහා ලොව පිළිගත් සහ ආරක්ෂිතම පුස්තකාලය වන්නේ Auth.js (පෙර නම: NextAuth.js) ය.

මෙය භාවිතා කිරීමෙන්:

- Google හෝ Facebook හරහා ලොගින් වීමට ඉඩ දිය හැක.

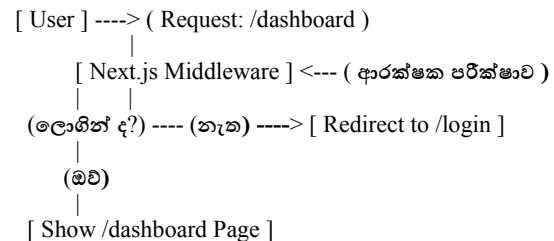
- මුරපද (Passwords) ඉතා ආරක්ෂිතව ගබඩා කළ හැක.

- Middleware සමඟ ඉතා පහසුවෙන් සම්බන්ධ කළ හැක.

99.6 එජ් රන්ටයිම් (Edge Runtime) - වේගයේ රහස

මැදිහත්කරු ක්‍රියාත්මක වන්නේ Edge Runtime නමැති ඉතා සැහැල්ලු පරිසරයකය. මෙහි ඇති විශේෂත්වය වන්නේ, ලෝකයේ විවිධ රටවල විසිරී ඇති සර්වර් (CDN nodes) හරහා පරිශීලකයාට ඉතාමත් ආසන්න ස්ථානයකදී මෙම ආරක්ෂක පරීක්ෂාව සිදුවීමයි. මේ නිසා වෙබ් අඩවිය අතිශය වේගවත් වේ.

99.7 ASCII රූපසටහන: Middleware ක්‍රියාවලිය



99.8 වෘත්තීය පැහැදිලි කිරීම (Professional Explanation)

මෘදුකාංග ඉංජිනේරු විද්‍යාවේදී Middleware යනු "Intercepting Pattern" එකකි. එය HTTP Request සහ Response වක්‍රය අතරට මැදිහත් වී දත්ත වෙනස් කිරීමට හෝ ගමනාන්තය තීරණය කිරීමට අවස්ථාව ලබා දෙයි. වෘත්තීය මට්ටමේදී, ආරක්ෂාවට අමතරව A/B Testing (පරිශීලකයන් කොටසකට එක පිටුවකුත්, අනෙක් අයට වෙනත් පිටුවකුත් පෙන්වීම) සහ Internationalization (භාෂාව අනුව පිටු මාරු කිරීම) සඳහා ද මෙය බහුලව භාවිතා වේ.

99.9 පොදු වැරදි (Common Mistakes)

1. අනන්ත පුඩු (Infinite Redirects): ඔබ /login පිටුවට යන විටත් මැදිහත්කරු ක්‍රියාත්මක වී නැවත /login වෙතම යොමු කළහොත් ඉවහරයාගේ හිත වේ. සැමවිටම /login සහ /api පාරවල් මැදිහත්කරුගෙන් ඉවත් කරන්න (Exclude).

2. සංකීර්ණ තර්කනයන් ලිවීම: මැදිහත්කරු ඉතා වේගවත් විය යුතුය. එහිදී දත්ත ගබඩාවල (Database) විශාල ගණනය කිරීම් සිදු නොකළ යුතුය. අවශ්‍යම ආරක්ෂක පරීක්ෂාව පමණක් සිදු කරන්න.
3. Matcher වැරදීම: නිවැරදිව පාරවල් සටහන් නොකිරීම නිසා ආරක්ෂිත පිටු ඕනෑම අයෙකුට බැලීමට හැකි වීම.

99.10 සාරාංශය (Key Points)

- Middleware යනු පිටුවකට ඇතුළු වීමට පෙර ක්‍රියාත්මක වන "ආරක්ෂක නිලධාරියා" වේ.
- එය middleware.ts ගොනුවේ අර්ථ දැක්විය යුතුය.
- NextResponse මගින් පරිශීලකයා වෙනත් පිටුවකට යොමු කළ හැක.
- Authentication සහ Authorization අතර වෙනස හඳුනා ගැනීම ආරක්ෂාවට අත්‍යවශ්‍යය.
- Edge Runtime භාවිතය නිසා එය ඉතා වේගවත්ය.

99.11 අභ්‍යාස (Exercises)

1. ඔබේ ව්‍යාපෘතියේ /admin නමින් පිටුවක් සාදා, පරිශීලකයා සතුව admin-token නමැති කුකිය (Cookie) නොමැති නම් ඔහු ව මුල් පිටුවට (/) හරවා යවන සරල මැදිහත්කරුවෙකු ලියන්න.
2. මැදිහත්කරුවෙකු භාවිතා කරමින්, පරිශීලකයා එන රට (Country) අනුව වෙනස් පණිවිඩයක් මුද්‍රණය කිරීමට උත්සාහ කරන්න. (ඉඟිය: request.geo භාවිතා කරන්න).
3. NextAuth.js වෙබ් අඩවියට පිවිස එහි "Getting Started" කොටස කියවා මූලික අවබෝධයක් ලබා ගන්න.

සුභ පැතුම්! ඔබ දැන් තෙක්ස්ට් ජේ.එස් හි සියලුම මූලික සහ උසස් තාක්ෂණික කරුණු ආවරණය කර අවසානය. පරිමාව 9 හි අවසාන පරිච්ඡේදය ලෙස මිළඟට අප ඉගෙන ගන්නේ සියලුම කරුණු එකතු කළ "මහා තෙක්ස්ට් ජේ.එස් ව්‍යාපෘතිය" (Capstone Next.js Project) පිළිබඳවයි.

සියවන පරිච්ඡේදය: මහා ව්‍යාපෘතිය - "InsightBlog" පූර්ණ ඉදිරිපෙළ පද්ධතිය (Capstone Project: InsightBlog Full-stack Dashboard)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම මුළු පාඨමාලාවේම උච්චතම අවස්ථාව වන මෙම සියවන පරිච්ඡේදය අවසානයේදී, ඔබ සැබෑ ලෝකයේ මෘදුකාංග ඉංජිනේරුවෙකු ලෙස පහත සියලුම දක්ෂතාවන් එකිනෙක මුසු කරමින් පූර්ණ මෘදුකාංගයක් නිර්මාණය කරනු ඇත:

1. Next.js App Router Architecture: මහා පරිමාණ ව්‍යාපෘතියක් ක්‍රමානුකූලව සංවිධානය කිරීම.
2. Full-stack Data Flow: සර්වර් සංරචක (Server Components) සහ සර්වර් ඇක්ෂන්ස් (Server Actions) භාවිතයෙන් දත්ත කළමනාකරණය.
3. Advanced UI/UX: Tailwind CSS සහ Next.js සංරචක (Image, Font) භාවිතා කර උසස් මට්ටමේ අතුරුමුහුණතක් සැකසීම.
4. Dynamic SEO: සෑම ලිපියකටම අනන්‍ය වූ මෙටා-දේවා (Metadata) උත්පාදනය කිරීම.
5. State & Persistence: දත්ත මතකයේ තබා ගැනීම සහ ප්‍රවේශන තත්ත්වයන් (Loading/Suspense) හැසිරවීම.
6. Professional Deployment: මෘදුකාංගය සර්වර් එකක් වෙත මුදාහැරීම සහ විගණනය (Audit) කිරීම.

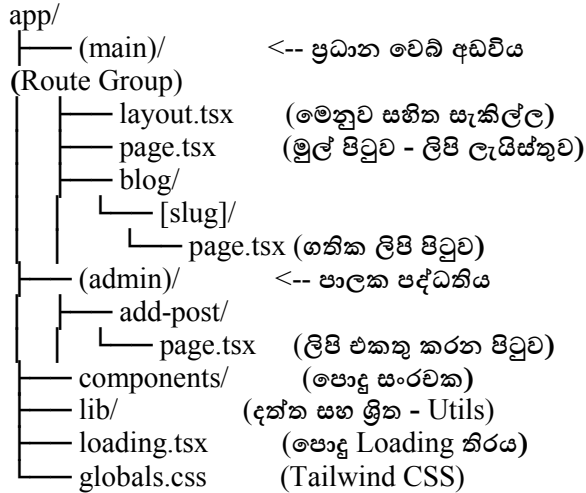
100.1 ව්‍යාපෘති විස්තරය: InsightBlog යනු කුමක්ද?

අපි අද නිර්මාණය කරන්නේ නවීන තාක්ෂණයෙන් සන්නද්ධ බ්ලොග් අඩවියකි. මෙහිදී පරිශීලකයාට:

- සියලුම ලිපි (Posts) මුල් පිටුවේදී නැරඹිය හැක.
- එක් ලිපියක් ක්ලික් කළ විට එහි සම්පූර්ණ විස්තරය වෙනම ගතික පිටුවකින් (Dynamic Route) බැලිය හැක.
- අලුත් ලිපියක් ඇතුළත් කිරීමට වෙන්වූ පද්ධතියක් (Server Action) ඇත.
- දත්ත ලබා ගන්නා තෙක් ලස්සන සැකිලි (Skeletons) දිස්වේ.
- ලිපි සෙවීමට (Search) සජීවී සෙවුම් පද්ධතියක් ඇත.

100.2 පියවර 1: වෘත්තීය ව්‍යාපෘති ව්‍යුහය (Project Structure)

පළමුව ඔබගේ app ෆෝල්ඩරය මෙලෙස සකස් කරන්න. අපි මෙහිදී Route Groups භාවිතා කරමු.



100.3 පියවර 2: දත්ත ගබඩාව සකස් කිරීම (Mock Database)

සැබෑ දත්ත ගබඩාවක් (Database) අප මිලහ Backend පාඨමාලාවේදී ඉගෙන ගන්නා බැවින්, දැනට අපි ජාවාස්ක්‍රිප්ට් Object එකක් භාවිතා කර දත්ත ගබඩාවක් අනුකරණය කරමු.

lib/data.ts

```
// සර්වර් එකේ පමණක් ක්‍රියා කරන බවට සහතික කරමු
import "server-only";
```

```
export type Post = {
  id: string;
  title: string;
  content: string;
  author: string;
  date: string;
};

export const blogPosts: Post[] = [
  { id: "1", title: "React ඉගෙන ගනිමු", content: "React ඉතා බලවත්ය...", author: "කසුන්", date: "2024-05-01" },
  { id: "2", title: "Next.js අභ්‍යන්තරය", content: "Next.js වේගවත්ය...", author: "නිමල්", date: "2024-05-05" },
];
```

```
// දත්ත ලබා ගැනීමට සර්වර් ශ්‍රිතය (Simulating a DB call)
export async function fetchPosts() {
  await new Promise((resolve) =>
    setTimeout(resolve, 1500)); // පමාව අනුකරණය කිරීම
  return blogPosts;
}
```

```
export async function fetchPostById(id: string) {
  return blogPosts.find((post) => post.id === id);
}
```

100.4 පියවර 3: මුල් පිටුව නිර්මාණය (Server Component)

මෙහිදී අපි සර්වර් එකේ සිටම දත්ත ලබාගෙන ලැයිස්තුව පෙන්වමු.

app/(main)/page.tsx

```
import { fetchPosts } from "@lib/data";
import Link from "next/link";

export default async function HomePage() {
  const posts = await fetchPosts();

  return (
    <div className="max-w-4xl mx-auto p-8">
      <h1 className="text-4xl font-extrabold mb-8 text-slate-900">අලුත්ම ලිපි 📄</h1>
      <div className="grid gap-6">
        {posts.map((post) => (
          <Link key={post.id}
            href={`\blog/${post.id}`}>
            <div className="p-6 border rounded-xl hover:shadow-lg transition-all bg-white cursor-pointer">
              <h2 className="text-2xl font-bold text-blue-600">{post.title}</h2>
              <p className="text-slate-500 mt-2">කර්තෘ: {post.author} | {post.date}</p>
            </div>
          </Link>
        ))}
      </div>
    </div>
  );
}
```


100.5 පියවර 4: සර්වර් ඇක්ෂන්ස් භාවිතය (Server Actions)

ලිපියක් එකතු කිරීම සඳහා අපි "use server" විධානය භාවිතා කරමු.

lib/actions.ts

"use server";

```
import { revalidatePath } from "next/cache";
import { redirect } from "next/navigation";
```

```
export async function addPostAction(formData:
FormData) {
  const title = formData.get("title") as string;
  const content = formData.get("content") as
string;
```

```
// සැබෑ ලෝකයේදී මෙහිදී DB එකට සේවි කරයි
console.log("අලුත් ලිපිය:", title, content);
```

```
// පරණ දත්ත Cache එකෙන් ඉවත් කර අලුත්
කරන්න
revalidatePath("/");
// නැවත මුල් පිටුවට යවන්න
redirect("/");
}
```

100.6 පියවර 5: අලුත් ලිපියක් එකතු කිරීමේ පෝරමය (Admin Page)

app/(admin)/add-post/page.tsx

```
import { addPostAction } from "@lib/actions";
```

```
export default function AddPostPage() {
  return (
    <div className="max-w-md mx-auto p-10
bg-slate-50 rounded-2xl shadow-xl mt-10">
      <h2 className="text-2xl font-bold mb-
6">අලුත් ලිපියක් එක් කරන්න</h2>
      <form action={addPostAction}
className="flex flex-col gap-4">
        <input
          name="title"
          placeholder="මාතෘකාව"
          className="p-3 border rounded-lg"
          required
        />
        <textarea
          name="content"
```

```
          placeholder="ලිපිය මෙහි ලියන්න..."
          className="p-3 border rounded-lg h-40"
          required
        />
        <button className="bg-blue-600 text-
white p-3 rounded-lg font-bold hover:bg-blue-
700">
          ප්‍රසිද්ධ කරන්න
        </button>
      </form>
    </div>
  );
}
```

100.7 පියවර 6: සජීවී සංවලනය සහ ප්‍රවේශනය (Loading & Suspense)

පරිශීලකයාට පිටුව ලෝඩ් වන තෙක් පෙන්වීමට අපි Skeleton UI එකක් සකස් කරමු.

app/(main)/loading.tsx

```
export default function Loading() {
  return (
    <div className="max-w-4xl mx-auto p-8
animate-pulse">
      <div className="h-10 bg-slate-200 rounded
w-1/3 mb-8"></div>
      <div className="space-y-6">
        <div className="h-32 bg-slate-200
rounded-xl"></div>
        <div className="h-32 bg-slate-200
rounded-xl"></div>
        <div className="h-32 bg-slate-200
rounded-xl"></div>
      </div>
    </div>
  );
}
```

100.8 පියවර 7: ගතික මෙටා-ඩේටා (Dynamic SEO)

බ්ලොග් ලිපියේ නමට අනුව Google සෙවුම් ප්‍රතිඵල මාරු වන සේ සකසමු.

app/(main)/blog/[slug]/page.tsx

```
import { fetchPostById } from "@lib/data";
import { notFound } from "next/navigation";
```

```
// SEO සඳහා ගතිකව මෙටා-ඩේටා සාදමු
```

```
export async function generateMetadata({
  params }: { params: { slug: string } }) {
  const post = await fetchPostById(params.slug);
  if (!post) return { title: "ලිපිය හමු නොවීය" };

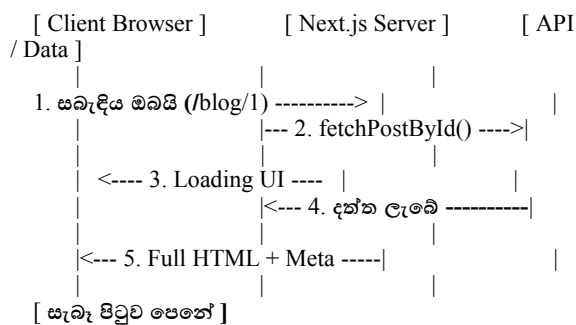
  return {
    title: `${post.title} | InsightBlog`,
    description: post.content.slice(0, 100),
  };
}

export default async function SinglePostPage({
  params }: { params: { slug: string } }) {
  const post = await fetchPostById(params.slug);

  if (!post) notFound(); // 404 පෙන්වන්න

  return (
    <article className="max-w-3xl mx-auto p-10 prose lg:prose-xl">
      <h1 className="text-5xl font-bold">{post.title}</h1>
      <p className="text-gray-500">කර්තෘ: {post.author} | {post.date}</p>
      <hr className="my-6" />
      <div className="text-lg leading-relaxed">{post.content}</div>
    </article>
  );
}
```

100.9 ASCII රූපසටහන: InsightBlog පද්ධති සැකැස්ම



100.10 වෘත්තීය විගණනය සහ ප්‍රසිද්ධ කිරීම (Audit & Deploy)

ඔබේ ව්‍යාපෘතිය නිම වූ පසු:

1. Build පරීක්ෂාව: ටර්මිනලයේ `npm run build` කර කිසිදු වැරද්දක් නැති බව තහවුරු කරන්න.

2. Vercel Deployment: ඔබගේ GitHub ගබඩාව (Repo) Vercel වෙත සම්බන්ධ කර විනාඩි 2කින් සජීවී ලිපිනයක් ලබා ගන්න.
3. Performance Check: Google Lighthouse හරහා ඔබේ SEO සහ Performance ලකුණු පරීක්ෂා කරන්න.

100.11 වෘත්තීය පැහැදිලි කිරීම (Professional Insight)

මෘදුකාංග ඉංජිනේරුවෙකු ලෙස ඔබ මෙහි ඉගෙන ගත් වැදගත්ම දෙය වන්නේ "Static vs Dynamic" සමතුලිතතාවයයි. Next.js මගින් අපට ලිපි ලැයිස්තුව ස්ථාවරව (Cache) තබා ගැනීමටත්, ලිපියක් එකතු කළ සැණින් එම Cache එක මකා දමා (Revalidate) අලුත් දත්ත පෙන්වීමටත් හැකි වේ. මෙය වෘත්තීය ලෝකයේ ISR (Incremental Static Regeneration) ලෙස හඳුන්වයි.

100.12 මුළු ඉදිරිපෙළ සංවර්ධන පාඨමාලාවේ අවසානය

සුභ පැතුම්!

ඔබ දැන් පරිච්ඡේද 1 සිට 100 දක්වා වූ අති දැවැන්ත ගමනක් අවසන් කර ඇත.

- ඔබ ආරම්භ කළේ "පරිගණකයක් යනු කුමක්ද?" යන මූලික පාඩමෙනි.
- අද ඔබ අවසන් කළේ ලෝකයේ දියුණුම ඉදිරිපෙළ තාක්ෂණය වන Next.js මගින් පූර්ණ මෘදුකාංගයක් නිර්මාණය කරමිනි.

ඔබ සතුව දැන්:

1. පිරිසිදු HTML/CSS පදනමක් ඇත.
2. ගැඹුරු JavaScript තර්කනයක් ඇත.
3. React සහ Next.js පිළිබඳ ප්‍රවීණතාවයක් ඇත.
4. Git, GitHub සහ Deployment පිළිබඳ වෘත්තීය පුහුණුවක් ඇත.

ඔබ දැන් Junior Frontend Engineer තනතුරක් සඳහා 200% ක් සුදුසුකම් ලබා ඇත.

වෘත්තීය ඉදිරිපෙළ සංවර්ධනය සහ වෘත්තීය ගමන (Professional Frontend Development and Career Roadmap)

එකසිය එක්වන පරිච්ඡේදය:

වෘත්තීය පෝට්ෆෝලියෝව - ඔබේ දක්ෂතාවයේ කැඩපත (The Professional Portfolio - Your Mirror of Skill)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අධ්‍යයනය කිරීමෙන් පසු ඔබට පහත කරුණු පිළිබඳව පූර්ණ අවබෝධයක් ලැබෙනු ඇත:

- පෝට්ෆෝලියෝවක් (Portfolio) යනු කුමක්ද සහ රැකියාවක් ලබා ගැනීමේදී එහි ඇති තීරණාත්මක වැදගත්කම.
- වෘත්තීය මට්ටමේ පෝට්ෆෝලියෝවක අඩංගු විය යුතු අනිවාර්ය අංග (Components).
- "පෙන්වන්න, පවසන්න එපා" (Show, Don't Tell) යන සංකල්පය වෙබ් අඩවියකට ආදේශ කරන ආකාරය.
- ව්‍යාපෘති විවරණයක් (Case Study) නිවැරදිව ලියන ආකාරය.
- පෝට්ෆෝලියෝ වෙබ් අඩවියක් නිර්මාණය කිරීමේදී අනුගමනය කළ යුතු උසස් UI/UX ප්‍රමිතීන්.
- ඔබේ පෝට්ෆෝලියෝව ලෝකයට විවෘත කරන ආකාරය (Deployment and Promotion).

101.1 හැඳින්වීම (Introduction)

ඔබ ලෝකයේ දක්ෂතම ඉදිරිපෙළ සංවර්ධකයා (Frontend Developer) විය හැකිය. ඔබ සතුව HTML, CSS, JavaScript සහ React පිළිබඳ අතිමහත් දැනුමක් තිබිය හැකිය. නමුත්, ඔබ එම දැනුම භාවිතා කර කුමක් හෝ නිර්මාණය කර ඇති බව ලෝකයට පෙන්වීමට ඔබට නොහැකි නම්, කිසිදු ආයතනයක් හෝ පාරිභෝගිකයෙකු ඔබට රැකියාවක් ලබා දෙන්නේ නැත.

මෘදුකාංග ක්ෂේත්‍රයේදී ඔබ සතු සහතික පත්‍රවලට (Certificates) වඩා ඔබ කළ වැඩවලට (Projects) ලැබෙන වටිනාකම ඉතා වැඩිය. ඔබේ එම සියලුම නිර්මාණ එක තැනක ලස්සනට ප්‍රදර්ශනය කරන

වෙබ් අඩවිය පෝට්ෆෝලියෝව (Portfolio) ලෙස හැඳින්වේ.

101.2 පෝට්ෆෝලියෝවක් යනු කුමක්ද? (Definition of a Portfolio)

පෝට්ෆෝලියෝවක් යනු සංවර්ධකයෙකුගේ දක්ෂතා, පළපුරුද්ද, ඔහු භාවිතා කරන තාක්ෂණයන් සහ ඔහු විසින් නිම කරන ලද ව්‍යාපෘති පිළිබඳ සාක්ෂි අඩංගු වෘත්තීය වෙබ් අඩවියකි.

සරල සිංහල පැහැදිලි කිරීම: මෙය හරියට වෙළඳසැලක ඇති "ප්‍රදර්ශන පුවරුවක්" (Showcase) වැනිය. පාරිභෝගිකයෙකු ඔබේ වෙළඳසැලට ඇතුළු වීමට පෙර එහි ඇති බඩු බාහිරාදිය දෙස බලා ඔබේ ගුණාත්මකභාවය තීරණය කරයි. පෝට්ෆෝලියෝව යනු ඔබේ වෘත්තීය ජීවිතයේ ප්‍රදර්ශන පුවරුවයි.

101.3 ඇයි පෝට්ෆෝලියෝවක් වැදගත් වන්නේ? (Purpose)

- විශ්වාසය ගොඩනැගීම (Building Trust): "මම React දන්නවා" කියා පවසනවාට වඩා React වලින් සාදන ලද වැඩ කරන මෘදුකාංගයක් පෙන්වීම මගින් සේවාදායකයාගේ විශ්වාසය 200% කින් වැඩි වේ.
- පළමු හැඟීම (First Impression): සේවා යෝජකයෙකු (Employer) ඔබව සම්මුඛ පරීක්ෂණයකට කැඳවීමට පෙර බලන පළමු දේ මෙයයි.
- තරඟකාරිත්වය: ලෝකයේ ලක්ෂ සංඛ්‍යාත සංවර්ධකයන් අතරින් ඔබ කැපී පෙනීමට නම් ඔබට ආවේණික වූ සන්නාමයක් (Personal Brand) තිබිය යුතුය.
- මුදල් ඉපයීමේ හැකියාව: ඉහළ මට්ටමේ Freelance ව්‍යාපෘති ලබා ගැනීමට නම් පෝට්ෆෝලියෝවක් අනිවාර්ය වේ.

101.4 පෞරුෂයාගේ වෘත්තීය ව්‍යුහය (Structure of a Professional Portfolio)

වෘත්තීය මට්ටමේ පෞරුෂයාගේ වෘත්තීය අනිවාර්යයෙන්ම පහත සඳහන් කොටස් (Sections) තිබිය යුතුය:

- 1. Hero Section (හඳුන්වාදීමේ කොටස):** තිරයේ මුලින්ම පෙනෙන කොටසයි. මෙහි ඔබේ නම, ඔබ කටුද (උදා: Frontend Engineer), සහ ඔබේ සුවිශේෂීත්වය කෙටියෙන් තිබිය යුතුය.
- 2. About Me (මා ගැන):** ඔබේ තාක්ෂණික ගමන, ඔබ කැමති දේවල් සහ ඔබ වැඩ කරන ආකාරය ගැන කෙටි ඡේදයක්. මෙහිදී "මම ඉතා වෙනෙස වි වැඩ කරන්නෙක්මි" වැනි සාමාන්‍ය වැනි වෙනුවට "මම සංකීර්ණ දත්ත සරල UI බවට පත් කිරීමට කැමති ඉංජිනේරුවෙක්මි" වැනි වෘත්තීය වැනි භාවිතා කරන්න.
- 3. Skills (දක්ෂතා):** ඔබ දන්නා තාක්ෂණයන් (HTML, CSS, JS, React, Tailwind) ලස්සනට අයිකන භාවිතා කර පෙන්වන්න.
- 4. Projects (ව්‍යාපෘති) - [වැදගත්ම කොටස]:** මෙහි අවම වශයෙන් හොඳම ව්‍යාපෘති 3-5ක් තිබිය යුතුය. සෑම ව්‍යාපෘතියක් සඳහාම:
 - පින්තූරයක් (Screenshot).
 - භාවිතා කළ තාක්ෂණයන් (Tech Stack).
 - වෙබ් අඩවියට පිවිසීමට ලින්ක් එකක් (Live Link).
 - කේතය බැලීමට ලින්ක් එකක් (GitHub Link).
- 5. Contact (සම්බන්ධ වන්න):** ඔබේ Email ලිපිනය, LinkedIn ලින්ක් එක සහ පණිවිඩයක් එවීමට Form එකක්.

101.5 ASCII රූපසටහන: පෞරුෂයාගේ සැකැස්ම

[Logo]	[Home]	[About]	[Projects]	[Contact]		
ආයුබෝවන්, මම කසුන් කල්හාර Frontend Architect & UI Engineer [වැඩ බලන්න බොත්තම]						
[දක්ෂතා: React JS CSS Tailwind Git]						
[ව්‍යාපෘති 1] (Weather App)					[ව්‍යාපෘති 2] (E-commerce)	[ව්‍යාපෘති 3] (Task Pro)
අප හා සම්බන්ධ වන්න [Email] [LinkedIn] [GitHub]						

101.6 ව්‍යාපෘති ඉදිරිපත් කිරීමේ කලාව: කේස් ස්ටඩීස් (Case Studies)

නිකම්ම පින්තූරයක් පෙන්වනවාට වඩා, ඔබ එම ව්‍යාපෘතිය කළේ ඇයි දැයි විස්තර කිරීම (Case Study) මගින් ඔබව "Senior" මට්ටමට ඔසවා තබයි. ඒ සඳහා මෙම ආකෘතිය භාවිතා කරන්න:

- **ප්‍රශ්නය (The Problem):** මෙම ඇප් එක හඳුනවේ කුමන ගැටලුව විසඳන්නද?
- **විසඳුම (The Solution):** ඔබ එය විසඳුවේ කෙසේද?
- **තාක්ෂණය (Tech Used):** ඇයි ඔබ React හෝ Tailwind තෝරාගත්තේ?
- **අභියෝග (Challenges):** ඔබට මුහුණ දීමට සිදු වූ අපරාධ තාක්ෂණික ගැටලුව සහ ඔබ එය විසඳූ ආකාරය. (මෙය සේවා යෝජකයන් ඉතා කැමැත්තෙන් කියවන කොටසකි).

101.7 පෞරුෂයාගේ ව්‍යුහය සඳහා ප්‍රෝ-මට්ටමේ උපදෙස් (Professional Tips)

- 1. Quality over Quantity:** බාල ව්‍යාපෘති 20ක් පෙන්වනවාට වඩා ඉතා උසස් තත්ත්වයේ ව්‍යාපෘති 3ක් පෙන්වීම වටී.
- 2. Responsive Design:** ඔබේ පෞරුෂයාගේ ව්‍යුහය ජංගම දුරකථනයෙන් බලන විට 100% ක් නිවැරදිව දිස්විය යුතුය. මන්ද බොහෝ සේවා යෝජකයන් පළමුව ලින්ක් එක විවෘත කරන්නේ දුරකථනයෙනි.
- 3. Speed (වේගය):** පෞරුෂයාගේ ව්‍යුහය තත්පර 1-2ක් ඇතුළත ලෝඩ් විය යුතුය. පින්තූර WebP වර්ගයෙන් භාවිතා කරන්න.
- 4. Blog (අත්‍යවශ්‍ය නොවේ):** ඔබට හැකි නම් තාක්ෂණික ලිපි කිහිපයක් ලියා පෞරුෂයාගේ ව්‍යුහය එක් කරන්න. එයින් ඔබ විෂය පිළිබඳව කෙතරම් ගැඹුරින් දන්නවාද යන්න පැහැදිලි වේ.
- 5. No Broken Links:** කිසිම ලින්ක් එකක් වැඩ නොකර තිබිය නොහැක. එය ඉතා අසම්මත දෙයකි.

101.8 වෘත්තීය පිරික්සුම් ලැයිස්තුව (Portfolio Checklist)

ඔබේ පෞරුෂයාගේ ව්‍යුහය ප්‍රසිද්ධ කිරීමට පෙර මේවා පරීක්ෂා කරන්න:

- [] මගේ පැහැදිලි ජායාරූපයක් හෝ වෘත්තීය ලාංඡනයක් තිබේද?
- [] මගේ CV (ජීව දත්ත පත්‍රිකාව) බාගත කිරීමට ලින්ක් එකක් තිබේද?

- [] සෑම ව්‍යාපෘතියකටම GitHub ලින්ක් එක තිබේද?
- [] අකුරු කියවීමට පහසුද (Typography)?
- [] සම්බන්ධ කර ගත හැකි විස්තර නිවැරදිද?

101.9 සාරාංශය (Key Points)

- පෝට්ෆෝලියෝව යනු ඔබේ දක්ෂතා පෙන්වන වෘත්තීය වෙබ් අඩවියයි.
- සහතික වලට වඩා සැබෑ ව්‍යාපෘති (Projects) වැදගත් වේ.
- Case Studies මගින් ඔබේ තාක්ෂණික තර්කනය (Logic) පෙන්විය හැක.
- පෝට්ෆෝලියෝව ඉතා වේගවත් සහ Responsive විය යුතුය.
- පෝට්ෆෝලියෝව යනු නිරන්තරයෙන් අලුත් කළ යුතු (Update) දෙයකි.

101.10 අභ්‍යාස (Exercises)

1. ඔබ මෙතෙක් නිර්මාණය කළ ව්‍යාපෘති අතරින් හොඳම 3 තෝරා ගන්න.
2. එම ව්‍යාපෘති 3 සඳහා "ප්‍රශ්නය, විසඳුම, සහ භාවිතා කළ තාක්ෂණය" ඇතුළත් කෙටි විස්තරයක් ලියන්න.
3. ඔබේ පෝට්ෆෝලියෝ වෙබ් අඩවියේ තිබිය යුතු වර්ණ සහ අකුරු වර්ග (Fonts) තෝරා ගන්න (Color Palette).
4. පෝට්ෆෝලියෝ වෙබ් අඩවියක තිබිය යුතු වැදගත්ම කොටස කුමක්ද? හේතු දක්වන්න.

101.11 සම්මුඛ පරීක්ෂණ ප්‍රශ්න (Interview Questions)

- ප්‍රශ්නය: "ඔබේ පෝට්ෆෝලියෝවේ ඇති අසමාරුම ව්‍යාපෘතිය කුමක්ද? එහිදී ඔබ මුහුණ දුන් තාක්ෂණික අභියෝගය කුමක්ද?"
- පිළිතුර: මෙයට පිළිතුරු දීමට ඔබ කලින් ලියූ Case Study එක උදව් වේ. ඔබ මුහුණ දුන් API error එකක් හෝ State management ගැටලුවක් ගැන සහ ඔබ එය විසඳූ ආකාරය පැහැදිලිව විස්තර කරන්න.

101.12 ගෙදර වැඩ (Homework)

Vite, React සහ Tailwind CSS භාවිතා කර ඔබේ පෝට්ෆෝලියෝ වෙබ් අඩවිය නිර්මාණය කිරීම ආරම්භ කරන්න. එහි Content එක සැබෑ ඒවා විය යුතුය (ලෝරම් ඉප්සම් - Lorem Ipsum භාවිතා නොකරන්න). ලබන සතිය වන විට එය Netlify හෝ Vercel හරහා සජීවීව ප්‍රසිද්ධ කරන්න.

එකසිය දෙවන පරිච්ඡේදය : නිදහස් වෘත්තිකයෙකු ලෙස ගමන ආරම්භ කිරීම (Entering the World of Freelancing)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

1. නිදහස් වෘත්තීය හෙවත් Freelancing යනු කුමක්දැයි වෘත්තීය මට්ටමින් තේරුම් ගැනීම.
2. ලෝකයේ ජනප්‍රියම නිදහස් වෘත්තීය වෙදිකා (Freelance Platforms) හඳුනා ගැනීම.
3. Fiverr සහ Upwork අතර ඇති තාක්ෂණික වෙනස වටහා ගැනීම.
4. වෘත්තීය මට්ටමේ ගිණුමක් (Profile) සකස් කරන ආකාරය පියවරෙන් පියවර ඉගෙන ගැනීම.
5. ඔබගේ සේවාවන් විකිණීම සඳහා "ගිග්" (Gig) එකක් නිර්මාණය කරන ආකාරය.
6. විදේශ මුදල් ලංකාවට ගෙන්වා ගන්නා ආකාරය (Payment Gateways).
7. පාරිභෝගිකයන් සමඟ ගනුදෙනු කිරීමේදී සැලකිලිමත් විය යුතු සදාචාරාත්මක නීති.

102.1 හැඳින්වීම: නිදහස් වෘත්තීය යනු කුමක්ද? (What is Freelancing?)

ඔබ මෙතෙක් ඉගෙන ගත් තාක්ෂණික දැනුම (HTML, CSS, JS, React) භාවිතා කර මුදල් ඉපයීමට ඇති පහසුම සහ වේගවත්ම ක්‍රමය වන්නේ Freelancing (නිදහස් වෘත්තීය) යි.

සාමාන්‍ය රැකියාවකදී ඔබ එක් ආයතනයකට උදේ 8 සිට සවස 5 දක්වා සේවය කරයි. නමුත් නිදහස් වෘත්තිකයෙකු (Freelancer) ලෙස ඔබ කිසිදු ආයතනයකට බැඳී නැත. ඔබ කරන්නේ ලෝකයේ ඕනෑම රටක සිටින පාරිභෝගිකයන්ගේ වෙබ් අඩවි නිර්මාණය කර දී, ඒ සඳහා කෙලින්ම මුදල් ලබා ගැනීමයි.

සැබෑ ලෝකයේ උපමාව: සිතන්න ඔබ දක්ෂ වඩු කාර්මිකයෙක් කියා.

- ඔබ ගෘහ භාණ්ඩ සාප්පුවක වැඩ කරනවා නම් ඔබ "සේවකයෙක්".

- නමුත් ඔබ ඔබේම වැඩපොළක සිට විවිධ අයගේ ඇණවුම් (Orders) භාරගෙන ගෘහ භාණ්ඩ හදා දෙනවා නම් ඔබ "නිදහස් වෘත්තිකයෙක්".

Frontend Developer කෙනෙකු ලෙස ඔබට දැන් ලෝකය පුරා සිටින ඕනෑම අයෙකුට වෙබ් අඩවි සාදා දීමේ හැකියාව ඇත.

102.2 ඇයි නිදහස් වෘත්තීයයක් තෝරා ගන්නේ? (Purpose and Benefits)

1. ස්වාධීනත්වය (Flexibility): ඔබට කැමති වේලාවක, කැමති තැනක සිට (නිවසේ සිට වුවද) වැඩ කළ හැක.
2. විදේශීය ආදායම (Global Income): ලංකාවේ සිටිමින් ඇමෙරිකානු ඩොලර් (USD) වලින් මුදල් ඉපයීමට හැකි වීම නිසා ඉහළ ආදායමක් ලැබිය හැක.
3. අත්දැකීම් (Experience): විවිධ රටවල පාරිභෝගිකයන් සමඟ වැඩ කිරීමෙන් ඔබේ තාක්ෂණික දැනුම ඉතා වේගයෙන් වර්ධනය වේ.
4. පාලනය (Control): ඔබ වැඩ කරන ව්‍යාපෘතිය සහ එහි මිල තීරණය කරන්නේ ඔබමය.

102.3 ප්‍රධාන නිදහස් වෘත්තීය වෙබ්සයිට්ස් (Top Platforms)

ලෝකයේ පාරිභෝගිකයන් (Clients) සහ සංවර්ධකයන් (Developers) එකිනෙක මුණගස්වන වෙබ් අඩවි කිහිපයක් ඇත. මින් ප්‍රධාන ඒවා දෙකකි:

1. Fiverr (ෆයිවර්)

මෙහිදී ඔබ කරන්නේ "මම වෙබ් අඩවියක් සාදා දෙමි" යනුවෙන් වෙළඳ දැන්වීමක් දැමීමයි. පාරිභෝගිකයා පැමිණ ඔබට තෝරා ගනී. මෙය ආරම්භකයින්ට ඉතා සුදුසුය.

- සේවාව හඳුන්වන නම: Gig (ගිග්).

2. Upwork (අප්වර්ක්)

මෙහිදී පාරිභෝගිකයා තමාට අවශ්‍ය වැඩය ගැන දැන්වීමක් දමයි. ඔබ ගොස් එම වැඩය ලබා ගැනීමට තරඟ කළ යුතුය. මෙය මදක් පළපුරුදු අයට සුදුසුය.

- සේවාව ලබා ගන්නා ක්‍රමය: Bidding (ලංසු තැබීම) / Proposals (යෝජනා ඉදිරිපත් කිරීම).

102.4 වෘත්තීය විස්තර පත්‍රිකාවක් (Profile) සකසන ආකාරය

ඔබේ ප්‍රොෆයිල් (Profile) එක යනු ඔබේ "අතටා වෙළඳසැල" (Virtual Shop) වේ. එය ඉතා පිරිසිදු සහ විශ්වාසවන්ත විය යුතුය.

පියවරෙන් පියවර මගපෙන්වීම:

1. වෘත්තීය ඡායාරූපය: පැහැදිලි, සිනහමුහුණු මුහුණක් සහිත ඡායාරූපයක් භාවිතා කරන්න. (Selfie ඡායාරූප භාවිතා නොකරන්න).
2. මාතෘකාව (Title): ඔබ කවුදැයි කෙටියෙන් කියන්න.

- උදා: "Professional React & Tailwind CSS Developer".

3. හැඳින්වීම (Description): ඔබ ලබා දෙන සේවාව සහ ඔබේ දක්ෂතා ගැන ඡේද 2ක විස්තරයක් ලියන්න.

- වෘත්තීය ඉඟිය: "මම වෙබ් අඩවි හඳුනවා" කියනවාට වඩා "මම ඔබේ ව්‍යාපාරය දියුණු කිරීමට නවීන සහ වේගවත් වෙබ් අඩවි නිර්මාණය කරනවා" ලෙස ලියන්න.

4. දක්ෂතා (Skills): HTML5, CSS3, JavaScript, React.js, Tailwind CSS, Responsive Design යන සියල්ල ඇතුළත් කරන්න.

102.5 පළමු "ගිග්" (Gig) එක නිර්මාණය කිරීම

Fiverr වැනි අඩවියක ඔබ ලියන දැන්වීම (Gig) සාර්ථක වීමට නම් මෙම කරුණු තිබිය යුතුය:

- Gig Title (මාතෘකාව): පාරිභෝගිකයන් සර්ව (Search) කරන වචන භාවිතා කරන්න.

- උදා: "I will build a modern responsive website using React and Tailwind CSS".

- Gig Image (ඡායාරූපය): ඔබ නිර්මාණය කළ වෙබ් අඩවියක පැහැදිලි පින්තූරයක් (Thumbnail) භාවිතා කරන්න. එය ඇසට හසුවන (Eye-catching) එකක් විය යුතුය.

- Pricing (මිල ගණන්): ආරම්භයේදී මිල මදක් අඩුවෙන් තබා පසුව වැඩි කරන්න.

- Basic: සරල එක් පිටුවක වෙබ් අඩවියක්.

- Standard: පිටු 3-5 කින් යුත් වෙබ් අඩවියක්.

- Premium: සම්පූර්ණ සංකීර්ණ වෙබ් අඩවියක්.

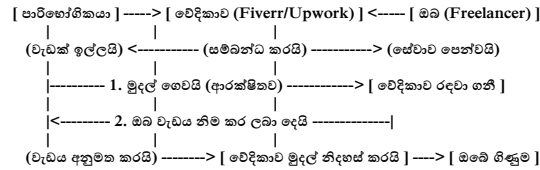
102.6 මුදල් ගෙත්වා ගන්නා ආකාරය (Getting Paid)

ලංකාවේ සිටින ඔබට ඔබ උපයන ඩොලර් ලබා ගැනීමට ක්‍රම කිහිපයක් ඇත:

1. Payoneer (පයනියර්): ලංකාවේ බොහෝ Freelancers ලා භාවිතා කරන්නේ මෙයයි. මෙය හරහා මුදල් ඔබේ ලංකාවේ බැංකු ගිණුමට කෙලින්ම ගෙන්වා ගත හැක.

2. Direct Bank Transfer: Upwork වැනි අඩවි වලින් කෙලින්ම ලංකාවේ බැංකුවකට මුදල් එවිය හැක (නමුත් ඒ සඳහා අමතර ගාස්තු අය වේ).

102.7 ASCII රූපසටහන: Freelancing ක්‍රියාවලිය



102.8 වෘත්තීය උපදෙස් (Professional Tips)

- Niche එකක් තෝරා ගන්න: "මම ඕනෑම වෙබ් අඩවියක් හඳුනවා" කියනවාට වඩා "මම ආපනශාලා සඳහා (Restaurants) විශේෂ වෙබ් අඩවි හඳුනවා" වැනි විශේෂිත දෙයක් තෝරා ගැනීමෙන් වැඩි ඇණවුම් ලබා ගත හැක.
- ඉක්මනින් පිළිතුරු දෙන්න: පාරිභෝගිකයෙකු පණිවිඩයක් එවූ සැණින් පිළිතුරු දීමට උත්සාහ කරන්න. (Mobile App එක භාවිතා කරන්න).
- අවංක වන්න: ඔබට කළ නොහැකි වැඩ භාර නොගන්න. එක තරක ප්‍රතිපෝෂණයක් (Negative Review) ඔබේ ගමනට බාධාවක් විය හැක.

102.9 පොදු වැරදි (Common Mistakes)

1. වෙනත් අයගේ පින්තූර සොරකම් කිරීම: වෙනත් අයෙකුගේ Portfolio එකක් ඔබේ ලෙස පෙන්වීමෙන් වළකින්න. එය හසු වූ සැණින් ඔබේ ගිණුම තහනම් (Ban) වේ.
2. Communication වැරදීම: පාරිභෝගිකයාට අවශ්‍ය දේ හරියටම තේරුම් නොගෙන වැඩ ආරම්භ කිරීම.
3. වේදිකාවෙන් පිටත මුදල් ගනුදෙනු කිරීම: පාරිභෝගිකයා "අපි WhatsApp වලින් කතා කරමු, මම කෙලින්ම සල්ලි එවන්නම්" කීවොත් එයට එකඟ නොවන්න. එය බොහෝ විට වංචාවක් (Scam) විය හැක.

102.10 සාරාංශය (Key Points)

- Freelancing යනු ලොව ඕනෑම අයෙකුට සේවය ලබා දී මුදල් ඉපයීමයි.
- Fiverr සහ Upwork යනු ප්‍රධාන වේදිකාවන් වේ.
- සාර්ථකත්වයේ රහස්‍ය වන්නේ හොඳ Profile එකක් සහ Portfolio එකක් තිබීමයි.
- මුදල් ලබා ගැනීමට Payoneer වැනි පද්ධතියක් අවශ්‍ය වේ.

102.11 අභ්‍යාස (Exercises)

1. Fiverr වෙබ් අඩවියට ගොස් "Frontend Developer" ලෙස සර්ච් කර, එහි සිටින ඉහළම ශ්‍රේණිගත (Top Rated) සංවර්ධකයන් පස් දෙනෙකුගේ Profile අධ්‍යයනය කර සටහන් කරගන්න.
2. ඔබේ සේවාව ගැන විස්තර කරන කෙටි ඡේදයක් (Bio) ඉංග්‍රීසියෙන් ලියන්න.
3. ඔබේ පළමු Gig එක සඳහා භාවිතා කළ හැකි පින්තූරයක් (Thumbnail) නිර්මාණය කිරීමට උත්සාහ කරන්න (Canva වැනි මෙවලමක් භාවිතා කළ හැක).

මිලිභ පරිච්ඡේදයෙන් අපි පාරිභෝගිකයන් සමඟ නිවැරදිව කතාබස් කරන ආකාරය සහ ඔවුන්ගේ අවශ්‍යතාවයන් තේරුම් ගන්නා ආකාරය (Client Communication) ඉගෙන ගනිමු.

එකසිය නෙවන පරිච්ඡේදය: පාරිභෝගික සන්නිවේදනය සහ මිල තීරණය කිරීම (Client Communication and Pricing Strategies)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

1. පාරිභෝගිකයෙකු සමඟ වෘත්තීය මට්ටමින් සන්නිවේදනය (Communication) කිරීමේ වැදගත්කම.
2. අවශ්‍යතා එක්රැස් කිරීම (Requirement Gathering) සහ නිවැරදි ප්‍රශ්න ඇසීමේ කලාව.
3. වැඩ සීමාව (Scope of Work) නිර්වචනය කිරීම සහ "Scope Creep" මගින් සිදුවන පාඩු වළක්වා ගැනීම.
4. ඔබගේ සේවාවන් සඳහා නිවැරදි මිලක් තීරණය කරන ආකාරය (Pricing Strategies).
5. පැයකට අය කිරීම (Hourly Rate) සහ ස්ථාවර මිල (Fixed Price) අතර වෙනස සහ ඒවා තෝරාගත යුතු අවස්ථා.
6. පාරිභෝගිකයා සමඟ සාර්ථකව මිල සාකච්ඡා (Negotiation) කරන ආකාරය.

103.1 හැඳින්වීම (Introduction)

බොහෝ දෙනෙක් සිතන්නේ වෙබ් සංවර්ධකයෙකුගේ සාර්ථකත්වය රඳා පවතින්නේ ඔහු දත්තා කේතකරණ (Coding) දැනුම මත පමණක් බවයි. නමුත් ඇත්තම තත්ත්වය නම්, සාර්ථක මෘදුකාංග ඉංජිනේරුවෙකු යනු කේත ලිවීමට මෙන්ම මිනිසුන් සමඟ ගනුදෙනු කිරීමට ද දක්ෂ අයෙකි.

ඔබ කොතරම් හොඳින් වෙබ් අඩවියක් නිර්මාණය කළත්, පාරිභෝගිකයාට අවශ්‍ය දේ ඔබ තේරුම් ගත්තේ නැතිනම් එම ව්‍යාපෘතිය අසාර්ථක වේ. එබැවින්, මෙම පරිච්ඡේදයෙන් අපි ඉගෙන ගන්නේ ඔබේ තාක්ෂණික දැනුම පාරිභෝගිකයාගේ ව්‍යාපාරික අවශ්‍යතාවයක් බවට පරිවර්තනය කරන්නේ කෙසේද සහ ඒ සඳහා නිවැරදි වටිනාකමක් ලබා ගන්නේ කෙසේද යන්නයි.

103.2 අවශ්‍යතා එක්රැස් කිරීම (Requirement Gathering)

පාරිභෝගිකයා "මට වෙබ් අඩවියක් ඕනේ" යැයි පැවසූ සැණින් කේත ලිවීම ආරම්භ නොකරන්න. මුලින්ම ඔහුට අවශ්‍ය කුමක්දැයි පැහැදිලිව දැනගත යුතුය.

ඇසිය යුතු ප්‍රධාන ප්‍රශ්න 5:

1. වෙබ් අඩවියේ අරමුණ කුමක්ද? (භාණ්ඩ විකිණීමටද? තොරතුරු ලබා දීමටද? සේවාවක් ප්‍රචාරණය කිරීමටද?)
2. ඔබේ ඉලක්කගත පාරිභෝගිකයන් කවුද? (Target Audience)
3. ඔබ කැමති වර්ණ සහ මෝස්තර මොනවාද? (Design Preferences)
4. වෙබ් අඩවියේ තිබිය යුතු අත්‍යවශ්‍යම විශේෂාංග (Features) මොනවාද? (උදා: Contact form, Payment gateway).
5. ඔබේ අයවැය (Budget) සහ කාලසීමාව (Deadline) කොපමණද?

වෘත්තීය උපදෙස: මෙම තොරතුරු ලබා ගැනීමට සරල ප්‍රශ්නාවලියක් (Questionnaire) හෝ Google Form එකක් භාවිතා කිරීම ඔබ වෘත්තීය මට්ටමේ අයෙකු බව පාරිභෝගිකයාට හැඟවීමට උදව් වේ.

103.3 වැඩ සීමාව තීරණය කිරීම (Defining the Scope)

ප්‍රතික්ෂේප කළ නොහැකි වැදගත්ම ලියවිල්ල වන්නේ Scope of Work (SOW) ය. මෙයින් පවසන්නේ ඔබ එම මුදලට කර දෙන වැඩ කොටස කුමක්ද යන්නයි.

Scope Creep (වැඩ සීමාව ඉක්මවා යාම):

ව්‍යාපෘතිය මැදදී පාරිභෝගිකයා පවසනවා, "තව මේ බොත්තමකුත් දාන්න, මේ පිටුවත් හදන්න" කියා. ඔබ මුලින්ම වැඩ සීමාව ලියා අත්සන් කර (හෝ Message එකකින් තහවුරු කර) නැතිනම්, ඔබට එම අමතර වැඩ නොමිලේ කිරීමට සිදු වේ. මෙය ඔබේ කාලය සහ මුදල නාස්ති කිරීමකි.

103.4 මිල තීරණය කිරීමේ උපාය මාර්ග (Pricing Strategies)

ඔබේ සේවාවට මිලක් නියම කිරීමට ප්‍රධාන ක්‍රම 3ක් ඇත.

1. පැයකට අය කිරීම (Hourly Rate)

ඔබ වැඩ කරන සෑම පැයකටම නිශ්චිත මුදලක් (උදා: \$15/hour) අය කිරීමයි.

- සුදුසු අවස්ථාව: ව්‍යාපෘතියේ වැඩ කොටස හරියටම නිශ්චිත නැති විට සහ දිගුකාලීන වැඩ වලදී.

2. ස්ථාවර මිල (Fixed Price)

මුළු ව්‍යාපෘතියටම එකම මිලක් (උදා: මුළු වෙබ් අඩවියටම \$500) නියම කිරීමයි.

- සුදුසු අවස්ථාව: ආරම්භකයින්ට මෙය ඉතා සුදුසුය. පාරිභෝගිකයා ද කැමති මුලින්ම මිල දැන ගැනීමටයි.

3. වටිනාකම මත පදනම් වූ මිල (Value-based Pricing)

පාරිභෝගිකයාට එම වෙබ් අඩවියෙන් ලැබෙන ලාභය අනුව මිල තීරණය කිරීමයි.

- උදා: ඔබ හදන වෙබ් අඩවිය නිසා පාරිභෝගිකයා ලක්ෂ 10ක ලාභයක් ලබන්නේ නම්, ඔබට එම වෙබ් අඩවියට ලක්ෂ 2ක් වුවද අය කළ හැක.

103.5 සාර්ථක මිල සාකච්ඡාවක් (Negotiation) කරන්නේ කෙසේද?

පාරිභෝගිකයා සැමවිටම මිල අඩු කිරීමට උත්සාහ කරනු ඇත. එවැනි විටකදී:

- මිල අඩු නොකර වැඩ කොටස අඩු කරන්න: පාරිභෝගිකයාට මුදල් මදි නම්, "මම මිල අඩු කරන්නම්, හැබැයි මේ විශේෂාංග දෙක දැනට අයින් කරමු" යැයි පවසන්න.

- වෘත්තීය මට්ටම පෙන්වන්න: ඔබ ඉහළ මිලක් අය කරන්නේ ඇයිදැයි පැහැදිලි කරන්න. (උදා: ඔබ සාදන වෙබ් අඩවිය ජංගම දුරකථන වලට 100% ක් ගැලපෙන බව (Responsive) සහ වේගවත් බව පවසන්න).

103.6 ASCII වගුව: මිල තීරණය කිරීමේ සංසන්දනය

ලක්ෂණය	පැයකට අය කිරීම (Hourly)	ස්ථාවර මිල (Fixed Price)
අවදානම (Risk)	අඩුයි (වැඩ කරන වෙලාවට මුදල් ලැබේ)	වැඩියි (වැඩ වැඩි වූණෙන පාඩු විය හැක)
පාරිභෝගික කැමැත්ත	අඩුයි (අවසාන මිල ගැන සැකයක් ඇති වේ)	වැඩියි (මුළු මිල කලින්ම දන්නා නිසා)
සුදුසු මට්ටම	මධ්‍යම / ජ්‍යෙෂ්ඨ	ආරම්භක (Junior)

103.7 සන්නිවේදන මෙවලම් (Communication Tools)

වෘත්තීය මට්ටමේදී ඔබ Facebook හෝ WhatsApp වලට වඩා පහත මෙවලම් භාවිතා කිරීමට හුරු විය යුතුය:

- Slack / Discord: පණිවිඩ හුවමාරු කර ගැනීමට.
- Zoom / Google Meet: විඩියෝ ඇමතුම් හරහා විස්තර සාකච්ඡා කිරීමට.
- Trello / Asana: ව්‍යාපෘතියේ ප්‍රගතිය පාරිභෝගිකයාට පෙන්වීමට (Project Tracking).
- Email: නිල ලියකියවිලි සහ එකඟතාවයන් යැවීමට.

103.8 වෘත්තීය උපදෙස් (Professional Tips)

- "නැහැ" කීමට පුරුදු වන්න: ඔබට කළ නොහැකි වැඩක් හෝ ඔබේ වටිනාකමට වඩා ඉතා අඩු මුදලක් දෙන වැඩ ප්‍රතික්ෂේප කරන්න. එය ඔබේ වෘත්තීය ගෞරවය රැක ගැනීමට උදව් වේ.
- තැන්පතුක් (Deposit) ලබා ගන්න: කිසිම විටක මුදල් ලබා නොගෙන වැඩ ආරම්භ නොකරන්න. අවම වශයෙන් මුළු මුදලින් 25% - 50% ක තැන්පතුක් මුලින් ලබා ගන්න.
- Update කරන්න: වැඩ අවසන් වන තෙක් නොසිට, සෑම දින 2කට වරක් ඔබ කළ වැඩ ගැන පාරිභෝගිකයාට කෙටි පණිවිඩයක් යවන්න.

103.9 පොදු වැරදි (Common Mistakes)

1. Underpricing (ඉතා අඩු මිලක් කීම): ඔබ මුලින්ම ඉතා අඩු මිලක් කීවාත්, පාරිභෝගිකයා සිතන්නේ ඔබේ දැනුම අඩු බවයි.
2. Over-promising (අසත්‍ය පොරොන්දු): දින 2කින් කළ නොහැකි වැඩක් දින 2කින් කරනවා යැයි පොරොන්දු වීම.
3. නොපැහැදිලි සන්නිවේදනය: පාරිභෝගිකයා දන්නා තාක්ෂණික වචන (උදා: Context API, Virtual DOM) භාවිතා නොකර, ඔහුට තේරෙන සරල වචන වලින් තාක්ෂණය පැහැදිලි කරන්න.

103.10 සාරාංශය (Key Points)

- සාර්ථකත්වයට තාක්ෂණික දැනුම මෙන්ම මෘදු කුසලතා (Soft Skills) ද වැදගත් වේ.
- අවශ්‍යතා ලබා ගැනීමට නිවැරදි ප්‍රශ්න අසන්න.
- වැඩ සීමාව (Scope) කලින්ම පැහැදිලිව සටහන් කරගන්න.
- ඔබේ පළපුරුද්ද සහ ව්‍යාපෘතියේ ස්වභාවය අනුව මිල තීරණය කරන්න.
- මුලින් තැන්පතුවක් ලබා ගැනීම අනිවාර්ය වේ.

103.11 අභ්‍යාස (Exercises)

1. පාරිභෝගිකයෙකුට යැවිය හැකි සරල අවශ්‍යතා පිරික්සුම් ලැයිස්තුවක් (Requirement Checklist) සිංහලෙන් සහ ඉංග්‍රීසියෙන් සකස් කරන්න.
2. ඔබේ පළමු වෙබ් අඩවිය නිර්මාණය කිරීමට පාරිභෝගිකයෙකු පැමිණියේ යැයි සිතන්න. ඔබ අය කරන ස්ථාවර මිල (Fixed Price) සහ එයට හේතු 3ක් ලියන්න.
3. පාරිභෝගිකයෙකු "මිල ගොඩක් වැඩියි, මට මේක මීට වඩා අඩුවට ඕනේ" කීවාත් ඔබ ඔහුට ලබා දෙන වෘත්තීය පිළිතුර කුමක්ද?

මිළඟ පරිච්ඡේදයෙන් අපි මෘදුකාංග සමාගමක රැකියාවක් ලබා ගැනීමට අවශ්‍ය සම්මුඛ පරීක්ෂණ සඳහා සුදානම් වන ආකාරය (Interview Preparation) ඉතාමත් විස්තරාත්මකව ඉගෙන ගනිමු.

එකසිය හතරවන පරිච්ඡේදය: සම්මුඛ පරීක්ෂණ සඳහා සූදානම් වීම - රැකියාව ලබා ගැනීමේ කලාව (Interview Preparation - Master the Art of Getting Hired)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

1. මෘදුකාංග ක්ෂේත්‍රයේ සම්මුඛ පරීක්ෂණ (Interviews) පවත්වන විවිධ අදියරයන් හඳුනා ගැනීම.
2. ඉදිරිපෙළ සංවර්ධකයෙකුගෙන් (Frontend Developer) අසනු ලබන පොදු තාක්ෂණික ප්‍රශ්නවලට පිළිතුරු දීමට පුහුණු වීම.
3. කේතකරණ අභියෝග (Coding Challenges) සහ Live Coding අවස්ථාවලට මුහුණ දෙන ආකාරය.
4. "මෘදු කුසලතා" (Soft Skills) සහ වර්යාත්මක ප්‍රශ්නවලට (Behavioral Questions) පිළිතුරු දෙන ආකාරය.
5. "STAR" ක්‍රමය භාවිතා කර ඔබගේ අත්දැකීම් විස්තර කරන ආකාරය.
6. සම්මුඛ පරීක්ෂණයකට පෙර සහ පසු අනුගමනය කළ යුතු වෘත්තීය ආචාරධර්ම.

104.1 හැඳින්වීම: සම්මුඛ පරීක්ෂණයක් යනු කුමක්ද?

සම්මුඛ පරීක්ෂණයක් (Interview) යනු හුදෙක් ඔබෙන් ප්‍රශ්න අසන අවස්ථාවක් පමණක් නොවේ. එය ඔබගේ තාක්ෂණික දැනුම (Hard Skills) සහ මිනිසුන් සමඟ වැඩ කිරීමේ හැකියාව (Soft Skills) යන දෙකම පරීක්ෂා කරන "ගැලපීම් පරීක්ෂණයකි".

වෘත්තීය Frontend Developer කෙනෙකු ලෙස, ඔබ ලියන කේතය (Code) මෙන්ම, එම කේතය ලියුවේ ඇයි දැයි පැහැදිලි කිරීමට ඇති හැකියාව රැකියාව ලබා ගැනීමට ඉතා වැදගත් වේ.

104.2 සම්මුඛ පරීක්ෂණයේ ප්‍රධාන අදියර (The Interview Process)

විශාල මෘදුකාංග සමාගමක (Software Company) සම්මුඛ පරීක්ෂණයක් සාමාන්‍යයෙන් පියවර කිහිපයකින් සමන්විත වේ:

1. HR Screening (මූලික පරීක්ෂාව): ඔබගේ CV එකේ ඇති තොරතුරු නිවැරදි දැයි සහ ඔබ එම රැකියාවට මූලිකව සුදුසු දැයි පරීක්ෂා කිරීම.
2. Technical Phone Screen / Quiz: මූලික තාක්ෂණික ප්‍රශ්න කිහිපයක් දුරකථනයෙන් හෝ Online Test එකක් හරහා ඇසීම.
3. Take-home Assignment: ඔබට කුඩා ව්‍යාපෘතියක් (උදා: සරල වෙබ් පිටුවක් නිර්මාණය කිරීම) නිවසේ සිට කිරීමට ලබා දීම.
4. Technical Interview (තාක්ෂණික සම්මුඛ පරීක්ෂණය): ජ්‍යෙෂ්ඨ ඉංජිනේරුවන් (Senior Engineers) සමඟ මුහුණට මුහුණ (හෝ වීඩියෝ ඇමතුම් හරහා) කතා කරමින් ගැඹුරු තාක්ෂණික කරුණු සාකච්ඡා කිරීම.
5. Cultural Fit / Behavioral Interview: ඔබ එම කණ්ඩායම සමඟ හොඳින් වැඩ කළ හැකි පුද්ගලයෙකු දැයි පරීක්ෂා කිරීම.

104.3 ප්‍රධාන තාක්ෂණික ප්‍රශ්න සහ පිළිතුරු (Core Technical Questions)

සම්මුඛ පරීක්ෂණයකට යාමට පෙර ඔබ පහත ප්‍රශ්න සඳහා පිළිතුරු සූදානම් කරගත යුතුය:

(අ) HTML සහ ප්‍රවේශ්‍යතාව (HTML & Accessibility)

- ප්‍රශ්නය: "Semantic HTML වැදගත් වන්නේ ඇයි?"

- පිළිතුර: SEO සඳහා සහ පෙරිමි දුර්වල අය භාවිතා කරන Screen Reader මෘදුකාංගවලට වෙබ් අඩවිය නිවැරදිව කියවා දීමට මෙය අත්‍යවශ්‍ය වේ.

- ප්‍රශ්නය: "Meta Tags වල කාර්යභාරය පැහැදිලි කරන්න."

(ආ) CSS සහ පිරිසැලසුම් (CSS & Layouts)

- ප්‍රශ්නය: "Flexbox සහ Grid අතර වෙනස කුමක්ද? ඔබ ඒවා භාවිතා කරන්නේ කුමන අවස්ථාවලදීද?"

- පිළිතුර: Flexbox එක දිශාවකට (1D) දේවල් පෙළගැස්වීමටත්, Grid දිශා දෙකකට (2D - ජේෂ්‍රි සහ තීරු) දේවල් පෙළගැස්වීමටත් භාවිතා කරයි.

- ප්‍රශ්නය: "CSS Specificity යනු කුමක්ද?"

(ඇ) JavaScript සහ Logic (ඡායාස්තූප්ව)

- ප්‍රශ්නය: "==" සහ === අතර වෙනස කුමක්ද?"

- ප්‍රශ්නය: "Closure එකක් යනු කුමක්ද?"

උදාහරණයක් දෙන්න."

- ප්‍රශ්නය: "Asynchronous JavaScript ක්‍රියා කරන්නේ කෙසේද? (Event Loop, Promises)."

(ඇ) React.js (රියැක්ට්)

- ප්‍රශ්නය: "Virtual DOM යනු කුමක්ද සහ එය රියැක්ට් වේගවත් කරන්නේ කෙසේද?"

- ප්‍රශ්නය: "useState සහ useEffect භාවිතා කළ යුත්තේ කුමන අවස්ථාවලදීද?"

- ප්‍රශ්නය: "Props සහ State අතර ඇති වෙනස කුමක්ද?"

104.4 STAR ක්‍රමය: අත්දැකීම් විස්තර කිරීමේ රහස

සම්මුඛ පරීක්ෂණයේදී ඔබෙන් "ඔබ මුහුණ දුන් අමාරුම තාක්ෂණික ගැටලුව කුමක්ද?" වැනි ප්‍රශ්න අසනු ඇත. එවිට පිළිතුරු දීමට STAR ක්‍රමය භාවිතා කරන්න:

- S - Situation (අවස්ථාව): ඔබ මුහුණ දුන් අවස්ථාව කුමක්ද?

- T - Task (කාර්යය): ඔබට කිරීමට තිබූ වැඩේ කුමක්ද?

- A - Action (ක්‍රියාව): ඔබ එම ගැටලුව විසඳීමට ගත් පියවර මොනවාද?

- R - Result (ප්‍රතිඵලය): අවසානයේදී ලැබුණු සාර්ථක ප්‍රතිඵලය කුමක්ද?

104.5 කේතකරණ අභියෝගවලට මුහුණ දීම (Live Coding Tips)

බොහෝ විට ඔවුන් පවසනු ඇත ඔබේ තිරය (Screen Share) විවෘත කර ඔවුන් පවසන කේතය ලියා පෙන්වන ලෙස. මෙහිදී:

1. කටහඬ අවදි කරන්න (Think Out Loud): ඔබ කේතය ලියන අතරතුර ඔබ සිතන දේ ඔවුන්ට පවසන්න. "මම මෙතන map() එකක් පාවිච්චි කරන්නේ මේ අරාමේ නියෙන දත්ත පෙන්වන්න" වැනි දේ පවසන්න. ඔවුන් බලන්නේ ඔබ දන්නා පිළිතුරට වඩා ඔබ සිතන ආකාරය (Logic) දෙසයි.
2. ප්‍රශ්න අසන්න: වැඩේ ආරම්භ කිරීමට පෙර ඔබට යම් දෙයක් පැහැදිලි නැතිනම් ප්‍රශ්න අසා එය තහවුරු කරගන්න.
3. පළමුව විසඳුම ලබා දෙන්න: මූලිකම කේතය ලේඛන කිරීමට උත්සාහ නොකර, වැඩේ හරිය වෙන ලෙස කේතය ලියන්න. පසුව එය ප්‍රශ්නය (Optimize) කළ හැක.

104.6 ASCII රූපසටහන: සම්මුඛ පරීක්ෂණයක ගලායෑම

```
[ ආරම්භය ]
|
[ හඳුන්වා දීම ] <--- (ඔබ ගැනත්, ඔවුන් ගැනත් දැන ගැනීම)
|
[ තාක්ෂණික ප්‍රශ්න ] <--- (HTML, CSS, JS, React දැනුම)
|
[ කේතකරණ පරීක්ෂණය ] <--- (Live Coding / Assignment)
|
[ වර්ගාත්මක ප්‍රශ්න ] <--- (කණ්ඩායමක් ලෙස වැඩ කිරීමේ හැකියාව)
|
[ ඔබ ප්‍රශ්න ඇසීම ] <--- (ඔබට ඇති ගැටලු ඇසීම)
|
[ අවසානය ]
```

104.7 සම්මුඛ පරීක්ෂණයේදී අසනු ලබන "මෘදු" ප්‍රශ්න (Soft Skills)

තාක්ෂණයට අමතරව ඔබෙන් මෙවැනි ප්‍රශ්න ද අසනු ඇත:

- "ඔබ අපේ සමාගමට සම්බන්ධ වීමට කැමති ඇයි?"
- "ඔබට වෙනත් කෙනෙකු සමඟ මතභේදයක් ඇති වුවහොත් එය විසඳා ගන්නේ කෙසේද?"
- "ඉදිරි වසර 5 ඇතුළත ඔබ ඔබව දකින්නේ කොතැනකද?"

104.8 වෘත්තීය උපදෙස් (Professional Tips)

- කල්තියා පරීක්ෂා කරන්න: Online Interview එකක් නම්, ඔබේ අන්තර්ජාලය, කැමරාව සහ මයික්‍රොෆෝනය නියමිත වේලාවට පෙර පරීක්ෂා කරන්න.

- ඔබේ ව්‍යාපෘති ගැන දැන සිටින්න: ඔබ CV එකේ සඳහන් කර ඇති ඕනෑම ව්‍යාපෘතියක් ඇතුළත ඇති කේතය ගැන පැහැදිලි කිරීමට සූදානම්ව සිටින්න.

- ස්තුති කරන්න: සම්මුඛ පරීක්ෂණය අවසන් වූ පසු, ඔවුන්ට ස්තුති කර කෙටි Email එකක් යැවීම ඔබව අනෙක් අයගෙන් වෙන් කර පෙන්වන ඉතා යහපත් ගුණාංගයකි.

104.9 පොදු වැරදි (Common Mistakes)

1. බොරුවක් කීම: ඔබ යම් දෙයක් නොදන්නේ නම්, "මම ඒ ගැන හරියටම දන්නේ නැහැ, නමුත් මම දන්නා විදිහට එය මෙලෙසයි" කියා අවංක වන්න. බොරු කීම හසු වුවහොත් ඔබට රැකියාව අහිමි වේ.

2. පරණ තාක්ෂණය ගැන පමණක් කතා කිරීම: සැමවිටම අලුත්ම තාක්ෂණයන් (උදා: React Hooks, Next.js) ගැන අවබෝධයෙන් සිටින්න.

3. පරක්කු වී පැමිණීම: සම්මුඛ පරීක්ෂණයට විනාඩි 10කට පෙරවත් සුදානම් වී සිටින්න.

104.10 අභ්‍යාස (Exercises)

- පහත ප්‍රශ්නයට STAR ක්‍රමය භාවිතා කර පිළිතුරක් ලියන්න: "ඔබේ වෙබ් අඩවියක ඇතිවූ දෝෂයක් (Bug) ඔබ නිවැරදි කළ ආකාරය විස්තර කරන්න."
- කණ්ණාඩියක් ඉදිරිපිට සිට ඔබ ගැන සහ ඔබ දන්නා තාක්ෂණයන් ගැන විනාඩි 2ක හැඳින්වීමක් (Self-introduction) ඉංග්‍රීසියෙන් කිරීමට පුහුණු වන්න.
- අන්තර්ජාලයේ ඇති "Frontend Interview Questions" ලැයිස්තුවක් ගෙන එයින් ප්‍රශ්න 10කට පිළිතුරු සොයා ගන්න.

104.11 සම්මුඛ පරීක්ෂකවරයාගෙන් ඔබ ඇසිය යුතු ප්‍රශ්න

සම්මුඛ පරීක්ෂණය අවසානයේ ඔවුන් අසනු ඇත, "ඔබට අපෙන් දැනගත යුතු දෙයක් තිබේද?" කියා. එවිට:

- "මම වැඩ කරන්න යන කණ්ඩායමේ වැඩ කරන ආකාරය (Workflow) මොන වගේද?"
- "සමාගම ඇතුළත මගේ දැනුම වර්ධනය කරගන්න මට ලැබෙන අවස්ථා මොනවාද?" වැනි ප්‍රශ්න ඇසීමෙන් ඔබ එම රැකියාවට කෙතරම් උනන්දුවක් දක්වනවාද යන්න ඔවුන්ට වැටහේ.

එකසිය පස්වන පරිච්ඡේදය: වෘත්තීය මාර්ග සිතියම - ඉදිරි ගමන සැලසුම් කරමු (The Professional Career Roadmap)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අවසානයේදී ඔබට පහත කරුණු පිළිබඳව මනා අවබෝධයක් ලැබෙනු ඇත:

- ඉදිරිපෙළ සංවර්ධකයෙකුගේ (Frontend Developer) වෘත්තීය මට්ටම් (Junior, Mid, Senior) හඳුනා ගැනීම.
- රැකියාවක නිරත වෙමින් ඔබේ දැනුම නිරන්තරයෙන් යාවත්කාලීන (Continuous Learning) කරගන්නා ආකාරය.
- වෘත්තීය සබඳතා ජාලයක් (Professional Networking) ගොඩනගා ගැනීමේ වැදගත්කම.

4. LinkedIn සහ GitHub වැනි වේදිකා හරහා ඔබේ "වෘත්තීය සන්නාමය" (Personal Brand) ගොඩනගන ආකාරය.

5. ඉදිරි වසර කිහිපය සඳහා ඔබේ තාක්ෂණික ගමන් මග (Tech Stack Roadmap) සැලසුම් කිරීම.

6. මෘදුකාංග ඉංජිනේරුවෙකු සතු විය යුතු මානසිකත්වය (Engineering Mindset).

105.1 හැඳින්වීම: ගමන ආරම්භය පමණි (The Journey Begins)

සුභ පැතුම්! ඔබ දැන් මෙම පොත් මාලාව හරහා ඉදිරිපෙළ සංවර්ධකයෙකු වීමට අවශ්‍ය තාක්ෂණික පදනම සම්පූර්ණයෙන්ම සකසා ගෙන අවසානය. නමුත් තාක්ෂණික ලෝකය යනු නිරන්තරයෙන් වෙනස් වන ක්ෂේත්‍රයකි. අද ඔබ ඉගෙන ගත් දෙය හෙට තවත් අලුත් වීමට පුළුවන.

එබැවින්, වෘත්තීය ලෝකයේ සාර්ථක වීමට නම් ඔබට නිවැරදි මාර්ග සිතියමක් (Roadmap) තිබිය යුතුය. මෙම අවසාන පරිච්ඡේදයෙන් අපි සාකච්ඡා කරන්නේ රැකියාවක් ලබා ගැනීමෙන් පසුව ඔබ ඔබේ වෘත්තීය ජීවිතය ඉහළටම රැගෙන යන්නේ කෙසේද යන්නයි.

105.2 වෘත්තීය මට්ටම් (Career Levels)

මෘදුකාංග සමාගමක ඔබ ඉහළට යන ආකාරය ප්‍රධාන මට්ටම් 4කට බෙදිය හැක:

1. Junior Developer (කනිෂ්ඨ සංවර්ධක): (වසර 0-2 අත්දැකීම්).

- ඔබේ කාර්යය වන්නේ ජ්‍යෙෂ්ඨයින් පවසන කුඩා වැඩ කොටස් (Tasks) නිවැරදිව නිම කිරීමයි. මෙහිදී ඔබ වැඩිපුරම ඉගෙන ගත යුතුය.

2. Mid-level Developer (මධ්‍යම මට්ටමේ සංවර්ධක): (වසර 2-5 අත්දැකීම්).

- ඔබට දැන් තනිවම වගකීම් සහ ව්‍යාපෘති කළමනාකරණය කළ හැක. ඔබ කේතයේ ගුණාත්මකභාවය (Code Quality) ගැන වැඩිපුර සිතිය යුතුය.

3. Senior Developer (ජ්‍යෙෂ්ඨ සංවර්ධක): (වසර 5+ අත්දැකීම්).

- ඔබ පද්ධති සැලසුම් කරයි (System Architecture). අලුත් අය පුහුණු කරයි. තාක්ෂණික තීරණ ගනී.

4. Tech Lead / Architect:

- මුළු කණ්ඩායමම මෙහෙයවන, ඉතා සංකීර්ණ තාක්ෂණික ගැටලු විසඳන මට්ටමයි.

105.3 ඉදිරි තාක්ෂණික මාවත (Tech Roadmap)

ඔබ දැන් HTML, CSS, JS සහ React දන්නවා. මිලහට ඔබ ඉගෙන ගත යුතු දෑ ලෝකයේ පිළිගත් පිළිවෙළකට පහතින් දැක්වේ:

- Next.js: React මත පදනම් වූ, අද ලෝකයේ වැඩිපුරම භාවිතා වන රාමුව (Framework) SEO සහ වේගය සඳහා මෙය අත්‍යවශ්‍ය වේ.
- TypeScript: ජාවාස්ක්‍රිප්ට් වලට වඩා ආරක්ෂිත සහ වෘත්තීය ක්‍රමයයි. (විශාල සමාගම්වල මෙය අනිවාර්ය වේ).
- Testing: ඔබ ලියූ කේතයේ වැරදි තිබේදැයි පරීක්ෂා කිරීමට මෘදුකාංග ලිවීම (Jest, Cypress)
- Mobile Development: React Native භාවිතා කර ජංගම දුරකථන Apps සෑදීම.

105.4 වෘත්තීය සන්නාමය ගොඩනැගීම (Personal Branding)

රැකියාවක් ඔබ සොයා එන මට්ටමට පත්වීමට නම් ඔබ "පෙනෙන්නට" සිටිය යුතුය.

1. LinkedIn: මෙය ඔබේ වෘත්තීය මුහුණුවරයි. ඔබේ සහතික, ඔබ කළ ව්‍යාපෘති සහ ඔබ දන්නා දේවල් එහි නිරන්තරයෙන් පළ කරන්න.
2. GitHub: ඔබේ කේතයන් ලෝකයට දැකගත හැකි ස්ථානයයි. දිනපතා යම් කේතයක් Push කිරීමට උත්සාහ කරන්න (Green Dots).
3. Personal Blog: ඔබ ඉගෙන ගන්නා දේවල් සිංහලෙන් හෝ ඉංග්‍රීසියෙන් ලිපියක් ලෙස ලියන්න. මෙය ඔබ ප්‍රවීණයෙකු බව පෙන්වීමට හොඳම ක්‍රමයයි.

105.5 නිරන්තර ඉගෙනීම (Continuous Learning)

මෘදුකාංග ඉංජිනේරුවෙකු කිසිදා ඉගෙන ගැනීම නතර නොකරයි. අලුත් දේ දැනගැනීමට මෙම මූලාශ්‍ර භාවිතා කරන්න:

- Official Documentation: React හෝ Tailwind වෙබ් අඩවිවල ඇති මුල් ලියකියවිලි කියවන්න.
- YouTube Channels: (Fireship, Web Dev Simplified, Traversy Media) වැනි නාලිකා අනුගමනය කරන්න.
- Newsletters: (JavaScript Weekly) වැනි රීමේල් පුවත්පත්වලට ලියාපදිංචි වන්න.

105.6 ASCII රූපසටහන: වෘත්තීය මාර්ග සිතියම

- [ආරම්භය: HTML/CSS/JS]
- v
- [React.js & Tailwind Master]
- v
- [ව්‍යාපෘති 5-10ක් නිම කිරීම]
- v
- [Portfolio & LinkedIn සැකසීම]
- v
- [Junior Frontend Role / Freelancing]
- v
- [TypeScript & Next.js ඉගෙනීම]
- v
- [Senior Frontend Engineer]

105.7 ඉංජිනේරු මානසිකත්වය (The Engineering Mindset)

සැබෑ මෘදුකාංග ඉංජිනේරුවෙකු යනු "Code ටයිප් කරන්නෙකු" නොවේ. ඔහු "ගැටලු විසඳන්නෙකි" (Problem Solver).

- වැරද්දක් (Bug) ආ විට කලබල නොවන්න. එය විසඳීමෙන් ඔබ අලුත් දෙයක් ඉගෙන ගන්නා බව සිතන්න.
- "ඇයි මෙහෙම වෙන්නේ?" (Why?) කියා නිතරම අසන්න.
- පරිශීලකයා ගැන නිතරම සිතන්න (User-centric design).

105.8 වෘත්තීය උපදෙස් (Professional Tips)

- ඉංග්‍රීසි භාෂාව: තාක්ෂණික ලෝකයේ ඉදිරියට යාමට ඉංග්‍රීසි භාෂාව පිළිබඳ මූලික අවබෝධයක් තිබීම ඉතා වැදගත්ය. එය පුහුණු වන්න.
- නම්‍යශීලී වන්න: අලුත් තාක්ෂණයක් පැමිණි විට එය වැළඳ ගැනීමට බිය නොවන්න.
- Networking: ක්ෂේත්‍රයේ සිටින අනෙකුත් අය සමඟ කතාබස් කරන්න (Meetups / Webinars).

105.9 සාරාංශය (Key Points)

- වෘත්තීය ගමන Junior සිට Architect දක්වා විහිදී යයි.
- React වලට පසු Next.js සහ TypeScript ඉගෙනීම වැදගත් වේ.
- LinkedIn සහ GitHub ඔබේ වෘත්තීය ජීවිතයේ හඳුන්වයි.

- ඉගෙනීම කිසිදා අවසන් නොවන ක්‍රියාවලියකි.
- තාක්ෂණයට වඩා ගැටලු විසඳීමේ හැකියාව (Problem Solving) වේ.

105.10 අභ්‍යාස (Exercises)

1. ඉදිරි මාස 6 සඳහා ඔබේ තාක්ෂණික ඉලක්ක (Learning Goals) ලැයිස්තුවක් සකස් කරන්න.
2. LinkedIn ගිණුමක් සාදා එයට ඔබගේ දැනට ඇති සියලුම දක්ෂතා ඇතුළත් කරන්න.
3. ඔබට ආදර්ශයට ගත හැකි (Role Model) ජ්‍යෙෂ්ඨ Frontend සංවර්ධකයෙකු අන්තර්ජාලයෙන් සොයාගෙන ඔහුගේ ගමන් මග

එකසිය හයවන පරිච්ඡේදය: ජයග්‍රාහී පෝට්ෆෝලියෝව සහ වෘත්තීය ජීව දත්ත පත්‍රිකාව (The Winning Portfolio and Developer Resume)

ඉගෙනුම් අරමුණු (Learning Objectives)

මෙම පරිච්ඡේදය අධ්‍යයනය කිරීමෙන් පසු ඔබට පහත කරුණු පිළිබඳව අතිශය ගැඹුරු වෘත්තීය දැනුමක් ලැබෙනු ඇත:

1. Portfolio (පෝට්ෆෝලියෝ) යනු කුමක්ද සහ එය සංවර්ධකයෙකුට අත්‍යවශ්‍ය වන්නේ ඇයිද යන්න.
2. ලෝකය පිළිගන්නා මට්ටමේ පෝට්ෆෝලියෝ වෙබ් අඩවියක තිබිය යුතු අනිවාර්ය අංග හඳුනා ගැනීම.
3. ප්‍රදර්ශනය කළ යුතු ව්‍යාපෘති (Projects) තෝරා ගන්නා ආකාරය.
4. සංවර්ධකයෙකු සඳහා වන Modern Tech Resume (CV) එකක් සකස් කරන ආකාරය.
5. LinkedIn සහ වෙනත් වෘත්තීය ජාල හරහා තමන්ගේ සන්නාමය (Personal Branding) ගොඩනගන ආකාරය.

106.1 පෝට්ෆෝලියෝව යනු කුමක්ද? (What is a Portfolio?)

මෘදුකාංග ක්ෂේත්‍රයේදී, ඔබ ලබාගෙන ඇති සහතික පත්‍රවලට වඩා ඔබ කළ හැකි දේ පෙන්වීම වැදගත් වේ. Portfolio (පෝට්ෆෝලියෝ) යනු ඔබ මෙතෙක් නිර්මාණය කළ ව්‍යාපෘති, ඔබ සතු තාක්ෂණික දක්ෂතා සහ ඔබේ පළපුරුද්ද ලෝකයට පෙන්වන සජීවී වෙබ් අඩවියයි.

ඇයි මෙය වැදගත්? ඔබ රැකියාවකට අයදුම් කළ විට, සම්මුඛ පරීක්ෂකවරයා මුලින්ම බලන්නේ ඔබේ පෝට්ෆෝලියෝ වෙබ් අඩවියයි. එයින් ඔබ කෙතරම් පිරිසිදුව කේත ලියනවාද (Code Quality) සහ ඔබට නිර්මාණශීලීව සිතිය හැකිද (UI/UX) යන්න ඔහු තීරණය කරයි.

106.2 පෝට්ෆෝලියෝ වෙබ් අඩවියක අන්තර්ගතය (Anatomy of a Portfolio)

සාර්ථක පෝට්ෆෝලියෝ අඩවියක කොටස් 5ක් තිබිය යුතුය:

1. Introduction (හඳුන්වාදීම): ඔබ කවුද, ඔබ කරන්නේ කුමක්ද යන්න කෙටියෙන් (උදා: "I am a Frontend Engineer who builds high-performance web apps").
2. Tech Stack (තාක්ෂණික දක්ෂතා): ඔබ දන්නා භාෂා සහ මෙවලම් (HTML, CSS, React, Next.js, Tailwind).
3. Projects (ව්‍යාපෘති): ඔබ කළ හොඳම නිර්මාණ 3-5ක්.
 - එක් එක් ව්‍යාපෘතිය සඳහා: රූපයක්, භාවිතා කළ තාක්ෂණයන්, GitHub ලින්ක් එක සහ සජීවී ලින්ක් එක (Live Demo).
4. About Me (මා ගැන): ඔබේ ඉතිහාසය, ඔබ ඉගෙන ගත් දේ සහ ඔබේ විනෝදාංශ.
5. Contact (සම්බන්ධ වන්න): ඔබව සම්බන්ධ කරගත හැකි Email ලිපිනය සහ LinkedIn ලින්ක් එක.

106.3 ප්‍රදර්ශනය කළ යුතු ව්‍යාපෘති (Which Projects to Show?)

නිකමිම කුඩා වැඩ පෙන්වීම ප්‍රමාණවත් නැත. ඔබේ පෝට්ෆෝලියෝවට ඇතුළත් කළ යුතු හොඳම ව්‍යාපෘති වර්ග 3 මෙන්න:

1. CRUD App: දත්ත ඇතුළත් කරන, කියවන, වෙනස් කරන සහ මකන ඇප් එකක්. (උදා: අපි කළ TaskMaster Pro).
2. API Integration Project: බාහිර දත්ත ලබාගෙන පෙන්වන ඇප් එකක්. (උදා: කාලගුණ පද්ධතිය හෝ Movie Database).
3. Landing Page: ඉතාමත් අලංකාර, Responsive සහ සජීවීකරණය සහිත වෙබ් පිටුවක්. වෘත්තීය රහස: සැමවිටම එම ව්‍යාපෘතිය නිර්මාණය කිරීමේදී ඔබ මුහුණ දුන් අභියෝගය සහ එය විසඳූ ආකාරය කුඩාවට සටහන් කරන්න.

106.4 වෘත්තීය ජීව දත්ත පත්‍රිකාව (Modern Developer Resume)

මෘදුකාංග ක්ෂේත්‍රයේ CV එකක් සාමාන්‍ය CV එකකට වඩා වෙනස්ය.

- **ATS Friendly:** විශාල සමාගම් භාවිතා කරන මෘදුකාංගවලට ඔබේ CV එක කියවිය හැකි විය යුතුය. ඒ නිසා රූප සහ මෝස්තර වැඩිපුර භාවිතා නොකර පිරිසිදු ලේඛනයක් ලෙස සකසන්න.
- **One Page Rule:** අත්දැකීම් අඩු අයෙකුගේ CV එක පිටු 1කට වඩා දිගු නොවිය යුතුය.
- **Action Verbs:** "Built", "Developed", "Optimized", "Architected" වැනි වචන භාවිතා කරන්න.

ASCII සටහන: Resume සැකැස්ම

```
+-----+
| NAME: KASUN KALHARA |
| [LinkedIn] [GitHub] [Portfolio Link] |
+-----+
| SUMMARY: Frontend Engineer passionate |
| about React and Modern Web standards. |
+-----+
| SKILLS: React, Next.js, TS, Tailwind |
+-----+
| PROJECTS: |
| - TaskMaster Pro: Built with Context API |
| - SkyCast Weather: Used Fetch API |
+-----+
| EDUCATION: G.C.E. O/L, Self-taught |
+-----+
```

106.5 ලින්ක්ඩ්‍යින් (LinkedIn) ප්‍රවීණතාවය

LinkedIn යනු රැකියා සොයන වෙබ් අඩවියක් පමණක් නොව, එය ඔබගේ වෘත්තීය සන්නාමයයි.

1. **Headline:** "Frontend Developer | React & Next.js" ලෙස පැහැදිලිව ලියන්න.
2. **Featured Section:** ඔබේ පෝට්ෆෝලියෝ ලින්ක් එක එහි පෙන්වන්න (Pin කරන්න).
3. **Content:** ඔබ අලුතින් ඉගෙන ගන්නා දේවල් ගැන කුඩා සටහන් පළ කරන්න.

106.6 වෘත්තීය උපදෙස් (Professional Tips)

- **GitHub Green Dots:** දිනපතා යම් කේතයක් ලියා GitHub වෙත යවන්න. ඔබ කෙතරම් උනන්දුවෙන් වැඩ කරනවාද යන්න එයින් තීරණය වේ.

- **Soft Skills:** තාක්ෂණයට අමතරව ඔබ මිනිසුන් සමඟ ගනුදෙනු කිරීමට දන්නා බව පෙන්වන්න.
- **Lighthouse Score:** ඔබේ පෝට්ෆෝලියෝ අඩවියේ Lighthouse ලකුණු 90+ තිබීම වෘත්තීය මට්ටමේ සාක්ෂියකි.

106.7 සාරාංශය (Key Points)

- පෝට්ෆෝලියෝව යනු ඔබේ දක්ෂතා පෙන්වන කැඩපතයි.
- ව්‍යාපෘති සඳහා GitHub සහ Live Demo ලින්ක්ස් අත්‍යවශ්‍යය.
- CV එක කෙටි සහ තාක්ෂණයට මුල් තැන දෙන එකක් විය යුතුය.
- LinkedIn සහ GitHub ඔබේ වෘත්තීය ගමනේ වැදගත්ම මෙවලම් දෙකයි.

106.8 අභ්‍යාස (Exercises)

1. පරිමාව 1 සිට 6 දක්වා ඔබ සාදන ලද ව්‍යාපෘති සියල්ල GitHub වෙත යවා ඒවායේ README ගොනු පිරිසිදුව සකසන්න.
2. Canvas හෝ Google Docs භාවිතා කර ඔබගේ පළමු "Developer Resume" එක සකස් කරන්න.
3. ඔබේ පෝට්ෆෝලියෝ වෙබ් අඩවිය සඳහා ලස්සන "Home Page" එකක් React සහ Tailwind භාවිතා කර නිර්මාණය කරන්න. අවසාන නිගමනය (Final Conclusion of the Course)
සුභ පැතුම්! ඔබ දැන් සර්ව සම්පූර්ණ ඉදිරිපෙළ සංවර්ධන (Frontend Development) පාඨමාලාව සාර්ථකව අවසන් කර ඇත.

ඔබ ආරම්භ කළේ "පරිගණකයක් යනු කුමක්ද?" යන මූලික පාඩමෙනි. අද ඔබ අවසන් කරන්නේ:

- HTML/CSS මගින් ලස්සන වෙබ් අඩවි නිර්මාණය කළ හැකි,
- JavaScript සහ TypeScript මගින් සංකීර්ණ තර්කනයන් ගොඩනැගිය හැකි,
- React සහ Next.js මගින් ලෝක මට්ටමේ මෘදුකාංග නිපදවිය හැකි,
- Git/GitHub මගින් වෘත්තීය මට්ටමේ කළමනාකරණය කළ හැකි, පූර්ණ මෘදුකාංග ඉංජිනේරුවෙකු (Software Engineer) ලෙසයි.

මෘදුකාංග ක්ෂේත්‍රය යනු ඉගෙනීම කිසිදා අවසන් නොවන ගමනකි. අලුත් තාක්ෂණයන් හමුවේ නොසැලී, දිගටම පර්යේෂණ කරමින් පෙරට යන්න. ඔබේ වෘත්තීය ජීවිතය සාර්ථක වේවා! මුළු පාඨමාලාව මෙයින් නිල වශයෙන් අවසන් වේ.

සම්පූර්ණ පාඨමාලාවේ නිමාව: නව ගමනක ඇරඹීම(Conclusion: The Beginning of a New Journey)

ඔබ දැන් මෙම අති දැවැන්ත "Full Stack Web Development" පාඨමාලාවේ අවසාන සීමාවට පැමිණ ඇත. පරිච්ඡේද 1 සිට 110 දක්වා ඔබ පැමිණි මෙම ගමන හුදෙක් පොතක් කියවීමක් පමණක් නොව, ඔබේ වෘත්තීය ජීවිතය උඩුයටිකුරු කළ හැකි ප්‍රබල තාක්ෂණික ආයෝජනයකි.

ඔබ දැන් කවුද?

මෙම පිටු පෙරළීමට පෙර ඔබ තාක්ෂණය දෙස බැලූ ආකාරය සහ අද ඔබ තාක්ෂණය දෙස බලන ආකාරය අතර විශාල වෙනසක් ඇත. අද ඔබ:

- වෙබ් අඩවියක පිටුපස ඇති සංකීර්ණ තර්කනය තේරුම් ගත හැකි,
- ඕනෑම මෘදුකාංග අභියෝගයකට නිර්මාණශීලී විසඳුම් සෙවිය හැකි,
- ලෝකයේ ඕනෑම මට්ටමක මෘදුකාංග ඉංජිනේරුවෙකු සමඟ කරට කර වැඩ කළ හැකි, වෘත්තීය මෘදුකාංග සංවර්ධකයෙකි (Professional Software Developer).

ඉදිරි මාවත (The Road Ahead)

මෙම පාඨමාලාව නිම කිරීම යනු ඉගෙනීමේ අවසානය නොව, සැබෑ ලෝකයේ අභියෝග ජය ගැනීමේ ඇරඹුමයි. තාක්ෂණික ලෝකය දිනෙන් දින වෙනස් වේ. ඒ නිසා:

1. නතර නොවන්න: සැමවිටම අලුත් Frameworks සහ තාක්ෂණයන් පිළිබඳව විමසිල්ලෙන් සිටින්න.
2. නිර්මාණය කරන්න: ඔබ ඉගෙනගත් දේ භාවිත කරමින් ලෝකයට ප්‍රයෝජනවත් කුමන හෝ දෙයක් (Side Projects) නිරන්තරයෙන් නිර්මාණය කරන්න.
3. බෙදාගන්න: ඔබ දන්නා දේ තවත් අයෙකුට කියා දෙන්න. දැනුම යනු බෙදා දෙන තරමට වර්ධනය වන දෙයකි.

කර්තෘගෙන් අවසාන පණිවිඩය

"මෘදුකාංග ඉංජිනේරු විද්‍යාව යනු හුදෙක් කේත ලිවීම (Coding) පමණක් නොව, එය ලෝකයේ පවතින ගැටළු වලට තාක්ෂණික විසඳුම් ලබාදීමේ කලාවකි. ශ්‍රී ලාංකික තාරුණ්‍යය ලෝකයේ

ඉහළම තාක්ෂණික මට්ටම් වලට රැගෙන යාම මගේ එකම සිහිනයයි. ඔබ අද ලබාගත් මෙම දැනුම ඔබේත්, ඔබේ රටේත් අනාගතය ආලෝකවත් කිරීමට ශක්තියක් කරගන්න."

ඔබේ අනාගත වෘත්තීය ගමන අතිශයින්ම සාර්ථක වේවායි මම හදවතින්ම ප්‍රාර්ථනා කරමි!



T. Sachintha Imesh [FYZIE]

Full Stack Web Developer & Software Engineering Mentor

[fyzie44@gmail.com]

== THE END ==

